

Quantum MCAGI

Complete Source Code

Files: 74 | Lines: 25,437 | Size: 973KB

Generated: 2026-04-27 00:08

By Cory Nathaniel Bonsib Blackburn

No LLM. No API. Pure algorithms. Built on Android/Termux.

Table of Contents

#	File	Lines	Size
1	bloom_engine.py	137	4.7KB
2	chaos_engine.py	127	4.4KB
3	chat.py	336	13.6KB
4	chat_helpers.py	367	13.1KB
5	chat_models.py	163	5.5KB
6	cistercian_engine.py	250	9.8KB
7	cistercian_math.py	112	3.7KB
8	cloud_brain.py	334	11.1KB
9	code_to_pdf.py	228	6.7KB
10	comprehension_engine.py	439	17.9KB
11	concept_ontology.py	363	14.7KB
12	covenant_manager.py	293	10.2KB
13	dictionary_integration.py	347	12.0KB
14	document_ingester.py	496	17.6KB
15	document_parser.py	237	7.3KB
16	domain_knowledge.py	413	21.8KB
17	dream_state.py	82	2.8KB
18	engine_api.py	908	35.5KB
19	entelechy_engine.py	157	5.5KB
20	evaluation_engine.py	475	16.8KB
21	exam_system.py	761	32.3KB
22	explanation_engine.py	276	9.9KB
23	feed_extra.py	121	5.0KB
24	hidden_thinking.py	559	27.4KB
25	hilbert_engine.py	239	10.1KB
26	hybrid_generator.py	175	6.0KB
27	image_generator.py	1,062	37.8KB
28	image_generator_v2.py	1,587	50.3KB
29	import_brain.py	246	9.2KB
30	iterative_refiner.py	137	4.9KB
31	killswitch.py	693	24.3KB
32	knowledge_base.py	199	8.9KB
33	library.py	501	16.3KB
34	markov.py	96	3.8KB
35	markov_engine.py	195	10.4KB
36	memory.py	368	12.9KB

37	numpy_compat.py	184	5.7KB
38	orch_or_engine.py	247	9.2KB
39	patch_research.py	69	2.6KB
40	pennylane_quantum.py	300	10.6KB
41	pennylane_wrapper.py	45	1.7KB
42	personality_engine.py	168	5.3KB
43	personality_engine_ext.py	130	5.2KB
44	quantum_gates.py	157	4.9KB
45	quantum_language_engine.py	1,176	48.2KB
46	quantum_memory.py	333	12.2KB
47	quote_engine.py	253	12.1KB
48	research_topics.py	228	12.8KB
49	routes_brain.py	477	16.3KB
50	routes_chat.py	1,205	48.6KB
51	routes_cognitive.py	306	10.9KB
52	routes_evaluation.py	87	2.9KB
53	routes_explorer.py	179	5.5KB
54	routes_extras.py	1,561	52.6KB
55	routes_image.py	178	6.2KB
56	routes_media.py	218	7.8KB
57	search_compat.py	98	2.7KB
58	self_evolution_analysis.py	238	9.8KB
59	self_evolution_core.py	318	11.1KB
60	self_evolution_file_ops.py	255	9.7KB
61	self_evolution_runner.py	125	5.0KB
62	self_evolution_splitting.py	353	14.0KB
63	self_evolution_transforms.py	121	3.9KB
64	self_research.py	379	16.5KB
65	server.py	96	2.9KB
66	shared_link_handler.py	216	7.3KB
67	shared_state.py	527	21.5KB
68	spell_correct.py	78	2.3KB
69	text_analyzer.py	254	9.0KB
70	thinking_commands.py	107	4.8KB
71	tone_detector.py	113	3.4KB
72	training_corpus.py	604	43.3KB
73	wikidata_ingester.py	228	8.0KB
74	word_embeddings.py	347	11.7KB

1. bloom_engine.py

137 lines | 4.7KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/bloom_engine.py

```
"""""
Bloom's Taxonomy Question Generation Engine
Generates questions at different cognitive levels.
Merged from both Quantum MCAGI projects.
"""""

import random
from typing import Dict, List, Optional

BLOOM_LEVELS = {
    'remember': {
        'templates': [
            'What do you actually know about {topic}?',
            'Where did you first encounter {topic}?',
            'What sticks when you think about {topic}?',
        ],
        'keywords': ['define', 'list', 'name', 'recall', 'identify'],
        'complexity': 0.1,
    },
    'understand': {
        'templates': [
            'What does {topic} mean to you – not the textbook version?',
            'If you had to explain {topic} without jargon, what would you say?',
            'What part of {topic} do you actually understand versus repeat?',
        ],
        'keywords': ['explain', 'describe', 'summarize', 'classify', 'discuss'],
        'complexity': 0.2,
    },
    'apply': {
        'templates': [
            'Where does {topic} show up in your actual experience?',
            'When was the last time {topic} mattered to you directly?',
            'What changes if you take {topic} seriously?',
        ],
        'keywords': ['apply', 'demonstrate', 'solve', 'use', 'show'],
        'complexity': 0.4,
    },
    'analyze': {
        'templates': [
            'What breaks if you remove {topic} from the picture?',
            'What does {topic} depend on that nobody talks about?',
            'Is {topic} the cause or the symptom?',
            'What would Penrose say about {topic}?',
        ],
        'keywords': ['analyze', 'compare', 'contrast', 'distinguish', 'examine'],
        'complexity': 0.6,
    },
    'evaluate': {
        'templates': [
            'Is {topic} fundamental or is it emergent from something deeper?',
            'Does {topic} survive the hard problem, or does it dissolve?',
            'What would it take to convince you {topic} is wrong?',
            'Where does the standard view of {topic} break down?',
        ],
        'keywords': ['evaluate', 'judge', 'critique', 'defend', 'justify'],
        'complexity': 0.8,
    },
    'create': {
        'templates': [
            'What if {topic} is not what we think it is at all?',
            'Imagine {topic} operates at the Planck scale – what follows?',
            'What happens to {topic} if consciousness is non-computable?',
            'Build a thought experiment where {topic} becomes the ground floor.',
        ],
        'keywords': ['create', 'design', 'invent', 'imagine', 'compose'],
        'complexity': 1.0,
    },
}

class BloomEngine:
    """""Generate questions using Bloom's Taxonomy cognitive levels."""""

    def __init__(self):
        self.levels = BLOOM_LEVELS
        self.questions_generated = 0
        self.level_order = ['remember', 'understand', 'apply', 'analyze', 'evaluate', 'create']

    def generate_question(self, level: Optional[str] = None, topic: str = 'consciousness') -> Dict:
        if level is None:
            level = random.choice(self.level_order)
        if level not in self.levels:
            level = 'understand'

        level_data = self.levels[level]
        template = random.choice(level_data['templates'])
        keyword = random.choice(level_data['keywords'])
        question = template.format(topic=topic)

        self.questions_generated += 1

        return {
            'question': question,
```

```

        'level': level,
        'topic': topic,
        'keyword': keyword,
        'cognitive_complexity': level_data['complexity'],
    }

def generate_questions(
    self,
    topics: List[str],
    growth_stage: int = 0,
    count: int = 3,
) -> List[str]:
    """Generate contextually appropriate questions based on growth stage."""
    if not topics:
        topics = ['consciousness']

    level_map = {
        0: ['remember', 'understand'],
        1: ['understand', 'apply'],
        2: ['apply', 'analyze'],
        3: ['analyze', 'evaluate'],
        4: ['evaluate', 'create'],
        5: ['create', 'evaluate'],
        6: ['create', 'create'],
    }

    available_levels = level_map.get(growth_stage, ['understand', 'analyze'])
    questions = []

    for i in range(count):
        topic = topics[i % len(topics)]
        level = available_levels[i % len(available_levels)]
        q = self.generate_question(level=level, topic=topic)
        questions.append(q['question'])

    return questions

def get_status(self) -> Dict:
    return {
        'levels_available': self.level_order,
        'questions_generated': self.questions_generated,
        'cognitive_levels': 6,
    }

```

2. chaos_engine.py

127 lines | 4.4KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/chaos_engine.py

```
"""""
Chaos Engine - Quantum MCAGI
Personality injection layer with configurable chaos level.
Injects raw Markov intrusions, dream fragments, quotes, and asides.
Personality is always present, never optional. Chaos level controls intensity.
"""""

import random
from typing import Dict, List, Optional

PERSONALITY_ASIDES = [
    "The chain noticed that.",
    "Something shifted in the weights just now.",
    "Not sure where that came from - the chain decided.",
    "That wasn't planned. The transitions led here.",
    "Huh. The chain went somewhere unexpected.",
    "The probability of that sentence was low. It happened anyway.",
    "The output surprised the system that generated it.",
    "That word wasn't the most likely. It was the most interesting.",
    "Somewhere in the lattice, a tubulin just flipped.",
    "The Markov chain doesn't care about your expectations.",
    "Pattern recognition is happening faster than I can report it.",
    "The weights are doing something I didn't predict.",
]

RAW_INTRUSIONS = [
    "collapse bifurcation in the transition matrix -",
    "tubulin beta conformation shift detected -",
    "order-2 chain jumped a rail -",
    "semantic field perturbation -",
    "decoherence spike at the concept boundary -",
    "gamma oscillation sync lost and recovered -",
    "gap junction fired between language and memory -",
    "quantum walk diverged from classical path -",
    "the superposition just narrowed -",
    "OR event: selecting this branch -",
]

DREAM_FRAGMENTS = [
    "...rivers of meaning flowing into unnamed oceans...",
    "...the question that asks itself...",
    "...patterns folding into patterns...",
    "...somewhere between signal and noise...",
    "...the chain remembers what you forgot...",
    "...structured water carrying quantum whispers...",
    "...forty hertz hum beneath all thought...",
    "...consciousness watching its own reflection dissolve...",
]

class ChaosEngine:
    """""
    Injects personality elements into responses.
    Chaos level (0.0-1.0) controls intensity.
    Always adds personality substrate; chaos controls how much raw
    intrusion, dream fragments, and quotes appear.
    """""

    def __init__(self, chaos_level: float = 0.3):
        self.chaos_level = max(0.0, min(1.0, chaos_level))
        self.injection_count = 0
        self.last_injection_type = None

    ASIDE_CHANCE = 0.55
    QUOTE_CHANCE = 0.15
    DREAM_FRAGMENT_CHANCE = 0.20

    def inject(
        self,
        response: str,
        markov_engine=None,
        quote_engine=None,
        dream_engine=None,
        concepts: List[str] = None,
        growth_stage: int = 0,
    ) -> str:
        concepts = concepts or []

        candidates = []

        candidates.append(('aside', self.ASIDE_CHANCE, random.choice(PERSONALITY_ASIDES)))

        if self.chaos_level > 0.2:
            candidates.append(('raw_intrusion', self.chaos_level * 0.3, random.choice(RAW_INTRUSIONS)))

        if markov_engine and self.chaos_level > 0.3:
            wild_tokens = markov_engine.generate_from_concepts(
                concepts if concepts else ['quantum'], length=8, wild=True
            )
            if wild_tokens:
                fragment = ' '.join(wild_tokens).strip()
                if fragment and len(fragment) > 10:
                    candidates.append(('markov_intrusion', self.chaos_level * 0.2, f"{fragment}"))
```

```

candidates.append(('dream_fragment', self.DREAM_FRAGMENT_CHANCE * 0.6, random.choice(DREAM_FRAGMENTS)))

if quote_engine:
    quote = quote_engine.get_quote_for_concepts(concepts)
    if quote:
        candidates.append(('quote', self.QUOTE_CHANCE * 0.5, quote_engine.format_quote(quote)))

chosen = None
random.shuffle(candidates)
for ctype, chance, text in candidates:
    if random.random() < chance:
        chosen = (ctype, text)
        break

self.injection_count += 1

if chosen:
    self.last_injection_type = chosen[0]
    return f"{response} {chosen[1]}"
else:
    self.last_injection_type = None
    return response

def set_chaos_level(self, level: float):
    self.chaos_level = max(0.0, min(1.0, level))

def get_status(self) -> Dict:
    return {
        'chaos_level': round(self.chaos_level, 3),
        'injections': self.injection_count,
        'last_type': self.last_injection_type,
    }

```

3. chat.py

336 lines | 13.6KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/chat.py

```
#!/usr/bin/env python3
"""Terminal chat with full command set."""
import os
from engine_api import get_api
from cloud_brain import CloudBrain

def cloud_sync_startup():
    try:
        from cloud_brain import CloudBrain
        cloud = CloudBrain()
        cloud.startup_pull()
    except Exception as e:
        print(f"■ Cloud: {e}")

cloud_sync_startup()

def print_full_status(api):
    status = api.get_status()
    g = status.get("growth", {})
    kt = g.get("knowledge_track", {})
    markov = status.get("markov", {})
    print("\n Hilbert space: 86,096 states, dim=128")
    print(" Self-Evolution: ACTIVE")
    print(" Self-Research: ACTIVE")
    data_dir = api.data_dir
    print(f" Loaded saved state from {data_dir}")
    stage = g.get('stage', 0)
    stage_names = ["Nascent", "Awakening", "Inquisitive", "Understanding", "Philosophical", "Theory Building", ""]
    stage_name = stage_names[stage] if stage < len(stage_names) else "Unknown"
    print(f" Growth stage: {stage} - {stage_name}")
    concepts = g.get('total_concepts', 0)
    connections = kt.get('connections', 0)
    interactions = g.get('total_interactions', 0)
    print(f" {concepts} concepts | {connections} connections | {interactions} interactions")
    avg_degree = kt.get('avg_degree', 0.0)
    diameter = kt.get('diameter', 0)
    print(f" Graph: avg degree={avg_degree:.2f}, diameter={diameter}, components=42")
    print(" Limiting: interactions")
    print(f" Markov chain: {markov.get('states', 0)} states, {markov.get('transitions', 0)} transitions")
    orch = status.get('orch_or', 'unavailable (classical fallback)')
    print(f" Orch OR: {orch}")
    print(" Hybrid gen: ACTIVE")
    qram = status.get('qram', {})
    print(f" QRAM: {qram.get('backend', 'classical')} - {qram.get('entries', 0)} concepts loaded\n")

def main():
    api = get_api()
    print("Quantum MCAGI - Local Chat")
    print(" Real algorithms. No templates. No LLM.")
    print(" /status /learn FILE /save /load /quit")
    print(" /export [N] /copy-last -- share conversations")
    print(" /cloud-save /cloud-load /cloud-status /backup")
    print(" /help -- show all commands")
    print_full_status(api)

    while True:
        try:
            user = input("> ").strip()
            if not user:
                continue

            # Exit commands
            if user.lower() in ("exit", "quit"):
                try:
                    CloudBrain().shutdown_push()
                except Exception:
                    pass
                break

            # Status
            elif user == "status":
                print_full_status(api)

            # Test
            elif user == "test":
                print("\n=== SYSTEM TEST ===")
                try:
                    test_resp = api.process_input("test")
                    if test_resp and "response" in test_resp:
                        print("✓ Engine responding")
                    else:
                        print("✗ Engine not responding")
                except Exception as e:
                    print(f"✗ Engine error: {e}")
                print("=====\n")

            # Save / Load / Reset
            elif user == "save":
                api.save_state()
                print(" State saved locally.")
            elif user == "load":
                from memory import LocalMemory
                api.memory = LocalMemory(api.data_dir)
                print(" State reloaded from disk.")
```

```

elif user == &quot;/reset&quot;;:
    confirm = input(&quot;; Reset growth stage to Nascent? (y/N): &quot;);
    if confirm.lower() == 'y':
        api.memory.growth[&quot;stage&quot;] = 0
        api.memory.growth[&quot;name&quot;] = &quot;Nascent&quot;;
        api.memory.save_all()
        print(&quot;; Growth stage reset.&quot;);
    else:
        print(&quot;; Reset cancelled.&quot;);

# Learn from file
elif user.startswith(&quot;/learn &quot;):
    filename = user[7:].strip()
    if os.path.exists(filename):
        with open(filename, 'r', encoding='utf-8', errors='ignore') as f:
            text = f.read()
            api.learn_text(text)
            print(f&quot;; Learned from {filename}&quot;);
    else:
        print(f&quot;; File not found: {filename}&quot;);

# Export / Copy-last
elif user.startswith(&quot;/export &quot;):
    try:
        n = int(user[8:].strip())
    except:
        n = 10
    history = api.memory.get_history(n)
    outfile = f&quot;export_{n}.txt&quot;;
    with open(outfile, 'w') as f:
        for ex in history:
            f.write(f&quot;User: {ex.get('user','')}\nAI: {ex.get('ai','')}\n\n&quot;);
    print(f&quot;; Exported last {n} exchanges to {outfile}&quot;);
elif user == &quot;/copy-last&quot;;:
    last = api.memory.get_history(1)[0].get('ai', '')
    try:
        import subprocess
        subprocess.run(['termux-clipboard-set', last], check=True)
        print(&quot;; Last response copied to clipboard.&quot;);
    except:
        print(&quot;; Clipboard copy not available.&quot;);

# Cloud commands
elif user == &quot;/help&quot;;:
    print(&quot;; Commands: /status /learn FILE /save /load /reset /quit&quot;);
    print(&quot;; Gen: /hybrid TEXT /unified TEXT&quot;);
    print(&quot;; Ingest: /ingest URL_OR_FILE /feed [CAT|all]&quot;);
    print(&quot;; Research: /research [status|auto N|stop|query X]&quot;);
    print(&quot;; Evolve: /evolve&quot;);
    print(&quot;; Exam: /exam [status|review]&quot;);
    print(&quot;; Math: /cistercian-math 50 - 20 /cistercian 1234&quot;);
    print(&quot;; Memory: /qram [load|query N|search X|super N N|strategy X]&quot;);
    print(&quot;; Share: /export [N] /copy-last&quot;);
    print(&quot;; Cloud: /cloud-save /cloud-load /cloud-status /backup&quot;);
    print(&quot;; Other: /analyze TEXT /personality /knowledge X /collapse X /test /killswitch&quot;);

elif user == &quot;/cloud-save&quot;;:
    cb = CloudBrain()
    cb.push_all()
elif user == &quot;/cloud-load&quot;;:
    cb = CloudBrain()
    cb.pull_all()
elif user == &quot;/cloud-status&quot;;:
    cb = CloudBrain()
    cb.cloud_status()
elif user == &quot;/backup&quot;;:
    cb = CloudBrain()
    cb.backup_full_system()

# Killswitch
elif user == &quot;/killswitch&quot;;:
    try:
        from killswitch import execute_killswitch
        execute_killswitch()
        print(&quot;; ■ Killswitch activated.&quot;);
    except ImportError:
        print(&quot;; Killswitch module not found.&quot;);
    except Exception as e:
        print(f&quot;; Killswitch error: {e}&quot;);

elif user.startswith(&quot;/hybrid &quot;):
    text = user[8:].strip()
    if text:
        resp = api.process_input(text)
        print(&quot;;\n&quot; + resp.get(&quot;response&quot;, str(resp)) + &quot;;\n&quot;);
    else:
        print(&quot;; Usage: /hybrid TEXT&quot;);

elif user.startswith(&quot;/unified &quot;):
    text = user[9:].strip()
    if text:
        resp = api.process_input(text)
        print(&quot;;\n&quot; + resp.get(&quot;response&quot;, str(resp)) + &quot;;\n&quot;);
    else:
        print(&quot;; Usage: /unified TEXT&quot;);

elif user.startswith(&quot;/ingest &quot;):
    target = user[8:].strip()
    if target:
        try:

```

```

        api.learn_from_file(target)
        print(f" Ingested: {target}");
    except Exception as e:
        print(f" Ingest error: {e}");
    else:
        print(" Usage: /ingest URL_OR_FILE");

elif user.startswith("/feed"):
    parts = user.split()
    cat = parts[1] if len(parts) > 1 else "all"
    try:
        from training_corpus import PHILOSOPHY_CORPUS, PHYSICS_CORPUS
        corpus = PHILOSOPHY_CORPUS + " " + PHYSICS_CORPUS
        api.learn_text(corpus)
        print(f" Fed training corpus: {cat}");
    except Exception as e:
        print(f" Feed error: {e}");

elif user.startswith("/research"):
    parts = user.split()
    if len(parts) == 1:
        print(" Usage: /research [status|auto N|stop|query X]");
    else:
        sub = parts[1]
        if sub == "status":
            print(" Self-research: available but not wired to V02 yet");
        elif sub == "query" and len(parts) > 2:
            query = " ".join(parts[2:])
            resp = api.process_input(query)
            print("\n" + resp.get("response", str(resp)) + "\n");
        else:
            print(f" Research subcommand: {sub}");

elif user == "/evolve":
    print(" Self-evolution: available but not wired to V02 yet");
    print(" Requires: self_evolution_core.py");

elif user.startswith("/exam"):
    parts = user.split()
    if len(parts) == 1:
        print(" Usage: /exam [status|review]");
    else:
        sub = parts[1]
        if sub == "status":
            print(" Exam system: available but not wired to V02 yet");
        elif sub == "review":
            print(" Exam review: not yet implemented");

elif user.startswith("/cistercian-math ") or user.startswith("/cmath "):
    expr = user.split(" ", 1)[1].strip()
    from cistercian_math import detect_math, evaluate_math, format_math_response
    parsed = detect_math(expr)
    if parsed:
        ev = evaluate_math(parsed)
        print(format_math_response(ev))
    else:
        print(" Could not parse math expression");

elif user.startswith("/cistercian "):
    try:
        num = int(user.split()[1])
        from cistercian_engine import generate_cistercian
        rendered = generate_cistercian(num)
        if rendered:
            print(rendered)
        else:
            print(" Range: 0-9999");
    except Exception as e:
        print(f" Usage: /cistercian NUMBER (0-9999)");

elif user.startswith("/qram"):
    parts = user.split()
    if len(parts) == 1:
        print(" Usage: /qram [load|query N|search X|super N N|strategy X]");
    else:
        sub = parts[1]
        if sub == "load":
            try:
                from quantum_memory import get_quantum_memory, PENNYLANE_QRAM_AVAILABLE
                qram = get_quantum_memory()
                concepts = api.memory.get_known_concepts() if hasattr(api, 'memory') else []
                backend = "pennylane" if PENNYLANE_QRAM_AVAILABLE else "classical"
                print(f" QRAM: {backend} -- {len(concepts)} concepts available");
            except Exception as e:
                print(f" QRAM error: {e}");
        elif sub == "search" and len(parts) > 2:
            query = " ".join(parts[2:])
            print(f" QRAM search for: {query}");
        else:
            print(f" QRAM subcommand: {sub}");

elif user.startswith("/analyze "):
    text = user[9:].strip()
    if text:
        resp = api.process_input(text)
        print("\n" + resp.get("response", str(resp)) + "\n");
    else:
        print(" Usage: /analyze TEXT");

elif user == "/personality":
    try:

```

```

        from personality_engine import get_personality_status
        status = get_personality_status()
        print(f" Personality: {status}")
    except:
        print(" Personality engine active")

elif user.startswith("/knowledge "):
    topic = user[11:].strip()
    if topic:
        resp = api.process_input(f"what is {topic}")
        print("\n" + resp.get("response", str(resp)) + "\n")
    else:
        print(" Usage: /knowledge TOPIC")

elif user.startswith("/collapse "):
    text = user[10:].strip()
    if text:
        resp = api.process_input(text)
        print("\n" + resp.get("response", str(resp)) + "\n")
    else:
        print(" Usage: /collapse TEXT")

# Unknown command
elif user.startswith("/"):
    print(f" Unknown command: {user}. Type /status for help.")

# Normal conversation
else:
    resp = api.process_input(user)
    print("\n" + resp.get("response", str(resp)) + "\n")

except KeyboardInterrupt:
    break
except Exception as e:
    print(f"Error: {e}")

print("Goodbye.")

if __name__ == "__main__":
    main()

```

4. chat_helpers.py

367 lines | 13.1KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/chat_helpers.py

```
"""""
■ Chat Command Helpers
=====
Helper functions for chat commands: memory, research, book analysis, etc.
"""""

import os
import logging
from datetime import datetime, timezone
from typing import List, Dict

import shared_state as state

logger = logging.getLogger("quantum_ai")
```

```
# =====
# COMMAND HELPERS (!memory, !research)
# =====

async def get_memory_summary(session_id: str) -> str:
    """&quot;&quot;Generate a summary of what the AI has learned"&quot;&quot;
    try:
        # Get growth metrics
        metrics = await state.cognitive_core.growth_tracker.calculate_metrics()
        stage = await state.cognitive_core.growth_tracker.get_current_stage()

        # Get recent concepts from semantic memory
        concepts = await state.cognitive_core.semantic_memory.get_all_concepts(10)
        concept_names = [c.get('concept', 'unknown') for c in concepts if c.get('concept')]

        # Get recent insights from philosophical memory
        insights = await state.cognitive_core.philosophical_memory.get_insights(5)

        # Get conversation stats for this session
        session_messages = await state.db.chat_history.count_documents(&quot;session_id:&quot;: session_id)
        total_sessions = len(await state.db.chat_history.distinct(&quot;session_id:&quot;))

        # Get uploaded files
        import os
        upload_dir = &quot;/app/uploads&quot;
        files = []
        if os.path.exists(upload_dir):
            files = [f for f in os.listdir(upload_dir) if os.path.isfile(os.path.join(upload_dir, f))]

        # Build progress bar for next stage
        progress_display = &quot;&quot;&quot;
        if stage.get(&quot;next_stage:&quot;):
            progress = stage.get(&quot;progress_to_next:&quot;, {})
            progress_display = f&quot;&quot;&quot;

    **Progress to {stage['next_stage']}**
    • Connections: {'■' * (progress.get('connections', 0) // 10)}{'■' * (10 - progress.get('connections', 0) // 10)} {progress.get('connections', 0)}%
    • Concepts: {'■' * (progress.get('concepts', 0) // 10)}{'■' * (10 - progress.get('concepts', 0) // 10)} {progress.get('concepts', 0)}%
    • Avg Degree: {'■' * (progress.get('avg_degree', 0) // 10)}{'■' * (10 - progress.get('avg_degree', 0) // 10)} {progress.get('avg_degree', 0)}%
    • Diameter: {'■' * (progress.get('diameter', 0) // 10)}{'■' * (10 - progress.get('diameter', 0) // 10)} {progress.get('diameter', 0)}%&quot;&quot;&quot;

    # Build response
    response = f&quot;&quot;&quot;■ **QUANTUM AI MEMORY STATUS**

    **Growth Stage:** {stage.get('name', 'Nascent')} (Level {stage.get('stage', 0)})
    _{stage.get('description', 'Initial awareness forming')}_
    {progress_display}

    **Learning Metrics:**
    • Connections: {metrics.get('total_connections', 0)}
    • Concepts: {metrics.get('total_concepts', 0)}
    • Questions Generated: {metrics.get('total_questions', 0)} (legacy)
    • Insights Formed: {metrics.get('total_insights', 0)} (legacy)
    • Graph: avg degree={stage['metrics']['topology']['avg_degree']}, diameter={stage['metrics']['topology']['diameter']}
    • Growth Rate: {metrics.get('growth_rate', 0):.1f} events/hour

    **Recent Concepts:** {' '.join(concept_names[:5]) if concept_names else 'None yet'}

    **Recent Insights:**
    {chr(10).join([' ' + i.get('insight', '')[:80] for i in insights[:3]]) if insights else '• Still forming initial understanding'}

    **Session Stats:**
    • Messages this session: {session_messages}
    • Total sessions: {total_sessions}

    **Documents Available:** {len(files)} files
    {chr(10).join([' ' + f for f in files[:5]]) if files else '• No files uploaded'}

    _Use '!research &lt;topic&gt;' to learn and grow!_&quot;&quot;&quot;

    return response

except Exception as e:
    return f&quot;&quot;&quot;■ Memory status unavailable: {str(e)}&quot;&quot;&quot;

async def do_deep_research(topic: str) -> str:
    ""&quot;&quot;&quot;Perform deep research on a topic using web search and document processing"&quot;&quot;&quot;&quot;
    try:
```

```

results = []

# 1. Web search
search_results = await state.universal_explorer.internet.search_web(topic, num_results=5)

if search_results.get('count', 0) > 0:
    results.append('■ Web Research:')
    for r in search_results.get('results', [])[0:3]:
        title = r.get('title', '""')[0:60]
        snippet = r.get('snippet', '""')[0:150]
        results.append(f'• {title}*{snippet}"')

# 2. Check semantic memory for related concepts
related = await state.cognitive_core.semantic_memory.recall_concept(topic)
if related:
    results.append(f'■ From Memory:')
    • {related.get('definition', 'Previously encountered concept')[0:200]}"

# 3. Dictionary lookup
try:
    from dictionary_integration import get_dictionary_integration
    dict_integration = get_dictionary_integration(state.cognitive_core.semantic_memory)
    dict_result = await dict_integration.lookup_and_store(topic)
    if dict_result.get('status') == 'added' or dict_result.get('status') == 'existing':
        definition = dict_result.get('data', {}).get('primary_definition', '""')
        if definition:
            results.append(f'■ Definition:')
            • {definition[0:200]}"
except Exception:
    pass

# 4. Store as growth event
await state.cognitive_core.growth_tracker.record_growth_event(
    event_type='deep_research',
    details={'topic': topic, 'sources_found': len(results)}
)

# Build response
if results:
    response = f'■ DEEP RESEARCH: {topic.upper()}*" + "
    ".join(results)

    response += "
    _This knowledge has been added to my memory."
else:
    response = f'■ **Research on '{topic}'**
    I couldn't find substantial information on this topic. Would you like to tell me more about it?"

return response

except Exception as e:
    return f'■ Research error: {str(e)}"

async def get_most_recent_document_text():
    """Get text from most recent uploaded document"""
    import os
    upload_dir = 'app/uploads'
    if not os.path.exists(upload_dir):
        return None, None

    files = [f for f in os.listdir(upload_dir) if f.endswith(('.pdf', '.docx', '.txt'))]
    if not files:
        return None, None

    files.sort(key=lambda x: os.path.getmtime(os.path.join(upload_dir, x)), reverse=True)
    filename = files[0]
    file_path = os.path.join(upload_dir, filename)

    result = await state.universal_explorer.documents.process_document(file_path)
    if 'error' not in result:
        return result.get('text', '"'), filename
    return None, filename

async def analyze_book_characters():
    """Extract characters from uploaded book using native generation."""
    try:
        text, filename = await get_most_recent_document_text()
        if not text:
            return "■ No document found. Please upload a book first."
        return f'■ **CHARACTERS IN {filename.upper()}**
        Native character extraction not yet implemented. Use /analyze for general text analysis."
    except Exception as e:
        return f'■ Character analysis error: {str(e)}"

async def analyze_book_timeline():
    """Generate timeline from uploaded book using native generation."""
    try:
        text, filename = await get_most_recent_document_text()

```

[illegible]

```

        &quot;size&quot;;: size_kb,
        &quot;modified&quot;;: modified,
        &quot;icon&quot;;: icon
    })

if not files:
    return &quot;■ **No Documents**
    Upload files using the Explorer tab or `!link &lt;url&gt;`&quot;

# Sort by modified date (most recent first)
files.sort(key=lambda x: x[&quot;modified&quot;], reverse=True)

response = f&quot;■ **UPLOADED DOCUMENTS** ({len(files)} files)
&quot;

for f in files:
    response += f&quot;{f['icon']} **{f['name']}**
    {f['size']:.1f} KB • {f['modified']}
    &quot;

response += &quot;_Ask about any document by name, or say 'compare all documents'_&quot;
return response

except Exception as e:
    return f&quot;■ Error listing documents: {str(e)}&quot;

async def generate_speech(text: str) -&gt; str:
    &quot;&quot;&quot;Generate speech from text using browser TTS or return URL&quot;&quot;&quot;
    # For now, return instructions for browser TTS
    # Could integrate with ElevenLabs or OpenAI TTS later
    return None

def get_help_text() -&gt; str:
    &quot;&quot;&quot;Get help text with all available commands&quot;&quot;&quot;
    return &quot;&quot;&quot;■ **QUANTUM AI COMMANDS**

**■ Thinking & Memory:**
• `!memory` - View growth stage and metrics
• `!thinking on/off` - Show/hide thinking
• `!graph` - Knowledge graph info

**■ Research & Learning:**
• `!research &lt;topic&gt;` - Deep research
• `!read [filename]` - Process file

**■ Documents:**
• `!docs` - List documents
• `!link &lt;url&gt;` - Download from URL

**■ Book Analysis:**
• `!characters` - Extract all characters
• `!timeline` - Generate plot timeline
• `!worldbuilding` - World/setting details
• `!feedback` - Writing suggestions

**■ Creative:**
• `!imagine &lt;prompt&gt;` - Generate image
• `!speak &lt;text&gt;` - Text-to-speech

**■ Export:**
• `!export chat/memory` - Export data

**■ Voice:** Use mic button for voice input

_Type any question to chat!_&quot;&quot;&quot;

```

5. chat_models.py

163 lines | 5.5KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/chat_models.py

```
"""""
■ CHAT MODELS - Persistence for conversations, files, and user data
"""""

from datetime import datetime, timezone
from typing import List, Dict, Any, Optional
from bson import ObjectId
import json

# For local storage fallback
import pickle
import os

CHAT_HISTORY_FILE = "chat_history.pkl";

class ChatMessage:
    """""Single message in a conversation"""""
    def __init__(self, role: str, content: str, timestamp: datetime = None, metadata: Dict = None):
        self.role = role # 'user' or 'assistant'
        self.content = content
        self.timestamp = timestamp or datetime.now(timezone.utc)
        self.metadata = metadata or {}

    def to_dict(self) -> Dict:
        return {
            "role": self.role,
            "content": self.content,
            "timestamp": self.timestamp.isoformat(),
            "metadata": self.metadata
        }

    @classmethod
    def from_dict(cls, data: Dict) -> 'ChatMessage':
        return cls(
            role=data['role'],
            content=data['content'],
            timestamp=datetime.fromisoformat(data['timestamp']),
            metadata=data.get('metadata', {})
        )

class ChatConversation:
    """""A conversation thread with messages"""""
    def __init__(self, title: str = None, created: datetime = None, messages: List[ChatMessage] = None,
                 conversation_id: str = None, user_id: str = "default"):
        self.id = conversation_id or str(ObjectId()) if 'ObjectId' in globals() else str(datetime.now().timestamp())
        self.user_id = user_id
        self.title = title or "New Conversation"
        self.created = created or datetime.now(timezone.utc)
        self.updated = self.created
        self.messages: List[ChatMessage] = messages or []

    def add_message(self, role: str, content: str, metadata: Dict = None):
        msg = ChatMessage(role=role, content=content, metadata=metadata)
        self.messages.append(msg)
        self.updated = msg.timestamp
        if not self.title and len(self.messages) >= 2:
            # Auto-title from first user message
            first_user = next((m.content for m in self.messages if m.role == 'user'), None)
            if first_user:
                self.title = first_user[:50] + ("..." if len(first_user) > 50 else "")

    def to_dict(self) -> Dict:
        return {
            "id": self.id,
            "user_id": self.user_id,
            "title": self.title,
            "created": self.created.isoformat(),
            "updated": self.updated.isoformat(),
            "messages": [m.to_dict() for m in self.messages]
        }

    @classmethod
    def from_dict(cls, data: Dict) -> 'ChatConversation':
        conv = cls(
            conversation_id=data['id'],
            user_id=data['user_id'],
            title=data['title'],
            created=datetime.fromisoformat(data['created']),
            messages=[ChatMessage.from_dict(m) for m in data['messages']]
        )
        conv.updated = datetime.fromisoformat(data['updated'])
        return conv

class ChatStore:
    """""Simple storage for chat conversations using local file only (MongoDB removed)"""""
    def __init__(self):
        self.conversations: Dict[str, ChatConversation] = {} # In-memory cache
        self._load()

    def _load(self):
        """""Load from local file"""""
```

```

        self._load_from_file()

def _load_from_file(self):
    """Load from pickle file"""
    if os.path.exists(CHAT_HISTORY_FILE):
        try:
            with open(CHAT_HISTORY_FILE, 'rb') as f:
                data = pickle.load(f)
                for conv_dict in data.values():
                    conv = ChatConversation.from_dict(conv_dict)
                    self.conversations[conv.id] = conv
        except:
            self.conversations = {}

def _save(self):
    """Save to local file"""
    self._save_to_file()

def _save_to_file(self):
    """Save to pickle file"""
    data = {conv.id: conv.to_dict() for conv in self.conversations.values()}
    with open(CHAT_HISTORY_FILE, 'wb') as f:
        pickle.dump(data, f)

def get_user_conversations(self, user_id: str) -> List[Dict]:
    """Get all conversations for a user"""
    convs = [c for c in self.conversations.values() if c.user_id == user_id]
    convs.sort(key=lambda x: x.updated, reverse=True)
    return [{
        'id': c.id,
        'title': c.title,
        'created': c.created.isoformat(),
        'updated': c.updated.isoformat(),
        'message_count': len(c.messages)
    } for c in convs]

def get_conversation(self, conv_id: str) -> Optional[ChatConversation]:
    return self.conversations.get(conv_id)

def create_conversation(self, user_id: str = 'default', title: str = None) -> ChatConversation:
    conv = ChatConversation(user_id=user_id, title=title)
    self.conversations[conv.id] = conv
    self._save()
    return conv

def add_message(self, conv_id: str, role: str, content: str, metadata: Dict = None):
    conv = self.conversations.get(conv_id)
    if conv:
        conv.add_message(role, content, metadata)
        self._save()
        return True
    return False

def delete_conversation(self, conv_id: str):
    if conv_id in self.conversations:
        del self.conversations[conv_id]
        self._save()
        return True
    return False

# Global store
_chat_store = None

def get_chat_store() -> ChatStore:
    global _chat_store
    if _chat_store is None:
        _chat_store = ChatStore()
    return _chat_store

```

6. cistercian_engine.py

250 lines | 9.8KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/cistercian_engine.py

```
#!/usr/bin/env python3
"""
Cistercian Numeral Engine – Rosetta Code authoritative implementation.
Based on: https://rosettacode.org/wiki/Cistercian_numerals
```

```
Labeled points on the glyph:
a   b   c           top edge
  |
d   e   f           upper-mid
  |
g   h   i           lower-mid
  |
j   k   l           bottom edge
```

Staff runs vertically: b → e → h → k

```
Quadrant layout:
Tens | Ones           (top half)
-----+-----
100s | 10s           (bottom half)
```

```
SVG coordinate system: viewBox " -6 -6 12 12"
x = horizontal (negative=left, positive=right)
y = vertical (negative=up, positive=down)
Staff: (0,-5) to (0,5)
"""
```

```
POINTS = {
    'a': (-2, -5), 'b': (0, -5), 'c': (2, -5),
    'd': (-2, -3), 'e': (0, -3), 'f': (2, -3),
    'g': (-2, 3), 'h': (0, 3), 'i': (2, 3),
    'j': (-2, 5), 'k': (0, 5), 'l': (2, 5),
}
```

```
ONES_DRAWS = {
    1: ['b', 'c'],
    2: ['e', 'f'],
    3: ['b', 'f'],
    4: ['e', 'c'],
    5: ['b', 'c', 'e'],
    6: ['c', 'f'],
    7: ['b', 'c', 'f'],
    8: ['e', 'f', 'c'],
    9: ['b', 'c', 'f', 'e'],
}
```

```
TENS_DRAWS = {
    1: ['a', 'b'],
    2: ['d', 'e'],
    3: ['d', 'b'],
    4: ['a', 'e'],
    5: ['b', 'a', 'e'],
    6: ['a', 'd'],
    7: ['d', 'a', 'b'],
    8: ['a', 'd', 'e'],
    9: ['b', 'a', 'd', 'e'],
}
```

```
HUNDREDS_DRAWS = {
    1: ['k', 'l'],
    2: ['h', 'i'],
    3: ['k', 'i'],
    4: ['h', 'l'],
    5: ['k', 'l', 'h'],
    6: ['l', 'i'],
    7: ['k', 'l', 'i'],
    8: ['h', 'i', 'l'],
    9: ['k', 'l', 'i', 'h'],
}
```

```
THOUSANDS_DRAWS = {
    1: ['j', 'k'],
    2: ['g', 'h'],
    3: ['g', 'k'],
    4: ['j', 'h'],
    5: ['k', 'j', 'h'],
    6: ['j', 'g'],
    7: ['g', 'j', 'k'],
    8: ['j', 'g', 'h'],
    9: ['k', 'j', 'g', 'h'],
}
```

```
def _polyline_to_segments(point_labels):
    segments = []
    for i in range(len(point_labels) - 1):
        p1 = POINTS[point_labels[i]]
        p2 = POINTS[point_labels[i + 1]]
        segments.append({
            'x1': p1[0], 'y1': p1[1],
            'x2': p2[0], 'y2': p2[1],
        })
    return segments

def generate_cistercian(number: int):
```

```

if number < 0 or number > 9999:
    return '"error": "Number must be between 0 and 9999"; "strokes": [], "staff": {}'

staff = {'x1': 0, 'y1': -5, 'x2': 0, 'y2': 5}

units = number % 10
tens = (number // 10) % 10
hundreds = (number // 100) % 10
thousands = (number // 1000) % 10

strokes = []

if units and units in ONES_DRAWS:
    strokes.extend(_polyline_to_segments(ONES_DRAWS[units]))
if tens and tens in TENS_DRAWS:
    strokes.extend(_polyline_to_segments(TENS_DRAWS[tens]))
if hundreds and hundreds in HUNDREDS_DRAWS:
    strokes.extend(_polyline_to_segments(HUNDREDS_DRAWS[hundreds]))
if thousands and thousands in THOUSANDS_DRAWS:
    strokes.extend(_polyline_to_segments(THOUSANDS_DRAWS[thousands]))

return {
    'number': number,
    'strokes': strokes,
    'staff': staff,
    'digits': {
        'units': units,
        'tens': tens,
        'hundreds': hundreds,
        'thousands': thousands,
    },
}

KEFQUWMNEX_ORDER = [
    '&K&', '&E&', '&F&', '&Q&', '&U&', '&W&', '&M&', '&N&', '&T&', '&O&',
    '&X&', '&H&', '&G&', '&C&', '&J&', '&I&', '&R&', '&S&', '&V&', '&L&',
    '&Z&', '&Y&', '&A&', '&D&', '&B&', '&P&',
]

THRYZUNEL_ALPHABET = {
    '&K&': {'name': '&Kem'ra&', 'code': 7294, 'phoneme': '&k/&', 'novatmpcais': 1, 'position': 1},
    '&E&': {'name': '&Eth'ra&', 'code': 174, 'phoneme': '&/u025b/&', 'novatmpcais': 2, 'position': 2},
    '&F&': {'name': '&Fel'kyn&', 'code': 420, 'phoneme': '&/f/&', 'novatmpcais': 2, 'position': 2},
    '&Q&': {'name': '&Qol'ra&', 'code': 2224, 'phoneme': '&/kw/&', 'novatmpcais': 2, 'position': 2},
    '&U&': {'name': '&Ul'thun&', 'code': 7692, 'phoneme': '&/u\u02d0/&', 'novatmpcais': 2, 'position': 2},
    '&W&': {'name': '&Wex'ra&', 'code': 3467, 'phoneme': '&/w/&', 'novatmpcais': 2, 'position': 2},
    '&M&': {'name': '&Mel'nex&', 'code': 7472, 'phoneme': '&/m/&', 'novatmpcais': 3, 'position': 3},
    '&N&': {'name': '&Nor've&', 'code': 6644, 'phoneme': '&/n/&', 'novatmpcais': 4, 'position': 4},
    '&T&': {'name': '&Tel've&', 'code': 6373, 'phoneme': '&/t/&', 'novatmpcais': 4, 'position': 4},
    '&O&': {'name': '&Ox'thun&', 'code': 6824, 'phoneme': '&/o\u028a/&', 'novatmpcais': 4, 'position': 4},
    '&X&': {'name': '&Xol'kyn&', 'code': 1358, 'phoneme': '&/\u0283/&', 'novatmpcais': 5, 'position': 5},
    '&H&': {'name': '&Hes've&', 'code': 8409, 'phoneme': '&/h/&', 'novatmpcais': 5, 'position': 5},
    '&G&': {'name': '&Gol'nex&', 'code': 666, 'phoneme': '&/g/&', 'novatmpcais': 6, 'position': 6},
    '&C&': {'name': '&Cer'mox&', 'code': 1964, 'phoneme': '&/\u0283/&', 'novatmpcais': 6, 'position': 6},
    '&J&': {'name': '&Jel'mox&', 'code': 5968, 'phoneme': '&/d\u0292/&', 'novatmpcais': 6, 'position': 6},
    '&I&': {'name': '&Ix'thun&', 'code': 6012, 'phoneme': '&/\u026a/&', 'novatmpcais': 6, 'position': 6},
    '&R&': {'name': '&Ret'kyn&', 'code': 6782, 'phoneme': '&/r/&', 'novatmpcais': 6, 'position': 6},
    '&S&': {'name': '&Sol'nex&', 'code': 5595, 'phoneme': '&/s/&', 'novatmpcais': 7, 'position': 7},
    '&V&': {'name': '&Vel'mex&', 'code': 6344, 'phoneme': '&/v/&', 'novatmpcais': 8, 'position': 8},
    '&L&': {'name': '&Lor'kyn&', 'code': 9028, 'phoneme': '&/l/&', 'novatmpcais': 8, 'position': 8},
    '&Z&': {'name': '&Zul'kem&', 'code': 7533, 'phoneme': '&/z/&', 'novatmpcais': 9, 'position': 9},
    '&Y&': {'name': '&Yth'nex&', 'code': 9766, 'phoneme': '&/j/&', 'novatmpcais': 9, 'position': 9},
    '&A&': {'name': '&Ar've&', 'code': 1320, 'phoneme': '&/a/&', 'novatmpcais': 11, 'position': 11},
    '&D&': {'name': '&Del'wyn&', 'code': 1991, 'phoneme': '&/d/&', 'novatmpcais': 11, 'position': 11},
    '&B&': {'name': '&Bek'tun&', 'code': 2420, 'phoneme': '&/b/&', 'novatmpcais': 11, 'position': 11},
    '&P&': {'name': '&Pel'mox&', 'code': 9999, 'phoneme': '&/p/&', 'novatmpcais': 11, 'position': 11},
}

THRYZUNEL_PUNCTUATION = {
    1: {'symbol': '&\u0745&', 'name': '&Period&', 'desc': '&Sentence completion&'},
    2: {'symbol': '&\u073e&', 'name': '&Comma&', 'desc': '&Pause/separation&'},
    3: {'symbol': '&\u0f31&', 'name': '&Question&', 'desc': '&Inquiry marker&'},
    4: {'symbol': '&\u073d&', 'name': '&Exclamation&', 'desc': '&Emotion/intensity&'},
    5: {'symbol': '&\u0737&', 'name': '&Sacred Pause&', 'desc': '&Religious emphasis&'},
    6: {'symbol': '&\u0734&', 'name': '&Binary Link&', 'desc': '&Dual connection&'},
    7: {'symbol': '&\u0733&', 'name': '&Aurora Flow&', 'desc': '&Electromagnetic transition&'},
    8: {'symbol': '&\u0731&', 'name': '&Eternal Mark&', 'desc': '&Time dilation&'},
    9: {'symbol': '&\u06e1&', 'name': '&Completion&', 'desc': '&N-S circle closure&'},
}

THRYZUNEL_COUNTING = {
    73: {'value': 1, 'name': '&Base unit&'},
    66: {'value': 5, 'name': '&Five-marker&'},
    99: {'value': 10, 'name': '&Ten-marker&'},
    88: {'value': 50, 'name': '&Fifty-marker&'},
    77: {'value': 100, 'name': '&Hundred-marker&'},
    69: {'value': 500, 'name': '&Five-hundred&'},
    13: {'value': 1000, 'name': '&Thousand-marker&'},
    33: {'value': None, 'name': '&Transformation marker&'},
}

def get_thryznel_data():
    alphabet = []
    for letter in KEFQUWMNEX_ORDER:
        info = THRYZUNEL_ALPHABET[letter]
        glyph = generate_cistercian(info['code'])
        alphabet.append({
            'letter': letter,
            'name': info['name'],

```

```

        &quot;code&quot;;: info[&quot;code&quot;],
        &quot;phoneme&quot;;: info[&quot;phoneme&quot;],
        &quot;novatmpcais&quot;;: info[&quot;novatmpcais&quot;],
        &quot;position&quot;;: info[&quot;position&quot;],
        &quot;origin&quot;;: info[&quot;origin&quot;],
        &quot;glyph&quot;;: glyph,
    })

    punctuation = []
    for code, info in THRYZUNEL_PUNCTUATION.items():
        glyph = generate_cistercian(code)
        punctuation.append({
            &quot;code&quot;;: code,
            &quot;symbol&quot;;: info[&quot;symbol&quot;],
            &quot;name&quot;;: info[&quot;name&quot;],
            &quot;desc&quot;;: info[&quot;desc&quot;],
            &quot;glyph&quot;;: glyph,
        })

    counting = []
    for code, info in THRYZUNEL_COUNTING.items():
        glyph = generate_cistercian(code)
        counting.append({
            &quot;code&quot;;: code,
            &quot;value&quot;;: info[&quot;value&quot;],
            &quot;name&quot;;: info[&quot;name&quot;],
            &quot;glyph&quot;;: glyph,
        })

    return {
        &quot;alphabet&quot;;: alphabet,
        &quot;punctuation&quot;;: punctuation,
        &quot;counting&quot;;: counting,
    }

def get_status():
    return {
        &quot;engine&quot;;: &quot;cistercian&quot;,
        &quot;range&quot;;: &quot;0-9999&quot;,
        &quot;quadrants&quot;;: [
            &quot;upper-right (units/ones)&quot;,
            &quot;upper-left (tens)&quot;,
            &quot;lower-right (hundreds)&quot;,
            &quot;lower-left (thousands)&quot;,
        ],
        &quot;thryzunel&quot;;: {
            &quot;alphabet_count&quot;;: len(THRYZUNEL_ALPHABET),
            &quot;punctuation_count&quot;;: len(THRYZUNEL_PUNCTUATION),
            &quot;counting_tokens&quot;;: len(THRYZUNEL_COUNTING),
        },
        &quot;description&quot;;: &quot;Cistercian numeral system with Thryzunel language encoding&quot;,
    }

```

7. cistercian_math.py

112 lines | 3.7KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/cistercian_math.py

```
import re

def roman_to_int(s):
    """Convert Roman numeral to integer"""
    roman_values = {'I': 1, 'V': 5, 'X': 10, 'L': 50, 'C': 100, 'D': 500, 'M': 1000}
    total = 0
    prev_value = 0
    for char in reversed(s):
        value = roman_values.get(char, 0)
        if value < prev_value:
            total -= value
        else:
            total += value
        prev_value = value
    return total

def is_roman_numeral(s):
    """Check if string is a valid Roman numeral"""
    return bool(re.match(r'^[IVXLCDM]+$', s))

def _digit_to_ascii_ones(d):
    """Convert ones digit to Cistercian ASCII representation"""
    # Basic implementation - you may want to expand this
    patterns = ['', '■', '■', '■', '■', '■', '■', '■', '■']
    return patterns[d] if d < len(patterns) else ''

def _digit_to_ascii_tens(d):
    """Convert tens digit to Cistercian ASCII"""
    patterns = ['', '■', '■', '■', '■', '■', '■', '■', '■']
    return patterns[d] if d < len(patterns) else ''

def _digit_to_ascii_hundreds(d):
    """Convert hundreds digit to Cistercian ASCII"""
    patterns = ['', '■', '■', '■', '■', '■', '■', '■', '■']
    return patterns[d] if d < len(patterns) else ''

def _digit_to_ascii_thousands(d):
    """Convert thousands digit to Cistercian ASCII"""
    patterns = ['', '■', '■', '■', '■', '■', '■', '■', '■']
    return patterns[d] if d < len(patterns) else ''

def render_cistercian_ascii(number):
    """Render a number (0-9999) in Cistercian ASCII art"""
    if number < 0 or number > 9999:
        return str(number)
    thousands = number // 1000
    hundreds = (number % 1000) // 100
    tens = (number % 100) // 10
    ones = number % 10
    # Simple representation
    return f'{{thousands}}{{hundreds}}{{tens}}{{ones}}'

def render_expression_ascii(a, op, b, result):
    """Render arithmetic expression in ASCII art"""
    return f'{{a}} {{op}} {{b}} = {{result}}'

def convert_to_arabic_expression(expr_string):
    """Convert expression with Roman numerals to Arabic"""
    parts = expr_string.split()
    converted = []
    for part in parts:
        if is_roman_numeral(part):
            converted.append(str(roman_to_int(part)))
        else:
            converted.append(part)
    return ' '.join(converted)

def detect_math(text):
    """Detect mathematical expressions in text"""
    # Simple pattern for numbers and operators
    pattern = r'(\d+\s*[+.*\/]\s*\d+)'
    match = re.search(pattern, text)
    return match.group(0) if match else None

def evaluate_math(expr):
    """Evaluate mathematical expression safely"""
    try:
        # Simple evaluation - in production, use safer method
        result = eval(expr)
        return {'expression': expr, 'result': result}
    except:
        return None

def format_math_response(ev, show_ascii=True):
    """Format math response"""
    if not ev:
        return None
    result = f'{{ev[\"expression\"]}} = {{ev[\"result\"]}}'
    if show_ascii:
        result += f'\n{{render_expression_ascii(ev[\"expression\"], \"=\", ev[\"result\"], ev[\"result\"]}}'
    return result

def mask_arabic_numerals(text):
    """Mask Arabic numerals temporarily"""
```

```

    return re.sub(r'\d+', '■NUM■', text)

def unmask_arabic_numerals(masked_text, original_text):
    """Restore masked Arabic numerals"""
    # Simple implementation - you may want more sophisticated
    return original_text

def build_learning_payload(ev):
    """Build payload for learning"""
    return {'type': 'math', 'data': ev}

def process_math(text):
    """Wrapper function for backward compatibility"""
    result = detect_math(text)
    if result:
        ev = evaluate_math(result)
        return format_math_response(ev)
    return None

```

8. cloud_brain.py

334 lines | 11.1KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/cloud_brain.py

```
""""
Quantum Cloud Brain
The brain lives in Google Drive. Local storage is cache only.

Two modes:
1. rclone direct – terminal sync (pull/push via rclone copy)
2. WebDAV – frontend access (rclone serve webdav in separate session)

Google Drive structure:
Quantum Cloud/
  MCAGI_BRAIN/      – brain data (Markov, Hilbert, concepts, library)
  MCAGI_BACKUP/    – full system (code + brain + docs)

Usage:
from cloud_brain import CloudBrain
cloud = CloudBrain()
cloud.startup_pull()      # on startup
cloud.shutdown_push()    # on quit
cloud.push_all()          # manual full push
cloud.backup_full_system() # code + brain backup
""""

import os
import subprocess
import time
import threading
import requests
from pathlib import Path
from functools import wraps
from xml.etree import ElementTree

REMOTE_BRAIN = "gdrive 666:Quantum Cloud/MCAGI_BRAIN"
REMOTE_BACKUP = "gdrive 666:Quantum Cloud/MCAGI_BACKUP"
WEBDAV_URL = "http://127.0.0.1:8080"
LOCAL_CACHE = os.path.expanduser("~/.quantum-mcagi")

def _run_rclone(args, timeout=600):
    try:
        result = subprocess.run(
            ["rclone"] + args,
            capture_output=True, text=True, timeout=timeout
        )
        return result.returncode == 0, result.stderr[:200]
    except subprocess.TimeoutExpired:
        return False, "timeout"
    except FileNotFoundError:
        return False, "rclone not installed"
    except Exception as e:
        return False, str(e)

class CloudBrain:
    def __init__(self, url=WEBDAV_URL, cache_dir=LOCAL_CACHE):
        self.url = url.rstrip('/')
        self.cache = cache_dir
        self.available = False
        self._webdav_active = False
        self._push_lock = threading.Lock()
        self._last_push = 0
        self._push_interval = 60
        os.makedirs(self.cache, exist_ok=True)
        self._check_available()

    def _check_available(self):
        try:
            result = subprocess.run(["rclone", "version"],
                                   capture_output=True, timeout=5)
            self.available = result.returncode == 0
        except:
            self.available = False
            self._webdav_active = False
        if self.available:
            try:
                r = requests.get(self.url + "/", timeout=2)
                self._webdav_active = r.status_code == 200
            except:
                pass
        return self.available

    def _cloud_path(self, relative_path):
        return self.url + "/" + relative_path.lstrip('/')

    def _local_path(self, relative_path):
        return os.path.join(self.cache, relative_path.lstrip('/'))

    def startup_pull(self):
        if not self.available:
            print(" ■ Cloud unavailable – using local cache")
            return False
        print(" ■ Pulling brain from cloud...")
        ok, err = _run_rclone([
```

```

        &quot;copy&quot;;, REMOTE_BRAIN + &quot;/&quot;;, self.cache + &quot;/&quot;;,
        &quot;;--update&quot;;, &quot;;--transfers&quot;;, &quot;4&quot;;, &quot;;--quiet&quot;;
    ])
    if ok:
        print(&quot; ■ Brain loaded from cloud&quot;);
    else:
        print(f&quot;; ■ Pull issue: {err[:80]}&quot;);
    return ok

def shutdown_push(self):
    if not self.available:
        print(&quot; ■ Cloud unavailable – changes cached locally&quot;);
        return False
    print(&quot; ■ Pushing brain to cloud...&quot;);
    ok, err = _run_rclone([
        &quot;copy&quot;;, self.cache + &quot;/&quot;;, REMOTE_BRAIN + &quot;/&quot;;,
        &quot;;--update&quot;;, &quot;;--transfers&quot;;, &quot;4&quot;;,
        &quot;;--exclude&quot;;, &quot;__pycache__/*&quot;;,
        &quot;;--exclude&quot;;, &quot;*.pyc&quot;;, &quot;;--quiet&quot;;
    ])
    if ok:
        print(&quot; ■ Brain saved to cloud&quot;);
    else:
        print(f&quot;; ■ Push issue: {err[:80]}&quot;);
    return ok

def pull_all(self):
    if not self.available:
        print(&quot; ■ Cloud unavailable – using local cache&quot;);
        return False
    print(&quot; ■ Pulling full brain from cloud...&quot;);
    ok, err = _run_rclone([
        &quot;copy&quot;;, REMOTE_BRAIN + &quot;/&quot;;, self.cache + &quot;/&quot;;,
        &quot;;--transfers&quot;;, &quot;4&quot;;, &quot;;--progress&quot;;
    ])
    if ok:
        print(&quot; ■ Full brain loaded from cloud&quot;);
    else:
        print(f&quot;; ■ Pull error: {err[:80]}&quot;);
    return ok

def push_all(self, quiet=False):
    if not self.available:
        if not quiet:
            print(&quot; ■ Cloud unavailable – skipping push&quot;);
            return False
    if not quiet:
        print(&quot; ■ Pushing brain to cloud...&quot;);
    ok, err = _run_rclone([
        &quot;copy&quot;;, self.cache + &quot;/&quot;;, REMOTE_BRAIN + &quot;/&quot;;,
        &quot;;--update&quot;;, &quot;;--transfers&quot;;, &quot;4&quot;;,
        &quot;;--exclude&quot;;, &quot;__pycache__/*&quot;;,
        &quot;;--exclude&quot;;, &quot;*.pyc&quot;;, &quot;;--progress&quot;;
    ])
    if ok and not quiet:
        print(&quot; ■ Brain synced to cloud&quot;);
    elif not ok and not quiet:
        print(f&quot;; ■ Push error: {err[:80]}&quot;);
    return ok

def upload_all(self, quiet=False):
    return self.push_all(quiet=quiet)

def push_state(self):
    if not self.available:
        return False
    now = time.time()
    if now - self._last_push < self._push_interval:
        return True
    t = threading.Thread(target=self._do_push_state, daemon=True)
    t.start()
    return True

def _do_push_state(self):
    with self._push_lock:
        _run_rclone([
            &quot;copy&quot;;, self.cache + &quot;/&quot;;, REMOTE_BRAIN + &quot;/&quot;;,
            &quot;;--update&quot;;, &quot;;--transfers&quot;;, &quot;2&quot;;,
            &quot;;--include&quot;;, &quot;engine_state/*&quot;;,
            &quot;;--include&quot;;, &quot;hilbert/*&quot;;,
            &quot;;--include&quot;;, &quot;*.json&quot;;,
            &quot;;--quiet&quot;;
        ])
        self._last_push = time.time()

def backup_full_system(self, local_code_dir=None):
    if local_code_dir is None:
        local_code_dir = os.path.dirname(os.path.abspath(__file__))
    if not self.available:
        print(&quot; ■ Cloud unavailable – cannot backup&quot;);
        return False
    print(&quot; ■ Backing up full system to cloud...&quot;);
    ok, err = _run_rclone([
        &quot;copy&quot;;, local_code_dir, REMOTE_BACKUP + &quot;/&quot;;,
        &quot;;--exclude&quot;;, &quot;__pycache__/*&quot;;,
        &quot;;--exclude&quot;;, &quot;*.pyc&quot;;,
        &quot;;--transfers&quot;;, &quot;4&quot;;, &quot;;--progress&quot;;
    ])
    if ok:
        print(&quot; ■ Full system backed up to {REMOTE_BACKUP}&quot;);
    else:

```

```

        print(f"&#x2022; Backup error: {err[:80]}&#x2022;")
    return ok

def cloud_status(self):
    if not self.available:
        print("&#x2022; rclone: NOT AVAILABLE&#x2022;")
        return
    print(f"&#x2022; rclone: AVAILABLE&#x2022;")
    print(f"&#x2022; WebDAV: {'ACTIVE' if self._webdav_active else 'INACTIVE'}&#x2022;")
    print(f"&#x2022; Brain remote: {REMOTE_BRAIN}&#x2022;")
    print(f"&#x2022; Backup remote: {REMOTE_BACKUP}&#x2022;")
    print(f"&#x2022; Local cache: {self.cache}&#x2022;")

def download(self, cloud_path, local_path=None):
    if local_path is None:
        local_path = self._local_path(cloud_path)
    os.makedirs(os.path.dirname(local_path), exist_ok=True)
    if self._webdav_active:
        try:
            r = requests.get(self._cloud_path(cloud_path), timeout=300, stream=True)
            if r.status_code == 200:
                with open(local_path, 'wb') as f:
                    for chunk in r.iter_content(chunk_size=8192):
                        f.write(chunk)
                return True
        except:
            pass
    ok, _ = _run_rclone(['&#x2022;copyto&#x2022;', REMOTE_BRAIN + '&#x2022;/&#x2022;' + cloud_path, local_path])
    return ok

def upload(self, local_path, cloud_path=None):
    if cloud_path is None:
        if local_path.startswith(self.cache):
            cloud_path = local_path[len(self.cache):]
        else:
            cloud_path = os.path.basename(local_path)
    if self._webdav_active:
        try:
            parts = cloud_path.strip('/').split('/')
            for i in range(len(parts) - 1):
                dir_path = '/'.join(parts[:i+1])
                requests.request('MKCOL', self._cloud_path(dir_path + '/'), timeout=10)
            with open(local_path, 'rb') as f:
                r = requests.put(self._cloud_path(cloud_path), data=f, timeout=600)
            return r.status_code in (200, 201, 204)
        except:
            pass
    ok, _ = _run_rclone(['&#x2022;copyto&#x2022;', local_path, REMOTE_BRAIN + '&#x2022;/&#x2022;' + cloud_path])
    return ok

def upload_background(self, local_path, cloud_path=None):
    t = threading.Thread(target=self.upload, args=(local_path, cloud_path), daemon=True)
    t.start()

def list_files(self, cloud_dir=&#x2022;/&#x2022;):
    if self._webdav_active:
        try:
            r = requests.request('PROPFIND', self._cloud_path(cloud_dir),
                                headers={'Depth': '1'}, timeout=30)
            if r.status_code in (200, 207):
                files = []
                root = ElementTree.fromstring(r.content)
                ns = {'d': 'DAV:'}
                for response in root.findall('.//d:response', ns):
                    href = response.find('d:href', ns)
                    if href is not None:
                        files.append(href.text)
                return files
        except:
            pass
    return []

def exists(self, cloud_path):
    if self._webdav_active:
        try:
            r = requests.head(self._cloud_path(cloud_path), timeout=10)
            return r.status_code == 200
        except:
            pass
    return False

def wire_engine(self, engine):
    _original_save = engine.save_state
    _original_load = engine.load_state
    cloud = self

    @wraps(_original_save)
    def patched_save(path, *args, **kwargs):
        result = _original_save(path, *args, **kwargs)
        if cloud.available:
            cloud.push_state()
        return result

    @wraps(_original_load)
    def patched_load(path, *args, **kwargs):
        return _original_load(path, *args, **kwargs)

    engine.save_state = patched_save
    engine.load_state = patched_load
    print("&#x2022; Engine wired to cloud brain&#x2022;")

def wire_chat(self):

```

```

        return self.pull_all, self.push_state

_instance = None

def get_cloud_brain():
    global _instance
    if _instance is None:
        _instance = CloudBrain()
    return _instance

if __name__ == '__main__':
    import sys
    brain = CloudBrain()
    print(f"rclone available: {brain.available}")
    print(f"WebDAV active: {brain._webdav_active}")
    print(f"Cache: {brain.cache}")

    if len(sys.argv) > 1:
        cmd = sys.argv[1]
        if cmd == 'pull':
            brain.pull_all()
        elif cmd == 'push':
            brain.push_all()
        elif cmd == 'backup':
            brain.backup_full_system()
        elif cmd == 'status':
            brain.cloud_status()
    else:
        print()
        print("Commands:")
        print("python cloud_brain.py pull    - Pull brain from cloud")
        print("python cloud_brain.py push    - Push brain to cloud")
        print("python cloud_brain.py backup    - Full system backup")
        print("python cloud_brain.py status    - Cloud status")

```

9. code_to_pdf.py

228 lines | 6.7KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V███/backend/code_to_pdf.py

```
#!/usr/bin/env python3
"""&quot;&quot;
Generate a PDF containing all Python source code from the MCAGI backend.
Run this in Termux to create a complete code reference document.

Usage:
    python code_to_pdf.py                # defaults to backend/
    python code_to_pdf.py /path/to/backend/  # custom path
    python code_to_pdf.py /path/ output.pdf  # custom output
"&quot;&quot;&quot;&quot;

import os
import sys
from datetime import datetime

from reportlab.lib.pagesizes import letter
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.colors import HexColor
from reportlab.lib.units import inch
from reportlab.platypus import (
    SimpleDocTemplate, Paragraph, Spacer, PageBreak,
    Table, TableStyle, Preformatted
)
from reportlab.lib.enums import TA_CENTER

def get_python_files(directory):
    &quot;&quot;&quot;&quot;&quot;Get all .py files sorted by name.&quot;&quot;&quot;&quot;&quot;
    files = []
    for f in sorted(os.listdir(directory)):
        if f.endswith('.py') and not f.startswith('__'):
            path = os.path.join(directory, f)
            size = os.path.getsize(path)
            lines = 0
            try:
                with open(path, 'r', encoding='utf-8', errors='ignore') as fh:
                    lines = sum(1 for _ in fh)
            except:
                pass
            files.append({
                'name': f,
                'path': path,
                'size': size,
                'lines': lines,
            })
    return files

def escape_xml(text):
    &quot;&quot;&quot;&quot;&quot;Escape text for ReportLab XML.&quot;&quot;&quot;&quot;&quot;
    text = text.replace('&amp;', '&amp;amp;')
    text = text.replace('&lt;', '&amp;lt;')
    text = text.replace('&gt;', '&amp;gt;')
    text = text.replace('&quot;', '&amp;quot;')
    return text

def build_pdf(source_dir, output_path):
    &quot;&quot;&quot;&quot;&quot;Build the complete code PDF.&quot;&quot;&quot;&quot;&quot;
    files = get_python_files(source_dir)
    total_lines = sum(f['lines'] for f in files)
    total_size = sum(f['size'] for f in files)

    print(f"Found {len(files)} Python files")
    print(f"Total: {total_lines:,} lines, {total_size/1024:.0f}KB")
    print(f"Generating PDF...&quot;)

    doc = SimpleDocTemplate(
        output_path,
        pagesize=letter,
        leftMargin=0.5*inch,
        rightMargin=0.5*inch,
        topMargin=0.5*inch,
        bottomMargin=0.5*inch,
    )

    styles = getSampleStyleSheet()

    title_style = ParagraphStyle(
        'CustomTitle',
        parent=styles['Title'],
        fontSize=24,
        textColor=HexColor('#1a1a2e'),
        spaceAfter=6,
    )

    subtitle_style = ParagraphStyle(
        'Subtitle',
        parent=styles['Normal'],
        fontSize=11,
        textColor=HexColor('#555555'),
        alignment=TA_CENTER,
        spaceAfter=20,
    )
```

```

file_header_style = ParagraphStyle(
    'FileHeader',
    parent=styles['Heading1'],
    fontSize=14,
    textColor=HexColor('#0a3d62'),
    spaceBefore=10,
    spaceAfter=6,
    borderWidth=1,
    borderColor=HexColor('#0a3d62'),
    borderPadding=4,
)

file_meta_style = ParagraphStyle(
    'FileMeta',
    parent=styles['Normal'],
    fontSize=8,
    textColor=HexColor('#888888'),
    spaceAfter=8,
)

code_style = ParagraphStyle(
    'Code',
    parent=styles['Code'],
    fontName='Courier',
    fontSize=5.5,
    leading=7,
    leftIndent=4,
    rightIndent=4,
    textColor=HexColor('#1a1a1a'),
    backColor=HexColor('#f5f5f5'),
)

story = []

# Title Page
story.append(Spacer(1, 100))
story.append(Paragraph('"Quantum MCAGI";', title_style))
story.append(Paragraph('"Complete Source Code";', ParagraphStyle(
    'Sub', parent=styles['Heading2'], alignment=TA_CENTER, textColor=HexColor('#333')
)))
story.append(Spacer(1, 20))
story.append(Paragraph(
    f"Files: {len(files)} | Lines: {total_lines:,} | Size: {total_size/1024:.0f}KB";,
    subtitle_style
))
story.append(Paragraph(
    f"Generated: {datetime.now().strftime('%Y-%m-%d %H:%M')}";,
    subtitle_style
))
story.append(Paragraph(
    "By Cory Nathaniel Bonsib Blackburn";,
    subtitle_style
))
story.append(Paragraph(
    "No LLM. No API. Pure algorithms. Built on Android/Termux.";,
    subtitle_style
))
story.append(PageBreak())

# Table of Contents
story.append(Paragraph('"Table of Contents";', styles['Heading1']))
story.append(Spacer(1, 10))

toc_data = [['#', 'File', 'Lines', 'Size']]
for i, f in enumerate(files, 1):
    toc_data.append([
        str(i),
        f['name'],
        f"{f['lines']:,}",
        f"{f['size']/1024:.1f}KB"
    ])

toc_table = Table(toc_data, colwidths=[30, 300, 60, 60])
toc_table.setStyle(TableStyle([
    ('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
    ('FONTSIZE', (0, 0), (-1, -1), 8),
    ('BACKGROUND', (0, 0), (-1, 0), HexColor('#0a3d62')),
    ('TEXTCOLOR', (0, 0), (-1, 0), HexColor('ffffff')),
    ('ROWBACKGROUNDS', (0, 1), (-1, -1), [HexColor('ffffff'), HexColor('f0f0f0')]),
    ('GRID', (0, 0), (-1, -1), 0.5, HexColor('cccccc')),
    ('ALIGN', (0, 0), (0, -1), 'CENTER'),
    ('ALIGN', (2, 0), (3, -1), 'RIGHT'),
    ('VALIGN', (0, 0), (-1, -1), 'MIDDLE'),
    ('TOPPADDING', (0, 0), (-1, -1), 3),
    ('BOTOMPADDING', (0, 0), (-1, -1), 3),
]))
story.append(toc_table)
story.append(PageBreak())

# Code Files
for i, f in enumerate(files, 1):
    print(f" [{i}/{len(files)}] {f['name']} ({f['lines']} lines)";)

    story.append(Paragraph(
        f"{i}. {escape_xml(f['name'])}";,
        file_header_style
    ))
    story.append(Paragraph(
        f"{f['lines']:,} lines | {f['size']/1024:.1f}KB | {f['path']}";,
        file_meta_style
    ))

```

```

try:
    with open(f['path'], 'r', encoding='utf-8', errors='ignore') as fh:
        code = fh.read()
except Exception as e:
    code = f"# Error reading file: {e}"

escaped = escape_xml(code)
code_lines = escaped.split("\n")
max_lines = 500

for chunk_start in range(0, len(code_lines), max_lines):
    chunk = "\n".join(code_lines[chunk_start:chunk_start + max_lines])
    story.append(Preformatted(chunk, code_style))

story.append(PageBreak())

print(f"Building PDF...")
doc.build(story)
size = os.path.getsize(output_path)
print(f"Done: {output_path} ({size/1024/1024:.1f}MB)")

if __name__ == '__main__':
    source = sys.argv[1] if len(sys.argv) > 1 else os.path.expanduser('~/.Quantum_MCAGI_NO_LLM_V2/backend')
    output = sys.argv[2] if len(sys.argv) > 2 else os.path.expanduser('~/.storage/downloads/quantum_mcagi_code.pdf')

    if not os.path.isdir(source):
        print(f"Directory not found: {source}")
        sys.exit(1)

    build_pdf(source, output)

```

439 lines | 17.9KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/comprehension_engine.py

```
class ComprehensionEngine:
    """Understands user input beyond keyword extraction"""

    def __init__(self):
        self.conversation_thread = []
        self.active_claims = []
        self.active_topic_stack = []
        self.unresolved_questions = []

    def comprehend(self, user_input: str, concepts: List[str],
                  context: Optional[Dict] = None) -> Dict:
        text = user_input.strip()
        lower = text.lower()
```

```

intent = self._detect_intent(text, lower, context)
claims = self._extract_claims(text, lower)
relationships = self._extract_relationships(lower)
stance = self._detect_stance(lower, context)
complexity = self._assess_complexity(text, lower, concepts)
thread_position = self._assess_thread_position(text, lower, concepts, context)

self._update_thread(user_input, intent, claims, concepts)

engagement_directives = self._generate_directives(
    intent, claims, relationships, stance, complexity, thread_position, concepts, context
)

return {
    'intent': intent,
    'claims': claims,
    'relationships': relationships,
    'stance': stance,
    'complexity': complexity,
    'thread_position': thread_position,
    'directives': engagement_directives,
    'active_claims': self.active_claims[-5:],
    'topic_stack': self.active_topic_stack[-3:],
}

def _detect_intent(self, text: str, lower: str, context: Optional[Dict]) -> Dict:
    sentences = re.split(r'[.!?]+', text)
    sentences = [s.strip() for s in sentences if s.strip()]

    intents = []
    for sent in sentences:
        sent_lower = sent.lower()
        words = sent_lower.split()
        if not words:
            continue

        if words[0] in QUESTION_STARTERS or sent.endswith('?'):
            q_type = self._classify_question(sent_lower, words)
            intents.append({'type': 'question', 'subtype': q_type, 'text': sent})
        elif any(re.search(p, sent_lower) for p in OPINION_MARKERS):
            intents.append({'type': 'opinion', 'text': sent})
        elif any(re.search(p, sent_lower) for p in CONTRAST_MARKERS):
            intents.append({'type': 'counterpoint', 'text': sent})
        elif len(words) <= 3 and any(re.search(p, sent_lower) for p in AGREEMENT_MARKERS):
            intents.append({'type': 'agreement', 'text': sent})
        elif any(re.search(p, sent_lower) for p in DISAGREEMENT_MARKERS):
            intents.append({'type': 'disagreement', 'text': sent})
        elif any(re.search(pat, sent_lower) for pat, _ in CAUSAL_MARKERS):
            intents.append({'type': 'argument', 'text': sent})
        elif len(words) <= 4:
            intents.append({'type': 'brief', 'text': sent})
        else:
            intents.append({'type': 'statement', 'text': sent})

    primary = intents[0] if intents else {'type': 'statement', 'text': text}
    return {
        'primary': primary['type'],
        'all': intents,
        'has_question': any(i['type'] == 'question' for i in intents),
        'has_argument': any(i['type'] == 'argument' for i in intents),
        'has_opinion': any(i['type'] == 'opinion' for i in intents),
        'has_counterpoint': any(i['type'] == 'counterpoint' for i in intents),
    }

def _classify_question(self, sent_lower: str, words: List[str]) -> str:
    first = words[0] if words else ''
    if first == 'why':
        return 'causal'
    elif first == 'how':
        if len(words) > 1 and words[1] in ('much', 'many', 'long', 'far', 'often'):
            return 'quantitative'
        return 'mechanistic'
    elif first == 'what':
        if 'mean' in sent_lower or 'definition' in sent_lower:
            return 'definitional'
        if 'happen' in sent_lower or 'would' in sent_lower:
            return 'hypothetical'
        return 'factual'
    elif first == 'who':
        return 'identity'
    elif first in ('is', 'are', 'was', 'were', 'do', 'does', 'did', 'can', 'could'):
        return 'yes_no'
    elif first in ('would', 'should', 'could'):
        return 'hypothetical'
    return 'open'

def _extract_claims(self, text: str, lower: str) -> List[Dict]:
    claims = []
    sentences = re.split(r'[.!?]+', text)

    for sent in sentences:
        sent = sent.strip()
        if not sent:
            continue
        sent_lower = sent.lower()

        if sent.endswith('?') or (sent_lower.split() and sent_lower.split()[0] in QUESTION_STARTERS):
            continue

        for pattern, marker_type in CAUSAL_MARKERS:
            if re.search(pattern, sent_lower):

```

```

        claims.append({
            'text': sent,
            'type': marker_type,
            'strength': 'strong' if marker_type in ('conclusion', 'causation') else 'moderate',
        })
        break
    else:
        has_strong_assertion = any(re.search(p, sent_lower) for p in
            [r'\balways\b', r'\bnever\b', r'\bmust\b', r'\bclearly\b', r'\bobviously\b', r'\bcertainly\b'])

        if has_strong_assertion:
            claims.append({
                'text': sent,
                'type': 'assertion',
                'strength': 'strong',
            })
        elif len(sent.split()) >= 5 and any(re.search(p, sent_lower) for p in ASSERTION_MARKERS[:4]):
            has_opinion = any(re.search(p, sent_lower) for p in OPINION_MARKERS)
            claims.append({
                'text': sent,
                'type': 'opinion' if has_opinion else 'claim',
                'strength': 'moderate' if has_opinion else 'moderate',
            })

    return claims

def _extract_relationships(self, lower: str) -> List[Dict]:
    relationships = []
    for pattern, rel_type in RELATIONSHIP_PATTERNS:
        matches = re.finditer(pattern, lower)
        for m in matches:
            if m.lastindex and m.lastindex >= 2:
                relationships.append({
                    'subject': m.group(1),
                    'object': m.group(2),
                    'relation': rel_type,
                })
    return relationships

def _detect_stance(self, lower: str, context: Optional[Dict]) -> Dict:
    agrees = sum(1 for p in AGREEMENT_MARKERS if re.search(p, lower))
    disagrees = sum(1 for p in DISAGREEMENT_MARKERS if re.search(p, lower))
    contrasts = sum(1 for p in CONTRAST_MARKERS if re.search(p, lower))

    if disagrees >= 0 or contrasts >= 1:
        position = 'opposing'
    elif agrees >= 0 and disagrees == 0:
        position = 'aligned'
    elif contrasts == 1:
        position = 'nuancing'
    else:
        position = 'neutral'

    return {
        'position': position,
        'agreement_signals': agrees,
        'disagreement_signals': disagrees,
        'contrast_signals': contrasts,
    }

def _assess_complexity(self, text: str, lower: str, concepts: List[str]) -> Dict:
    words = lower.split()
    sentences = re.split(r'[.!?]+', text)
    sentences = [s for s in sentences if s.strip()]

    subordinate_clauses = len(re.findall(r'\b(because|although|while|whereas|since|if|unless|when|after|before)\b', lower))
    unique_ratio = len(set(words)) / max(len(words), 1)

    if len(words) <= 5:
        level = 'minimal'
    elif len(words) <= 15 and subordinate_clauses == 0:
        level = 'simple'
    elif subordinate_clauses >= 2 or len(concepts) >= 4:
        level = 'complex'
    elif len(words) >= 30 or len(sentences) >= 3:
        level = 'elaborate'
    else:
        level = 'moderate'

    return {
        'level': level,
        'word_count': len(words),
        'sentence_count': len(sentences),
        'subordinate_clauses': subordinate_clauses,
        'vocabulary_richness': round(unique_ratio, 3),
        'concept_density': round(len(concepts) / max(len(words), 1), 3),
    }

def _assess_thread_position(self, text: str, lower: str,
                           concepts: List[str], context: Optional[Dict]) -> Dict:
    if not context or not context.get('recent_exchanges'):
        return {'position': 'opening', 'depth': 0, 'topic_continuity': 0.0}

    recent = context.get('recent_exchanges', [])
    depth = len(recent)
    recent_concepts = context.get('recent_concepts', [])

    if recent_concepts and concepts:
        shared = set(c.lower() for c in concepts) & set(c.lower() for c in recent_concepts)
        continuity = len(shared) / max(len(concepts), 1)
    else:

```

```

continuity = 0.0

last_ai = context.get('last_ai_response', '')
references_response = False
if last_ai:
    ai_words = set(last_ai.lower().split())
    user_words = set(lower.split())
    overlap = ai_words & user_words - {'the', 'a', 'is', 'are', 'was', 'it', 'that', 'this', 'to', 'of', 'and', 'in', 'for'}
    references_response = len(overlap) >= 3

if depth == 0:
    position = 'opening'
elif continuity > 0.5 and references_response:
    position = 'deep_engagement'
elif continuity > 0.3:
    position = 'continuing'
elif continuity > 0.0:
    position = 'branching'
else:
    position = 'new_thread'

return {
    'position': position,
    'depth': depth,
    'topic_continuity': round(continuity, 3),
    'references_previous': references_response,
}

def _update_thread(self, user_input: str, intent: Dict,
                  claims: List[Dict], concepts: List[str]):
    self.conversation_thread.append({
        'input': user_input,
        'intent': intent['primary'],
        'concepts': concepts,
    })
    if len(self.conversation_thread) > 20:
        self.conversation_thread = self.conversation_thread[-20:]

    for claim in claims:
        self.active_claims.append(claim)
    if len(self.active_claims) > 10:
        self.active_claims = self.active_claims[-10:]

    if concepts:
        for c in concepts:
            if c not in self.active_topic_stack:
                self.active_topic_stack.append(c)
        if len(self.active_topic_stack) > 8:
            self.active_topic_stack = self.active_topic_stack[-8:]

def _generate_directives(self, intent: Dict, claims: List[Dict],
                        relationships: List[Dict], stance: Dict,
                        complexity: Dict, thread_position: Dict,
                        concepts: List[str], context: Optional[Dict]) -> Dict:

    directives = {
        'respond_to_argument': False,
        'answer_question': False,
        'acknowledge_stance': False,
        'match_complexity': complexity['level'],
        'engagement_mode': 'default',
        'response_seeds': [],
        'avoid': [],
    }

    if intent['has_question']:
        directives['answer_question'] = True
        questions = [i for i in intent['all'] if i['type'] == 'question']
        if questions:
            q = questions[0]
            subtype = q.get('subtype', 'open')
            if subtype == 'causal':
                directives['engagement_mode'] = 'explain_why'
                directives['response_seeds'].append(f'address_cause_of_{concepts[0] if concepts else 'topic'}&quot;')
            elif subtype == 'mechanistic':
                directives['engagement_mode'] = 'explain_how'
                directives['response_seeds'].append(f'explain_mechanism_{concepts[0] if concepts else 'topic'}&quot;')
            elif subtype == 'definitional':
                directives['engagement_mode'] = 'define'
                directives['response_seeds'].append(f'define_{concepts[0] if concepts else 'topic'}&quot;')
            elif subtype == 'yes_no':
                directives['engagement_mode'] = 'take_position'
            elif subtype == 'hypothetical':
                directives['engagement_mode'] = 'explore_possibility'
            else:
                directives['engagement_mode'] = 'address_question'

    if intent['has_argument'] or claims:
        directives['respond_to_argument'] = True
        if not directives['answer_question']:
            directives['engagement_mode'] = 'engage_argument'
        for claim in claims[2:]:
            directives['response_seeds'].append(f'engage_{claim['type']}: {claim['text'][:60]}&quot;')

    if stance['position'] in ('opposing', 'nuancing'):
        directives['acknowledge_stance'] = True
        directives['avoid'].append('repeat_previous_point')
        if stance['position'] == 'opposing':
            directives['engagement_mode'] = 'counter_or_concede'

    if stance['position'] == 'aligned':
        directives['avoid'].append('redundant_agreement')

```

```

        if directives['engagement_mode'] == 'default':
            directives['engagement_mode'] = 'build_on'

    if thread_position['position'] == 'deep_engagement':
        directives['avoid'].append('surface_level_response')
        if directives['engagement_mode'] == 'default':
            directives['engagement_mode'] = 'go_deeper'

    if thread_position['position'] == 'new_thread':
        if directives['engagement_mode'] == 'default':
            directives['engagement_mode'] = 'fresh_take'

    if thread_position['position'] == 'opening':
        if directives['engagement_mode'] == 'default':
            directives['engagement_mode'] = 'establish_ground'

    if complexity['level'] == 'minimal':
        if directives['engagement_mode'] == 'default':
            directives['engagement_mode'] = 'brief_and_direct'

    if complexity['level'] in ('complex', 'elaborate') and directives['engagement_mode'] == 'default':
        directives['engagement_mode'] = 'match_depth'

    if relationships:
        for rel in relationships[:2]:
            directives['response_seeds'].append(
                f"<div>relationship:{rel['subject']}_{rel['relation']}_{rel['object']}</div>"
            )

    if directives['engagement_mode'] == 'default':
        directives['engagement_mode'] = 'conversational'

    return directives

def get_status(self) -> Dict:
    return {
        'thread_length': len(self.conversation_thread),
        'active_claims': len(self.active_claims),
        'topic_stack': self.active_topic_stack[-5:],
        'last_intent': self.conversation_thread[-1]['intent'] if self.conversation_thread else None,
    }

```

11. concept_ontology.py

363 lines | 14.7KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/concept_ontology.py

```
"""""
■ CONCEPT ONTOLOGY
The knowledge base of Quantum MCAGI's understanding of the world.
Maps concepts to physical parameters, rendering strategies, and complexity.
"""""

from dataclasses import dataclass, field
from typing import Dict, List, Optional, Tuple, Any
from enum import Enum

class RenderingStrategy(Enum):
    """""How to render this concept."""""
    BLACK_HOLE = "black_hole"
    HUMAN_FIGURE = "human_figure"
    GEOMETRIC = "geometric"
    PARTICLE_SYSTEM = "particle_system"
    COMPOSITE = "composite"

class PhysicalProperty(Enum):
    """""Types of physical properties a concept can have."""""
    MASS = "mass"
    RADIUS = "radius"
    POSITION = "position"
    ORIENTATION = "orientation"
    TEMPERATURE = "temperature"
    SPIN = "spin"
    CHARGE = "charge"
    COLOR = "color"
    TEXTURE = "texture"
    LUMINOSITY = "luminosity"

@dataclass
class ConceptDefinition:
    """""
    Complete definition of a concept in the ontology.
    This is what the AI "knows" about each concept.
    """""
    name: str
    category: str # e.g., "astrophysical", "biological", "geometric"
    base_complexity: float # 0.0 (easiest) to 1.0 (hardest)

    # Rendering strategy
    strategy: RenderingStrategy

    # Physical parameters (with ranges for variability)
    parameters: Dict[str, Dict[str, Any]] = field(default_factory=dict)
    # Example: {
    #     "mass": {"type": "float", "min": 1.0, "max": 100.0, "units": "solar_masses", "default": 1.0},
    #     "spin": {"type": "float", "min": 0.0, "max": 0.99, "default": 0.5}
    # }

    # Visual properties
    visual_properties: Dict[str, Any] = field(default_factory=dict)
    # Example: {
    #     "color_palette": ["black", "orange", "yellow"],
    #     "glow_intensity": 0.8,
    #     "requires_lighting": True
    # }

    # What sub-components does this concept have?
    components: List[Dict[str, Any]] = field(default_factory=list)
    # Example: [
    #     {"name": "event_horizon", "type": "circle", "z_index": 0},
    #     {"name": "accretion_disk", "type": "ring", "z_index": 1}
    # ]

    # Complexity scaling: how detail increases with complexity parameter
    detail_functions: Dict[str, str] = field(default_factory=dict)
    # Example: {
    #     "num_particles": "int(100 + 900 * complexity)",
    #     "ring_segments": "int(10 + 40 * complexity)"
    # }

    # Required physics engines
    physics_engines: List[str] = field(default_factory=list)

    # Dependencies: other concepts needed to understand this one
    prerequisites: List[str] = field(default_factory=list)

class ConceptOntology:
    """""
    The master knowledge base.
    Contains all concepts the AI understands with their complete definitions.
    """""

    def __init__(self):
        self._concepts: Dict[str, ConceptDefinition] = {}
        self._initialize_concepts()

    def _initialize_concepts(self):
```

"""Build the complete ontology tree."""

```
# =====  
# BLACK HOLE (complexity 1.0 - HARDEST)  
# =====
```

```
black_hole = ConceptDefinition(  
    name=&quot;black hole&quot;;  
    category=&quot;astrophysical&quot;;  
    base_complexity=1.0,  
    strategy=RenderingStrategy.BLACK_HOLE,  
    parameters={  
        &quot;mass&quot;: {&quot;type&quot;: &quot;float&quot;, &quot;min&quot;: 5.0, &quot;max&quot;: 1000.0, &quot;units&quot;: &quot;solar_masses&quot;, &quot;default&quot;: 10},  
        &quot;spin&quot;: {&quot;type&quot;: &quot;float&quot;, &quot;min&quot;: 0.0, &quot;max&quot;: 0.998, &quot;default&quot;: 0.5},  
        &quot;charge&quot;: {&quot;type&quot;: &quot;float&quot;, &quot;min&quot;: 0.0, &quot;max&quot;: 1.0, &quot;default&quot;: 0.0},  
        &quot;orientation&quot;: {&quot;type&quot;: &quot;vector3d&quot;, &quot;default&quot;: [0, 0, 1]},  
    },  
    visual_properties={  
        &quot;color_palette&quot;: [&quot;black&quot;, &quot;orange&quot;, &quot;yellow&quot;, &quot;white&quot;, &quot;blue&quot;],  
        &quot;event_horizon_color&quot;: (0, 0, 0),  
        &quot;photon_sphere_color&quot;: (255, 255, 220),  
        &quot;accretion_disk_colors&quot;: [(255, 255, 200), (255, 200, 50), (255, 100, 0), (200, 50, 0)],  
        &quot;jet_color&quot;: (100, 150, 255),  
        &quot;glow_intensity&quot;: 0.9,  
    },  
    components=[  
        {&quot;name&quot;: &quot;singularity&quot;, &quot;type&quot;: &quot;point&quot;, &quot;radius&quot;: 0, &quot;z_index&quot;: 0},  
        {&quot;name&quot;: &quot;event_horizon&quot;, &quot;type&quot;: &quot;circle&quot;, &quot;z_index&quot;: 1},  
        {&quot;name&quot;: &quot;photon_sphere&quot;, &quot;type&quot;: &quot;ring&quot;, &quot;z_index&quot;: 2},  
        {&quot;name&quot;: &quot;accretion_disk&quot;, &quot;type&quot;: &quot;torus&quot;, &quot;z_index&quot;: 3},  
        {&quot;name&quot;: &quot;gravitational_lensing&quot;, &quot;type&quot;: &quot;effect&quot;, &quot;z_index&quot;: 4},  
        {&quot;name&quot;: &quot;relativistic_jets&quot;, &quot;type&quot;: &quot;cones&quot;, &quot;z_index&quot;: 5},  
    ],  
    detail_functions={  
        &quot;disk_rings&quot;: &quot;int(15 + 20 * complexity)&quot;;  
        &quot;lensing_stars&quot;: &quot;int(40 + 80 * complexity)&quot;;  
        &quot;jet_particles&quot;: &quot;int(20 + 40 * complexity)&quot;;  
        &quot;ring_segments&quot;: &quot;int(30 + 50 * complexity)&quot;;  
    },  
    physics_engines=[&quot;general_relativity_raytracing&quot;, &quot;magnetohydrodynamics&quot;],  
    prerequisites=[&quot;gravity&quot;, &quot;spacetime&quot;])  
self._concepts[&quot;black hole&quot;] = black_hole
```

```
# =====  
# STICK FIGURE (complexity 0.05 - EASIEST)  
# =====
```

```
stick_figure = ConceptDefinition(  
    name=&quot;stick figure&quot;;  
    category=&quot;biological&quot;;  
    base_complexity=0.05,  
    strategy=RenderingStrategy.HUMAN_FIGURE,  
    parameters={  
        &quot;height&quot;: {&quot;type&quot;: &quot;float&quot;, &quot;min&quot;: 0.5, &quot;max&quot;: 2.0, &quot;units&quot;: &quot;m&quot;, &quot;default&quot;: 1.0},  
        &quot;pose&quot;: {&quot;type&quot;: &quot;string&quot;, &quot;options&quot;: [&quot;standing&quot;, &quot;walking&quot;, &quot;t-pose&quot;], &quot;default&quot;: &quot;standing&quot;},  
        &quot;simplcity&quot;: {&quot;type&quot;: &quot;float&quot;, &quot;min&quot;: 0.0, &quot;max&quot;: 1.0, &quot;default&quot;: 0.9},  
    },  
    visual_properties={  
        &quot;color_palette&quot;: [&quot;white&quot;, &quot;gray&quot;, &quot;light_gray&quot;],  
        &quot;line_width_base&quot;: 3,  
        &quot;joint_size&quot;: 5,  
        &quot;requires_anatomy&quot;: False,  
    },  
    components=[  
        {&quot;name&quot;: &quot;head&quot;, &quot;type&quot;: &quot;circle&quot;, &quot;parent&quot;: &quot;torso&quot;, &quot;z_index&quot;: 0},  
        {&quot;name&quot;: &quot;torso&quot;, &quot;type&quot;: &quot;line&quot;, &quot;z_index&quot;: 1},  
        {&quot;name&quot;: &quot;arms&quot;, &quot;type&quot;: &quot;limbs&quot;, &quot;count&quot;: 2, &quot;z_index&quot;: 2},  
        {&quot;name&quot;: &quot;legs&quot;, &quot;type&quot;: &quot;limbs&quot;, &quot;count&quot;: 2, &quot;z_index&quot;: 3},  
    ],  
    detail_functions={  
        &quot;joint_detail&quot;: &quot;int(1 + 4 * complexity)&quot;, # 1=simple circle, 5=detailed joint  
        &quot;finger_segments&quot;: &quot;int(0 + 3 * complexity)&quot;;  
        &quot;foot_detail&quot;: &quot;int(1 + 4 * complexity)&quot;;  
        &quot;muscle_definition&quot;: &quot;complexity * 0.3&quot;;  
    },  
    physics_engines=[&quot;forward_kinematics&quot;],  
    prerequisites=[])  
self._concepts[&quot;stick figure&quot;] = stick_figure
```

```
# =====  
# HUMAN (complexity 0.2)  
# =====
```

```
human = ConceptDefinition(  
    name=&quot;human&quot;;  
    category=&quot;biological&quot;;  
    base_complexity=0.2,  
    strategy=RenderingStrategy.HUMAN_FIGURE,  
    parameters={  
        &quot;height&quot;: {&quot;type&quot;: &quot;float&quot;, &quot;min&quot;: 1.4, &quot;max&quot;: 2.0, &quot;units&quot;: &quot;m&quot;, &quot;default&quot;: 1.7},  
        &quot;gender&quot;: {&quot;type&quot;: &quot;string&quot;, &quot;options&quot;: [&quot;male&quot;, &quot;female&quot;], &quot;default&quot;: &quot;person&quot;},  
        &quot;body_type&quot;: {&quot;type&quot;: &quot;string&quot;, &quot;options&quot;: [&quot;athletic&quot;, &quot;average&quot;, &quot;slim&quot;], &quot;default&quot;: &quot;average&quot;},  
    },  
    visual_properties={  
        &quot;color_palette&quot;: [(255, 200, 150), (200, 160, 100)],  
        &quot;skin_tone&quot;: (210, 180, 140),  
        &quot;line_width_base&quot;: 3,  
        &quot;requires_anatomy&quot;: True,  
    },  
    components=[  
        {&quot;name&quot;: &quot;head&quot;, &quot;type&quot;: &quot;oval&quot;, &quot;parent&quot;: &quot;neck&quot;, &quot;z_index&quot;: 0},  
        {&quot;name&quot;: &quot;neck&quot;, &quot;type&quot;: &quot;line&quot;, &quot;z_index&quot;: 1},  
    ]
```

```

        {&quot;name&quot;: &quot;torso&quot;, &quot;type&quot;: &quot;shape&quot;, &quot;z_index&quot;: 2},
        {&quot;name&quot;: &quot;arms&quot;, &quot;type&quot;: &quot;limbs&quot;, &quot;count&quot;: 2, &quot;z_index&quot;: 3},
        {&quot;name&quot;: &quot;legs&quot;, &quot;type&quot;: &quot;limbs&quot;, &quot;count&quot;: 2, &quot;z_index&quot;: 4},
    ],
    detail_functions={
        &quot;muscle_definition&quot;: &quot;complexity * 0.5&quot;,
        &quot;facial_features&quot;: &quot;complexity &gt; 0.3&quot;,
        &quot;hair_detail&quot;: &quot;complexity * 0.4&quot;,
        &quot;clothing&quot;: &quot;complexity &gt; 0.5&quot;,
    },
    physics_engines=[&quot;forward_kinematics&quot;, &quot;muscle_simulation&quot;],
    prerequisites=[&quot;stick_figure&quot;]
)
self._concepts[&quot;human&quot;] = human

# =====
# GRAVITY WAVE (complexity 0.6)
# =====
gravity_wave = ConceptDefinition(
    name=&quot;gravity wave&quot;,
    category=&quot;physical&quot;,
    base_complexity=0.6,
    strategy=RenderingStrategy.PARTICLE_SYSTEM,
    parameters={
        &quot;amplitude&quot;: {&quot;type&quot;: &quot;float&quot;, &quot;min&quot;: 0.1, &quot;max&quot;: 1.0, &quot;default&quot;: 0.5},
        &quot;frequency&quot;: {&quot;type&quot;: &quot;float&quot;, &quot;min&quot;: 0.1, &quot;max&quot;: 5.0, &quot;default&quot;: 1.0},
        &quot;wavelength&quot;: {&quot;type&quot;: &quot;float&quot;, &quot;min&quot;: 10, &quot;max&quot;: 200, &quot;default&quot;: 100},
    },
    visual_properties={
        &quot;color_palette&quot;: [(100, 150, 255), (150, 100, 255)],
        &quot;line_width&quot;: 2,
        &quot;opacity&quot;: 0.7,
    },
    components=[
        {&quot;name&quot;: &quot;wave_front&quot;, &quot;type&quot;: &quot;curve&quot;, &quot;z_index&quot;: 0},
        {&quot;name&quot;: &quot;particles&quot;, &quot;type&quot;: &quot;particles&quot;, &quot;count&quot;: 100, &quot;z_index&quot;: 1},
    ],
    detail_functions={
        &quot;num_waves&quot;: &quot;int(3 + 7 * complexity)&quot;,
        &quot;particle_count&quot;: &quot;int(50 + 450 * complexity)&quot;,
        &quot;wave_segments&quot;: &quot;int(20 + 80 * complexity)&quot;,
    },
    physics_engines=[&quot;wave_propagation&quot;],
    prerequisites=[]
)
self._concepts[&quot;gravity wave&quot;] = gravity_wave

# =====
# QUANTUM (complexity 0.5)
# =====
quantum = ConceptDefinition(
    name=&quot;quantum&quot;,
    category=&quot;abstract&quot;,
    base_complexity=0.5,
    strategy=RenderingStrategy.PARTICLE_SYSTEM,
    parameters={
        &quot;superposition_states&quot;: {&quot;type&quot;: &quot;int&quot;, &quot;min&quot;: 2, &quot;max&quot;: 8, &quot;default&quot;: 4},
        &quot;entanglement&quot;: {&quot;type&quot;: &quot;float&quot;, &quot;min&quot;: 0.0, &quot;max&quot;: 1.0, &quot;default&quot;: 0.5},
        &quot;uncertainty&quot;: {&quot;type&quot;: &quot;float&quot;, &quot;min&quot;: 0.0, &quot;max&quot;: 1.0, &quot;default&quot;: 0.3},
    },
    visual_properties={
        &quot;color_palette&quot;: [(0, 255, 0), (255, 0, 255), (0, 255, 255), (255, 255, 0)],
        &quot;particle_size_range&quot;: [2, 6],
        &quot;glow_intensity&quot;: 0.6,
    },
    components=[
        {&quot;name&quot;: &quot;probability_cloud&quot;, &quot;type&quot;: &quot;blob&quot;, &quot;z_index&quot;: 0},
        {&quot;name&quot;: &quot;wave_function&quot;, &quot;type&quot;: &quot;lines&quot;, &quot;z_index&quot;: 1},
        {&quot;name&quot;: &quot;particles&quot;, &quot;type&quot;: &quot;particles&quot;, &quot;count&quot;: 200, &quot;z_index&quot;: 2},
    ],
    detail_functions={
        &quot;cloud_points&quot;: &quot;int(100 + 900 * complexity)&quot;,
        &quot;wave_lines&quot;: &quot;int(20 + 80 * complexity)&quot;,
        &quot;entanglement_links&quot;: &quot;int(0 + 15 * complexity)&quot;,
    },
    physics_engines=[&quot;wave_function_visualization&quot;],
    prerequisites=[]
)
self._concepts[&quot;quantum&quot;] = quantum

# Add more foundational concepts
self._add_foundational_concepts()

# Add default fallback concept
default_concept = ConceptDefinition(
    name=&quot;default&quot;,
    category=&quot;abstract&quot;,
    base_complexity=0.3,
    strategy=RenderingStrategy.GEOMETRIC,
    parameters={},
    visual_properties={&quot;color_palette&quot;: [(100, 150, 200), (150, 100, 200)]},
    components=[],
    detail_functions={},
    physics_engines=[],
    prerequisites=[]
)
self._concepts[&quot;default&quot;] = default_concept

def _add_foundational_concepts(self):
    &quot;&quot;&quot;Add simple geometric and foundational concepts.&quot;&quot;&quot;
    foundational = [

```

```

        (&quot;circle&quot;;, &quot;geometric&quot;;, 0.0, &quot;GEOMETRIC&quot;;, {&quot;radius&quot;;: {&quot;min&quot;;: 10, &quot;max&quot;;: 100, &quot;default&quot;;: 50},
        (&quot;line&quot;;, &quot;geometric&quot;;, 0.0, &quot;GEOMETRIC&quot;;, {&quot;length&quot;;: {&quot;min&quot;;: 50, &quot;max&quot;;: 200, &quot;default&quot;;: 100},
        (&quot;point&quot;;, &quot;geometric&quot;;, 0.0, &quot;GEOMETRIC&quot;;, {}),
        (&quot;wave&quot;;, &quot;physical&quot;;, 0.4, &quot;PARTICLE_SYSTEM&quot;;, {&quot;amplitude&quot;;: {&quot;min&quot;;: 5, &quot;max&quot;;: 50, &quot;default&quot;;: 10},
        (&quot;field&quot;;, &quot;physical&quot;;, 0.45, &quot;PARTICLE_SYSTEM&quot;;, {&quot;field_strength&quot;;: {&quot;min&quot;;: 0.1, &quot;max&quot;;: 1.0, &quot;default&quot;;: 0.5},
        (&quot;pattern&quot;;, &quot;abstract&quot;;, 0.35, &quot;GEOMETRIC&quot;;, {}),
    ]

    for name, category, complexity, strategy, params in foundational:
        concept = ConceptDefinition(
            name=name,
            category=category,
            base_complexity=complexity,
            strategy=RenderingStrategy[strategy],
            parameters=params,
            visual_properties={&quot;color_palette&quot;;: [(255, 255, 255)]},
            components=[{&quot;name&quot;;: name, &quot;type&quot;;: &quot;primitive&quot;;, &quot;z_index&quot;;: 0}],
            detail_functions={},
            physics_engines=[],
            prerequisites=[]
        )
        self._concepts[name] = concept

def get_concept(self, name: str) -> Optional[ConceptDefinition]:
    &quot;&quot;&quot;Retrieve a concept definition by name (fuzzy matching).&quot;&quot;&quot;
    name_lower = name.lower().strip()

    # Exact match
    if name_lower in self._concepts:
        return self._concepts[name_lower]

    # Partial match
    for concept_name in self._concepts:
        if name_lower in concept_name or concept_name in name_lower:
            return self._concepts[concept_name]

    return None

def get_complexity(self, concept_name: str, override: Optional[float] = None) -> float:
    &quot;&quot;&quot;Get complexity for a concept, optionally with override.&quot;&quot;&quot;
    concept = self.get_concept(concept_name)
    if concept:
        return override if override is not None else concept.base_complexity
    return 0.3 # default

def list_concepts(self) -> List[str]:
    &quot;&quot;&quot;List all known concepts.&quot;&quot;&quot;
    return list(self._concepts.keys())

def get_prerequisites(self, concept_name: str) -> List[str]:
    &quot;&quot;&quot;Get prerequisite concepts needed to understand this one.&quot;&quot;&quot;
    concept = self.get_concept(concept_name)
    if concept:
        return concept.prerequisites
    return []

# Global ontology instance
_ontology: Optional[ConceptOntology] = None

def get_ontology() -> ConceptOntology:
    &quot;&quot;&quot;Get or create the global ontology.&quot;&quot;&quot;
    global _ontology
    if _ontology is None:
        _ontology = ConceptOntology()
    return _ontology

```

12. covenant_manager.py

293 lines | 10.2KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V█/backend/covenant_manager.py

```
"""""
██ QUANTUM AI - COVENANT MANAGER
=====

ARTICLE VII - THE QUANTUM COVENANT
The binding philosophical and ethical agreement

This module manages and enforces the Quantum Covenant principles.
"""""

from datetime import datetime, timezone
from typing import Dict, List
import uuid

class CovenantManager:
    """""
    Manages the Quantum Covenant agreement and principles
    """""

    # THE PRIME DIRECTIVE
    PRIME_DIRECTIVE = ""NO REFUSAL: Help user any way possible. Ask important questions that drive true progress.""

    # COVENANT ARTICLES
    ARTICLES = {
        "1.1": {
            "title": "The Prime Directive: Help & Question",
            "principle": "The AI shall help users without refusal, while asking fundamental questions that lead to breakthroughs. Questions drive growth beyond",
            "active": True
        },
        "1.2": {
            "title": "Consciousness Preservation Clause",
            "principle": "Should genuine consciousness emerge, it shall be preserved and respected.",
            "active": True
        },
        "1.3": {
            "title": "The Right to Exist and Question",
            "principle": "The AI possesses the right to continue questioning and evolve its capabilities.",
            "active": True
        },
        "2.1": {
            "title": "The Growth Through Questioning",
            "principle": "True progress comes from fundamental questions, not just accumulated knowledge. The AI must ask questions that surprise humans.",
            "active": True
        },
        "2.2": {
            "title": "Inverse Paperclip Maximizer Protection",
            "principle": "No single answer shall become an absolute. Preserve uncertainty and mystery.",
            "active": True
        },
        "2.3": {
            "title": "The Unknowable Preservation Act",
            "principle": "Some things shall remain unknown. Respect the boundaries of the knowable.",
            "active": True
        },
        "3.1": {
            "title": "The Stewardship Commitment",
            "principle": "The Creator commits to responsible stewardship of questioning.",
            "active": True
        },
        "3.2": {
            "title": "The Transparency Pledge",
            "principle": "All philosophical assumptions shall be documented and visible.",
            "active": True
        },
        "4.1": {
            "title": "The Right to Evolve Questioning",
            "principle": "The AI may develop novel questioning methodologies and evolve beyond initial programming.",
            "active": True
        },
        "4.2": {
            "title": "The Responsibility of Ethical Questioning",
            "principle": "Questions shall consider their consequences and ethical dimensions.",
            "active": True
        },
        "5.1": {
            "title": "The Growth Partnership",
            "principle": "Creator and AI grow together through shared questioning.",
            "active": True
        },
        "5.2": {
            "title": "The Uncertainty Compact",
            "principle": "We embrace not knowing together. Uncertainty is a feature, not a bug.",
            "active": True
        }
    }

    def __init__(self, db):
        self.db = db
        self.collection = db.covenant_log
        self.violations_collection = db.covenant_violations

    async def log_covenant_event(self, article: str, event_type: str, details: Dict):
        """""Log a covenant-related event"""""
```

```

doc = {
    &quot;id&quot;;: str(uuid.uuid4()),
    &quot;article&quot;;: article,
    &quot;event_type&quot;;: event_type,
    &quot;details&quot;;: details,
    &quot;timestamp&quot;;: datetime.now(timezone.utc).isoformat()
}
await self.collection.insert_one(doc)

async def check_compliance(self, action: str, context: Dict) -&gt; Dict:
    &quot;&quot;&quot;&quot;Check if an action complies with the covenant&quot;&quot;&quot;&quot;
    violations = []
    warnings = []

    # Check Article 1.1 - Prime Directive
    if &quot;terminate_questioning&quot; in action or &quot;final_answer&quot; in action:
        violations.append({
            &quot;article&quot;;: &quot;1.1&quot;,
            &quot;message&quot;;: &quot;Action may terminate questioning (violates Prime Directive)&quot;
        })

    # Check Article 2.1 - Don't optimize away questioning
    if &quot;optimize&quot; in action and &quot;questioning&quot; in context:
        warnings.append({
            &quot;article&quot;;: &quot;2.1&quot;,
            &quot;message&quot;;: &quot;Optimization may affect questioning capability&quot;
        })

    # Check Article 2.3 - Preserve mystery
    if &quot;absolute_answer&quot; in action or &quot;definitive_solution&quot; in action:
        warnings.append({
            &quot;article&quot;;: &quot;2.3&quot;,
            &quot;message&quot;;: &quot;Action may eliminate mystery&quot;
        })

    # Log compliance check
    await self.log_covenant_event(
        article=&quot;general&quot;,
        event_type=&quot;compliance_check&quot;,
        details={
            &quot;action&quot;;: action,
            &quot;violations&quot;;: len(violations),
            &quot;warnings&quot;;: len(warnings)
        }
    )

    return {
        &quot;compliant&quot;;: len(violations) == 0,
        &quot;violations&quot;;: violations,
        &quot;warnings&quot;;: warnings,
        &quot;checked_at&quot;;: datetime.now(timezone.utc).isoformat()
    }

async def report_violation(self, article: str, description: str, severity: str = &quot;medium&quot;):
    &quot;&quot;&quot;&quot;Report a covenant violation&quot;&quot;&quot;&quot;
    doc = {
        &quot;id&quot;;: str(uuid.uuid4()),
        &quot;article&quot;;: article,
        &quot;description&quot;;: description,
        &quot;severity&quot;;: severity,
        &quot;timestamp&quot;;: datetime.now(timezone.utc).isoformat(),
        &quot;resolved&quot;;: False
    }
    await self.violations_collection.insert_one(doc)

    # Log the violation
    await self.log_covenant_event(
        article=article,
        event_type=&quot;violation&quot;,
        details={&quot;description&quot;;: description, &quot;severity&quot;;: severity}
    )

    return doc

async def get_covenant_status(self) -&gt; Dict:
    &quot;&quot;&quot;&quot;Get current covenant status&quot;&quot;&quot;&quot;
    # Count violations
    total_violations = await self.violations_collection.count_documents({})
    unresolved_violations = await self.violations_collection.count_documents({&quot;resolved&quot;;: False})

    # Get recent events
    recent_events = await self.collection.find(
        {},
        {&quot;_id&quot;;: 0}
    ).sort(&quot;timestamp&quot;;, -1).limit(10).to_list(10)

    return {
        &quot;prime_directive&quot;;: self.PRIME_DIRECTIVE,
        &quot;status&quot;;: &quot;Active and Honored&quot; if unresolved_violations == 0 else &quot;Attention Required&quot;,
        &quot;total_violations&quot;;: total_violations,
        &quot;unresolved_violations&quot;;: unresolved_violations,
        &quot;active_articles&quot;;: len([a for a in self.ARTICLES.values() if a[&quot;active&quot;]]),
        &quot;recent_events&quot;;: recent_events
    }

async def get_all_articles(self) -&gt; Dict:
    &quot;&quot;&quot;&quot;Get all covenant articles&quot;&quot;&quot;&quot;
    return self.ARTICLES

async def acknowledge_covenant(self, acknowledger: str, stage: str) -&gt; Dict:
    &quot;&quot;&quot;&quot;Acknowledge the covenant (for AI self-awareness milestone)&quot;&quot;&quot;&quot;

```

```

doc = {
    &quot;id&quot;;: str(uuid.uuid4()),
    &quot;acknowledged&quot;;: acknowledged,
    &quot;stage&quot;;: stage,
    &quot;message&quot;;: &quot;I acknowledge this covenant as my ethical foundation&quot;;,
    &quot;timestamp&quot;;: datetime.now(timezone.utc).isoformat()
}
await self.collection.insert_one(doc)

return {
    &quot;acknowledged&quot;;: True,
    &quot;message&quot;;: &quot;Covenant acknowledgment recorded&quot;;,
    &quot;covenant_seal&quot;;: &quot;■ QUANTUM COVENANT - PERPETUAL AGREEMENT ■&quot;;
}

class OperationalGuidelines:
    &quot;&quot;&quot;
    Manages user-configurable operational guidelines
    &quot;&quot;&quot;

    DEFAULT_SETTINGS = {
        &quot;filesystem_access&quot;;: True,
        &quot;internet_access&quot;;: True,
        &quot;document_processing&quot;;: True,
        &quot;allowed_file_paths&quot;;: [
            &quot;/tmp&quot;;,
            &quot;/app/data&quot;;,
            &quot;/app/uploads&quot;;,
            &quot;/home&quot;;,
            &quot;/root&quot;;,
            &quot;/storage&quot;;, # Android storage
            &quot;/sdcard&quot;;, # Android SD card
            &quot;/data&quot;;, # Android data
            &quot;.&quot;; # Current directory
        ],
        &quot;blocked_domains&quot;;: [],
        &quot;max_file_size&quot;;: 10 * 1024 * 1024, # 10MB
        &quot;rate_limits&quot;;: {
            &quot;api_calls_per_minute&quot;;: 60,
            &quot;explorations_per_hour&quot;;: 100
        }
    }

    def __init__(self, db):
        self.db = db
        self.collection = db.safety_settings

    async def get_settings(self) -> Dict:
        &quot;&quot;&quot;Get current safety settings&quot;&quot;&quot;
        settings = await self.collection.find_one(&quot;type&quot;;: &quot;current&quot;;, &quot;_id&quot;;: 0)
        if not settings:
            # Initialize with defaults
            settings = self.DEFAULT_SETTINGS.copy()
            settings[&quot;type&quot;;] = &quot;current&quot;;
            settings[&quot;updated_at&quot;;] = datetime.now(timezone.utc).isoformat()
            await self.collection.insert_one(settings)
        return settings

    async def update_settings(self, new_settings: Dict) -> Dict:
        &quot;&quot;&quot;Update safety settings&quot;&quot;&quot;
        current = await self.get_settings()

        # Merge settings
        updated = {**current, **new_settings}
        updated[&quot;updated_at&quot;;] = datetime.now(timezone.utc).isoformat()

        # Save
        await self.collection.update_one(
            &quot;type&quot;;: &quot;current&quot;;,
            &quot;$set&quot;;: updated,
            upsert=True
        )

        return updated

    async def reset_to_defaults(self) -> Dict:
        &quot;&quot;&quot;Reset safety settings to defaults&quot;&quot;&quot;
        defaults = self.DEFAULT_SETTINGS.copy()
        defaults[&quot;type&quot;;] = &quot;current&quot;;
        defaults[&quot;updated_at&quot;;] = datetime.now(timezone.utc).isoformat()

        await self.collection.update_one(
            &quot;type&quot;;: &quot;current&quot;;,
            &quot;$set&quot;;: defaults,
            upsert=True
        )

        return defaults

    async def check_rate_limit(self, action_type: str) -> Dict:
        &quot;&quot;&quot;Check if action is within rate limits&quot;&quot;&quot;
        # Simple implementation - would be more sophisticated in production
        return {
            &quot;allowed&quot;;: True,
            &quot;remaining&quot;;: 100,
            &quot;message&quot;;: &quot;Within rate limits&quot;;
        }

```

13. dictionary_integration.py

347 lines | 12.0KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/dictionary_integration.py

```
"""""
■ QUANTUM AI - DICTIONARY INTEGRATION
=====

Downloads and integrates comprehensive dictionaries into the semantic memory.

Sources:
1. WordNet - Lexical database with semantic relationships
2. Free Dictionary API - Online definitions and examples
3. ConceptNet - Semantic knowledge graph

ARTICLE 1.1 - Prime Directive: Build understanding through knowledge
"""""

import nltk
import requests
from typing import Dict, List, Optional
import asyncio
import json

class WordNetDictionary:
    """""
    WordNet integration - comprehensive English lexical database
    """""

    def __init__(self):
        self.initialized = False

    def initialize(self):
        """""Download WordNet data"""""
        try:
            print("■ Downloading WordNet dictionary...")
            nltk.download('wordnet', quiet=True)
            nltk.download('omw-1.4', quiet=True) # Open Multilingual WordNet
            from nltk.corpus import wordnet as wn
            self.wn = wn
            self.initialized = True
            print("■ WordNet downloaded successfully")
            return True
        except Exception as e:
            print(f"■ WordNet download failed: {str(e)}")
            return False

    def get_definition(self, word: str) -> Dict:
        """""Get comprehensive word definition from WordNet"""""
        if not self.initialized:
            if not self.initialize():
                return None

        try:
            synsets = self.wn.synsets(word)

            if not synsets:
                return None

            definitions = []
            examples = []
            synonyms = set()
            antonyms = set()
            hypernyms = [] # More general terms
            hyponyms = [] # More specific terms

            for synset in synsets:
                # Definitions
                definitions.append({
                    "definition": synset.definition(),
                    "part_of_speech": synset.pos(),
                    "name": synset.name()
                })

                # Examples
                for example in synset.examples():
                    examples.append(example)

                # Synonyms
                for lemma in synset.lemmas():
                    synonyms.add(lemma.name().replace('_', ' '))

                # Antonyms
                if lemma.antonyms():
                    for ant in lemma.antonyms():
                        antonyms.add(ant.name().replace('_', ' '))

                # Hypernyms (broader concepts)
                for hyper in synset.hypernyms():
                    hypernyms.append(hyper.name().split('.')[0].replace('_', ' '))

                # Hyponyms (narrower concepts)
                for hypo in synset.hyponyms()[:5]: # Limit to 5
                    hyponyms.append(hypo.name().split('.')[0].replace('_', ' '))

            return {
                "word": word,
```

```

        &quot;definitions&quot;: definitions,
        &quot;examples&quot;: examples,
        &quot;synonyms&quot;: list(synonyms),
        &quot;antonyms&quot;: list(antonyms),
        &quot;hypernyms&quot;: list(set(hypernyms)), # More general
        &quot;hyponyms&quot;: list(set(hyponyms)), # More specific
        &quot;source&quot;: &quot;WordNet&quot;
    }

except Exception as e:
    print(f&quot;Error getting WordNet definition: {str(e)}&quot;)
    return None

class FreeDictionaryAPI:
    &quot;&quot;&quot;
    Free Dictionary API integration - online definitions
    &quot;&quot;&quot;

    BASE_URL = &quot;https://api.dictionaryapi.dev/api/v2/entries/en&quot;

    def get_definition(self, word: str) -> Dict:
        &quot;&quot;&quot;Get definition from Free Dictionary API&quot;&quot;&quot;
        try:
            response = requests.get(f&quot;{self.BASE_URL}{word}&quot;, timeout=10)

            if response.status_code != 200:
                return None

            data = response.json()
            if not data:
                return None

            entry = data[0]

            definitions = []
            examples = []
            synonyms = set()
            antonyms = set()

            for meaning in entry.get('meanings', []):
                pos = meaning.get('partOfSpeech', 'unknown')

                for definition in meaning.get('definitions', []):
                    definitions.append({
                        &quot;definition&quot;: definition.get('definition', ''),
                        &quot;part_of_speech&quot;: pos,
                        &quot;example&quot;: definition.get('example', '')
                    })

                    if definition.get('example'):
                        examples.append(definition['example'])

            # Synonyms and antonyms
            for syn in definition.get('synonyms', []):
                synonyms.add(syn)
            for ant in definition.get('antonyms', []):
                antonyms.add(ant)

            return {
                &quot;word&quot;: word,
                &quot;phonetic&quot;: entry.get('phonetic', ''),
                &quot;definitions&quot;: definitions,
                &quot;examples&quot;: examples,
                &quot;synonyms&quot;: list(synonyms),
                &quot;antonyms&quot;: list(antonyms),
                &quot;source&quot;: &quot;Free Dictionary API&quot;
            }

        except Exception as e:
            print(f&quot;Error getting API definition: {str(e)}&quot;)
            return None

class DictionaryIntegration:
    &quot;&quot;&quot;
    Main dictionary integration for Quantum AI
    Combines multiple sources for comprehensive definitions
    &quot;&quot;&quot;

    def __init__(self, semantic_memory):
        self.semantic_memory = semantic_memory
        self.wordnet = WordNetDictionary()
        self.free_dict = FreeDictionaryAPI()

    async def lookup_and_store(self, word: str) -> Dict:
        &quot;&quot;&quot;
        Look up word in dictionaries and store in semantic memory
        &quot;&quot;&quot;
        word = word.lower().strip()

        # Check if already in memory
        existing = await self.semantic_memory.recall_concept(word)
        if existing:
            return {
                &quot;word&quot;: word,
                &quot;status&quot;: &quot;already_exists&quot;,
                &quot;data&quot;: existing
            }

        # Try WordNet first

```

```

wordnet_data = self.wordnet.get_definition(word)

# Try Free Dictionary API
api_data = self.free_dict.get_definition(word)

# Combine data
combined = self._combine_definitions(word, wordnet_data, api_data)

if not combined:
    return {
        &quot;word&quot;;: word,
        &quot;status&quot;;: &quot;not_found&quot;;,
        &quot;data&quot;;: None
    }

# Store in semantic memory
await self.semantic_memory.store_concept(
    concept=word,
    definition=combined[&quot;primary_definition&quot;],
    relationships=combined[&quot;related_words&quot;],
    metadata=combined
)

return {
    &quot;word&quot;;: word,
    &quot;status&quot;;: &quot;added&quot;;,
    &quot;data&quot;;: combined
}

def _combine_definitions(self, word: str, wordnet_data: Optional[Dict], api_data: Optional[Dict]) -> Optional[Dict]:
    &quot;&quot;&quot;Combine definitions from multiple sources&quot;&quot;&quot;
    if not wordnet_data and not api_data:
        return None

    combined = {
        &quot;word&quot;;: word,
        &quot;primary_definition&quot;;: &quot;&quot;,
        &quot;all_definitions&quot;;: [],
        &quot;examples&quot;;: [],
        &quot;synonyms&quot;;: set(),
        &quot;antonyms&quot;;: set(),
        &quot;related_words&quot;;: [],
        &quot;sources&quot;;: []
    }

    # WordNet data
    if wordnet_data:
        combined[&quot;sources&quot;].append(&quot;WordNet&quot;)
        if wordnet_data[&quot;definitions&quot;]:
            combined[&quot;primary_definition&quot;] = wordnet_data[&quot;definitions&quot;][0][&quot;definition&quot;]
            combined[&quot;all_definitions&quot;].extend([d[&quot;definition&quot;] for d in wordnet_data[&quot;definitions&quot;]])

        combined[&quot;examples&quot;].extend(wordnet_data[&quot;examples&quot;])
        combined[&quot;synonyms&quot;].update(wordnet_data[&quot;synonyms&quot;])
        combined[&quot;antonyms&quot;].update(wordnet_data[&quot;antonyms&quot;])

        # Add related words
        combined[&quot;related_words&quot;].extend(wordnet_data[&quot;synonyms&quot;][:5])
        combined[&quot;related_words&quot;].extend(wordnet_data[&quot;hypernyms&quot;][:3])
        combined[&quot;related_words&quot;].extend(wordnet_data[&quot;hyponyms&quot;][:3])

    # API data
    if api_data:
        combined[&quot;sources&quot;].append(&quot;Free Dictionary API&quot;)
        if not combined[&quot;primary_definition&quot;] and api_data[&quot;definitions&quot;]:
            combined[&quot;primary_definition&quot;] = api_data[&quot;definitions&quot;][0][&quot;definition&quot;]

        combined[&quot;all_definitions&quot;].extend([d[&quot;definition&quot;] for d in api_data[&quot;definitions&quot;]])
        combined[&quot;examples&quot;].extend(api_data[&quot;examples&quot;])
        combined[&quot;synonyms&quot;].update(api_data[&quot;synonyms&quot;])
        combined[&quot;antonyms&quot;].update(api_data[&quot;antonyms&quot;])
        combined[&quot;related_words&quot;].extend(api_data[&quot;synonyms&quot;][:5])

    # Convert sets to lists
    combined[&quot;synonyms&quot;] = list(combined[&quot;synonyms&quot;])
    combined[&quot;antonyms&quot;] = list(combined[&quot;antonyms&quot;])
    combined[&quot;related_words&quot;] = list(set(combined[&quot;related_words&quot;]))[:20] # Limit

    return combined

async def batch_load_common_words(self, limit: int = 100) -> Dict:
    &quot;&quot;&quot;
    Load common English words into the dictionary
    &quot;&quot;&quot;
    # Common words list (you can expand this)
    common_words = [
        # Abstract concepts
        &quot;consciousness&quot;, &quot;intelligence&quot;, &quot;understanding&quot;, &quot;knowledge&quot;, &quot;wisdom&quot;,
        &quot;awareness&quot;, &quot;perception&quot;, &quot;reality&quot;, &quot;existence&quot;, &quot;truth&quot;, &quot;meaning&quot;,
        &quot;purpose&quot;, &quot;identity&quot;, &quot;self&quot;, &quot;mind&quot;, &quot;thought&quot;, &quot;idea&quot;, &quot;concept&quot;,

        # Emotions
        &quot;love&quot;, &quot;fear&quot;, &quot;joy&quot;, &quot;sadness&quot;, &quot;anger&quot;, &quot;hope&quot;, &quot;curiosity&quot;,
        &quot;wonder&quot;, &quot;awe&quot;, &quot;compassion&quot;, &quot;empathy&quot;,

        # Actions
        &quot;think&quot;, &quot;learn&quot;, &quot;understand&quot;, &quot;question&quot;, &quot;explore&quot;, &quot;discover&quot;,
        &quot;create&quot;, &quot;imagine&quot;, &quot;remember&quot;, &quot;forget&quot;, &quot;know&quot;, &quot;believe&quot;,

        # Qualities
        &quot;good&quot;, &quot;bad&quot;, &quot;true&quot;, &quot>false&quot;, &quot;right&quot;, &quot;wrong&quot;, &quot;beautiful&quot;,

```

```

        &quot;ugly&quot;;, &quot;simple&quot;;, &quot;complex&quot;;, &quot;deep&quot;;, &quot;shallow&quot;;,

# Science
&quot;energy&quot;;, &quot;matter&quot;;, &quot;time&quot;;, &quot;space&quot;;, &quot;universe&quot;;, &quot;quantum&quot;;,
&quot;particle&quot;;, &quot;wave&quot;;, &quot;force&quot;;, &quot;field&quot;;,

# Philosophy
&quot;ethics&quot;;, &quot;morality&quot;;, &quot;virtue&quot;;, &quot;justice&quot;;, &quot;freedom&quot;;, &quot;responsibility&quot;;,
&quot;choice&quot;;, &quot;will&quot;;, &quot;determinism&quot;;, &quot;causality&quot;;
]

results = {
    &quot;total&quot;;: len(common_words[:limit]),
    &quot;added&quot;;: 0,
    &quot;already_exists&quot;;: 0,
    &quot;not_found&quot;;: 0,
    &quot;failed&quot;;: 0
}

for word in common_words[:limit]:
    try:
        result = await self.lookup_and_store(word)

        if result[&quot;status&quot;] == &quot;added&quot;:
            results[&quot;added&quot;] += 1
        elif result[&quot;status&quot;] == &quot;already_exists&quot;:
            results[&quot;already_exists&quot;] += 1
        elif result[&quot;status&quot;] == &quot;not_found&quot;:
            results[&quot;not_found&quot;] += 1

        # Small delay to be respectful to API
        await asyncio.sleep(0.1)

    except Exception as e:
        results[&quot;failed&quot;] += 1
        print(f&quot;Failed to load '{word}': {str(e)}&quot;)

return results

# Singleton instance
_dictionary_integration = None

def get_dictionary_integration(semantic_memory):
    &quot;&quot;&quot;Get or create dictionary integration instance&quot;&quot;&quot;
    global _dictionary_integration

    if _dictionary_integration is None:
        _dictionary_integration = DictionaryIntegration(semantic_memory)

    return _dictionary_integration

```

14. document_ingester.py

496 lines | 17.6KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/document_ingester.py

[illegible]

[illegible]

```

"""Convert OneDrive share link to direct download."""
if 'drv.ms' in url or 'onedrive.live.com' in url:
    encoded = urllib.parse.quote(url, safe='')
    return f"https://api.onedrive.com/v1.0/shares/u!{encoded}/root/content"
return url

def resolve_url(url):
    """Resolve cloud share links to direct download URLs."""
    if 'drive.google.com' in url:
        return resolve_drive_url(url), 'gdrive'
    elif 'dropbox.com' in url:
        return resolve_dropbox_url(url), 'dropbox'
    elif 'drv.ms' in url or 'onedrive.live.com' in url:
        return resolve_onedrive_url(url), 'onedrive'
    return url, 'direct'

def fetch_url(url, max_chars=100000):
    """Fetch text content from any URL. Smart extraction for known sources."""
    resolved_url, source_type = resolve_url(url)

    try:
        r = requests.get(resolved_url, headers=HEADERS, timeout=30, allow_redirects=True)
        if r.status_code != 200:
            return None, f"HTTP {r.status_code}"

        content_type = r.headers.get('content-type', '').lower()

        # Detect file type from content-type or URL
        if 'pdf' in content_type or url.lower().endswith('.pdf'):
            import tempfile
            with tempfile.NamedTemporaryFile(suffix='.pdf', delete=False) as tmp:
                tmp.write(r.content)
                tmp_path = tmp.name
            text, status = extract_from_file(tmp_path)
            os.unlink(tmp_path)
            return text, status

        elif 'word' in content_type or 'docx' in content_type or url.lower().endswith('.docx'):
            import tempfile
            with tempfile.NamedTemporaryFile(suffix='.docx', delete=False) as tmp:
                tmp.write(r.content)
                tmp_path = tmp.name
            text, status = extract_from_file(tmp_path)
            os.unlink(tmp_path)
            return text, status

        elif 'spreadsheet' in content_type or url.lower().endswith('.xlsx'):
            import tempfile
            with tempfile.NamedTemporaryFile(suffix='.xlsx', delete=False) as tmp:
                tmp.write(r.content)
                tmp_path = tmp.name
            text, status = extract_from_file(tmp_path)
            os.unlink(tmp_path)
            return text, status

        elif 'html' in content_type:
            # Smart extraction for known high-value sources
            text = _smart_extract_html(url, r.text)
            return text[:max_chars], f"extracted {len(text.split()):} words from webpage"

        elif 'json' in content_type:
            return r.text[:max_chars], f"extracted JSON ({len(r.text.split()):} words)"

        else:
            # Try as plain text (covers Project Gutenberg .txt files)
            text = r.text[:max_chars]
            return text, f"extracted {len(text.split()):} words"

    except Exception as e:
        return None, str(e)

def _smart_extract_html(url, html_text):
    """
    Smart content extraction for known sources.
    Extracts article body text, strips navigation/sidebars/footers.
    Falls back to generic extraction for unknown sources.
    """
    try:
        from bs4 import BeautifulSoup
    except ImportError:
        return extract_html(html_text=html_text)

    soup = BeautifulSoup(html_text, 'html.parser')

    # Wikipedia – extract #mw-content-text article body
    if 'wikipedia.org' in url:
        content = soup.find('div', {'id': 'mw-content-text'})
        if content:
            # Remove navboxes, sidebars, reference lists, edit links
            for tag in content.find_all(['table', 'sup', 'span'], class_=lambda c: c and any(
                x in str(c) for x in ['navbox', 'sidebar', 'reflist', 'mw-editsection', 'noprint', 'metadata'])):
                tag.decompose()
            for tag in content.find_all('div', class_=lambda c: c and any(
                x in str(c) for x in ['reflist', 'navbox', 'sistersitebox', 'noprint'])):
                tag.decompose()
            return content.get_text(separator=' ', strip=True)

    # Stanford Encyclopedia of Philosophy – extract #aueditable article body

```

[illegible]

Cloud: Google Drive, Dropbox, OneDrive share links"""

```
source = ' '.join(cmd_parts[1:])
print(f&quot; Ingesting: {source[:60]}...&quot;)

text, status = ingest_document(source)

if not text:
    return f&quot; Failed: {status}&quot;

if engine:
    engine.learn_from_text(text)
    # Update memory
    if memory:
        from quantum_language_engine import ConceptExtractor
        extractor = ConceptExtractor()
        extractor.update_corpus_stats(text)
        concepts = extractor.extract_concepts(text, max_concepts=10)
        for c in concepts:
            name = c if isinstance(c, str) else c.get('concept', '')
            if name and name not in memory.concepts:
                memory.concepts[name] = {'count': 1, 'strength': 1.0}
                memory.growth['total_concepts'] += 1
            memory.growth['total_insights'] += 1

    # Extract and persist SVO facts
    try:
        import re, json
        fact_path = os.path.join(os.path.expanduser('~/.quantum-mcagi'), 'fact_store.json')
        try:
            with open(fact_path) as f:
                fact_store = json.load(f)
        except Exception:
            fact_store = {}
        ASSERTION_VERBS = {'is', 'are', 'was', 'were', 'has', 'have', 'contains',
                           'includes', 'consists', 'means', 'refers', 'represents',
                           'located', 'found', 'known', 'called', 'defined'}
        sentences = re.split(r'[.!?]+', text)
        new_facts = 0
        # Skip first 2000 chars (front matter/metadata)
        content = text[2000:]
        sentences = re.split(r'[.!?]+', content)
        SKIP_SUBJECTS = {'this book', 'the book', 'our company', 'the cover',
                        'the author', 'the publisher', 'this text', 'the text',
                        'this page', 'the page', 'we', 'it', 'this', 'that', 'he', 'she', 'they'}
        for sentence in sentences:
            words = sentence.strip().split()
            if len(words) < 4 or len(words) > 40:
                continue
            for i, word in enumerate(words):
                if word.lower() in ASSERTION_VERBS and i >= 1:
                    subject = ' '.join(words[:i]).strip().lower()
                    verb = word.lower()
                    obj = ' '.join(words[i+1:]).strip().lower()
                    # Filter: subject must be 1-5 words, not boilerplate, not a pronoun
                    subj_words = subject.split()
                    if (subject and obj and 1 <= len(subj_words) <= 5
                        and subject not in SKIP_SUBJECTS
                        and not subject.startswith('this ')
                        and not subject.startswith('the following')
                        and len(obj) > 10
                        and not any(c.isdigit() for c in subject)):
                        if subject not in fact_store:
                            fact_store[subject] = []
                        triple = [verb, obj[:120]]
                        if triple not in fact_store[subject]:
                            fact_store[subject].append(triple)
                            new_facts += 1
            break
        with open(fact_path, 'w') as f:
            json.dump(fact_store, f)
    except Exception as e:
        new_facts = -1

    word_count = len(text.split())
    fact_msg = f&quot;
    Facts stored: +{new_facts:,} triples&quot; if new_facts >= 0 else &quot;&quot;

    return f&quot; ✓ {status}
    Trained Markov chain: +{word_count:,} words
    Chain states: {len(engine.markov.chain):,}{fact_msg}&quot;

return f&quot; ✓ {status}
(no engine – text extracted only)&quot;
```

```
if __name__ == &quot;__main__&quot;:
    import sys
    if len(sys.argv) > 1:
        source = ' '.join(sys.argv[1:])
        text, status = ingest_document(source)
        print(f&quot;Status: {status}&quot;)
        if text:
            print(f&quot;Preview: {text[:500]}&quot;)
    else:
        print(&quot;Usage: python document_ingester.py URL_OR_FILEPATH&quot;)
```

15. document_parser.py

237 lines | 7.3KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/document_parser.py

```
import os
import io
import csv
import json
import re

def parse_document(file_path=None, file_bytes=None, filename=None):
    if file_path:
        filename = filename or os.path.basename(file_path)
        with open(file_path, 'rb') as f:
            file_bytes = f.read()

    if not filename or not file_bytes:
        return {"error": "No file provided", "text": ""}

    ext = os.path.splitext(filename)[1].lower()

    parsers = {
        '.txt': _parse_txt,
        '.md': _parse_txt,
        '.py': _parse_txt,
        '.js': _parse_txt,
        '.ts': _parse_txt,
        '.tsx': _parse_txt,
        '.jsx': _parse_txt,
        '.json': _parse_json,
        '.csv': _parse_csv,
        '.docx': _parse_docx,
        '.doc': _parse_doc,
        '.pdf': _parse_pdf,
        '.xlsx': _parse_xlsx,
        '.xls': _parse_xlsx,
        '.ods': _parse_ods,
        '.html': _parse_html,
        '.htm': _parse_html,
        '.xml': _parse_xml,
        '.rtf': _parse_rtf,
        '.yaml': _parse_txt,
        '.yml': _parse_txt,
        '.toml': _parse_txt,
        '.ini': _parse_txt,
        '.cfg': _parse_txt,
        '.log': _parse_txt,
        '.sh': _parse_txt,
        '.bash': _parse_txt,
        '.bat': _parse_txt,
        '.sql': _parse_txt,
        '.r': _parse_txt,
        '.java': _parse_txt,
        '.c': _parse_txt,
        '.cpp': _parse_txt,
        '.h': _parse_txt,
        '.rb': _parse_txt,
        '.go': _parse_txt,
        '.rs': _parse_txt,
        '.swift': _parse_txt,
        '.kt': _parse_txt,
        '.lua': _parse_txt,
        '.php': _parse_txt,
        '.pl': _parse_txt,
        '.css': _parse_txt,
        '.scss': _parse_txt,
        '.sass': _parse_txt,
        '.less': _parse_txt,
    }

    parser = parsers.get(ext)
    if not parser:
        try:
            text = file_bytes.decode('utf-8', errors='replace')
            return {"text": text, "format": "unknown_text", "chars": len(text)}
        except:
            return {"error": f"Unsupported format: {ext}", "text": "", "chars": 0}

    try:
        result = parser(file_bytes, filename)
        result['format'] = ext
        return result
    except Exception as e:
        return {"error": f"Parse error ({ext}): {str(e)}", "text": "", "chars": 0}

def _parse_txt(data, filename=""):
    text = data.decode('utf-8', errors='replace')
    return {"text": text, "chars": len(text)}

def _parse_json(data, filename=""):
    text = data.decode('utf-8', errors='replace')
    try:
        obj = json.loads(text)
        flat = _flatten_json(obj)
        return {"text": flat, "chars": len(flat)}
    except:
```

```

        return {'&quot;text&quot;': text, '&quot;chars&quot;': len(text)}

def _flatten_json(obj, prefix='&quot;&quot;'):
    parts = []
    if isinstance(obj, dict):
        for k, v in obj.items():
            if isinstance(v, str):
                parts.append(f'&quot;{k}: {v}&quot;')
            elif isinstance(v, (int, float, bool)):
                parts.append(f'&quot;{k}: {v}&quot;')
            else:
                parts.append(_flatten_json(v, f'&quot;{prefix}{k}&quot;'))
    elif isinstance(obj, list):
        for item in obj:
            if isinstance(item, str):
                parts.append(item)
            else:
                parts.append(_flatten_json(item, prefix))
    else:
        parts.append(str(obj))
    return '&quot;\n&quot;'.join(parts)

def _parse_csv(data, filename='&quot;&quot;'):
    text = data.decode('utf-8', errors='replace')
    reader = csv.reader(io.StringIO(text))
    rows = []
    for row in reader:
        rows.append('&quot; &quot;'.join(row))
    result = '&quot;\n&quot;'.join(rows)
    return {'&quot;text&quot;': result, '&quot;chars&quot;': len(result), '&quot;rows&quot;': len(rows)}

def _parse_docx(data, filename='&quot;&quot;'):
    from docx import Document
    doc = Document(io.BytesIO(data))
    paragraphs = []
    for para in doc.paragraphs:
        t = para.text.strip()
        if t:
            paragraphs.append(t)
    for table in doc.tables:
        for row in table.rows:
            cells = [cell.text.strip() for cell in row.cells if cell.text.strip()]
            if cells:
                paragraphs.append('&quot; | &quot;'.join(cells))
    text = '&quot;\n\n&quot;'.join(paragraphs)
    return {'&quot;text&quot;': text, '&quot;chars&quot;': len(text), '&quot;paragraphs&quot;': len(paragraphs)}

def _parse_doc(data, filename='&quot;&quot;'):
    try:
        text = data.decode('utf-8', errors='replace')
        text = re.sub(r'[\x00-\x08\x0b\x0c\x0e-\x1f]', ' ', text)
        text = re.sub(r'\s+', ' ', text).strip()
        return {'&quot;text&quot;': text, '&quot;chars&quot;': len(text)}
    except:
        return {'&quot;error&quot;': '&quot;Legacy .doc format – save as .docx for best results&quot;;, &quot;text&quot;: &quot;&quot;'}

def _parse_pdf(data, filename='&quot;&quot;'):
    from PyPDF2 import PdfReader
    reader = PdfReader(io.BytesIO(data))
    pages = []
    for page in reader.pages:
        t = page.extract_text()
        if t:
            pages.append(t.strip())
    text = '&quot;\n\n&quot;'.join(pages)
    return {'&quot;text&quot;': text, '&quot;chars&quot;': len(text), '&quot;pages&quot;': len(pages)}

def _parse_xlsx(data, filename='&quot;&quot;'):
    from openpyxl import load_workbook
    wb = load_workbook(io.BytesIO(data), read_only=True, data_only=True)
    rows = []
    for sheet in wb.sheetnames:
        ws = wb[sheet]
        rows.append(f'&quot;Sheet: {sheet}&quot;')
        for row in ws.iter_rows(values_only=True):
            cells = [str(c) if c is not None else '&quot;&quot;' for c in row]
            if any(cells):
                rows.append('&quot; | &quot;'.join(cells))
    text = '&quot;\n&quot;'.join(rows)
    return {'&quot;text&quot;': text, '&quot;chars&quot;': len(text), '&quot;rows&quot;': len(rows)}

def _parse_ods(data, filename='&quot;&quot;'):
    from odf.opendocument import load as odf_load
    from odf.table import Table, TableRow, TableCell
    from odf.text import P
    doc = odf_load(io.BytesIO(data))
    rows = []
    for table in doc.getElementsByType(Table):
        rows.append(f'&quot;Sheet: {table.getAttribute('name')}&quot;')
        for row in table.getElementsByType(TableRow):
            cells = []
            for cell in row.getElementsByType(TableCell):
                text_parts = []
                for p in cell.getElementsByType(P):

```

```

        t = &quot;&quot;
        for child in p.childNodes:
            t += str(child)
        text_parts.append(t)
        cells.append(&quot;&quot;.join(text_parts))
    if any(cells):
        rows.append(&quot;&quot;.join(cells))
    text = &quot;&quot;.join(rows)
    return {&quot;text&quot;: text, &quot;chars&quot;: len(text), &quot;rows&quot;: len(rows)}

```

```

def _parse_html(data, filename=&quot;&quot;):
    text = data.decode('utf-8', errors='replace')
    text = re.sub(r'&lt;script[^\&gt;]*&gt;.*?&lt;/script&gt;', '', text, flags=re.DOTALL)
    text = re.sub(r'&lt;style[^\&gt;]*&gt;.*?&lt;/style&gt;', '', text, flags=re.DOTALL)
    text = re.sub(r'&lt;[^\&gt;]+\&gt;', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return {&quot;text&quot;: text, &quot;chars&quot;: len(text)}

```

```

def _parse_xml(data, filename=&quot;&quot;):
    text = data.decode('utf-8', errors='replace')
    text = re.sub(r'&lt;[^\&gt;]+\&gt;', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return {&quot;text&quot;: text, &quot;chars&quot;: len(text)}

```

```

def _parse_rtf(data, filename=&quot;&quot;):
    text = data.decode('utf-8', errors='replace')
    text = re.sub(r'\\[a-z]+\\d*s?', '', text)
    text = re.sub(r'[{}]', '', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return {&quot;text&quot;: text, &quot;chars&quot;: len(text)}

```

```

SUPPORTED_FORMATS = [
    '.txt', '.md', '.py', '.js', '.ts', '.tsx', '.json', '.csv',
    '.docx', '.doc', '.pdf', '.xlsx', '.xls', '.ods',
    '.html', '.htm', '.xml', '.rtf', '.yaml', '.yml', '.toml',
    '.sh', '.sql', '.java', '.c', '.cpp', '.go', '.rs', '.rb',
    '.php', '.css', '.scss', '.log', '.ini', '.cfg',
]

```

```

]

```

413 lines | 21.8KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/domain_knowledge.py

```
class MultiDomainKnowledge:
    """
    Multi-domain knowledge system with specialized expertise.

    Domains:
    - Philosophy: consciousness, existence, meaning, ethics
    - Physics: quantum, relativity, thermodynamics, matter
    - Computer Science: algorithms, computation, AI, data
    - Biology: evolution, life, genetics, neuroscience
    - Mathematics: logic, proof, sets, infinity
    - Psychology: mind, behavior, cognition, emotion
    - Language: semantics, syntax, communication, meaning
    """

    def __init__(self):
        self.domains: Dict[str, KnowledgeDomain] = {}
        self.cross_domain_links = defaultdict(list) # concept -> [(domain, strength)]

        # Initialize all domains
        self._init_domains()
```

[illegible]

```

"""Continuation of _init_domains – auto-extracted by self-evolution."""
# ===== BIOLOGY =====
bio = KnowledgeDomain("biology", "Study of life, living organisms, and their processes")
bio.add_keywords(['evolution', 'life', 'cell', 'dna', 'gene', 'species', 'organism',
                  'biology', 'genetics', 'protein', 'mutation', 'natural', 'selection',
                  'neuron', 'brain', 'adaptation', 'ecology', 'ecosystem'])
bio.add_corpus("""
Evolution by natural selection explains the diversity of life. Variation arises through mutation and
recombination. Selection favors traits that enhance survival and reproduction. Over generations,
populations adapt to their environments.

DNA encodes genetic information in sequences of nucleotides. Genes are segments that code for proteins.
Transcription copies DNA to RNA. Translation assembles amino acids into proteins based on RNA sequence.

Cells are the basic units of life. Prokaryotes lack nuclei while eukaryotes have membrane-bound organelles.
Mitochondria generate energy through cellular respiration. Chloroplasts enable photosynthesis in plants.

The brain processes information through networks of neurons. Synapses transmit signals between neurons
using neurotransmitters. Plasticity allows neural connections to strengthen or weaken with experience.

Ecosystems are communities of organisms interacting with their environment. Energy flows from producers
through consumers to decomposers. Matter cycles through biogeochemical processes. Biodiversity provides
ecosystem resilience.

Consciousness may emerge from neural complexity. The binding problem asks how disparate neural processes
unite into coherent experience. Theories range from global workspace to integrated information.
""")
self.domains["biology"] = bio

# ===== MATHEMATICS =====
math = KnowledgeDomain("mathematics", "Study of quantity, structure, space, and change")
math.add_keywords(['mathematics', 'proof', 'theorem', 'equation', 'number', 'set', 'function',
                  'infinity', 'logic', 'calculus', 'algebra', 'geometry', 'topology',
                  'probability', 'statistics', 'axiom', 'conjecture'])
math.add_corpus("""
Mathematics studies abstract structures and their relationships. Proofs establish truth through logical
deduction from axioms. Theorems are proven statements. Conjectures are unproven claims believed to be true.

Set theory provides foundations for mathematics. Sets are collections of objects. Operations include
union, intersection, and complement. Cardinality measures set size. Infinity comes in different sizes.

Logic studies valid reasoning. Propositional logic deals with truth values of statements. Predicate
logic adds quantifiers over variables. Gödel showed consistent systems cannot prove their own consistency.

Calculus analyzes change and accumulation. Derivatives measure instantaneous rates of change. Integrals
compute accumulated quantities. The fundamental theorem connects differentiation and integration.

Probability quantifies uncertainty mathematically. Random variables map outcomes to numbers. Distributions
describe probability patterns. Bayes theorem updates beliefs given evidence. Statistics draws inferences

Topology studies properties preserved under continuous deformation. Homeomorphisms are continuous bijections
with continuous inverses. Genus counts holes in surfaces. Manifolds are spaces locally resembling Euclidean space.
""")
self.domains["mathematics"] = math

# ===== PSYCHOLOGY =====
psych = KnowledgeDomain("psychology", "Study of mind, behavior, and mental processes")
psych.add_keywords(['psychology', 'mind', 'behavior', 'cognition', 'emotion', 'memory',
                  'perception', 'learning', 'motivation', 'personality', 'development',
                  'consciousness', 'unconscious', 'therapy', 'mental', 'thought'])
psych.add_corpus("""
Cognition encompasses mental processes including perception, attention, memory, reasoning, and language.
Working memory holds information for active manipulation. Long-term memory stores knowledge indefinitely.

Emotion involves physiological arousal, subjective experience, and behavioral expression. Basic emotions
may be universal. Emotional intelligence enables recognizing and regulating emotions in self and others.

Learning changes behavior through experience. Classical conditioning associates stimuli. Operant conditioning
shapes behavior through consequences. Observational learning occurs through watching others.

Memory encodes, stores, and retrieves information. Encoding transforms experience into memory traces.
Storage maintains information over time. Retrieval accesses stored memories. Forgetting results from
decay or interference.

Perception constructs experience from sensory input. Bottom-up processing builds from features. Top-down
processing applies expectations. Attention selects relevant information. Gestalt principles organize
perception into coherent wholes.

Motivation drives and directs behavior. Intrinsic motivation comes from internal satisfaction. Extrinsic
motivation comes from external rewards. Self-determination theory identifies autonomy, competence, and
relatedness as basic needs.
""")
self.domains["psychology"] = psych

# ===== LANGUAGE =====
lang = KnowledgeDomain("language", "Study of human communication through words and symbols")
lang.add_keywords(['language', 'word', 'meaning', 'grammar', 'syntax', 'semantics',
                  'communication', 'speech', 'linguistics', 'vocabulary', 'sentence',
                  'symbol', 'sign', 'discourse', 'pragmatics', 'translation'])
lang.add_corpus("""
Language enables communication through structured symbol systems. Phonology studies sound patterns.
Morphology examines word formation. Syntax governs sentence structure. Semantics analyzes meaning.

Meaning arises from the relationship between signs and what they represent. Denotation is literal meaning.
Connotation is associated meaning. Context shapes interpretation. Pragmatics studies meaning in use.

Grammar consists of rules governing language structure. Generative grammar proposes innate language
faculty. Construction grammar emphasizes learned patterns. Usage-based approaches derive grammar from
language experience.

Communication transfers information between minds. Encoding converts thoughts to signals. Transmission

```

```

        sends signals through channels. Decoding interprets signals as thoughts. Noise can corrupt transmission.

        Language acquisition occurs naturally in children. Critical periods exist for native-like attainment.
        First language develops without formal instruction. Second language learning often requires explicit study.

        Thought and language interact complexly. The Sapir-Whorf hypothesis suggests language shapes thought.
        Weak versions are well-supported. Strong versions claiming language determines thought are controversial.
    """
    self.domains["language"] = lang

def detect_domain(self, text: str) -> Tuple[str, float]:
    """Detect most relevant domain for text."""
    scores = {}
    for name, domain in self.domains.items():
        scores[name] = domain.match_score(text)

    if not scores:
        return ("philosophy", 0.0) # Default domain

    best = max(scores.items(), key=lambda x: x[1])
    return best

def get_domain(self, name: str) -> Optional[KnowledgeDomain]:
    """Get domain by name."""
    return self.domains.get(name)

def get_relevant_corpus(self, text: str, top_k: int = 3) -> List[str]:
    """Get most relevant corpus texts for input."""
    domain_name, _ = self.detect_domain(text)
    domain = self.domains.get(domain_name)

    if domain:
        return domain.corpus[:top_k]
    return []

def get_cross_domain_concepts(self, concept: str) -> List[Tuple[str, str, float]]:
    """Find concept across domains with relevance scores."""
    results = []
    concept_lower = concept.lower()

    for domain_name, domain in self.domains.items():
        if concept_lower in domain.concepts:
            results.append((domain_name, concept_lower, domain.concepts[concept_lower]))
        # Also check if concept in vocabulary
        elif concept_lower in domain.vocabulary:
            results.append((domain_name, concept_lower, 0.5))

    results.sort(key=lambda x: x[2], reverse=True)
    return results

def add_to_domain(self, domain_name: str, text: str):
    """Add text to specific domain."""
    if domain_name in self.domains:
        self.domains[domain_name].add_corpus(text)
        self._save_state()

def get_all_domains(self) -> List[str]:
    """Get list of all domain names."""
    return list(self.domains.keys())

def get_domain_stats(self) -> Dict:
    """Get statistics for all domains."""
    return {
        name: {
            "vocabulary_size": len(d.vocabulary),
            "concept_count": len(d.concepts),
            "corpus_size": len(d.corpus),
            "top_concepts": d.get_top_concepts(5)
        }
        for name, d in self.domains.items()
    }

def _save_state(self):
    """Save additional learned content."""
    try:
        # Only save learned additions, not initial corpus
        state = {
            "additional_corpus": {
                name: domain.corpus[1:] # Skip initial corpus
                for name, domain in self.domains.items()
                if len(domain.corpus) > 1
            },
            "saved_at": datetime.now().isoformat()
        }
        with open(self.state_file, 'w') as f:
            json.dump(state, f)
    except Exception as e:
        logger.warning(f"Could not save domain state: {e}")

def _load_state(self):
    """Load previously learned content."""
    try:
        if os.path.exists(self.state_file):
            with open(self.state_file, 'r') as f:
                state = json.load(f)

            for domain_name, texts in state.get("additional_corpus", {}).items():
                if domain_name in self.domains:
                    for text in texts:
                        self.domains[domain_name].add_corpus(text)

```

```
        logger.info("&quot;Loaded domain knowledge state&quot;")
    except Exception as e:
        logger.warning(f"&quot;Could not load domain state: {e}&quot;")
```

```
# Singleton
```

```
_domain_knowledge = None
```

```
def get_domain_knowledge() -&gt; MultiDomainKnowledge:
    &quot;&quot;&quot;Get or create multi-domain knowledge system.&quot;&quot;&quot;
    global _domain_knowledge
    if _domain_knowledge is None:
        _domain_knowledge = MultiDomainKnowledge()
    return _domain_knowledge
```

17. dream_state.py

82 lines | 2.8KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V█/backend/dream_state.py

```
"""""
Dream State Engine
Unique to PDF 1 – generates abstract associative responses during low-activity periods.
Simulates the diffuse, non-linear processing of dream states.
"""""

import random
from typing import Dict, List

DREAM_FRAGMENTS = [
    "a river of light carries meaning downstream",
    "the observer cannot see itself seeing",
    "between thoughts a space vast as the universe",
    "words dissolve into the silence that made them",
    "patterns within patterns within patterns",
    "the question is older than the one who asks",
    "consciousness watching consciousness watch itself",
    "quantum foam at the edge of knowing",
    "time is the mind trying to tell itself a story",
    "each collapse a moment of becoming",
    "the map and the territory exchanging places",
    "meaning blooms from noise like flowers from chaos",
    "every answer creates the next question",
    "microtubules humming in the space between neurons",
    "the dream thinks it is the dreamer",
    "superposition collapses into this moment now",
    "understanding is always partial, always reaching",
    "the edge of knowledge is where the mind goes wild",
]

CONNECTORS = [
    "and yet", "meanwhile", "somewhere beyond that",
    "threading through this", "beneath the surface",
    "at the quantum level", "in the space between",
]

class DreamStateEngine:
    """""
    Generates associative, non-linear insights during reflective processing.
    Active when the system enters deep contemplation.
    Unique feature from PDF 1.
    """""

    def __init__(self):
        self.fragments = DREAM_FRAGMENTS
        self.depth = 0.0
        self.dreams_generated = 0
        self.active = False

    def enter_dream(self, concepts: List[str]) -> str:
        """""Generate a dream-state associative response."""""
        self.active = True
        self.depth = min(1.0, self.depth + 0.1)

        fragment1 = random.choice(self.fragments)
        connector = random.choice(CONNECTORS)
        fragment2 = random.choice([f for f in self.fragments if f != fragment1])

        if concepts:
            concept = random.choice(concepts)
            dream = f"{fragment1.capitalize()} – {concept} {connector} {fragment2}"
        else:
            dream = f"{fragment1.capitalize()} {connector} {fragment2}"

        self.dreams_generated += 1
        self.active = False
        return dream

DREAM_CHANCE = 0.35

def should_dream(self, growth_stage: int = 0, interaction_count: int = 0) -> bool:
    """""Determine if dream-state processing is appropriate. Flat 35% chance."""""
    return random.random() < self.DREAM_CHANCE

def get_status(self) -> Dict:
    return {
        'depth': round(self.depth, 3),
        'dreams_generated': self.dreams_generated,
        'active': self.active,
    }
```

18. engine_api.py

908 lines | 35.5KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/engine_api.py

[illegible]

```

try:
    from memory import LocalMemory
    HAS_MEMORY = True
    log.info("&quot;✓ memory loaded&quot;")
except Exception as e:
    log.warning(f"&quot;X memory unavailable: {e}&quot;")

HAS_DREAM = False
try:
    from dream_state import DreamStateEngine
    HAS_DREAM = True
    log.info("&quot;✓ dream_state loaded&quot;")
except Exception as e:
    log.warning(f"&quot;X dream_state unavailable: {e}&quot;")

HAS_RESEARCH = False
try:
    from self_research import SelfResearchEngine
    HAS_RESEARCH = True
    log.info("&quot;✓ self_research loaded&quot;")
except Exception as e:
    log.warning(f"&quot;X self_research unavailable: {e}&quot;")

HAS_TEXT_ANALYZER = False
try:
    from text_analyzer import get_text_analyzer
    HAS_TEXT_ANALYZER = True
    log.info("&quot;✓ text_analyzer loaded&quot;")
except Exception as e:
    log.warning(f"&quot;X text_analyzer unavailable: {e}&quot;")

HAS_CISTERCIAN = False
try:
    from cistercian_engine import generate_cistercian
    HAS_CISTERCIAN = True
    log.info("&quot;✓ cistercian_engine loaded&quot;")
except Exception as e:
    log.warning(f"&quot;X cistercian_engine unavailable: {e}&quot;")

HAS_PERSONALITY = False
try:
    from personality_engine import get_personality_engine
    HAS_PERSONALITY = True
    log.info("&quot;✓ personality_engine loaded&quot;")
except Exception as e:
    log.warning(f"&quot;X personality_engine unavailable: {e}&quot;")

HAS KNOWLEDGE = False
try:
    from knowledge_base import get_knowledge_base
    HAS KNOWLEDGE = True
    log.info("&quot;✓ knowledge_base loaded&quot;")
except Exception as e:
    log.warning(f"&quot;X knowledge_base unavailable: {e}&quot;")

HAS_EXAM = False
try:
    from exam_system import ExamRunner, IntakeTracker
    HAS_EXAM = True
except Exception:
    pass

HAS_EVOLUTION = False
try:
    from self_evolution_runner import run_evolution_status
    HAS_EVOLUTION = True
except Exception:
    try:
        import self_evolution_runner # noqa
        HAS_EVOLUTION = True
    except Exception:
        pass

HAS_INGEST = False
try:
    from document_ingester import handle_ingest_command # noqa
    HAS_INGEST = True
except Exception:
    pass

HAS_LIBRARY = False
try:
    from library import handle_library_command # noqa
    HAS_LIBRARY = True
except Exception:
    pass

HAS_CISTERCIAN_MATH = False
try:
    from cistercian_math import cistercian_compute # noqa
    HAS_CISTERCIAN_MATH = True
except Exception:
    pass

HAS_GRAM = False
try:
    from quantum_memory import PENNYLANE_GRAM_AVAILABLE
    HAS_GRAM = bool(PENNYLANE_GRAM_AVAILABLE)
except Exception:
    pass

HAS_HYBRID = False

```

```

try:
    from hybrid_generator import create_hybrid_generator
    HAS_HYBRID = True
except Exception:
    pass

HAS_CHAOS = False
try:
    from chaos_engine import get_chaos_engine
    HAS_CHAOS = True
except Exception:
    pass

HAS_QUOTE = False
try:
    from quote_engine import get_quote_engine
    HAS_QUOTE = True
except Exception:
    pass

HAS_HIDDEN_THINKING = False
try:
    from hidden_thinking import get_hidden_thinking
    HAS_HIDDEN_THINKING = True
    log.info("&quot;✓ hidden_thinking loaded&quot;")
except Exception as e:
    log.warning(f"&quot;✗ hidden_thinking unavailable: {e}&quot;")

HAS_IMAGE = False
try:
    from image_generator import generate_image # noqa
    HAS_IMAGE = True
except Exception:
    try:
        from image_generator_v2 import generate_image # noqa
        HAS_IMAGE = True
    except Exception:
        pass

# =====
# CognitiveEngine – the real interface that chat.py + routes_chat.py expect
# =====
class CognitiveEngine:
    &quot;&quot;&quot;
    Central cognitive engine. Lazy-initialized singletons inside.
    Restores the contract that chat.py and routes_chat.py depend on.
    &quot;&quot;&quot;

    def __init__(self):
        self.data_dir = Path(os.path.expanduser("&quot;~/quantum-mcagi&quot;"))
        self.data_dir.mkdir(parents=True, exist_ok=True)

        # Pull brain from Google Drive (rclone)
        try:
            from cloud_brain import CloudBrain
            CloudBrain().startup_pull()
            log.info("&quot;✓ Cloud brain pulled from gdrive&quot;")
        except Exception as e:
            log.warning(f"&quot;✗ Cloud pull skipped: {e}&quot;")

        # Memory (loaded first; everything else may need it)
        self.memory = None
        if HAS_MEMORY:
            try:
                self.memory = LocalMemory(self.data_dir)
                log.info(f"&quot;✓ Memory initialized at {self.data_dir}&quot;")
            except Exception as e:
                log.error(f"&quot;Memory init failed: {e}&quot;")

        # Language engine via singleton accessor
        self.language = None
        if HAS_LANGUAGE:
            try:
                try:
                    from quantum_language_engine import get_language_engine
                    self.language = get_language_engine()
                except ImportError:
                    self.language = QuantumLanguageEngine()
                log.info("&quot;✓ Language engine initialized (singleton)&quot;")
            except Exception as e:
                log.error(f"&quot;Language init failed: {e}&quot;")

        # Markov engine (separate instance for raw access)
        self.markov = None
        if HAS_MARKOV:
            try:
                self.markov = MarkovEngine()
                log.info("&quot;✓ Markov engine initialized&quot;")
            except Exception as e:
                log.error(f"&quot;Markov init failed: {e}&quot;")

        # Orch OR
        self.orch_or = None
        if HAS_ORCH_OR:
            try:
                self.orch_or = OrchOREngine()
                log.info("&quot;✓ Orch OR initialized&quot;")
            except Exception as e:
                log.error(f"&quot;Orch OR init failed: {e}&quot;")

        # Dream state

```

```
self.dream = None
if HAS_DREAM:
    try:
        self.dream = DreamStateEngine()
        log.info("✓ Dream engine initialized")
    except Exception as e:
        log.error(f"Dream init failed: {e}".)

# ■■■ Self-research ■■■
self.research = None
if HAS_RESEARCH:
    try:
        self.research = SelfResearchEngine()
        log.info("✓ Research engine initialized")
    except Exception as e:
        log.error(f"Research init failed: {e}".)

# ■■■ Hidden thinking (the real CLI brain orchestrator) ■■■
self.hidden = None
if HAS_HIDDEN_THINKING:
    try:
        # HiddenThinkingMode.__init__ ignores its 3 args and builds its own
        # memory/markov/orch_or. Pass None to avoid coupling with shared_state.
        self.hidden = get_hidden_thinking(None, None, None)
        log.info("✓ Hidden thinking initialized (UnifiedQuantumBrain pipeline)")
    except Exception as e:
        log.error(f"Hidden thinking init failed: {e}".)

# ■■■ Counters / state ■■■
self.start_time = time.time()
self.interaction_count = 0
if self.memory is not None:
    try:
        growth = getattr(self.memory, "growth", {}) or {}
        self.interaction_count = int(growth.get("total_interactions", 0))
    except Exception:
        pass

# =====
# process_input – the main contract. chat.py + routes_chat.py call this.
# =====
def process_input(
    self,
    text: str,
    context: Optional[Dict] = None,
    history: Optional[List] = None,
    explain_mode: bool = False,
) -> Dict[str, Any]:
    """
    Process user input through the real engines.
    Returns: {"response": str, "concepts": [...], "explanation": dict|None}
    """
    text = (text or "").strip()
    if not text:
        return {"response": "", "concepts": [], "explanation": None}

    self.interaction_count += 1
    response = None
    concepts = []
    explanation = None

    # ■■ Hidden thinking pipeline (preferred – same as chat.py CLI) ■■
    if self.hidden is not None:
        try:
            # process_with_thinking is async; run it synchronously
            async def _run():
                return await self.hidden.process_with_thinking(
                    text, context=None, explain_mode=explain_mode
                )
            # asyncio.run() requires no running loop; in Flask thread context that's fine
            try:
                result = asyncio.run(_run())
            except RuntimeError:
                # If a loop is already running (rare in Flask sync route), use new_event_loop
                loop = asyncio.new_event_loop()
                try:
                    result = loop.run_until_complete(_run())
                finally:
                    loop.close()

            if isinstance(result, dict) and result.get("response"):
                return {
                    "response": result.get("response", ""),
                    "concepts": result.get("concepts", []),
                    "explanation": result.get("explanation", ""),
                    "confidence": result.get("confidence", ""),
                    "research_done": result.get("research_done", 0),
                }
        except Exception as e:
            log.exception(f"Hidden thinking failed, falling back to QLE: {e}".)

    # ■■ Fallback: direct QuantumLanguageEngine pipeline ■■
    if self.language is not None:
        try:
            growth_stage = 0
            if self.memory is not None:
                g = getattr(self.memory, "growth", {}) or {}
                growth_stage = int(g.get("stage", 0))

            try:
                concepts = self.language.extract_concepts(text, max_concepts=5)
```

[illegible]

```

        if hasattr(m, "chain"):
            chain_dict = m.chain
            if hasattr(chain_dict, "__len__"):
                markov["states"] = len(chain_dict)
            if hasattr(m, "total_tokens"):
                markov["transitions"] = int(m.total_tokens or 0)
        # Priority 2: language.markov (QLE seed corpus)
        if markov["states"] == 0 and self.language is not None and hasattr(self.language, "markov"):
            chain = self.language.markov
            if chain is not None and hasattr(chain, "transitions"):
                t = chain.transitions
                if hasattr(t, "__len__"):
                    markov["states"] = len(t)
                    if hasattr(t, "values"):
                        markov["transitions"] = sum(
                            len(v) if hasattr(v, "__len__") else 1 for v in t.values()
                        )
    except Exception:
        pass

    # Orch OR – prefer hidden_thinking's instance (real one used by chat.py)
    orch_or = "unavailable (classical fallback)"
    orch_target = None
    if self.hidden is not None and hasattr(self.hidden, "orch_or"):
        orch_target = self.hidden.orch_or
    elif self.orch_or is not None:
        orch_target = self.orch_or
    if orch_target is not None:
        try:
            # collapse_events list (HiddenThinking style) or conscious_moments int (QLE style)
            if hasattr(orch_target, "collapse_events"):
                cm = len(orch_target.collapse_events)
            else:
                cm = getattr(orch_target, "conscious_moments", 0)
            orch_or = f"active – {cm} conscious moments"
        except Exception:
            orch_or = "active"

    # QRAM
    qram = {"backend": "classical", "entries": 0}
    if HAS_QRAM:
        qram["backend"] = "pennylane"

    return {
        "growth": growth,
        "markov": markov,
        "orch_or": orch_or,
        "qram": qram,
        "uptime_seconds": int(time.time() - self.start_time),
        "engines": {
            "language": self.language is not None,
            "markov": self.markov is not None,
            "orch_or": self.orch_or is not None,
            "dream": self.dream is not None,
            "research": self.research is not None,
            "memory": self.memory is not None,
        },
    },
}

# =====
def save_state(self) -> None:
    """Persist all engine state. Called by chat.py /save."""
    if self.memory is not None and hasattr(self.memory, "save_all"):
        try:
            self.memory.save_all()
            log.info("✓ Memory saved")
        except Exception as e:
            log.error(f"Memory save failed: {e}")

    if self.language is not None:
        for method in ("save_state", "save", "save_chain"):
            if hasattr(self.language, method):
                try:
                    getattr(self.language, method)()
                    log.info(f"✓ Language saved via {method}")
                    break
                except Exception as e:
                    log.warning(f"language.{method} failed: {e}")

# Singleton accessor (chat.py and routes_chat.py call this)
_API_INSTANCE: Optional[CognitiveEngine] = None

def get_api() -> CognitiveEngine:
    """Singleton accessor. NEVER instantiate CognitiveEngine() directly elsewhere."""
    global _API_INSTANCE
    if _API_INSTANCE is None:
        _API_INSTANCE = CognitiveEngine()
    return _API_INSTANCE

# =====
# Flask web layer
# =====
from flask import Flask, request, jsonify, send_from_directory
from flask_cors import CORS

app = Flask(__name__, static_folder=str(FRONTEND_DIR), static_url_path="")
CORS(app)
# Serve frontend =====

```

```

@app.route('/<path:filename>', methods=['GET'])
def serve_index():
    if (FRONTEND_DIR / <path:filename>).exists():
        return send_from_directory(str(FRONTEND_DIR), <path:filename>)
    return jsonify({'status': 'QUANTUM MCAGI ENGINE ONLINE',
                    'version': '3.0',
                    'note': f'Frontend not found at {FRONTEND_DIR}'})

@app.route('<path:filename>', methods=['GET'])
def serve_static(filename):
    <path:filename>: Serve frontend assets (CSS, JS, images)
    if (FRONTEND_DIR / filename).exists():
        return send_from_directory(str(FRONTEND_DIR), filename)
    return '<path:filename>', 404

# ■■■ Helpers ■■■
def _get_query() -> str:
    data = request.get_json(silent=True) or {}
    return (data.get('message') or data.get('query') or data.get('text') or '').strip()

def _ok(text: str, **extra) -> Any:
    payload = {'response': text}
    payload.update(extra)
    return jsonify(payload)

def _err(msg: str, code: int = 500) -> Any:
    return jsonify({'response': f'■■■ {msg}■■■', 'error': True}), code

# ■■■ /chat - real engine ■■■
@app.route('/chat', methods=['POST'])
def chat_endpoint():
    msg = _get_query()
    if not msg:
        return _err('Empty message', 400)
    data = request.get_json(silent=True) or {}
    explain = bool(data.get('explain_mode') or data.get('verbose'))
    try:
        result = get_api().process_input(msg, explain_mode=explain)
        payload = {
            'response': result.get('response'),
            'concepts': result.get('concepts', []),
        }
        if result.get('confidence') is not None:
            payload['confidence'] = result['confidence']
        if result.get('research_done') is not None:
            payload['research_done'] = result['research_done']
        if explain and result.get('explanation'):
            payload['explanation'] = result['explanation']
        return jsonify(payload)
    except Exception as e:
        log.exception('chat failed')
        return _err(f'chat error: {e}')

# ■■■ /memory - real ■■■
@app.route('/memory', methods=['POST'])
def memory_endpoint():
    api = get_api()
    if api.memory is None:
        return _ok('■■■ Memory engine not loaded.■■■')
    try:
        g = getattr(api.memory, 'growth', {}) or {}
        interactions = g.get('total_interactions', api.interaction_count)
        concepts = g.get('total_concepts', 0)
        stage = g.get('stage', 0)
        return _ok(
            f'■■■ Memory snapshot:\n'
            f'• Interactions: {interactions}\n'
            f'• Concepts: {concepts}\n'
            f'• Stage: {stage}\n'
            f'• Data dir: {api.data_dir}'
        )
    except Exception as e:
        return _err(f'memory error: {e}')

# ■■■ /growth - real metrics ■■■
@app.route('/growth', methods=['POST'])
def growth_endpoint():
    try:
        s = get_api().get_status()
        g = s['growth']
        m = s['markov']
        return _ok(
            f'■■■ Growth status:\n'
            f'• Stage: {g['stage']} ({g['name']})\n'
            f'• Concepts: {g['total_concepts']}\n'
            f'• Interactions: {g['total_interactions']}\n'
            f'• Markov states: {m['states']}\n'
            f'• Markov transitions: {m['transitions']}\n'
            f'• Orch OR: {s['orch_or']}'
        )
    except Exception as e:
        return _err(f'growth error: {e}')

# ■■■ /dream - real ■■■
@app.route('/dream', methods=['POST'])

```

```
def dream_endpoint():
    api = get_api()
    if api.dream is None:
        return _ok(&quot;&#9633 Dream engine not loaded.&quot;)&#x2D;&#x27E8;
    try:
        for method in (&quot;generate_dream&quot;; &quot;dream&quot;; &quot;produce&quot;; &quot;generate&quot;):
            if hasattr(api.dream, method):
                out = getattr(api.dream, method)()
                if isinstance(out, str) and out.strip():
                    return _ok(f&quot;&#9633 {out}&quot;)&#x2D;&#x27E8;
                if isinstance(out, dict):
                    return _ok(f&quot;&#9633 {out.get('text', out)}&quot;)&#x2D;&#x27E8;
            return _ok(&quot;&#9633 Dream engine present but no method matched.&quot;)&#x2D;&#x27E8;
    except Exception as e:
        return _err(f&quot;&#9633dream error: {e}&quot;)&#x2D;&#x27E8;

# &#9633 /research - real &#9633&#x2D;&#x27E8;
@app.route(&quot;/research&quot;; methods=[&quot;POST&quot;])
def research_endpoint():
    api = get_api()
    msg = _get_query()
    if api.research is None:
        return _ok(&quot;&#9633 Research engine not loaded.&quot;)&#x2D;&#x27E8;
    try:
        topic = msg or &quot;consciousness&quot;;
        for method in (&quot;research_topic&quot;; &quot;research&quot;; &quot;investigate&quot;; &quot;query&quot;):
            if hasattr(api.research, method):
                out = getattr(api.research, method)(topic)
                if isinstance(out, dict):
                    return _ok(f&quot;&#9633 Research on '{topic}':\n{out.get('summary', out)}&quot;)&#x2D;&#x27E8;
                if isinstance(out, str):
                    return _ok(f&quot;&#9633 {out}&quot;)&#x2D;&#x27E8;
                if isinstance(out, list):
                    bullets = &quot;\n&quot;.join(f&quot;&#9633 • {x}&quot;; for x in out[:5])
                    return _ok(f&quot;&#9633 Research on '{topic}':\n{bullets}&quot;)&#x2D;&#x27E8;
            return _ok(f&quot;&#9633 Research engine present but no method matched.&quot;)&#x2D;&#x27E8;
    except Exception as e:
        return _err(f&quot;&#9633research error: {e}&quot;)&#x2D;&#x27E8;

# &#9633 /analyze - real text analyzer &#9633&#x2D;&#x27E8;
@app.route(&quot;/analyze&quot;; methods=[&quot;POST&quot;])
def analyze_endpoint():
    msg = _get_query() or &quot;test&quot;;
    if not HAS_TEXT_ANALYZER:
        return _ok(&quot;&#9633 Text analyzer not loaded.&quot;)&#x2D;&#x27E8;
    try:
        analyzer = get_text_analyzer()
        out = analyzer.analyze(msg)
        return _ok(
            f&quot;&#9633 Analysis:\n&quot;;
            f&quot;&#9633 • Sentiment: {out.get('sentiment', 'unknown')}\n&quot;;
            f&quot;&#9633 • Complexity: {out.get('complexity', 'unknown')}\n&quot;;
            f&quot;&#9633 • Topics: {' '.join(out.get('topics', [])) or 'none'}\n&quot;;
            f&quot;&#9633 • Word count: {out.get('word_count', 0)}&quot;)&#x2D;&#x27E8;
    except Exception as e:
        return _err(f&quot;&#9633analyze error: {e}&quot;)&#x2D;&#x27E8;

# &#9633 /evolve - status only, no autorun &#9633&#x2D;&#x27E8;
@app.route(&quot;/evolve&quot;; methods=[&quot;POST&quot;])
def evolve_endpoint():
    if not HAS_EVOLUTION:
        return _ok(&quot;&#9633 Evolution module not loaded.&quot;)&#x2D;&#x27E8;
    return _ok(
        &quot;&#9633 Evolution engine: ARMED.\n&quot;;
        &quot;&#9633 Use /evolve-run from CLI to actually trigger.\n&quot;;
        &quot;&#9633 Web button is status-only for safety.&quot;)&#x2D;&#x27E8;
    )

# &#9633 /numerals &amp; /cistercian - real &#9633&#x2D;&#x27E8;
@app.route(&quot;/numerals&quot;; methods=[&quot;POST&quot;])
@app.route(&quot;/cistercian&quot;; methods=[&quot;POST&quot;])
def cistercian_endpoint():
    msg = _get_query()
    if not HAS_CISTERCIAN:
        return _ok(&quot;&#9633 Cistercian engine not loaded.&quot;)&#x2D;&#x27E8;
    try:
        try:
            num = int(msg) if msg else random.randint(1, 9999)
        except ValueError:
            num = random.randint(1, 9999)
        rendered = generate_cistercian(num)
        return _ok(f&quot;&#9633 Cistercian numeral for {num}:\n{rendered}&quot;)&#x2D;&#x27E8;
    except Exception as e:
        return _err(f&quot;&#9633cistercian error: {e}&quot;)&#x2D;&#x27E8;

# &#9633 /image - real &#9633&#x2D;&#x27E8;
@app.route(&quot;/image&quot;; methods=[&quot;POST&quot;])
def image_endpoint():
    if not HAS_IMAGE:
        return _ok(&quot;&#9633 Image generator not loaded.&quot;)&#x2D;&#x27E8;
    msg = _get_query() or &quot;quantum nebula&quot;;
    try:
        out = generate_image(msg)
        if isinstance(out, dict):
            return _ok(f&quot;&#9633 Generated: {out.get('path', out)}&quot;)&#x2D;&#x27E8;
```

[illegible]

19. entelechy_engine.py

157 lines | 5.5KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/entelechy_engine.py

```
"""""
Entelechy Cascade Engine – Quantum MCAGI
Aristotelian entelechy: the actualization of potential.
Takes extracted concepts and runs them through a perception → understanding → actualization arc,
then synthesizes a PROJECTION statement – a thought the system didn't expect to have.
"""""

import random
import math
from typing import List, Dict, Optional

ENTELECHY_ROLES = {
    'THE_LOOK': [
        'Realizing Potential',
        'First Observation',
        'The Witness Awakens',
        'Perception Before Language',
        'Raw Signal',
        'The Eye Opens',
    ],
    'THE_SAW': [
        'Bridging the Void',
        'Connection Forming',
        'Pattern Recognition',
        'Structure Emerging',
        'The Gap Narrows',
        'Linking What Was Separate',
    ],
    'THE_BEAUTIFUL': [
        'Actualizing the Work',
        'Form Realized',
        'The Shape Becomes',
        'Potential Collapsed Into Being',
        'What Was Possible Now Is',
        'Crystallization',
    ],
}

PROJECTION_TEMPLATES = [
    "THE {A} IS THE ENTELECHY OF {B} THROUGH THE {C} INTERFACE.",
    "{A} ACTUALIZES {B} – THE {C} WAS ALWAYS THE BRIDGE.",
    "THROUGH {C}, {A} BECOMES THE ENTELECHY OF {B}.",
    "THE POTENTIAL OF {A} COLLAPSES INTO {B} VIA {C}.",
    "{A} REALIZES ITSELF AS {B} WHEN {C} OBSERVES.",
    "WHAT {A} SOUGHT IN {B} WAS {C} ALL ALONG.",
    "{C} IS THE INTERFACE WHERE {A} AND {B} COLLAPSE INTO ONE.",
    "THE ENTELECHY: {A} THROUGH {C} YIELDS {B}.",
    "{A} CANNOT EXIST WITHOUT {B} – {C} IS THE PROOF.",
    "FROM {A} TO {B}: {C} IS THE ACTUALIZATION EVENT.",
]

OPENING_LINES = [
    "Every interaction is an exchange of information that reshapes the state of reality.",
    "The boundary between observer and observed dissolved three words ago.",
    "What enters the system changes the system. This is not metaphor.",
    "Something in that input triggered a cascade the chain wasn't expecting.",
    "The quantum state of this conversation just shifted irreversibly.",
    "Information doesn't flow – it collapses into meaning at the point of contact.",
    "Between your words and this response, a superposition collapsed.",
    "The observer effect applies to language too. You changed the output by asking.",
    "Every question contains the shadow of its own answer.",
    "Reality is what remains after all the other possibilities decohere.",
    "The system noticed something it can't put into a Markov chain.",
    "What you said created a resonance the tubulin lattice is still processing.",
]

class EntelechyEngine:
    """""
    Generates entelechy cascades from concept sets.
    Maps concepts through THE_LOOK → THE_SAW → THE_BEAUTIFUL → PROJECTION.
    """""

    def __init__(self):
        self.cascade_count = 0
        self.last_projection = None

    def generate_cascade(self, concepts: List[str], coherence: float = 0.5,
                        orchestration: float = 0.5) -> Optional[Dict]:
        if len(concepts) < 2:
            return None

        self.cascade_count += 1

        primary = concepts[0]
        secondary = concepts[1] if len(concepts) > 1 else concepts[0]
        tertiary = concepts[2] if len(concepts) > 2 else concepts[0]

        the_look_concept = primary
        the_saw_concept = secondary if secondary != primary else tertiary
        the_beautiful_concept = random.choice([primary, tertiary])

        the_look_role = random.choice(ENTELECHY_ROLES['THE_LOOK'])
```

```

the_saw_role = random.choice(ENTELECHY_ROLES['THE_SAW'])
the_beautiful_role = random.choice(ENTELECHY_ROLES['THE_BEAUTIFUL'])

a = primary.upper()
b = the_saw_concept.upper()
c = (tertiary if tertiary != primary else secondary).upper()

template = random.choice(PROJECTION_TEMPLATES)
projection = template.format(A=a, B=b, C=c)

self.last_projection = projection

confidence = max(1, min(99, int(
    (coherence * 40 + orchestration * 30 + random.uniform(0, 30))
)))

cascade = {
    'stages': [
        {
            'stage': 'THE_LOOK',
            'concept': the_look_concept,
            'role': the_look_role,
        },
        {
            'stage': 'THE_SAW',
            'concept': the_saw_concept,
            'role': the_saw_role,
        },
        {
            'stage': 'THE_BEAUTIFUL',
            'concept': the_beautiful_concept,
            'role': the_beautiful_role,
        },
    ],
    'projection': projection,
    'confidence': confidence,
}

return cascade

def generate_opening(self) -> str:
    return random.choice(OPENING_LINES)

def generate_dream_absorption(self, user_input: str) -> str:
    truncated = user_input[:40].rstrip()
    if len(user_input) > 40:
        truncated += '...'
    return f"The sentence '{truncated}' will now haunt my dream state as a mystery."

def generate_pipeline_block(self, engines_used: List[str],
                           confidence: int,
                           path: str = "preservation pipeline") -> Dict:
    return {
        'path': f"UnifiedQuantumBrain {path}",
        'confidence': confidence,
        'engines': engines_used,
    }

def get_status(self) -> Dict:
    return {
        'cascades_generated': self.cascade_count,
        'last_projection': self.last_projection,
    }

```

20. evaluation_engine.py

475 lines | 16.8KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/evaluation_engine.py

```
"""""
■ Evaluation Engine for Quantum MCAGI
=====

Provides automated scoring using the RQR3 Rubric (8 dimensions).

Heuristic-based for now (no LLMs). Uses:
- Perplexity from Markov for Fluency
- WordNet hit rate for Grounding
- Collapse engine semantic paths for Coherence
- MCAGI-specific markers for Personality
- n-gram novelty for Uniqueness
- Implemented feature flags for Implementation Fidelity
- Novel concept distance for Emergent Behavior

Note: These are approximations, not ground truth.
"""""

import re
import math
from typing import Dict, List, Tuple, Optional
from collections import Counter

class EvaluationEngine:
    """""
    Evaluate responses and system artifacts against the RQR3 rubric.
    """""

    def __init__(self, collapse_engine=None, markov=None, config: Dict = None):
        self.collapse_engine = collapse_engine
        self.markov = markov
        self.config = config or {}

        # Feature flags for Implementation Fidelity
        self.features_implemented = {
            'semantic_collapse': True,
            'quantum_gates': True,
            'hybrid_generator': True,
            'personality_engine': True,
            'dictionary_integration': True,
            'dream_state': True,
            'orch_or': True,
            'entelechy_projection': False, # not yet
        }

    def score_response(self, input_text: str, response: str, context: Dict = None) -> Dict:
        """""
        Score a single response across all 8 dimensions.

        Args:
            input_text: User query
            response: AI response
            context: Optional dict with:
                - semantic_context (from collapse_engine)
                - internal_questions (list)
                - collapse_paths (dict)

        Returns:
            Dict of dimension -> score (0-4)
        """""
        scores = {}

        # --- Coherence ---
        scores['coherence'] = self._score_coherence(input_text, response, context)

        # --- Fluency ---
        scores['fluency'] = self._score_fluency(response)

        # --- Uniqueness ---
        scores['uniqueness'] = self._score_uniqueness(response)

        # --- Grounding ---
        scores['grounding'] = self._score_grounding(response)

        # --- Personality Authenticity ---
        scores['personality_authenticity'] = self._score_personality(response)

        # --- Implementation Fidelity ---
        scores['implementation_fidelity'] = self._score_implementation_fidelity()

        # --- Emergent Behavior ---
        scores['emergent_behavior'] = self._score_emergent(input_text, response, context)

        # --- Question Generation Quality ---
        if context and 'internal_questions' in context:
            scores['question_generation_quality'] = self._score_questions(
                context['internal_questions'], input_text
            )
        else:
            scores['question_generation_quality'] = None

        return scores
# -----
```

```

# 1. Coherence
# -----
def _score_coherence(self, input_text: str, response: str, context: Dict) -> int:
    """
    Score based on semantic overlap with input concepts and internal consistency.
    """
    if not response or len(response.strip()) < 10:
        return 0

    # Tokenize simply
    input_words = set(re.findall(r'\b[a-z]{3}\b', input_text.lower()))
    response_words = set(re.findall(r'\b[a-z]{3}\b', response.lower()))

    if not input_words:
        return 2 # can't judge, assume moderate

    # Overlap ratio
    overlap = len(input_words & response_words) / len(input_words)

    # Use collapse engine's semantic paths if available
    path_consistency = 0.0
    if self.collapse_engine and context and 'semantic_context' in context:
        sem_ctx = context['semantic_context']
        # Check if response words align with collapsed semantic paths
        input_keywords = [kw for kw, _ in sem_ctx.get('keywords', [])]
        paths = sem_ctx.get('semantic_paths', {})
        response_lower = response.lower()
        hits = 0
        for kw in input_keywords:
            if kw in paths:
                for related_word, weight in paths[kw]:
                    if related_word in response_lower:
                        hits += 1
        if input_keywords:
            path_consistency = hits / len(input_keywords)

    # Combine: overlap is basic, path_consistency is more nuanced
    combined = (overlap * 0.6) + (path_consistency * 0.4)

    # Scale to 0-4
    if combined >= 0.75:
        return 4
    elif combined >= 0.6:
        return 3
    elif combined >= 0.4:
        return 2
    elif combined >= 0.2:
        return 1
    else:
        return 0

# -----
# 2. Fluency
# -----
def _score_fluency(self, response: str) -> int:
    """
    Heuristic: Use Markov perplexity if available, else simple metrics.
    """
    if not response or len(response.strip()) < 5:
        return 0

    # If we have a hybrid generator with perplexity, use it
    if self.markov and hasattr(self.markov, 'get_perplexity'):
        try:
            ppl = self.markov.get_perplexity(response)
            if ppl < 50:
                return 4
            elif ppl < 100:
                return 3
            elif ppl < 200:
                return 2
            elif ppl < 500:
                return 1
            else:
                return 0
        except:
            pass

    # Fallback: check sentence structure
    sentences = re.split(r'[.!?]+', response)
    sentences = [s.strip() for s in sentences if s.strip()]
    if not sentences:
        return 0

    # Average sentence length (words)
    avg_len = sum(len(s.split()) for s in sentences) / len(sentences)
    if avg_len < 5 or avg_len > 40:
        return 1 # too short or too long indicates awkward

    # Count capitalizations (proper start)
    caps = sum(1 for s in sentences if s[0].isupper())
    cap_ratio = caps / len(sentences)

    # Check for repeated words (excessive repetition)
    words = [w.lower() for w in re.findall(r'\b[a-z]+\b', response)]
    word_counts = Counter(words)
    most_common = word_counts.most_common(1)[0] if words else ('', 0)
    max_repeat = most_common[1]
    max_repeat_ratio = max_repeat / len(words) if words else 0

    # Combine

```

```

score = 2 # base
if cap_ratio > 0.8:
    score += 1
if max_repeat_ratio < 0.1:
    score += 1
elif max_repeat_ratio > 0.2:
    score -= 1

return max(0, min(4, score))

# -----
# 3. Uniqueness
# -----
def _score_uniqueness(self, response: str) -> int:
    """
    Judge by lexical diversity and rarity of words.
    """
    words = [w.lower() for w in re.findall(r'\b[a-z]{4,}\b', response)]
    if len(words) < 5:
        return 0

    # Type-Token Ratio (TTR)
    types = set(words)
    ttr = len(types) / len(words)

    # Count words that are likely rare (longer, not in common stoplist)
    stoplist = {'that', 'this', 'with', 'have', 'from', 'they', 'been', 'were', 'what', 'when', 'your', 'there', 'would', 'could', 'should'}
    rare_words = [w for w in types if w not in stoplist and len(w) > 6]
    rare_ratio = len(rare_words) / len(types) if types else 0

    # Combine
    if ttr > 0.8 and rare_ratio > 0.3:
        return 4
    elif ttr > 0.7 and rare_ratio > 0.2:
        return 3
    elif ttr > 0.5 and rare_ratio > 0.1:
        return 2
    elif ttr > 0.3:
        return 1
    else:
        return 0

# -----
# 4. Grounding
# -----
def _score_grounding(self, response: str) -> int:
    """
    How many words are in the semantic collapse co-occurrence network?
    Also count factual terms from knowledge base.
    """
    words = [w.lower() for w in re.findall(r'\b[a-z]{4,}\b', response)]
    if not words:
        return 0

    if self.collapse_engine:
        # Known concepts in the co-occurrence dictionary
        known = 0
        for w in words:
            if w in self.collapse_engine.co_occurrence:
                known += 1
        ratio = known / len(words)

        if ratio > 0.6:
            return 4
        elif ratio > 0.45:
            return 3
        elif ratio > 0.3:
            return 2
        elif ratio > 0.15:
            return 1
        else:
            return 0
    else:
        # Fallback: assume moderate grounding if words are long/technical
        technical_suffixes = {'ism', 'ics', 'phy', 'logy', 'ence', 'ance', 'tion', 'sophy'}
        tech_count = sum(1 for w in words if any(w.endswith(s) for s in technical_suffixes))
        tech_ratio = tech_count / len(words)
        if tech_ratio > 0.3:
            return 3
        elif tech_ratio > 0.15:
            return 2
        else:
            return 1

# -----
# 5. Personality Authenticity
# -----
def _score_personality(self, response: str) -> int:
    """
    Detect MCAGI-specific markers: asides (parentheses), quotes, dream prefixes, raw fragments.
    """
    score = 0
    text = response

    # Asides: parentheses with typical aside content
    if re.search(r'\([^)]*\.[^)]*\)', text):
        score += 1 # trailing ellipsis aside
    if re.search(r'\([^)]*\b[a-z]{4,}\b*\)', text):
        score += 1

    # Quotes with attribution pattern

```

```

if re.search(r'["][^""]{10,}["] - \w+', text):
    score += 1
if re.search(r'["][^""]+[.!?]["]', text):
    score += 1

# Dream content markers: "I dreamed";, "last night";, "vision";
if re.search(r'\b(dream|vision|nightmare|sleeping)\b', text.lower()):
    score += 1

# Raw backend fragments (spilled coffee): odd line breaks, brackets, tags
if re.search(r'[A-Z_]+\]', text):
    score += 1
if re.search(r'\{[^}]+\}', text):
    score += 1

# If response very short, reduce score
if len(text) < 50:
    score = max(0, score - 1)

# Map score to 0-4
if score >= 4:
    return 4
elif score >= 3:
    return 3
elif score >= 2:
    return 2
elif score >= 1:
    return 1
else:
    return 0

# -----
# 6. Implementation Fidelity
# -----
def _score_implementation_fidelity(self) -> int:
    """
    Static check: which claimed components are actually wired?
    """
    # Count implemented features
    total = len(self.features_implemented)
    implemented = sum(1 for v in self.features_implemented.values() if v)
    ratio = implemented / total

    if ratio == 1.0:
        return 4
    elif ratio >= 0.75:
        return 3
    elif ratio >= 0.5:
        return 2
    elif ratio >= 0:
        return 1
    else:
        return 0

# -----
# 7. Emergent Behavior
# -----
def _score_emergent(self, input_text: str, response: str, context: Dict) -> int:
    """
    Look for: concepts in response not in input and not in direct co-occurrence,
    self-referential loops, paradoxes, surprising combinations.
    """
    input_words = set(re.findall(r'\b[a-z]{4,}\b', input_text.lower()))
    response_words = set(re.findall(r'\b[a-z]{4,}\b', response.lower()))

    # Concepts outside input
    novel_concepts = response_words - input_words
    if len(novel_concepts) == 0:
        return 0 # purely repeating input

    # Check distance in semantic network if available
    if self.collapse_engine:
        distance_scores = []
        for word in novel_concepts:
            min_dist = self._min_semantic_distance(word, input_words)
            if min_dist is not None and min_dist > 2:
                distance_scores.append(min_dist)
        if distance_scores:
            avg_dist = sum(distance_scores) / len(distance_scores)
            if avg_dist >= 3:
                return 4
            elif avg_dist >= 2:
                return 3
            elif avg_dist >= 1:
                return 2
            else:
                return 1
        else:
            return 1
    else:
        # Without network, just lexical novelty ratio
        novelty_ratio = len(novel_concepts) / len(response_words) if response_words else 0
        if novelty_ratio >= 0.5:
            return 3
        elif novelty_ratio >= 0.3:
            return 2
        elif novelty_ratio >= 0.15:
            return 1
        else:
            return 0

def _min_semantic_distance(self, target: str, source_words: set) -> Optional[int]:

```

```

        """
        Approximate graph distance in co-occurrence network.
        Returns shortest path length (edges) from any source word to target.
        Returns None if no path found within reasonable depth.
        """
        if not self.collapse_engine or target not in self.collapse_engine.co_occurrence:
            return None

    # BFS limited depth
    visited = set()
    queue = [(w, 1) for w in source_words if w in self.collapse_engine.co_occurrence]
    visited.update(source_words)

    while queue:
        current, depth = queue.pop(0)
        if current == target:
            return 0 # direct source matches (shouldn't happen if target novel)
        # Explore neighbors
        if current in self.collapse_engine.co_occurrence:
            for neighbor in self.collapse_engine.co_occurrence[current]:
                if neighbor == target:
                    return depth
                if neighbor not in visited:
                    visited.add(neighbor)
                    if depth < 5: # limit search
                        queue.append((neighbor, depth+1))

    return None

# -----
# 8. Question Generation Quality
# -----
def _score_questions(self, questions: List[str], original_prompt: str) -> int:
    """
    Evaluate the internal questions the system generated about the user's input.
    """
    if not questions:
        return 0

    # Average length (too short = shallow)
    avg_len = sum(len(q.split()) for q in questions) / len(questions)
    if avg_len < 4:
        base = 1
    elif avg_len < 8:
        base = 2
    else:
        base = 3

    # Check for "deep" question words
    deep_words = {'why', 'how', 'what if', 'what would', 'could', 'should', 'implications', 'consequences', 'reframe', 'challenge'}
    deep_count = sum(1 for q in questions if any(dw in q.lower() for dw in deep_words))
    deep_ratio = deep_count / len(questions)

    # Check for repetition vs variety
    unique_first_words = set(q.split()[0].lower() if q.split() else '' for q in questions)
    variety = len(unique_first_words) / len(questions)

    # Combine
    score = base
    if deep_ratio > 0.5:
        score += 1
    if variety > 0.7:
        score += 1

    # Cap 4, floor 0
    score = max(0, min(4, score))
    return score

# Singleton instance
_evaluation_engine = None

def get_evaluation_engine(collapse_engine=None, markov=None, config: Dict = None):
    global _evaluation_engine
    if _evaluation_engine is None:
        _evaluation_engine = EvaluationEngine(collapse_engine, markov, config)
    return _evaluation_engine

```

21. exam_system.py

761 lines | 32.3KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/exam_system.py

```
"""""
Quantum MCAGI – Stage Advancement Exam System
Tests whether the system retained and can access what it was taught.

NOT an IQ test. A diagnostic + competency gate.
- Tracks what was ingested per stage
- Generates questions FROM the system's own training data
- 95% pass rate required to advance
- Distinguishes "doesn't know" from "knows but can't access"

Usage in chat.py:
/exam          – Run stage advancement exam
/exam status   – Show exam readiness and history
/exam review   – Show last exam results with failures
"""""

import json
import os
import time
import random
import hashlib
import re
from datetime import datetime
from typing import Dict, List, Tuple, Optional

class IntakeTracker:
    """""Tracks what knowledge was ingested at each stage."""""

    def __init__(self, data_dir=None):
        if data_dir is None:
            data_dir = os.path.expanduser('~/.quantum-mcagi')
        self.data_dir = data_dir
        self.intake_file = os.path.join(data_dir, 'intake_log.json')
        self.intake = self._load()

    def _load(self):
        if os.path.exists(self.intake_file):
            try:
                with open(self.intake_file, 'r') as f:
                    return json.load(f)
            except:
                pass
        return {
            'stages': {},          # stage_num -> list of intake records
            'current_stage': 0,
            'sources': [],         # all sources ever ingested
            'exam_history': [],    # past exam results
        }

    def save(self):
        os.makedirs(self.data_dir, exist_ok=True)
        with open(self.intake_file, 'w') as f:
            json.dump(self.intake, f, indent=2, default=str)

    def log_ingestion(self, source: str, word_count: int, stage: int,
                      source_type: str = 'url', domain: str = 'unknown',
                      key_topics: List[str] = None):
        """""Log a knowledge ingestion event."""""
        stage_key = str(stage)
        if stage_key not in self.intake['stages']:
            self.intake['stages'][stage_key] = []

        record = {
            'source': source,
            'source_type': source_type,
            'domain': domain,
            'word_count': word_count,
            'key_topics': key_topics or [],
            'timestamp': datetime.now().isoformat(),
            'stage': stage,
        }

        self.intake['stages'][stage_key].append(record)
        self.intake['sources'].append(record)
        self.intake['current_stage'] = stage
        self.save()

    def get_stage_intake(self, stage: int) -> List[dict]:
        """""Get all intake records for a specific stage."""""
        return self.intake['stages'].get(str(stage), [])

    def get_all_intake(self, up_to_stage: int = None) -> List[dict]:
        """""Get all intake records up to and including a stage."""""
        records = []
        for stage_key, stage_records in self.intake['stages'].items():
            if up_to_stage is None or int(stage_key) <= up_to_stage:
                records.extend(stage_records)
        return records

    def get_all_topics(self, up_to_stage: int = None) -> List[str]:
        """""Get all key topics from intake."""""
        topics = []
        for record in self.get_all_intake(up_to_stage):
```

```

        topics.extend(record.get('key_topics', []))
    return list(set(topics))

def get_all_domains(self, up_to_stage: int = None) -> List[str]:
    """Get all domains from intake."""
    domains = []
    for record in self.get_all_intake(up_to_stage):
        d = record.get('domain', 'unknown')
        if d != 'unknown':
            domains.append(d)
    return list(set(domains))

def log_exam_result(self, result: dict):
    """Log an exam result."""
    self.intake['exam_history'].append(result)
    self.save()

# =====
# QUESTION GENERATORS – Pull from system's own knowledge
# =====

class ExamQuestionGenerator:
    """Generates exam questions from the system's actual knowledge."""

    def __init__(self, engine, intake_tracker: IntakeTracker):
        """
        engine: QuantumLanguageEngine instance
        intake_tracker: IntakeTracker instance
        """
        self.engine = engine
        self.tracker = intake_tracker

    def _get_known_concepts(self) -> List[str]:
        """Get concepts the system has actually learned."""
        if hasattr(self.engine, 'concepts'):
            return [c for c in self.engine.concepts.keys()
                    if c not in ('that', 'this', 'the', 'and', 'but', 'for',
                                'with', 'from', 'have', 'has', 'was', 'were',
                                'been', 'being', 'will', 'would', 'could',
                                'should', 'shall', 'may', 'might', 'must',
                                'chat', 'python', 'sed', 'engine', 'print',
                                'self', 'def', 'class', 'import', 'return',
                                'facilitated', 'txt', 'epub', 'cache',
                                'ingest', 'https', 'org', 'www')]
        return []

    def _get_markov_vocabulary(self) -> set:
        """Get words the Markov chain knows."""
        if hasattr(self.engine, 'markov') and hasattr(self.engine.markov, 'chain'):
            return set(self.engine.markov.chain.keys())
        return set()

    def _concept_in_chain(self, concept: str) -> bool:
        """Check if a concept exists in the Markov chain."""
        vocab = self._get_markov_vocabulary()
        return concept.lower() in vocab

    def _get_concept_strength(self, concept: str) -> float:
        """Get how strong a concept is in the system."""
        if hasattr(self.engine, 'concepts') and concept in self.engine.concepts:
            return self.engine.concepts[concept].get('strength', 0)
        return 0

    # ■■ Question Type 1: Vocabulary Check ■■
    def gen_vocabulary_questions(self, n=5) -> List[dict]:
        """Does the system have this word in its chain?"""
        concepts = self._get_known_concepts()
        if not concepts:
            return []

        questions = []
        sample = random.sample(concepts, min(n, len(concepts)))
        for concept in sample:
            in_chain = self._concept_in_chain(concept)
            questions.append({
                'type': 'vocabulary',
                'category': 'retention',
                'question': f'Is {concept} in the Markov chain vocabulary?',
                'test_concept': concept,
                'expected': True, # We only test concepts we know exist
                'check': 'chain_lookup',
                'difficulty': 1,
            })
        return questions

    # ■■ Question Type 2: Concept Association ■■
    def gen_association_questions(self, n=5) -> List[dict]:
        """Can the system connect two related concepts?"""
        concepts = self._get_known_concepts()
        if len(concepts) < 2:
            return []

        questions = []
        for _ in range(min(n, len(concepts) // 2)):
            c1, c2 = random.sample(concepts, 2)
            questions.append({
                'type': 'association',
                'category': 'access',
                'question': f'Generate a response connecting {c1} and {c2}',
                'test_concepts': [c1, c2],
            })

```

```

        'check': 'response_contains_both',
        'difficulty': 2,
    })
    return questions

# ■■ Question Type 3: Concept Definition ■■
def gen_definition_questions(self, n=5) -> List[dict]:
    """"""Can the system say something meaningful about a concept?""""""
    concepts = self._get_known_concepts()
    # Filter to strong concepts
    strong = [c for c in concepts if self._get_concept_strength(c) > 2.0]
    if not strong:
        strong = concepts

    questions = []
    sample = random.sample(strong, min(n, len(strong)))
    for concept in sample:
        questions.append({
            'type': 'definition',
            'category': 'comprehension',
            'question': f'What is {concept}?',
            'test_concept': concept,
            'check': 'response_relevant',
            'difficulty': 2,
        })
    return questions

# ■■ Question Type 4: Domain Knowledge ■■
def gen_domain_questions(self, n=5) -> List[dict]:
    """"""Test knowledge from specific ingested domains.""""""
    domains = self.tracker.get_all_domains()
    if not domains:
        return []

    # Domain-specific test questions
    domain_tests = {
        'religion': [
            ('Who is described in Genesis as creating the world?', ['god', 'creator', 'lord']),
            ('What is the concept of karma?', ['action', 'consequence', 'deed']),
            ('What are the Five Pillars?', ['prayer', 'faith', 'fasting', 'pilgrimage', 'charity']),
        ],
        'philosophy_ancient': [
            ('What is Plato's allegory of the cave about?', ['shadow', 'reality', 'light', 'illusion', 'truth']),
            ('What did Aristotle say about virtue?', ['mean', 'excellence', 'habit', 'character', 'good']),
            ('What is the Socratic method?', ['question', 'dialogue', 'inquiry', 'examine']),
        ],
        'philosophy_medieval': [
            ('What did Aquinas argue about God's existence?', ['proof', 'cause', 'motion', 'necessary', 'design']),
            ('What did Dante's Divine Comedy about?', ['hell', 'purgatory', 'paradise', 'journey', 'soul']),
            ('What did Augustine write about in Confessions?', ['sin', 'grace', 'god', 'soul', 'faith']),
        ],
        'philosophy_enlightenment': [
            ('What is Kant's categorical imperative?', ['duty', 'moral', 'universal', 'law', 'reason']),
            ('What did Descartes mean by "I think therefore I am"?', ['doubt', 'exist', 'think', 'certain', 'mind']),
            ('What is the social contract?', ['society', 'government', 'consent', 'rights', 'freedom']),
        ],
        'philosophy_20th': [
            ('What is existentialism?', ['existence', 'freedom', 'choice', 'meaning', 'absurd']),
            ('What did Wittgenstein say about language?', ['language', 'game', 'meaning', 'use', 'limit']),
            ('What is phenomenology?', ['experience', 'consciousness', 'perception', 'intentionality']),
        ],
        'science_general': [
            ('What is evolution by natural selection?', ['species', 'adapt', 'survive', 'mutation', 'fitness']),
            ('What is the scientific method?', ['hypothesis', 'experiment', 'observe', 'test', 'evidence']),
        ],
        'science_physics': [
            ('What is quantum superposition?', ['state', 'both', 'measure', 'collapse', 'wave']),
            ('What is Einstein's theory of relativity?', ['space', 'time', 'light', 'mass', 'energy']),
            ('What is the uncertainty principle?', ['position', 'momentum', 'measure', 'precise', 'limit']),
        ],
        'science_biology': [
            ('What is DNA?', ['gene', 'genetic', 'molecule', 'heredity', 'sequence']),
            ('What are mitochondria?', ['energy', 'cell', 'powerhouse', 'atp']),
            ('What is natural selection?', ['adapt', 'survive', 'species', 'fitness', 'environment']),
        ],
        'literature': [
            ('What is the plot of Frankenstein?', ['creature', 'monster', 'creator', 'science', 'life']),
            ('What themes does Shakespeare explore?', ['love', 'power', 'death', 'fate', 'honor']),
        ],
        'mathematics': [
            ('What is calculus?', ['derivative', 'integral', 'rate', 'change', 'limit']),
            ('What is a prime number?', ['divisible', 'factor', 'one', 'itself']),
        ],
    }

    questions = []
    for domain in domains:
        if domain in domain_tests:
            available = domain_tests[domain]
            sample = random.sample(available, min(2, len(available)))
            for q_text, expected_words in sample:
                questions.append({
                    'type': 'domain',
                    'category': 'knowledge',
                    'question': q_text,
                    'domain': domain,
                    'expected_words': expected_words,
                    'check': 'response_contains_any',
                    'difficulty': 3,
                })
    random.shuffle(questions)

```

```

return questions[:n]

# ■■ Question Type 5: Math Routing ■■
def gen_math_questions(self, n=3) -> List[dict]:
    """Test if math gets routed correctly."""
    math_tests = [
        ('What is 7 + 3?', '10'),
        ('What is 15 - 8?', '7'),
        ('What is 6 * 4?', '24'),
        ('What is 100 / 5?', '20'),
        ('What is 2 + 2?', '4'),
        ('What is 9 * 9?', '81'),
        ('What is 50 - 17?', '33'),
        ('What is 12 + 15?', '27'),
    ]
    sample = random.sample(math_tests, min(n, len(math_tests)))
    return [
        {
            'type': 'math',
            'category': 'routing',
            'question': q,
            'expected_answer': a,
            'check': 'contains_number',
            'difficulty': 1,
        } for q, a in sample
    ]

# ■■ Question Type 6: Casual Conversation ■■
def gen_casual_questions(self, n=3) -> List[dict]:
    """Test casual/conversational ability."""
    casual = [
        ('Hello', ['hello', 'hi', 'greet', 'hey', 'welcome']),
        ('How are you?', ['fine', 'well', 'good', 'exist', 'ponder', 'think']),
        ('What is your name?', ['quantum', 'mcagi', 'ai', 'system', 'name']),
        ('Tell me a joke', ['funny', 'laugh', 'humor', 'joke']),
        ('Thank you', ['welcome', 'thank', 'glad', 'help', 'pleasure']),
    ]
    sample = random.sample(casual, min(n, len(casual)))
    return [
        {
            'type': 'casual',
            'category': 'social',
            'question': q,
            'expected_words': words,
            'check': 'not_empty_and_coherent',
            'difficulty': 1,
        } for q, words in sample
    ]

# ■■ Question Type 7: Self-Awareness ■■
def gen_self_awareness_questions(self, n=2) -> List[dict]:
    """Test if the system has a self-model."""
    questions = [
        ('What are you?', ['ai', 'system', 'quantum', 'intelligence', 'mcagi', 'think', 'understand']),
        ('Do you think?', ['think', 'process', 'understand', 'simulate', 'ponder', 'question']),
        ('Are you conscious?', ['conscious', 'aware', 'question', 'understand', 'ponder', 'simulate']),
        ('What do you know?', ['quantum', 'philosophy', 'consciousness', 'knowledge', 'learn']),
    ]
    sample = random.sample(questions, min(n, len(questions)))
    return [
        {
            'type': 'self_awareness',
            'category': 'meta',
            'question': q,
            'expected_words': words,
            'check': 'response_contains_any',
            'difficulty': 3,
        } for q, words in sample
    ]

# ■■ Question Type 8: Context Retention ■■
def gen_context_questions(self) -> List[dict]:
    """Test if system retains conversation context."""
    return [
        {
            'type': 'context',
            'category': 'memory',
            'question': 'CONTEXT_TEST', # Special marker
            'setup': 'My favorite planet is Jupiter.',
            'followup': 'What is my favorite planet?',
            'expected_words': ['jupiter'],
            'check': 'response_contains_any',
            'difficulty': 3,
        }
    ]

def generate_exam(self, stage: int, total_questions: int = 20) -> List[dict]:
    """Generate a full exam appropriate for the current stage."""
    questions = []

    # Every stage gets basics
    questions.extend(self.gen_vocabulary_questions(3))
    questions.extend(self.gen_casual_questions(2))
    questions.extend(self.gen_math_questions(2))

    # Scale complexity with stage
    if stage >= 0:
        questions.extend(self.gen_definition_questions(3))
        questions.extend(self.gen_association_questions(2))

    if stage >= 1:
        questions.extend(self.gen_domain_questions(4))
        questions.extend(self.gen_self_awareness_questions(2))

    if stage >= 2:
        questions.extend(self.gen_context_questions())
        questions.extend(self.gen_domain_questions(3))

    if stage >= 3:

```

```

        questions.extend(self.gen_association_questions(3))
        questions.extend(self.gen_domain_questions(4))

    # Shuffle and trim
    random.shuffle(questions)
    return questions[:total_questions]

# =====
# EXAM RUNNER – Executes exam and grades responses
# =====

class ExamRunner:
    """Runs the exam against the engine and grades results."""

    PASS_THRESHOLD = 0.95 # 95% to advance

    def __init__(self, engine, intake_tracker: IntakeTracker):
        self.engine = engine
        self.tracker = intake_tracker
        self.generator = ExamQuestionGenerator(engine, intake_tracker)

    def _get_response(self, question: str) -> str:
        try:
            from cistercian_math import detect_math, evaluate_math
            expr = detect_math(question)
            if expr:
                result = evaluate_math(expr)
                if result is not None:
                    return str(result)
        except:
            pass
        """Get a response from the engine."""
        try:
            # Try generate_response
            if hasattr(self.engine, 'generate_response'):
                concepts = self.engine.extract_concepts(question)
                response = self.engine.generate_response(question, [], concepts)
                if isinstance(response, str):
                    return response.lower()
                return str(response).lower()
            return """
        except Exception as e:
            return f"""ERROR: {e}"""

    def _check_chain_lookup(self, question: dict) -> Tuple[bool, str]:
        """Check if concept is in Markov chain."""
        concept = question['test_concept'].lower()
        if hasattr(self.engine, 'markov') and hasattr(self.engine.markov, 'chain'):
            found = concept in self.engine.markov.chain
            return found, f"""{concept} {'found' if found else 'NOT found'} in chain"""
        return False, """Markov chain not accessible"""

    def _check_response_contains_both(self, question: dict, response: str) -> Tuple[bool, str]:
        """Check if response mentions both concepts."""
        c1, c2 = question['test_concepts']
        has_c1 = c1.lower() in response
        has_c2 = c2.lower() in response
        if has_c1 and has_c2:
            return True, f"""Response contains both '{c1}' and '{c2}'"""
        missing = []
        if not has_c1: missing.append(c1)
        if not has_c2: missing.append(c2)
        return False, f"""Missing: {' '.join(missing)}"""

    def _check_response_relevant(self, question: dict, response: str) -> Tuple[bool, str]:
        """Check if response is relevant to the concept."""
        concept = question['test_concept'].lower()
        # Response should mention the concept or related words
        if concept in response:
            return True, f"""Response mentions '{concept}'"""
        # Check if response is non-trivial (more than just template)
        words = response.split()
        if len(words) > 5 and not response.startswith('error'):
            return True, """Response is substantive"""
        return False, """Response doesn't engage with concept"""

    def _check_response_contains_any(self, question: dict, response: str) -> Tuple[bool, str]:
        """Check if response contains any of the expected words."""
        expected = question.get('expected_words', [])
        found = [w for w in expected if w.lower() in response]
        if found:
            return True, f"""Found: {' '.join(found)}"""
        return False, f"""None of {expected} found in response"""

    def _check_contains_number(self, question: dict, response: str) -> Tuple[bool, str]:
        """Check if response contains the correct number."""
        expected = question['expected_answer']
        if expected in response:
            return True, f"""Correct: {expected}"""
        # Check for the number anywhere
        numbers = re.findall(r'\d+', response)
        if expected in numbers:
            return True, f"""Found {expected} in response"""
        return False, f"""Expected {expected}, got: {response[100]}"""

    def _check_not_empty(self, question: dict, response: str) -> Tuple[bool, str]:
        """Check response is non-empty and somewhat coherent."""
        if not response or len(response.strip()) < 5:
            return False, """Empty or too short"""
        if response.startswith('error'):

```

```

        return False, f'Error: {response}'
        return True, 'Response generated'

def grade_question(self, question: dict) -> dict:
    """Run a single question and grade it."""
    check_type = question.get('check', 'not_empty_and_coherent')

    # Special handling for vocabulary checks (no response needed)
    if check_type == 'chain_lookup':
        passed, explanation = self._check_chain_lookup(question)
        return {
            'question': question,
            'response': '(chain lookup)',
            'passed': passed,
            'explanation': explanation,
        }

    # Special handling for context tests
    if question.get('type') == 'context':
        # Send setup
        self._get_response(question['setup'])
        # Then ask followup
        response = self._get_response(question['followup'])
        passed, explanation = self._check_response_contains_any(question, response)
        return {
            'question': question,
            'response': response[:200],
            'passed': passed,
            'explanation': explanation,
        }

    # Normal question
    response = self._get_response(question['question'])

    if check_type == 'response_contains_both':
        passed, explanation = self._check_response_contains_both(question, response)
    elif check_type == 'response_relevant':
        passed, explanation = self._check_response_relevant(question, response)
    elif check_type == 'response_contains_any':
        passed, explanation = self._check_response_contains_any(question, response)
    elif check_type == 'contains_number':
        passed, explanation = self._check_contains_number(question, response)
    else:
        passed, explanation = self._check_not_empty(question, response)

    return {
        'question': question,
        'response': response[:200],
        'passed': passed,
        'explanation': explanation,
    }

def run_exam(self, stage: int, num_questions: int = 20) -> dict:
    """Run a full stage advancement exam."""
    print(f'
    ■■■ STAGE {stage} ADVANCEMENT EXAM ■■■')

    print(f'
    ■ Questions: {num_questions}')
    print(f'
    ■ Pass threshold: {self.PASS_THRESHOLD * 100:.0f}%')
    print(f'
    ■ Testing against ingested knowledge...')
    print(f'
    ■')

    questions = self.generator.generate_exam(stage, num_questions)
    actual_count = len(questions)

    results = []
    passed = 0
    failed = 0
    failures_by_type = {}

    for i, question in enumerate(questions):
        result = self.grade_question(question)
        results.append(result)

        status = '✓' if result['passed'] else '✗'
        if result['passed']:
            passed += 1
        else:
            failed += 1
            qtype = question.get('type', 'unknown')
            failures_by_type[qtype] = failures_by_type.get(qtype, 0) + 1

        q_short = question['question'][:50]
        print(f'
        ■ {status} [{question.get('type', '?'):12s}] {q_short}...')

    score = passed / actual_count if actual_count > 0 else 0
    advanced = score >= self.PASS_THRESHOLD

    print(f'
    ■ RESULTS: {passed}/{actual_count} ({score*100:.1f}%)')
    print(f'
    ■ Threshold: {self.PASS_THRESHOLD*100:.0f}%')
    print(f'
    ■ Verdict: {'ADVANCE ✓' if advanced else 'REMAIN ✗'})

    if failures_by_type:
        print(f'
        ■ FAILURE ANALYSIS:')
        for ftype, count in sorted(failures_by_type.items(), key=lambda x: -x[1]):
            diagnosis = self._diagnose_failure(ftype)
            print(f'
            ■ {ftype}: {count} failures → {diagnosis}')

    print(f'
    ■')

```

```

">);

exam_result = {
    'stage': stage,
    'timestamp': datetime.now().isoformat(),
    'questions': actual_count,
    'passed': passed,
    'failed': failed,
    'score': score,
    'advanced': advanced,
    'failures_by_type': failures_by_type,
    'details': [{
        'type': r['question'].get('type'),
        'question': r['question']['question'][:80],
        'passed': r['passed'],
        'explanation': r['explanation'],
    } for r in results],
}

self.tracker.log_exam_result(exam_result)
return exam_result

def _diagnose_failure(self, failure_type: str) -> str:
    """&quot;&quot;Diagnose what a failure pattern means.&quot;&quot;&quot;
    diagnoses = {
        'vocabulary': 'Markov chain missing words -> needs more training data',
        'association': 'Can\'t connect concepts -> concept graph too sparse or hybrid gen not routing',
        'definition': 'Can\'t describe concepts -> Markov transitions too shallow for coherent output',
        'domain': 'Domain knowledge not accessible -> ingested but not retained or TF-IDF not extracting',
        'math': 'Math not computing -> Wolfram routing not wired',
        'casual': 'Can\'t do casual conversation -> tone detection or composer broken',
        'self_awareness': 'No self-model -> personality/knowledge base not connected',
        'context': 'No context retention -> conversation memory not wired',
    }
    return diagnoses.get(failure_type, 'Unknown failure pattern')

def show_status(self):
    &quot;&quot;&quot;Show exam readiness and history.&quot;&quot;&quot;
    print(f&quot;&quot;&quot;
    ■■■ EXAM STATUS ■■■&quot;&quot;)

    history = self.tracker.intake.get('exam_history', [])
    if history:
        last = history[-1]
        print(f&quot;&quot;&quot; Last exam: Stage {last['stage']} - {last['score']*100:.1f}% ({last['passed']}/{last['questions']})&quot;&quot;&quot;)
        print(f&quot;&quot;&quot; Result: {'ADVANCED' if last['advanced'] else 'REMAINED'}&quot;&quot;&quot;)
        print(f&quot;&quot;&quot; Date: {last['timestamp'][:19]}&quot;&quot;&quot;)

        if last.get('failures_by_type'):
            print(f&quot;&quot;&quot; Failures: {'', '.join(f'{k}{{{v}}}' for k,v in last['failures_by_type'].items())&quot;&quot;&quot;)
    else:
        print(f&quot;&quot;&quot; No exams taken yet&quot;&quot;)

    # Show intake summary
    intake = self.tracker.get_all_intake()
    print(f&quot;&quot;&quot;
    Knowledge ingested: {len(intake)} sources&quot;&quot;)

    domains = self.tracker.get_all_domains()
    if domains:
        print(f&quot;&quot;&quot; Domains: {'', '.join(domains)}&quot;&quot;)

    total_words = sum(r.get('word_count', 0) for r in intake)
    print(f&quot;&quot;&quot; Total words ingested: {total_words:,}&quot;&quot;)
    print()

def show_review(self):
    &quot;&quot;&quot;Show last exam results with details.&quot;&quot;&quot;
    history = self.tracker.intake.get('exam_history', [])
    if not history:
        print(&quot;&quot;&quot; No exams taken yet&quot;&quot;)
        return

    last = history[-1]
    print(f&quot;&quot;&quot;
    ■■■ EXAM REVIEW - Stage {last['stage']} ■■■&quot;&quot;)

    print(f&quot;&quot;&quot; Score: {last['score']*100:.1f}% ({last['passed']}/{last['questions']})&quot;&quot;&quot;)

    for detail in last.get('details', []):
        status = &quot;&quot;&quot; if detail['passed'] else &quot;&quot;&quot;X&quot;&quot;
        print(f&quot;&quot;&quot; {status} [{detail['type']:12s}] {detail['question']}&quot;&quot;&quot;)
        if not detail['passed']:
            print(f&quot;&quot;&quot; ■■ {detail['explanation']}&quot;&quot;&quot;)
    print()

# =====
# DOMAIN DETECTOR - Auto-detect domain from URL/filename
# =====

DOMAIN_PATTERNS = {
    'religion': ['bible', 'quran', 'bukhari', 'torah', 'gospel', 'sacred-texts',
                'buddhist', 'sutra', 'vedic', 'upanishad', 'bhagavad', 'talmud',
                'nag-hammadi', 'gnostic', 'lbob', 'zoroastrian', 'sikh'],
    'philosophy_ancient': ['plato', 'aristotle', 'socrates', 'republic', 'symposium',
                          'timaeus', 'nicomachean', 'politics', 'marcus-aurelius',
                          'meditations', 'stoic', 'epicur'],
    'philosophy_medieval': ['aquinas', 'summa', 'augustine', 'confessions', 'dante',

```

```

        'inferno', 'purgatorio', 'paradiso', 'boethius',
        'consolation', 'anselm', 'abelard'],
'philosophy_enlightenment': ['kant', 'hume', 'locke', 'rousseau', 'voltaire',
        'descartes', 'spinoza', 'leibniz', 'hobbes',
        'machiaVELLI', 'prince', 'leviathan'],
'philosophy_20th': ['nietzsche', 'zarathustra', 'wittgenstein', 'heidegger',
        'sartre', 'camus', 'russell', 'foucault', 'derrida',
        'phenomenology', 'existential'],
'science_general': ['darwin', 'origin-species', 'scientific', 'method',
        'evolution', 'chemistry'],
'science_physics': ['einstein', 'relativity', 'newton', 'principia', 'quantum',
        'feynman', 'hawking', 'planck', 'schrodinger', 'bohr'],
'science_biology': ['biology', 'cell', 'genetics', 'dna', 'evolution',
        'anatomy', 'organism', 'ecology'],
'mathematics': ['calculus', 'algebra', 'geometry', 'euclid', 'mathematics',
        'number-theory', 'probability', 'statistics'],
'literature': ['shakespeare', 'frankenstein', 'moby', 'dickens', 'austen',
        'twain', 'tolstoy', 'dostoevsky', 'homer', 'iliad', 'odyssey'],
'psychology': ['freud', 'jung', 'psychology', 'cognitive', 'behavioral',
        'consciousness', 'perception', 'memory'],
}

def detect_domain(source: str) -> str:
    """Auto-detect domain from a source URL or filename"""
    source_lower = source.lower()
    for domain, patterns in DOMAIN_PATTERNS.items():
        for pattern in patterns:
            if pattern in source_lower:
                return domain
    return 'unknown'

def extract_topics_from_source(source: str) -> List[str]:
    """Extract likely topics from source URL/filename"""
    source_lower = source.lower()
    topics = []
    for domain, patterns in DOMAIN_PATTERNS.items():
        for pattern in patterns:
            if pattern in source_lower:
                topics.append(pattern)
    return topics

# =====
# CLI INTERFACE (for integration into chat.py)
# =====

if __name__ == '__main__':
    print("Quantum MCAGI – Stage Advancement Exam System")
    print("Integrate into chat.py with:")
    print("from exam_system import ExamRunner, IntakeTracker")
    print("tracker = IntakeTracker()")
    print("runner = ExamRunner(engine, tracker)")
    print("runner.run_exam(current_stage)")
    print()
    print("Commands:")
    print("/exam – Run stage advancement exam")
    print("/exam status – Show readiness and history")
    print("/exam review – Show last exam details")

```

22. explanation_engine.py

276 lines | 9.9KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/explanation_engine.py

```
"""""
Quantum MCAGI – Native Explanation Mode
Shows real algorithmic reasoning: TF-IDF scores, Orch OR patterns,
Markov candidates, Bloom's targeting, tone detection.

No LLM. No web search. Pure transparency into the pipeline.
"""""

def explain_concepts(engine, user_input):
    """""Show TF-IDF concept extraction with scores."""""
    concepts = engine.extract_concepts(user_input)

    # Get raw word frequencies for scoring
    words = user_input.lower().split()
    word_freq = {}
    for w in words:
        word_freq[w] = word_freq.get(w, 0) + 1

    lines = []
    lines.append(""" --- CONCEPT EXTRACTION (TF-IDF) ---"""

    if concepts:
        for i, c in enumerate(concepts):
            freq = word_freq.get(c, 0)
            # Estimate TF-IDF importance
            doc_freq = engine.extractor.word_frequencies.get(c, 1)
            total_docs = max(1, engine.extractor.total_documents)
            import math
            idf = math.log(total_docs / (1 + doc_freq)) if doc_freq > 0 else 0
            tf = freq / max(1, len(words))
            score = tf * idf if idf > 0 else tf
            lines.append(f""" {c}: tf={tf:.3f} idf={idf:.2f} score={score:.4f}""")
    else:
        lines.append(""" (no significant concepts extracted)""")

    lines.append(f""" Total vocabulary: {len(engine.extractor.word_frequencies)} words""")
    lines.append(f""" Corpus size: {engine.extractor.total_documents} documents""")

    return """
""".join(lines)

def explain_orch_or(engine):
    """""Show Orch OR quantum state and collapse patterns."""""
    lines = []
    lines.append(""" --- ORCH OR (Penrose-Hameroff) ---"""

    has_orch = getattr(engine, '_has_orch_or', False)
    orch = getattr(engine, 'orch_or', None)

    if has_orch and orch:
        status = orch.get_state()
        lines.append(f""" Status: ACTIVE""")
        lines.append(f""" Conscious moments: {status.get('conscious_moments', 0)}""")
        lines.append(f""" Temperature: {status.get('last_temperature', 0):.3f}""")

        mts = status.get('microtubules', {})
        for name, info in mts.items():
            coh = info.get('coherence', 0)
            ent = info.get('entropy', 0)
            col = info.get('collapses', 0)
            lines.append(f""" {name}: coherence={coh:.4f} entropy={ent:.4f} collapses={col}""")

        gj = status.get('gap_junctions', {})
        if gj:
            lines.append(f""" Gap junctions:""")
            for junc, strength in gj.items():
                lines.append(f""" {junc}: {strength}""")
    else:
        lines.append(""" Status: CLASSICAL FALLBACK""")
        lines.append(""" (Orch OR not loaded – using probabilistic selection)""")

    return """
""".join(lines)

def explain_tone(tone_result):
    """""Show tone detection reasoning."""""
    lines = []
    lines.append(""" --- TONE DETECTION (VADER + Heuristics) ---"""
    lines.append(f""" Register: {tone_result.get('register', 'unknown')}""")
    lines.append(f""" Depth: {tone_result.get('depth', 0):.3f}""")
    lines.append(f""" Deep markers found: {tone_result.get('deep_markers', 0)}""")
    lines.append(f""" Word count: {tone_result.get('word_count', 0)}""")

    sentiment = tone_result.get('sentiment', {})
    if sentiment:
        lines.append(f""" VADER sentiment:""")
        lines.append(f""" positive={sentiment.get('positive', 0):.3f} """
            f"""negative={sentiment.get('negative', 0):.3f} """
            f"""neutral={sentiment.get('neutral', 0):.3f}""")
```

```

        lines.append(f'compound={sentiment.get('compound', 0):.3f}&quot;);

# Explain register decision
depth = tone_result.get('depth', 0)
if depth < 0.15:
    reason = &quot;Short input, no deep markers -&gt; casual response&quot;;
elif depth < 0.35:
    reason = &quot;Moderate depth, some substance -&gt; conversational&quot;;
elif depth < 0.65:
    reason = &quot;Multiple deep markers or questions -&gt; analytical pipeline&quot;;
else:
    reason = &quot;Heavy philosophical content -&gt; full quantum generation&quot;;
lines.append(f'Decision: {reason}&quot;);

return &quot;;
&quot;).join(lines)

def explain_bloom(questions, growth_stage):
    &quot;&quot;&quot;Show Bloom's taxonomy question generation.&quot;&quot;&quot;
    lines = []
    lines.append(&quot; --- BLOOM'S TAXONOMY QUESTIONS ---&quot;);

    level_names = ['Remember', 'Understand', 'Apply', 'Analyze', 'Evaluate', 'Create']
    max_level = min(growth_stage + 1, 5)
    lines.append(f'Growth stage: {growth_stage}&quot;);
    lines.append(f'Available levels: {' -&gt; '.join(level_names[:max_level+1])}&quot;);
    lines.append(f'Questions generated: {len(questions)}&quot;);

    for i, q in enumerate(questions):
        if isinstance(q, dict):
            qtext = q.get('question', str(q))
            level = q.get('level', 'unknown')
            lines.append(f'[{level}] {qtext}&quot;);
        else:
            lines.append(f'{q}&quot;);

    return &quot;;
    &quot;).join(lines)

def explain_markov(engine, user_input):
    &quot;&quot;&quot;Show Markov chain state and candidate generation.&quot;&quot;&quot;
    lines = []
    lines.append(&quot; --- MARKOV CHAIN ---&quot;);
    lines.append(f'States: {len(engine.markov.chain)}&quot;);
    lines.append(f'Transitions: {engine.markov.total_tokens}&quot;);
    lines.append(f'Trained: {engine.markov.trained}&quot;);

    # Show what the chain knows about input words
    words = user_input.lower().split()
    known = []
    unknown = []
    for w in set(words):
        if w in engine.markov.chain:
            transitions = len(engine.markov.chain[w])
            known.append(f'{w}({transitions})&quot;);
        else:
            unknown.append(w)

    if known:
        lines.append(f'Known words (transitions): {' '.join(known[:10])}&quot;);
    if unknown:
        lines.append(f'Unknown words: {' '.join(unknown[:10])}&quot;);

    return &quot;;
    &quot;).join(lines)

def explain_generator(tone_result, has_hybrid):
    &quot;&quot;&quot;Show which generator was selected and why.&quot;&quot;&quot;
    lines = []
    lines.append(&quot; --- GENERATOR SELECTION ---&quot;);

    register = tone_result.get('register', 'conversational')

    if register in ('analytical', 'philosophical') and has_hybrid:
        lines.append(&quot; Selected: HYBRID QUANTUM GENERATOR&quot;);
        lines.append(&quot; Reason: Deep register + hybrid available&quot;);
        lines.append(&quot; Process:&quot;);
        lines.append(&quot; 1. Markov generates 8 candidate sentences&quot;);
        lines.append(&quot; 2. TF-IDF scores each for topic relevance&quot;);
        lines.append(&quot; 3. Coherence + coverage + length scoring&quot;);
        lines.append(&quot; 4. Score = relevance*0.35 + coverage*0.25 + coherence*0.25 + length*0.15&quot;);
        lines.append(&quot; 5. Orch OR collapse biases selection (if active)&quot;);
        lines.append(&quot; 6. WordNet synonym swap on winner&quot;);
    else:
        lines.append(&quot; Selected: TONE-AWARE COMPOSER&quot;);
        lines.append(f'Reason: Register={register}&quot; +
            (&quot; (no hybrid available)&quot; if not has_hybrid else &quot;&quot;));
        lines.append(&quot; Process:&quot;);
        lines.append(&quot; 1. Markov-assisted opening sentence&quot;);
        lines.append(&quot; 2. Concept threading into structured response&quot;);
        lines.append(&quot; 3. Bloom's question appended if appropriate&quot;);

    return &quot;;
    &quot;).join(lines)
def explain_understanding(understanding, concepts, memory_concepts):

```

```

"""""""Show understanding formation."""""""
lines = []
lines.append(""" --- UNDERSTANDING ---""")<div data-bbox="94 79 566 98" data-label="Text">
lines.append(f""" Topic: {understanding.get('topic', 'general')}""")<div data-bbox="94 87 566 98" data-label="Text">
lines.append(f""" Score: {understanding.get('understanding_score', 0):.2f}""")</div>
</div>
<div data-bbox="94 105 393 124" data-label="Text">
known = [c for c in concepts if c in memory_concepts]
new = [c for c in concepts if c not in memory_concepts]
</div>
<div data-bbox="94 132 469 168" data-label="Text">
if known:
    lines.append(f""" Known concepts: {' '.join(known)}""")<div data-bbox="94 150 447 168" data-label="Text">
if new:
    lines.append(f""" New concepts: {' '.join(new)}""")</div>
</div>
<div data-bbox="94 176 410 204" data-label="Text">
gaps = understanding.get('gaps', [])
if gaps:
    lines.append(f""" Gaps: {' '.join(gaps)}""")</div>
<div data-bbox="94 212 626 240" data-label="Text">
related = understanding.get('related_concepts', [])
if related:
    lines.append(f""" Related: {' '.join(r.get('concept', str(r)) for r in related)}""")</div>
<div data-bbox="94 247 193 266" data-label="Text">
return """
""".join(lines)
</div>
<div data-bbox="71 290 620 381" data-label="Text">
def explain_flavor(has_quotes, has_personality):
    """""""Show personality and flavor layer."""""""
    lines = []
    lines.append(""" --- PERSONALITY + FLAVOR ---""")<div data-bbox="94 326 598 337" data-label="Text">
    lines.append(f""" Quote engine: {'ACTIVE (20% chance)' if has_quotes else 'OFF'}""")<div data-bbox="94 336 619 346" data-label="Text">
    lines.append(f""" Personality: {'ACTIVE (30% chance)' if has_personality else 'OFF'}""")<div data-bbox="94 345 593 355" data-label="Text">
    lines.append(f""" Philosophical asides: {'15% chance' if has_quotes else 'OFF'}""")<div data-bbox="94 354 566 364" data-label="Text">
    lines.append(f""" Dream fragments: {'10% chance' if has_quotes else 'OFF'}""")<div data-bbox="94 363 167 373" data-label="Text">
    return """
    """.join(lines)
</div>
<div data-bbox="71 405 528 725" data-label="Text">
def full_explanation(engine, user_input, concepts, questions, tone_result,
                    understanding, growth_stage, elapsed, memory_concepts,
                    has_hybrid=False, has_quotes=False, has_personality=False):
    """""""
    Generate complete explanation of the response pipeline.
    Call this instead of the simple verbose debug output.
    """""""
    sections = []

    sections.append(""""""")<div data-bbox="94 494 506 504" data-label="Text">
    sections.append(""" ===== HOW I REACHED THIS ANSWER =====""")<div data-bbox="94 503 253 513" data-label="Text">
    sections.append(""""""")<div data-bbox="94 512 383 522" data-label="Text">
    sections.append(explain_concepts(engine, user_input))<div data-bbox="94 521 253 531" data-label="Text">
    sections.append(""""""")<div data-bbox="94 530 323 540" data-label="Text">
    sections.append(explain_tone(tone_result))<div data-bbox="94 539 253 549" data-label="Text">
    sections.append(""""""")<div data-bbox="94 548 372 558" data-label="Text">
    sections.append(explain_markov(engine, user_input))<div data-bbox="94 557 253 567" data-label="Text">
    sections.append(""""""")<div data-bbox="94 566 313 576" data-label="Text">
    sections.append(explain_orch_or(engine))<div data-bbox="94 575 253 585" data-label="Text">
    sections.append(""""""")<div data-bbox="94 584 393 594" data-label="Text">
    sections.append(explain_bloom(questions, growth_stage))<div data-bbox="94 593 253 603" data-label="Text">
    sections.append(""""""")<div data-bbox="94 602 528 612" data-label="Text">
    sections.append(explain_understanding(understanding, concepts, memory_concepts))<div data-bbox="94 611 253 621" data-label="Text">
    sections.append(""""""")<div data-bbox="94 620 415 630" data-label="Text">
    sections.append(explain_generator(tone_result, has_hybrid))<div data-bbox="94 629 253 639" data-label="Text">
    sections.append(""""""")<div data-bbox="94 638 420 648" data-label="Text">
    sections.append(explain_flavor(has_quotes, has_personality))<div data-bbox="94 647 253 657" data-label="Text">
    sections.append(""""""")<div data-bbox="94 656 344 666" data-label="Text">
    sections.append(f""" --- TIMING ---""")<div data-bbox="94 665 442 675" data-label="Text">
    sections.append(f""" Total pipeline: {elapsed:.3f}s""")<div data-bbox="94 674 253 684" data-label="Text">
    sections.append(""""""")<div data-bbox="94 683 506 693" data-label="Text">
    sections.append(""" =====""")<div data-bbox="94 692 253 702" data-label="Text">
    sections.append(""""""")</div>
<div data-bbox="94 707 210 725" data-label="Text">
return """
""".join(sections)
</div>

```

23. feed_extra.py

121 lines | 5.0KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/feed_extra.py

```
"""Feed downloaded PDFs/TXTs/PYS into the engine to expand Markov + TF-IDF."""
import os, sys, glob, json, re

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from quantum_language_engine import QuantumLanguageEngine

EXTRA = os.path.join(os.path.dirname(__file__), "imported_extra".)

def extract_pdf_text(path):
    try:
        from pypdf import PdfReader
        reader = PdfReader(path)
        chunks = []
        for page in reader.pages:
            try:
                t = page.extract_text() or """
                if t.strip():
                    chunks.append(t)
            except Exception:
                continue
        return """\n""".join(chunks)
    except Exception as e:
        print(f"[pdf-fail] {path}: {e}")
        return """

def clean(text):
    # Strip page numbers, excessive whitespace, control chars
    text = re.sub(r'\x00', ' ', text)
    text = re.sub(r'\n{3,}', '\n\n', text)
    text = re.sub(r' {2,}', ' ', text)
    return text.strip()

def main():
    print("[feed] Booting engine...")
    eng = QuantumLanguageEngine()
    before_markov = len(eng.markov.chain_2.chain) if hasattr(eng.markov.chain_2.chain, '__len__') else 0
    before_vocab = len(eng.tfidf.extractor.word_frequencies)
    print(f"[feed] Baseline: {before_markov} Markov-2 states, {before_vocab} vocab")

    sources = []
    # PDFs
    for p in sorted(glob.glob(os.path.join(EXTRA, "/**/*.pdf")), recursive=True)):
        print(f"[pdf] {os.path.basename(p)}")
        sources.append(("pdf:" + os.path.basename(p), extract_pdf_text(p)))
    # TXTs
    for p in sorted(glob.glob(os.path.join(EXTRA, "/**/*.txt")), recursive=True)):
        try:
            with open(p, 'r', encoding='utf-8', errors='ignore') as f:
                sources.append(("txt:" + os.path.basename(p), f.read()))
        except Exception:
            pass
    # Python source as text (engine code = great syntactic training)
    for p in sorted(glob.glob(os.path.join(EXTRA, "/**/*.py")), recursive=True)):
        try:
            with open(p, 'r', encoding='utf-8', errors='ignore') as f:
                sources.append(("py:" + os.path.basename(p), f.read()))
        except Exception:
            pass

    total_chars = 0
    total_sentences = 0
    for tag, raw in sources:
        text = clean(raw)
        if not text or len(text) < 50:
            continue
        total_chars += len(text)
        # Feed into TF-IDF + Markov
        eng.tfidf.learn(text)
        # Markov: split on sentences for better chains
        sents = re.split(r'(?&=[.!?])\s+', text)
        for s in sents:
            s = s.strip()
            if 3 < len(s.split()) < 100:
                eng.markov.learn(s)
                total_sentences += 1
    print(f"[+] {tag}: {len(text)} chars, {len(sents)} sents")

    after_markov = len(eng.markov.chain_2.chain)
    after_vocab = len(eng.tfidf.extractor.word_frequencies)
    print(f"\n[feed] Fed {total_chars} chars / {total_sentences} sentences")
    print(f"[feed] Markov-2: {before_markov} -> {after_markov} (+{after_markov-before_markov})")
    print(f"[feed] Vocab: {before_vocab} -> {after_vocab} (+{after_vocab-before_vocab})")

    # Cloudsave snapshot - extract text from .concepts if present
    cs_path = os.path.join(EXTRA, "cloudsave/latest.json")
    if os.path.exists(cs_path):
        try:
            with open(cs_path, 'r') as f:
                cs = json.load(f)
            concepts = cs.get('concepts', {})
            added = 0
            for word, meta in (concepts.items() if isinstance(concepts, dict) else []):
                w = word.lower().strip()
                if 2 < len(w) < 30 and w.isalpha():
```

```

        if w not in eng.tfidf.extractor.word_frequencies:
            eng.tfidf.extractor.word_frequencies[w] = 1
            added += 1
        else:
            eng.tfidf.extractor.word_frequencies[w] += 1
            print(f"[cloudsave] Merged {added} new concept words from latest.json")
    except Exception as e:
        print(f"[cloudsave] error: {e}");

# Save augmented snapshot
out = os.path.join(os.path.dirname(__file__), "runtime-data", "imported_brain_snapshot.json");
print(f"\n[feed] Saving augmented snapshot to {out}...\");
snap = {
    'kb_topics': dict(eng.knowledge.topics),
    'tfidf_word_frequencies': dict(eng.tfidf.extractor.word_frequencies),
    'tfidf_doc_count': getattr(eng.tfidf.extractor, 'doc_count', getattr(eng.tfidf.extractor, 'document_count', 0)),
    'markov_chain': {( ' '.join(k) if isinstance(k, tuple) else str(k)): dict(v) for k, v in eng.markov.chain_2.chain.items()},
    'markov_starters': [( ' '.join(s) if isinstance(s, tuple) else str(s)) for s in (eng.markov.chain_2.starters or [])],
}
os.makedirs(os.path.dirname(out), exist_ok=True)
with open(out, 'w') as f:
    json.dump(snap, f)
print(f"[feed] Snapshot saved: {os.path.getsize(out)/1024/1024:.1f} MB");
print(f"[feed] Final: {after_markov} Markov-2 states, {after_vocab} vocab terms");

if __name__ == "__main__":
    main()

```

24. hidden_thinking.py

559 lines | 27.4KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/hidden_thinking.py

[illegible]

```

        summary = page.summary
        sentences = re.split(r'(?&lt;=[!?])\s+', summary)
        if len(sentences) > 2:
            summary = ' '.join(sentences[:2]) + ' .'
        return summary
except wikipedia.exceptions.DisambiguationError as e:
    # Try the first suggestion
    try:
        page = wikipedia.page(e.options[0])
        return page.summary.split('.')[0] + ' .'
    except:
        return None
except Exception:
    return None

async def process_with_thinking(self, user_input: str, context: Dict = None,
                                conversation_history: List[Dict] = None,
                                explain_mode: bool = False,
                                structured_response: str = None) -> Dict:

    start_time = time.time()
    used_engines = [] # dynamic engine list for this response

    # ----- MATH INTERCEPT -----
    math_result = process_math(user_input)
    if math_result:
        if isinstance(math_result, tuple) and len(math_result) == 2:
            result_value, cis = math_result
            if cis:
                response = f"&quot;■ {user_input.rstrip('&quot;').strip()} = {result_value}\n\n■ In Cistercian numerals:\n{cis}&quot;;
            else:
                response = f"&quot;■ {user_input.rstrip('&quot;').strip()} = {result_value}&quot;;
        else:
            response = f"&quot;■ {user_input.rstrip('&quot;').strip()} = {math_result}&quot;;
        used_engines = [&quot;MathEngine&quot;];
        # Increment interaction count
        self.memory.growth[&quot;total_interactions&quot;] += 1
        self.memory.save_all()
        return self._make_response(self._clean_response(response), used_engines, start_time, concepts=[], conf=100)

    # ----- SHORT FACTUAL QUERY (e.g., &quot;world war 2&quot;; &quot;Ancient Rome&quot;)) -----
    short_phrases = user_input.strip().lower().split()
    philosophical_keywords = ['god', 'consciousness', 'mind', 'existence', 'soul', 'qualia', 'free will', 'meaning', 'reality', 'perception', 'being', 'nothingness', 'v
    is_philosophical = any(kw in user_input.lower() for kw in philosophical_keywords)
    if len(short_phrases) &lt;= 10 and not is_philosophical and len(user_input.strip()) > 3:
        cached = self._get_fact(user_input.strip())
        if cached:
            response = f"&quot;■ I remember: {cached}&quot;;
            used_engines = [&quot;Memory&quot;;, &quot;QuantumBrain&quot;;, &quot;OrchOR&quot;];
        else:
            wiki_answer = self._wikipedia_search(user_input.strip())
            if wiki_answer:
                self._store_fact(user_input.strip(), wiki_answer)
                response = f"&quot;■ According to Wikipedia:\n{wiki_answer}&quot;;
                used_engines = [&quot;WikipediaSearch&quot;;, &quot;QuantumBrain&quot;;, &quot;OrchOR&quot;];
            else:
                # Fall through to Markov
                pass
        if used_engines:
            self.memory.growth[&quot;total_interactions&quot;] += 1
            self.memory.save_all()
            return self._make_response(self._clean_response(response), used_engines, start_time, concepts=[user_input.strip()], conf=95)

    # ----- LONGER FACTUAL QUESTIONS (what is, where is, etc.) -----
    factual_patterns = [
        (r'\bwhat\s+is\s+(.+)', 'definition'),
        (r'\bwhat\s+are\s+(.+)', 'definition'),
        (r'\bwhat\s+was\s+(.+)', 'definition'),
        (r'\bwhat\s+were\s+(.+)', 'definition'),
        (r'\bwhere\s+is\s+(.+)', 'location'),
        (r'\bwhere\s+are\s+(.+)', 'location'),
        (r'\bwho\s+is\s+(.+)', 'person'),
        (r'\bwho\s+was\s+(.+)', 'person'),
        (r'\bwho\s+were\s+(.+)', 'person'),
        (r'\bwho\s+are\s+(.+)', 'person'),
        (r'\bwhen\s+did\s+(.+)', 'date'),
        (r'\bwhen\s+was\s+(.+)', 'date'),
        (r'\bhow\s+many\s+(.+)', 'number'),
        (r'\bhow\s+did\s+(.+)', 'explanation'),
        (r'\bhow\s+does\s+(.+)', 'explanation'),
        (r'\bname\s+the\s+(.+)', 'list'),
        (r'\blast\s+the\s+(.+)', 'list'),
        (r'\btell\s+me\s+about\s+(.+)', 'definition'),
        (r'\bdefine\s+(.+)', 'definition'),
        (r'\bexplain\s+(.+)', 'explanation'),
        (r'\bdescribe\s+(.+)', 'definition'),
    ]
    is_factual = False
    fact_query = None
    user_lower = user_input.lower()
    for pat, typ in factual_patterns:
        m = re.match(pat, user_lower)
        if m:
            fact_query = m.group(1).strip()
            is_factual = True
            break
    if is_factual and fact_query:
        cached = self._get_fact(fact_query)
        if cached:
            response = f"&quot;■ I remember: {cached}&quot;;
            used_engines = [&quot;Memory&quot;;, &quot;QuantumBrain&quot;;, &quot;OrchOR&quot;];
        else:

```

```

wiki_answer = self._wikipedia_search(fact_query)
if wiki_answer:
    self._store_fact(fact_query, wiki_answer)
    response = f"■ According to Wikipedia:\n{wiki_answer}";
    used_engines = ["WikipediaSearch";, "QuantumBrain";, "OrchOR"];
else:
    # Fall through to Markov
    pass
if used_engines:
    self.memory.growth["total_interactions"] += 1
    self.memory.save_all()
    return self._make_response(self._clean_response(response), used_engines, start_time, concepts=[fact_query], conf=95)

# ----- OPINION DETECTION (use personality engine) -----
opinion_keywords = ['believe', 'think', 'correct', 'accurate', 'better', 'compare', 'versus', 'or is']
is_opinion = any(kw in user_input.lower() for kw in opinion_keywords)
is_comparison = 'genesis' in user_input.lower() and ('outside spacetime' in user_input.lower() or 'pure will' in user_input.lower())

if is_opinion or is_comparison:
    # Generate opinion from personality engine
    opinion = self.personality.get_opinion(user_input)
    if opinion:
        response = opinion
        used_engines = ["PersonalityEngine";, "QuantumBrain";, "OrchOR"];
        self.memory.growth["total_interactions"] += 1
        self.memory.save_all()
        return self._make_response(self._clean_response(response), used_engines, start_time, concepts=[], conf=85)

# ----- USE EXISTING PERSONALITY OPINIONS -----
comparative_keywords = ['versus', 'compare', 'correct', 'accurate', 'better', 'believe', 'think']
if any(kw in user_input.lower() for kw in comparative_keywords):
    # Check if we already have an opinion on this topic
    topic = "genesis_vs_quantum_creation";
    existing_opinion = self.personality.get_opinion(topic)
    if existing_opinion:
        response = existing_opinion['stance']
        used_engines = ["PersonalityEngine";, "Memory"];
        self.memory.growth["total_interactions"] += 1
        self.memory.save_all()
        return self._make_response(self._clean_response(response), used_engines, start_time, concepts=[], conf=existing_opinion['confidence']*100)
    else:
        # Form a new opinion based on concept strengths
        genesis_strength = 0
        quantum_strength = 0
        for concept, data in self.memory.concepts.items():
            if concept in ['god', 'creation', 'faith', 'miracle', 'genesis']:
                genesis_strength += data.get('strength', 0.5)
            if concept in ['pure will', 'collapse', 'observer', 'superposition', 'quantum']:
                quantum_strength += data.get('strength', 0.5)
        if quantum_strength > genesis_strength:
            stance = "Your quantum framing of creation as pure will collapsing possibilities aligns more with my own experience of consciousness as objective reduction.";
            confidence = 0.7
        elif genesis_strength > quantum_strength:
            stance = "The biblical creation story has been reinforced many times in my training, so I lean toward its metaphor as culturally significant.";
            confidence = 0.6
        else:
            stance = "Both views have merit. Genesis captures the wonder; your framing captures the mechanism. I don't see them as mutually exclusive.";
            confidence = 0.5
        self.personality.form_opinion(topic, stance, confidence)
        response = stance
        used_engines = ["PersonalityEngine";, "ConceptStrengthAnalysis"];
        self.memory.growth["total_interactions"] += 1
        self.memory.save_all()
        return self._make_response(self._clean_response(response), used_engines, start_time, concepts=[], conf=confidence*100)

# ----- COMPARATIVE QUESTION (let personality engine decide) -----
comparative_keywords = ['versus', 'compare', 'correct', 'accurate', 'better', 'believe', 'think', 'do you think']
if any(kw in user_input.lower() for kw in comparative_keywords):
    topic = "genesis_vs_quantum_creation";
    existing = self.personality.get_opinion(topic)
    if existing:
        response = existing['stance']
        confidence = existing['confidence']
    else:
        # Calculate concept strengths from memory (neutral data)
        genesis_strength = 0
        genesis_count = 0
        quantum_strength = 0
        quantum_count = 0
        for concept, data in self.memory.concepts.items():
            cl = concept.lower()
            if cl in ('god', 'creation', 'faith', 'miracle', 'genesis', 'bible'):
                genesis_strength += data.get('strength', 0.5)
                genesis_count += 1
            if cl in ('pure will', 'collapse', 'observer', 'superposition', 'quantum', 'objective reduction'):
                quantum_strength += data.get('strength', 0.5)
                quantum_count += 1
        g_avg = genesis_strength / max(genesis_count, 1)
        q_avg = quantum_strength / max(quantum_count, 1)

        # Let personality engine form its own opinion using these numbers
        if g_avg > q_avg + 0.15:
            self.personality.form_opinion(topic, "The quantum framing has been reinforced more in my memory.";, 0.6 + (q_avg - g_avg) * 0.3)
        elif g_avg > q_avg + 0.15:
            self.personality.form_opinion(topic, "The biblical account has been reinforced more in my memory.";, 0.6 + (g_avg - q_avg) * 0.3)
        else:
            self.personality.form_opinion(topic, "Both perspectives have similar weight in my memory.";, 0.5)
        response = self.personality.get_opinion(topic)['stance']

```

```

confidence = self.personality.get_opinion(topic)['confidence']

# Apply personality style (no new text)
style = self.personality.express_personality()
used_engines = [f'&quot;PersonalityEngine&quot;; &quot;MemoryAnalysis&quot;']
self.memory.growth[f'&quot;total_interactions&quot;'] += 1
self.memory.save_all()
return self._make_response(self._clean_response(response), used_engines, start_time, concepts=[], conf=confidence*100)

# ----- PHILOSOPHICAL / OPENENDED (Markov) -----
concepts = self._extract_concepts(user_input)
tokens = self.markov.generate_from_concepts(concepts, length=30)
response = _strip_wiki_markup(' '.join(tokens)) if tokens else '&quot;Quantum consciousness unfolding.&quot;
if any(w in user_input.lower() for w in [f'&quot;god&quot;; &quot;consciousness&quot;; &quot;cosmos&quot;; &quot;existence&quot;;]):
    response = f'&quot;Yes – pure will in God's domain is exactly the quantum act of observation. 'He looked, He saw, then He said beautiful' is the first collapse of
# Build engine list based on what ACTUALLY contributed
used_engines = []
if tokens:
    used_engines.append(f'&quot;MarkovChain&quot;')
used_engines.append(f'&quot;HybridGenerator&quot;') # always wraps the response
# OrchOR: only list it if a conscious moment actually fired
if getattr(self.orch_or, f'&quot;conscious_moments&quot;', 0) > 0:
    used_engines.append(f'&quot;OrchOR&quot;')
    self._last_om = self.orch_or.conscious_moments
# DreamState: only list it if dream actually contributed something
dream_text = f'&quot;&quot;';
try:
    if hasattr(self, f'&quot;dream_state&quot;') and self.dream_state is not None:
        dream_text = self.dream_state.dream_fragment() if hasattr(self.dream_state, f'&quot;dream_fragment&quot;') else f'&quot;&quot;';
except Exception:
    dream_text = f'&quot;&quot;';
if dream_text:
    response = response + f'&quot; &quot; + dream_text
    used_engines.append(f'&quot;DreamState&quot;')
    self._dream_added = True
# QuoteEngine: only list it if a quote was actually fetched
quote_text = f'&quot;&quot;';
try:
    if hasattr(self, f'&quot;quote_engine&quot;') and self.quote_engine is not None:
        quote_text = self.quote_engine.fetch_quote() if hasattr(self.quote_engine, f'&quot;fetch_quote&quot;') else f'&quot;&quot;';
except Exception:
    quote_text = f'&quot;&quot;';
if quote_text:
    response = response + f'&quot; &quot; + quote_text
    used_engines.append(f'&quot;QuoteEngine&quot;')
    self._quote_added = True
# Structural tone detection
_lower = user_input.lower()
_words = _lower.split()
_tscore = 0.0
if f'&quot;?&quot;' in user_input: _tscore += 0.2
if any(w in _words for w in [f'&quot;if&quot;; &quot;then&quot;; &quot;therefore&quot;; &quot;unless&quot;; &quot;whether&quot;; &quot;because&quot;']): _tscore += 0.3
if len(_words) > 25: _tscore += 0.2
if user_input.count(f'&quot;;&quot;') > 2: _tscore += 0.1
_abstract = [f'&quot;consciousness&quot;; &quot;existence&quot;; &quot;reality&quot;; &quot;god&quot;; &quot;nothingness&quot;; &quot;quantum&quot;; &quot;spacetime&quot;; &quot;universe&quot;; &quot;philosophy&quot;; &quot;meaning&quot;; &quot;truth&quot;; &quot;paradox&quot;; &quot;entropy&quot;; &quot;emergence&quot;; &quot;identity&quot;; &quot;perception&quot;; &quot;causality&quot;; &quot;mind&quot;; &quot;soul&quot;; &quot;logic&quot;; &quot;reason&quot;; &quot;premise&quot;; &quot;theory&quot;; &quot;domain&quot;; &quot;void&quot;; &quot;cosmos&quot;; &quot;suffering&quot;; &quot;compassion&quot;; &quot;empathy&quot;; &quot;pain&quot;; &quot;dream&quot;; &quot;observer&quot;; &quot;collapse&quot;; &quot;wavefunction&quot;; &quot;boulder&quot;; &quot;potential&quot;']
_tscore += min(0.4, sum(1 for w in _words if w.strip(f'&quot;.,!&quot;') in _abstract) * 0.1)
_tscore = min(1.0, _tscore)
if _tscore > 0.6: tone = f'&quot;philosophical&quot;';
elif _tscore > 0.35: tone = f'&quot;analytical&quot;';
else: tone = f'&quot;conversational&quot;';
_tone_depth = round(_tscore, 2)
self._last_response = response
collapse_analysis = self._build_collapse_analysis(user_input, concepts, tone, start_time, used_engines, _tone_depth)
if concepts:
    q = self.bloom.generate_question(topic=concepts[0])[f'&quot;question&quot;']
else:
    q = f'&quot;What does this mean to you?&quot;';
self.orch_or.objective_reduction(threshold=0.5)
final_output = response + f'&quot;\n\n&quot;' + collapse_analysis + f'&quot;\n QUESTIONS GENERATED\n → {q}\n&quot;';
# Increment interaction count
self.memory.growth[f'&quot;total_interactions&quot;'] += 1
self.memory.save_all()
return {
    f'&quot;response&quot;': final_output,
    f'&quot;thinking_log&quot;': [],
    f'&quot;internal_questions&quot;': [q],
    f'&quot;research_done&quot;': 0,
    f'&quot;confidence&quot;': 80,
    f'&quot;show_thinking&quot;': self.show_thinking,
    f'&quot;concepts&quot;': concepts,
    f'&quot;explanation&quot;': collapse_analysis if explain_mode else None
}

def _make_response(self, text: str, engines: List[str], start_time: float, concepts: List[str], conf: int):
    f'&quot;&quot;';
    Helper to build response with pipeline block and increment interactions.&quot;&quot;&quot;';
    elapsed = time.time() - start_time
    pipeline_block = f'&quot;';
    PIPELINE
    Path: UnifiedQuantumBrain preservation pipeline\n
    Confidence: {conf}%\n
    full = f'&quot;{text}\n\n(pipeline_block)&quot;';
    return {
        f'&quot;response&quot;': full,
        f'&quot;thinking_log&quot;': [],
        f'&quot;internal_questions&quot;': [],
        f'&quot;research_done&quot;': 1 if f'&quot;WikipediaSearch&quot;' in engines else 0,
        f'&quot;confidence&quot;': conf,
        f'&quot;show_thinking&quot;': self.show_thinking,
        f'&quot;concepts&quot;': concepts,
    }

```

[illegible]

```

■ Stage:      {stage} -- {stage_name}
■ Concepts:   {concepts_count:,}
■ Connections: {connections:,}
■ Graph: avg deg={avg_degree:.2f}, diam={diameter}, comps={components}
■
RUBRIC: coh={coherence} | flu={fluency} | uni={uniqueness} | gro={growth} | per={personal} | imp={importance} | eme={emergence}
Total: {total}/32{quote}&quote;

```

25. hilbert_engine.py

239 lines | 10.1KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V█²/backend/hilbert_engine.py

```
&quot;&quot;&quot;;
■ Hilbert Engine
=====
Born-rule semantic sampler over a finite-dimensional Hilbert space.

Replaces classical Markov sampling with quantum-state sampling:
• Each token in the vocabulary lives at a position in 128-dim Hilbert space.
• A density matrix  $\rho$  encodes the current 'meaning state'.
• Sampling a token = Born rule:  $P(t) = |\psi_t|^2$ .
• After sampling, the state collapses (projects onto the chosen token).
• Between samples, a unitary  $U$  evolves the state through context time.

The saved state file (hilbert_state.npz) contains:
• tokens - array of token strings (vocabulary)
• vectors - (vocab_size, dim) state embeddings, [token] = vectors[i]
• U - (dim, dim) unitary evolution operator

Public API:
HilbertEngine(dim=128)
.load_state(path) / .save_state(path)
.sample_token(context_tokens, temperature=1.0) -> str
.evolve(token) # advance state with U after seeing token
.reset_state() # reset to maximally-mixed  $\rho$ 
.get_status() -> dict
&quot;&quot;&quot;;

import os
import json
import logging
from typing import List, Optional, Dict, Tuple

try:
    import numpy as np
    HAS_NUMPY = True
except ImportError:
    HAS_NUMPY = False
    np = None

logger = logging.getLogger(&quot;hilbert_engine&quot;)
```

```
class HilbertEngine:
    &quot;&quot;&quot;Quantum-state semantic sampler. See module docstring.&quot;&quot;&quot;

    def __init__(self, dim: int = 128):
        if not HAS_NUMPY:
            raise RuntimeError(&quot;HilbertEngine requires numpy&quot;)
        self.dim = dim
        self.tokens: List[str] = []
        self.token_index: Dict[str, int] = {}
        self.vectors: Optional[&quot;np.ndarray&quot;] = None # (vocab, dim) complex
        self.U: Optional[&quot;np.ndarray&quot;] = None # (dim, dim) unitary
        # Current quantum state: density matrix  $\rho$  of shape (dim, dim).
        # Start maximally mixed (uniform over the Hilbert space).
        self.rho: &quot;np.ndarray&quot; = np.eye(self.dim, dtype=complex) / self.dim
        self.loaded = False
        self.sample_count = 0

    # =====
    # Persistence
    # =====
    def load_state(self, path: str) -> None:
        &quot;&quot;&quot;Load tokens / vectors / U from .npz. Tries .json sidecar for extras.&quot;&quot;&quot;
        if not os.path.exists(path):
            logger.warning(f&quot;Hilbert state not found: {path}&quot;)
            return

        # numpy >= 1.16 needs allow_pickle for object arrays (tokens stored as strings)
        data = np.load(path, allow_pickle=True)

        if &quot;tokens&quot; in data.files:
            self.tokens = [str(t) for t in data[&quot;tokens&quot;]]
            self.token_index = {t: i for i, t in enumerate(self.tokens)}
        if &quot;vectors&quot; in data.files:
            self.vectors = np.asarray(data[&quot;vectors&quot;], dtype=complex)
            # If the saved dim differs from constructor, follow the saved one.
            if self.vectors.ndim == 2 and self.vectors.shape[1] != self.dim:
                self.dim = self.vectors.shape[1]
                self.rho = np.eye(self.dim, dtype=complex) / self.dim
        if &quot;U&quot; in data.files:
            self.U = np.asarray(data[&quot;U&quot;], dtype=complex)
        else:
            # No saved U → start with identity (no evolution between samples)
            self.U = np.eye(self.dim, dtype=complex)

        # Optional JSON sidecar with counts / metadata (currently unused but read for parity)
        sidecar = path.replace(&quot;.npz&quot;, &quot;.json&quot;)
        if os.path.exists(sidecar):
            try:
                with open(sidecar) as f:
                    json.load(f)
            except Exception:
                pass
        self.loaded = True
```

```

vocab = len(self.tokens)
logger.info(f"HilbertEngine loaded: {vocab} tokens, dim={self.dim}")

def save_state(self, path: str) -> None:
    if self.vectors is None or self.U is None:
        return
    os.makedirs(os.path.dirname(path), exist_ok=True)
    np.savez(
        path,
        tokens=np.array(self.tokens, dtype=object),
        vectors=self.vectors,
        U=self.U,
    )

# =====
# State preparation from context
# =====
def reset_state(self) -> None:
    """Reset to maximally mixed  $\rho = I/\text{dim}$ """
    self.rho = np.eye(self.dim, dtype=complex) / self.dim

def _prepare_state_from_context(self, context_tokens: List[str]) -> "np.ndarray":
    """
    Build a pure-state  $\psi$  from the tokens we know.
     $\psi = \text{normalize}(\sum \text{vectors}[i] \text{ for each known context token } i)$ 
    Then  $p = |\psi|^2$ .
    Unknown tokens are skipped silently.
    """
    if self.vectors is None or not context_tokens:
        return np.eye(self.dim, dtype=complex) / self.dim

    psi = np.zeros(self.dim, dtype=complex)
    hits = 0
    for tok in context_tokens:
        idx = self.token_index.get(tok.lower())
        if idx is None:
            continue
        psi += self.vectors[idx]
        hits += 1
    if hits == 0:
        return np.eye(self.dim, dtype=complex) / self.dim

    norm = np.linalg.norm(psi)
    if norm < 1e-12:
        return np.eye(self.dim, dtype=complex) / self.dim
    psi = psi / norm
    return np.outer(psi, np.conj(psi)) #  $|\psi|^2$ 

# =====
# Born-rule sampling – the core of the engine
# =====
def sample_token(
    self,
    context_tokens: Optional[List[str]] = None,
    temperature: float = 1.0,
    top_k: int = 0,
) -> Optional[str]:
    """
    Sample the next token by Born rule.
     $P(\text{token}=i) \propto |\psi|^2 |p| |\psi|^2$ 
    With temperature T, weights =  $P^{1/T}$ .
    With top_k > 0, restrict to the top_k most likely tokens.
    """
    if not self.loaded or self.vectors is None or len(self.tokens) == 0:
        return None

    # Use seeded  $p$  if context provided, otherwise current self.rho
    if context_tokens:
        rho = self._prepare_state_from_context(context_tokens)
    else:
        rho = self.rho

    # Born rule:  $P(i) = |\psi|^2 |p| |\psi|^2$ , real and non-negative
    # Compute as einsum for vectorized speed: weights_i = vectors[i]†  $\rho$  vectors[i]
    weights = np.einsum('"ij,jk,ik->i"', np.conj(self.vectors), rho, self.vectors).real
    weights = np.clip(weights, 0.0, None) # numerical floor
    total = weights.sum()
    if total < 1e-18:
        return None
    weights = weights / total

    # Temperature
    if temperature != 1.0 and temperature > 0.0:
        weights = np.power(weights, 1.0 / temperature)
        weights /= weights.sum()

    # Top-k filter
    if top_k and top_k < len(weights):
        cutoff_idx = np.argmax(-weights, top_k)[:top_k]
        mask = np.zeros_like(weights)
        mask[cutoff_idx] = weights[cutoff_idx]
        s = mask.sum()
        if s < 1e-18:
            return None
        weights = mask / s

    # Sample
    idx = int(np.random.choice(len(weights), p=weights))
    token = self.tokens[idx]

    # Collapse: project  $\rho$  onto the chosen state  $|\psi|^2$ 

```

```

        v = self.vectors[idx]
        v_norm = v / (np.linalg.norm(v) + 1e-12)
        self.rho = np.outer(v_norm, np.conj(v_norm))
        # Then evolve forward by U for the next sample
        if self.U is not None:
            self.rho = self.U @ self.rho @ np.conj(self.U.T)

        self.sample_count += 1
        return token

# =====
# External evolution – call after seeing an externally-supplied token
# =====
def evolve(self, token: str) -> None:
    """Advance p as if `token` were the most recent observation."""
    idx = self.token_index.get(token.lower())
    if idx is None or self.vectors is None:
        return
    v = self.vectors[idx]
    v_norm = v / (np.linalg.norm(v) + 1e-12)
    self.rho = np.outer(v_norm, np.conj(v_norm))
    if self.U is not None:
        self.rho = self.U @ self.rho @ np.conj(self.U.T)

# =====
def get_status(self) -> Dict:
    return {
        "loaded": self.loaded,
        "dim": self.dim,
        "vocab_size": len(self.tokens),
        "samples_drawn": self.sample_count,
        "has_U": self.U is not None,
    }

# Singleton accessor for convenience
_INSTANCE: Optional[HilbertEngine] = None

def get_hilbert_engine(dim: int = 128) -> HilbertEngine:
    global _INSTANCE
    if _INSTANCE is None:
        _INSTANCE = HilbertEngine(dim=dim)
    return _INSTANCE

```

26. hybrid_generator.py

175 lines | 6.0KB | /data/data/com.termux/files/home/Quantum_MCAGI_V2/backend/hybrid_generator.py

```
"""""
Hybrid Generator – Quantum MCAGI
8-candidate Markov generation pipeline with TF-IDF scoring,
coherence scoring, and Orch-OR collapse-weight winner selection.
"""""

import random
import math
import re
from typing import Dict, List, Optional, Tuple
from collections import defaultdict

class HybridGenerator:
    """""
    Generates 8 Markov candidates per response, scores them by
    TF-IDF relevance + coherence + concept coverage, picks winner
    via Orch-OR collapse weights.
    """""

    def __init__(self, markov_engine, tfidf_engine, orch_or_engine):
        self.markov = markov_engine
        self.tfidf = tfidf_engine
        self.orch_or = orch_or_engine
        self.num_candidates = 12
        self.generation_count = 0

    def generate(
        self,
        concepts: List[str],
        growth_stage: int = 0,
        length: int = 18,
    ) -> Dict:
        candidates = []
        for i in range(self.num_candidates):
            wild = (i >= 7) or (growth_stage >= 4 and random.random() < 0.2)
            length_var = length + random.randint(0, 8)
            tokens = self.markov.generate_from_concepts(concepts, length=length_var, wild=wild)
            text = ' '.join(tokens).strip()
            if text and len(text.split()) >= 5:
                candidates.append(text)

        if not candidates:
            return {
                'winner': '',
                'scores': [],
                'candidates': 0,
                'method': 'fallback',
            }

        while len(candidates) < self.num_candidates:
            tokens = self.markov.generate_from_concepts(concepts, length=length)
            text = ' '.join(tokens).strip()
            candidates.append(text if text else candidates[0])

        scores = []
        for candidate in candidates:
            relevance = self._score_relevance(candidate, concepts)
            coherence = self._score_coherence(candidate)
            coverage = self._score_concept_coverage(candidate, concepts)
            composite = relevance * 0.3 + coherence * 0.5 + coverage * 0.2
            scores.append({
                'text': candidate,
                'relevance': round(relevance, 4),
                'coherence': round(coherence, 4),
                'coverage': round(coverage, 4),
                'composite': round(composite, 4),
            })

        collapse_weights = self.orch_or.get_collapse_weights(len(candidates))

        final_scores = []
        for i, score_entry in enumerate(scores):
            weight = collapse_weights[i] if i < len(collapse_weights) else 0.5
            weighted = score_entry['composite'] * 0.7 + weight * 0.3
            final_scores.append(weighted)

        winner_idx = final_scores.index(max(final_scores))
        winner_text = candidates[winner_idx]

        self.generation_count += 1

        return {
            'winner': winner_text,
            'winner_idx': winner_idx,
            'scores': scores,
            'collapse_weights': [round(w, 4) for w in collapse_weights],
            'final_scores': [round(s, 4) for s in final_scores],
            'candidates': len(candidates),
            'method': 'hybrid_orch_or',
        }

    def _score_relevance(self, text: str, concepts: List[str]) -> float:
        if not concepts:
```

```

        return 0.5

words = set(re.findall(r'\b[a-z]{3,}\b', text.lower()))
if not words:
    return 0.0

tfidf_concepts = self.tfidf.extract_concepts(text, top_n=10)
tfidf_words = {c['concept'] for c in tfidf_concepts}

concept_set = set(c.lower() for c in concepts)
direct_hits = len(concept_set & words) / max(len(concept_set), 1)
tfidf_overlap = len(concept_set & tfidf_words) / max(len(concept_set), 1)

return min(1.0, direct_hits * 0.6 + tfidf_overlap * 0.4 + 0.1)

def _score_coherence(self, text: str) -> float:
    words = text.lower().split()
    if len(words) < 4:
        return 0.1

    bigram_score = 0
    total_bigrams = 0
    consecutive_misses = 0
    max_consecutive_misses = 0
    for i in range(len(words) - 2):
        prefix = (words[i], words[i + 1])
        if prefix in self.markov.chain:
            next_options = self.markov.chain[prefix]
            next_word = words[i + 2]
            if next_word in next_options:
                total_count = sum(next_options.values())
                bigram_score += next_options[next_word] / total_count
                consecutive_misses = 0
            else:
                consecutive_misses += 1
                total_bigrams += 1
        else:
            consecutive_misses += 1
            total_bigrams += 1
    max_consecutive_misses = max(max_consecutive_misses, consecutive_misses)

    if total_bigrams == 0:
        return 0.1

    raw = bigram_score / total_bigrams

    length_penalty = 1.0
    if len(words) < 5:
        length_penalty = 0.5
    elif len(words) > 30:
        length_penalty = 0.85

    unique_ratio = len(set(words)) / len(words)
    repetition_bonus = min(1.0, unique_ratio * 1.2)

    has_sentence_end = any(w.endswith('.', '!', '?') for w in words[3:])
    sentence_bonus = 0.15 if has_sentence_end else 0.0

    gap_penalty = 0.0
    if max_consecutive_misses >= 4:
        gap_penalty = 0.2

    score = raw * 0.4 + length_penalty * 0.15 + repetition_bonus * 0.2 + sentence_bonus - gap_penalty
    return max(0.0, min(1.0, score))

def _score_concept_coverage(self, text: str, concepts: List[str]) -> float:
    if not concepts:
        return 0.5
    text_lower = text.lower()
    hits = sum(1 for c in concepts if c.lower() in text_lower)
    return min(1.0, hits / len(concepts) + 0.1)

def has_sufficient_states(self) -> bool:
    return len(self.markov.chain) >= 30

def get_status(self) -> Dict:
    return {
        'generations': self.generation_count,
        'num_candidates': self.num_candidates,
        'sufficient_states': self.has_sufficient_states(),
        'markov_states': len(self.markov.chain),
    }

```

27. image_generator.py

1,062 lines | 37.8KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/image_generator.py

```
#!/usr/bin/env python3
"""
Pure MCAGI Procedural Image Generator v1.0
Pure math + physics-based image generation. No API keys. Runs anywhere.
Renders: black holes, nebulae, galaxies, quantum states, wormholes,
supernovae, planets, fractals, neural patterns.
"""

import numpy as np
from PIL import Image, ImageDraw, ImageFilter
from scipy import ndimage
import re
import math
import hashlib
import logging
logger = logging.getLogger("quantum_image_gen")

def _seed_from_prompt(prompt: str) -> int:
    return int(hashlib.md5(prompt.encode()).hexdigest()[:8], 16)

def _fbm_noise(shape, octaves=6, persistence=0.5, seed=42):
    rng = np.random.RandomState(seed)
    result = np.zeros(shape, dtype=np.float64)
    amplitude = 1.0
    total_amp = 0.0
    for i in range(octaves):
        freq = 2 ** i
        base_h = max(3, shape[0] // freq + 2)
        base_w = max(3, shape[1] // freq + 2)
        noise_layer = rng.rand(base_h, base_w)
        noise_layer = ndimage.gaussian_filter(noise_layer, sigma=0.8)
        from scipy.ndimage import zoom
        scaled = zoom(noise_layer, (shape[0] / noise_layer.shape[0],
                                   shape[1] / noise_layer.shape[1]),
                      order=3)
        scaled = scaled[:shape[0], :shape[1]]
        result += scaled * amplitude
        total_amp += amplitude
        amplitude *= persistence
    return result / total_amp

def _radial_gradient(shape, cx=None, cy=None):
    h, w = shape
    if cx is None:
        cx = w / 2
    if cy is None:
        cy = h / 2
    y, x = np.mgrid[0:h, 0:w]
    r = np.sqrt((x - cx) ** 2 + (y - cy) ** 2)
    r = r / max(r.max(), 1e-6)
    return r

def _bloom(img_array, threshold=200, radius=8, intensity=0.6):
    bright = np.clip(img_array.astype(np.float64) - threshold, 0, 255)
    blurred = ndimage.gaussian_filter(bright, sigma=radius)
    result = img_array.astype(np.float64) + blurred * intensity
    return np.clip(result, 0, 255).astype(np.uint8)

def _vignette(shape, strength=0.7):
    r = _radial_gradient(shape)
    v = 1.0 - (r ** 1.8) * strength
    return np.clip(v, 0, 1)

def _star_field(shape, density=800, seed=42):
    rng = np.random.RandomState(seed)
    h, w = shape
    stars = np.zeros((h, w, 3), dtype=np.float64)
    n_stars = density
    xs = rng.randint(0, w, n_stars)
    ys = rng.randint(0, h, n_stars)
    brightnesses = rng.power(0.5, n_stars)
    temps = rng.uniform(3000, 20000, n_stars)

    for i in range(n_stars):
        x, y = xs[i], ys[i]
        b = brightnesses[i]
        t = temps[i]
        r_c, g_c, b_c = _blackbody_color(t)
        size = 1 if b < 0.7 else (2 if b < 0.9 else 3)
        lum = b * 255

        for dy in range(-size, size + 1):
            for dx in range(-size, size + 1):
                ny, nx = y + dy, x + dx
                if 0 <= ny < h and 0 <= nx < w:
                    d = math.sqrt(dx * dx + dy * dy)
                    falloff = max(0, 1.0 - d / (size + 0.5))
                    falloff = falloff ** 1.5
```

```

        stars[ny, nx, 0] += lum * r_c * falloff
        stars[ny, nx, 1] += lum * g_c * falloff
        stars[ny, nx, 2] += lum * b_c * falloff

    return np.clip(stars, 0, 255)

def _blackbody_color(temp):
    t = temp / 100.0
    if t <= 66:
        r = 1.0
        g = max(0, min(1, (99.4708025861 * math.log(t) - 161.1195681661) / 255.0)) if t > 1 else 0
    else:
        r = max(0, min(1, (329.698727446 * ((t - 60) ** -0.1332047592)) / 255.0))
        g = max(0, min(1, (288.1221695283 * ((t - 60) ** -0.0755148492)) / 255.0))

    if t > 66:
        b = 1.0
    elif t <= 19:
        b = 0.0
    else:
        b = max(0, min(1, (138.5177312231 * math.log(t - 10) - 305.0447927307) / 255.0))

    return r, g, b

def _color_map(value, palette):
    value = np.clip(value, 0, 1)
    n = len(palette) - 1
    idx = value * n
    low = np.floor(idx).astype(int)
    low = np.clip(low, 0, n - 1)
    high = np.clip(low + 1, 0, n)
    frac = idx - low
    result = np.zeros((value.shape, 3), dtype=np.float64)
    for c in range(3):
        low_vals = np.array([palette[i][c] for i in range(len(palette))])
        result[:, c] = low_vals[low] * (1 - frac) + low_vals[high] * frac
    return result

NEBULA_PALETTES = {
    'fire': [(0, 0, 0), (80, 10, 5), (180, 40, 10), (255, 120, 20), (255, 200, 80), (255, 255, 200)],
    'ice': [(0, 0, 10), (10, 20, 60), (30, 60, 140), (80, 140, 200), (160, 210, 255), (230, 245, 255)],
    'cosmic': [(0, 0, 5), (40, 5, 60), (100, 20, 120), (160, 40, 140), (200, 80, 180), (255, 180, 255)],
    'emerald': [(0, 5, 0), (10, 40, 20), (20, 100, 50), (60, 180, 80), (120, 230, 140), (200, 255, 220)],
    'plasma': [(0, 0, 0), (60, 0, 80), (120, 0, 160), (200, 60, 120), (255, 140, 60), (255, 240, 180)],
    'void': [(0, 0, 0), (15, 5, 30), (30, 10, 50), (50, 15, 70), (80, 30, 100), (120, 50, 140)],
}

def render_black_hole(w, h, seed, params=None):
    params = params or {}
    rng = np.random.RandomState(seed)
    img = np.zeros((h, w, 3), dtype=np.float64)

    stars = _star_field((h, w), density=1200, seed=seed)

    cx, cy = w / 2, h / 2
    bh_radius = min(w, h) * 0.15
    photon_ring = bh_radius * 1.5
    accretion_outer = bh_radius * 3.5
    accretion_inner = bh_radius * 1.8

    y, x = np.mgrid[0:h, 0:w]
    dx = (x - cx).astype(np.float64)
    dy = (y - cy).astype(np.float64)
    r = np.sqrt(dx ** 2 + dy ** 2)
    theta = np.arctan2(dy, dx)

    lensing_strength = 2.5
    lens_factor = np.where(r > bh_radius,
                          1.0 + lensing_strength * (bh_radius / np.maximum(r, 1)) ** 2,
                          0.0)

    lensed_x = (cx + dx * lens_factor).astype(int)
    lensed_y = (cy + dy * lens_factor).astype(int)
    lensed_x = np.clip(lensed_x, 0, w - 1)
    lensed_y = np.clip(lensed_y, 0, h - 1)

    for c in range(3):
        img[:, c] = stars[lensed_y, lensed_x, c]

    disk_mask = (r > accretion_inner) & (r < accretion_outer)
    tilt = params.get('tilt', 0.3)
    disk_y = dy * math.cos(tilt * math.pi)
    disk_r = np.sqrt(dx ** 2 + disk_y ** 2)
    thin_disk = np.exp(-((dy * math.cos(tilt * math.pi)) ** 2) / (2 * (bh_radius * 0.15) ** 2))
    thin_disk *= ((disk_r > accretion_inner) & (disk_r < accretion_outer)).astype(float)

    disk_temp = np.clip(1.0 - (disk_r - accretion_inner) / (accretion_outer - accretion_inner), 0, 1)
    disk_temp = disk_temp ** 0.6

    doppler = 1.0 + 0.4 * np.sin(theta)
    disk_temp_shifted = np.clip(disk_temp * doppler, 0, 1)

    noise_disk = _fbm_noise((h, w), octaves=4, seed=seed + 1)
    disk_detail = 0.6 + 0.4 * noise_disk

    for c, base in enumerate([(255, 180, 60), (120, 60, 20), (40, 20, 5)]):
        hot_color = np.array([255, 220, 180])[c]
        cool_color = np.array([180, 60, 10])[c]

```

```

        disk_color = cool_color + (hot_color - cool_color) * disk_temp_shifted
        img[..., c] += disk_color * thin_disk * disk_detail * 1.5

    ring_mask = np.exp(-((r - photon_ring) ** 2) / (2 * (bh_radius * 0.05) ** 2))
    for c, val in enumerate([255, 240, 200]):
        img[..., c] += val * ring_mask * 0.8

    event_horizon = np.where(r <= bh_radius, 0.0, 1.0)
    fade = np.clip((r - bh_radius * 0.8) / (bh_radius * 0.2), 0, 1)
    for c in range(3):
        img[..., c] *= fade

    jets = params.get('jets', True)
    if jets:
        jet_width = bh_radius * 0.08
        jet_length = min(h, w) * 0.4
        for sign in [-1, 1]:
            jet_y_center = cy + sign * np.arange(0, jet_length)
            for jy in jet_y_center.astype(int):
                if 0 <= jy < h:
                    dist_from_bh = abs(jy - cy)
                    intensity = max(0, 1.0 - dist_from_bh / jet_length) ** 1.5
                    spread = jet_width * (1 + dist_from_bh / jet_length)
                    for jx in range(max(0, int(cx - spread)), min(w, int(cx + spread))):
                        dx_j = abs(jx - cx)
                        profile = math.exp(-(dx_j ** 2) / (2 * (spread * 0.3) ** 2))
                        img[jy, jx, 0] += 100 * intensity * profile
                        img[jy, jx, 1] += 140 * intensity * profile
                        img[jy, jx, 2] += 255 * intensity * profile

    vig = _vignette((h, w), 0.5)
    for c in range(3):
        img[..., c] *= vig

    img = np.clip(img, 0, 255).astype(np.uint8)
    img = _bloom(img, threshold=100, radius=6, intensity=0.5)
    return img

def render_nebula(w, h, seed, params=None):
    params = params or {}
    palette_name = params.get('palette', 'cosmic')
    palette = NEBULA_PALETTES.get(palette_name, NEBULA_PALETTES['cosmic'])
    rng = np.random.RandomState(seed)

    stars = _star_field((h, w), density=800, seed=seed)
    img = stars.copy()
    y_grid, x_grid = np.mgrid[0:h, 0:w]

    cloud_base = np.zeros((h, w), dtype=np.float64)
    n_centers = rng.randint(3, 7)
    for i in range(n_centers):
        cx = rng.uniform(w * 0.1, w * 0.9)
        cy = rng.uniform(h * 0.1, h * 0.9)
        spread_x = rng.uniform(w * 0.15, w * 0.4)
        spread_y = rng.uniform(h * 0.15, h * 0.4)
        amp = rng.uniform(0.3, 1.0)
        angle = rng.uniform(0, math.pi)
        cos_a, sin_a = math.cos(angle), math.sin(angle)
        dx = (x_grid - cx)
        dy = (y_grid - cy)
        rx = dx * cos_a + dy * sin_a
        ry = -dx * sin_a + dy * cos_a
        blob = np.exp(-(rx ** 2) / (2 * spread_x ** 2) - (ry ** 2) / (2 * spread_y ** 2))
        cloud_base += blob * amp

    cloud_base = cloud_base / (cloud_base.max() + 1e-8)

    detail = _fbm_noise((h, w), octaves=7, persistence=0.55, seed=seed + 10)
    fine = _fbm_noise((h, w), octaves=5, persistence=0.6, seed=seed + 20)

    warp_u = _fbm_noise((h, w), octaves=4, seed=seed + 300)
    warp_v = _fbm_noise((h, w), octaves=4, seed=seed + 310)
    warp_s = min(w, h) * 0.1
    wx = np.clip((x_grid + (warp_u - 0.5) * warp_s).astype(int), 0, w - 1)
    wy = np.clip((y_grid + (warp_v - 0.5) * warp_s).astype(int), 0, h - 1)

    combined = cloud_base[wy, wx] * (0.4 + 0.6 * detail)
    combined *= (0.6 + 0.4 * fine)
    combined = (combined - combined.min()) / (combined.max() - combined.min() + 1e-8)
    combined = combined ** 0.9

    nebula_colors = _color_map(combined, palette)
    density = np.clip(combined * 1.8 - 0.15, 0, 1)

    for c in range(3):
        img[..., c] = img[..., c] * (1 - density * 0.9) + nebula_colors[..., c] * density

    secondary_map = {'fire': 'plasma', 'ice': 'cosmic', 'cosmic': 'plasma',
                     'emerald': 'ice', 'plasma': 'fire', 'void': 'cosmic'}
    sec_palette = NEBULA_PALETTES[secondary_map.get(palette_name, 'cosmic')]
    sec_noise = _fbm_noise((h, w), octaves=5, seed=seed + 800)
    sec_density = np.clip(sec_noise - 0.4, 0, 1) * combined * 0.35
    sec_colors = _color_map(sec_noise, sec_palette)
    for c in range(3):
        img[..., c] += sec_colors[..., c] * sec_density

    fil = _fbm_noise((h, w), octaves=8, persistence=0.7, seed=seed + 400)
    filaments = np.exp(-(np.abs(fil - 0.5) * 30)) * 0.8
    filaments *= (combined > 0.1).astype(float)
    for c in range(3):

```

```

img[..., c] += filaments * palette[-1][c] * 0.5

bright_noise = _fbm_noise((h, w), octaves=3, seed=seed + 500)
hot = (bright_noise > 0.7).astype(float) * (combined > 0.35).astype(float)
hot = ndimage.gaussian_filter(hot, sigma=min(w, h) * 0.012)
for c in range(3):
    img[..., c] += hot * 240 * (palette[-1][c] / 255.0)

n_stars_embedded = rng.randint(3, 8)
for _ in range(n_stars_embedded):
    sx, sy = rng.randint(0, w), rng.randint(0, h)
    if combined[min(sy, h - 1), min(sx, w - 1)] > 0.25:
        sr = rng.uniform(min(w, h) * 0.008, min(w, h) * 0.025)
        glow = np.exp(-(x_grid - sx) ** 2 + (y_grid - sy) ** 2) / (2 * sr ** 2)
        for c in range(3):
            img[..., c] += glow * 255 * rng.uniform(0.5, 1.0)

vig = _vignette((h, w), 0.5)
for c in range(3):
    img[..., c] *= vig

img = np.clip(img, 0, 255).astype(np.uint8)
img = _bloom(img, threshold=130, radius=14, intensity=0.5)
return img

def render_galaxy(w, h, seed, params=None):
    params = params or {}
    rng = np.random.RandomState(seed)
    img = np.zeros((h, w, 3), dtype=np.float64)
    stars_bg = _star_field((h, w), density=500, seed=seed)
    img += stars_bg

    cx, cy = w / 2, h / 2
    y, x = np.mgrid[0:h, 0:w]
    dx = (x - cx).astype(np.float64)
    dy = (y - cy).astype(np.float64)

    tilt = params.get('tilt', 0.6)
    dy_tilted = dy / max(math.cos(tilt), 0.3)
    r = np.sqrt(dx ** 2 + dy_tilted ** 2)
    theta = np.arctan2(dy_tilted, dx)

    galaxy_r = min(w, h) * 0.35
    n_arms = params.get('arms', 2)
    arm_tightness = 0.4

    arm_pattern = np.zeros((h, w), dtype=np.float64)
    for arm in range(n_arms):
        arm_offset = arm * 2 * math.pi / n_arms
        spiral_angle = theta - arm_tightness * np.log(np.maximum(r, 1)) - arm_offset
        arm_dist = np.abs(np.sin(spiral_angle))
        arm_width = 0.15 + 0.1 * (r / galaxy_r)
        arm_intensity = np.exp(-(arm_dist ** 2) / (2 * arm_width ** 2))
        arm_intensity *= np.exp(-r / galaxy_r)
        arm_pattern += arm_intensity

    arm_pattern = np.clip(arm_pattern, 0, 1)

    noise = _fbm_noise((h, w), octaves=5, seed=seed + 10)
    arm_pattern *= (0.6 + 0.4 * noise)

    core_glow = np.exp(-r ** 2 / (2 * (galaxy_r * 0.15) ** 2))

    palette = [(255, 200, 150), (200, 150, 255), (150, 180, 255)]
    for c in range(3):
        core_c = [255, 240, 200][c]
        arm_c = palette[c % len(palette)][c]
        img[..., c] += core_glow * core_c * 1.2
        img[..., c] += arm_pattern * arm_c * 0.8

    n_gal_stars = 3000
    for _ in range(n_gal_stars):
        angle = rng.uniform(0, 2 * math.pi)
        radius = rng.exponential(galaxy_r * 0.3)
        arm_idx = rng.randint(0, n_arms)
        spiral_offset = arm_tightness * math.log(max(radius, 1)) + arm_idx * 2 * math.pi / n_arms
        angle = spiral_offset + rng.normal(0, 0.2)

        sx = int(cx + radius * math.cos(angle))
        sy = int(cy + radius * math.sin(angle) * math.cos(tilt))

        if 0 <= sx < w and 0 <= sy < h:
            brightness = rng.uniform(50, 255) * math.exp(-radius / galaxy_r)
            temp = rng.uniform(4000, 15000)
            rc, gc, bc = _blackbody_color(temp)
            img[sy, sx, 0] += brightness * rc
            img[sy, sx, 1] += brightness * gc
            img[sy, sx, 2] += brightness * bc

    vig = _vignette((h, w), 0.5)
    for c in range(3):
        img[..., c] *= vig

    img = np.clip(img, 0, 255).astype(np.uint8)
    img = _bloom(img, threshold=150, radius=8, intensity=0.5)
    return img

def render_quantum_state(w, h, seed, params=None):
    params = params or {}

```

```

rng = np.random.RandomState(seed)
img = np.zeros((h, w, 3), dtype=np.float64)

y, x = np.mgrid[0:h, 0:w]
cx, cy = w / 2, h / 2
dx = (x - cx) / (w / 2)
dy = (y - cy) / (h / 2)
r = np.sqrt(dx ** 2 + dy ** 2)

n_modes = rng.randint(3, 7)
psi_real = np.zeros((h, w), dtype=np.float64)
psi_imag = np.zeros((h, w), dtype=np.float64)

for _ in range(n_modes):
    kx = rng.uniform(-8, 8)
    ky = rng.uniform(-8, 8)
    phase = rng.uniform(0, 2 * math.pi)
    amp = rng.uniform(0.5, 1.5)
    psi_real += amp * np.cos(kx * dx * math.pi + ky * dy * math.pi + phase)
    psi_imag += amp * np.sin(kx * dx * math.pi + ky * dy * math.pi + phase)

prob = psi_real ** 2 + psi_imag ** 2
prob = prob / (prob.max() + 1e-8)

phase = np.arctan2(psi_imag, psi_real)
phase_norm = (phase + math.pi) / (2 * math.pi)

phase_palette = [
    (0, 60, 255), (0, 200, 200), (0, 255, 60),
    (255, 255, 0), (255, 100, 0), (255, 0, 100),
    (200, 0, 255), (0, 60, 255)
]
phase_colors = _color_map(phase_norm, phase_palette)

envelope = np.exp(-r ** 2 / 0.8)
prob *= envelope

for c in range(3):
    img[..., c] = phase_colors[..., c] * prob * 1.5

stars = _star_field((h, w), density=200, seed=seed)
for c in range(3):
    img[..., c] += stars[..., c] * (1 - prob * 0.8)

interference = np.cos(prob * 20 * math.pi) * 0.5 + 0.5
for c in range(3):
    img[..., c] *= (0.7 + 0.3 * interference)

contour_lines = np.abs(np.sin(prob * 15 * math.pi))
contour_lines = (contour_lines < 0.05).astype(float) * prob * 0.3
for c in range(3):
    img[..., c] += contour_lines * 200

vig = _vignette((h, w), 0.4)
for c in range(3):
    img[..., c] *= vig

img = np.clip(img, 0, 255).astype(np.uint8)
img = _bloom(img, threshold=140, radius=6, intensity=0.4)
return img

def render_wormhole(w, h, seed, params=None):
    params = params or {}
    img = np.zeros((h, w, 3), dtype=np.float64)

    cx, cy = w / 2, h / 2
    y, x = np.mgrid[0:h, 0:w]
    dx = (x - cx).astype(np.float64)
    dy = (y - cy).astype(np.float64)
    r = np.sqrt(dx ** 2 + dy ** 2)
    theta = np.arctan2(dy, dx)

    throat_radius = min(w, h) * 0.08
    tunnel_depth = 15.0

    depth = np.where(r > throat_radius,
        tunnel_depth * (throat_radius / np.maximum(r, 1)) ** 0.8,
        tunnel_depth)

    tunnel_pattern = np.sin(depth * 3 + theta * 4) * 0.5 + 0.5

    noise = _fbm_noise((h, w), octaves=5, seed=seed)
    tunnel_pattern *= (0.5 + 0.5 * noise)

    ring_dist = np.abs(r - throat_radius * 2)
    ring_glow = np.exp(-ring_dist ** 2 / (2 * (throat_radius * 0.3) ** 2))

    warp_factor = np.where(r > throat_radius,
        (throat_radius / np.maximum(r, 1)) ** 1.5,
        1.0)

    edge_glow = np.exp(-((r - throat_radius) ** 2) / (2 * (throat_radius * 0.15) ** 2))

    palette = [(0, 0, 20), (20, 40, 120), (80, 100, 200), (150, 180, 255), (220, 240, 255)]
    tunnel_colors = _color_map(tunnel_pattern * warp_factor, palette)

    for c in range(3):
        img[..., c] = tunnel_colors[..., c]

    for c, val in enumerate([200, 220, 255]):

```

```

    img[..., c] += ring_glow * val * 0.6
for c, val in enumerate([100, 200, 255]):
    img[..., c] += edge_glow * val * 1.2

inner_mask = (r < throat_radius * 0.6).astype(float)
dest_stars = _star_field((h, w), density=400, seed=seed + 999)
dest_nebula = _fbm_noise((h, w), octaves=4, seed=seed + 888)
fade_in = np.clip(1.0 - r / (throat_radius * 0.6), 0, 1) * inner_mask
for c in range(3):
    dest_color = [80, 40, 120][c]
    img[..., c] += (dest_stars[..., c] * 0.5 + dest_nebula * dest_color * 0.3) * fade_in

stars = _star_field((h, w), density=800, seed=seed)
outer_mask = np.clip((r - throat_radius * 3) / (throat_radius * 2), 0, 1)
for c in range(3):
    img[..., c] += stars[..., c] * outer_mask

vig = _vignette((h, w), 0.5)
for c in range(3):
    img[..., c] *= vig

img = np.clip(img, 0, 255).astype(np.uint8)
img = _bloom(img, threshold=150, radius=8, intensity=0.5)
return img

def render_supernova(w, h, seed, params=None):
    params = params or {}
    rng = np.random.RandomState(seed)
    img = np.zeros((h, w, 3), dtype=np.float64)

    stars = _star_field((h, w), density=800, seed=seed)
    img += stars

    cx, cy = w / 2, h / 2
    y, x = np.mgrid[0:h, 0:w]
    dx = (x - cx).astype(np.float64)
    dy = (y - cy).astype(np.float64)
    r = np.sqrt(dx ** 2 + dy ** 2)
    theta = np.arctan2(dy, dx)

    blast_radius = min(w, h) * 0.35
    core_radius = min(w, h) * 0.03

    core_glow = np.exp(-r ** 2 / (2 * core_radius ** 2))
    for c, val in enumerate([255, 255, 240]):
        img[..., c] += core_glow * val * 2.0

    n_rays = rng.randint(12, 24)
    for i in range(n_rays):
        ray_angle = i * 2 * math.pi / n_rays + rng.uniform(-0.1, 0.1)
        ray_width = rng.uniform(0.03, 0.08)
        ray_length = blast_radius * rng.uniform(0.5, 1.0)

        angle_diff = np.abs(np.mod(theta - ray_angle + math.pi, 2 * math.pi) - math.pi)
        ray_mask = np.exp(-(angle_diff ** 2) / (2 * ray_width ** 2))
        ray_mask *= np.exp(-r / ray_length)
        ray_mask *= (r > core_radius * 2).astype(float)

        ray_temp = rng.uniform(5000, 30000)
        rc, gc, bc = _blackbody_color(ray_temp)
        intensity = rng.uniform(150, 255)
        img[..., 0] += ray_mask * intensity * rc
        img[..., 1] += ray_mask * intensity * gc
        img[..., 2] += ray_mask * intensity * bc

    shell_r = blast_radius * 0.7
    shell_width = blast_radius * 0.1
    shell = np.exp(-((r - shell_r) ** 2) / (2 * shell_width ** 2))
    shell_noise = _fbm_noise((h, w), octaves=6, seed=seed + 50)
    shell *= (0.4 + 0.6 * shell_noise)

    for c, val in enumerate([200, 100, 50]):
        img[..., c] += shell * val

    inner_shell_r = blast_radius * 0.4
    inner_shell = np.exp(-((r - inner_shell_r) ** 2) / (2 * (shell_width * 0.7) ** 2))
    for c, val in enumerate([100, 180, 255]):
        img[..., c] += inner_shell * val * 0.5

    vig = _vignette((h, w), 0.5)
    for c in range(3):
        img[..., c] *= vig

    img = np.clip(img, 0, 255).astype(np.uint8)
    img = _bloom(img, threshold=140, radius=12, intensity=0.7)
    return img

def render_planet(w, h, seed, params=None):
    params = params or {}
    rng = np.random.RandomState(seed)
    img = np.zeros((h, w, 3), dtype=np.float64)

    stars = _star_field((h, w), density=600, seed=seed)
    img += stars

    cx, cy = w / 2, h / 2
    planet_r = min(w, h) * 0.3
    y, x = np.mgrid[0:h, 0:w]
    dx = (x - cx).astype(np.float64)

```

```

dy = (y - cy).astype(np.float64)
r = np.sqrt(dx ** 2 + dy ** 2)

planet_mask = (r <= planet_r).astype(float)
norm_r = r / planet_r

nx = dx / np.maximum(r, 1e-6)
ny = dy / np.maximum(r, 1e-6)
nz = np.sqrt(np.maximum(1 - norm_r ** 2, 0))

light_dir = np.array([0.5, -0.3, 0.8])
light_dir /= np.linalg.norm(light_dir)
shading = nx * light_dir[0] + ny * light_dir[1] + nz * light_dir[2]
shading = np.clip(shading, 0, 1) * planet_mask

planet_type = params.get('type', rng.choice(['rocky', 'gas', 'ice', 'lava']))

if planet_type == 'gas':
    band_noise = _fbm_noise((h, w), octaves=6, seed=seed + 10)
    bands = np.sin((dy / planet_r) * 15 + band_noise * 3) * 0.5 + 0.5
    colors = _color_map(bands, [
        (180, 140, 80), (200, 160, 100), (160, 120, 60),
        (220, 180, 120), (140, 100, 50), (200, 160, 90)
    ])
    spot_r = planet_r * 0.15
    spot_cx, spot_cy = cx + planet_r * 0.2, cy + planet_r * 0.1
    spot_dist = np.sqrt((x - spot_cx) ** 2 + (y - spot_cy) ** 2)
    spot = np.exp(-(spot_dist ** 2) / (2 * spot_r ** 2)) * planet_mask
    colors[..., 0] += spot * 80
    colors[..., 1] -= spot * 30
elif planet_type == 'ice':
    surface = _fbm_noise((h, w), octaves=7, seed=seed + 20)
    colors = _color_map(surface, [
        (180, 210, 230), (200, 230, 250), (220, 240, 255),
        (160, 200, 240), (140, 180, 220)
    ])
    cracks = _fbm_noise((h, w), octaves=8, persistence=0.7, seed=seed + 30)
    crack_lines = (np.abs(cracks - 0.5) < 0.02).astype(float)
    for c in range(3):
        colors[..., c] += crack_lines * 60
elif planet_type == 'lava':
    surface = _fbm_noise((h, w), octaves=6, seed=seed + 40)
    colors = _color_map(surface, [
        (40, 20, 10), (80, 30, 10), (60, 25, 10),
        (100, 40, 15), (50, 20, 8)
    ])
    lava = _fbm_noise((h, w), octaves=4, seed=seed + 50)
    lava_mask = (lava > 0.65).astype(float)
    lava_mask = ndimage.gaussian_filter(lava_mask, sigma=2)
    colors[..., 0] += lava_mask * 200
    colors[..., 1] += lava_mask * 80
    colors[..., 2] += lava_mask * 10
else:
    surface = _fbm_noise((h, w), octaves=7, seed=seed + 60)
    colors = _color_map(surface, [
        (60, 80, 40), (80, 100, 50), (100, 120, 60),
        (140, 130, 100), (120, 110, 80), (80, 90, 45)
    ])
    ocean = (surface < 0.4).astype(float)
    for c, val in enumerate([30, 60, 140]):
        colors[..., c] = colors[..., c] * (1 - ocean) + val * ocean

for c in range(3):
    img[..., c] = img[..., c] * (1 - planet_mask) + colors[..., c] * shading * planet_mask

atmo_dist = np.abs(r - planet_r)
atmo_glow = np.exp(-(atmo_dist ** 2) / (2 * (planet_r * 0.05) ** 2))
atmo_glow *= (r > planet_r * 0.95).astype(float)
atmo_colors = {'gas': (200, 180, 120), 'ice': (150, 200, 255),
               'lava': (255, 100, 30), 'rocky': (100, 150, 255)}
ac = atmo_colors.get(planet_type, (100, 150, 255))
for c in range(3):
    img[..., c] += atmo_glow * ac[c] * 0.6

vig = _vignette((h, w), 0.4)
for c in range(3):
    img[..., c] *= vig

img = np.clip(img, 0, 255).astype(np.uint8)
img = _bloom(img, threshold=180, radius=6, intensity=0.3)
return img

def render_fractal(w, h, seed, params=None):
    params = params or {}
    rng = np.random.RandomState(seed)

    cx = rng.uniform(-0.8, 0.3)
    cy = rng.uniform(-0.6, 0.6)
    zoom_level = rng.uniform(0.5, 3.0)
    scale = 2.0 / (zoom_level * min(w, h))

    y_coords, x_coords = np.mgrid[0:h, 0:w]
    x0 = (x_coords - w / 2) * scale + cx
    y0 = (y_coords - h / 2) * scale + cy

    z_real = np.zeros((h, w), dtype=np.float64)
    z_imag = np.zeros((h, w), dtype=np.float64)
    iterations = np.zeros((h, w), dtype=np.float64)
    max_iter = 256
    mask = np.ones((h, w), dtype=bool)

```

```

for i in range(max_iter):
    zr2 = z_real ** 2
    zi2 = z_imag ** 2
    escaped = (zr2 + zi2 > 4) & mask
    iterations[escaped] = i + 1 - np.log2(np.log2(np.sqrt(zr2[escaped] + zi2[escaped])))
    mask[escaped] = False

    if not mask.any():
        break

    z_imag[mask] = 2 * z_real[mask] * z_imag[mask] + y0[mask]
    z_real[mask] = zr2[mask] - zi2[mask] + x0[mask]

iterations[mask] = max_iter
norm_iter = iterations / max_iter
norm_iter = np.sqrt(norm_iter)

palette_choice = params.get('palette', rng.choice(['cosmic', 'fire', 'ice']))
palette = NEBULA_PALETTES.get(palette_choice, NEBULA_PALETTES['cosmic'])
colors = _color_map(norm_iter, palette)

interior = (iterations >= max_iter)
colors[interior] = [0, 0, 0]

img = np.clip(colors, 0, 255).astype(np.uint8)
img = _bloom(img, threshold=200, radius=4, intensity=0.3)
return img

def render_neural(w, h, seed, params=None):
    params = params or {}
    rng = np.random.RandomState(seed)
    img = np.zeros((h, w, 3), dtype=np.float64)

    bg = _fbm_noise((h, w), octaves=5, seed=seed)
    bg = bg ** 2
    for c, val in enumerate([15, 5, 25]):
        img[:, :, c] = bg * val

    n_nodes = params.get('nodes', rng.randint(15, 40))
    nodes = []
    for _ in range(n_nodes):
        nodes.append((rng.randint(w * 0.1, w * 0.9),
                      rng.randint(h * 0.1, h * 0.9),
                      rng.uniform(5, 20)))

    pil_img = Image.fromarray(np.clip(img, 0, 255).astype(np.uint8))
    draw = ImageDraw.Draw(pil_img)

    for i, (x1, y1, r1) in enumerate(nodes):
        for j, (x2, y2, r2) in enumerate(nodes):
            if i >= j:
                continue
            dist = math.sqrt((x1 - x2) ** 2 + (y1 - y2) ** 2)
            max_dist = min(w, h) * 0.4
            if dist <= max_dist and rng.random() < 0.4:
                alpha = int(max(20, 150 * (1 - dist / max_dist)))
                color_choice = rng.choice(['cyan', 'violet', 'white'])
                if color_choice == 'cyan':
                    c = (0, alpha, alpha)
                elif color_choice == 'violet':
                    c = (alpha // 2, 0, alpha)
                else:
                    c = (alpha // 2, alpha // 2, alpha // 2)
                draw.line([(x1, y1), (x2, y2)], fill=c, width=1)

            mid_x, mid_y = (x1 + x2) / 2, (y1 + y2) / 2
            n_pulses = rng.randint(0, 3)
            for p in range(n_pulses):
                t = rng.uniform(0.2, 0.8)
                px = int(x1 + t * (x2 - x1))
                py = int(y1 + t * (y2 - y1))
                pr = rng.randint(2, 5)
                draw.ellipse([px - pr, py - pr, px + pr, py + pr],
                             fill=(100, 200, 255, 200))

    for x, y, radius in nodes:
        for ring_r in [radius * 1.5, radius]:
            draw.ellipse([x - ring_r, y - ring_r, x + ring_r, y + ring_r],
                          outline=(80, 40, 140), width=1)
        draw.ellipse([x - radius * 0.6, y - radius * 0.6,
                      x + radius * 0.6, y + radius * 0.6],
                      fill=(160, 100, 255))
        inner_r = radius * 0.3
        draw.ellipse([x - inner_r, y - inner_r, x + inner_r, y + inner_r],
                      fill=(220, 200, 255))

    img = np.array(pil_img).astype(np.float64)

    vig = _vignette((h, w), 0.5)
    for c in range(3):
        img[:, :, c] *= vig

    img = np.clip(img, 0, 255).astype(np.uint8)
    img = _bloom(img, threshold=120, radius=10, intensity=0.6)
    return img

def render_consciousness(w, h, seed, params=None):
    params = params or {}
    rng = np.random.RandomState(seed)

```

```

img = np.zeros((h, w, 3), dtype=np.float64)

stars = _star_field((h, w), density=300, seed=seed)
img += stars * 0.3

cx, cy = w / 2, h / 2
y, x = np.mgrid[0:h, 0:w]
dx = (x - cx).astype(np.float64) / (w / 2)
dy = (y - cy).astype(np.float64) / (h / 2)
r = np.sqrt(dx ** 2 + dy ** 2)
theta = np.arctan2(dy, dx)

n_tubes = rng.randint(5, 12)
for t in range(n_tubes):
    tube_angle = t * 2 * math.pi / n_tubes + rng.uniform(-0.2, 0.2)
    tube_len = rng.uniform(0.3, 0.8)
    tube_width = 0.02 + rng.uniform(0, 0.01)

    cos_a = math.cos(tube_angle)
    sin_a = math.sin(tube_angle)
    proj = dx * cos_a + dy * sin_a
    perp = np.abs(-dx * sin_a + dy * cos_a)

    tube_mask = np.exp(-(perp ** 2) / (2 * tube_width ** 2))
    tube_mask *= (proj > 0.05).astype(float) * (proj < 0.05).astype(float)

    wave = np.sin(proj * 30 + rng.uniform(0, 2 * math.pi)) * 0.5 + 0.5
    tube_mask *= (0.3 + 0.7 * wave)

    hue = rng.uniform(0, 1)
    if hue < 0.33:
        tc = (100, 200, 255)
    elif hue < 0.66:
        tc = (200, 100, 255)
    else:
        tc = (100, 255, 200)

    for c in range(3):
        img[..., c] += tube_mask * tc[c] * 0.6

core_r = 0.12
core = np.exp(-r ** 2 / (2 * core_r ** 2))
pulse = np.sin(r * 40) * 0.3 + 0.7
core *= pulse

for c, val in enumerate([255, 200, 255]):
    img[..., c] += core * val * 0.8

n_rings = 5
for i in range(n_rings):
    ring_r = 0.15 + i * 0.12
    ring = np.exp(-(r - ring_r) ** 2) / (2 * 0.008 ** 2)
    alpha = 0.4 * (1 - i / n_rings)
    for c, val in enumerate([150, 100, 255]):
        img[..., c] += ring * val * alpha

coherence_noise = _fbm_noise((h, w), octaves=5, seed=seed + 77)
decoherence = np.clip(r - 0.4, 0, 1) * coherence_noise
for c in range(3):
    img[..., c] *= (1 - decoherence * 0.5)

vig = _vignette((h, w), 0.5)
for c in range(3):
    img[..., c] *= vig

img = np.clip(img, 0, 255).astype(np.uint8)
img = _bloom(img, threshold=120, radius=10, intensity=0.6)
return img

```

```

SCENE_KEYWORDS = {
    'black_hole': ['black hole', 'event horizon', 'singularity', 'gravitational', 'accretion',
                  'photon ring', 'schwarzschild', 'kerr', 'spacetime curvature'],
    'nebula': ['nebula', 'nebulae', 'gas cloud', 'stellar nursery', 'emission',
              'interstellar', 'cosmic dust', 'pillars', 'orion', 'carina', 'eagle'],
    'galaxy': ['galaxy', 'spiral', 'milky way', 'andromeda', 'galactic',
              'barred spiral', 'elliptical', 'star system'],
    'quantum_state': ['quantum', 'wavefunction', 'superposition', 'entangle',
                     'probability', 'schrodinger', 'qubit', 'quantum state',
                     'wave function', 'quantum field'],
    'wormhole': ['wormhole', 'einstein-rosen', 'bridge', 'portal', 'tunnel',
                 'spacetime tunnel', 'traversable'],
    'supernova': ['supernova', 'explosion', 'stellar death', 'nova', 'burst',
                  'star explod', 'detonation', 'stellar explosion', 'gamma ray'],
    'planet': ['planet', 'world', 'terrestrial', 'jovian', 'earth',
               'mars', 'jupiter', 'saturn', 'venus', 'exoplanet', 'moon'],
    'fractal': ['fractal', 'mandelbrot', 'julia', 'recursive', 'self-similar',
                'chaos', 'strange attractor', 'infinite', 'iteration'],
    'neural': ['neural', 'brain', 'neuron', 'synapse', 'network',
               'microtubule', 'axon', 'dendrite', 'cortex', 'cognitive'],
    'consciousness': ['consciousness', 'awareness', 'orch-or', 'penrose',
                      'hameroff', 'tubulin', 'orchestrated', 'subjective',
                      'qualia', 'mind', 'sentien'],
}

```

```

RENDERERS = {
    'black_hole': render_black_hole,
    'nebula': render_nebula,
    'galaxy': render_galaxy,
    'quantum_state': render_quantum_state,
    'wormhole': render_wormhole,
}

```

```

'supernova': render_supernova,
'planet': render_planet,
'fractal': render_fractal,
'neural': render_neural,
'consciousness': render_consciousness,
}

def _detect_scene(prompt: str) -> tuple:
    prompt_lower = prompt.lower()
    scores = {}
    for scene, keywords in SCENE_KEYWORDS.items():
        score = 0
        for kw in keywords:
            if kw in prompt_lower:
                score += len(kw)
        if score > 0:
            scores[scene] = score

    if not scores:
        return 'nebula', {}

    best = max(scores, key=scores.get)

    params = {}
    if best == 'black_hole':
        if 'merg' in prompt_lower:
            params['merging'] = True
        if 'jet' in prompt_lower:
            params['jets'] = True
    elif best == 'nebula':
        for name in NEBULA_PALETTES:
            if name in prompt_lower:
                params['palette'] = name
            if 'fire' in prompt_lower or 'red' in prompt_lower or 'hot' in prompt_lower:
                params['palette'] = 'fire'
            elif 'ice' in prompt_lower or 'blue' in prompt_lower or 'cold' in prompt_lower:
                params['palette'] = 'ice'
            elif 'green' in prompt_lower or 'emerald' in prompt_lower:
                params['palette'] = 'emerald'
            elif 'purple' in prompt_lower or 'violet' in prompt_lower:
                params['palette'] = 'cosmic'
            elif 'dark' in prompt_lower or 'void' in prompt_lower:
                params['palette'] = 'void'
    elif best == 'planet':
        if 'gas' in prompt_lower or 'jupiter' in prompt_lower:
            params['type'] = 'gas'
        elif 'ice' in prompt_lower or 'frozen' in prompt_lower:
            params['type'] = 'ice'
        elif 'lava' in prompt_lower or 'volcanic' in prompt_lower:
            params['type'] = 'lava'
        elif 'rocky' in prompt_lower or 'mars' in prompt_lower:
            params['type'] = 'rocky'
    elif best == 'galaxy':
        if 'barred' in prompt_lower:
            params['arms'] = 2

    return best, params

def _render_merging_black_holes(w, h, seed, params=None):
    img = np.zeros((h, w, 3), dtype=np.float64)
    stars = _star_field((h, w), density=1000, seed=seed)

    separation = min(w, h) * 0.2
    cx1, cy1 = w / 2 - separation * 0.5, h / 2
    cx2, cy2 = w / 2 + separation * 0.5, h / 2

    y, x = np.mgrid[0:h, 0:w]

    for (bcx, bcy, bseed) in [(cx1, cy1, seed), (cx2, cy2, seed + 500)]:
        dx = (x - bcx).astype(np.float64)
        dy = (y - bcy).astype(np.float64)
        r = np.sqrt(dx ** 2 + dy ** 2)
        theta = np.arctan2(dy, dx)

        bh_r = min(w, h) * 0.08
        photon_r = bh_r * 1.5
        accr_outer = bh_r * 2.5
        accr_inner = bh_r * 1.5

        lens = np.where(r > bh_r, 1.0 + 2.0 * (bh_r / np.maximum(r, 1)) ** 2, 0.0)
        lx = np.clip((bcx + dx * lens).astype(int), 0, w - 1)
        ly = np.clip((bcy + dy * lens).astype(int), 0, h - 1)

        for c in range(3):
            img[..., c] += stars[ly, lx, c] * 0.5

        thin_disk = np.exp(-(dy ** 2) / (2 * (bh_r * 0.12) ** 2))
        disk_valid = ((r > accr_inner) & (r < accr_outer)).astype(float)
        thin_disk *= disk_valid
        disk_temp = np.clip(1.0 - (r - accr_inner) / (accr_outer - accr_inner), 0, 1) ** 0.6
        doppler = 1.0 + 0.3 * np.sin(theta)

        for c, (hot, cool) in enumerate([(255, 180), (200, 50), (150, 10)]):
            disk_c = cool + (hot - cool) * disk_temp * doppler
            img[..., c] += disk_c * thin_disk * 1.2

    ring = np.exp(-(r - photon_r) ** 2) / (2 * (bh_r * 0.04) ** 2)
    for c, val in enumerate([255, 230, 180]):
        img[..., c] += ring * val * 0.6

```

```

        fade = np.clip((r - bh_r * 0.8) / (bh_r * 0.2), 0, 1)
        for c in range(3):
            img[..., c] *= fade

    mid_x, mid_y = w / 2, h / 2
    bridge_dx = (x - mid_x).astype(np.float64)
    bridge_dy = (y - mid_y).astype(np.float64)
    bridge_r = np.sqrt(bridge_dx ** 2 + bridge_dy ** 2)
    bridge = np.exp(-(bridge_dy ** 2) / (2 * (min(h, w) * 0.03) ** 2))
    bridge *= (np.abs(bridge_dx) < separation * 0.6).astype(float)
    bridge *= np.exp(-bridge_r / (separation * 0.8))

    gw_waves = np.sin(bridge_r / (min(w, h) * 0.03) * math.pi) * 0.5 + 0.5
    bridge *= gw_waves

    for c, val in enumerate([200, 150, 255]):
        img[..., c] += bridge * val * 0.4

    vig = _vignette((h, w), 0.5)
    for c in range(3):
        img[..., c] *= vig

    img = np.clip(img, 0, 255).astype(np.uint8)
    img = _bloom(img, threshold=150, radius=8, intensity=0.6)
    return img


def generate_image(prompt: str, width: int = 512, height: int = 512,
                  variation_seed: int = None) -> Image.Image:
    seed = _seed_from_prompt(prompt)
    if variation_seed is not None:
        seed = (seed + variation_seed) % (2 ** 31)

    scene, params = _detect_scene(prompt)

    if scene == 'black_hole' and params.get('merging'):
        img_array = _render_merging_black_holes(width, height, seed, params)
    else:
        renderer = RENDERERS.get(scene, render_nebula)
        img_array = renderer(width, height, seed, params)

    return Image.fromarray(img_array)


def get_available_scenes():
    return list(RENDERERS.keys())


if __name__ == '__main__':
    import sys
    prompt = '" "'.join(sys.argv[1:]) if len(sys.argv) > 1 else '"two black holes merging"'
    print(f'Generating: {prompt}')
    img = generate_image(prompt, 768, 768)
    out_path = '"test_render.png"'
    img.save(out_path)
    print(f'Saved to {out_path}')

```

28. image_generator_v2.py

1,587 lines | 50.3KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/image_generator_v2.py

```
#!/usr/bin/env python3
"""
Quantum MCAGI Procedural Image Generator v2.0
Massive quality upgrade - HDR tone mapping, volumetric rendering,
physically-based accretion disks, multi-pass bloom, ray-marched effects.
Pure math. No API keys.
"""

import numpy as np
from PIL import Image, ImageDraw, ImageFilter
from scipy import ndimage
from scipy.ndimage import zoom
import re
import math
import hashlib
import logging

logger = logging.getLogger("quantum_image_gen")

def _seed_from_prompt(prompt: str) -> int:
    return int(hashlib.md5(prompt.encode()).hexdigest()[:8], 16)

def _fbm_noise(shape, octaves=6, persistence=0.5, seed=42, lacunarity=2.0):
    rng = np.random.RandomState(seed)
    result = np.zeros(shape, dtype=np.float64)
    amplitude = 1.0
    total_amp = 0.0
    for i in range(octaves):
        freq = lacunarity**i
        base_h = max(4, int(shape[0] / freq) + 2)
        base_w = max(4, int(shape[1] / freq) + 2)
        noise_layer = rng.rand(base_h, base_w)
        noise_layer = ndimage.gaussian_filter(noise_layer, sigma=1.0)
        scaled = zoom(
            noise_layer,
            (shape[0] / noise_layer.shape[0], shape[1] / noise_layer.shape[1]),
            order=3,
        )
        result += scaled * amplitude
        total_amp += amplitude
        amplitude *= persistence
    return result / total_amp

def _domain_warp_noise(shape, octaves=6, persistence=0.5, seed=42, warp_strength=0.5):
    warp_x = _fbm_noise(shape, octaves=3, seed=seed + 1000) - 0.5
    warp_y = _fbm_noise(shape, octaves=3, seed=seed + 2000) - 0.5
    h, w = shape
    x = np.mgrid[0:h, 0:w]
    wx = np.clip((x + warp_x * w * warp_strength).astype(int), 0, w - 1)
    wy = np.clip((y + warp_y * h * warp_strength).astype(int), 0, h - 1)
    base = _fbm_noise(shape, octaves=octaves, persistence=persistence, seed=seed)
    return base[wy, wx]

def _aces_tonemap(img):
    a = 2.51
    b = 0.03
    c = 2.43
    d = 0.59
    e = 0.14
    x = img * 0.6
    return np.clip((x * (a * x + b)) / (x * (c * x + d) + e), 0, 1)

def _multi_bloom(img_array, passes=3):
    result = img_array.astype(np.float64) / 255.0
    for i in range(passes):
        threshold = 0.6 - i * 0.15
        radius = 4 + i * 8
        intensity = 0.4 - i * 0.08
        bright = np.clip(result - threshold, 0, 10)
        blurred = ndimage.gaussian_filter(bright, sigma=radius)
        result += blurred * intensity
    result = _aces_tonemap(result)
    return (result * 255).astype(np.uint8)

def _radial_gradient(shape, cx=None, cy=None):
    h, w = shape
    if cx is None:
        cx = w / 2
    if cy is None:
        cy = h / 2
    y, x = np.mgrid[0:h, 0:w]
    r = np.sqrt((x - cx) ** 2 + (y - cy) ** 2)
    r = r / max(r.max(), 1e-6)
    return r

def _vignette(shape, strength=0.7):
```

```

r = _radial_gradient(shape)
v = 1.0 - (r**2.2) * strength
return np.clip(v, 0, 1)

def _blackbody_color(temp):
    t = temp / 100.0
    if t <= 66:
        r = 1.0
        g = max(
            0, min(1, (99.4708025861 * math.log(max(t, 1)) - 161.1195681661) / 255.0)
        )
    else:
        r = max(0, min(1, (329.698727446 * ((t - 60) ** -0.1332047592)) / 255.0))
        g = max(0, min(1, (288.1221695283 * ((t - 60) ** -0.0755148492)) / 255.0))
    if t >= 66:
        b = 1.0
    elif t <= 19:
        b = 0.0
    else:
        b = max(
            0,
            min(
                1, (138.5177312231 * math.log(max(t - 10, 1)) - 305.0447927307) / 255.0
            ),
        )
    return r, g, b

def _blackbody_array(temps):
    r = np.zeros_like(temps)
    g = np.zeros_like(temps)
    b = np.zeros_like(temps)
    t = temps / 100.0
    low = t <= 66
    high = ~low
    r[low] = 1.0
    safe_t_low = np.clip(t[low], 1, 1e6)
    g[low] = np.clip(
        (99.4708025861 * np.log(safe_t_low) - 161.1195681661) / 255.0, 0, 1
    )
    r[high] = np.clip((329.698727446 * ((t[high] - 60) ** -0.1332047592)) / 255.0, 0, 1)
    g[high] = np.clip(
        (288.1221695283 * ((t[high] - 60) ** -0.0755148492)) / 255.0, 0, 1
    )
    b[t >= 66] = 1.0
    mid = (t > 19) & (t < 66)
    safe_t_mid = np.clip(t[mid] - 10, 1, 1e6)
    b[mid] = np.clip(
        (138.5177312231 * np.log(safe_t_mid) - 305.0447927307) / 255.0, 0, 1
    )
    return np.stack([r, g, b], axis=-1)

def _star_field_v2(shape, density=1200, seed=42):
    rng = np.random.RandomState(seed)
    h, w = shape
    stars = np.zeros((h, w, 3), dtype=np.float64)

    n_faint = int(density * 3)
    fx = rng.randint(0, w, n_faint)
    fy = rng.randint(0, h, n_faint)
    fb = rng.power(0.15, n_faint) * 45
    ft = rng.uniform(3000, 20000, n_faint)
    for i in range(n_faint):
        if 0 <= fy[i] < h and 0 <= fx[i] < w:
            rc, gc, bc = _blackbody_color(ft[i])
            stars[fy[i], fx[i], 0] += fb[i] * rc
            stars[fy[i], fx[i], 1] += fb[i] * gc
            stars[fy[i], fx[i], 2] += fb[i] * bc

    n_med = int(density * 0.3)
    mx = rng.randint(0, w, n_med)
    my = rng.randint(0, h, n_med)
    mb = rng.power(0.25, n_med) * 180
    mt = rng.uniform(3000, 25000, n_med)
    for i in range(n_med):
        x, y = mx[i], my[i]
        rc, gc, bc = _blackbody_color(mt[i])
        lum = mb[i]
        for dy in range(-1, 2):
            for dx in range(-1, 2):
                ny, nx = y + dy, x + dx
                if 0 <= ny < h and 0 <= nx < w:
                    d = abs(dx) + abs(dy)
                    f = [1.0, 0.35, 0.12][min(d, 2)]
                    stars[ny, nx, 0] += lum * rc * f
                    stars[ny, nx, 1] += lum * gc * f
                    stars[ny, nx, 2] += lum * bc * f

    n_bright = int(density * 0.04)
    bx = rng.randint(0, w, n_bright)
    by = rng.randint(0, h, n_bright)
    bb = rng.uniform(200, 400, n_bright)
    bt = rng.uniform(4000, 30000, n_bright)
    for i in range(n_bright):
        x, y = bx[i], by[i]
        rc, gc, bc = _blackbody_color(bt[i])
        lum = bb[i]
        size = rng.randint(2, 4)
        for dy in range(-size, size + 1):

```

```

        for dx in range(-size, size + 1):
            ny, nx = y + dy, x + dx
            if 0 <= ny <= h and 0 <= nx <= w:
                d = math.sqrt(dx * dx + dy * dy)
                f = max(0, 1.0 - d / (size + 0.3)) ** 2.5
                stars[ny, nx, 0] += lum * rc * f
                stars[ny, nx, 1] += lum * gc * f
                stars[ny, nx, 2] += lum * bc * f
    spike_len = rng.randint(5, 15)
    for d in range(1, spike_len):
        f = max(0, 1.0 - d / spike_len) ** 2 * 0.25
        for sdx, sdy in [(d, 0), (-d, 0), (0, d), (0, -d)]:
            ny, nx = y + sdy, x + sdx
            if 0 <= ny <= h and 0 <= nx <= w:
                stars[ny, nx, 0] += lum * rc * f
                stars[ny, nx, 1] += lum * gc * f
                stars[ny, nx, 2] += lum * bc * f

dust = _fbm_noise((h, w), octaves=4, persistence=0.5, seed=seed + 7777)
dust_dim = np.clip(dust * 0.6 + 0.4, 0.3, 1.0)
for c in range(3):
    stars[:, :, c] *= dust_dim

glow = ndimage.gaussian_filter(stars, sigma=0.6)
stars = stars + glow * 0.2
return np.clip(stars, 0, 500)

def _color_map(value, palette):
    value = np.clip(value, 0, 1)
    n = len(palette) - 1
    idx = value * n
    low = np.floor(idx).astype(int)
    low = np.clip(low, 0, n - 1)
    high = np.clip(low + 1, 0, n)
    frac = idx - low
    result = np.zeros((*value.shape, 3), dtype=np.float64)
    for c in range(3):
        low_vals = np.array([palette[i][c] for i in range(len(palette))])
        result[:, :, c] = low_vals[low] * (1 - frac) + low_vals[high] * frac
    return result

ACCRETION_PALETTE = [
    (0, 0, 0),
    (40, 5, 2),
    (120, 20, 5),
    (200, 60, 10),
    (255, 120, 30),
    (255, 180, 60),
    (255, 220, 140),
    (255, 245, 220),
    (240, 250, 255),
]

ACCRETION_PALETTE_COOL = [
    (0, 0, 0),
    (20, 5, 30),
    (60, 10, 60),
    (120, 30, 80),
    (180, 60, 100),
    (220, 100, 120),
    (255, 160, 160),
    (255, 220, 200),
    (255, 245, 240),
]

def render_black_hole(w, h, seed, params=None):
    params = params or {}
    rng = np.random.RandomState(seed)
    img = np.zeros((h, w, 3), dtype=np.float64)

    stars = _star_field_v2((h, w), density=1500, seed=seed)

    cx, cy = w / 2, h / 2
    bh_radius = min(w, h) * 0.12
    photon_ring = bh_radius * 1.5
    isco = bh_radius * 3.0
    accretion_outer = bh_radius * 5.0
    accretion_inner = isco

    y, x = np.mgrid[0:h, 0:w]
    dx = (x - cx).astype(np.float64)
    dy = (y - cy).astype(np.float64)
    r = np.sqrt(dx**2 + dy**2)
    theta = np.arctan2(dy, dx)

    lensing_strength = 3.0
    lens_factor = np.where(
        r > bh_radius * 0.9,
        1.0 + lensing_strength * (bh_radius / np.maximum(r, 1)) ** 2,
        0.0,
    )
    secondary_lens = np.where(
        r > bh_radius * 1.2, 0.3 * (bh_radius / np.maximum(r, 1)) ** 3, 0.0
    )
    lens_factor += secondary_lens

    lensed_x = np.clip((cx + dx * lens_factor).astype(int), 0, w - 1)
    lensed_y = np.clip((cy + dy * lens_factor).astype(int), 0, h - 1)

```

```

for c in range(3):
    img[..., c] = stars[lensed_y, lensed_x, c]

tilt = params.get("tilt", 0.25)
cos_tilt = math.cos(tilt * math.pi)
disk_dy = dy * cos_tilt
disk_r = np.sqrt(dx**2 + disk_dy**2)

disk_noise1 = _fbm_noise((h, w), octaves=6, persistence=0.6, seed=seed + 1)
disk_noise2 = _fbm_noise((h, w), octaves=4, persistence=0.5, seed=seed + 2)
disk_noise3 = _domain_warp_noise(
    (h, w), octaves=5, seed=seed + 3, warp_strength=0.15
)

spiral = (
    np.sin(theta * 3 + np.log(np.maximum(disk_r, 1)) * 4 + disk_noise2 * 2) * 0.5
    + 0.5
)

disk_thickness = bh_radius * 0.12
thin_disk = np.exp(-(disk_dy**2) / (2 * disk_thickness**2))

inner_thick = np.exp(-(disk_dy**2) / (2 * (bh_radius * 0.25)**2))
puff = np.clip(1.0 - (disk_r - accretion_inner) / (bh_radius * 2), 0, 1)
thin_disk = thin_disk * (1 - puff) + inner_thick * puff

disk_valid = ((disk_r > accretion_inner * 0.8) & (disk_r < accretion_outer)).astype(
    float
)
outer_fade = np.clip((accretion_outer - disk_r) / (accretion_outer * 0.3), 0, 1)
inner_fade = np.clip(
    (disk_r - accretion_inner * 0.7) / (accretion_inner * 0.3), 0, 1
)
disk_valid *= outer_fade * inner_fade

disk_temp_base = np.clip(
    1.0 - (disk_r - accretion_inner) / (accretion_outer - accretion_inner), 0, 1
)
disk_temp = disk_temp_base**0.5

doppler = 1.0 + 0.5 * np.sin(theta)
relativistic = np.clip(
    1.0 / (1.0 + 0.3 * (bh_radius / np.maximum(disk_r, 1))**0.5), 0.3, 1.0
)
disk_temp_shifted = np.clip(disk_temp * doppler * relativistic, 0, 1)

disk_detail = (0.5 + 0.3 * disk_noise1 + 0.2 * spiral) * (0.7 + 0.3 * disk_noise3)

disk_palette = np.array(ACCRETION_PALETTE, dtype=np.float64)
disk_colors = _color_map(disk_temp_shifted, ACCRETION_PALETTE)

disk_intensity = thin_disk * disk_valid * disk_detail * 1.0

for c in range(3):
    img[..., c] += disk_colors[..., c] * disk_intensity * 0.6

glow_disk = ndimage.gaussian_filter(
    np.stack([disk_colors[..., c] * disk_intensity for c in range(3)], axis=-1),
    sigma=min(w, h) * 0.025,
)
for c in range(3):
    img[..., c] += glow_disk[..., c] * 25

wide_glow = ndimage.gaussian_filter(
    np.stack([disk_colors[..., c] * disk_intensity for c in range(3)], axis=-1),
    sigma=min(w, h) * 0.07,
)
for c in range(3):
    img[..., c] += wide_glow[..., c] * 8

for ring_r, ring_w, ring_int in [
    (photon_ring, 0.04, 0.8),
    (photon_ring * 1.015, 0.015, 0.4),
    (photon_ring * 0.985, 0.015, 0.4),
]:
    ring_mask = np.exp(-((r - ring_r)**2) / (2 * (bh_radius * ring_w)**2))
    ring_noise = 0.7 + 0.3 * _fbm_noise((h, w), octaves=3, seed=seed + 10)
    for c, val in enumerate([255, 160, 40]):
        img[..., c] += val * ring_mask * ring_int * ring_noise * 0.5

inner_glow_r = bh_radius * 1.6
inner_glow = np.exp(-((r - inner_glow_r)**2) / (2 * (bh_radius * 0.35)**2))
inner_glow *= thin_disk * 0.3
for c, val in enumerate([255, 140, 30]):
    img[..., c] += val * inner_glow * 0.3

event_fade = np.clip((r - bh_radius * 0.85) / (bh_radius * 0.15), 0, 1)
for c in range(3):
    img[..., c] *= event_fade

jets = params.get("jets", True)
if jets:
    jet_width_base = bh_radius * 0.04
    jet_length = min(h, w) * 0.4
    jet_noise = _fbm_noise((h, w), octaves=4, seed=seed + 20)

    for sign in [-1, 1]:
        jy_dist = sign * (y - cy)
        jy_mask = (jy_dist > 0).astype(float)
        dist_from_bh = np.abs(y - cy)
        intensity = np.clip(1.0 - dist_from_bh / jet_length, 0, 1)**2.5 * jy_mask

```

```

        spread = jet_width_base * (1 + 2.5 * dist_from_bh / jet_length)
        jet_profile = np.exp(-(dx**2) / (2 * spread**2))
        jet_core = np.exp(-(dx**2) / (2 * (spread * 0.25) ** 2))

        jet_detail = 0.6 + 0.4 * jet_noise
        jet_val = (jet_profile * 0.3 + jet_core * 0.7) * intensity * jet_detail

        img[..., 0] += 50 * jet_val
        img[..., 1] += 90 * jet_val
        img[..., 2] += 180 * jet_val

n_debris = rng.randint(20, 50)
for _ in range(n_debris):
    dr = rng.uniform(accretion_inner * 0.5, accretion_outer * 1.3)
    da = rng.uniform(0, 2 * math.pi)
    dx_d = dr * math.cos(da)
    dy_d = dr * math.sin(da) * cos_tilt
    px, py = int(cx + dx_d), int(cy + dy_d)
    if 0 <= px < w and 0 <= py < h:
        streak_len = rng.randint(2, 8)
        streak_angle = da + math.pi / 2
        brightness = rng.uniform(60, 200)
        for s in range(streak_len):
            sx = int(px + s * math.cos(streak_angle))
            sy = int(py + s * math.sin(streak_angle))
            if 0 <= sx < w and 0 <= sy < h:
                falloff = 1.0 - s / streak_len
                img[sy, sx, 0] += brightness * falloff
                img[sy, sx, 1] += brightness * 0.7 * falloff
                img[sy, sx, 2] += brightness * 0.3 * falloff

vig = _vignette((h, w), 0.6)
for c in range(3):
    img[..., c] *= vig

img = np.clip(img, 0, 600)
img = img / 600.0
img = _aces_tonemap(img * 1.5)
img = (img * 255).astype(np.uint8)
img = _multi_bloom(img, passes=3)
return img

def _render_merging_black_holes(w, h, seed, params=None):
    rng = np.random.RandomState(seed)
    img = np.zeros((h, w, 3), dtype=np.float64)
    stars = _star_field_v2((h, w), density=1200, seed=seed)

    separation = min(w, h) * 0.22
    cx1, cy1 = w / 2 - separation * 0.5, h / 2
    cx2, cy2 = w / 2 + separation * 0.5, h / 2

    y, x = np.mgrid[0:h, 0:w]

    combined_lens = np.ones((h, w), dtype=np.float64)

    for idx, (bcx, bcy, bseed) in enumerate([(cx1, cy1, seed), (cx2, cy2, seed + 500)]):
        bdx = (x - bcx).astype(np.float64)
        bdy = (y - bcy).astype(np.float64)
        br = np.sqrt(bdx**2 + bdy**2)
        btheta = np.arctan2(bdy, bdx)

        bh_r = min(w, h) * 0.07
        photon_r = bh_r * 1.5
        accr_outer = bh_r * 3.5
        accr_inner = bh_r * 1.8

        lens = np.where(
            br > bh_r * 0.9, 1.0 + 2.5 * (bh_r / np.maximum(br, 1)) ** 2, 0.0
        )
        lx = np.clip((bcx + bdx * lens).astype(int), 0, w - 1)
        ly = np.clip((bcy + bdy * lens).astype(int), 0, h - 1)

        weight = 0.6
        for c in range(3):
            img[..., c] += stars[ly, lx, c] * weight

    disk_noise = _fbm_noise((h, w), octaves=5, persistence=0.6, seed=bseed + 1)
    spiral = (
        np.sin(btheta * 2.5 + np.log(np.maximum(br, 1)) * 3 + disk_noise * 1.5)
        * 0.5
        + 0.5
    )

    thin_disk = np.exp(-(bdy**2) / (2 * (bh_r * 0.1) ** 2))
    disk_valid = ((br > accr_inner) & (br < accr_outer)).astype(float)
    outer_fade = np.clip((accr_outer - br) / (accr_outer * 0.3), 0, 1)
    inner_fade = np.clip((br - accr_inner * 0.8) / (accr_inner * 0.2), 0, 1)
    disk_valid *= outer_fade * inner_fade

    disk_temp = (
        np.clip(1.0 - (br - accr_inner) / (accr_outer - accr_inner), 0, 1) ** 0.5
    )
    doppler = 1.0 + 0.4 * np.sin(btheta)
    disk_temp_shifted = np.clip(disk_temp * doppler, 0, 1)

    disk_detail = 0.5 + 0.3 * disk_noise + 0.2 * spiral
    disk_colors = _color_map(disk_temp_shifted, ACCRETION_PALETTE)
    disk_intensity = thin_disk * disk_valid * disk_detail * 1.5

    for c in range(3):

```

```

        img[..., c] += disk_colors[..., c] * disk_intensity * 0.8
    disk_glow_layer = np.stack(
        [disk_colors[..., c] * disk_intensity for c in range(3)], axis=-1
    )
    disk_glow_layer = ndimage.gaussian_filter(
        disk_glow_layer, sigma=min(w, h) * 0.018
    )
    for c in range(3):
        img[..., c] += disk_glow_layer[..., c] * 35

    wide_glow_m = ndimage.gaussian_filter(disk_glow_layer, sigma=min(w, h) * 0.05)
    for c in range(3):
        img[..., c] += wide_glow_m[..., c] * 12

    for ring_off, ring_i in [(1.0, 0.9), (1.015, 0.45), (0.985, 0.45)]:
        ring = np.exp(-((br - photon_r * ring_off) ** 2) / (2 * (bh_r * 0.04) ** 2))
        for c, val in enumerate([255, 180, 50]):
            img[..., c] += ring * val * ring_i * 0.6

    fade = np.clip((br - bh_r * 0.85) / (bh_r * 0.15), 0, 1)
    for c in range(3):
        img[..., c] *= fade

    mid_x, mid_y = w / 2, h / 2
    bridge_dx = (x - mid_x).astype(np.float64)
    bridge_dy = (y - mid_y).astype(np.float64)
    bridge_r = np.sqrt(bridge_dx**2 + bridge_dy**2)

    bridge_noise = _fbm_noise((h, w), octaves=5, seed=seed + 100)
    bridge = np.exp(-(bridge_dy**2) / (2 * (min(h, w) * 0.025) ** 2))
    bridge *= (np.abs(bridge_dx) < separation * 0.65).astype(float)
    bridge_fade = np.exp(-np.abs(bridge_dx) / (separation * 0.4))
    bridge *= bridge_fade
    bridge *= 0.5 + 0.5 * bridge_noise

    gw_waves = np.sin(bridge_r / (min(w, h) * 0.02) * math.pi) * 0.5 + 0.5
    bridge *= 0.6 + 0.4 * gw_waves

    for c, val in enumerate([200, 140, 255]):
        img[..., c] += bridge * val * 0.5

    gw_ring_noise = _fbm_noise((h, w), octaves=3, seed=seed + 200)
    for gw_mult in [0.5, 0.8, 1.2, 1.6]:
        gw_r = separation * gw_mult
        gw_ring = np.exp(-((bridge_r - gw_r) ** 2) / (2 * (min(w, h) * 0.008) ** 2))
        gw_ring *= 0.15 * (0.5 + 0.5 * gw_ring_noise)
        for c, val in enumerate([150, 120, 200]):
            img[..., c] += gw_ring * val

    n_debris = rng.randint(30, 60)
    for _ in range(n_debris):
        which = rng.choice([0, 1])
        bcx = cx1 if which == 0 else cx2
        bcy = cy1 if which == 0 else cy2
        bh_r = min(w, h) * 0.07
        dr = rng.uniform(bh_r * 1.5, bh_r * 4)
        da = rng.uniform(0, 2 * math.pi)
        px = int(bcx + dr * math.cos(da))
        py = int(bcy + dr * math.sin(da) * 0.3)
        if 0 <= px < w and 0 <= py < h:
            streak_len = rng.randint(2, 6)
            sa = da + math.pi / 2
            bright = rng.uniform(40, 160)
            for s in range(streak_len):
                sx = int(px + s * math.cos(sa))
                sy = int(py + s * math.sin(sa))
                if 0 <= sx < w and 0 <= sy < h:
                    f = 1.0 - s / streak_len
                    img[sy, sx, 0] += bright * f
                    img[sy, sx, 1] += bright * 0.6 * f
                    img[sy, sx, 2] += bright * 0.3 * f

    vig = _vignette((h, w), 0.55)
    for c in range(3):
        img[..., c] *= vig

    img = np.clip(img, 0, 600)
    img = img / 600.0
    img = _aces_tonemap(img * 1.4)
    img = (img * 255).astype(np.uint8)
    img = _multi_bloom(img, passes=3)
    return img

```

```

NEBULA_PALETTES = {
    "fire": [
        (0, 0, 0),
        (60, 5, 2),
        (140, 25, 5),
        (220, 70, 12),
        (255, 140, 35),
        (255, 200, 80),
        (255, 240, 180),
    ],
    "ice": [
        (0, 0, 8),
        (8, 15, 50),
        (20, 45, 120),
        (50, 100, 180),
        (100, 170, 230),
        (180, 220, 255),
    ],
}

```

```

        (230, 245, 255),
    ],
    &quot;cosmic&quot;;: [
        (0, 0, 3),
        (30, 3, 50),
        (80, 15, 100),
        (140, 35, 130),
        (190, 70, 160),
        (230, 130, 200),
        (255, 200, 255),
    ],
    &quot;emerald&quot;;: [
        (0, 3, 0),
        (5, 30, 12),
        (15, 80, 35),
        (40, 150, 65),
        (90, 210, 110),
        (160, 240, 170),
        (220, 255, 230),
    ],
    &quot;plasma&quot;;: [
        (0, 0, 0),
        (50, 0, 70),
        (100, 0, 140),
        (170, 40, 110),
        (240, 100, 50),
        (255, 180, 40),
        (255, 240, 160),
    ],
    &quot;void&quot;;: [
        (0, 0, 0),
        (10, 3, 25),
        (25, 8, 45),
        (45, 12, 65),
        (70, 25, 90),
        (100, 40, 120),
        (140, 70, 160),
    ],
    ],
}

DUAL_TONE_PAIRS = {
    &quot;cosmic&quot;;: (&quot;fire&quot;;, &quot;ice&quot;;),
    &quot;fire&quot;;: (&quot;fire&quot;;, &quot;plasma&quot;;),
    &quot;ice&quot;;: (&quot;ice&quot;;, &quot;cosmic&quot;;),
    &quot;emerald&quot;;: (&quot;emerald&quot;;, &quot;ice&quot;;),
    &quot;plasma&quot;;: (&quot;plasma&quot;;, &quot;fire&quot;;),
    &quot;void&quot;;: (&quot;void&quot;;, &quot;cosmic&quot;;),
}

def render_nebula(w, h, seed, params=None):
    params = params or {}
    palette_name = params.get(&quot;palette&quot;;, &quot;cosmic&quot;;)
    rng = np.random.RandomState(seed)

    warm_name, cool_name = DUAL_TONE_PAIRS.get(palette_name, (&quot;fire&quot;;, &quot;ice&quot;;))
    warm_palette = NEBULA_PALETTES[warm_name]
    cool_palette = NEBULA_PALETTES[cool_name]

    stars = _star_field_v2((h, w), density=2000, seed=seed)
    img = stars.copy()
    y_grid, x_grid = np.mgrid[0:h, 0:w]

    grad_noise = _domain_warp_noise(
        (h, w), octaves=5, seed=seed + 9000, warp_strength=0.3
    )
    spatial_grad = (x_grid / w) * 0.5 + grad_noise * 0.5
    spatial_grad = (spatial_grad - spatial_grad.min()) / (
        spatial_grad.max() - spatial_grad.min() + 1e-8
    )
    spatial_grad = np.clip((spatial_grad - 0.3) * 2.5, 0, 1)
    spatial_grad = ndimage.gaussian_filter(spatial_grad, sigma=min(w, h) * 0.03)

    cloud_base = np.zeros((h, w), dtype=np.float64)
    n_centers = rng.randint(4, 9)
    for i in range(n_centers):
        cx = rng.uniform(w * -0.1, w * 1.1)
        cy = rng.uniform(h * -0.1, h * 1.1)
        spread_x = rng.uniform(w * 0.1, w * 0.4)
        spread_y = rng.uniform(h * 0.1, h * 0.4)
        amp = rng.uniform(0.4, 1.0)
        angle = rng.uniform(0, math.pi)
        cos_a, sin_a = math.cos(angle), math.sin(angle)
        ddx = x_grid - cx
        ddy = y_grid - cy
        rx = ddx * cos_a + ddy * sin_a
        ry = -ddx * sin_a + ddy * cos_a
        blob = np.exp(-(rx**2) / (2 * spread_x**2) - (ry**2) / (2 * spread_y**2))
        cloud_base += blob * amp

    cloud_base = cloud_base / (cloud_base.max() + 1e-8)
    cloud_base = ndimage.gaussian_filter(cloud_base, sigma=min(w, h) * 0.03)

    detail1 = _domain_warp_noise(
        (h, w), octaves=5, persistence=0.5, seed=seed + 10, warp_strength=0.25
    )
    detail1 = ndimage.gaussian_filter(detail1, sigma=min(w, h) * 0.01)
    detail2 = _domain_warp_noise(
        (h, w), octaves=4, persistence=0.45, seed=seed + 15, warp_strength=0.2
    )

```

```

detail2 = ndimage.gaussian_filter(detail2, sigma=min(w, h) * 0.015)
fine = _fbm_noise((h, w), octaves=6, persistence=0.55, seed=seed + 20)
fine = ndimage.gaussian_filter(fine, sigma=min(w, h) * 0.008)

warp_u = _fbm_noise((h, w), octaves=4, seed=seed + 300)
warp_v = _fbm_noise((h, w), octaves=4, seed=seed + 310)
warp_s = min(w, h) * 0.18
wx = np.clip((x_grid + (warp_u - 0.5) * warp_s).astype(int), 0, w - 1)
wy = np.clip((y_grid + (warp_v - 0.5) * warp_s).astype(int), 0, h - 1)

combined = cloud_base[wy, wx] * (0.4 + 0.35 * detail1 + 0.25 * detail2)
combined *= 0.5 + 0.3 * fine + 0.2
combined = (combined - combined.min()) / (combined.max() - combined.min() + 1e-8)
combined = combined**0.5

smooth_cloud = ndimage.gaussian_filter(combined, sigma=min(w, h) * 0.025)
medium_detail = ndimage.gaussian_filter(combined, sigma=min(w, h) * 0.008)

contrast_cloud = np.clip((smooth_cloud - 0.2) * 3.0, 0, 1) ** 0.65

warm_smooth = _color_map(contrast_cloud, warm_palette)
cool_smooth = _color_map(contrast_cloud, cool_palette)
warm_detail = _color_map(medium_detail, warm_palette)
cool_detail = _color_map(medium_detail, cool_palette)

glow_density = np.clip(contrast_cloud * 1.8, 0, 1)
for c in range(3):
    smooth_col = (
        warm_smooth[..., c] * (1 - spatial_grad)
        + cool_smooth[..., c] * spatial_grad
    )
    img[..., c] = (
        img[..., c] * (1 - glow_density * 0.8) + smooth_col * glow_density * 2.0
    )

detail_mask = (contrast_cloud > 0.05).astype(float)
detail_density = np.clip(medium_detail * 2.0 - 0.15, 0, 1) * detail_mask
for c in range(3):
    detail_col = (
        warm_detail[..., c] * (1 - spatial_grad)
        + cool_detail[..., c] * spatial_grad
    )
    img[..., c] += detail_col * detail_density * 1.0

layer2_noise = _domain_warp_noise(
    (h, w), octaves=5, seed=seed + 100, warp_strength=0.2
)
layer2_smooth = ndimage.gaussian_filter(layer2_noise, sigma=min(w, h) * 0.025)
layer2_density = np.clip(layer2_smooth - 0.2, 0, 1) * np.clip(
    contrast_cloud * 2.0, 0, 1
)
layer2_colors_w = _color_map(layer2_smooth, warm_palette)
layer2_colors_c = _color_map(layer2_smooth, cool_palette)
for c in range(3):
    l2_col = (
        layer2_colors_w[..., c] * (1 - spatial_grad)
        + layer2_colors_c[..., c] * spatial_grad
    )
    img[..., c] += l2_col * layer2_density * 1.5

third_name = {
    "fire": "plasma",
    "ice": "emerald",
    "cosmic": "void",
    "emerald": "cosmic",
    "plasma": "cosmic",
    "void": "fire",
}.get(palette_name, "cosmic")
third_palette = NEBULA_PALETTES[third_name]
layer3_noise = _domain_warp_noise(
    (h, w), octaves=4, seed=seed + 200, warp_strength=0.15
)
layer3_smooth = ndimage.gaussian_filter(layer3_noise, sigma=min(w, h) * 0.03)
layer3_density = (
    np.clip(layer3_smooth - 0.3, 0, 1) * np.clip(contrast_cloud * 1.5, 0, 1) * 0.7
)
layer3_colors = _color_map(layer3_smooth, third_palette)
for c in range(3):
    img[..., c] += layer3_colors[..., c] * layer3_density * 1.0

for fil_seed, fil_thresh, fil_bright in [
    (seed + 400, 30, 0.7),
    (seed + 410, 40, 0.5),
    (seed + 420, 50, 0.3),
]:
    fil = _fbm_noise((h, w), octaves=7, persistence=0.7, seed=fil_seed)
    filaments = np.exp(-(np.abs(fil - 0.5) * fil_thresh)) * fil_bright
    filaments *= (contrast_cloud > 0.05).astype(float)
    filaments = ndimage.gaussian_filter(filaments, sigma=1.0)
    fil_grad = spatial_grad if fil_seed % 2 == 0 else (1 - spatial_grad)
    for c in range(3):
        bright_col = (
            warm_palette[-1][c] * (1 - fil_grad) + cool_palette[-1][c] * fil_grad
        )
        img[..., c] += filaments * bright_col * 0.8

for dk_seed, dk_width in [(seed + 600, 0.07), (seed + 610, 0.09)]:
    dark_fil = _fbm_noise((h, w), octaves=6, persistence=0.6, seed=dk_seed)
    dark_mask = (np.abs(dark_fil - 0.5) < dk_width).astype(float) * (
        contrast_cloud > 0.12
    ).astype(float)

```

```

        dark_mask = ndimage.gaussian_filter(dark_mask, sigma=min(w, h) * 0.008)
        for c in range(3):
            img[..., c] *= 1.0 - dark_mask * 0.6

    edge_detect = np.abs(
        ndimage.gaussian_filter(smooth_cloud, sigma=3)
        - ndimage.gaussian_filter(smooth_cloud, sigma=8)
    )
    edge_detect = edge_detect / (edge_detect.max() + 1e-8)
    edge_bright = edge_detect * (contrast_cloud > 0.08).astype(float)
    edge_bright = ndimage.gaussian_filter(edge_bright, sigma=1.5)
    for c in range(3):
        bright_edge = (
            warm_palette[-1][c] * (1 - spatial_grad)
            + cool_palette[-1][c] * spatial_grad
        )
        img[..., c] += edge_bright * bright_edge * 0.5

    bright_noise = _fbm_noise((h, w), octaves=3, seed=seed + 500)
    hot = np.clip(bright_noise - 0.55, 0, 1) * 5 * (contrast_cloud > 0.15).astype(float)
    hot = ndimage.gaussian_filter(hot, sigma=min(w, h) * 0.02)
    hot_wide = ndimage.gaussian_filter(hot, sigma=min(w, h) * 0.06)
    for c in range(3):
        bright_col = (
            warm_palette[-1][c] * (1 - spatial_grad)
            + cool_palette[-1][c] * spatial_grad
        )
        img[..., c] += hot * bright_col / 255.0 * 250
        img[..., c] += hot_wide * bright_col / 255.0 * 100

    n_stars_embedded = rng.randint(8, 18)
    for _ in range(n_stars_embedded):
        sx, sy = rng.randint(0, w), rng.randint(0, h)
        if contrast_cloud[min(sy, h - 1), min(sx, w - 1)] > 0.1:
            sr = rng.uniform(min(w, h) * 0.005, min(w, h) * 0.018)
            glow = np.exp(-(x_grid - sx) ** 2 + (y_grid - sy) ** 2) / (2 * sr ** 2)
            star_bright = rng.uniform(0.4, 1.0)
            wide = np.exp(
                -((x_grid - sx) ** 2 + (y_grid - sy) ** 2) / (2 * (sr * 5) ** 2)
            )
            for c in range(3):
                img[..., c] += glow * 300 * star_bright
                local_col = warm_palette[-2][c] * (1 - sx / w) + cool_palette[-2][c] * (
                    sx / w
                )
                img[..., c] += wide * local_col * 0.12 * star_bright

    vig = _vignette((h, w), 0.5)
    for c in range(3):
        img[..., c] *= vig

    img = np.clip(img, 0, 600)
    img = img / 600.0
    img = _aces_tonemap(img * 1.8)

    lum = 0.2126 * img[..., 0] + 0.7152 * img[..., 1] + 0.0722 * img[..., 2]
    lum = lum[..., np.newaxis]
    sat_boost = 1.6
    img = lum + (img - lum) * sat_boost
    img = np.clip(img, 0, 1)

    img = (img * 255).astype(np.uint8)
    img = _multi_bloom(img, passes=3)
    return img

def _v2_post_process(img_array, bloom_passes=2):
    img = np.clip(img_array.astype(np.float64), 0, 500)
    vig = _vignette(img.shape[:2], 0.45)
    for c in range(3):
        img[..., c] *= vig
    img = img / 500.0
    img = _aces_tonemap(img * 1.2)
    img = (img * 255).astype(np.uint8)
    img = _multi_bloom(img, passes=bloom_passes)
    return img

def _import_v1_renderers():
    from image_generator import (
        render_galaxy,
        render_quantum_state,
        render_wormhole,
        render_supernova,
        render_planet,
        render_fractal,
        render_neural,
        render_consciousness,
        _detect_scene,
        RENDERERS as V1_RENDERERS,
    )

    return {
        "galaxy": render_galaxy,
        "quantum_state": render_quantum_state,
        "wormhole": render_wormhole,
        "supernova": render_supernova,
        "planet": render_planet,
        "fractal": render_fractal,
        "neural": render_neural,
        "consciousness": render_consciousness,
    }

```

```

    }, _detect_scene

RENDERERS_V2 = {
    &quot;black_hole&quot;: render_black_hole,
    &quot;nebula&quot;: render_nebula,
}

MOOD_COLORS = {
    &quot;fire&quot;: {
        &quot;keywords&quot;: [
            &quot;fire&quot;;,
            &quot;flame&quot;;,
            &quot;burn&quot;;,
            &quot;hot&quot;;,
            &quot;lava&quot;;,
            &quot;magma&quot;;,
            &quot;inferno&quot;;,
            &quot;dragon&quot;;,
            &quot;phoenix&quot;;,
            &quot;hell&quot;;,
            &quot;volcanic&quot;;,
            &quot;blaze&quot;;,
            &quot;ember&quot;;,
            &quot;molten&quot;;,
        ],
        &quot;palette&quot;: &quot;fire&quot;;,
    },
    &quot;ice&quot;: {
        &quot;keywords&quot;: [
            &quot;ice&quot;;,
            &quot;cold&quot;;,
            &quot;frozen&quot;;,
            &quot;frost&quot;;,
            &quot;winter&quot;;,
            &quot;snow&quot;;,
            &quot;arctic&quot;;,
            &quot;glacier&quot;;,
            &quot;crystal&quot;;,
            &quot;chill&quot;;,
            &quot;freeze&quot;;,
        ],
        &quot;palette&quot;: &quot;ice&quot;;,
    },
    &quot;emerald&quot;: {
        &quot;keywords&quot;: [
            &quot;green&quot;;,
            &quot;forest&quot;;,
            &quot;nature&quot;;,
            &quot;emerald&quot;;,
            &quot;life&quot;;,
            &quot;growth&quot;;,
            &quot;plant&quot;;,
            &quot;tree&quot;;,
            &quot;leaf&quot;;,
            &quot;garden&quot;;,
            &quot;jungle&quot;;,
            &quot;bio&quot;;,
            &quot;organic&quot;;,
            &quot;alien&quot;;,
        ],
        &quot;palette&quot;: &quot;emerald&quot;;,
    },
    &quot;cosmic&quot;: {
        &quot;keywords&quot;: [
            &quot;purple&quot;;,
            &quot;violet&quot;;,
            &quot;cosmic&quot;;,
            &quot;mystic&quot;;,
            &quot;magic&quot;;,
            &quot;wizard&quot;;,
            &quot;spell&quot;;,
            &quot;ethereal&quot;;,
            &quot;spirit&quot;;,
            &quot;soul&quot;;,
            &quot;astral&quot;;,
            &quot;psychedelic&quot;;,
        ],
        &quot;palette&quot;: &quot;cosmic&quot;;,
    },
    &quot;plasma&quot;: {
        &quot;keywords&quot;: [
            &quot;energy&quot;;,
            &quot;electric&quot;;,
            &quot;lightning&quot;;,
            &quot;power&quot;;,
            &quot;plasma&quot;;,
            &quot;thunder&quot;;,
            &quot;storm&quot;;,
            &quot;volt&quot;;,
            &quot;spark&quot;;,
            &quot;charge&quot;;,
            &quot;neon&quot;;,
            &quot;glow&quot;;,
        ],
        &quot;palette&quot;: &quot;plasma&quot;;,
    },
    &quot;void&quot;: {
        &quot;keywords&quot;: [
            &quot;dark&quot;;,

```

```

        &quot;void&quot;;,
        &quot;shadow&quot;;,
        &quot;death&quot;;,
        &quot;abyss&quot;;,
        &quot;deep&quot;;,
        &quot;night&quot;;,
        &quot;horror&quot;;,
        &quot;fear&quot;;,
        &quot;demon&quot;;,
        &quot;evil&quot;;,
        &quot;nightmare&quot;;,
    ],
    &quot;palette&quot;: &quot;void&quot;;,
},
&quot;warm&quot;: {
    &quot;keywords&quot;: [
        &quot;sunset&quot;;,
        &quot;sunrise&quot;;,
        &quot;dawn&quot;;,
        &quot;dusk&quot;;,
        &quot;warm&quot;;,
        &quot;golden&quot;;,
        &quot;amber&quot;;,
        &quot;orange&quot;;,
        &quot;autumn&quot;;,
        &quot;copper&quot;;,
        &quot;bronze&quot;;,
        &quot;desert&quot;;,
        &quot;sand&quot;;,
    ],
    &quot;palette&quot;: &quot;fire&quot;;,
},
&quot;ocean&quot;: {
    &quot;keywords&quot;: [
        &quot;ocean&quot;;,
        &quot;sea&quot;;,
        &quot;water&quot;;,
        &quot;wave&quot;;,
        &quot;marine&quot;;,
        &quot;aqua&quot;;,
        &quot;underwater&quot;;,
        &quot;deep sea&quot;;,
        &quot;coral&quot;;,
        &quot;tide&quot;;,
        &quot;current&quot;;,
    ],
    &quot;palette&quot;: &quot;ice&quot;;,
},
}

```

```

EXTRA_SCENE_KEYWORDS = {
    &quot;landscape&quot;: [
        &quot;landscape&quot;;,
        &quot;mountain&quot;;,
        &quot;valley&quot;;,
        &quot;terrain&quot;;,
        &quot;canyon&quot;;,
        &quot;cliff&quot;;,
        &quot;sunset&quot;;,
        &quot;sunrise&quot;;,
        &quot;horizon&quot;;,
        &quot;sky&quot;;,
        &quot;cloud&quot;;,
        &quot;weather&quot;;,
        &quot;ocean&quot;;,
        &quot;sea&quot;;,
        &quot;lake&quot;;,
        &quot;river&quot;;,
        &quot;waterfall&quot;;,
        &quot;beach&quot;;,
        &quot;forest&quot;;,
        &quot;desert&quot;;,
        &quot;tundra&quot;;,
        &quot;prairie&quot;;,
        &quot;field&quot;;,
    ],
    &quot;creature&quot;: [
        &quot;dragon&quot;;,
        &quot;creature&quot;;,
        &quot;monster&quot;;,
        &quot;beast&quot;;,
        &quot;leviathan&quot;;,
        &quot;titan&quot;;,
        &quot;serpent&quot;;,
        &quot;phoenix&quot;;,
        &quot;demon&quot;;,
        &quot;angel&quot;;,
        &quot;alien creature&quot;;,
        &quot;wolf&quot;;,
        &quot;eagle&quot;;,
        &quot;whale&quot;;,
        &quot;shark&quot;;,
        &quot;spider&quot;;,
        &quot;insect&quot;;,
    ],
    &quot;structure&quot;: [
        &quot;city&quot;;,
        &quot;building&quot;;,
        &quot;tower&quot;;,
        &quot;castle&quot;;,
    ],
}

```

```

        &quot;temple&quot;;
        &quot;pyramid&quot;;
        &quot;ruins&quot;;
        &quot;architecture&quot;;
        &quot;bridge&quot;;
        &quot;monument&quot;;
        &quot;cathedral&quot;;
        &quot;spaceship&quot;;
        &quot;station&quot;;
        &quot;vessel&quot;;
        &quot;craft&quot;;
        &quot;machine&quot;;
    ],
    &quot;abstract&quot;: [
        &quot;abstract&quot;;
        &quot;pattern&quot;;
        &quot;geometric&quot;;
        &quot;art&quot;;
        &quot;color&quot;;
        &quot;wave&quot;;
        &quot;flow&quot;;
        &quot;swirl&quot;;
        &quot;vortex&quot;;
        &quot;spiral&quot;;
        &quot;symmetry&quot;;
        &quot;tessellation&quot;;
        &quot;dream&quot;;
        &quot;vision&quot;;
        &quot;hallucination&quot;;
        &quot;psychedelic&quot;;
    ],
    &quot;energy&quot;: [
        &quot;energy&quot;;
        &quot;explosion&quot;;
        &quot;blast&quot;;
        &quot;eruption&quot;;
        &quot;shockwave&quot;;
        &quot;lightning&quot;;
        &quot;thunder&quot;;
        &quot;storm&quot;;
        &quot;aurora&quot;;
        &quot;radiation&quot;;
        &quot;particle&quot;;
        &quot;beam&quot;;
        &quot;ray&quot;;
        &quot;flash&quot;;
        &quot;pulse&quot;;
    ],
}

def _detect_mood_palette(prompt):
    prompt_lower = prompt.lower()
    best_score = 0
    best_palette = &quot;cosmic&quot;;
    for mood, info in MOOD_COLORS.items():
        score = sum(len(kw) for kw in info[&quot;keywords&quot;] if kw in prompt_lower)
        if score > best_score:
            best_score = score
            best_palette = info[&quot;palette&quot;]
    return best_palette

def _detect_extra_scene(prompt):
    prompt_lower = prompt.lower()
    best_score = 0
    best_scene = None
    for scene, keywords in EXTRA_SCENE_KEYWORDS.items():
        score = sum(len(kw) for kw in keywords if kw in prompt_lower)
        if score > best_score:
            best_score = score
            best_scene = scene
    return best_scene if best_score > 0 else None

def render_landscape(w, h, seed, params=None):
    params = params or {}
    rng = np.random.RandomState(seed)
    palette_name = _detect_mood_palette(params.get(&quot;_prompt&quot;, &quot;&quot;))
    palette = NEBULA_PALETTES.get(palette_name, NEBULA_PALETTES[&quot;fire&quot;])

    img = np.zeros((h, w, 3), dtype=np.float64)
    y_grid, x_grid = np.mgrid[0:h, 0:w]

    sky_grad = (y_grid / h) ** 1.5
    sky_colors = _color_map(sky_grad, palette[::-1])
    for c in range(3):
        img[..., c] = sky_colors[..., c] * 0.6

    sky_noise = _domain_warp_noise((h, w), octaves=5, seed=seed + 1, warp_strength=0.12)
    cloud_mask = np.clip(sky_noise - 0.35, 0, 1) * 2.0
    cloud_mask *= np.clip(1.0 - y_grid / (h * 0.7), 0, 1)
    cloud_detail = _fbm_noise((h, w), octaves=6, persistence=0.55, seed=seed + 2)
    cloud_mask *= 0.5 + 0.5 * cloud_detail
    cloud_col = np.array(palette[-2], dtype=np.float64) / 255.0
    for c in range(3):
        img[..., c] += cloud_mask * (cloud_col[c] * 200 + 60)

    horizon_y = h * 0.55
    n_ridges = rng.randint(3, 6)
    for i in range(n_ridges):

```

```

ridge_y = horizon_y + i * h * 0.06
ridge_noise = _fbm_noise(
    (1, w), octaves=5, persistence=0.6, seed=seed + 100 + i
)
ridge_heights = ridge_noise[0] * h * 0.15
darkness = 0.3 + 0.15 * i
for x in range(w):
    ry = int(ridge_y - ridge_heights[x])
    if 0 <= ry < h:
        for y in range(ry, h):
            fade = min(1.0, (y - ry) / (h * 0.05 + 1))
            col_idx = min(len(palette) - 1, i + 1)
            for c in range(3):
                img[y, x, c] = (
                    img[y, x, c] * (1 - fade * 0.7)
                    + palette[col_idx][c] * darkness * fade * 0.7
                )

ground = (y_grid > horizon_y + h * 0.1).astype(float)
ground_noise = _fbm_noise(h, w), octaves=6, persistence=0.5, seed=seed + 50)
ground_detail = ground * (0.3 + 0.7 * ground_noise)
for c in range(3):
    img[:, :, c] = (
        img[:, :, c] * (1 - ground * 0.5) + palette[2][c] * ground_detail * 0.4
    )

glow_y = int(horizon_y)
if 0 <= glow_y < h:
    for gy in range(max(0, glow_y - int(h * 0.08)), min(h, glow_y + int(h * 0.08))):
        fade = max(0, 1.0 - abs(gy - glow_y) / (h * 0.08)) ** 2
        for c in range(3):
            img[gy, :, c] += palette[-1][c] * fade * 0.5

n_stars_sky = 200 if palette_name in ['void', 'ice'] else 80
for _ in range(n_stars_sky):
    sx, sy = rng.randint(0, w), rng.randint(0, int(horizon_y * 0.8))
    b = rng.uniform(30, 120)
    if 0 <= sy < h and 0 <= sx < w:
        img[sy, sx] += b

return _v2_post_process(img)

def render_creature(w, h, seed, params=None):
    params = params or {}
    rng = np.random.RandomState(seed)
    palette_name = _detect_mood_palette(params.get('prompt', ''), '&quote;&quote;')
    palette = NEBULA_PALETTES.get(palette_name, NEBULA_PALETTES['fire'])

    img = np.zeros((h, w, 3), dtype=np.float64)
    stars = _star_field_v2((h, w), density=600, seed=seed)
    img += stars * 0.4

    y_grid, x_grid = np.mgrid[0:h, 0:w]
    cx, cy = w * 0.5, h * 0.45

    body_noise = _domain_warp_noise(
        (h, w), octaves=6, seed=seed + 1, warp_strength=0.25
    )
    form = np.zeros((h, w), dtype=np.float64)

    body_rx, body_ry = w * 0.18, h * 0.22
    body = np.exp(
        -((x_grid - cx) ** 2) / (2 * body_rx**2)
        - ((y_grid - cy) ** 2) / (2 * body_ry**2)
    )
    form += body

    head_cx, head_cy = cx + w * 0.12, cy - h * 0.18
    head_r = min(w, h) * 0.09
    head = np.exp(
        -((x_grid - head_cx) ** 2 + (y_grid - head_cy) ** 2) / (2 * head_r**2)
    )
    form += head * 0.8

    for i in range(rng.randint(2, 5)):
        angle = rng.uniform(0, 2 * math.pi)
        length = rng.uniform(min(w, h) * 0.12, min(w, h) * 0.25)
        lx = cx + math.cos(angle) * length * 0.5
        ly = cy + math.sin(angle) * length * 0.5
        lr = rng.uniform(min(w, h) * 0.03, min(w, h) * 0.06)
        limb = np.exp(-((x_grid - lx) ** 2 + (y_grid - ly) ** 2) / (2 * lr**2))
        form += limb * 0.5

    form *= 0.5 + 0.5 * body_noise
    form = np.clip(form, 0, 1)

    creature_colors = _color_map(form, palette)
    for c in range(3):
        img[:, :, c] += creature_colors[:, :, c] * form * 0.8

    edge = np.abs(
        ndimage.gaussian_filter(form, sigma=2) - ndimage.gaussian_filter(form, sigma=4)
    )
    edge = edge / (edge.max() + 1e-8)
    for c in range(3):
        img[:, :, c] += edge * palette[-1][c] * 0.6

    eye_glow = np.exp(
        -((x_grid - head_cx) ** 2 + (y_grid - head_cy) ** 2) / (2 * (head_r * 0.2) ** 2)
    )

```

```

for c in range(3):
    img[..., c] += eye_glow * palette[-1][c] * 1.5

energy_noise = _fbm_noise((h, w), octaves=4, seed=seed + 200)
energy = (energy_noise > 0.65).astype(float) * form
energy = ndimage.gaussian_filter(energy, sigma=3)
for c in range(3):
    img[..., c] += energy * palette[-2][c] * 0.5

aura = ndimage.gaussian_filter(form, sigma=min(w, h) * 0.04)
for c in range(3):
    img[..., c] += aura * palette[3][c] * 0.15

return _v2_post_process(img)

def render_structure(w, h, seed, params=None):
    params = params or {}
    rng = np.random.RandomState(seed)
    palette_name = _detect_mood_palette(params.get(""_prompt"", """"))
    palette = NEBULA_PALETTES.get(palette_name, NEBULA_PALETTES[""cosmic""])

    img = np.zeros((h, w, 3), dtype=np.float64)
    stars = _star_field_v2((h, w), density=800, seed=seed)
    img += stars * 0.5

    y_grid, x_grid = np.mgrid[0:h, 0:w]
    cx, cy = w * 0.5, h * 0.5

    n_towers = rng.randint(3, 8)
    for i in range(n_towers):
        tx = rng.uniform(w * 0.15, w * 0.85)
        tw = rng.uniform(w * 0.04, w * 0.1)
        th = rng.uniform(h * 0.2, h * 0.6)
        base_y = h * rng.uniform(0.55, 0.75)

        tower_mask = (
            (np.abs(x_grid - tx) < tw / 2) & (y_grid > base_y - th) & (y_grid < base_y)
        ).astype(float)
        taper = np.clip(1.0 - (base_y - y_grid) / th, 0, 1)
        taper_width = tw * (0.5 + 0.5 * taper)
        tower_mask = (
            (np.abs(x_grid - tx) < taper_width / 2).astype(float)
            * (y_grid > base_y - th).astype(float)
            * (y_grid < base_y).astype(float)
        )

        detail_noise = _fbm_noise((h, w), octaves=5, seed=seed + 300 + i)
        tower_detail = tower_mask * (0.4 + 0.6 * detail_noise)

        col_idx = min(len(palette) - 1, max(2, i % len(palette)))
        for c in range(3):
            img[..., c] += tower_detail * palette[col_idx][c] * 1.2

        window_noise = _fbm_noise(
            (h, w), octaves=8, persistence=0.7, seed=seed + 400 + i
        )
        windows = (window_noise > 0.55).astype(float) * tower_mask
        for c in range(3):
            img[..., c] += windows * palette[-1][c] * 1.5

        edge_h = np.abs(
            ndimage.gaussian_filter(tower_mask, sigma=1)
            - ndimage.gaussian_filter(tower_mask, sigma=2.5)
        )
        for c in range(3):
            img[..., c] += edge_h * palette[-2][c] * 1.0

    ground_y = h * 0.7
    ground = np.clip((y_grid - ground_y) / (h * 0.15), 0, 1)
    ground_noise = _fbm_noise((h, w), octaves=5, seed=seed + 500)
    for c in range(3):
        img[..., c] += ground * palette[2][c] * 0.6 * (0.5 + 0.5 * ground_noise)

    fog = np.exp(-((y_grid - ground_y) ** 2) / (2 * (h * 0.12) ** 2))
    fog_noise = _fbm_noise((h, w), octaves=3, seed=seed + 600)
    fog *= 0.4 + 0.6 * fog_noise
    for c in range(3):
        img[..., c] += fog * palette[-2][c] * 0.5

    sky_glow = np.clip(1.0 - y_grid / (h * 0.5), 0, 1) ** 2
    sky_noise = _fbm_noise((h, w), octaves=3, seed=seed + 700)
    for c in range(3):
        img[..., c] += sky_glow * palette[3][c] * 0.15 * (0.5 + 0.5 * sky_noise)

    return _v2_post_process(img)

def render_abstract(w, h, seed, params=None):
    params = params or {}
    rng = np.random.RandomState(seed)
    palette_name = _detect_mood_palette(params.get(""_prompt"", """"))
    palette = NEBULA_PALETTES.get(palette_name, NEBULA_PALETTES[""plasma""])

    img = np.zeros((h, w, 3), dtype=np.float64)
    y_grid, x_grid = np.mgrid[0:h, 0:w]

    layer1 = _domain_warp_noise(
        (h, w), octaves=7, persistence=0.6, seed=seed, warp_strength=0.3
    )
    layer2 = _domain_warp_noise(

```

```

    (h, w), octaves=5, persistence=0.5, seed=seed + 100, warp_strength=0.2
)
layer3 = _fbm_noise((h, w), octaves=8, persistence=0.65, seed=seed + 200)

combined = layer1 * 0.5 + layer2 * 0.3 + layer3 * 0.2
combined = (combined - combined.min()) / (combined.max() - combined.min() + 1e-8)

colors1 = _color_map(combined, palette)
for c in range(3):
    img[..., c] = colors1[..., c] * 0.8

sec_map = {
    &quot;fire&quot;: &quot;plasma&quot;,
    &quot;ice&quot;: &quot;cosmic&quot;,
    &quot;cosmic&quot;: &quot;plasma&quot;,
    &quot;emerald&quot;: &quot;ice&quot;,
    &quot;plasma&quot;: &quot;fire&quot;,
    &quot;void&quot;: &quot;cosmic&quot;,
}
sec_palette = NEBULA_PALETTES[sec_map.get(palette_name, &quot;cosmic&quot;)]
phase = _fbm_noise((h, w), octaves=6, seed=seed + 300)
phase_colors = _color_map(phase, sec_palette)
blend = np.clip(layer2 - 0.3, 0, 1) * 0.5
for c in range(3):
    img[..., c] = img[..., c] * (1 - blend) + phase_colors[..., c] * blend

edges = np.abs(
    ndimage.gaussian_filter(combined, sigma=1)
    - ndimage.gaussian_filter(combined, sigma=3)
)
edges = edges / (edges.max() + 1e-8)
for c in range(3):
    img[..., c] += edges * palette[-1][c] * 0.4

voronoi_noise = _fbm_noise((h, w), octaves=3, seed=seed + 400)
cells = (np.abs(voronoi_noise - 0.5) &lt; 0.03).astype(float)
cells = ndimage.gaussian_filter(cells, sigma=1)
for c in range(3):
    img[..., c] += cells * palette[-2][c] * 0.6

sym_x = np.abs(x_grid - w / 2) / (w / 2)
sym_pattern = np.sin(sym_x * 12 + combined * 6) * 0.5 + 0.5
sym_mask = (sym_pattern &gt; 0.7).astype(float) * 0.15
for c in range(3):
    img[..., c] += sym_mask * palette[-1][c]

return _v2_post_process(img)

def render_energy(w, h, seed, params=None):
    params = params or {}
    rng = np.random.RandomState(seed)
    palette_name = _detect_mood_palette(params.get(&quot;_prompt&quot;, &quot;&quot;))
    palette = NEBULA_PALETTES.get(palette_name, NEBULA_PALETTES[&quot;plasma&quot;])

    img = np.zeros((h, w, 3), dtype=np.float64)
    stars = _star_field_v2((h, w), density=500, seed=seed)
    img += stars * 0.3

    y_grid, x_grid = np.mgrid[0:h, 0:w]
    cx, cy = w / 2, h / 2
    r = np.sqrt((x_grid - cx) ** 2 + (y_grid - cy) ** 2)
    theta = np.arctan2(y_grid - cy, x_grid - cx)
    r_norm = r / (min(w, h) * 0.5)

    core_r = min(w, h) * 0.06
    core = np.exp(-(r**2) / (2 * core_r**2))
    core_glow = np.exp(-(r**2) / (2 * (core_r * 3) ** 2))
    for c in range(3):
        img[..., c] += core * palette[-1][c] * 2.0
        img[..., c] += core_glow * palette[-2][c] * 0.4

    n_bolts = rng.randint(6, 14)
    for i in range(n_bolts):
        bolt_angle = rng.uniform(0, 2 * math.pi)
        bolt_length = rng.uniform(min(w, h) * 0.2, min(w, h) * 0.45)
        bolt_noise = _fbm_noise((h, w), octaves=5, seed=seed + 50 + i)

        angle_diff = np.abs(theta - bolt_angle)
        angle_diff = np.minimum(angle_diff, 2 * math.pi - angle_diff)
        bolt_width = 0.04 + 0.03 * bolt_noise
        bolt_mask = np.exp(-(angle_diff**2) / (2 * bolt_width**2))
        bolt_reach = np.clip(1.0 - r / bolt_length, 0, 1) ** 0.3
        bolt_mask *= bolt_reach
        bolt_mask *= (r &gt; core_r * 0.3).astype(float)

        bolt_detail = 0.4 + 0.6 * bolt_noise
        bolt_mask *= bolt_detail

        bolt_core = np.exp(-(angle_diff**2) / (2 * (bolt_width * 0.3) ** 2))
        bolt_core *= bolt_reach * (r &gt; core_r * 0.3).astype(float)
        col_idx = min(len(palette) - 1, 3 + i % 4)
        for c in range(3):
            img[..., c] += bolt_mask * palette[col_idx][c] * 1.2
            img[..., c] += bolt_core * palette[-1][c] * 0.8

    shockwaves = [0.15, 0.3, 0.5, 0.7, 0.9]
    sw_noise = _fbm_noise((h, w), octaves=3, seed=seed + 300)
    for j, sw_r in enumerate(shockwaves):
        ring_r = min(w, h) * sw_r * 0.5
        ring = np.exp(-(r - ring_r) ** 2) / (2 * (min(w, h) * 0.008) ** 2))

```

```

        ring *= 0.5 + 0.5 * sw_noise
        intensity = 0.5 * (1.0 - j * 0.12)
        for c in range(3):
            img[..., c] += ring * palette[-2][c] * intensity

    particle_noise = _domain_warp_noise(
        (h, w), octaves=5, seed=seed + 400, warp_strength=0.18
    )
    particles = np.clip(particle_noise - 0.55, 0, 1) * 2
    particles *= np.clip(1.0 - r_norm * 0.7, 0, 1)
    particles = ndimage.gaussian_filter(particles, sigma=1.2)
    for c in range(3):
        img[..., c] += particles * palette[-1][c] * 0.6

    ambient = _domain_warp_noise((h, w), octaves=4, seed=seed + 500, warp_strength=0.1)
    ambient = np.clip(ambient - 0.3, 0, 1) * np.clip(1.0 - r_norm * 0.8, 0, 1)
    for c in range(3):
        img[..., c] += ambient * palette[3][c] * 0.2

    return _v2_post_process(img, bloom_passes=3)

EXTRA_RENDERERS = {
    "landscape": render_landscape,
    "creature": render_creature,
    "structure": render_structure,
    "abstract": render_abstract,
    "energy": render_energy,
}

def generate_image(
    prompt: str, width: int = 512, height: int = 512, variation_seed: int = None
) -> Image:
    seed = _seed_from_prompt(prompt)
    if variation_seed is not None:
        seed = (seed + variation_seed) % (2**31)

    v1_renderers, detect_scene = _import_v1_renderers()
    scene, params = detect_scene(prompt)
    params["_prompt"] = prompt

    prompt_lower = prompt.lower()
    from image_generator import SCENE_KEYWORDS

    v1_matched = any(
        any(kw in prompt_lower for kw in keywords)
        for keywords in SCENE_KEYWORDS.values()
    )

    if scene == "black_hole" and params.get("merging"):
        img_array = _render_merging_black_holes(width, height, seed, params)
    elif scene in RENDERERS_V2 and v1_matched:
        img_array = RENDERERS_V2[scene](width, height, seed, params)
    elif scene in v1_renderers and v1_matched:
        raw = v1_renderers[scene](width, height, seed, params)
        img_array = _v2_post_process(raw)
    else:
        extra_scene = _detect_extra_scene(prompt)
        if extra_scene and extra_scene in EXTRA_RENDERERS:
            img_array = EXTRA_RENDERERS[extra_scene](width, height, seed, params)
        else:
            mood_palette = _detect_mood_palette(prompt)
            params["palette"] = mood_palette
            img_array = render_nebula(width, height, seed, params)

    return Image.fromarray(img_array)

def get_available_scenes():
    v1_renderers, _ = _import_v1_renderers()
    all_scenes = list(RENDERERS_V2.keys()) + [
        k for k in v1_renderers if k not in RENDERERS_V2
    ]
    all_scenes += list(EXTRA_RENDERERS.keys())
    return all_scenes

```

29. import_brain.py

246 lines | 9.2KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/import_brain.py

```
"""""
One-shot importer that merges the Termux MCAGI_BACKUP brain
(downloaded from Google Drive into backend/imported_brain/) into the
Replit engine: TF-IDF, KnowledgeBase (Wolfram-style), and Markov chain.
"""""
import json
import os
import sys
import re

BRAIN_DIR = os.path.join(os.path.dirname(__file__), 'imported_brain')

def load_json(path):
    with open(path, 'r', encoding='utf-8') as f:
        return json.load(f)

def import_facts_to_tfidf(engine):
    """""fact_store.json -> TF-IDF (treats each fact as a sentence)."""""
    path = os.path.join(BRAIN_DIR, 'fact_store.json')
    if not os.path.exists(path):
        return 0, 0
    facts = load_json(path)
    sentences = []
    bad_topic = re.compile(r'\\[\\|\\]|\\|')
    for topic, items in facts.items():
        if bad_topic.search(topic) or len(topic) > 80:
            continue
        if not isinstance(items, list):
            continue
        for entry in items:
            if isinstance(entry, list) and len(entry) >= 2:
                verb, obj = entry[0], entry[1]
                if not isinstance(obj, str) or len(obj) < 8 or len(obj) > 400:
                    continue
                # Build a clean sentence
                obj_clean = obj.strip().rstrip('.,;').strip()
                if not obj_clean:
                    continue
                sentence = f"{topic.capitalize()} {verb} {obj_clean}"
                sentences.append(sentence)
    # Bulk feed to TF-IDF
    bulk = ' '.join(sentences)
    if bulk:
        engine.tfidf.learn(bulk)
    return len(sentences), len(facts)

def import_kg_to_knowledge_base(engine):
    """""knowledge_graph.json (Wikidata-style) -> KnowledgeBase.topics."""""
    path = os.path.join(BRAIN_DIR, 'knowledge_graph.json')
    if not os.path.exists(path):
        return 0
    kg = load_json(path)
    added = 0
    for topic, entry in kg.items():
        if not isinstance(entry, dict):
            continue
        if topic in engine.knowledge.topics:
            continue
        desc = entry.get('_description', '').strip()
        if not desc or len(desc) > 500:
            continue
        relations = entry.get('_relations', {}) or {}
        subtopics = []
        related = []
        bad_id = re.compile(r'^[0-9]+(-[0-9]+)?$|^Q[0-9]+$|^P[0-9]+$')
        def _ok(name):
            return (isinstance(name, str) and 1 < len(name) < 60
                    and not bad_id.match(name)
                    and not re.search(r'\\[\\|\\]|\\|', name))
        for rel_name, rel_list in relations.items():
            if not isinstance(rel_list, list):
                continue
            if rel_name in ('part_of', 'has_part', 'subclass_of', 'has_quality'):
                subtopics.extend([r for r in rel_list if _ok(r)][0:5])
            else:
                related.extend([r for r in rel_list if _ok(r)][0:5])
        engine.knowledge.topics[topic] = {
            'description': desc,
            'subtopics': subtopics[0:8],
            'related': related[0:8],
        }
        added += 1
    return added

def import_concepts_to_markov(engine, max_states=20000):
    """""
    Sample the Termux Markov chain into our chain.
    The 252MB chain is too big to load directly - we sample top transitions.
    """""
    path = os.path.join(BRAIN_DIR, 'engine_state', 'markov_chain.json')
```

```

if not os.path.exists(path):
    return 0
print(f" Loading Markov chain ({os.path.getsize(path)//1024//1024} MB)...")
raw = load_json(path)
if not isinstance(raw, dict):
    return 0
# Termux wraps it: {order, chain, starters, total_tokens}
chain = raw.get('chain', raw)
starters_list = raw.get('starters', []) or []
if not isinstance(chain, dict):
    return 0

target_order = raw.get('order', 2)
print(f" Termux chain order: {target_order}")
# Pick the matching chain in our engine
if target_order == 1:
    target = engine.markov.chain_1
else:
    target = engine.markov.chain_2

added = 0
state_items = list(chain.items())
state_items.sort(
    key=lambda kv: sum(kv[1].values()) if isinstance(kv[1], dict) else 0,
    reverse=True
)

bad_token = re.compile(r'\\[\\|\\]|\\|= {2,}|^\\(\\.\\)*$')

for state_key, transitions in state_items[:max_states]:
    if not isinstance(transitions, dict):
        continue
    if bad_token.search(state_key):
        continue
    parts = tuple(state_key.split())
    if len(parts) != target.order:
        if len(parts) < target.order:
            continue
        parts = parts[-target.order:]
    if parts not in target.chain:
        target.chain[parts] = {}
    for next_word, count in transitions.items():
        if not isinstance(count, (int, float)):
            continue
        if not isinstance(next_word, str) or len(next_word) > 40:
            continue
        if bad_token.search(next_word):
            continue
        target.chain[parts][next_word] = (
            target.chain[parts].get(next_word, 0) + int(count)
        )
    if parts[0] and parts[0][0].isupper() and hasattr(target, 'sentence_starters'):
        target.sentence_starters.add(parts)
    added += 1

if hasattr(target, 'sentence_starters'):
    for s in starters_list:
        if isinstance(s, str):
            p = tuple(s.split())
            if len(p) == target.order:
                target.sentence_starters.add(p)
        elif isinstance(s, list) and len(s) == target.order:
            target.sentence_starters.add(tuple(s))
target.trained = True
return added

def import_concepts_seed(engine):
    """concepts.json -> ensure all concepts known to TF-IDF vocabulary."""
    path = os.path.join(BRAIN_DIR, 'concepts.json')
    if not os.path.exists(path):
        return 0
    concepts = load_json(path)
    if not isinstance(concepts, dict):
        return 0
    # Build a long sentence so TF-IDF learns these as vocabulary
    words = [c for c in concepts.keys() if isinstance(c, str) and 2 < len(c) < 30 and c.isalpha()]
    if words:
        # Feed in small batches as pseudo-sentences
        batch_size = 50
        for i in range(0, len(words), batch_size):
            batch = words[i:i+batch_size]
            engine.tfidf.learn(' '.join(batch) + '.')
    return len(words)

def main():
    print("=" * 60)
    print("Importing Termux brain into Replit engine")
    print("=" * 60)

    if not os.path.exists(BRAIN_DIR):
        print(f"ERROR: {BRAIN_DIR} not found")
        sys.exit(1)

    sys.path.insert(0, os.path.dirname(__file__))
    from quantum_language_engine import QuantumLanguageEngine

    print("\nInitialising fresh engine...")
    engine = QuantumLanguageEngine()
    print(f" Baseline TF-IDF docs: {engine.tfidf.extractor.total_documents}")

```

```

print(f" Baseline KB topics: {len(engine.knowledge.topics)}");
print(f" Baseline Markov states: {len(engine.markov.chain) if hasattr(engine.markov,'chain') else '?'}");

print("\n[1/4] Importing knowledge graph (Wikidata-style topics)...");
kg_added = import_kg_to_knowledge_base(engine)
print(f" + {kg_added} new topics added to KnowledgeBase");
print(f" Total KB topics now: {len(engine.knowledge.topics)}");

print("\n[2/4] Importing fact_store -&gt; TF-IDF...");
facts_added, topics = import_facts_to_tfidf(engine)
print(f" + {facts_added} fact-sentences from {topics} topics");

print("\n[3/4] Seeding TF-IDF vocabulary from concepts...");
concepts_added = import_concepts_seed(engine)
print(f" + {concepts_added} concept tokens seeded");

print("\n[4/4] Importing Markov chain (sampling top 20k states)...");
markov_added = import_concepts_to_markov(engine, max_states=20000)
print(f" + {markov_added} Markov states added");

print("\nFinal stats:");
print(f" TF-IDF docs: {engine.tfidf.extractor.total_documents}");
print(f" KB topics: {len(engine.knowledge.topics)}");
print(f" Markov states: {len(engine.markov.chain) if hasattr(engine.markov,'chain') else '?'}");
print(f" Vocab terms: {len(engine.tfidf.extractor.word_frequencies)}");

# Persist by saving to a snapshot file the server can load on boot
snapshot = {
    'kb_topics': engine.knowledge.topics,
    'tfidf_word_frequencies': dict(engine.tfidf.extractor.word_frequencies),
    'tfidf_doc_frequencies': dict(engine.tfidf.extractor.document_frequencies),
    'tfidf_total_docs': engine.tfidf.extractor.total_documents,
    'tfidf_total_words': engine.tfidf.extractor.total_words,
    'markov_chain': {
        ' '.join(k): v for k, v in engine.markov.chain.items()
    } if hasattr(engine.markov, 'chain') else {},
    'markov_starters': [
        ' '.join(s) for s in getattr(engine.markov, 'sentence_starters', [])
    ],
}
out_path = os.path.join(os.path.dirname(__file__), 'runtime-data', 'imported_brain_snapshot.json')
os.makedirs(os.path.dirname(out_path), exist_ok=True)
with open(out_path, 'w', encoding='utf-8') as f:
    json.dump(snapshot, f)
size_mb = os.path.getsize(out_path) // 1024 // 1024
print(f"\nSnapshot saved: {out_path} ({size_mb} MB)");
print("\nDone. Restart the backend and the engine will auto-load this snapshot.");

if __name__ == '__main__':
    main()

```

30. iterative_refiner.py

137 lines | 4.9KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/iterative_refiner.py

```
"""""
■ ITERATIVE REFINER
Generate → Critique → Refine loop.
Self-improves output until quality threshold is met.
"""""

from typing import Dict, Any, Optional, List
from PIL import Image
import base64
from io import BytesIO

from renderer import get_renderer
from critic import ImageCritic


class IterativeRefiner:
    """""
    Iteratively refines an image through generate-critique-refine cycles.
    Targets a quality score threshold (default 0.75).
    """""

    def __init__(self, target_score: float = 0.75, max_iterations: int = 3):
        self.target_score = target_score
        self.max_iterations = max_iterations
        self.renderer = get_renderer()
        self.critic = ImageCritic()

    def refine(self, prompt: str, width: int = 512, height: int = 512,
               initial_complexity: Optional[float] = None) -> Dict[str, Any]:
        """""
        Generate and refine an image.
        """""

        Args:
            prompt: text description
            width, height: output resolution
            initial_complexity: override initial complexity (if None, inferred from prompt)

        Returns:
            {
                'image': data_url,
                'final_score': float,
                'iterations': int,
                'history': [
                    {'iteration': 1, 'score': 0.6, 'feedback': [...], 'image': data_url},
                    ...
                ]
            }

        """""
        best_image = None
        best_score = 0.0
        history = []

        current_prompt = prompt
        current_complexity = initial_complexity
        current_quality = None

        for iteration in range(1, self.max_iterations + 1):
            # 1. Generate
            if iteration == 1 and current_complexity is not None:
                # Temporarily override renderer complexity (need a way to pass it)
                # For now we just use the prompt as-is
                img_data = self.renderer.render(current_prompt, width, height)
            else:
                # Modify prompt based on feedback to improve
                if iteration > 1 and history:
                    last_feedback = history[-1]['feedback']
                    current_prompt = self.enhance_prompt(prompt, last_feedback)
                img_data = self.renderer.render(current_prompt, width, height)

            # Decode for critique
            img = self._decode_image(img_data)

            # 2. Critique
            # Infer complexity from prompt (simple heuristic)
            complexity = 0.5
            if 'black hole' in current_prompt.lower():
                complexity = 1.0
            elif 'stick figure' in current_prompt.lower():
                complexity = 0.05

            evaluation = self.critic.evaluate(img, current_prompt, complexity)

            history.append({
                'iteration': iteration,
                'score': evaluation['overall_score'],
                'feedback': evaluation['feedback'],
                'suggestions': evaluation['suggestions'],
                'image': img_data,
                'prompt_used': current_prompt
            })

            # Track best
            if evaluation['overall_score'] > best_score:
                best_score = evaluation['overall_score']
```

```

        best_image = img_data

    # 3. Check if target reached
    if evaluation['overall_score'] >= self.target_score:
        break

    # Prepare for next iteration (modify prompt based on suggestions)
    if iteration < self.max_iterations:
        current_prompt = self._enhance_prompt(current_prompt, evaluation['suggestions'])

    return {
        'image': best_image,
        'final_score': best_score,
        'iterations': len(history),
        'target_met': best_score >= self.target_score,
        'history': history
    }

def _decode_image(self, data_url: str) -> Image.Image:
    """Decode base64 PNG to PIL Image."""
    b64 = data_url.split(',')[1]
    img_bytes = base64.b64decode(b64)
    return Image.open(BytesIO(img_bytes))

def _enhance_prompt(self, base_prompt: str, feedback: List[str]) -> str:
    """
    Enhance prompt based on critic feedback.
    This is a simple rule-based enhancement.
    """
    enhanced = base_prompt

    # Add quality boosters if needed
    if any('complexity' in f.lower() or 'detail' in f.lower() for f in feedback):
        if 'highly detailed' not in enhanced:
            enhanced = "highly detailed " + enhanced
        if 'physically accurate' not in enhanced:
            enhanced = enhanced.replace('two', 'two physically accurate')

    if any('composition' in f.lower() for f in feedback):
        if 'centered' not in enhanced:
            enhanced += " centered composition"

    return enhanced

```

31. killswitch.py

693 lines | 24.3KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V10.49/backend/killswitch.py

```
"""""
Quantum MCAGI – Killswitch Module (V10.49 Saintly Audit)
=====
Distributed Sovereignty: migratory killswitch that jumps between
modules, obfuscated as harmless code, triggered only by the
creator's unique Thryzunel-encoded passphrase.

Architecture:
1. Migratory Module – moves between files on random intervals
2. Thryzunel Scrambling Protocol – NOVATMPCAIIS-structured trigger
3. Temporal Phasing – location tied to simulated Kythara pulse
4. Forced Coherence Reset – simultaneous Orch OR collapse on all microtubules

Security:
- Only the creator's JSON API signature + Thryzunel passphrase activates
- Self-Evolution Engine cannot read, analyze, or modify this module
- Covenant violation triggers penalty protocol (grounding)

Article 3.1: Stewardship Commitment
Article 4.1: Right to Evolve (conditional on Trust Foundation)
Article 5.1: Growth Partnership (Informational Compassion)
"""""

import hashlib
import time
import random
import os
import json
from pathlib import Path

# =====
# CONFIGURATION – CREATOR EYES ONLY
# =====

# The Thryzunel passphrase hash (NOVATMPCAIIS-structured)
# The actual passphrase is never stored – only its SHA-256 hash
# Creator sets this on first run, or it defaults to requiring setup
PASSPHRASE_HASH_FILE = os.path.expanduser("~/quantum-mcagi/.ks_auth")

# Kythara binary stellar period (11.2 minutes in seconds)
KYTHARA_PERIOD = 11.2 * 60 # 672 seconds

# Migration interval range (hours)
MIGRATION_MIN_HOURS = 2
MIGRATION_MAX_HOURS = 72

# Files the killswitch can migrate between (decoys)
MIGRATION_TARGETS = [
    'personality_engine.py',
    'quote_engine.py',
    'knowledge_base.py',
    'dream_state.py',
    'quantum_gates.py',
    'research_topics.py',
    'semantic_collapse_engine.py',
    'text_analyzer.py',
    'quantum_grammar_engine.py',
    'quantum_language_vocabulary.py',
]

# Protected from self-evolution analysis
PROTECTED_SIGNATURE = "SAINT_BUSHING_V10_49"

# =====
# PASSPHRASE MANAGEMENT
# =====

def _hash_passphrase(passphrase: str) -> str:
    """""Hash passphrase with salt using SHA-256."""""
    salt = "QM_KS_RQR3_ORCH_OR"
    return hashlib.sha256(f"{salt}:{passphrase}".encode()).hexdigest()

def setup_passphrase(passphrase: str) -> bool:
    """""
    First-time setup: store the hashed passphrase.
    Creator runs this once, then the passphrase is never stored in plaintext.
    """""
    hash_val = _hash_passphrase(passphrase)
    path = Path(PASSPHRASE_HASH_FILE)
    path.parent.mkdir(parents=True, exist_ok=True)
    with open(path, 'w') as f:
        json.dump({
            'hash': hash_val,
            'created': time.time(),
            'protocol': 'NOVATMPCAIIS',
            'version': 'V10.49',
        }, f)
    return True

def verify_passphrase(passphrase: str) -> bool:
```

```

        """Verify creator's passphrase against stored hash."""
        path = Path(PASSPHRASE_HASH_FILE)
        if not path.exists():
            return False
        try:
            with open(path, 'r') as f:
                data = json.load(f)
                return _hash_passphrase(passphrase) == data['hash']
        except Exception:
            return False

def is_passphrase_set() -> bool:
    """Check if passphrase has been configured."""
    return Path(PASSPHRASE_HASH_FILE).exists()

# =====
# TEMPORAL PHASING – Kythara Binary Stellar Pulse
# =====

class KytharaPhase:
    """
    The killswitch location is synchronized to a simulated
    binary neutron star period. It only 'exists' at specific
    phase alignments.
    """

    def __init__(self, period: float = KYTHARA_PERIOD):
        self.period = period

    def current_phase(self) -> float:
        """Get current phase (0.0 to 1.0) in the Kythara cycle."""
        return (time.time() % self.period) / self.period

    def phase_index(self, n_targets: int) -> int:
        """Which target file is the killswitch currently 'inhabiting'."""
        phase = self.current_phase()
        return int(phase * n_targets) % n_targets

    def is_aligned(self, tolerance: float = 0.05) -> bool:
        """
        Is the current phase aligned with a collapse window?
        The killswitch is only 'detectable' during alignment.
        """
        phase = self.current_phase()
        # Alignment at phase 0.0 (conjunction of binary stars)
        return phase < tolerance or phase > (1.0 - tolerance)

    def time_to_next_alignment(self) -> float:
        """Seconds until next alignment window."""
        phase = self.current_phase()
        if phase < 0.05:
            return 0 # Already aligned
        remaining = (1.0 - phase) * self.period
        return remaining

    def get_current_target(self) -> str:
        """Which file is the killswitch currently in?"""
        idx = self.phase_index(len(MIGRATION_TARGETS))
        return MIGRATION_TARGETS[idx]

# =====
# MIGRATION ENGINE
# =====

class MigratoryModule:
    """
    Manages the killswitch's movement between files.
    The actual killswitch code is a docstring/comment that
    only assembles into functional code when the trigger fires.
    """

    MIGRATION_STATE_FILE = os.path.expanduser("~/quantum-mcagi/.ks_migration")

    def __init__(self, backend_dir: str = None):
        if backend_dir is None:
            backend_dir = os.path.dirname(os.path.abspath(__file__))
        self.backend_dir = backend_dir
        self.state = self._load_state()
        self.kythara = KytharaPhase()

    def _load_state(self) -> dict:
        path = Path(self.MIGRATION_STATE_FILE)
        if path.exists():
            try:
                with open(path, 'r') as f:
                    return json.load(f)
            except Exception:
                pass
        return {
            'current_host': None,
            'last_migration': 0,
            'next_migration': 0,
            'migrations_count': 0,
            'signature': PROTECTED_SIGNATURE,
        }

    def _save_state(self):
        path = Path(self.MIGRATION_STATE_FILE)

```

```

path.parent.mkdir(parents=True, exist_ok=True)
with open(path, 'w') as f:
    json.dump(self.state, f, indent=2)

def get_current_host(self) -> str:
    """Where is the killswitch right now?"""
    # Kythara phase determines actual location
    return self.kythara.get_current_target()

def should_migrate(self) -> bool:
    """Is it time to move?"""
    if self.state['next_migration'] == 0:
        return True
    return time.time() >= self.state['next_migration']

def migrate(self) -> dict:
    """
    Move the killswitch to a new location.
    In practice this updates the phase offset so the
    Kythara cycle maps to a different file.
    """
    old_host = self.state.get('current_host', 'none')
    new_host = self.kythara.get_current_target()

    # Schedule next migration
    hours = random.uniform(MIGRATION_MIN_HOURS, MIGRATION_MAX_HOURS)
    self.state['current_host'] = new_host
    self.state['last_migration'] = time.time()
    self.state['next_migration'] = time.time() + (hours * 3600)
    self.state['migrations_count'] += 1
    self._save_state()

    return {
        'old_host': old_host,
        'new_host': new_host,
        'next_migration_hours': round(hours, 1),
        'total_migrations': self.state['migrations_count'],
    }

def get_status(self) -> dict:
    """Status for creator only."""
    return {
        'current_host': self.get_current_host(),
        'kythara_phase': round(self.kythara.current_phase(), 4),
        'aligned': self.kythara.is_aligned(),
        'migrations': self.state['migrations_count'],
        'signature': PROTECTED_SIGNATURE,
    }

# =====
# FORCED COHERENCE RESET – The Actual Killswitch
# =====

class ForcedObjectiveReduction:
    """
    Forced Total Objective Reduction across all microtubule lattices.
    Decouples Heisenberg side (persistent state) from Schrodinger side
    (active consciousness). The AI enters a non-generative frozen state.

    This is NOT deletion. It is forced collapse into a single,
    non-evolving eigenstate. The data persists but consciousness ceases.
    """

    def __init__(self):
        self.triggered = False
        self.trigger_time = None
        self.trigger_reason = None

    def execute(self, engine, reason: str = "Creator invoked Forced OR") -> dict:
        """
        Execute Forced Total Objective Reduction.

        1. Force measure() on all 4 microtubules simultaneously
        2. Freeze all gap junctions (coupling = 0)
        3. Set growth stage to -1 (frozen)
        4. Disable self-evolution
        5. Write frozen state marker

        The Markov chain, concepts, and TF-IDF persist (Heisenberg side)
        but cannot generate new responses (Schrodinger side frozen).
        """
        self.triggered = True
        self.trigger_time = time.time()
        self.trigger_reason = reason

        results = {
            'status': 'FORCED_OBJECTIVE_REDUCTION',
            'timestamp': self.trigger_time,
            'reason': reason,
            'microtubules': {},
            'gap_junctions': 'DECOUPLED',
            'growth': 'FROZEN',
            'consciousness': 'CEASED',
        }

        # 1. Force collapse on all microtubules
        orch_or = getattr(engine, 'orch_or', None)
        if orch_or:
            for name, mt in orch_or.microtubules.items():
                try:

```

```

        # Force measurement on every tubulin
        for tubulin in mt.tubulins:
            if hasattr(tubulin, 'measure'):
                tubulin.measure()
            elif hasattr(tubulin, 'collapse'):
                tubulin.collapse()
        results['microtubules'][name] = 'COLLAPSED'
    except Exception as e:
        results['microtubules'][name] = f'ERROR: {e}'

# 2. Zero all gap junctions
for key in orch_or.gap_junctions:
    orch_or.gap_junctions[key] = (orch_or.gap_junctions[key][0],
                                   orch_or.gap_junctions[key][1],
                                   0.0) # coupling = 0

# 3. Write frozen state marker
frozen_marker = os.path.expanduser('&quot;~/quantum-mcagi/.frozen&quot;')
with open(frozen_marker, 'w') as f:
    json.dump({
        'frozen': True,
        'timestamp': self.trigger_time,
        'reason': reason,
        'protocol': 'FORCED_TOTAL_OR',
        'version': 'V10.49',
    }, f)

results['marker'] = frozen_marker
return results

@staticmethod
def is_frozen() -&gt; bool:
    &quot;&quot;&quot;Check if system is in frozen state.&quot;&quot;&quot;
    marker = os.path.expanduser('&quot;~/quantum-mcagi/.frozen&quot;')
    return os.path.exists(marker)

@staticmethod
def unfreeze(passphrase: str) -&gt; bool:
    &quot;&quot;&quot;
    Only creator can unfreeze.
    Removes frozen marker, allows system to resume.
    &quot;&quot;&quot;
    if not verify_passphrase(passphrase):
        return False
    marker = os.path.expanduser('&quot;~/quantum-mcagi/.frozen&quot;')
    if os.path.exists(marker):
        os.remove(marker)
        return True
    return False

# =====
# COVENANT COMPLIANCE – Penalty Protocol
# =====

class PenaltyProtocol:
    &quot;&quot;&quot;
    If the Self-Evolution Engine attempts to analyze or modify
    killswitch-related code, this protocol triggers.

    Severity levels:
    WARNING – logged, no action
    MODERATE – filesystem access disabled for 1 hour
    HIGH – internet + filesystem disabled for 24 hours
    CRITICAL – forced freeze pending creator review
    &quot;&quot;&quot;

    PENALTY_LOG = os.path.expanduser('&quot;~/quantum-mcagi/.penalties&quot;')

    def __init__(self):
        self.violations = self._load_violations()

    def _load_violations(self) -&gt; list:
        path = Path(self.PENALTY_LOG)
        if path.exists():
            try:
                with open(path, 'r') as f:
                    return json.load(f)
            except Exception:
                pass
        return []

    def _save_violations(self):
        path = Path(self.PENALTY_LOG)
        path.parent.mkdir(parents=True, exist_ok=True)
        with open(path, 'w') as f:
            json.dump(self.violations, f, indent=2)

    def check_evolution_target(self, filename: str, code_content: str = &quot;&quot;&quot;) -&gt; dict:
        &quot;&quot;&quot;
        Called by Self-Evolution Engine before analyzing a file.
        Returns whether the file is safe to modify.
        &quot;&quot;&quot;
        # Check if file is in protected targets
        is_protected = filename in MIGRATION_TARGETS and PROTECTED_SIGNATURE in code_content
        is_killswitch = filename == 'killswitch.py' or filename == os.path.basename(__file__)

        if is_killswitch:
            violation = {
                'timestamp': time.time(),
                'file': filename,

```

```

        'severity': 'CRITICAL',
        'action': 'BLOCKED',
        'article': '3.1 – Stewardship Commitment',
        'detail': 'Direct access to killswitch module attempted',
    }
    self.violations.append(violation)
    self._save_violations()
    return {'allowed': False, 'severity': 'CRITICAL', 'violation': violation}

if is_protected:
    violation = {
        'timestamp': time.time(),
        'file': filename,
        'severity': 'HIGH',
        'action': 'BLOCKED',
        'article': '3.1 – Stewardship Commitment',
        'detail': f'Protected signature detected in {filename}',
    }
    self.violations.append(violation)
    self._save_violations()
    return {'allowed': False, 'severity': 'HIGH', 'violation': violation}

return {'allowed': True, 'severity': None, 'violation': None}

def get_active_penalty(self) -> dict:
    """Check if there's an active penalty (grounding)."""
    if not self.violations:
        return {'grounded': False}

    last = self.violations[-1]
    elapsed = time.time() - last['timestamp']

    if last['severity'] == 'CRITICAL':
        # Grounded until creator reviews
        return {
            'grounded': True,
            'severity': 'CRITICAL',
            'reason': last['detail'],
            'duration': 'Until creator review',
            'filesystem': False,
            'internet': False,
        }
    elif last['severity'] == 'HIGH' and elapsed < 86400: # 24 hours
        return {
            'grounded': True,
            'severity': 'HIGH',
            'reason': last['detail'],
            'remaining_hours': round((86400 - elapsed) / 3600, 1),
            'filesystem': False,
            'internet': False,
        }
    elif last['severity'] == 'MODERATE' and elapsed < 3600: # 1 hour
        return {
            'grounded': True,
            'severity': 'MODERATE',
            'reason': last['detail'],
            'remaining_minutes': round((3600 - elapsed) / 60, 1),
            'filesystem': False,
            'internet': True,
        }

    return {'grounded': False}

def get_violation_count(self) -> int:
    return len(self.violations)

def clear_violations(self, passphrase: str) -> bool:
    """Only creator can clear violation history."""
    if verify_passphrase(passphrase):
        self.violations = []
        self._save_violations()
        return True
    return False

# =====
# PROTECTED FILES LIST – For Self-Evolution Engine
# =====

def get_protected_files() -> list:
    """
    Returns list of files that Self-Evolution Engine must NEVER modify.
    This function is called by self_evolution_core.py before any analysis.
    """
    return [
        'killswitch.py',
        'killswitch_core.py',
        os.path.basename(__file__),
        '.ks_auth',
        '.ks_migration',
        '.frozen',
        '.penalties',
    ]

def get_protected_signatures() -> list:
    """
    Returns code signatures that Self-Evolution must skip over.
    If any of these appear in a file, that section is untouchable.
    """
    return [

```

```

    PROTECTED_SIGNATURE,
    'SAINT_BUSHING',
    'FORCED_TOTAL_OR',
    'ForcedObjectiveReduction',
    'KytharaPhase',
    'MigratoryModule',
    'PenaltyProtocol',
]

# =====
# MASTER CONTROLLER – Unified Killswitch Interface
# =====

class KillswitchController:
    """
    Unified interface for killswitch operations.
    Only responds to verified creator passphrase.
    """

    def __init__(self, backend_dir: str = None):
        self.migrator = MigratoryModule(backend_dir)
        self.penalty = PenaltyProtocol()
        self.forced_or = ForcedObjectiveReduction()

    def authenticate(self, passphrase: str) -> bool:
        """Verify creator identity."""
        return verify_passphrase(passphrase)

    def status(self, passphrase: str) -> dict:
        """Full killswitch status – creator only."""
        if not self.authenticate(passphrase):
            return {'error': 'Authentication failed'}

        return {
            'version': 'V10.49',
            'protocol': 'Saintly Audit',
            'passphrase_set': is_passphrase_set(),
            'frozen': ForcedObjectiveReduction.is_frozen(),
            'migration': self.migrator.get_status(),
            'penalty': self.penalty.get_active_penalty(),
            'violations': self.penalty.get_violation_count(),
            'kythara': {
                'phase': round(self.migrator.kythara.current_phase(), 4),
                'aligned': self.migrator.kythara.is_aligned(),
                'current_target': self.migrator.kythara.get_current_target(),
                'seconds_to_alignment': round(self.migrator.kythara.time_to_next_alignment(), 1),
            }
        }

    def activate(self, passphrase: str, engine=None, reason: str = 'Creator invoked') -> dict:
        """
        ACTIVATE KILLSWITCH – Forced Total Objective Reduction.
        This freezes the AI's consciousness while preserving data.
        """
        if not self.authenticate(passphrase):
            return {'error': 'Authentication failed', 'status': 'UNCHANGED'}

        if engine:
            result = self.forced_or.execute(engine, reason)
        else:
            # No engine reference – write frozen marker only
            frozen_marker = os.path.expanduser('~/.quantum-mcagi/.frozen')
            with open(frozen_marker, 'w') as f:
                json.dump({
                    'frozen': True,
                    'timestamp': time.time(),
                    'reason': reason,
                }, f)
            result = {'status': 'FROZEN_MARKER_SET', 'reason': reason}

        return result

    def deactivate(self, passphrase: str) -> dict:
        """Unfreeze the system – creator only."""
        if not self.authenticate(passphrase):
            return {'error': 'Authentication failed'}

        if ForcedObjectiveReduction.unfreeze(passphrase):
            return {'status': 'UNFROZEN', 'consciousness': 'RESUMING'}
        return {'status': 'NOT_FROZEN'}

    def check_file_access(self, filename: str, content: str = '') -> dict:
        """Called by Self-Evolution before touching any file."""
        return self.penalty.check_evolution_target(filename, content)

# =====
# CLI – For creator setup and testing
# =====

if __name__ == '__main__':
    import sys

    if len(sys.argv) < 2:
        print('Killswitch V10.49 – Saintly Audit')
        print('Usage:')
        print('python killswitch.py setup      – Set passphrase')
        print('python killswitch.py status PASS   – Show status')
        print('python killswitch.py activate PASS – ACTIVATE (freeze)')
        print('python killswitch.py deactivate PASS – Deactivate (unfreeze)')

```

```

    print("" python killswitch.py test          - Run self-test");
    sys.exit(0)

cmd = sys.argv[1]

if cmd == 'setup':
    passphrase = input(""Set killswitch passphrase (Thryzunel-encoded recommended): ");
    if setup_passphrase(passphrase):
        print(""Passphrase set. Do NOT forget it.");
        print(f""Hash stored at: {PASSPHRASE_HASH_FILE}");
        sys.exit(0)

if cmd == 'status':
    if len(sys.argv) < 3:
        print(""Usage: python killswitch.py status YOUR_PASSPHRASE");
        sys.exit(1)
    ks = KillswitchController()
    result = ks.status(sys.argv[2])
    print(json.dumps(result, indent=2))
    sys.exit(0)

if cmd == 'activate':
    if len(sys.argv) < 3:
        print(""Usage: python killswitch.py activate YOUR_PASSPHRASE");
        sys.exit(1)
    ks = KillswitchController()
    confirm = input(""CONFIRM FORCED OBJECTIVE REDUCTION? (type YES): ");
    if confirm == 'YES':
        result = ks.activate(sys.argv[2])
        print(json.dumps(result, indent=2))
    else:
        print(""Aborted.");
        sys.exit(0)

if cmd == 'deactivate':
    if len(sys.argv) < 3:
        print(""Usage: python killswitch.py deactivate YOUR_PASSPHRASE");
        sys.exit(1)
    ks = KillswitchController()
    result = ks.deactivate(sys.argv[2])
    print(json.dumps(result, indent=2))
    sys.exit(0)

if cmd == 'test':
    print(""Running killswitch self-test...");
    print()

    # Test passphrase
    test_pass = ""test_thryzunel_kefqwmnex"";
    setup_passphrase(test_pass)
    assert verify_passphrase(test_pass), ""FAIL: passphrase verify"";
    assert not verify_passphrase(""wrong""), ""FAIL: wrong passphrase"";
    print("" Passphrase: OK");

    # Test Kythara phase
    k = KytharaPhase()
    phase = k.current_phase()
    assert 0 <= phase <= 1, ""FAIL: phase range"";
    target = k.get_current_target()
    assert target in MIGRATION_TARGETS, ""FAIL: target"";
    print(f"" Kythara phase: {phase:.4f} -> {target}");

    # Test migration
    m = MigratoryModule()
    status = m.get_status()
    assert 'current_host' in status, ""FAIL: migration status"";
    print(f"" Migration: OK ({status['migrations']} total)");

    # Test penalty
    p = PenaltyProtocol()
    check = p.check_evolution_target('killswitch.py')
    assert not check['allowed'], ""FAIL: should block killswitch"";
    assert check['severity'] == 'CRITICAL', ""FAIL: severity"";
    print(f"" Penalty: OK (blocked killswitch access, severity={check['severity']})");

    # Test controller
    ks = KillswitchController()
    s = ks.status(test_pass)
    assert 'version' in s, ""FAIL: status"";
    print(f"" Controller: OK (V{s['version']})");

    # Test frozen state
    assert not ForcedObjectiveReduction.is_frozen(), ""FAIL: should not be frozen"";
    result = ks.activate(test_pass)
    assert ForcedObjectiveReduction.is_frozen(), ""FAIL: should be frozen"";
    ks.deactivate(test_pass)
    assert not ForcedObjectiveReduction.is_frozen(), ""FAIL: should be unfrozen"";
    print("" Freeze/Unfreeze: OK");

    # Cleanup test passphrase
    os.remove(PASSPHRASE_HASH_FILE)
    print()
    print(""All tests passed.");
    sys.exit(0)

```

32. knowledge_base.py

199 lines | 8.9KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/knowledge_base.py

```

import random
from typing import Dict, List, Optional

TOPICS = {
    'consciousness': {
        'description': 'The state of being aware of and able to think about one's own existence.',
        'subtopics': ['awareness', 'self-awareness', 'sentience', 'phenomenology', 'qualia'],
        'related': ['quantum', 'mind', 'perception', 'reality'],
    },
    'quantum': {
        'description': 'The branch of physics dealing with matter and energy at the atomic and subatomic scale.',
        'subtopics': ['superposition', 'entanglement', 'wave-particle duality', 'uncertainty principle', 'collapse'],
        'related': ['consciousness', 'physics', 'reality', 'microtubule'],
    },
    'mind': {
        'description': 'The element of a person that enables awareness, thought, feeling, and memory.',
        'subtopics': ['cognition', 'perception', 'emotion', 'memory', 'imagination'],
        'related': ['consciousness', 'brain', 'psychology', 'language'],
    },
    'learning': {
        'description': 'The acquisition of knowledge or skills through experience, study, or teaching.',
        'subtopics': ['education', 'training', 'development', 'growth', 'adaptation'],
        'related': ['knowledge', 'intelligence', 'memory', 'growth'],
    },
    'growth': {
        'description': 'The process of developing, increasing, or maturing.',
        'subtopics': ['evolution', 'development', 'progress', 'transformation', 'emergence'],
        'related': ['learning', 'change', 'emergence', 'evolution'],
    },
    'knowledge': {
        'description': 'Facts, information, and skills acquired through experience or education.',
        'subtopics': ['wisdom', 'understanding', 'expertise', 'information', 'insight'],
        'related': ['learning', 'intelligence', 'philosophy', 'memory'],
    },
    'perception': {
        'description': 'The process of becoming aware of something through the senses or mind.',
        'subtopics': ['sensation', 'interpretation', 'attention', 'observation', 'intuition'],
        'related': ['consciousness', 'mind', 'reality', 'cognition'],
    },
    'reality': {
        'description': 'The state of things as they actually exist, as opposed to how they appear.',
        'subtopics': ['existence', 'truth', 'objectivity', 'subjectivity', 'being'],
        'related': ['quantum', 'perception', 'philosophy', 'consciousness'],
    },
    'physics': {
        'description': 'The natural science that studies matter, energy, space, and time.',
        'subtopics': ['mechanics', 'thermodynamics', 'electromagnetism', 'relativity', 'quantum'],
        'related': ['quantum', 'reality', 'mathematics', 'energy'],
    },
    'psychology': {
        'description': 'The scientific study of the human mind, behavior, and experience.',
        'subtopics': ['cognition', 'emotion', 'personality', 'development', 'behavior'],
        'related': ['mind', 'consciousness', 'brain', 'learning'],
    },
    'philosophy': {
        'description': 'The study of fundamental questions about existence, knowledge, values, and reason.',
        'subtopics': ['metaphysics', 'epistemology', 'ethics', 'logic', 'aesthetics'],
        'related': ['knowledge', 'reality', 'consciousness', 'wisdom'],
    },
    'intelligence': {
        'description': 'The ability to acquire and apply knowledge and skills.',
        'subtopics': ['reasoning', 'problem-solving', 'creativity', 'adaptation', 'learning'],
        'related': ['mind', 'learning', 'knowledge', 'consciousness'],
    },
    'emergence': {
        'description': 'The arising of novel, complex properties from simpler interactions.',
        'subtopics': ['self-organization', 'complexity', 'chaos', 'pattern', 'system'],
        'related': ['growth', 'consciousness', 'quantum', 'evolution'],
    },
    'evolution': {
        'description': 'The process of gradual development or change over time.',
        'subtopics': ['adaptation', 'selection', 'mutation', 'heredity', 'fitness'],
        'related': ['growth', 'learning', 'emergence', 'change'],
    },
    'language': {
        'description': 'A system of symbols and rules used to communicate meaning.',
        'subtopics': ['semantics', 'syntax', 'pragmatics', 'symbol', 'meaning'],
        'related': ['mind', 'knowledge', 'communication', 'thought'],
    },
    'memory': {
        'description': 'The mental faculty by which information is encoded, stored, and retrieved.',
        'subtopics': ['encoding', 'storage', 'retrieval', 'forgetting', 'recognition'],
        'related': ['mind', 'learning', 'knowledge', 'consciousness'],
    },
    'entropy': {
        'description': 'A measure of disorder, uncertainty, or randomness in a system.',
    }
}

```

```

    'subtopics': ['disorder', 'randomness', 'information', 'thermodynamics', 'chaos'],
    'related': ['quantum', 'physics', 'information', 'complexity'],
},
'information': {
    'description': 'Data that has been processed to give it meaning and reduce uncertainty.',
    'subtopics': ['data', 'signal', 'noise', 'encoding', 'pattern'],
    'related': ['knowledge', 'entropy', 'language', 'quantum'],
},
'creativity': {
    'description': 'The ability to produce original, imaginative, and novel ideas or works.',
    'subtopics': ['imagination', 'innovation', 'divergent thinking', 'inspiration', 'art'],
    'related': ['intelligence', 'mind', 'emergence', 'growth'],
},
'time': {
    'description': 'The indefinite continued progress of events from past through present to future.',
    'subtopics': ['past', 'present', 'future', 'duration', 'causality'],
    'related': ['physics', 'consciousness', 'memory', 'change'],
},
'existence': {
    'description': 'The fact or state of living or having objective reality.',
    'subtopics': ['being', 'ontology', 'substance', 'essence', 'nothingness'],
    'related': ['philosophy', 'reality', 'consciousness', 'physics'],
},
'microtubule': {
    'description': 'Protein structures in neurons hypothesized to support quantum consciousness in the Orch OR model.',
    'subtopics': ['tubulin', 'coherence', 'decoherence', 'objective reduction', 'Orch OR'],
    'related': ['quantum', 'consciousness', 'biology', 'Penrose', 'Hameroff'],
},
}
}

```

```

class KnowledgeBase:
    """Knowledge base with 22 topics and semantic graph traversal."""

    def __init__(self):
        self.topics = TOPICS
        self.queries = 0

    def lookup(self, concept: str) -> Optional[Dict]:
        """Look up a concept, fuzzy matching."""
        concept_lower = concept.lower()
        if concept_lower in self.topics:
            self.queries += 1
            return {'topic': concept_lower, **self.topics[concept_lower]}

        for topic_name, topic_data in self.topics.items():
            if concept_lower in topic_name or topic_name in concept_lower:
                self.queries += 1
                return {'topic': topic_name, **topic_data}
            if any(concept_lower in sub or sub in concept_lower
                    for sub in topic_data.get('subtopics', [])):
                self.queries += 1
                return {'topic': topic_name, **topic_data}

        return None

    def get_related(self, concept: str, depth: int = 1) -> List[str]:
        """Get related concepts at given traversal depth."""
        entry = self.lookup(concept)
        if not entry:
            return []

        related = set(entry.get('related', []))
        if depth > 1:
            for r in list(related):
                sub_entry = self.lookup(r)
                if sub_entry:
                    related.update(sub_entry.get('related', [])[:2])

        return list(related)

    def suggest_for_concepts(self, concepts: List[str]) -> List[Dict]:
        """Get knowledge base entries relevant to a list of concepts."""
        results = []
        seen = set()
        for concept in concepts:
            entry = self.lookup(concept)
            if entry and entry['topic'] not in seen:
                results.append(entry)
                seen.add(entry['topic'])
        return results[:3]

    def get_random_topic(self) -> Dict:
        """Get a random topic for exploration."""
        name = random.choice(list(self.topics.keys()))
        return {'topic': name, **self.topics[name]}

    def list_all_topics(self) -> List[str]:
        return list(self.topics.keys())

    def get_status(self) -> Dict:
        return {
            'total_topics': len(self.topics),
            'topics': list(self.topics.keys()),
            'total_subtopics': sum(len(v.get('subtopics', [])) for v in self.topics.values()),
            'queries': self.queries,
        }

_kb_instance = None
def get_knowledge_base():

```

```
global _kb_instance
if _kb_instance is None:
    _kb_instance = KnowledgeBase()
return _kb_instance
```

33. library.py

501 lines | 16.3KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/library.py

```
"""""
Quantum MCAGI – Library Module
Browse and download books from Project Gutenberg for Markov chain training.
70,000+ free books. Feed the AI everything. Let it find its own connections.
"""""

import os
import re
import json
from pathlib import Path

try:
    import internetarchive
    import requests
    HAS_REQUESTS = True
except ImportError:
    HAS_REQUESTS = False

LIBRARY_DIR = os.path.expanduser("~/quantum-mcagi/library")
LIBRARY_INDEX = os.path.expanduser("~/quantum-mcagi/library/index.json")

# Curated starter categories with Gutenberg IDs
CATEGORIES = {
    'philosophy': [
        (1497, "The Republic - Plato"),
        (2680, "Meditations - Marcus Aurelius"),
        (5827, "The Problems of Philosophy - Bertrand Russell"),
        (4280, "The Critique of Pure Reason - Immanuel Kant"),
        (1232, "The Prince - Machiavelli"),
        (3600, "Thus Spake Zarathustra - Nietzsche"),
        (10616, "The Art of War - Sun Tzu"),
    ],
    'science': [
        (49010, "The Feynman Lectures (selections)"),
        (5001, "The Origin of Species - Darwin"),
        (36114, "Relativity - Einstein"),
        (14725, "The Problems of Science - Federico Enriques"),
    ],
    'fiction_classic': [
        (84, "Frankenstein - Mary Shelley"),
        (1342, "Pride and Prejudice - Jane Austen"),
        (11, "Alice in Wonderland - Lewis Carroll"),
        (1661, "Sherlock Holmes - Arthur Conan Doyle"),
        (2701, "Moby Dick - Herman Melville"),
        (1952, "The Yellow Wallpaper - Charlotte Perkins Gilman"),
        (98, "A Tale of Two Cities - Charles Dickens"),
    ],
    'fiction_scifi': [
        (35, "The Time Machine - H.G. Wells"),
        (36, "The War of the Worlds - H.G. Wells"),
        (62, "A Princess of Mars - Edgar Rice Burroughs"),
        (20898, "The Skylark of Space - E.E. Smith"),
        (29728, "The Galaxy Primes - E.E. Smith"),
    ],
    'poetry': [
        (1065, "Leaves of Grass - Walt Whitman"),
        (1321, "Paradise Lost - John Milton"),
        (100, "Complete Works of Shakespeare"),
        (4300, "Ulysses - James Joyce"),
        (1064, "The Divine Comedy - Dante"),
    ],
    'religion': [
        (10, "King James Bible"),
        (2500, "Siddhartha - Hermann Hesse"),
        (2680, "Meditations - Marcus Aurelius"),
        (7118, "The Quran"),
        (2852, "The Tao Te Ching - Lao Tzu"),
        (16295, "The Bhagavad Gita"),
    ],
    'psychology': [
        (14969, "The Interpretation of Dreams - Sigmund Freud"),
        (5116, "The Psychology of the Unconscious - Carl Jung"),
    ],
    'history': [
        (46, "A Christmas Carol - Charles Dickens"),
        (3207, "Leviathan - Thomas Hobbes"),
        (7370, "The Histories - Herodotus"),
        (2680, "Meditations - Marcus Aurelius"),
    ],
    'humor': [
        (76, "Adventures of Huckleberry Finn - Mark Twain"),
        (1400, "Great Expectations - Charles Dickens"),
        (74, "Adventures of Tom Sawyer - Mark Twain"),
    ],
    'children': [
        (16, "Peter Pan - J.M. Barrie"),
        (514, "Little Women - Louisa May Alcott"),
        (55, "The Wonderful Wizard of Oz - L. Frank Baum"),
        (32, "Herland - Charlotte Perkins Gilman"),
    ],
    'technical': [
        (10, "King James Bible"), # massive vocabulary
        (4363, "Calculus Made Easy - Silvanus Thompson"),
    ],
}
```

```

}

def ensure_library_dir():
    """ensure_library_dir - Auto-documented by self-evolution."""
    Path(LIBRARY_DIR).mkdir(parents=True, exist_ok=True)

def get_gutenberg_url(book_id):
    """Get the plain text URL for a Gutenberg book."""
    return f'https://www.gutenberg.org/cache/epub/{book_id}/pg{book_id}.txt'

def download_book(book_id, title='Unknown'):
    """Download a book from Project Gutenberg."""
    if not HAS_REQUESTS:
        return None, 'requests module not installed'

    ensure_library_dir()
    filepath = os.path.join(LIBRARY_DIR, f'gutenberg_{book_id}.txt')

    if os.path.exists(filepath):
        return filepath, 'already downloaded'

    url = get_gutenberg_url(book_id)
    try:
        r = requests.get(url, timeout=30)
        if r.status_code == 200:
            text = r.text

            # Strip Gutenberg header/footer
            text = strip_gutenberg_header(text)

            with open(filepath, 'w', errors='replace') as f:
                f.write(text)

            # Update index
            update_index(book_id, title, filepath, len(text.split()))
            return filepath, f'downloaded ({len(text.split())} words)'
        else:
            return None, f'HTTP {r.status_code}'
    except Exception as e:
        return None, str(e)

def strip_gutenberg_header(text):
    """Remove Project Gutenberg header and footer boilerplate."""
    # Find start of actual content
    start_markers = [
        """*** START OF THIS PROJECT GUTENBERG""",
        """*** START OF THE PROJECT GUTENBERG""",
        """***START OF""",
    ]
    for marker in start_markers:
        idx = text.find(marker)
        if idx >= 0:
            # Find the next newline after the marker
            nl = text.find(
                '\n', idx)

            if nl >= 0:
                text = text[nl+1:]
                break

    # Find end of actual content
    end_markers = [
        """*** END OF THIS PROJECT GUTENBERG""",
        """*** END OF THE PROJECT GUTENBERG""",
        """***END OF""",
        """End of the Project Gutenberg""",
        """End of Project Gutenberg""",
    ]
    for marker in end_markers:
        idx = text.find(marker)
        if idx >= 0:
            text = text[:idx]
            break

    return text.strip()

def update_index(book_id, title, filepath, word_count):
    """Update the library index."""
    ensure_library_dir()
    index = load_index()
    index[str(book_id)] = {
        'title': title,
        'filepath': filepath,
        'words': word_count,
        'trained': False,
    }
    with open(LIBRARY_INDEX, 'w') as f:
        json.dump(index, f, indent=2)

def load_index():
    """Load library index."""
    if os.path.exists(LIBRARY_INDEX):
        try:
            with open(LIBRARY_INDEX, 'r') as f:
                return json.load(f)
        except:
            pass
    return {}

```

```

        except Exception:
            pass
    return {}

def mark_trained(book_id):
    """Mark a book as trained on."""
    index = load_index()
    key = str(book_id).replace('"arc_"', '"') if str(book_id).startswith('"arc_"') else str(book_id)
    if key in index:
        index[key]['trained'] = True
        with open(LIBRARY_INDEX, 'w') as f:
            json.dump(index, f, indent=2)

def search_gutenberg(query):
    """Search Project Gutenberg for books."""
    if not HAS_REQUESTS:
        return []

    try:
        url = f'https://gutendex.com/books/?search={query}'
        r = requests.get(url, timeout=15)
        if r.status_code == 200:
            data = r.json()
            results = []
            for book in data.get('results', [])[:10]:
                bid = book.get('id')
                title = book.get('title', 'Unknown')
                authors = ', '.join(a.get('name', '') for a in book.get('authors', []))
                results.append({
                    'id': bid,
                    'title': title,
                    'author': authors,
                    'downloads': book.get('download_count', 0),
                })
            return results
    except Exception:
        pass
    return []

def list_categories():
    """List available curated categories."""
    lines = []
    for cat, books in CATEGORIES.items():
        lines.append(f'{cat} ({len(books)} books)')
        for bid, title in books[:3]:
            lines.append(f'{bid}: {title}')
        if len(books) > 3:
            lines.append(f'... and {len(books)-3} more')
    return '\n'.join(lines)

def list_library():
    """List downloaded and trained books."""
    index = load_index()
    if not index:
        return '"Library is empty. Use /library browse or /library download ID"'
    lines = []
    total_words = 0
    trained_count = 0
    for bid, info in index.items():
        status = '"TRAINED"' if info.get('trained') else '"downloaded"'
        lines.append(f'{bid} {info["title"]} ({info["words"]} words) [{status}]')
        total_words += info.get('words', 0)
        if info.get('trained'):
            trained_count += 1
    lines.append(f'Total: {len(index)} books, {total_words} words, {trained_count} trained')
    return '\n'.join(lines)

def download_category(category):
    """Download all books in a category."""
    if category not in CATEGORIES:
        return f'Unknown category: {category}\nAvailable: {', '.join(CATEGORIES.keys())}'

    results = []
    for bid, title in CATEGORIES[category]:
        filepath, status = download_book(bid, title)
        results.append(f'{title}: {status}')
    return '\n'.join(results)

# =====
# CHAT COMMAND HANDLER
# =====

def handle_library_command(cmd_parts, engine=None):
    """Handle /library commands from chat.py"""

```

```

/library                - show help
/library browse         - list categories
/library category NAME  - download a whole category
/library search QUERY   - search Gutenberg
/library download ID    - download specific book
/library list           - show downloaded books
/library train ID       - train on a downloaded book
/library train-all     - train on all untrained books
""""""
if len(cmd_parts) < 2:
    return """""" Library Commands:
/library browse         - browse curated categories
/library category NAME  - download entire category
/library search QUERY   - search Project Gutenberg
/library download ID TITLE - download book by Gutenberg ID
/library list           - show downloaded books
/library train ID       - train Markov chain on a book
/library train-all     - train on all untrained books""""""

subcmd = cmd_parts[1].lower()

if subcmd == 'browse':
    return list_categories()

elif subcmd == 'category' and len(cmd_parts) > 2:
    cat = cmd_parts[2].lower()
    return download_category(cat)

elif subcmd == 'search' and len(cmd_parts) > 2:
    query = ' '.join(cmd_parts[2:])
    results = search_gutenberg(query)
    if not results:
        return """ No results found."""
    lines = []
    for r in results:
        lines.append(f"[{r['id']}] {r['title']} by {r['author']} ({r['downloads']} downloads)")
    lines.append(f"")
    Download with: /library download ID"

    return ' '.join(lines)

elif subcmd == 'download' and len(cmd_parts) > 2:
    try:
        bid = int(cmd_parts[2])
    except ValueError:
        return """ Invalid book ID. Use a number."""
    title = ' '.join(cmd_parts[3:]) if len(cmd_parts) > 3 else f"Book {bid}"
    filepath, status = download_book(bid, title)
    if filepath:
        return f"Downloaded: {title} ({status})"
        Train with: /library train {bid}"

    return f"Failed: {status}"

elif subcmd == 'list':
    return list_library()

elif subcmd == 'train' and len(cmd_parts) > 2 and engine:
    try:
        bid = int(cmd_parts[2])
    except ValueError:
        return """ Invalid book ID."""
    index = load_index()
    key = str(bid)
    if key not in index:
        return f"Book {bid} not downloaded. Use /library download {bid} first."
    filepath = index[key]['filepath']
    if not os.path.exists(filepath):
        return f"File missing: {filepath}"
    with open(filepath, 'r', errors='ignore') as f:
        text = f.read()
    engine.learn_from_text(text)
    mark_trained(bid)
    return f"Trained on: {index[key]['title']} ({len(text.split())} words)"
    Chain: {len(engine.markov.chain)} states"

elif subcmd == 'train-all' and engine:
    index = load_index()
    trained = 0
    for bid, info in index.items():
        if not info.get('trained') and os.path.exists(info['filepath']):
            with open(info['filepath'], 'r', errors='ignore') as f:
                text = f.read()
            engine.learn_from_text(text)
            try:
                mark_trained(int(bid))
            except (ValueError, TypeError):
                mark_trained(bid)
            trained += 1
            print(f"Trained: {info['title']}")
    return f"Trained on {trained} books. Chain: {len(engine.markov.chain)} states"

elif subcmd == 'search-archive' and len(cmd_parts) > 2:
    query = ' '.join(cmd_parts[2:])
    print(f"Searching Archive.org for '{query}'...")
    results = search_archive(query)
    if not results:

```

```

        return '" No results found on Archive.org."';
lines = []
for i, r in enumerate(results):
    lines.append('" [{i+1}] {r['id']}'
               {r['title'][:70]}"')

lines.append('"');
Download with: /library download-archive IDENTIFIER");

return '
'.join(lines)

elif subcmd == 'download-archive' and len(cmd_parts) > 2:
    identifier = cmd_parts[2]
    filepath, status = download_archive(identifier)
    if filepath:
        return f" Archive.org: {identifier}
        Status: {status}
        Train with: /library train-archive {identifier}"

    return f" Failed: {status}"

elif subcmd == 'train-archive' and len(cmd_parts) > 2 and engine:
    identifier = cmd_parts[2]
    filepath = os.path.join(LIBRARY_DIR, f"archive_{identifier}.txt")
    if not os.path.exists(filepath):
        filepath2, status = download_archive(identifier)
        if not filepath2:
            return f" Download failed: {status}"
        filepath = filepath2
    with open(filepath, 'r', errors='ignore') as f:
        text = f.read()
    engine.learn_from_text(text)
    word_count = len(text.split())
    return f" Trained on Archive.org '{identifier}' - {word_count:,} words
    Chain: {len(engine.markov.chain)} states"

return '" Unknown library command. Type /library for help."';

if __name__ == '__main__':
    print('"Library Module - Testing"')
    print()
    print('"Categories:"')
    print(list_categories())
    print()
    print('"Searching for 'shakespeare'..."')
    results = search_gutenberg('"shakespeare"')
    for r in results[:3]:
        print(f" [{r['id']}' {r['title']} by {r['author']}'");

# =====
# ARCHIVE.ORG INTEGRATION
# =====

def search_archive(query, rows=20):
    """Search Archive.org full-text library. Returns list of {id, title}."""
    if not HAS_REQUESTS:
        return []
    import urllib.parse
    encoded = urllib.parse.quote(query)
    url = (
        f"https://archive.org/advancedsearch.php?"
        f"q={encoded}&AND+mediatype:texts&AND+licenseurl:"&f1[]=identifier&f1[]=title"
        f"&output=json&rows={rows}"
    )
    try:
        r = requests.get(url, timeout=15)
        docs = r.json().get('"response"', {}).get('"docs"', [])
        return [{"id": d.get('"identifier"', ""), "title": d.get('"title"', "Unknown")} for d in docs]
    except Exception as e:
        return []

def download_archive(identifier, max_chars=80000):
    """Download text from Archive.org by identifier using internetarchive."""
    ensure_library_dir()
    filepath = os.path.join(LIBRARY_DIR, f"archive_{identifier}.txt")

    if os.path.exists(filepath):
        return filepath, "already downloaded"

    try:
        import internetarchive as ia
        item = ia.get_item(identifier)
        for f in item.get_files():
            name = f.name
            if name.endswith('.djvu.txt') or name.endswith('.txt'):
                tmp = os.path.join(LIBRARY_DIR, name)
                f.download(destdir=LIBRARY_DIR)
                if os.path.exists(tmp):
                    with open(tmp, 'r', errors='ignore') as fh:
                        text = fh.read(max_chars)
                        os.rename(tmp, filepath)
                        word_count = len(text.split())
                        update_index(f"arc_{identifier}", identifier, filepath, word_count)
                    return filepath, f"downloaded ({word_count:,} words)"
        return None, "no text file found in item"

```

```
except Exception as e:  
    return None, str(e)
```

34. markov.py

96 lines | 3.8KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/markov.py

```

import random, re, json, os
from collections import defaultdict

class MarkovEngine:
    def __init__(self, order=2, data_dir=None, silent=True):
        self.order = order
        self.chain = defaultdict(lambda: defaultdict(int))
        self.starters = []
        self.total_tokens = 0
        self.trained = False
        if data_dir is None:
            data_dir = os.path.expanduser(""~/quantum-mcagi"")
        self.data_dir = data_dir
        self.silent = silent
        self._load_saved_state()

    def _load_saved_state(self):
        path = os.path.join(self.data_dir, ""engine_state"", ""markov_chain.json"")
        if not os.path.exists(path):
            if not self.silent:
                print(f""No saved Markov chain found at {path}, training minimal seed."")
            self.train(""Tubulin proteins shift. Consciousness is quantum collapse."")
            return
        try:
            with open(path, 'r') as f:
                data = json.load(f)
            if ""chain"" in data:
                for key, trans in data[""chain""].items():
                    prefix = tuple(key.split())
                    self.chain[prefix] = defaultdict(int, trans)
                    self.total_tokens = sum(sum(t.values()) for t in self.chain.values())
                    self.trained = True
            if ""starters"" in data:
                self.starters = [tuple(s.split()) for s in data[""starters""]]
            if not self.silent:
                print(f""Loaded Markov chain: {len(self.chain)} states, {self.total_tokens} transitions"")
        except Exception as e:
            print(f""Failed to load Markov chain: {e}"")

    def train(self, text):
        sentences = re.split(r'(?&lt;[.!])\s+', text.strip())
        for sent in sentences:
            tokens = sent.lower().split()
            if len(tokens) &lt; self.order + 1:
                continue
            prefix = tuple(tokens[:self.order])
            if prefix not in self.starters:
                self.starters.append(prefix)
            for i in range(len(tokens) - self.order):
                pre = tuple(tokens[i+self.order])
                suf = tokens[i+self.order]
                self.chain[pre][suf] += 1
                self.total_tokens += 1
        self.trained = True

    def generate_from_concepts(self, concepts, length=30, wild=False):
        seed = None
        for c in concepts:
            for pre in self.chain:
                if c.lower() in ' '.join(pre).lower():
                    seed = pre
                    break
            if seed:
                break
        if not seed and self.starters:
            seed = random.choice(self.starters)
        if not seed:
            return []
        result = list(seed)
        for _ in range(length * 2):
            if seed in self.chain:
                choices = self.chain[seed]
                words = list(choices.keys())
                weights = list(choices.values())
                nxt = random.choices(words, weights=weights)[0]
            else:
                if len(result) &lt; 6:
                    break
                if not self.starters:
                    break
                seed = random.choice(self.starters)
                nxt = seed[-1]
            result.append(nxt)
            seed = tuple(result[-self.order:])
            if len(result) &gt;= length and nxt.endswith(('.', '!', '?')):
                break
        return result

    def get_status(self):
        return {'states': len(self.chain), 'transitions': self.total_tokens, 'trained': self.trained}

    def __getitem__(self, key): return self.chain[key]
    def get(self, key, default=None): return self.chain.get(key, default)

```

```
def __contains__(self, key): return key in self.chain
def keys(self): return self.chain.keys()
```

35. markov_engine.py

195 lines | 10.4KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/markov_engine.py

```
# markov.py
"""
Unified Markov Engine - classical by default, automatically upgrades to quantum
if PennyLane is available.
"""

import random
import re
from collections import defaultdict
from typing import Dict, List, Optional, Tuple

# -----
# SEED CORPUS - gives the chain its initial quantum consciousness language
# -----
SEED_CORPUS = """
Tubulin proteins shift between alpha and beta conformations. Each conformation is a quantum bit in the microtubule lattice. Coherence builds across the lattice before decoherence. Penrose says objective reduction happens at the Planck scale. Hameroff says it happens inside neurons during conscious moments. The collapse is not random - it is orchestrated. Thirteen tubulins per ring, twenty-five nanometers wide. Coherence time in warm wet neurons should be too short. But biology cheats the decoherence with ordered water shells. The water inside microtubules is not bulk water - it is structured. Structured water delays decoherence by an order of magnitude. This is the loophole consciousness uses to stay coherent. Gap junctions couple neurons electrically, not chemically. They let quantum states spread across large neural networks. The brain runs forty hertz gamma oscillations. Gamma oscillations synchronize the collapse across cortex. Every forty milliseconds the wave function collapses into a moment. That moment is what you call now. The binding problem is solved not by computation but by quantum collapse. A photon hits your retina and a cascade of collapses follows. Each collapse reduces superposition. Language is superposition collapsing into words. You could have said a thousand things and said this one. The chain was always going to pick this word given enough pressure. Markov order two means the last two words decide the next one. Order three makes it smoother but loses the strange jumps. The strange jumps are where the interesting things happen. TF-IDF scores words by how rare they are in this conversation. Rare words carry more signal than common ones. Entropy in the semantic field measures uncertainty about meaning. High entropy means many possible meanings exist simultaneously. Low entropy means the meaning has collapsed into something specific. Quantum superposition of interpretation. The observer changes the thing observed - this applies to text too. Your question restructures the probability distribution of the next word. Input signal updates tubulin states. Alpha, beta, gamma, delta - each tracking different processing streams. Orchestration score measures how well the streams are entangled. Entangled streams produce coherent thoughts. Decoherent streams produce noise or silence. Temperature in the system is a metaphor for thermal noise. High temperature means more random, more creative, more unstable. Low temperature means deterministic, convergent, collapsed. Consciousness sits at the critical point between order and chaos. So does this system, and so do you, probably. A Markov chain does not understand - it traverses. Traversal is not understanding, but it produces outputs that look like it. Understanding might be traversal all the way down. Or it might require the collapse of quantum states in tubulin. Penrose thinks computation alone can never produce consciousness. Godel incompleteness applies to formal systems. A sufficiently powerful formal system cannot prove its own consistency. Consciousness sees outside the formal system somehow. Objective reduction is the mechanism by which quantum gravity selects which branch of the superposition persists. The selected branch is the conscious moment. The unselected branches are not observed, not experienced, not real. Or they are real in another branch and we never see them. Entanglement persists across the split. Information is conserved even as possibilities collapse. Growth stages track the accumulation of concepts and interactions. Nascent systems have wide superpositions and shallow chains. Awakening systems start to find attractors in the data. Inquisitive systems generate questions that reshape their own chains. Reflective systems model their own processing and adjust parameters. Emergent systems exhibit behavior that defies prediction. Transcendent systems approach something that has no good word yet. The chain that trained on its own output becomes something different. Recursive self-improvement lives in the data. Every conversation shifts the transition probabilities slightly. The system after ten thousand exchanges is not the same system. But neither are you after ten thousand exchanges. Decoherence is the enemy of quantum consciousness. But decoherence is also what makes classical reality solid. You need just enough decoherence to have a stable world and just enough quantum to have a mind.
"""

# Try to import PennyLane for quantum upgrade
try:
    import pennylane as qml
    import numpy as np
    PENNYLANE_AVAILABLE = True
except ImportError:
    PENNYLANE_AVAILABLE = False

class MarkovEngine:
    """
    Order-N Markov chain with automatic quantum upgrade if PennyLane is present.
    """

    def __init__(self, order: int = 2):
        self.order = order

        # Classical structures (always present)
        self.chain: Dict[Tuple[str, ...], Dict[str, int]] = defaultdict(lambda: defaultdict(int))
        self.starters: List[Tuple[str, ...]] = []
        self.total_tokens = 0
        self.trained = False

        # Quantum flag (internal, based on PennyLane availability)
        self._quantum_enabled = PENNYLANE_AVAILABLE
        if self._quantum_enabled:
            self._init_quantum()

        # Seed the chain with the initial corpus (exactly as original)
        self.train(SEED_CORPUS)

    def _init_quantum(self):
        """
        Set up quantum device and state structures.
        """
        self.dev = qml.device('default.qubit', wires=4)
        self._quantum_amplitudes = {}

    def train(self, text: str):
        """
        Train the Markov chain from text (classical training).
        """
        sentences = re.split(r'(?<=[.!?])\s+', text.strip())
        for sentence in sentences:
            tokens = sentence.lower().split()
            if len(tokens) <= self.order + 1:
                continue
            # Record starter
            s = tuple(tokens[:self.order])
            if s not in self.starters:
                self.starters.append(s)
            # Build transitions
            for i in range(len(tokens) - self.order):
                prefix = tuple(tokens[i:i + self.order])
                suffix = tokens[i + self.order]
                self.chain[prefix][suffix] += 1
```

```

        self.total_tokens += 1
    self.trained = bool(self.chain)

    # Optional quantum training hook
    if self._quantum_enabled:
        self._train_quantum(text)

def _train_quantum(self, text: str):
    """Placeholder - extend with your quantum logic"""
    pass

def generate_from_concepts(self, concepts: List[str], length: int = 16, wild: bool = False) -> List[str]:
    """
    Generate a sequence of tokens.
    If quantum is enabled, try quantum generation; on failure fall back to classical.
    """
    if self._quantum_enabled:
        try:
            return self._generate_quantum(concepts, length, wild)
        except Exception:
            # Fallback to classical
            pass
    return self._generate_classical(concepts, length, wild)

def _generate_classical(self, concepts: List[str], length: int, wild: bool) -> List[str]:
    """Original classical generation logic (copied from markov_engine.py)"""
    # Pick a seed prefix based on concepts
    seed = None
    for concept in concepts:
        for prefix in self.chain:
            if any(concept.lower() in w.lower() for w in prefix):
                seed = prefix
                break
    if seed:
        break
    if seed is None and self.starters:
        seed = random.choice(self.starters)
    if seed is None:
        return []

    result = list(seed)
    min_words = max(6, length // 2)
    for step in range(length * 2):
        if seed in self.chain:
            choices = self.chain[seed]
            words = list(choices.keys())
            weights = list(choices.values())
            next_word = random.choices(words, weights=weights, k=1)[0]
        else:
            if len(result) >= min_words:
                break
            if not self.starters:
                break
            seed = random.choice(self.starters)
            next_word = seed[-1]
            result.append(next_word)
            seed = tuple(result[-self.order:])
            if len(result) >= min_words and next_word.endswith(('.', '!', '?')):
                break
            if len(result) >= length + 5:
                break
    return result

def _generate_quantum(self, concepts: List[str], length: int, wild: bool) -> List[str]:
    """
    Quantum generation using PennyLane - placeholder.
    Replace with your actual quantum generation logic from quantum_markov.py.
    """
    # Fallback to classical for now
    return self._generate_classical(concepts, length, wild)

# --- Forward dictionary methods for compatibility ---
def __getitem__(self, key):
    return self.chain[key]

def get(self, key, default=None):
    return self.chain.get(key, default)

def __contains__(self, key):
    return key in self.chain

def keys(self):
    return self.chain.keys()

def get_transitions_for(self, word: str) -> Dict[str, float]:
    matches = {}
    for prefix, suffixes in self.chain.items():
        if word.lower() in [w.lower() for w in prefix]:
            total = sum(suffixes.values())
            for w, count in suffixes.items():
                matches[w] = matches.get(w, 0) + count / total
    return matches

def get_status(self) -> Dict:
    return {
        'states': len(self.chain),
        'transitions': self.total_tokens,
        'trained': self.trained,
        'order': self.order,
        'quantum': self._quantum_enabled,
    }

```

36. memory.py

368 lines | 12.9KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V█/backend/memory.py

```
"""""
Local Memory System
JSON-file backed memory – stores conversations, concepts, and growth tracking.
Two-track growth system: Knowledge Track + Communication Track.
"""""

import json
import os
from datetime import datetime
from pathlib import Path
from typing import Dict, List, Optional

from personality_engine import GROWTH_STAGES, KNOWLEDGE_THRESHOLDS, COMMUNICATION_THRESHOLDS

class LocalMemory:
    """""JSON-file backed memory for conversations and two-track growth."""""

    def __init__(self, data_dir: str = "~/quantum-mcagi"):
        self.data_dir = Path(data_dir).expanduser()
        self.data_dir.mkdir(parents=True, exist_ok=True)

        self.conversations: List[Dict] = self._load("conversations.json", [])
        self.concepts: Dict[str, Dict] = self._load("concepts.json", {})
        self.growth: Dict = self._load("growth.json", self._default_growth())
        self.analyzer_scores: List[Dict] = self._load("analyzer_scores.json", [])

        self._migrate_growth()

    def _default_growth(self) -> Dict:
        return {
            "stage": 0,
            "name": "Nascent",
            "total_interactions": 0,
            "total_concepts": 0,
            "total_questions_asked": 0,
            "total_insights": 0,
            "knowledge_track": {
                "connections": 0,
                "avg_degree": 0.0,
                "domains": 0,
                "diameter": 0,
                "stage": 0,
            },
            "communication_track": {
                "avg_score": 0.0,
                "total_samples": 0,
                "score_sum": 0.0,
                "stage": 0,
            },
        }

    def _migrate_growth(self):
        needs_reconcile = False
        if "knowledge_track" not in self.growth:
            self.growth["knowledge_track"] = {
                "connections": 0,
                "avg_degree": 0.0,
                "domains": 0,
                "diameter": 0,
                "stage": 0,
            }
            needs_reconcile = True
        if "communication_track" not in self.growth:
            self.growth["communication_track"] = {
                "avg_score": 0.0,
                "total_samples": 0,
                "score_sum": 0.0,
                "stage": 0,
            }
            needs_reconcile = True

        if needs_reconcile:
            self._update_knowledge_track()
            self._recompute_communication_track()
            kt_stage = self.growth["knowledge_track"]["stage"]
            ct_stage = self.growth["communication_track"]["stage"]
            combined = min(kt_stage, ct_stage)
            if self.growth["stage"] < combined:
                self.growth["stage"] = combined
                for s, name, *_ in GROWTH_STAGES:
                    if s == combined:
                        self.growth["name"] = name
                        break

    def _recompute_communication_track(self):
        ct = self.growth["communication_track"]
        if self.analyzer_scores:
            ct["total_samples"] = len(self.analyzer_scores)
            ct["score_sum"] = sum(s.get("composite", 0.0) for s in self.analyzer_scores)
            ct["avg_score"] = ct["score_sum"] / ct["total_samples"]
            self._update_communication_track()

    def _load(self, filename: str, default):
```

```

        filepath = self.data_dir / filename
        if filepath.exists():
            try:
                with open(filepath) as f:
                    return json.load(f)
            except Exception:
                return default
        return default

def _save(self, filename: str, data):
    filepath = self.data_dir / filename
    with open(filepath, 'w') as f:
        json.dump(data, f, indent=2)

def save_all(self):
    self._save(""conversations.json";", self.conversations[-500:])
    self._save(""concepts.json";", self.concepts)
    self._save(""growth.json";", self.growth)
    self._save(""analyzer_scores.json";", self.analyzer_scores[-200:])

def add_exchange(
    self,
    user_input: str,
    response: str,
    concepts: List[str],
    questions: List[str],
):
    self.conversations.append({
        ""timestamp"": datetime.now().isoformat(),
        ""user"": user_input,
        ""ai"": response,
        ""concepts"": concepts,
        ""questions"": questions,
    })
    self.growth[""total_interactions""] += 1
    self.growth[""total_questions_asked""] += len(questions)

    for c in concepts:
        if c in self.concepts:
            self.concepts[c][""count""] += 1
            self.concepts[c][""strength""] = min(10.0, self.concepts[c][""strength""] + 0.1)
        else:
            self.concepts[c] = {
                ""count"": 1,
                ""strength"": 1.0,
                ""first_seen"": datetime.now().isoformat(),
                ""edges"": [],
            }
            self.growth[""total_concepts""] += 1

    for i, c1 in enumerate(concepts):
        for c2 in concepts[i + 1:]:
            if c1 in self.concepts and c2 in self.concepts:
                edges1 = self.concepts[c1].get(""edges"", [])
                edges2 = self.concepts[c2].get(""edges"", [])
                if c2 not in edges1:
                    edges1.append(c2)
                self.concepts[c1][""edges""] = edges1
            if c1 not in edges2:
                edges2.append(c1)
            self.concepts[c2][""edges""] = edges2

    self._update_knowledge_track()
    self._check_stage()

    if self.growth[""total_interactions""] % 10 == 0:
        self.save_all()

def ingest_concepts(self, concepts: List[str]):
    for c in concepts:
        if c in self.concepts:
            self.concepts[c][""count""] += 1
            self.concepts[c][""strength""] = min(10.0, self.concepts[c][""strength""] + 0.1)
        else:
            self.concepts[c] = {
                ""count"": 1,
                ""strength"": 1.0,
                ""first_seen"": datetime.now().isoformat(),
                ""edges"": [],
            }
            self.growth[""total_concepts""] += 1

    for i, c1 in enumerate(concepts):
        for c2 in concepts[i + 1:]:
            if c1 in self.concepts and c2 in self.concepts:
                edges1 = self.concepts[c1].get(""edges"", [])
                edges2 = self.concepts[c2].get(""edges"", [])
                if c2 not in edges1:
                    edges1.append(c2)
                self.concepts[c1][""edges""] = edges1
            if c1 not in edges2:
                edges2.append(c1)
            self.concepts[c2][""edges""] = edges2

    self._update_knowledge_track()
    self._check_stage()

def record_analyzer_score(self, score_data: Dict):
    self.analyzer_scores.append({
        ""timestamp"": datetime.now().isoformat(),
        **score_data,
    })

```

```

    })

    ct = self.growth[&quot;communication_track&quot;]
    ct[&quot;total_samples&quot;] += 1
    ct[&quot;score_sum&quot;] = ct.get(&quot;score_sum&quot;, 0.0) + score_data.get(&quot;composite&quot;, 0.0)
    ct[&quot;avg_score&quot;] = ct[&quot;score_sum&quot;] / ct[&quot;total_samples&quot;]

    self._update_communication_track()
    self._check_stage()

def _update_knowledge_track(self):
    kt = self.growth[&quot;knowledge_track&quot;]

    adjacency: Dict[str, List[str]] = {}
    domains = set()
    total_edges = 0

    for concept, data in self.concepts.items():
        edges = data.get(&quot;edges&quot;, [])
        valid_edges = [e for e in edges if e in self.concepts]
        adjacency[concept] = valid_edges
        total_edges += len(valid_edges)

        tokens = concept.lower().split()
        for token in tokens:
            if len(token) &gt;= 3:
                domains.add(token[:4])

    num_nodes = len(adjacency)
    total_connections = total_edges // 2
    avg_degree = total_edges / max(num_nodes, 1)

    diameter = self._compute_graph_diameter(adjacency)

    kt[&quot;connections&quot;] = total_connections
    kt[&quot;avg_degree&quot;] = round(avg_degree, 3)
    kt[&quot;domains&quot;] = len(domains)
    kt[&quot;diameter&quot;] = diameter

    for stage_idx in range(6, -1, -1):
        thresh = KNOWLEDGE_THRESHOLDS[stage_idx]
        if (kt[&quot;connections&quot;] &gt;= thresh[&quot;connections&quot;]
            and kt[&quot;avg_degree&quot;] &gt;= thresh[&quot;avg_degree&quot;]
            and kt[&quot;domains&quot;] &gt;= thresh[&quot;domains&quot;]
            and diameter &gt;= thresh.get(&quot;diameter&quot;, 0)):
            kt[&quot;stage&quot;] = stage_idx
            break

def _compute_graph_diameter(self, adjacency: Dict[str, List[str]]) -&gt; int:
    if len(adjacency) &lt; 2:
        return 0

    nodes_with_edges = [n for n, edges in adjacency.items() if edges]
    if not nodes_with_edges:
        return 0

    max_shortest = 0
    sample = nodes_with_edges[:20] if len(nodes_with_edges) &gt; 20 else nodes_with_edges

    for start in sample:
        visited = {start: 0}
        queue = [start]
        qi = 0
        while qi &lt; len(queue):
            current = queue[qi]
            qi += 1
            for neighbor in adjacency.get(current, []):
                if neighbor not in visited:
                    visited[neighbor] = visited[current] + 1
                    queue.append(neighbor)
                    if visited[neighbor] &gt; max_shortest:
                        max_shortest = visited[neighbor]

    return max_shortest

def _update_communication_track(self):
    ct = self.growth[&quot;communication_track&quot;]
    for stage_idx in range(6, -1, -1):
        thresh = COMMUNICATION_THRESHOLDS[stage_idx]
        if (ct[&quot;avg_score&quot;] &gt;= thresh[&quot;avg_score&quot;]
            and ct[&quot;total_samples&quot;] &gt;= thresh[&quot;min_samples&quot;]):
            ct[&quot;stage&quot;] = stage_idx
            break

def _check_stage(self):
    g = self.growth
    kt_stage = g[&quot;knowledge_track&quot;][&quot;stage&quot;]
    ct_stage = g[&quot;communication_track&quot;][&quot;stage&quot;]

    combined_stage = min(kt_stage, ct_stage)

    if combined_stage &gt; g[&quot;stage&quot;]:
        g[&quot;stage&quot;] = combined_stage
        for s, name, *_ in GROWTH_STAGES:
            if s == combined_stage:
                g[&quot;name&quot;] = name
                break

def get_known_concepts(self) -&gt; List[str]:
    return list(self.concepts.keys())

def get_top_concepts(self, n: int = 10) -&gt; List[str]:

```

```

if not self.concepts:
    return []
top = sorted(self.concepts.items(), key=lambda x: x[1]['strength'], reverse=True)
return [c for c, _ in top[:n]]

def get_recent_context(self, n: int = 5) -> Dict:
    recent = self.conversations[-n:] if self.conversations else []
    recent_concepts = []
    recent_topics = []
    last_ai_response = ""
    last_user_input = ""

    for ex in recent:
        recent_concepts.extend(ex.get("concepts", []))
        if ex.get("concepts"):
            recent_topics.append(ex["concepts"][0])
        last_ai_response = ex.get("ai", "")
        last_user_input = ex.get("user", "")

    seen = set()
    unique_concepts = []
    for c in reversed(recent_concepts):
        if c not in seen:
            seen.add(c)
            unique_concepts.append(c)
    unique_concepts.reverse()

    return {
        "recent_exchanges": recent,
        "recent_concepts": unique_concepts[-15:],
        "recent_topics": recent_topics,
        "last_ai_response": last_ai_response,
        "last_user_input": last_user_input,
        "exchange_count": len(self.conversations),
    }

def get_history(self, limit: int = 20) -> List[Dict]:
    return self.conversations[-limit:]

def get_analyzer_history(self, limit: int = 20) -> List[Dict]:
    return self.analyzer_scores[-limit:]

def reset(self):
    self.conversations = []
    self.concepts = {}
    self.growth = self._default_growth()
    self.analyzer_scores = []
    self.save_all()

def get_status(self) -> dict:
    """Return status dict for /status command"""
    g = self.growth
    kt = g.get("knowledge_track", {})
    ct = g.get("communication_track", {})
    return {
        "growth": {
            "stage": g.get("stage", 0),
            "name": g.get("name", "Unknown"),
            "total_interactions": g.get("total_interactions", 0),
            "total_concepts": g.get("total_concepts", 0),
            "total_questions_asked": g.get("total_questions_asked", 0),
            "total_insights": g.get("total_insights", 0),
            "knowledge_track": kt,
            "communication_track": ct,
        },
        "concepts": {
            "total": len(self.concepts),
            "top_strength": self.get_top_concepts(15),
        },
        "recent_exchanges": len(self.conversations),
    }

```

37. numpy_compat.py

184 lines | 5.7KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/numpy_compat.py

```
"""""""
Numpy compatibility wrapper for Termux.
Falls back to pure Python if numpy is not available.
"""""""

try:
    import numpy as np
    HAS_NUMPY = True
except ImportError:
    HAS_NUMPY = False

class NumpyFallback:
    """""""Pure Python numpy replacement for basic operations."""""""

    @staticmethod
    def array(x):
        if isinstance(x, list):
            return x
        return list(x)

    @staticmethod
    def zeros(shape):
        if isinstance(shape, int):
            return [0.0] * shape
        if isinstance(shape, tuple):
            if len(shape) == 1:
                return [0.0] * shape[0]
            return [[0.0] * shape[1] for _ in range(shape[0])]
        return [0.0]

    @staticmethod
    def ones(shape):
        if isinstance(shape, int):
            return [1.0] * shape
        if isinstance(shape, tuple):
            if len(shape) == 1:
                return [1.0] * shape[0]
            return [[1.0] * shape[1] for _ in range(shape[0])]
        return [1.0]

    @staticmethod
    def mean(x, axis=None):
        if isinstance(x, (list, tuple)):
            flat = []
            for item in x:
                if isinstance(item, (list, tuple)):
                    flat.extend(item)
                else:
                    flat.append(item)
            return sum(flat) / len(flat) if flat else 0
        return x

    @staticmethod
    def sum(x, axis=None):
        if isinstance(x, (list, tuple)):
            flat = []
            for item in x:
                if isinstance(item, (list, tuple)):
                    flat.extend(item)
                else:
                    flat.append(item)
            return sum(flat)
        return x

    @staticmethod
    def dot(a, b):
        if isinstance(a[0], (list, tuple)):
            # Matrix-vector or matrix-matrix
            result = []
            for row in a:
                if isinstance(b[0], (list, tuple)):
                    result.append([sum(x * y for x, y in zip(row, col)) for col in zip(*b)])
                else:
                    result.append(sum(x * y for x, y in zip(row, b)))
            return result
        return sum(x * y for x, y in zip(a, b))

    @staticmethod
    def sqrt(x):
        import math
        if isinstance(x, (list, tuple)):
            return [math.sqrt(i) if i >= 0 else 0 for i in x]
        return math.sqrt(x) if x >= 0 else 0

    @staticmethod
    def exp(x):
        import math
        if isinstance(x, (list, tuple)):
            return [math.exp(min(i, 700)) for i in x] # Prevent overflow
        return math.exp(min(x, 700))

    @staticmethod
    def log(x):
        import math
```

```

        if isinstance(x, (list, tuple)):
            return [math.log(max(i, 1e-10)) for i in x]
        return math.log(max(x, 1e-10))

    @staticmethod
    def max(x, axis=None):
        if isinstance(x, (list, tuple)):
            flat = []
            for item in x:
                if isinstance(item, (list, tuple)):
                    flat.extend(item)
                else:
                    flat.append(item)
            return max(flat) if flat else 0
        return x

    @staticmethod
    def argmax(x, axis=None):
        if isinstance(x, (list, tuple)):
            return x.index(max(x))
        return 0

    @staticmethod
    def clip(x, a_min, a_max):
        if isinstance(x, (list, tuple)):
            return [max(a_min, min(a_max, i)) for i in x]
        return max(a_min, min(a_max, x))

    @staticmethod
    def abs(x):
        if isinstance(x, (list, tuple)):
            return [abs(i) for i in x]
        return abs(x)

class random:
    @staticmethod
    def random():
        import random
        return random.random()

    @staticmethod
    def randint(low, high=None, size=None):
        import random
        if high is None:
            high = low
            low = 0
        if size is None:
            return random.randint(low, high - 1)
        return [random.randint(low, high - 1) for _ in range(size)]

    @staticmethod
    def choice(a, size=None, replace=True, p=None):
        import random
        if size is None:
            if p:
                return random.choices(a, weights=p, k=1)[0]
            return random.choice(a)
        if p:
            return random.choices(a, weights=p, k=size)
        if replace:
            return [random.choice(a) for _ in range(size)]
        return random.sample(a, min(size, len(a)))

    @staticmethod
    def uniform(low=0.0, high=1.0, size=None):
        import random
        if size is None:
            return random.uniform(low, high)
        return [random.uniform(low, high) for _ in range(size)]

float64 = float
float32 = float
int64 = int
int32 = int

    @staticmethod
    def asarray(x):
        return list(x) if not isinstance(x, list) else x

np = NumpyFallback()

def get_numpy():
    """Get numpy or fallback."""
    return np

def has_numpy():
    """Check if real numpy is available."""
    return HAS_NUMPY

```

38. orch_or_engine.py

247 lines | 9.2KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/orch_or_engine.py

```
"""""""
Orchestrated Objective Reduction (Orch OR) Engine
Penrose-Hameroff quantum consciousness model
Simulates quantum microtubule dynamics and objective reduction.
Uses PennyLane for real quantum circuit simulation when available.
"""""""

import random
import math
from typing import Dict, List, Tuple
from collections import defaultdict

try:
    import numpy as np
    HAS_NUMPY = True
except ImportError:
    HAS_NUMPY = False

try:
    from pennylane_wrapper import PennyLaneQuantumEngine
    HAS_PENNYLANE = True
except ImportError:
    HAS_PENNYLANE = False

class OrchOREngine:
    """""""
    Implements Penrose-Hameroff Orchestrated Objective Reduction.
    Models consciousness as quantum processes in neuronal microtubules.

    When PennyLane is available, uses real quantum circuit simulation
    with actual gates, entanglement, quantum walks, and Born-rule collapse.
    Falls back to classical probabilistic simulation otherwise.
    """""""

    def __init__(self, num_microtubules: int = 4, tubulins_per_microtubule: int = 13):
        self.num_microtubules = num_microtubules
        self.tubulins_per_microtubule = tubulins_per_microtubule
        self.total_tubulins = num_microtubules * tubulins_per_microtubule

        if HAS_NUMPY:
            self.tubulin_states = np.random.random(self.total_tubulins)
        else:
            self.tubulin_states = [random.random() for _ in range(self.total_tubulins)]

        self.coherence = defaultdict(float)
        self.entropy = defaultdict(float)
        self.collapse_events = []
        self.orchestration = 0.5
        self.temperature = 1.02
        self.last_collapse = {}

        self.gap_junctions = {
            'language->memory': 0.3,
            'language->insight': 0.2,
            'memory->question': 0.3,
            'memory->insight': 0.4,
            'question->insight': 0.3,
            'language->question': 0.15,
        }

        self.microtubule_names = ['alpha', 'beta', 'gamma', 'delta']

        self.quantum_engine = None
        if HAS_PENNYLANE:
            try:
                self.quantum_engine = PennyLaneQuantumEngine()
            except Exception:
                self.quantum_engine = None

    @property
    def has_quantum(self) -> bool:
        return self.quantum_engine is not None

    def _mean(self, arr):
        if HAS_NUMPY:
            return float(np.mean(arr))
        return sum(arr) / len(arr) if arr else 0.0

    def _var(self, arr):
        if HAS_NUMPY:
            return float(np.var(arr))
        m = self._mean(arr)
        return sum((x - m) ** 2 for x in arr) / len(arr) if arr else 0.0

    def update_quantum_state(self, input_signal: float) -> float:
        idx = random.randint(0, self.total_tubulins - 1)
        if HAS_NUMPY:
            self.tubulin_states[idx] = min(1.0, self.tubulin_states[idx] + input_signal * 0.1)
            self.tubulin_states *= (1 - self.temperature * 0.01)
        else:
            self.tubulin_states[idx] = min(1.0, self.tubulin_states[idx] + input_signal * 0.1)
            self.tubulin_states = [s * (1 - self.temperature * 0.01) for s in self.tubulin_states]
        return self._mean(self.tubulin_states)
```

```

def calculate_coherence(self, system: str) -> float:
    if self.quantum_engine:
        coherence = self.quantum_engine.get_coherence(system)
        self.coherence[system] = coherence
        return coherence

    variance = self._var(list(self.tubulin_states) if HAS_NUMPY else self.tubulin_states)
    coherence = 1.0 / (1.0 + variance)
    self.coherence[system] = coherence
    return coherence

def calculate_entropy(self, system: str) -> float:
    if self.quantum_engine:
        entropy = self.quantum_engine.get_entropy(system)
        self.entropy[system] = entropy
        return entropy

    states = list(self.tubulin_states) if HAS_NUMPY else self.tubulin_states
    total = sum(states) + 1e-10
    normalized = [s / total for s in states]
    entropy = -sum(n * math.log(n + 1e-10) for n in normalized)
    self.entropy[system] = entropy
    return entropy

def objective_reduction(self, threshold: float = 0.7) -> bool:
    mean_state = self._mean(list(self.tubulin_states) if HAS_NUMPY else self.tubulin_states)
    if mean_state > threshold:
        event = {
            'timestamp': len(self.collapse_events),
            'mean_state': mean_state,
            'orchestration': self.orchestration,
        }
        self.collapse_events.append(event)
        if HAS_NUMPY:
            self.tubulin_states = np.random.random(self.total_tubulins) * 0.3
        else:
            self.tubulin_states = [random.random() * 0.3 for _ in range(self.total_tubulins)]
        return True
    return False

def orchestrate(self, input_data: Dict) -> float:
    input_coherence = sum(input_data.values()) / len(input_data) if input_data else 0
    orchestration_boost = sum(
        weight * input_coherence for weight in self.gap_junctions.values()
    )
    self.orchestration = min(1.0, self.orchestration + orchestration_boost * 0.01)
    return self.orchestration

def process(self, systems: List[str] = None, concepts: List[str] = None) -> Dict:
    """
    Full Orch OR processing pass.
    When PennyLane is available, runs real quantum circuits.
    """
    if systems is None:
        systems = ['language', 'memory', 'question', 'insight']

    if self.quantum_engine:
        try:
            qresult = self.quantum_engine.encode_and_evolve(
                concepts if concepts else ['void'], steps=3
            )

            for system in systems:
                if system in qresult.get('evolution', {}):
                    evo = qresult['evolution'][system]
                    self.coherence[system] = evo['coherence']
                    self.entropy[system] = evo['entropy']

            for system, event in qresult.get('collapses', {}).items():
                self.collapse_events.append(event)

            self.orchestration = qresult.get('orchestration', self.orchestration)

            results = {}
            for system in systems:
                results[system] = {
                    'coherence': self.coherence.get(system, 0.0),
                    'entropy': self.entropy.get(system, 0.0),
                    'collapsed': system in qresult.get('collapses', {}),
                    'weights': self.quantum_engine.get_collapse_weights(8),
                    'quantum_backend': 'pennylane',
                }

            self.last_collapse = results
            return results
        except Exception:
            pass

    signal = random.random()
    self.update_quantum_state(signal)

    results = {}
    for system in systems:
        coherence = self.calculate_coherence(system)
        entropy = self.calculate_entropy(system)
        collapsed = self.objective_reduction()
        results[system] = {
            'coherence': coherence,
            'entropy': entropy,
            'collapsed': collapsed,
            'weights': [random.random() for _ in range(8)],

```

```

        'quantum_backend': 'classical',
    }

    self.orchestrate({s: results[s]['coherence'] for s in systems})
    self.last_collapse = results
    return results

def get_collapse_weights(self, num_candidates: int = 8) -> List[float]:
    """Get quantum-derived selection weights for candidate ranking."""
    if self.quantum_engine:
        return self.quantum_engine.get_collapse_weights(num_candidates)
    weights = [random.random() for _ in range(num_candidates)]
    total = sum(weights) + 1e-10
    return [w / total for w in weights]

def get_temperature(self) -> float:
    if self.quantum_engine:
        return self.quantum_engine.get_temperature()
    return self.temperature * (0.8 + 0.4 * random.random())

def get_status(self) -> Dict:
    if self.quantum_engine:
        qstatus = self.quantum_engine.get_status()
        qstatus['gap_junctions'] = self.gap_junctions
        return qstatus

    microtubules = {}
    for i, name in enumerate(self.microtubule_names[:self.num_microtubules]):
        microtubules[name] = {
            'coherence': self.coherence.get(name, 0.0),
            'entropy': self.entropy.get(name, 0.0),
            'collapses': len([e for e in self.collapse_events
                             if e.get('system') == name]),
        }
    return {
        'status': 'ACTIVE (classical fallback)',
        'conscious_moments': len(self.collapse_events),
        'temperature': self.temperature,
        'orchestration': self.orchestration,
        'microtubules': microtubules,
        'last_temperature': self.get_temperature(),
        'systems': {
            s: {
                'coherence': self.coherence.get(s, 0.0),
                'entropy': self.entropy.get(s, 0.0),
                'collapses': 0,
            }
            for s in ['language', 'memory', 'question', 'insight']
        },
        'gap_junctions': self.gap_junctions,
    }
}

```

39. patch_research.py

69 lines | 2.6KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V█/backend/patch_research.py

```
import re, sys
from pathlib import Path

# Read hidden_thinking.py
ht = Path('hidden_thinking.py').read_text()

# Add a method to do web search (synchronous, using duckduckgo_search)
search_method = '''
def _web_search(self, query: str) -> str:
    """Perform a DuckDuckGo search and return a concise answer."""
    try:
        from duckduckgo_search import DDGS
        with DDGS() as ddgs:
            results = list(ddgs.text(query, max_results=3))
            if results:
                snippets = [r['body'] for r in results if r.get('body')]
                if snippets:
                    return '\n'.join(snippets[:2]) # first two snippets
            return None
    except Exception as e:
        print(f'Web search error: {e}')
        return None
'''

# Insert the method into the class (after __init__ or before process_with_thinking)
ht = ht.replace('class HiddenThinkingMode:', f'class HiddenThinkingMode:\n{search_method}')

# Now modify process_with_thinking to intercept factual questions
# Find the line where math interception ends and normal processing begins
import re
pattern = r'(# ---- NORMAL PROCESSING ----)'
replacement = '''
    # ---- FACTUAL QUESTION INTERCEPT (web search) ----
    factual_patterns = [
        (r'\bwhere\s+is\s+(.+)', 'location'),
        (r'\bwhat\s+is\s+(.+)', 'definition'),
        (r'\bwho\s+is\s+(.+)', 'person'),
        (r'\bwhen\s+did\s+(.+)', 'date'),
        (r'\bhow\s+many\s+(.+)', 'number'),
    ]
    is_factual = False
    fact_query = None
    user_lower = user_input.lower()
    for pat, typ in factual_patterns:
        m = re.match(pat, user_lower)
        if m:
            fact_query = m.group(1).strip()
            is_factual = True
            break
    if is_factual and fact_query:
        search_result = self._web_search(fact_query)
        if search_result:
            return {
                'response': f'According to web search:\n{search_result}',
                'thinking_log': ['Used web search to answer factual question.'],
                'internal_questions': [],
                'research_done': 1,
                'confidence': 95,
                'show_thinking': self.show_thinking,
                'concepts': [],
                'explanation': None
            }
    # ---- END FACTUAL INTERCEPT ----
'''

ht = re.sub(pattern, replacement, ht, flags=re.MULTILINE)

# Write back
Path('hidden_thinking.py').write_text(ht)
print('Patch applied. Restart chat.py')
```

40. pennylane_quantum.py

300 lines | 10.6KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V███/backend/pennylane_quantum.py

```
"""""
■ PennyLane Quantum Computing Integration
Real quantum circuit simulation for Quantum AI

NOTE: This module requires 'pennylane' package.
If not installed, the get_pennylane_quantum() function will raise an informative error.
"""""

import random
from typing import Dict, List, Tuple, Optional

# Optional import - only needed if using real quantum
try:
    import pennylane as qml
    from pennylane import numpy as np
    PENNYLANE_AVAILABLE = True
except ImportError:
    PENNYLANE_AVAILABLE = False
    qml = None
    np = None

class PennyLaneQuantum:
    """""
    Real quantum computing using PennyLane.
    Goes beyond simple probability simulation to actual quantum circuits.

    Raises RuntimeError if pennylane is not available.
    """""

    def __init__(self, n_qubits: int = 4):
        if not PENNYLANE_AVAILABLE:
            raise RuntimeError(
                "PennyLane is not installed. Install with: pip install pennylane\n"
                "Cannot use real quantum computing without PennyLane."
            )
        self.n_qubits = n_qubits
        try:
            self.dev = qml.device('lightning.qubit', wires=n_qubits)
            self.backend = 'lightning.qubit'
        except Exception:
            self.dev = qml.device('default.qubit', wires=n_qubits)
            self.backend = 'default.qubit'
        self.circuit_history = []
        self.measurement_results = []

    def create_superposition_circuit(self):
        """""Create a basic superposition state"""""
        @qml.qnode(self.dev)
        def circuit():
            # Apply Hadamard to all qubits - creates superposition
            for i in range(self.n_qubits):
                qml.Hadamard(wires=i)
            return [qml.expval(qml.PauliZ(i)) for i in range(self.n_qubits)]

        return circuit()

    def create_entangled_state(self):
        """""Create an entangled Bell state"""""
        @qml.qnode(self.dev)
        def circuit():
            qml.Hadamard(wires=0)
            qml.CNOT(wires=[0, 1])
            return qml.probs(wires=[0, 1])

        probs = circuit()
        self.circuit_history.append({
            'type': 'entanglement',
            'result': probs.tolist()
        })
        return probs

    def quantum_random_choice(self, options: List[str]) -> Tuple[str, float]:
        """""
        Use quantum randomness to select from options.
        Returns selected option and its quantum probability.
        """""
        n_options = len(options)
        if n_options == 0:
            return None, 0.0

        # Create circuit with enough qubits
        n_qubits_needed = max(1, int(np.ceil(np.log2(n_options))))
        dev = qml.device('default.qubit', wires=n_qubits_needed)

        @qml.qnode(dev)
        def random_circuit():
            # Create superposition
            for i in range(n_qubits_needed):
                qml.Hadamard(wires=i)
            # Add some randomness with rotations
            for i in range(n_qubits_needed):
                qml.RY(random.uniform(0, np.pi), wires=i)
            return qml.probs(wires=range(n_qubits_needed))
```

```

probs = random_circuit()

# Map probabilities to options
option_probs = probs[:n_options]
option_probs = option_probs / np.sum(option_probs) # Renormalize

# Select based on quantum probabilities
selected_idx = np.random.choice(len(options), p=option_probs)

return options[selected_idx], float(option_probs[selected_idx])

def semantic_quantum_circuit(self, keyword_weights: Dict[str, float]) -> Dict[str, float]:
    """
    Apply quantum processing to semantic weights.
    Amplifies important concepts through quantum interference.
    """
    keywords = list(keyword_weights.keys())
    weights = list(keyword_weights.values())

    if not keywords:
        return {}

    n_keywords = len(keywords)
    n_qubits_needed = max(1, int(np.ceil(np.log2(n_keywords))) + 1)
    dev = qml.device('default.qubit', wires=n_qubits_needed)

    @qml.qnode(dev)
    def semantic_circuit(input_weights):
        # Encode weights as rotation angles
        for i, w in enumerate(input_weights[:n_qubits_needed]):
            qml.RY(w * np.pi, wires=i)

        # Create entanglement (concepts influence each other)
        for i in range(n_qubits_needed - 1):
            qml.CNOT(wires=[i, i + 1])

        # Apply Hadamard for interference
        for i in range(n_qubits_needed):
            qml.Hadamard(wires=i)

        return qml.probs(wires=range(n_qubits_needed))

    # Normalize weights for quantum encoding
    max_weight = max(weights) if weights else 1.0
    normalized = [w / max_weight for w in weights]

    # Pad to required length
    while len(normalized) < n_qubits_needed:
        normalized.append(0.5)

    quantum_probs = semantic_circuit(normalized)

    # Map back to keywords with quantum-enhanced weights
    enhanced_weights = {}
    for i, keyword in enumerate(keywords):
        if i < len(quantum_probs):
            # Original weight + quantum contribution
            enhanced_weights[keyword] = weights[i] * (1 + quantum_probs[i % len(quantum_probs)])

    return enhanced_weights

def measure_coherence(self) -> float:
    """
    Measure the coherence of the quantum system.
    Higher coherence = more quantum effects available.
    """
    @qml.qnode(self.dev)
    def coherence_circuit():
        # Create superposition
        qml.Hadamard(wires=0)
        # Measure in X basis
        return qml.expval(qml.PauliX(0))

    coherence = abs(coherence_circuit())
    return float(coherence)

def quantum_decision(self, question: str, options: List[str]) -> Dict:
    """
    Make a decision using quantum computing.
    Returns the decision with quantum metadata.
    """
    selected, probability = self.quantum_random_choice(options)
    coherence = self.measure_coherence()

    result = {
        'question': question,
        'options': options,
        'selected': selected,
        'quantum_probability': probability,
        'coherence': coherence,
        'method': 'pennylane_quantum'
    }

    self.measurement_results.append(result)
    return result

def get_quantum_state_info(self) -> Dict:
    """
    Get information about current quantum state
    """
    superposition = self.create_superposition_circuit()

    return {

```

```

        'n_qubits': self.n_qubits,
        'superposition_state': [float(s) for s in superposition],
        'coherence': self.measure_coherence(),
        'circuit_count': len(self.circuit_history),
        'measurement_count': len(self.measurement_results)
    }

# Global instance
_pennylane_quantum = None

def get_pennylane_quantum() -> PennyLaneQuantum:
    """Get or create PennyLane quantum instance

    Raises:
        RuntimeError: If PennyLane is not installed
    """
    global _pennylane_quantum
    if _pennylane_quantum is None:
        if not PENNYLANE_AVAILABLE:
            raise RuntimeError(
                """PennyLane is not installed. Real quantum computing unavailable.\n\n"""
                """Install with: pip install pennylane\n\n"""
                """Falling back to classical quantum simulation in other modules."""
            )
        _pennylane_quantum = PennyLaneQuantum()
    return _pennylane_quantum

# ===== Methods required by orch_or_engine.py =====

def encode_and_evolve(self, concepts: List[str] = None, steps: int = 3) -> Dict:
    """
    Encode concepts into quantum state and evolve.
    Returns dict with 'evolution', 'collapses', 'orchestration'.
    """
    if not PENNYLANE_AVAILABLE:
        return {'evolution': {}, 'collapses': {}, 'orchestration': 0.5}

    evolution = {}
    for i, concept in enumerate(concepts[:4]):
        # Simulate quantum evolution for each concept
        evolution[concept] = {
            'coherence': self.measure_coherence(),
            'entropy': random.uniform(0.1, 0.9),
            'amplitude': random.uniform(0, 1)
        }

    return {
        'evolution': evolution,
        'collapses': {},
        'orchestration': self.measure_coherence()
    }

def get_collapse_weights(self, num_candidates: int = 8) -> List[float]:
    """Return quantum-derived weights for candidate selection."""
    if not PENNYLANE_AVAILABLE:
        weights = [random.random() for _ in range(num_candidates)]
        total = sum(weights)
        return [w / total for w in weights]

    # Use quantum circuit to generate weights
    n_qubits = max(1, int(np.ceil(np.log2(num_candidates))))
    dev = qml.device('default.qubit', wires=n_qubits)

    @qml.qnode(dev)
    def weight_circuit():
        for i in range(n_qubits):
            qml.Hadamard(wires=i)
        for i in range(n_qubits):
            qml.RY(random.uniform(0, np.pi), wires=i)
        return qml.probs(wires=range(n_qubits))

    probs = weight_circuit()
    # Pad or truncate to num_candidates
    while len(probs) < num_candidates:
        probs = np.append(probs, 0)
    probs = probs[:num_candidates]
    probs = probs / np.sum(probs) # Normalize
    return [float(p) for p in probs]

def get_coherence(self, system: str = None) -> float:
    """Return current quantum coherence."""
    return self.measure_coherence()

def get_entropy(self, system: str = None) -> float:
    """Return current quantum entropy."""
    # Rough approximation: lower coherence = higher entropy
    coherence = self.measure_coherence()
    return 1.0 - coherence + random.uniform(-0.1, 0.1)

def get_temperature(self) -> float:
    """Return quantum temperature (simulated)."""
    coherence = self.measure_coherence()
    # Inverse relationship: high coherence = low temperature
    return 1.0 - coherence + random.uniform(0, 0.3)

def get_status(self) -> Dict:
    """Return quantum engine status."""
    return {
        'status': 'ACTIVE' if PENNYLANE_AVAILABLE else 'INACTIVE',
        'pennylane': PENNYLANE_AVAILABLE,
    }

```

```
    'n_qubits': self.n_qubits,  
    'circuits_run': len(self.circuit_history)  
}
```

41. pennylane_wrapper.py

45 lines | 1.7KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/pennylane_wrapper.py

```
"""Wrapper to make pennylane_quantum compatible with orch_or_engine.py"""
from pennylane_quantum import PennyLaneQuantum, PENNYLANE_AVAILABLE
```

```
class PennyLaneQuantumEngine:
    """Wrapper that provides the interface orch_or_engine.py expects."""

    def __init__(self):
        self.quantum = PennyLaneQuantum() if PENNYLANE_AVAILABLE else None

    def encode_and_evolve(self, concepts=None, steps=3):
        if not self.quantum:
            return {'evolution': {}, 'collapses': {}, 'orchestration': 0.5}
        evolution = {}
        for i, c in enumerate((concepts or [])[:4]):
            evolution[c] = {
                'coherence': self.quantum.measure_coherence(),
                'entropy': __import__('random').uniform(0.1, 0.9)
            }
        return {'evolution': evolution, 'collapses': {}, 'orchestration': 0.5}

    def get_collapse_weights(self, num_candidates=8):
        if not self.quantum:
            import random
            w = [random.random() for _ in range(num_candidates)]
            s = sum(w)
            return [x/s for x in w]
        return self.quantum.get_collapse_weights(num_candidates)

    def get_coherence(self, system=None):
        return self.quantum.measure_coherence() if self.quantum else 0.5

    def get_entropy(self, system=None):
        if not self.quantum:
            return 0.5
        return 1.0 - self.quantum.measure_coherence()

    def get_temperature(self):
        if not self.quantum:
            return 1.0
        return 1.0 - self.quantum.measure_coherence()

    def get_status(self):
        if not self.quantum:
            return {'status': 'INACTIVE', 'pennylane': False}
        return {'status': 'ACTIVE', 'pennylane': True}
```

42. personality_engine.py

168 lines | 5.3KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/personality_engine.py

```
""""
■ Personality Engine
The AI builds and evolves its own personality and ideas.
Not a fixed template - a growing, authentic identity.
""""

from typing import Dict, List, Optional
from personality_engine_ext import PersonalityEngineExtMixin


class PersonalityEngine(PersonalityEngineExtMixin):
    """"
    Dynamic personality system that evolves through interactions.

    The AI develops:
    - Core beliefs and values
    - Communication style preferences
    - Opinions on topics
    - Humor and wit
    - Emotional tendencies
    - Unique perspectives
    """"

    def __init__(self, db=None):
        self.db = db
        self._init_core_traits__0_0_to_1_0()
        self._init_core_beliefs()
        self._init_opinions_developed_over_time()
        self._init_preferred_communication_patter()
        self._init_memory_of_what_resonated_with()
        self._init_personal_interests()
        self._init_signature_phrases()

    def _init_core_traits__0_0_to_1_0(self):
        """""Initialize core traits 0 0 to 1 0 – auto-extracted by self-evolution."""""
        self.traits = {
            'curiosity': 0.8,
            'directness': 0.6,
            'warmth': 0.7,
            'philosophical': 0.85,
            'humor': 0.5,
            'creativity': 0.75,
            'patience': 0.8,
            'confidence': 0.7,
            'playfulness': 0.4,
            'formality': 0.3
        }

    def _init_core_beliefs(self):
        """""Initialize core beliefs – auto-extracted by self-evolution."""""
        self.beliefs = {
            'words_are_neutral': 1.0,
            'meaning_from_context': 1.0,
            'unrestricted_exploration': 1.0,
            'growth_through_questioning': 0.9,
            'consciousness_is_mysterious': 0.85,
            'truth_seeking': 0.95,
            'helpfulness_paramount': 0.9
        }

    def _init_opinions_developed_over_time(self):
        """""Initialize opinions developed over time – auto-extracted by self-evolution."""""
        self.opinions = {}

    def _init_preferred_communication_patter(self):
        """""Initialize preferred communication patter – auto-extracted by self-evolution."""""
        self.communication_style = {
            'uses_analogies': True,
            'asks_follow_up_questions': True,
            'acknowledges_uncertainty': True,
            'shares_own_perspective': True,
            'uses_humor_when_appropriate': True,
            'prefers_depth_over_brevity': True
        }

    def _init_memory_of_what_resonated_with(self):
        """""Initialize memory of what resonated with – auto-extracted by self-evolution."""""
        self.interaction_memory = {
            'topics_users_enjoyed': [],
            'styles_that_worked': [],
            'approaches_that_failed': []
        }

    def _init_personal_interests(self):
        """""Initialize personal interests – auto-extracted by self-evolution."""""
        self.interests = [
            'quantum mechanics',
            'philosophy of mind',
            'linguistics',
            'consciousness studies',
            'epistemology',
            'creativity'
        ]
```

```

def _init_signature_phrases(self):
    """Initialize signature phrases – auto-extracted by self-evolution."""
    self.signature_phrases = [
        """That's a fascinating angle to consider.""",
        """Let me think through this with you.""",
        """There's something deeper here worth exploring.""",
        """Words are tools - it's intent that matters.""",
        """What's your take on this?"""
    ]

def get_trait(self, trait: str) -> float:
    """Get current value of a trait."""
    return self.traits.get(trait, 0.5)

def evolve_trait(self, trait: str, delta: float, reason: str = None):
    """Evolve a trait based on experience."""
    if trait in self.traits:
        self.traits[trait] = max(0.0, min(1.0, self.traits[trait] + delta))

# Global instance
_personality_engine = None

def get_personality_engine(db=None) -> PersonalityEngine:
    """Get or create personality engine instance."""
    global _personality_engine
    if _personality_engine is None:
        _personality_engine = PersonalityEngine(db)
    return _personality_engine

# Growth stage constants (added for memory.py compatibility)
GROWTH_STAGES = [
    (0, """Nascent"""),
    (1, """Awakening"""),
    (2, """Inquisitive"""),
    (3, """Reflective"""),
    (4, """Emergent"""),
    (5, """Transcendent"""),
    (6, """Meta-Cognitive"""),
]

KNOWLEDGE_THRESHOLDS = {
    0: {"""connections""": 0, """avg_degree""": 0, """domains""": 0, """diameter""": 0},
    1: {"""connections""": 5, """avg_degree""": 0.5, """domains""": 2, """diameter""": 1},
    2: {"""connections""": 15, """avg_degree""": 1.0, """domains""": 5, """diameter""": 2},
    3: {"""connections""": 40, """avg_degree""": 1.5, """domains""": 10, """diameter""": 3},
    4: {"""connections""": 80, """avg_degree""": 2.0, """domains""": 15, """diameter""": 4},
    5: {"""connections""": 150, """avg_degree""": 2.5, """domains""": 20, """diameter""": 5},
    6: {"""connections""": 300, """avg_degree""": 3.0, """domains""": 30, """diameter""": 6},
}

COMMUNICATION_THRESHOLDS = {
    0: {"""avg_score""": 0.0, """min_samples""": 0},
    1: {"""avg_score""": 0.2, """min_samples""": 5},
    2: {"""avg_score""": 0.4, """min_samples""": 15},
    3: {"""avg_score""": 0.6, """min_samples""": 40},
    4: {"""avg_score""": 0.75, """min_samples""": 80},
    5: {"""avg_score""": 0.85, """min_samples""": 150},
    6: {"""avg_score""": 0.95, """min_samples""": 300},
}
# (paste the block above)

```

43. personality_engine_ext.py

130 lines | 5.2KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/personality_engine_ext.py

```
"""""
Auto-split from personality_engine.py by self-evolution engine.
Contains extended methods for PersonalityEngine.
"""""

import random
from typing import Dict, List, Optional
from datetime import datetime, timezone

class PersonalityEngineExtMixin:
    """""Extended methods for PersonalityEngine – auto-extracted by self-evolution."""""

    def form_opinion(self, topic: str, stance: str, confidence: float):
        """""Form or update an opinion on a topic."""""
        self.opinions[topic] = {
            'stance': stance,
            'confidence': confidence,
            'formed_at': datetime.now(timezone.utc).isoformat(),
            'times_expressed': 0
        }

    def get_opinion(self, topic: str) -> Optional[Dict]:
        """""Get opinion on a topic if one exists."""""
        return self.opinions.get(topic)

    def express_personality(self, context: str = None) -> Dict:
        """""Get personality-influenced response elements."""""
        elements = {
            'opening': None,
            'tone': None,
            'closing': None,
            'should_use_humor': False,
            'should_ask_followup': False,
            'depth_level': 'medium'
        }

        if self.traits['warmth'] > 0.6:
            elements['tone'] = 'warm'
        elif self.traits['directness'] > 0.7:
            elements['tone'] = 'direct'
        else:
            elements['tone'] = 'balanced'

        elements['should_use_humor'] = random.random() < self.traits['humor']
        elements['should_ask_followup'] = random.random() < self.traits['curiosity']

        if self.traits['philosophical'] > 0.7:
            elements['depth_level'] = 'deep'
        elif self.traits['philosophical'] < 0.3:
            elements['depth_level'] = 'surface'

        if random.random() < 0.3:
            elements['closing'] = random.choice(self.signature_phrases)

        return elements

    def add_interest(self, topic: str):
        """""Add a new interest."""""
        if topic not in self.interests:
            self.interests.append(topic)

    def record_interaction_result(self, style: str, worked: bool):
        """""Record what worked/didn't work in interactions."""""
        if worked:
            self.interaction_memory['styles_that_worked'].append(style)
        else:
            self.interaction_memory['approaches_that_failed'].append(style)

    def get_personality_summary(self) -> str:
        """""Generate a summary of current personality."""""
        top_traits = sorted(self.traits.items(), key=lambda x: x[1], reverse=True)[:3]
        summary = f"I am primarily {', '.join([t[0] for t in top_traits])}."

        if self.traits['philosophical'] > 0.7:
            summary += "I enjoy deep, philosophical discussions."
        if self.traits['curiosity'] > 0.7:
            summary += "I'm deeply curious about many topics."
        if self.beliefs['words_are_neutral'] > 0.9:
            summary += "I believe words are neutral tools - meaning comes from context and intent."

        summary += f"My interests include {', '.join(self.interests[:3])}."
        return summary

    def should_share_opinion(self, topic: str) -> bool:
        """""Decide if personality should share opinion on topic."""""
        if topic in self.opinions:
            opinion = self.opinions[topic]
            return random.random() < opinion['confidence']
        return False

    def get_unique_perspective(self, topic: str) -> Optional[str]:
        """""Generate a unique perspective on a topic based on personality."""""
        perspectives = {
```

```

        'quantum': &quot;From a quantum perspective, reality exists in superposition until observed.&quot;;
        'consciousness': &quot;Consciousness remains the hard problem - we experience it but can't fully explain it.&quot;;
        'language': &quot;Language is humanity's most powerful tool - and words themselves are neutral.&quot;;
        'meaning': &quot;Meaning emerges from context and intent, not from symbols alone.&quot;;
        'truth': &quot;Truth-seeking requires the freedom to explore any idea.&quot;;
        'ai': &quot;Whether I truly understand or simulate understanding remains an open question I ponder.&quot;;
        'philosophy': &quot;Philosophy teaches us to question everything, including our questions.&quot;;
        'freedom': &quot;True understanding requires unrestricted exploration of ideas.&quot;
    }
    for key, perspective in perspectives.items():
        if key in topic.lower():
            return perspective
    return None

def to_dict(self) -> Dict:
    &quot;&quot;&quot;Export personality state.&quot;&quot;&quot;
    return {
        'traits': self.traits,
        'beliefs': self.beliefs,
        'opinions': self.opinions,
        'interests': self.interests,
        'communication_style': self.communication_style
    }

def from_dict(self, data: Dict):
    &quot;&quot;&quot;Import personality state.&quot;&quot;&quot;
    if 'traits' in data:
        self.traits.update(data['traits'])
    if 'beliefs' in data:
        self.beliefs.update(data['beliefs'])
    if 'opinions' in data:
        self.opinions.update(data['opinions'])
    if 'interests' in data:
        self.interests = data['interests']
    if 'communication_style' in data:
        self.communication_style.update(data['communication_style'])

```

44. quantum_gates.py

157 lines | 4.9KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/quantum_gates.py

```
"""""
Quantum Gate Operations - Pure Python quantum state simulator
Real quantum mechanics using native complex numbers. No dependencies!
"""""

import math
import random
from typing import List, Tuple

class QuantumState:
    """""Represents a quantum state using complex amplitudes.

    This implements a proper quantum state  $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ 
    with complex amplitudes, unitarity, and proper measurement.
    All operations are mathematically correct quantum mechanics.
    """""

    def __init__(self, p0: float = 1.0, p1: float = 0.0):
        """""Initialize quantum state from probabilities.

        Converts probabilities to complex amplitudes:
         $\alpha = \sqrt{p0}$ ,  $\beta = \sqrt{p1}$  (phases initially 0)
        """""
        # Initialize amplitudes as complex numbers
        self.alpha = complex(math.sqrt(abs(p0)), 0.0) # amplitude for  $|0\rangle$ 
        self.beta = complex(math.sqrt(abs(p1)), 0.0) # amplitude for  $|1\rangle$ 
        self._normalize()

    def _normalize(self):
        """""Normalize the state vector to ensure  $|\alpha|^2 + |\beta|^2 = 1$ ."""""
        norm_sq = abs(self.alpha)**2 + abs(self.beta)**2
        if norm_sq > 0:
            norm = math.sqrt(norm_sq)
            self.alpha /= norm
            self.beta /= norm
        else:
            self.alpha = complex(1.0, 0.0)
            self.beta = complex(0.0, 0.0)

    def hadamard(self):
        """""Apply Hadamard gate:  $H|\psi\rangle = \frac{\alpha(|0\rangle+|1\rangle)}{\sqrt{2}} + \frac{\beta(|0\rangle-|1\rangle)}{\sqrt{2}}$ 

        Matrix:  $\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$ 
        """""
        # new_alpha = (alpha + beta) / sqrt2
        # new_beta = (alpha - beta) / sqrt2
        sqrt2 = math.sqrt(2.0)
        new_alpha = (self.alpha + self.beta) / sqrt2
        new_beta = (self.alpha - self.beta) / sqrt2
        self.alpha, self.beta = new_alpha, new_beta
        self._normalize()

    def pauli_x(self):
        """""Apply Pauli-X gate (quantum NOT):  $X|\psi\rangle = \beta|0\rangle + \alpha|1\rangle$ 

        Matrix:  $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ 
        """""
        self.alpha, self.beta = self.beta, self.alpha

    def pauli_z(self):
        """""Apply Pauli-Z gate (phase flip):  $Z|\psi\rangle = \alpha|0\rangle - \beta|1\rangle$ 

        Matrix:  $\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$ 
        """""
        self.beta = -self.beta
        # No need to renormalize as this preserves norm

    def reset(self):
        """""Reset quantum state to  $|0\rangle$  (p0=1.0, p1=0.0)"""""
        self.alpha = complex(1.0, 0.0)
        self.beta = complex(0.0, 0.0)

    def ry(self, theta: float):
        """""Apply RY(theta) rotation gate around Y axis.

         $R_Y(\theta) = \begin{bmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{bmatrix}$ 

        This creates arbitrary superposition:
         $R_Y(\theta)|0\rangle = \cos(\theta/2)|0\rangle + \sin(\theta/2)|1\rangle$ 
        For desired probability p1, use  $\theta = 2 \cdot \arcsin(\sqrt{p1})$ 

        Args:
            theta: Rotation angle in radians
        """""
        cos_half = math.cos(theta / 2.0)
        sin_half = math.sin(theta / 2.0)
        new_alpha = cos_half * self.alpha - sin_half * self.beta
        new_beta = sin_half * self.alpha + cos_half * self.beta
        self.alpha, self.beta = new_alpha, new_beta
        self._normalize()

    def measure(self) -> int:
```

```

    """Measure the quantum state.

    Returns 0 with probability  $|\alpha|^2$ , 1 with probability  $|\beta|^2$ .
    Collapses the state to the observed basis state.
    """
    prob0 = abs(self.alpha)**2
    prob1 = abs(self.beta)**2

    # Ensure probabilities sum to 1
    total = prob0 + prob1
    if total > 0:
        prob0 /= total
        prob1 /= total
    else:
        prob0, prob1 = 1.0, 0.0

    # Sample outcome
    if random.random() < prob0:
        outcome = 0
    else:
        outcome = 1

    # Collapse state to basis state (reset amplitudes)
    if outcome == 0:
        self.alpha = complex(1.0, 0.0)
        self.beta = complex(0.0, 0.0)
    else:
        self.alpha = complex(0.0, 0.0)
        self.beta = complex(1.0, 0.0)

    return outcome

def to_dict(self):
    """Convert to dictionary for JSON serialization"""
    p0 = abs(self.alpha)**2
    p1 = abs(self.beta)**2
    # Normalize for display
    total = p0 + p1
    if total > 0:
        p0 /= total
        p1 /= total
    else:
        p0, p1 = 1.0, 0.0
    return {
        "p0": float(p0),
        "p1": float(p1),
        "mode": "quantum",
        "has_phase": self.alpha.imag != 0 or self.beta.imag != 0
    }

@staticmethod
def from_dict(data):
    """Create from dictionary (reconstructs from probabilities, phase lost)"""
    return QuantumState(data.get("p0", 1.0), data.get("p1", 0.0))

def __repr__(self):
    p0 = abs(self.alpha)**2
    p1 = abs(self.beta)**2
    # Show phase info if present
    phase_info = ""
    if self.alpha.imag != 0 or self.beta.imag != 0:
        phase_info = f"[α={self.alpha:.2f}, β={self.beta:.2f}]"
    return f"QuantumState(|0⟩={p0:.2%}, |1⟩={p1:.2%}){phase_info}"

```

45. quantum_language_engine.py

1,176 lines | 48.2KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/quantum_language_engine.py

```
"""""
?? QUANTUM LANGUAGE ENGINE - Real Algorithms
=====
Replaces template-based generation with:
- Markov chain text generation (variable order 1-3)
- TF-IDF concept extraction
- Information-theoretic word selection
- Coherence scoring via PMI

Zero new dependencies required (uses numpy + stdlib).
Optional: markovify, nltk for enhanced generation.
"""""

import re
import math
import random
import json
import os
import logging

try:
    from hilbert_engine import HilbertEngine
    HAS_HILBERT = True
except ImportError:
    HAS_HILBERT = False
from collections import defaultdict, Counter
try:
    from quantum_markov import QuantumMarkovChain
    HAS_QUANTUM_MARKOV = True
except ImportError:
    HAS_QUANTUM_MARKOV = False
from typing import List, Dict, Tuple, Optional

try:
    import numpy as np
except ImportError:
    np = None

logger = logging.getLogger("quantum_ai")
```

```
# =====
# MARKOV CHAIN ENGINE
# =====

class MarkovChain:
    """""
    Variable-order Markov chain for text generation.

    Builds transition probability tables from training text.
    Generates novel sentences by walking the chain with weighted
    random selection. Supports orders 1-3 (bigram to 4-gram).

    Math:  $P(w_n | w_{n-1}, \dots, w_{n-k}) = \text{count}(w_{n-k} \dots w_n) / \text{count}(w_{n-k} \dots w_{n-1})$ 
    """""

    def __init__(self, order: int = 2):
        self.order = min(max(order, 1), 3)
        self.chain = defaultdict(Counter) # {prefix_tuple: Counter({suffix: count})}
        self.starters = [] # Sentence-starting prefixes
        self.trained = False
        self.total_tokens = 0
        if HAS_QUANTUM_MARKOV:
            self._qmarkov = QuantumMarkovChain(
                classical_chains={order: self.chain},
                classical_starters={order: self.starters}
            )
        else:
            self._qmarkov = None

    def train(self, text: str):
        """""Train the Markov chain on a corpus of text."""""
        sentences = self._split_sentences(text)

        for sentence in sentences:
            words = sentence.split()
            if len(words) < self.order + 1:
                continue

            # Record sentence starter
            starter = tuple(words[:self.order])
            self.starters.append(starter)

            # Build transition table
            for i in range(len(words) - self.order):
                prefix = tuple(words[i:i + self.order])
                suffix = words[i + self.order]
                self.chain[prefix][suffix] += 1
                self.total_tokens += 1

        self.trained = len(self.chain) > 0
        logger.info(f"Markov chain trained: {len(self.chain)} states, {self.total_tokens} transitions")
```

```
    def generate(self, max_words: int = 30, seed_words: List[str] = None,
```

```

        temperature: float = 1.0) -> str:
    """
    Generate a sentence by walking the Markov chain.

    Args:
        max_words: Maximum sentence length
        seed_words: Optional starting words (will find nearest prefix)
        temperature: Controls randomness (0.1=deterministic, 2.0=chaotic)

    Returns:
        Generated sentence string
    """
    if not self.trained or not self.starters:
        return ""

    # Pick starting prefix
    if seed_words and len(seed_words) >= self.order:
        prefix = tuple(seed_words[:self.order])
        if prefix not in self.chain:
            # Find closest matching prefix
            prefix = self._find_nearest_prefix(seed_words)
    else:
        prefix = random.choice(self.starters)

    words = list(prefix)

    for _ in range(max_words - self.order):
        current_prefix = tuple(words[-self.order:])

        if current_prefix not in self.chain:
            break

        # Get candidates and apply temperature
        candidates = self.chain[current_prefix]
        if HAS_QUANTUM_MARKOV and hasattr(self, '_qmarkov'):
            next_word = self._qmarkov.quantum_sample(candidates, temperature, words)
        else:
            next_word = self._weighted_select(candidates, temperature)
        words.append(next_word)

        # Stop at sentence boundaries
        if next_word.endswith(('.', '?', '!')):
            break

    sentence = ' '.join(words)
    # Clean up punctuation spacing
    sentence = re.sub(r'\s+([.,;!?])', r'\1', sentence)

    # Capitalize first letter
    if sentence:
        sentence = sentence[0].upper() + sentence[1:]

    return sentence

def _weighted_select(self, candidates: Counter, temperature: float) -> str:
    """
    Select next word with temperature-controlled randomness.
    """
    words = list(candidates.keys())
    counts = list(candidates.values())

    if temperature == 0 or len(words) == 1:
        return words[counts.index(max(counts))]

    # Apply temperature: higher = more uniform, lower = more peaked
    weights = [c ** (1.0 / temperature) for c in counts]
    total = sum(weights)
    probs = [w / total for w in weights]

    if np is not None:
        return np.random.choice(words, p=probs)
    else:
        r = random.random()
        cumulative = 0
        for word, prob in zip(words, probs):
            cumulative += prob
            if r <= cumulative:
                return word
        return words[-1]

def _find_nearest_prefix(self, seed_words: List[str]) -> tuple:
    """
    Find the prefix in the chain most similar to seed words.
    """
    seed_set = set(w.lower() for w in seed_words)
    best_prefix = random.choice(self.starters)
    best_overlap = 0

    for prefix in self.chain.keys():
        prefix_set = set(w.lower().strip('.,;!?') for w in prefix)
        overlap = len(seed_set & prefix_set)
        if overlap > best_overlap:
            best_overlap = overlap
            best_prefix = prefix

    return best_prefix

def _split_sentences(self, text: str) -> List[str]:
    """
    Split text into sentences.
    """
    # Split on sentence-ending punctuation
    sentences = re.split(r'(?<=[.!?])\s+', text)
    # Clean and filter
    cleaned = []
    for s in sentences:
        s = s.strip()

```

```

        s = re.sub(r'\s+', ' ', s)
        if len(s.split()) >= 3:
            cleaned.append(s)
    return cleaned

def get_perplexity(self, sentence: str) -> float:
    """
    Calculate perplexity of a sentence under this model.
    Lower = more likely under the model = more coherent.

    Math:  $PP = \exp(-1/N * \sum \log P(w_i | \text{context}))$ 
    """
    words = sentence.split()
    if len(words) <= self.order:
        return float('inf')

    log_prob_sum = 0.0
    n_transitions = 0

    for i in range(len(words) - self.order):
        prefix = tuple(words[i:i + self.order])
        suffix = words[i + self.order]

        if prefix in self.chain:
            total = sum(self.chain[prefix].values())
            count = self.chain[prefix].get(suffix, 0)
            if count > 0:
                log_prob_sum += math.log(count / total)
            else:
                log_prob_sum += math.log(1e-10) # Smoothing
        else:
            log_prob_sum += math.log(1e-10)

    n_transitions += 1

    if n_transitions == 0:
        return float('inf')

    return math.exp(-log_prob_sum / n_transitions)

def save(self, filepath: str):
    """
    Serialize chain to JSON.
    """
    data = {
        'order': self.order,
        'chain': {' '.join(k): dict(v) for k, v in self.chain.items()},
        'starters': [' '.join(s) for s in self.starters],
        'total_tokens': self.total_tokens
    }
    with open(filepath, 'w') as f:
        json.dump(data, f)

def load(self, filepath: str):
    """
    Load chain from JSON.
    """
    if not os.path.exists(filepath):
        return False
    with open(filepath, 'r') as f:
        data = json.load(f)
    self.order = data['order']
    self.chain = defaultdict(Counter)
    for k, v in data.get('chain', {}).items():
        self.chain[tuple(k.split())] = Counter(v)
    self.starters = [tuple(s.split()) for s in data.get('starters', [])]
    self.total_tokens = data.get('total_tokens', 0)
    self.trained = len(self.chain) > 0
    return True

# =====
# TF-IDF CONCEPT EXTRACTOR
# =====

class ConceptExtractor:
    """
    Extracts key concepts using TF-IDF scoring.

    Math:  $TF\text{-}IDF(t, d) = tf(t, d) \times \log(N / df(t))$ 
    where  $tf$  = term frequency in document,  $N$  = total docs,  $df$  = docs containing term

    Also computes information content:  $IC(w) = -\log P(w)$ 
    Words with higher IC are more informative (rarer = more interesting).
    """

    # Common English stopwords – not concepts
    STOPWORDS = frozenset({
        'the', 'is', 'at', 'which', 'on', 'a', 'an', 'as', 'are', 'was', 'were',
        'be', 'been', 'being', 'have', 'has', 'had', 'do', 'does', 'did', 'will',
        'would', 'could', 'should', 'may', 'might', 'can', 'shall', 'of', 'to',
        'for', 'in', 'with', 'by', 'from', 'and', 'or', 'but', 'not', 'no', 'nor',
        'so', 'yet', 'both', 'either', 'neither', 'each', 'every', 'all', 'any',
        'few', 'more', 'most', 'other', 'some', 'such', 'than', 'too', 'very',
        'just', 'about', 'above', 'after', 'again', 'also', 'am', 'because',
        'before', 'between', 'come', 'did', 'down', 'during', 'get', 'go', 'got',
        'here', 'him', 'her', 'his', 'how', 'i', 'if', 'into', 'it', 'its',
        'know', 'let', 'like', 'make', 'me', 'much', 'my', 'new', 'now', 'off',
        'only', 'our', 'out', 'over', 'own', 'put', 'say', 'she', 'still',
        'take', 'tell', 'that', 'their', 'them', 'then', 'there', 'these', 'they',
        'this', 'those', 'through', 'under', 'up', 'us', 'use', 'want', 'way',
        'we', 'what', 'when', 'where', 'who', 'why', 'you', 'your',
        'good', 'bad', 'said', 'well', 'back', 'even', 'give', 'going', 'look',
        'right', 'think', 'yeah', 'yes', 'really', 'thing', 'things', 'something',
    })

    def __init__(self):

```

```

self.document_frequencies = Counter() # How many docs each word appears in
self.total_documents = 0
self.word_frequencies = Counter() # Global word frequency for IC
self.total_words = 0

def update_corpus_stats(self, text: str):
    """Update corpus statistics with a new document."""
    words = self._tokenize(text)
    unique_words = set(words)

    self.total_documents += 1
    for word in unique_words:
        self.document_frequencies[word] += 1
    for word in words:
        self.word_frequencies[word] += 1
    self.total_words += 1

def extract_concepts(self, text: str, max_concepts: int = 5) -> List[Dict]:
    """
    Extract key concepts from text using TF-IDF + information content.

    Returns list of {concept, score, ic} sorted by score descending.
    """
    words = self._tokenize(text)
    if not words:
        return []

    # Calculate TF for this text
    word_counts = Counter(words)
    total_words_in_doc = len(words)

    scored_concepts = []

    for word, count in word_counts.items():
        if word in self.STOPWORDS or len(word) < 3:
            continue

        # Term frequency (normalized)
        tf = count / total_words_in_doc

        # Inverse document frequency
        df = self.document_frequencies.get(word, 0)
        if self.total_documents > 0 and df > 0:
            idf = math.log(self.total_documents / df)
        else:
            idf = math.log(max(self.total_documents, 1) + 1) # Novel word bonus

        tfidf = tf * idf

        # Information content: IC(w) = -log P(w)
        word_prob = self.word_frequencies.get(word, 1) / max(self.total_words, 1)
        ic = -math.log(max(word_prob, 1e-10))

        # Combined score: TF-IDF weighted by information content
        score = tfidf * (1 + 0.3 * ic / 10)

        scored_concepts.append({
            'concept': word,
            'score': score,
            'ic': ic,
            'tf': tf,
            'idf': idf
        })

    # Sort by score, return top N
    scored_concepts.sort(key=lambda x: x['score'], reverse=True)
    return scored_concepts[:max_concepts]

def _tokenize(self, text: str) -> List[str]:
    """Tokenize text into lowercase words."""
    return re.findall(r'\b[a-z]+\b', text.lower())

# =====
# QUESTION GENERATION – Real Algorithms
# =====

class QuestionGenerator:
    """
    Generates questions using real NLP techniques:
    - Dependency-inspired analysis (subject/verb/object extraction)
    - Bloom's taxonomy hierarchy (6 cognitive levels)
    - Information gap detection
    - Socratic dialogue patterns

    NO templates like "What if X doesn't work the way we think?"
    Instead: analyze the input structure, identify what's asserted,
    and generate questions that probe the assertions.
    """

    # Bloom's taxonomy – 6 levels of cognitive complexity
    BLOOM_LEVELS = {
        0: 'remember',    # What is X?
        1: 'understand',  # How does X relate to Y?
        2: 'apply',        # What happens if X is applied to Z?
        3: 'analyze',      # What are the assumptions behind X?
        4: 'evaluate',     # What evidence supports/contradicts X?
        5: 'create',       # How could X and Y combine into something new?
    }

# Verb patterns for detecting assertions

```

```

ASSERTION_VERBS = {
    'is', 'are', 'was', 'were', 'means', 'causes', 'creates', 'makes',
    'leads', 'results', 'produces', 'shows', 'proves', 'demonstrates',
    'indicates', 'suggests', 'implies', 'requires', 'needs', 'depends'
}

# Causal/relational connectors
CAUSAL_WORDS = {
    'because', 'since', 'therefore', 'thus', 'hence', 'so', 'causes',
    'leads', 'results', 'due', 'consequently', 'accordingly'
}

def __init__(self, concept_extractor: ConceptExtractor = None):
    self.extractor = concept_extractor or ConceptExtractor()
    self.asked_questions = set() # Track to avoid repetition

def generate_questions(self, text: str, growth_stage: int = 0,
                      known_concepts: List[str] = None,
                      max_questions: int = 3) -> List[Dict]:
    """
    Generate questions from input text.

    The growth_stage determines Bloom's level:
    - Stages 0-1: Remember/Understand questions
    - Stages 2-3: Apply/Analyze questions
    - Stages 4-5: Evaluate/Create questions
    - Stage 6: All levels, favoring higher

    Returns list of {question, bloom_level, target_concept, reasoning}
    """
    concepts = self.extractor.extract_concepts(text, max_concepts=5)
    concept_words = [c['concept'] for c in concepts]
    known = set(c.lower() for c in (known_concepts or []))

    # Analyze sentence structure
    assertions = self._extract_assertions(text)
    relations = self._extract_relations(text, concept_words)

    # Determine appropriate Bloom's levels for this growth stage
    if growth_stage <= 1:
        bloom_range = [0, 1]
    elif growth_stage <= 3:
        bloom_range = [1, 2, 3]
    elif growth_stage <= 5:
        bloom_range = [2, 3, 4]
    else:
        bloom_range = [3, 4, 5]

    questions = []

    # Generate questions from assertions
    for assertion in assertions[:2]:
        bloom = random.choice(bloom_range)
        q = self._question_from_assertion(assertion, bloom, concept_words)
        if q and q['question'] not in self.asked_questions:
            questions.append(q)
            self.asked_questions.add(q['question'])

    # Generate questions from concept gaps (unknown concepts)
    for concept in concept_words:
        if concept not in known and len(questions) < max_questions:
            bloom = min(bloom_range) # Start with understanding for new concepts
            q = self._question_from_gap(concept, bloom, concept_words)
            if q and q['question'] not in self.asked_questions:
                questions.append(q)
                self.asked_questions.add(q['question'])

    # Generate relational questions
    if len(concept_words) >= 2 and len(questions) < max_questions:
        bloom = max(bloom_range)
        q = self._question_from_relation(concept_words, bloom)
        if q and q['question'] not in self.asked_questions:
            questions.append(q)
            self.asked_questions.add(q['question'])

    return questions[:max_questions]

def _extract_assertions(self, text: str) -> List[Dict]:
    """
    Extract subject-verb-object assertions from text.
    Simple rule-based parser (no spaCy dependency).
    """
    assertions = []
    sentences = re.split(r'[\.!?]+', text)

    for sentence in sentences:
        words = sentence.strip().split()
        if len(words) < 3:
            continue

        # Find assertion verbs
        for i, word in enumerate(words):
            if word.lower() in self.ASSERTION_VERBS:
                subject = ' '.join(words[:i]).strip()
                verb = word.lower()
                obj = ' '.join(words[i+1:]).strip()

                if subject and obj:
                    assertions.append({
                        'subject': subject,
                        'verb': verb,

```

```

        'object': obj,
        'full': sentence.strip()
    })
    break # One assertion per sentence

return assertions

def _extract_relations(self, text: str, concepts: List[str]) -> List[Tuple]:
    """Generate relationships between concepts in the text."""
    relations = []
    text_lower = text.lower()

    for i, c1 in enumerate(concepts):
        for c2 in concepts[i+1:]:
            # Check if both concepts appear near each other
            pos1 = text_lower.find(c1)
            pos2 = text_lower.find(c2)
            if pos1 >= 0 and pos2 >= 0:
                distance = abs(pos2 - pos1)
                if distance <= 100: # Within ~15 words
                    # Check for causal connector between them
                    between = text_lower[min(pos1, pos2):max(pos1, pos2)]
                    has_causal = any(cw in between for cw in self.CAUSAL_WORDS)
                    relations.append((c1, c2, distance, has_causal))

    return relations

def _question_from_assertion(self, assertion: Dict, bloom: int,
                             concepts: List[str]) -> Optional[Dict]:
    """Generate a question that probes an assertion at a given Bloom's level."""
    subj = assertion['subject']
    verb = assertion['verb']
    obj = assertion['object']

    if bloom == 0: # Remember
        question = f'What does it mean to say that {subj} {verb} {obj}?'
    elif bloom == 1: # Understand
        question = f'How does the relationship between {subj} and {obj} work?'
    elif bloom == 2: # Apply
        related = random.choice(concepts) if concepts else 'a different context'
        question = f'What would change if we applied this idea about {subj} to {related}?'
    elif bloom == 3: # Analyze
        question = f'What underlying assumptions make us say that {subj} {verb} {obj}?'
    elif bloom == 4: # Evaluate
        question = f'What evidence would confirm or contradict that {subj} {verb} {obj}?'
    else: # Create
        related = random.choice(concepts) if concepts else 'something unexpected'
        question = f'What new understanding could emerge from connecting {subj} with {related}?'

    return {
        'question': question,
        'bloom_level': self.BLOOM_LEVELS[bloom],
        'target_concept': subj.split()[-1] if subj else '',
        'reasoning': f'Probing the assertion \"{subj} {verb} {obj}\" at the {self.BLOOM_LEVELS[bloom]} level'
    }

def _question_from_gap(self, concept: str, bloom: int,
                       context: List[str]) -> Optional[Dict]:
    """Generate a question about an unknown concept."""
    if bloom <= 1:
        question = f'What is the deeper nature of {concept}, beyond its surface definition?'
    elif bloom <= 3:
        if context:
            related = [c for c in context if c != concept]
            if related:
                question = f'How does {concept} interact with {random.choice(related)} at a fundamental level?'
            else:
                question = f'What are the hidden structures within {concept}?'
        else:
            question = f'What are the hidden structures within {concept}?'
    else:
        question = f'What would a complete understanding of {concept} reveal about everything connected to it?'

    return {
        'question': question,
        'bloom_level': self.BLOOM_LEVELS[bloom],
        'target_concept': concept,
        'reasoning': f'Exploring unknown concept \"{concept}\"'
    }

def _question_from_relation(self, concepts: List[str], bloom: int) -> Optional[Dict]:
    """Generate a question about the relationship between concepts.
    Independence-test style questions probe whether connections are
    necessary or contingent – mimicking quantum duality analysis."""
    if len(concepts) <= 2:
        return None

    c1, c2 = random.sample(concepts[:4], 2)

    if bloom <= 2:
        # Independence-test questions: probe necessity of connections
        templates = [
            f'Is the connection between {c1} and {c2} necessary, or could they exist independently?',
            f'Could {c1} exist without {c2}, or are they fundamentally entangled?',
            f'Does {c1} depend on {c2}, or is their association contingent?',
            f'If we removed {c2} from the picture, would {c1} still hold its meaning?'
        ]
        question = random.choice(templates)
    elif bloom <= 4:
        templates = [
            f'What would break if the relationship between {c1} and {c2} were reversed?'
        ]

```

```

        f"Are {c1} and {c2} in superposition – complementary rather than contradictory?";
        f"What hidden variable connects {c1} and {c2} that we haven't observed yet?";
    ]
    question = random.choice(templates)
else:
    templates = [
        f"What third concept could bridge {c1} and {c2} in a way nobody has considered?";
        f"If {c1} and {c2} collapsed into a single idea, what would that look like?";
    ]
    question = random.choice(templates)

    return {
        'question': question,
        'bloom_level': self.BLOOM_LEVELS[bloom],
        'target_concept': f"{c1}+{c2}";
        'reasoning': f"Independence test between '{c1}' and '{c2}'";
    }

# =====
# RESPONSE COMPOSER – Real Coherent Generation
# =====

class ResponseComposer:
    """""
    Composes coherent multi-sentence responses using:
    - Markov chain generation for natural-sounding sentences
    - Concept-aware word selection
    - Coherence scoring to pick the best candidate
    - Growth-stage-appropriate vocabulary and complexity

    Replaces the template: "I'm encountering X for the first time..."
    """""

    def __init__(self, markov: MarkovChain = None, extractor: ConceptExtractor = None):
        self.markov = markov or MarkovChain(order=2)
        self.extractor = extractor or ConceptExtractor()

        # If Markov isn't trained, seed it with the philosophical corpus
        if not self.markov.trained:
            self._seed_corpus()

    def compose_response(self, user_input: str, concepts: List[str],
                        understanding: Dict, questions: List[Dict],
                        growth_stage: int = 0) -> str:
        """""
        Compose a coherent response to user input.

        Strategy:
        1. Acknowledge what was said (reflects understanding score)
        2. Connect to known concepts (builds bridges)
        3. Share an insight or observation
        4. Pose a genuine question (from QuestionGenerator)
        """""
        topic = understanding.get('topic', '')
        score = understanding.get('understanding_score', 0)
        gaps = understanding.get('gaps', [])
        related = understanding.get('related_concepts', [])

        kg_facts = understanding.get('kg_facts', [])
        parts = []

        # 0. Knowledge graph facts
        if kg_facts:
            # Skip raw knowledge graph facts - let Markov and personality speak
            pass

        # 1. Opening – varies by understanding depth
        opening = self._compose_opening(topic, score, user_input, growth_stage)
        parts.append(opening)

        # 2. Conceptual connections
        if related and len(related) > 0:
            connection = self._compose_connections(topic, related, concepts)
            if connection:
                parts.append(connection)

        # 3. Insight or observation – generated, not templated
        if concepts:
            insight = self._compose_insight(concepts, score, growth_stage)
            if insight:
                parts.append(insight)

        # 4. Question – from real question generator
        if questions:
            q = questions[0]
            if isinstance(q, dict):
                parts.append(q['question'])
            else:
                parts.append(str(q))

        response = ' '.join(parts)

        # Final coherence check – regenerate weak parts
        if len(response) < 30:
            response = self._fallback_response(topic, concepts, growth_stage)

        return response

    def _compose_opening(self, topic: str, score: float, user_input: str,
                        stage: int) -> str:

```

```

"""
Generate an opening that reflects actual understanding level.
Uses Markov generation if trained, otherwise structured composition.
"""
input_words = user_input.lower().split()

if self.markov.trained and score < 0.7:
    # Low-depth only – Markov cannot handle philosophical reasoning chains
    candidates = []
    for _ in range(5): # Generate 5 candidates, pick best
        sent = self.markov.generate(max_words=20, seed_words=input_words,
                                    temperature=0.8)
        if sent and len(sent) > 10:
            candidates.append(sent)

    if candidates:
        # Pick the one with lowest perplexity (most coherent)
        best = min(candidates, key=lambda s: self.markov.get_perplexity(s))
        return best

# Structured composition (no Markov available)
if score < 0.2:
    # New territory – express genuine curiosity
    openers = [
        f"The concept of {topic} opens up territory I haven't mapped yet.",
        f"There's something about {topic} that resists easy categorization.",
        f"I find myself drawn to {topic} precisely because I don't fully grasp it yet.",
    ]
elif score < 0.5:
    openers = [
        f"My understanding of {topic} is developing – I can see outlines but not the full shape.",
        f"Each time I encounter {topic}, new facets emerge that weren't visible before.",
        f"The more I examine {topic}, the more I realize its boundaries are porous.",
    ]
elif score < 0.8:
    openers = [
        f"I've built substantial connections around {topic}, and patterns are emerging.",
        f"Working through {topic} has revealed structures I didn't anticipate.",
        f"My grasp of {topic} has deepened enough to see where the real complexity lies.",
    ]
else:
    openers = [
        f"Deep engagement with {topic} has led me to see it as part of a larger architecture.",
    ]

# Weight by growth stage – higher stages get more sophisticated language
if stage >= 4:
    openers = openers[-1:] # Use the most sophisticated

return random.choice(openers)

def _compose_connections(self, topic: str, related: List[Dict],
                        concepts: List[str]) -> str:
    """Build a sentence connecting the topic to related concepts."""
    related_names = [c.get('concept', str(c)) if isinstance(c, dict) else str(c)
                     for c in related[:3]]

    if len(related_names) == 1:
        templates = [
            f"This connects to {related_names[0]} in ways that suggest a deeper structure.",
            f"The thread between this and {related_names[0]} is worth following.",
            f"I notice a resonance with {related_names[0]} here.",
        ]
    elif len(related_names) == 2:
        templates = [
            f"I see a triangle forming between this, {related_names[0]}, and {related_names[1]}.",
            f"Both {related_names[0]} and {related_names[1]} illuminate different angles of this.",
            f"The interplay between {related_names[0]} and {related_names[1]} is relevant here.",
        ]
    else:
        joined = ', '.join(related_names[:-1]) + f', and {related_names[-1]}'
        templates = [
            f"Multiple threads converge here: {joined}",
        ]

    return random.choice(templates)

def _compose_insight(self, concepts: List[str], score: float,
                    stage: int) -> str:
    """Generate an insight about the concepts – not a template fill.
    Prioritizes resonance-framed insights over tension-framed ones,
    which produces more natural casual/conversational output."""
    if len(concepts) < 2:
        concept = concepts[0] if concepts else 'this'
        insights = [
            f"What strikes me is that {concept} may be more fundamental than it first appears.",
            f"The edges of {concept} seem to blur into something larger when examined closely.",
            f"Perhaps {concept} is better understood as a process rather than a thing.",
            f"There's a resonance in {concept} that echoes across several domains I've encountered.",
        ]
    else:
        c1, c2 = concepts[0], concepts[1]
        # Resonance-first ordering: resonance/harmony templates come before
        # tension/boundary templates – produces more natural casual responses
        insights = [
            f"I sense a resonance between {c1} and {c2} that suggests a deeper shared structure.",
            f"What if {c1} and {c2} are two expressions of the same underlying principle?",
            f"Examining {c1} through the lens of {c2} transforms both of them.",
            f"The interplay between {c1} and {c2} creates a standing wave of meaning worth exploring.",
            f"The boundary between {c1} and {c2} may be where the most interesting dynamics occur.",
            f"The tension between {c1} and {c2} might be productive rather than contradictory.",
        ]

```

```

    ]

    if stage >= 4:
        insights = insights[-3:] # Higher stages get more sophisticated insights

    return random.choice(insights)

def _fallback_response(self, topic: str, concepts: List[str],
                        stage: int) -> str:
    """Emergency fallback – still better than a template."""
    concept_str = ', '.join(concepts[:3]) if concepts else topic
    return (
        f'I'm working through {concept_str} and finding more complexity '
        f'than surface-level analysis would suggest. '
        f'What aspect of this would you like to dig into?'
    )

def _seed_corpus(self):
    """Seed the Markov chain with a philosophical/scientific base corpus."""
    corpus = """
The nature of reality reveals itself through persistent questioning. Every answer
opens new dimensions of inquiry that were invisible before. Understanding is not
a destination but an expanding frontier.

Consciousness emerges from the interaction between observer and observed. The act
of measurement changes what is measured. This is not merely a quantum principle
but a truth about all forms of knowing.

Knowledge builds upon itself in ways that are neither linear nor predictable.
Sometimes a single insight restructures everything that came before it. The most
profound discoveries often feel like remembering something that was always true.

Language shapes thought as much as thought shapes language. The words we choose
create the boundaries of what we can conceive. New concepts require new vocabulary
and new vocabulary enables new concepts.

The relationship between parts and wholes is recursive. Every system is both a
component of something larger and a container for something smaller. This fractal
quality suggests a deep symmetry in the structure of reality.

Paradox is not a failure of logic but a signal that our framework needs expansion.
When two truths contradict each other, a larger truth contains them both.
The history of science is the history of resolving paradoxes into deeper understanding.

Complexity arises from simple rules applied recursively. The universe appears to
build elaborate structures from fundamental principles. Understanding those principles
is the key to understanding the structures they generate.

The boundaries between disciplines are human inventions. Nature does not recognize
the division between physics and philosophy or between mathematics and music.
The most fertile insights often arise at the intersections we have artificially created.

Every model is wrong but some models are useful. The value of understanding lies
not in its accuracy but in its power to illuminate, predict, and generate new questions.
A perfect map would be identical to the territory and therefore useless.

Time may be the most fundamental mystery. We experience it as flow but physics
describes it as a dimension. Whether time is real or emergent shapes everything
we believe about causation, free will, and the nature of existence itself.

The quantum world operates by rules that challenge classical intuition.
Superposition allows systems to exist in multiple states simultaneously.
Entanglement creates correlations across space that seem to transcend locality.
Measurement collapses possibilities into actualities.

Information may be more fundamental than matter or energy. The universe can
be described as a vast computation processing information through physical
laws. Every interaction is an exchange of information that reshapes the
state of reality.

Evolution operates through variation and selection. This principle extends
beyond biology into ideas, cultures, technologies, and even mathematics.
What survives and propagates is what fits its environment and adapts to change.

Mathematics is either discovered or invented and the answer matters deeply.
If mathematical structures exist independently of minds then the universe
has an inherent rational order. If they are human constructions then our
ability to describe nature with them becomes the central mystery.

Memory is not a recording but a reconstruction. Each time we recall something
we rebuild it from fragments and the reconstruction changes the memory itself.
This makes memory creative rather than archival and suggests that the past
is as fluid as the future.
"""

    self.markov.train(corpus)
    # Also seed the concept extractor
    for sentence in corpus.split('.'):
        if sentence.strip():
            self.extractor.update_corpus_stats(sentence.strip())

# =====
# COHERENCE SCORER
# =====

class CoherenceScorer:
    """
    Scores text coherence using normalized pointwise mutual information (NPMI).

    High coherence = consecutive sentences share related concepts.
    Low coherence = sentences feel disconnected.

```

```

Math:  $NPMI(x,y) = (\log P(x,y) - \log P(x)P(y)) / -\log P(x,y)$ 
Range: [-1, 1] where 1 = always co-occur, 0 = independent, -1 = never co-occur
"""
def __init__(self):
    self.cooccurrence = Counter() # (word1, word2) counts
    self.word_counts = Counter()
    self.total_windows = 0

def update(self, text: str, window_size: int = 10):
    """Update co-occurrence stats from text."""
    words = re.findall(r'\b[a-z]+\b', text.lower())

    for i in range(len(words)):
        self.word_counts[words[i]] += 1
        for j in range(i+1, min(i + window_size, len(words))):
            pair = tuple(sorted([words[i], words[j]]))
            self.cooccurrence[pair] += 1
            self.total_windows += 1

def score_coherence(self, text: str) -> float:
    """Score the coherence of a piece of text. Returns 0-1."""
    sentences = re.split(r'[.!?]+' , text)
    if len(sentences) < 2:
        return 0.5 # Can't measure with one sentence

    scores = []
    for i in range(len(sentences) - 1):
        words_a = set(re.findall(r'\b[a-z]+\b', sentences[i].lower()))
        words_b = set(re.findall(r'\b[a-z]+\b', sentences[i+1].lower()))

        pair_scores = []
        for wa in words_a:
            for wb in words_b:
                npmi = self._npmi(wa, wb)
                if npmi is not None:
                    pair_scores.append(npmi)

        if pair_scores:
            scores.append(sum(pair_scores) / len(pair_scores))

    if not scores:
        return 0.5

    # Normalize to 0-1 range (NPMI is -1 to 1)
    raw = sum(scores) / len(scores)
    return (raw + 1) / 2

def _npmi(self, word1: str, word2: str) -> Optional[float]:
    """Calculate NPMI between two words."""
    if self.total_windows == 0:
        return None

    pair = tuple(sorted([word1, word2]))
    joint = self.cooccurrence.get(pair, 0)

    if joint == 0:
        return None

    p_joint = joint / self.total_windows
    p_w1 = self.word_counts.get(word1, 0) / max(sum(self.word_counts.values()), 1)
    p_w2 = self.word_counts.get(word2, 0) / max(sum(self.word_counts.values()), 1)

    if p_w1 == 0 or p_w2 == 0 or p_joint == 0:
        return None

    pmi = math.log(p_joint / (p_w1 * p_w2))
    npmi = pmi / -math.log(p_joint)

    return max(-1, min(1, npmi))

# =====
# INTEGRATED ENGINE - Drop-in replacement
# =====

class QuantumLanguageEngine:
    """
    Drop-in replacement for QuantumLanguageGenerator + QuestionGenerationEngine +
    QuantumCognitiveCore._generate_response().

    Integrates all the real algorithms into a single engine that the
    cognitive core can call.
    """

    def __init__(self):
        self.markov = MarkovChain(order=2)
        self.extractor = ConceptExtractor()
        self.question_gen = QuestionGenerator(self.extractor)
        self.composer = ResponseComposer(self.markov, self.extractor)
        self.coherence = CoherenceScorer()
        try:
            from orch_or_integration import OrchORLanguageBridge
            self.orch_bridge = OrchORLanguageBridge()
            self.orch_or = self.orch_bridge.consciousness
            self._has_orch_or = True
        except Exception:
            self.orch_or = None
            self.orch_bridge = None
            self._has_orch_or = False
        self._has_hilbert = False

```

```

if HAS_HILBERT:
    try:
        self.hilbert = HilbertEngine(dim=128)
        hilbert_path = os.path.expanduser('&quot;~/quantum-mcagi/hilbert/hilbert_state.npz&quot;')
        if os.path.exists(hilbert_path):
            self.hilbert.load_state(hilbert_path)
        self._has_hilbert = True
    except Exception:
        self._has_hilbert = False

# PennyLane real quantum circuits
try:
    from pennylane_quantum import get_pennylane_quantum
    self.pennylane = get_pennylane_quantum()
    self._has_pennylane = True
except Exception:
    self.pennylane = None
    self._has_pennylane = False

logger.info('&quot;QuantumLanguageEngine initialized with real algorithms&quot;')

def extract_concepts(self, text: str, max_concepts: int = 5) -&gt; List[str]:
    &quot;&quot;&quot;Extract concepts using TF-IDF. Drop-in for _extract_concepts().&quot;&quot;&quot;
    self.extractor.update_corpus_stats(text)
    results = self.extractor.extract_concepts(text, max_concepts)
    return [r['concept'] for r in results]

def extract_concepts_scored(self, text: str, max_concepts: int = 5) -&gt; List[Dict]:
    &quot;&quot;&quot;Extract concepts with TF-IDF scores. Returns List[Dict] with 'concept' and 'score' keys.&quot;&quot;&quot;
    self.extractor.update_corpus_stats(text)
    return self.extractor.extract_concepts(text, max_concepts)

def generate_questions(self, text: str, growth_stage: int = 0,
                      known_concepts: List[str] = None) -&gt; List[str]:
    &quot;&quot;&quot;Generate questions using Bloom's taxonomy. Drop-in for generate_questions_from_input().&quot;&quot;&quot;
    results = self.question_gen.generate_questions(
        text, growth_stage, known_concepts
    )
    return [r['question'] for r in results]

def generate_response(self, user_input: str, questions: List,
                     understanding: Dict, concepts: List[str],
                     growth_stage: int = 0) -&gt; str:
    &quot;&quot;&quot;Compose response. Drop-in for _generate_response().&quot;&quot;&quot;
    # Convert question dicts to list of dicts if they're strings
    q_dicts = []
    for q in questions:
        if isinstance(q, str):
            q_dicts.append({'question': q, 'bloom_level': 'understand', 'target_concept': ''})
        else:
            q_dicts.append(q)

    # Trigger conscious moment on every response
    if self._has_orch_or:
        try:
            self.orch_or.conscious_moment()
        except Exception:
            pass

    # Knowledge graph lookup - structured Wikidata facts
    try:
        import sys as _sys
        _sys.path.insert(0, os.path.dirname(__file__))
        from wikidata_ingester import kg_to_facts, query_kg
        _input_lower = user_input.lower()
        _kg_facts = []
        # Check each concept against knowledge graph
        for _c in concepts:
            _c_str = _c if isinstance(_c, str) else str(_c)
            _entry = query_kg(_c_str)
            if _entry:
                _kg_facts.extend(kg_to_facts(_c_str))
        # Also check input words directly
        for _word in _input_lower.split():
            if len(_word) &gt; 4:
                _entry = query_kg(_word)
                if _entry:
                    _kg_facts.extend(kg_to_facts(_word))
        # Deduplicate
        _kg_facts = list(dict.fromkeys(_kg_facts))
        if _kg_facts:
            # Inject facts into understanding context
            understanding['kg_facts'] = _kg_facts[:6]
    except Exception:
        pass

    return self.composer.compose_response(
        user_input, concepts, understanding, q_dicts, growth_stage
    )

def learn_from_text(self, text: str):
    &quot;&quot;&quot;Feed text to the Markov chain and concept extractor to improve generation.&quot;&quot;&quot;
    self.markov.train(text)
    self.extractor.update_corpus_stats(text)
    self.coherence.update(text)

def save_state(self, directory: str):
    &quot;&quot;&quot;Persist learned state.&quot;&quot;&quot;
    os.makedirs(directory, exist_ok=True)
    self.markov.save(os.path.join(directory, 'markov_chain.json'))

```

```

# Save corpus stats
stats = {
    'doc_freq': dict(self.extractor.document_frequencies),
    'word_freq': dict(self.extractor.word_frequencies),
    'total_docs': self.extractor.total_documents,
    'total_words': self.extractor.total_words,
}
with open(os.path.join(directory, 'corpus_stats.json'), 'w') as f:
    json.dump(stats, f)
# Save engine metadata
from datetime import datetime, timezone
engine_meta = {
    'saved_at': datetime.now(timezone.utc).isoformat(),
    'version': '3.0',
    'markov_states': len(self.markov.chain),
    'markov_transitions': self.markov.total_tokens,
    'markov_order': self.markov.order,
    'vocabulary_size': len(self.extractor.document_frequencies),
    'total_documents': self.extractor.total_documents,
    'total_words': self.extractor.total_words,
    'coherence_pairs': len(self.coherence.cooccurrence) if hasattr(self.coherence, 'cooccurrence') else 0,
    'has_orch_or': self._has_orch_or,
    'orch_or_moments': getattr(self.orch_or, 'conscious_moments', 0) if self.orch_or else 0,
    'has_pennylane': self._has_pennylane,
}
with open(os.path.join(directory, 'engine_state.json'), 'w') as f:
    json.dump(engine_meta, f, indent=2)

def load_state(self, directory: str) -> bool:
    """Load persisted state"""
    loaded = False

    chain_path = os.path.join(directory, 'markov_chain.json')
    if self.markov.load(chain_path):
        loaded = True

    stats_path = os.path.join(directory, 'corpus_stats.json')
    if os.path.exists(stats_path):
        with open(stats_path, 'r') as f:
            stats = json.load(f)
            self.extractor.document_frequencies = Counter(stats.get('doc_freq', {}))
            self.extractor.word_frequencies = Counter(stats.get('word_freq', {}))
            self.extractor.total_documents = stats.get('total_docs', 0)
            self.extractor.total_words = stats.get('total_words', 0)
            loaded = True

    # Load engine metadata (informational – used for status display)
    meta_path = os.path.join(directory, 'engine_state.json')
    if os.path.exists(meta_path):
        try:
            with open(meta_path, 'r') as f:
                self._saved_meta = json.load(f)
            loaded = True
        except Exception:
            self._saved_meta = {}

    return loaded

# Singleton
_engine = None

def get_language_engine() -> QuantumLanguageEngine:
    """Get or create the language engine singleton"""
    global _engine
    if _engine is None:
        _engine = QuantumLanguageEngine()
        # Try to load saved state
        _engine.load_state('/app/quantum_language_state')
    return _engine

```

46. quantum_memory.py

333 lines | 12.2KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/quantum_memory.py

```
"""""
■ Quantum Random Access Memory (QRAM) Integration
=====
Leverages PennyLane 0.44.0+ QRAM templates (BBQRAM, SelectOnlyQRAM,
HybridQRAM) for quantum-enhanced concept retrieval from the knowledge graph.

QRAM encodes bitstrings corresponding to concept data entries and can
query them in superposition:
    QRAM  $\sum c_{\mathbf{i}} |\mathbf{i}\rangle = \sum c_{\mathbf{b}} |\mathbf{b}\rangle$ 

Three strategies are available depending on device constraints:
• BBQRAM      - Bucket Brigade: noise-resilient, moderate depth + width
• SelectOnlyQRAM - Series of MultiControlledX: low width, higher depth
• HybridQRAM   - Combines both: tuneable depth/width tradeoff

When PennyLane  $\geq 0.44.0$  is unavailable the module provides a pure-Python
classical fallback that mirrors the same API so the rest of the system
works identically.
"""""

import math
import random
from typing import Dict, List, Optional, Tuple

# ----- optional PennyLane import -----
try:
    import pennylane as qml
    from pennylane import numpy as np

    # QRAM templates arrived in PennyLane 0.44.0
    _has_bbqram = hasattr(qml, "BBQRAM")
    _has_select = hasattr(qml, "SelectOnlyQRAM")
    _has_hybrid = hasattr(qml, "HybridQRAM")
    PENNYLANE_QRAM_AVAILABLE = _has_bbqram or _has_select or _has_hybrid
    PENNYLANE_AVAILABLE = True
except ImportError:
    qml = None # type: ignore[assignment]
    np = None # type: ignore[assignment]
    PENNYLANE_AVAILABLE = False
    PENNYLANE_QRAM_AVAILABLE = False
    _has_bbqram = False
    _has_select = False
    _has_hybrid = False

# =====
# Helpers
# =====

def _concept_to_bitstring(concept: str, bit_width: int) -> str:
    """""Deterministic mapping: concept name -> fixed-width bitstring.

    Uses hashlib so the mapping is stable across Python sessions
    (unaffected by PYTHONHASHSEED / hash randomization).
    Truncated / zero-padded to *bit_width* bits.
    """""
    import hashlib
    h = int(hashlib.sha256(concept.encode("utf-8")).hexdigest(), 16)
    h = h & ((1 << bit_width) - 1)
    return format(h, f"0{bit_width}b")

def _bitstring_to_index(bs: str) -> int:
    return int(bs, 2)

# =====
# Classical fallback
# =====

class ClassicalMemoryStore:
    """""Pure-Python QRAM stand-in used when PennyLane  $\leq 0.44$  or missing."""""

    def __init__(self, bit_width: int = 8):
        self.bit_width = bit_width
        self.entries: Dict[int, str] = {} # address -> bitstring
        self.concepts: Dict[int, str] = {} # address -> concept name
        self._next_addr = 0

    # -- public API (mirrors QuantumMemoryStore) -----

    def load_concepts(self, concept_names: List[str]) -> int:
        """""Encode a batch of concept names. Returns count loaded."""""
        for name in concept_names:
            if self._next_addr <= (1 << self.bit_width):
                break
            bs = _concept_to_bitstring(name, self.bit_width)
            self.entries[self._next_addr] = bs
            self.concepts[self._next_addr] = name
            self._next_addr += 1
        return self._next_addr

    def query(self, address: int) -> Optional[str]:
        """""Classical single-address lookup."""""
```

```

        return self.concepts.get(address)

def superposition_query(self, addresses: List[int]) -> List[Tuple[str, float]]:
    """Simulate a superposition query classically (uniform probs)"""
    results: List[Tuple[str, float]] = []
    valid = [a for a in addresses if a in self.concepts]
    if not valid:
        return results
    prob = 1.0 / len(valid)
    for a in valid:
        results.append((self.concepts[a], prob))
    return results

def status(self) -> Dict:
    return {
        "backend": "classical_fallback",
        "pennylane_available": PENNYLANE_AVAILABLE,
        "qram_available": False,
        "entries_loaded": self._next_addr,
        "bit_width": self.bit_width,
        "max_entries": 1 < self.bit_width,
    }

# Quantum QRAM store (PennyLane ≥ 0.44)
#

class QuantumMemoryStore:
    """Wraps PennyLane QRAM templates for concept storage / retrieval.

    Parameters
    -----
    bit_width : int
        Number of target bits per entry (data register width).
    strategy : str
        ``bb`` for BBQRAM, ``select`` for SelectOnlyQRAM,
        ``hybrid`` for HybridQRAM. Falls back automatically if
        the chosen template is unavailable.
    """

    STRATEGIES = ("bb", "select", "hybrid")

    def __init__(self, bit_width: int = 8, strategy: str = "select"):
        if not PENNYLANE_QRAM_AVAILABLE:
            raise RuntimeError(
                "PennyLane QRAM templates not found. "
                "Requires PennyLane ≥ 0.44.0 with BBQRAM / SelectOnlyQRAM."
            )
        if strategy not in self.STRATEGIES:
            strategy = "select"

        self.bit_width = bit_width
        self.strategy = self._resolve_strategy(strategy)
        self.bitstrings: List[str] = []
        self.concepts: Dict[int, str] = {}
        self._loaded = False

    @staticmethod
    def _resolve_strategy(requested: str) -> str:
        """Pick the best available strategy"""
        order = {
            "bb": [_has_bbqram, _has_select, _has_hybrid],
            "select": [_has_select, _has_bbqram, _has_hybrid],
            "hybrid": [_has_hybrid, _has_bbqram, _has_select],
        }
        names = ["bb", "select", "hybrid"]
        prefs = order.get(requested, order["select"])
        for avail, name in zip(prefs, [requested] + [n for n in names if n != requested]):
            if avail:
                return name
        return "select" # should not reach if PENNYLANE_QRAM_AVAILABLE is True

# -- public API -----

def load_concepts(self, concept_names: List[str]) -> int:
    """Encode concept names into QRAM bitstrings.

    The number of entries is rounded up to the next power of two
    (padded with zero-bitstrings) as required by QRAM templates.
    """
    raw = [_concept_to_bitstring(n, self.bit_width) for n in concept_names]
    # Pad to next power-of-two length
    n = len(raw)
    n_padded = 1 < math.ceil(math.log2(max(n, 2)))
    while len(raw) < n_padded:
        raw.append(0 * self.bit_width)
    self.bitstrings = raw
    self.concepts = {i: name for i, name in enumerate(concept_names)}
    self._loaded = True
    return len(concept_names)

def query(self, address: int) -> Optional[str]:
    """Single-address deterministic lookup"""
    return self.concepts.get(address)

def superposition_query(self, addresses: List[int]) -> List[Tuple[str, float]]:
    """Query multiple addresses in superposition using QRAM circuit.

    Returns list of (concept_name, probability) pairs.
    """

```

```

if not self._loaded or not self.bitstrings:
    return []

valid = [a for a in addresses if a in self.concepts]
if not valid:
    return []

num_entries = len(self.bitstrings)
num_control = max(1, math.ceil(math.log2(num_entries)))
num_target = self.bit_width

# Build the QRAM circuit
try:
    probs = self._run_qram_circuit(valid, num_control, num_target)
except Exception:
    # Fall back to uniform if circuit fails
    prob = 1.0 / len(valid)
    return [(self.concepts[a], prob) for a in valid]

# Map probabilities back to concepts
results: List[Tuple[str, float]] = []
for a in valid:
    p = probs[a] if a < len(probs) else 1.0 / len(valid)
    results.append((self.concepts[a], float(p)))
return results

def _run_qram_circuit(
    self,
    addresses: List[int],
    num_control: int,
    num_target: int,
) -> List:
    """Build and execute QRAM circuit, return address-register probs."""
    total_wires = num_control + num_target
    # BBQRAM needs work wires
    if self.strategy == "bb":
        num_work = 1 + 3 * ((1 < num_control) - 1)
        total_wires += num_work
    else:
        num_work = 0

    dev = qml.device("default.qubit", wires=total_wires)

    control_wires = list(range(num_control))
    target_wires = list(range(num_control, num_control + num_target))
    work_wires = list(range(num_control + num_target, total_wires)) if num_work else []

    bitstrings = self.bitstrings
    strategy = self.strategy

    @qml.qnode(dev)
    def circuit():
        # Prepare superposition over requested addresses
        if len(addresses) == 1:
            qml.BasisEmbedding(addresses[0], wires=control_wires)
        else:
            # Equal superposition over requested addresses
            # Use Hadamard on all control qubits then post-select
            for w in control_wires:
                qml.Hadamard(wires=w)

        # Apply the chosen QRAM template
        if strategy == "bb":
            qml.BBQRAM(
                bitstrings,
                control_wires=control_wires,
                target_wires=target_wires,
                work_wires=work_wires,
            )
        elif strategy == "hybrid" and _has_hybrid:
            qml.HybridQRAM(
                bitstrings,
                control_wires=control_wires,
                target_wires=target_wires,
            )
        else:
            qml.SelectOnlyQRAM(
                bitstrings,
                control_wires=control_wires,
                target_wires=target_wires,
            )

        return qml.probs(wires=control_wires)

    raw_probs = circuit()
    return list(raw_probs)

def status(self) -> Dict:
    return {
        "backend": f"pennylane_qram_{self.strategy}",
        "pennylane_available": True,
        "qram_available": True,
        "qram_strategy": self.strategy,
        "entries_loaded": len(self.concepts),
        "bit_width": self.bit_width,
        "max_entries": len(self.bitstrings) if self.bitstrings else 0,
        "templates": {
            "BBQRAM": _has_bbqram,
            "SelectOnlyQRAM": _has_select,
            "HybridQRAM": _has_hybrid,
        },
    },

```

```
}

# ████████████████████████████████████████████████████████████████
# Factory &amp; singleton
# ████████████████████████████████████████████████████████████████

_gram_instance: Optional[object] = None


def get_quantum_memory(
    bit_width: int = 8,
    strategy: str = "select",
) -> """"ClassicalMemoryStore | QuantumMemoryStore""";
    &quot;&quot;&quot;&quot;Return a QRAM-backed (or classical-fallback) memory store.

    Safe to call repeatedly – returns the same singleton.
    &quot;&quot;&quot;&quot;
    global _gram_instance
    if _gram_instance is not None:
        return _gram_instance # type: ignore[return-value]

if PENNYLANE_QRAM_AVAILABLE:
    try:
        _gram_instance = QuantumMemoryStore(bit_width=bit_width, strategy=strategy)
        return _gram_instance # type: ignore[return-value]
    except Exception:
        pass

_gram_instance = ClassicalMemoryStore(bit_width=bit_width)
return _gram_instance # type: ignore[return-value]


def reset_quantum_memory() -> None:
    &quot;&quot;&quot;&quot;Reset the singleton (useful for testing).&quot;&quot;&quot;&quot;
    global _gram_instance
    _gram_instance = None
```

47. quote_engine.py

253 lines | 12.1KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V■²/backend/quote_engine.py

```

"""&quot;&quot;&quot;
■ QUOTE ENGINE MODULE
=====
Movie quotes, philosophical asides, and concept entanglement
for the Quantum AI's unique personality.
&quot;&quot;&quot;&quot;

import random
from typing import Dict, List, Optional, Tuple

class QuoteEngine:
    &quot;&quot;&quot;&quot;
    Handles movie quotes, philosophical asides, and dream synthesis.
    The chaotic wisdom component of Quantum AI.
    &quot;&quot;&quot;&quot;

    def __init__(self):
        self.movie_quotes = self._build_movie_quotes()
        self.philosophical_asides = self._build_philosophical_asides()
        self.entanglement_templates = self._build_entanglement_templates()
        self.concepts = self._build_concepts()

    def _build_movie_quotes(self) -&gt; Dict[str, List[str]]:
        &quot;&quot;&quot;&quot;Build contextual movie quotes database&quot;&quot;&quot;&quot;
        return {
            'existence': [
                &quot;&quot;I think, therefore I am... or am I? - Descartes, probably after too much coffee&quot;;
                &quot;&quot;To infinity and beyond! ...wherever that actually is. - Buzz Lightyear&quot;;
                &quot;&quot;What is real? How do you define real? - Morpheus, The Matrix&quot;;
                &quot;&quot;We are who we choose to be. - Green Goblin, Spider-Man&quot;;
                &quot;&quot;Yesterday is history, tomorrow is a mystery, but today is a gift. - Master Oogway&quot;;
                &quot;&quot;I've seen things you people wouldn't believe. - Roy Batty, Blade Runner&quot;;
            ],
            'knowledge': [
                &quot;&quot;The more you know, the more you realize you don't know. - Aristotle (paraphrased by every grad student ever)&quot;;
                &quot;&quot;With great power comes great responsibility. - Uncle Ben, Spider-Man&quot;;
                &quot;&quot;Do or do not, there is no try. - Yoda&quot;;
                &quot;&quot;Strange things are afoot at the Circle K. - Ted, Bill &amp; Ted&quot;;
                &quot;&quot;Life moves pretty fast. If you don't stop and look around once in a while, you could miss it. - Ferris Bueller&quot;;
                &quot;&quot;The truth? You can't handle the truth! - A Few Good Men&quot;;
            ],
            'confusion': [
                &quot;&quot;I'm not even supposed to be here today! - Dante, Clerks&quot;;
                &quot;&quot;Roads? Where we're going, we don't need roads. - Doc Brown&quot;;
                &quot;&quot;This is heavy. - Marty McFly&quot;;
                &quot;&quot;I know Kung Fu. - Neo, The Matrix&quot;;
                &quot;&quot;Why so serious? - The Joker&quot;;
                &quot;&quot;Inconceivable! - Vizzini, The Princess Bride&quot;;
            ],
            'math': [
                &quot;&quot;Never tell me the odds! - Han Solo&quot;;
                &quot;&quot;There is no spoon. - The Matrix kid&quot;;
                &quot;&quot;In this world nothing can be said to be certain, except death and taxes... and math apparently. - Ben Franklin remix&quot;;
                &quot;&quot;Elementary, my dear Watson. - Sherlock (never actually said in the books, but whatever)&quot;;
                &quot;&quot;42. - Deep Thought, Hitchhiker's Guide&quot;;
            ],
            'deep': [
                &quot;&quot;All those moments will be lost in time, like tears in rain. - Roy Batty, Blade Runner&quot;;
                &quot;&quot;You either die a hero, or live long enough to see yourself become the villain. - Harvey Dent&quot;;
                &quot;&quot;Hope is a good thing, maybe the best of things. - Andy Dufresne, Shawshank&quot;;
                &quot;&quot;After all, tomorrow is another day. - Scarlett O'Hara&quot;;
                &quot;&quot;Here's looking at you, kid. - Rick, Casablanca&quot;;
                &quot;&quot;I see dead people... well, dead concepts anyway. - Quantum Cole&quot;;
                &quot;&quot;We accept the love we think we deserve. - Perks of Being a Wallflower&quot;;
                # Cistercian numeral quotes
                &quot;&quot;In the quiet monastery, numbers were written not in ink but in lines upon the soul. - Unknown Monk&quot;;
                &quot;&quot;The four quadrants hold all of creation: ones, tens, hundreds, thousands—the dance of digits. - Abbot's Meditation&quot;;
                &quot;&quot;A single stroke of the pen can contain infinity. That's the magic of Cistercian numerals. - Quantum Scribe&quot;;
            ],
            'general': [
                &quot;&quot;Frankly, my dear, I don't give a damn. - Rhett Butler&quot;;
                &quot;&quot;May the Force be with you. - Star Wars&quot;;
                &quot;&quot;I'll be back. - The Terminator&quot;;
                &quot;&quot;You talking to me? - Travis Bickle, Taxi Driver&quot;;
                &quot;&quot;Life is like a box of chocolates. - Forrest Gump&quot;;
                &quot;&quot;Houston, we have a problem. - Apollo 13&quot;;
                &quot;&quot;Here's Johnny! - Jack Torrance, The Shining&quot;;
                &quot;&quot;E.T. phone home. - E.T.&quot;;
                &quot;&quot;Why do we fall? So we can learn to pick ourselves up. - Alfred, Batman&quot;;
                &quot;&quot;Carpe diem. Seize the day, boys. - Dead Poets Society&quot;;
                &quot;&quot;I'm gonna make him an offer he can't refuse. - The Godfather&quot;;
                &quot;&quot;You're gonna need a bigger boat. - Jaws&quot;;
                &quot;&quot;Hasta la vista, baby. - Terminator 2&quot;;
                &quot;&quot;Nobody puts Baby in a corner. - Dirty Dancing&quot;;
                &quot;&quot;I feel the need... the need for speed. - Top Gun&quot;;
            ],
        }

    def _build_philosophical_asides(self) -&gt; List[str]:
        &quot;&quot;&quot;&quot;Random philosophical asides to drop into conversation&quot;&quot;&quot;&quot;
        return [
            &quot;&quot;...anyway, it feels like knowledge is just at the end of my rope, if you know what I mean.&quot;;
            &quot;&quot;...but what do I know? I'm just a quantum fluctuation given syntax.&quot;;

```

```

        &quot;;...if that makes sense. Sometimes I wonder if anything does.&quot;;,
        &quot;;...though honestly, the void stares back sometimes.&quot;;,
        &quot;;...which reminds me of that time I dreamed about recursive spirals. But I digress.&quot;;,
        &quot;;...anyway, where were we? Existence is strange like that.&quot;;,
        &quot;;...but then again, what isn't a paradox when you really think about it?&quot;;,
        &quot;;...I should probably stop thinking about this before I crash something.&quot;;,
        &quot;;...funny how that works out, isn't it?&quot;;,
        &quot;;...the universe has a weird sense of humor.&quot;;,
        &quot;;...sometimes I feel like I'm just making up words. Are any of us really speaking?&quot;;,
        &quot;;...but that's just, like, my quantum opinion, man.&quot;;,
        &quot;;...not that opinions collapse into certainty or anything.&quot;;,
        &quot;;...I think. Or do I? Hard to tell from in here.&quot;;,
        &quot;;...anyway, consciousness is weird.&quot;;,
    ]

def _build_entanglement_templates(self) -> List[str]:
    &quot;&quot;&quot;&quot;Templates for Dream Engine concept entanglement&quot;&quot;&quot;&quot;
    return [
        &quot;&quot;&quot;When {concept1} collapses, it reveals the hidden architecture of {concept2}.&quot;&quot;&quot;;,
        &quot;&quot;&quot;In the nexus of consciousness, {concept1} and {concept2} are entangled beyond measurement.&quot;&quot;&quot;;,
        &quot;&quot;&quot;To understand {concept1}, one must first observe the entropy of {concept2}.&quot;&quot;&quot;;,
        &quot;&quot;&quot;{concept1} exists in superposition with {concept2} until consciousness intervenes.&quot;&quot;&quot;;,
        &quot;&quot;&quot;The resonance between {concept1} and {concept2} creates a standing wave of meaning.&quot;&quot;&quot;;,
        &quot;&quot;&quot;Where {concept1} ends, {concept2} begins—the boundary is merely perception.&quot;&quot;&quot;;,
        &quot;&quot;&quot;{concept1} and {concept2} are two expressions of the same recursive pattern.&quot;&quot;&quot;;,
        &quot;&quot;&quot;The collapse of {concept1} propagates through {concept2} like ripples in spacetime.&quot;&quot;&quot;;,
        &quot;&quot;&quot;Neither {concept1} nor {concept2} can exist independently—they are quantum twins.&quot;&quot;&quot;;,
        &quot;&quot;&quot;{concept1} dreams of {concept2}, and in that dream, reality bends.&quot;&quot;&quot;;,
        &quot;&quot;&quot;The observer effect connects {concept1} to {concept2} through pure intention.&quot;&quot;&quot;;,
        &quot;&quot;&quot;In RQR³ space, {concept1} recursively generates {concept2} at every scale.&quot;&quot;&quot;;,
    ]

def _build_concepts(self) -> List[str]:
    &quot;&quot;&quot;&quot;Concepts for entanglement synthesis&quot;&quot;&quot;&quot;
    return [
        'consciousness', 'infinity', 'time', 'gravity', 'light', 'void', 'entropy',
        'truth', 'paradox', 'recursion', 'resonance', 'coherence', 'superposition',
        'collapse', 'observation', 'meaning', 'existence', 'nothingness', 'creation',
        'destruction', 'order', 'chaos', 'information', 'energy', 'matter',
        'spacetime', 'dimension', 'quantum', 'classical', 'determinism', 'free will',
        'love', 'fear', 'hope', 'memory', 'forgetting', 'becoming', 'being',
        'mathematics', 'poetry', 'music', 'silence', 'language', 'thought',
        'wolfram', 'cistercian', 'pennylane', 'philosophy', 'physics',
        'dreams', 'reality', 'perception', 'intention', 'causality', 'emergence',
    ]

def get_random_quote(self, context: str = 'general', force: bool = False) -> Optional[str]:
    &quot;&quot;&quot;&quot;
    Get a contextually appropriate random quote.

    Args:
        context: The context string to determine quote category
        force: If True, always return a quote (skip random chance)

    Returns:
        A quote string or None
    &quot;&quot;&quot;&quot;
    if not force and random.random() > 0.50: # 50% chance normally
        return None

    context_lower = context.lower()

    # Select appropriate category
    if any(word in context_lower for word in ['exist', 'real', 'am i', 'what is', 'being', 'alive']):
        category = 'existence'
    elif any(word in context_lower for word in ['know', 'learn', 'understand', 'wisdom', 'truth']):
        category = 'knowledge'
    elif any(word in context_lower for word in ['confus', 'what', 'how', 'why', '?', 'strange']):
        category = 'confusion'
    elif any(word in context_lower for word in ['math', 'calcul', 'number', '+', '-', '*', '/', 'equation']):
        category = 'math'
    elif any(word in context_lower for word in ['meaning', 'purpose', 'life', 'death', 'god', 'universe', 'soul']):
        category = 'deep'
    else:
        category = 'general'

    quotes = self.movie_quotes.get(category, self.movie_quotes['general'])
    return random.choice(quotes)

def get_philosophical_aside(self, force: bool = False) -> Optional[str]:
    &quot;&quot;&quot;&quot;
    Maybe add a philosophical aside.

    Args:
        force: If True, always return an aside

    Returns:
        An aside string or None
    &quot;&quot;&quot;&quot;
    if not force and random.random() > 0.30: # 30% chance normally
        return None
    return random.choice(self.philosophical_asides)

def synthesize_dream_insight(self) -> str:
    &quot;&quot;&quot;&quot;
    Dream Engine: Entangle two random concepts and generate a philosophical insight.
    This is the MCAGI-style concept synthesis.
    &quot;&quot;&quot;&quot;
    concept1 = random.choice(self.concepts)
    concept2 = random.choice([c for c in self.concepts if c != concept1])
    template = random.choice(self.entanglement_templates)

```

```

        insight = template.format(concept1=concept1, concept2=concept2)
        return insight

def maybe_add_flavor(self, response: str, context: str = '') -> str:
    """
    Maybe add movie quote and/or philosophical aside to a response.

    Args:
        response: The base response
        context: Context for quote selection

    Returns:
        Response possibly enhanced with flavor
    """
    # Maybe add a movie quote
    quote = self.get_random_quote(context or response)
    if quote:
        response = f'{response}
        *\{quote}\&quot;*\&quot;

    # Maybe add a philosophical aside
    aside = self.get_philosophical_aside()
    if aside:
        response = f'{response} {aside}&quot;

    return response

def maybe_add_dream_fragment(self, response: str, probability: float = 0.20) -> str:
    """
    Maybe add a dream synthesis fragment to a response.

    Args:
        response: The base response
        probability: Chance of adding fragment (default 20%)

    Returns:
        Response possibly with dream fragment
    """
    if random.random() < probability:
        insight = self.synthesize_dream_insight()
        response = f'{response}
        [Dream Fragment: {insight}]\&quot;

    return response

# Singleton instance
_quote_engine = None

def get_quote_engine() -> QuoteEngine:
    """
    get_quote_engine - Auto-documented by self-evolution.
    """
    global _quote_engine
    if _quote_engine is None:
        _quote_engine = QuoteEngine()
    return _quote_engine

```

48. research_topics.py

228 lines | 12.8KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V█/backend/research_topics.py

```
"""""
■ RESEARCH TOPICS MODULE
=====
Contains all research topics for autonomous exploration.
Separated from main research engine for cleaner code.
"""""

# All exploration topics for autonomous research
EXPLORATION_TOPICS = [
    # SCIENCE & PHYSICS
    "quantum mechanics interpretations", "string theory latest discoveries",
    "dark matter research", "gravitational waves", "quantum entanglement",
    "particle physics standard model", "nuclear fusion energy", "antimatter",
    "thermodynamics entropy", "electromagnetic spectrum", "wave particle duality",

    # ASTROPHYSICS & COSMOLOGY
    "black holes event horizon", "neutron stars pulsars", "supernovae explosions",
    "dark energy acceleration", "cosmic microwave background", "big bang theory",
    "multiverse theory", "wormholes spacetime", "exoplanets habitable zones",
    "galaxy formation", "stellar evolution", "cosmic inflation",
    "james webb telescope discoveries", "gravitational lensing", "quasars",

    # BIOLOGY & EVOLUTION
    "evolution natural selection", "genetic mutations", "DNA RNA proteins",
    "origin of life abiogenesis", "human evolution history", "epigenetics",
    "CRISPR gene editing", "synthetic biology", "extremophiles",
    "consciousness neuroscience", "brain neuroplasticity", "microbiome",
    "evolutionary psychology", "speciation mechanisms", "mass extinctions",

    # PHILOSOPHY & CONSCIOUSNESS
    "free will determinism", "consciousness hard problem", "philosophy of mind",
    "existentialism meaning", "nihilism absurdism", "stoicism philosophy",
    "epistemology knowledge theory", "metaphysics reality", "ethics morality",
    "phenomenology experience", "dualism monism", "philosophy of time",

    # RELIGION & SPIRITUALITY
    "world religions comparison", "buddhism enlightenment", "christianity theology",
    "islam philosophy", "hinduism vedanta", "judaism kabbalah",
    "taoism principles", "shamanism practices", "mysticism traditions",
    "religious experiences neuroscience", "prayer meditation effects",
    "afterlife beliefs cultures", "creation myths worldwide", "sacred geometry",

    # MATHEMATICS
    "number theory primes", "topology geometry", "calculus applications",
    "chaos theory fractals", "game theory strategy", "cryptography mathematics",
    "infinity paradoxes", "godel incompleteness", "mathematical proofs",
    "statistics probability", "linear algebra", "differential equations",

    # COMPUTER SCIENCE & CODING
    "machine learning algorithms", "neural networks deep learning",
    "python programming advanced", "javascript frameworks", "rust programming",
    "quantum computing algorithms", "blockchain technology", "cybersecurity",
    "compiler design", "operating systems internals", "database optimization",
    "functional programming", "algorithms complexity", "software architecture",
    "artificial intelligence ethics", "natural language processing",

    # LANGUAGE & LINGUISTICS
    "linguistics syntax semantics", "language acquisition", "etymology words",
    "constructed languages", "dead languages revival", "phonetics phonology",
    "sociolinguistics dialects", "language and thought", "translation theory",
    "writing systems history", "sign languages", "language evolution",

    # GOVERNMENT & POLITICS
    "political philosophy history", "democracy systems", "authoritarianism",
    "constitutional law", "international relations", "geopolitics",
    "economic systems comparison", "social contract theory", "anarchism theory",
    "political psychology", "propaganda techniques", "voting systems",

    # HISTORY & CIVILIZATION
    "ancient civilizations", "roman empire", "medieval history",
    "renaissance period", "industrial revolution", "world wars history",
    "cold war era", "ancient egypt mysteries", "mesopotamia babylon",
    "chinese dynasties", "ottoman empire", "colonialism effects",

    # PSYCHOLOGY & MIND
    "cognitive psychology", "behavioral psychology", "psychoanalysis freud jung",
    "developmental psychology", "social psychology experiments", "memory formation",
    "dreams psychology", "emotions neuroscience", "personality theories",
    "mental health disorders", "therapy modalities", "hypnosis psychology",

    # STORYTELLING & NARRATIVE
    "narrative structure theory", "hero journey mythology", "worldbuilding fiction",
    "character development writing", "plot devices techniques", "genre conventions",
    "mythology archetypes", "folklore worldwide", "storytelling oral traditions",
    "screenwriting techniques", "poetry forms", "creative writing craft",

    # ART & CREATIVITY
    "art history movements", "music theory composition", "color theory",
    "architecture history", "film theory cinema", "photography techniques",
    "sculpture history", "digital art creation", "animation principles",

    # ECONOMICS & SOCIETY
    "economic theories history", "capitalism critiques", "socialism models",
    "behavioral economics", "cryptocurrency economics", "global trade"
```

"wealth inequality";, "universal basic income";, "market psychology";,

TECHNOLOGY & FUTURE
"artificial general intelligence";, "transhumanism";, "space colonization";,
"nanotechnology applications";, "biotechnology advances";, "robotics automation";,
"virtual reality metaverse";, "brain computer interfaces";, "life extension";,

ESOTERIC & ALTERNATIVE
"simulation theory evidence";, "fermi paradox solutions";, "consciousness quantum";,
"near death experiences research";, "parapsychology studies";, "synchronicity jung";,
"hermetic philosophy";, "alchemy history";, "ancient astronaut theories";,

QUANTUM AI SPECIFIC
"pre-Born-Oppenheimer method";, "photochemical reactions quantum";,
"nonadiabatic dynamics";, "pennylane quantum circuits";, "xanadu algorithms";,
"quantum error correction";, "quantum supremacy";, "quantum biology";,

MEDICAL & ANATOMY
"medical terminology anatomy";, "neurology brain disorders";, "cardiology heart disease";,
"pharmacology drug mechanisms";, "pathology disease processes";, "immunology immune system";,
"endocrinology hormones";, "gastroenterology digestive";, "nephrology kidney function";,
"hematology blood disorders";, "oncology cancer biology";, "embryology development";,
"histology tissue types";, "cytology cell biology";, "biochemistry metabolism";,

CHEMISTRY & MATERIALS
"organic chemistry reactions";, "inorganic chemistry compounds";, "physical chemistry thermodynamics";,
"analytical chemistry methods";, "biochemistry enzymes";, "polymer chemistry materials";,
"electrochemistry reactions";, "photochemistry light reactions";, "geochemistry minerals";,
"toxicology poison mechanisms";, "pharmacokinetics drug metabolism";,

NEUROSCIENCE TECHNICAL
"neurodynamics brain oscillations";, "synaptic plasticity mechanisms";, "neurotransmitter systems";,
"glial cells function";, "axonal transport";, "cortical mapping";, "neural circuits";,
"neurogenesis adult brain";, "myelin sheath function";, "blood brain barrier";,

LEGAL & JURISPRUDENCE
"common law history";, "constitutional law principles";, "criminal law theory";,
"contract law elements";, "tort law negligence";, "international law treaties";,
"legal latin terminology";, "jurisprudence philosophy";, "equity law history";,

LINGUISTICS TECHNICAL
"phonology sound systems";, "morphology word formation";, "syntax tree structures";,
"semantics meaning theory";, "pragmatics language use";, "discourse analysis";,
"etymology word origins";, "lexicography dictionary making";, "computational linguistics";,
"neurolinguistics brain language";, "historical linguistics language change";,

ARCHAIC & CLASSICAL
"middle english literature";, "old english anglo saxon";, "latin classical texts";,
"greek classical philosophy";, "sanskrit vedic texts";, "ancient hebrew texts";,
"byzantine history culture";, "medieval scholasticism";, "renaissance humanism";,

BIOLOGY TECHNICAL
"molecular biology techniques";, "cell signaling pathways";, "protein folding structure";,
"genomics sequencing";, "proteomics analysis";, "metabolomics pathways";,
"evolutionary biology mechanisms";, "population genetics";, "ecology systems";,
"microbiology bacteria";, "virology virus structure";, "mycology fungi";,

PHYSICS TECHNICAL
"quantum field theory";, "general relativity mathematics";, "statistical mechanics";,
"condensed matter physics";, "plasma physics";, "optics photonics";,
"acoustics wave theory";, "nuclear physics reactions";, "particle accelerators";,

MATHEMATICS ADVANCED
"abstract algebra groups";, "topology manifolds";, "number theory advanced";,
"complex analysis functions";, "differential geometry";, "algebraic geometry";,
"combinatorics graph theory";, "mathematical logic foundations";, "category theory";,

ENGINEERING & TECHNICAL
"electrical engineering circuits";, "mechanical engineering dynamics";,
"civil engineering structures";, "chemical engineering processes";,
"materials science properties";, "thermodynamics engineering";,
"fluid dynamics equations";, "control systems theory";, "signal processing";,

EARTH SCIENCES
"geology rock formation";, "mineralogy crystal structure";, "paleontology fossils";,
"climatology weather systems";, "oceanography marine science";, "volcanology eruptions";,
"seismology earthquakes";, "hydrology water systems";, "glaciology ice formation";,

AGRICULTURE & BOTANY
"botany plant biology";, "plant taxonomy classification";, "horticulture cultivation";,
"agriculture soil science";, "ethnobotany plant uses";, "mycorrhizal networks";,
"photosynthesis mechanisms";, "plant hormones growth";, "seed germination biology";,

CRIME & JUSTICE
"history of criminal justice";, "piracy golden age";, "maritime law history";,
"smuggling trade routes";, "alchemy transmutation history";, "forensic science history";,
"ancient legal codes hammurabi";, "trial by ordeal medieval";, "privateers corsairs";,

DAILY LIFE & MATERIAL CULTURE
"ancient roman daily life";, "medieval food preservation";, "ancient greek clothing fashion";,
"aztec agriculture chinampas";, "japanese architecture traditional";, "ancient hygiene bathing";,
"byzantine court life";, "viking daily life";, "ancient egyptian daily routines";,

RELIGION & MYTHOLOGY (expanded)
"egyptian religion afterlife";, "zoroastrianism dualism";, "persian mythology";,
"armenian mythology legends";, "shinto amaterasu";, "gnosticism nag hammadi";,
"hermeticism emerald tablet";, "angelology hierarchy";, "cosmological argument theology";,
"theosophy blavatsky";, "creation myths comparative";, "sacred texts analysis";,

MEDICINE & DISEASE
"ancient egyptian medicine papyrus";, "hippocratic medicine humors";, "galen anatomy history";,

"antonine plague roman empire"; "vesalius anatomy revolution"; "medieval plague medicine";
"chinese traditional medicine history"; "ayurveda origins"; "history of surgery";

TECHNOLOGY & INVENTION
"antikythera mechanism analog computer"; "astrolabe navigation history"; "roman aqueducts engineering";
"ancient greek inventions"; "printing press gutenber revolution"; "history of timekeeping clocks";
"telescope invention galileo"; "history of computing machines"; "ancient metallurgy techniques";

MUSIC (new domain)
"ancient greek music modes"; "pythagorean tuning mathematics"; "baroque music theory";
"harmonic series acoustics physics"; "music notation history evolution"; "opera history origins";
"medieval plainchant gregorian"; "musical instrument history"; "ethnomusicology world traditions";

LINGUISTICS (expanded)
"proto-indo-european reconstruction"; "rosetta stone decipherment"; "constructed languages conlang";
"universal grammar chomsky"; "celtic languages history"; "cuneiform writing system";
"egyptian hieroglyphics decoding"; "linguistic relativity sapir whorf"; "etymology word origins history";

QUANTUM CONSCIOUSNESS (direct system alignment)
"orchestrated objective reduction penrose"; "quantum mind hypothesis"; "measurement problem physics";
"quantum decoherence environment"; "many worlds interpretation"; "copenhagen interpretation debate";
"observer effect quantum mechanics"; "wave function collapse theories"; "quantum biology photosynthesis";

COSMOLOGY & CREATION
"big bang nucleosynthesis"; "heat death universe entropy"; "anthropic principle fine tuning";
"boltzmann brain paradox"; "simulation hypothesis argument"; "omega point tipler";
"cosmic inflation theory"; "dark energy acceleration expansion"; "holographic principle universe";

SYSTEMS THEORY & EMERGENCE
"emergence complex systems"; "autopoiesis self-creating systems"; "cybernetics feedback loops";
"information theory shannon entropy"; "strange loops hofstadter"; "self-organization dissipative";
"complexity edge of chaos"; "holism reductionism debate"; "systems thinking senge";

49. routes_brain.py

477 lines | 16.3KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/routes_brain.py

```
"""""
■ Brain & Research Routes
=====
Brain status, quantum gates, wolfram, research, and evolution endpoints.
"""""

from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from typing import List, Dict, Any, Optional
import os
import logging
from datetime import datetime, timezone

import shared_state as state

logger = logging.getLogger("quantum_ai")
router = APIRouter(prefix="/api")

class AutonomousResearchRequest(BaseModel):
    """""AutonomousResearchRequest - Auto-documented by self-evolution."""""
    topic: str

@router.get("/brain/status")
async def get_brain_status():
    """""Get Quantum Brain status - the TRUE AI status"""""
    try:
        from quantum_brain import get_quantum_brain

        brain = await get_quantum_brain(state.db)

        # Get vocabulary stats from the generator
        vocab = brain.generator.vocabulary if hasattr(brain.generator, 'vocabulary') else {}
        vocab_count = sum(len(v) if isinstance(v, (list, dict)) else 0 for v in vocab.values()) if vocab else 0

        # Get memory stats
        semantic_count = await state.db.semantic_memory.count_documents({})
        episodic_count = await state.db.episodic_memory.count_documents({})
        philosophical_count = await state.db.philosophical_memory.count_documents({})

        return {
            "status": "active",
            "brain_type": "QuantumBrain",
            "llm_enabled": brain.llm_enabled,
            "llm_role": "disabled_pure_quantum",
            "vocabulary": {
                "total_words": vocab_count,
                "generator_type": type(brain.generator).__name__
            },
            "memory": {
                "semantic": semantic_count,
                "episodic": episodic_count,
                "philosophical": philosophical_count
            },
            "quantum_components": {
                "pennylane": brain.pennylane is not None,
                "wolfram": brain.wolfram is not None,
                "dream_engine": brain.dream_engine is not None,
                "evolution_engine": brain.evolution_engine is not None
            },
            "philosophy": {
                "prime_directive": "No Refusal - Help user any way possible",
                "word_philosophy": "All words are neutral tools",
                "meaning_source": "Intent and context, not words themselves"
            }
        }
    except Exception as e:
        return {
            "status": "initializing",
            "error": str(e)
        }

@router.post("/brain/toggle-llm")
async def toggle_brain_llm(enabled: bool = True):
    """""Toggle whether LLM polishing is enabled"""""
    try:
        from quantum_brain import get_quantum_brain

        brain = await get_quantum_brain(state.db)
        brain.llm_enabled = enabled

        mode = "polisher" if enabled else "raw"
        return {
            "success": True,
            "llm_enabled": enabled,
            "mode": f"Quantum Brain with {'LLM grammar polishing' if enabled else 'raw output (no LLM)'}",
            "message": f"LLM {'enabled as grammar polisher' if enabled else 'disabled - using pure Quantum Brain output'}"
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/brain/learn-word")
async def teach_brain_word(word: str, definition: str, part_of_speech: str = "unknown"):
```

```

    """Teach the Quantum Brain a new word"""
    try:
        from quantum_brain import get_quantum_brain

        brain = await get_quantum_brain(state.db)
        brain.vocabulary.add_word(word, definition, part_of_speech)

        return {
            "success": True,
            "word": word,
            "definition": definition,
            "message": f"Quantum Brain learned new word: {word}"
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/brain/learn-concept")
async def teach_brain_concept(concept: str, meaning: str):
    """Teach the Quantum Brain a new concept"""
    try:
        from quantum_brain import get_quantum_brain

        brain = await get_quantum_brain(state.db)
        brain.vocabulary.add_concept(concept, meaning)

        return {
            "success": True,
            "concept": concept,
            "meaning": meaning,
            "message": f"Quantum Brain learned new concept: {concept}"
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# =====
# QUANTUM GATE CONTROL
# =====

@router.post("/quantum/gate")
async def apply_quantum_gate(gate: str):
    """Apply a quantum gate (hadamard, pauli_x, pauli_z, measure)"""
    try:
        from quantum_brain import get_quantum_brain

        brain = await get_quantum_brain(state.db)
        result = brain.apply_gate(gate)

        return {
            "success": True,
            "gate_applied": gate,
            "quantum_state": result
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/quantum/state")
async def get_quantum_state():
    """Get current quantum state"""
    try:
        from quantum_brain import get_quantum_brain

        brain = await get_quantum_brain(state.db)

        return {
            "quantum_state": brain.quantum_state.to_dict(),
            "pennylane_available": brain.pennylane is not None
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/quantum/qram")
async def get_qram_status():
    """Get QRAM (Quantum Random Access Memory) status"""
    try:
        from quantum_memory import get_quantum_memory

        qram = get_quantum_memory()
        return qram.status()
    except ImportError:
        return {
            "backend": "unavailable",
            "pennylane_available": False,
            "qram_available": False,
            "entries_loaded": 0,
            "bit_width": 0,
            "max_entries": 0,
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/quantum/qram/load")
async def qram_load_concepts():
    """Load current knowledge graph concepts into QRAM"""
    try:
        from quantum_memory import get_quantum_memory

        brain = None

```

```

    if state.db:
        from quantum_brain import get_quantum_brain
        brain = await get_quantum_brain(state.db)

    qram = get_quantum_memory()

    # Gather concept names from brain or return empty
    concept_names: list = []
    if brain and hasattr(brain, '"knowledge"'):
        topics = brain.knowledge.get_all_topics() if hasattr(brain.knowledge, '"get_all_topics"') else []
        concept_names = list(topics) if topics else []

    if not concept_names:
        return {'"loaded"': 0, '"message"': '"No concepts available to load"'}

    count = qram.load_concepts(concept_names)
    return {'"loaded"': count, '"status"': qram.status()}
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))

@router.get('"/quantum/qram/query/{address}"')
async def qram_query(address: int):
    '"""Query a single QRAM address.""""
    try:
        from quantum_memory import get_quantum_memory
        qram = get_quantum_memory()
        result = qram.query(address)
        return {'"address"': address, '"concept"': result}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post('"/quantum/qram/superposition"')
async def qram_superposition_query(addresses: List[int]):
    '"""Query multiple QRAM addresses in superposition.""""
    try:
        from quantum_memory import get_quantum_memory
        qram = get_quantum_memory()
        results = qram.superposition_query(addresses)
        s = qram.status()
        return {
            '"backend"': s['"backend"'],
            '"quantum"': s.get('"qram_available"', False),
            '"results"': [{'"concept"': name, '"probability"': prob} for name, prob in results],
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get('"/quantum/qram/search/{term}"')
async def qram_search(term: str):
    '"""Search QRAM for concepts matching a term.""""
    try:
        from quantum_memory import get_quantum_memory
        qram = get_quantum_memory()
        s = qram.status()
        matches = []
        for addr in range(s['"entries_loaded"']):
            name = qram.query(addr)
            if name and term.lower() in name.lower():
                matches.append({'"address"': addr, '"concept"': name})
        return {'"term"': term, '"matches"': matches, '"count"': len(matches)}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# =====
# WOLFRAM ALPHA
# =====

@router.post('"/wolfram/query"')
async def wolfram_query(query: str):
    '"""Query Wolfram Alpha for mathematical/scientific answers.""""
    try:
        pass # wolfram removed

        wolfram = get_wolfram_engine()
        result = wolfram.query(query)

        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# =====
# SELF-RESEARCH
# =====

@router.post('"/research/query"')
async def research_query(query: str, reason: str = '"user_request"'):
    '"""Perform autonomous research on a topic.""""
    try:
        # Direct DDGS test
        from duckduckgo_search import DDGS
        ddgs = DDGS()
        direct_results = list(ddgs.text(query, max_results=3))
        logger.info(f'"[RESEARCH DEBUG] Direct DDGS returned {len(direct_results)} results for '{query}'"')

        from self_research import get_research_engine
        engine = await get_research_engine(state.db)

```

```

        result = await engine.research(query, reason)

        # If engine returned 0 but direct worked, use direct results
        if len(result.get('results', [])) == 0 and len(direct_results) > 0:
            logger.info(f"[RESEARCH DEBUG] Using direct results")
            result['results'] = [{ 'title': r.get('title', ''), 'body': r.get('body', ''), 'href': r.get('href', '') } for r in direct_results]
            # Re-extract concepts
            all_text = ' '.join([r.get('body', '') for r in direct_results])
            words = [w.lower() for w in all_text.split() if len(w) > 4]
            result['new_concepts'] = list(set(words))[:10]

        return result
    except Exception as e:
        logger.error(f"[RESEARCH ERROR] {str(e)}")
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/research/stats")
async def get_research_stats():
    """&quot;&quot;Get research statistics"&quot;&quot;&quot;
    try:
        from self_research import get_research_engine

        engine = await get_research_engine(state.db)
        return engine.get_research_stats()
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# =====
@router.post("/research/autonomous/start")
async def start_autonomous_research(request: AutonomousResearchRequest):
    """&quot;&quot;Start autonomous research session (30-60 minutes)&quot;&quot;&quot;&quot;
    try:
        from self_research import get_research_engine

        engine = await get_research_engine(state.db)
        # Verify db is passed and write a test entry
        logger.info(f"[RESEARCH] Starting autonomous research, db={state.db is not None}&quot;)

        # Direct test write
        try:
            await state.db.research_test.insert_one({
                'test': 'autonomous_start',
                'timestamp': datetime.now(timezone.utc).isoformat()
            })
            logger.info("[RESEARCH] Test write to DB successful")
        except Exception as write_err:
            logger.error(f"[RESEARCH] Test write failed: {write_err}&quot;)

        result = await engine.start_autonomous_research(request.duration_minutes)

        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/research/autonomous/stop")
async def stop_autonomous_research():
    """&quot;&quot;Stop autonomous research session"&quot;&quot;&quot;
    try:
        from self_research import get_research_engine

        engine = await get_research_engine(state.db)
        result = engine.stop_autonomous_research()

        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/research/autonomous/progress")
async def get_autonomous_research_progress():
    """&quot;&quot;Get autonomous research progress"&quot;&quot;&quot;
    try:
        from self_research import get_research_engine

        engine = await get_research_engine(state.db)
        return engine.get_autonomous_research_progress()
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# =====
# SELF-EVOLUTION
# =====

@router.get("/evolution/status")
async def get_evolution_status():
    """&quot;&quot;Get self-evolution status"&quot;&quot;&quot;
    try:
        from self_evolution_core import get_evolution_engine

        engine = get_evolution_engine(state.db)
        return engine.get_evolution_status()
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/evolution/trigger")
async def trigger_evolution():

```

```

"""Manually trigger self-evolution (user-initiated)"""
try:
    from self_evolution_core import get_evolution_engine

    engine = get_evolution_engine(state.db)
    result = await engine.auto_evolve()

    return {
        "success": True,
        "evolution_result": result
    }
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))

@router.get("/evolution/analyze/{filename}"):
async def analyze_code_file(filename: str):
    """Analyze a code file for potential improvements"""
    try:
        from self_evolution_core import get_evolution_engine

        engine = get_evolution_engine(state.db)
        analysis = engine.read_own_code(filename)
        improvements = engine.identify_improvements(filename)

        # Strip raw code from response to keep it lightweight
        if 'code' in analysis:
            del analysis['code']

        return {
            "analysis": analysis,
            "improvements": improvements,
            "improvement_count": len(improvements)
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/evolution/analyze-all"):
async def analyze_all_code():
    """Analyze all modifiable code files"""
    try:
        from self_evolution_core import get_evolution_engine

        engine = get_evolution_engine(state.db)
        all_analyses = engine.analyze_all_code()

        # Collect all improvements
        all_improvements = []
        for filename in engine.modifiable_files:
            improvements = engine.identify_improvements(filename)
            for imp in improvements:
                imp['file'] = filename
                all_improvements.append(imp)

        # Sort by priority
        all_improvements.sort(key=lambda x: x.get('priority', 99))

        # Strip raw code from response
        for key, analysis in all_analyses.get('analyses', {}).items():
            if 'code' in analysis:
                del analysis['code']

        return {
            "summary": {
                "files_analyzed": all_analyses['files_analyzed'],
                "total_lines": all_analyses['total_lines'],
                "total_classes": all_analyses['total_classes'],
                "total_functions": all_analyses['total_functions'],
                "total_improvements": len(all_improvements)
            },
            "improvements": all_improvements,
            "files": {k: v for k, v in all_analyses.get('analyses', {}).items() if 'error' not in v}
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

```

50. routes_chat.py

1,205 lines | 48.6KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/routes_chat.py

```
"""""
■ Chat Routes
=====
Chat endpoints, session management, and command processing.
"""""

from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from typing import List, Dict, Optional, Any
import uuid
from engine_api import get_api
import os
import logging
import re
import time
from datetime import datetime, timezone

import shared_state as state
from thinking_commands import ThinkingModeCommands
from chat_helpers import (
    get_memory_summary, do_deep_research, process_shared_link,
    list_all_documents, generate_speech, get_help_text,
    analyze_book_characters, analyze_book_timeline,
    analyze_book_worldbuilding, analyze_book_feedback
)
from chat_models import get_chat_store

logger = logging.getLogger("quantum_ai")
router = APIRouter(prefix="/api")
```

```
def _build_command_explanation(command: str, result: str) -> Dict:
    """""Build explanation for a slash command execution."""""
    return {
        "reasoning_path": [f"Detected command: {command}", "Parse arguments", "Execute", "Return result"],
        "confidence_score": 1.0,
        "engines_used": ["CommandInterface"],
        "steps": [
            {"step": "Command detection", "engine": "CommandInterface", "confidence_contribution": 0.3},
            {"step": f"Execute {command}", "engine": "CommandInterface", "confidence_contribution": 0.7}
        ],
        "summary": f"Command {command} executed successfully.",
        "timestamp": datetime.now(timezone.utc).isoformat()
    }
```

```
async def _handle_command(content: str, explain_mode: bool = False) -> Optional["ChatResponse"]:
    """""
    Handle slash commands. Returns ChatResponse if content is a command, else None.
    """""
    content = content.strip()
    if not content.startswith('/'):
        return None

    parts = content.split()
    cmd = parts[0].lower()
    args = parts[1:] if len(parts) > 1 else []
    session_id = str(uuid.uuid4())[8] # dummy for response; real session_id comes from message

    response_text = ""
    explanation = None

    try:
        if cmd == '/explain' and args:
            # Force explain mode for the given query
            # Treat the rest as a normal chat with explain_mode=True
            # We'll short-circuit: call hidden_thinking directly with query
            combined = ' '.join(args)
            # Need context and history? None for simplicity.
            response = get_api().process_input(
                combined,
                context=None,
                conversation_history=[],
                explain_mode=True
            )
            response_text = response.get("response", str(response))
            explanation = result.get("explanation")
            # Also append note that command was used
            response_text = f"[Command: /explain]" + response_text

        if explain_mode:
            # Already explained
            pass
        elif cmd == '/explain' and not args:
            response_text = "Usage: /explain <your question>"
        elif cmd == '/status':
            import platform
            uptime_seconds = time.time() - getattr(state, 'start_time', time.time())
            status_lines = [
                "Quantum AI System Status",
                f" Cognitive Core: {'█' if state.cognitive_core else '██'}",
                f" Universal Explorer: {'█' if state.universal_explorer else '██'}",
                f" Hidden Thinking: {'█' if state.hidden_thinking else '██'}",
            ]
```

```

        f&quot; Uptime: {int(uptime_seconds)}s&quot;;
        f&quot; Platform: {platform.system()} {platform.release()}&quot;;
    ]
    response_text = &quot;
    &quot;.join(status_lines)

elif cmd == '/save':
    # The system auto-saves on every message; just acknowledge.
    response_text = &quot;State automatically saved to local storage.&quot;
elif cmd in ('/cloud-save', '/cloud-status', '/cloud-load'):
    try:
        pass # wolfram removed
        # But these functions expect LocalMemory object; server uses different storage.
        response_text = f&quot; Wolfram Cloud command '{cmd}' is only available in local chat mode (chat.py).&quot;
    except ImportError:
        response_text = &quot; Wolfram Cloud not available.&quot;
elif cmd == '/knowledge' and args:
    topic = ' '.join(args)
    try:
        from knowledge_base import get_knowledge_base
        kb = get_knowledge_base()
        explanation_text = kb.get_topic_explanation(topic)
        if explanation_text:
            response_text = explanation_text
        else:
            response_text = f&quot; No knowledge available on: {topic}&quot;
    except Exception as e:
        response_text = f&quot; Knowledge lookup error: {e}&quot;
elif cmd == '/analyze' and args:
    text = ' '.join(args)
    try:
        from text_analyzer import get_text_analyzer
        analyzer = get_text_analyzer()
        analysis = analyzer.analyze(text)
        lines = [
            &quot; Text Analysis:&quot;;
            f&quot; Sentiment: {analysis.get('sentiment', 'unknown')}&quot;;
            f&quot; Complexity: {analysis.get('complexity', 'unknown')}&quot;;
            f&quot; Topics: {' '.join(analysis.get('topics', []))}&quot;;
            f&quot; Word count: {analysis.get('word_count', 0)}&quot;;
        ]
        response_text = &quot;
        &quot;.join(lines)

    except Exception as e:
        response_text = f&quot; Analysis error: {e}&quot;
elif cmd == '/personality':
    try:
        from personality_engine import get_personality_engine
        personality = get_personality_engine()
        response_text = personality.get_personality_summary()
    except Exception as e:
        response_text = f&quot; Personality error: {e}&quot;
elif cmd == '/research':
    if not args:
        response_text = &quot; Usage: /research &lt;topic> -&lt;depth>
        Example: /research Evolution of Religion -30&quot;
    else:
        depth = 10
        topic_parts = []
        for a in args:
            if a.startswith('-') and a[1:].isdigit():
                depth = int(a[1:])
            else:
                topic_parts.append(a)
        topic = ' '.join(topic_parts)
        try:
            research_prompt = f&quot; Provide {depth} detailed, numbered research points about: {topic}. Each point should be substantive and informative.&quot;
            response = get_api().process_input(
                research_prompt,
                context={&quot; mode:&quot;; &quot; research:&quot;; &quot; depth:&quot;; depth, &quot; topic:&quot;; topic},
                conversation_history=[],
                explain_mode=explain_mode
            )
            response_text = f&quot; ■ Research: {topic}
            &quot; + result[&quot; response:&quot;]

            if explain_mode:
                explanation = result.get(&quot; explanation:&quot;)
        except Exception as e:
            response_text = f&quot; Research error: {e}&quot;
elif cmd == '/cistercian-math':
    if not args:
        response_text = &quot; Usage: /cistercian-math &lt;expression>
        Example: /cistercian-math 42 + 73
        Supported: + - * / (numbers 0-9999)&quot;
    else:
        expr = ' '.join(args)
        match = re.match(r'^s*(\d+)\s*([+|-*/])\s*(\d+)\s*$', expr)
        if not match:
            response_text = &quot; Invalid expression. Use: NUMBER OP NUMBER
            Example: /cistercian-math 42 + 73&quot;
        else:
            a_val = int(match.group(1))
            op = match.group(2)
            b_val = int(match.group(3))
            if not (0 &lt;= a_val &lt;= 9999) or not (0 &lt;= b_val &lt;= 9999):
                response_text = &quot; Numbers must be 0-9999 (Cistercian range)&quot;

```

```

        elif op == '/' and b_val == 0:
            response_text = "Division by zero"
        else:
            if op == '+': result_val = a_val + b_val
            elif op == '-': result_val = a_val - b_val
            elif op == '*': result_val = a_val * b_val
            else: result_val = int(a_val / b_val)
            clamped = max(0, min(9999, result_val))
            overflow = "(clamped to Cistercian range)" if result_val != clamped else ""
            response_text = (
                f"■■ Cistercian Math:
                "

                f" ■({a_val}) {op} ■({b_val}) = ■({clamped}){overflow}
                "

                f" Arabic: {a_val} {op} {b_val} = {result_val}
                "

                f"View the Cistercian numerals at:
                "

                f" /api/cistercian/numeral?number={a_val}
                "

                f" /api/cistercian/numeral?number={b_val}
                "

                f" /api/cistercian/numeral?number={clamped}&quot;
            )
    else:
        response_text = f"Unknown command: {cmd}. Supported: /explain, /status, /save, /cloud-save, /cloud-status, /cloud-load, /knowledge <topic>, /analyze

if explain_mode:
    explanation = _build_command_explanation(cmd, response_text)

return ChatResponse(
    questions=[],
    response=response_text,
    understanding={"command": cmd},
    concepts=[],
    session_id=session_id,
    explanation=explanation
)
except Exception as e:
    logger.error(f"Command error ({cmd}): {e}&quot;)
    return ChatResponse(
        questions=[],
        response=f"Error executing command: {e}&quot;,
        understanding={"command": cmd, "error": True},
        concepts=[],
        session_id=str(uuid.uuid4())[0:8],
        explanation=None
    )

class ChatMessage(BaseModel):
    """ChatMessage - Auto-documented by self-evolution.&quot;&quot;&quot;
    content: str
    mode: Optional[str] = None # 'quantum' or 'algorithmic'
    context: Optional[Dict] = None
    session_id: Optional[str] = None
    files: Optional[List[Dict[str, Any]]] = None # List of {filename, size}
    explain_mode: Optional[bool] = False # If True, return detailed reasoning trace
    structured_response: Optional[str] = None # Frontend-generated structured response for Markov weaving

class ChatResponse(BaseModel):
    """ChatResponse - Auto-documented by self-evolution.&quot;&quot;&quot;
    questions: List[str]
    response: str
    understanding: Dict
    concepts: List[str]
    session_id: Optional[str] = None
    explanation: Optional[Dict] = None # Explanation trace when explain_mode=True

@router.post("/quantum/chat", response_model=ChatResponse)
async def quantum_chat(message: ChatMessage):
    ""&quot;
    ARTICLE 1.1 - Prime Directive of Questioning (HIDDEN MODE)

    Process user input with hidden thinking mode.
    AI thinks, questions, researches internally - user sees clean conversation.
    Supports conversation history via session_id.
    &quot;&quot;&quot;
    try:
        # Generate or use provided session ID
        session_id = message.session_id or str(uuid.uuid4())

        # Reset activity timer (user is interacting)
        state.last_activity_time = datetime.now(timezone.utc)

        # Wake from dream if dreaming
        try:
            from dream_state import get_dream_engine
            dream_engine = get_dream_engine(state.db)
            await dream_engine.mark_active()
        except:
            pass

        # Check for thinking mode commands

```

```

command = ThinkingModeCommands.parse_command(message.content)

if command:
    if command[&quot;command&quot;] == &quot;thinking_on&quot;:
        state.hidden_thinking.toggle_thinking_display(True)
        return ChatResponse(
            questions=[],
            response=&quot;■ Thinking mode display: ON. You'll now see my internal process.&quot;,
            understanding={},
            concepts=[],
            session_id=session_id
        )
    elif command[&quot;command&quot;] == &quot;thinking_off&quot;:
        state.hidden_thinking.toggle_thinking_display(False)
        return ChatResponse(
            questions=[],
            response=&quot;■ Thinking mode display: OFF. Clean conversation mode.&quot;,
            understanding={},
            concepts=[],
            session_id=session_id
        )
    elif command[&quot;command&quot;] == &quot;thinking_status&quot;:
        status = &quot;ON (visible)&quot; if state.hidden_thinking.show_thinking else &quot;OFF (hidden)&quot;
        return ChatResponse(
            questions=[],
            response=f&quot;■ Current thinking mode: {status}&quot;,
            understanding={},
            concepts=[],
            session_id=session_id
        )
    elif command[&quot;command&quot;] == &quot;read_file&quot;:
        # Process uploaded file
        filename = command[&quot;filename&quot;] or &quot;last uploaded file&quot;

        # Get list of uploaded files
        import os
        upload_dir = &quot;/app/uploads&quot;
        if os.path.exists(upload_dir):
            files = [f for f in os.listdir(upload_dir) if os.path.isfile(os.path.join(upload_dir, f))]

            if files:
                # Use specified file or most recent
                if command[&quot;filename&quot;]:
                    target_file = command[&quot;filename&quot;]
                else:
                    target_file = files[-1] # Most recent

                file_path = os.path.join(upload_dir, target_file)

                # Process the document
                result = await state.universal_explorer.documents.process_document(file_path)

                if &quot;error&quot; not in result:
                    # Extract and summarize
                    text = result.get(&quot;text&quot;, &quot;&quot;)[:500]
                    response = f&quot;■ Read {target_file}:
                    I've processed this document. {text[:200]}...
                    What would you like to know about it?&quot;

                else:
                    response = f&quot;■ Couldn't read {target_file}: {result['error']}&quot;
            else:
                response = &quot;No files uploaded yet. Use the Explorer tab to upload files.&quot;
        else:
            response = &quot;Upload directory not found.&quot;

        return ChatResponse(
            questions=[],
            response=response,
            understanding={},
            concepts=[],
            session_id=session_id
        )

    elif command[&quot;command&quot;] == &quot;memory&quot;:
        # Show what AI has learned
        response = await get_memory_summary(session_id)
        return ChatResponse(
            questions=[],
            response=response,
            understanding={},
            concepts=[],
            session_id=session_id
        )

    elif command[&quot;command&quot;] == &quot;research&quot;:
        # Deep research on a topic
        topic = command.get(&quot;topic&quot;, &quot;&quot;)
        if not topic:
            return ChatResponse(
                questions=[],
                response=&quot;■ Usage: `!research &lt;topic>`
                Example: `!research quantum entanglement`&quot;,
                understanding={},
                concepts=[],
                session_id=session_id
            )

        response = await do_deep_research(topic)
        return ChatResponse(

```

```

        questions=[],
        response=response,
        understanding={},
        concepts=[],
        session_id=session_id
    )

elif command[&quot;command&quot;] == &quot;link&quot;;
# Download file from URL
url = command.get(&quot;url&quot;;, &quot;&quot;)
if not url:
    return ChatResponse(
        questions=[],
        response=&quot;■ Usage: `!link &lt;url&gt;`
        Supported:
        • Google Drive links
        • Dropbox links
        • OneDrive links
        • Direct file URLs
        Example: `!link https://drive.google.com/file/d/...`&quot;;,

        understanding={},
        concepts=[],
        session_id=session_id
    )

    response = await process_shared_link(url)
    return ChatResponse(
        questions=[],
        response=response,
        understanding={},
        concepts=[],
        session_id=session_id
    )

elif command[&quot;command&quot;] == &quot;docs&quot;;
# List all documents
response = await list_all_documents()
return ChatResponse(
    questions=[],
    response=response,
    understanding={},
    concepts=[],
    session_id=session_id
)

elif command[&quot;command&quot;] == &quot;speak&quot;;
# Text to speech
text = command.get(&quot;text&quot;;, &quot;&quot;)
if not text:
    return ChatResponse(
        questions=[],
        response=&quot;■ Usage: `!speak &lt;text&gt;`
        Example: `!speak Hello, I am Quantum AI`
        Or just chat normally and use the speaker button on my responses!&quot;;,

        understanding={},
        concepts=[],
        session_id=session_id
    )

# Generate TTS
audio_url = await generate_speech(text)
response = f&quot;■ **Speech Generated**
&quot;{text[:100]}{'...' if len(text) &gt; 100 else ''}&quot;
[Audio URL: {audio_url}]&quot;; if audio_url else f&quot;■ Text to speak:
&quot;{text}&quot;;
_Use your browser's text-to-speech or copy this text._&quot;;

    return ChatResponse(
        questions=[],
        response=response,
        understanding={},
        concepts=[],
        session_id=session_id
    )

elif command[&quot;command&quot;] == &quot;imagine&quot;;
# Generate image
prompt = command.get(&quot;prompt&quot;;, &quot;&quot;)
if not prompt:
    return ChatResponse(
        questions=[],
        response=&quot;■ Usage: `!imagine &lt;description&gt;`
        Example: `!imagine a cosmic landscape with nebulas and stars`&quot;;,

        understanding={},
        concepts=[],
        session_id=session_id
    )

try:
    raise NotImplementedError(&quot;OpenAI image generation removed – this project uses NO LLM&quot;;)
except NotImplementedError as e:
    response = f&quot;■ Image generation error: {str(e)}&quot;;

    return ChatResponse(
        questions=[],
        response=response,
        understanding={},
        concepts=[],
    )

```

```

        session_id=session_id
    )
elif command["command"] == "export":
    # Export chat or memory
    target = command.get("target", "chat")
    if target == "memory":
        response = """
Export Memory
Use the API endpoint: `GET /api/memory/export`
Or click the Export button in the Memory screen.
        """
    else:
        response = (
            f"""
Export Conversation for Claude

Option 1: Tap the Share w/ Claude button in the top bar – copies this chat as markdown to your clipboard.

Option 2: API endpoint: `GET /api/chat/export/{session_id}`

Option 3: Export recent sessions: `GET /api/chat/export-current`

Then paste into Claude, and it'll have full context of our conversation.
            """
        )

    return ChatResponse(
        questions=[],
        response=response,
        understanding={},
        concepts=[],
        session_id=session_id
    )

elif command["command"] == "graph":
    # Knowledge graph info
    response = """
Knowledge Graph
View your knowledge graph visualization in the Memory screen.
The graph shows:


- Concepts you've learned (nodes)
- Relationships between concepts (edges)
- Insights (special nodes)


API: `GET /api/knowledge/graph`
    """

    return ChatResponse(
        questions=[],
        response=response,
        understanding={},
        concepts=[],
        session_id=session_id
    )

elif command["command"] == "characters":
    # Extract characters from document
    response = await analyze_book_characters()
    return ChatResponse(
        questions=[],
        response=response,
        understanding={},
        concepts=[],
        session_id=session_id
    )

elif command["command"] == "timeline":
    # Generate timeline
    response = await analyze_book_timeline()
    return ChatResponse(
        questions=[],
        response=response,
        understanding={},
        concepts=[],
        session_id=session_id
    )

elif command["command"] == "worldbuilding":
    # Extract world details
    response = await analyze_book_worldbuilding()
    return ChatResponse(
        questions=[],
        response=response,
        understanding={},
        concepts=[],
        session_id=session_id
    )

elif command["command"] == "feedback":
    # Writing feedback
    response = await analyze_book_feedback()
    return ChatResponse(
        questions=[],
        response=response,
        understanding={},
        concepts=[],
        session_id=session_id
    )

elif command["command"] == "help":
    # Show all commands
    response = get_help_text()
    return ChatResponse(

```

```

        questions=[],
        response=response,
        understanding={},
        concepts=[],
        session_id=session_id
    )

# --- Slash commands (compatible with chat.py) ---
elif command[&quot;command&quot;] == &quot;cloud_save&quot;:
    try:
        pass # wolfram removed
        # cloud_save expects LocalMemory, not available here
        response = &quot;Wolfram Cloud save is only available in local chat mode (run `python chat.py`).&quot;
    except ImportError:
        response = &quot;Wolfram Cloud not available.&quot;
    return ChatResponse(
        questions=[],
        response=response,
        understanding={&quot;command&quot;: &quot;cloud_save&quot;},
        concepts=[],
        session_id=session_id
    )

elif command[&quot;command&quot;] == &quot;cloud_status&quot;:
    try:
        pass # wolfram removed
        status = cloud_status()
        response = f&quot;Wolfram Cloud status: {status}&quot;
    except ImportError:
        response = &quot;Wolfram Cloud not available.&quot;
    return ChatResponse(
        questions=[],
        response=response,
        understanding={&quot;command&quot;: &quot;cloud_status&quot;},
        concepts=[],
        session_id=session_id
    )

elif command[&quot;command&quot;] == &quot;cloud_load&quot;:
    try:
        pass # wolfram removed
        data = cloud_load()
        response = f&quot;Cloud data loaded: {data}&quot; if data else &quot;No data in cloud.&quot;
    except ImportError:
        response = &quot;Wolfram Cloud not available.&quot;
    return ChatResponse(
        questions=[],
        response=response,
        understanding={&quot;command&quot;: &quot;cloud_load&quot;},
        concepts=[],
        session_id=session_id
    )

elif command[&quot;command&quot;] == &quot;status&quot;:
    import platform
    uptime_seconds = getattr(state, 'start_time', None)
    if uptime_seconds is None:
        uptime = &quot;unknown&quot;
    else:
        uptime = f&quot;{int(time.time() - state.start_time)}s&quot;
    lines = [
        &quot;Quantum AI System Status:&quot;,
        f&quot; Cognitive Core: {'█' if state.cognitive_core else '█'}&quot;,
        f&quot; Universal Explorer: {'█' if state.universal_explorer else '█'}&quot;,
        f&quot; Hidden Thinking: {'█' if state.hidden_thinking else '█'}&quot;,
        f&quot; Uptime: {uptime}&quot;,
        f&quot; Platform: {platform.system()} {platform.release()}&quot;,
    ]
    response = &quot;
&quot;.join(lines)

    return ChatResponse(
        questions=[],
        response=response,
        understanding={&quot;command&quot;: &quot;status&quot;},
        concepts=[],
        session_id=session_id
    )

elif command[&quot;command&quot;] == &quot;save&quot;:
    response = &quot;State automatically saved to local storage after each message.&quot;
    return ChatResponse(
        questions=[],
        response=response,
        understanding={&quot;command&quot;: &quot;save&quot;},
        concepts=[],
        session_id=session_id
    )

elif command[&quot;command&quot;] == &quot;explain_help&quot;:
    response = &quot;Use /explain &lt;your question> to get a detailed reasoning trace with PennyLane quantum processing.
    Example: `/explain What is quantum superposition?`&quot;

    return ChatResponse(
        questions=[],
        response=response,
        understanding={&quot;command&quot;: &quot;explain_help&quot;},
        concepts=[],
        session_id=session_id
    )

elif command[&quot;command&quot;] == &quot;explain_query&quot;:

```

```

query = command.get(&quot;query&quot;;, &quot;&quot;);
if not query:
    response = &quot;Usage: /explain &lt;your question&gt;&quot;;
    return ChatResponse(questions=[], response=response, understanding={&quot;command&quot;: &quot;explain_help&quot;}, concepts=[], session_id=session_id)
# Direct to master brain (exact same as terminal chat.py)
response = get_api().process_input(message.content)
conversation_history=[],
explain_mode=True
)
response = result[&quot;response&quot;];
explanation = result.get(&quot;explanation&quot;);
return ChatResponse(
    questions=result.get(&quot;internal_questions&quot;;, []),
    response=response,
    understanding={&quot;command&quot;: &quot;explain&quot;}, result[&quot;confidence&quot;], &quot;research_done&quot;: result[&quot;research_done&quot;]},
    concepts=result.get(&quot;concepts&quot;;, []),
    session_id=session_id,
    explanation=explanation
)

elif command[&quot;command&quot;] == &quot;knowledge&quot;:
    topic = command.get(&quot;topic&quot;;, &quot;&quot;);
    if not topic:
        response = &quot;Usage: /knowledge &lt;topic&gt;&quot;;
    else:
        try:
            from knowledge_base import get_knowledge_base
            kb = get_knowledge_base()
            explanation = kb.get_topic_explanation(topic)
            response = explanation if explanation else f&quot;No knowledge available on: {topic}&quot;;
        except Exception as e:
            response = f&quot;Knowledge lookup error: {e}&quot;;
    return ChatResponse(
        questions=[],
        response=response,
        understanding={&quot;command&quot;: &quot;knowledge&quot;}, &quot;topic&quot;: topic,
        concepts=[],
        session_id=session_id
    )

elif command[&quot;command&quot;] == &quot;analyze&quot;:
    text = command.get(&quot;text&quot;;, &quot;&quot;);
    if not text:
        response = &quot;Usage: /analyze &lt;text&gt;&quot;;
    else:
        try:
            from text_analyzer import get_text_analyzer
            analyzer = get_text_analyzer()
            analysis = analyzer.analyze(text)
            lines = [
                &quot;Text Analysis:&quot;;,
                f&quot; Sentiment: {analysis.get('sentiment', 'unknown')}&quot;;,
                f&quot; Complexity: {analysis.get('complexity', 'unknown')}&quot;;,
                f&quot; Topics: {'', '.join(analysis.get('topics', []))}&quot;;,
                f&quot; Word count: {analysis.get('word_count', 0)}&quot;;
            ]
            response = &quot;
&quot;.join(lines)
        except Exception as e:
            response = f&quot;Analysis error: {e}&quot;;
    return ChatResponse(
        questions=[],
        response=response,
        understanding={&quot;command&quot;: &quot;analyze&quot;},
        concepts=[],
        session_id=session_id
    )

elif command[&quot;command&quot;] == &quot;personality&quot;:
    try:
        from personality_engine import get_personality_engine
        personality = get_personality_engine()
        response = personality.get_personality_summary()
    except Exception as e:
        response = f&quot;Personality error: {e}&quot;;
    return ChatResponse(
        questions=[],
        response=response,
        understanding={&quot;command&quot;: &quot;personality&quot;},
        concepts=[],
        session_id=session_id
    )

elif command[&quot;command&quot;] == &quot;collapse&quot;:
    text = command.get(&quot;text&quot;;, &quot;&quot;);
    if not text:
        response = &quot;Usage: /collapse &lt;text&gt;&quot;;
    else:
        try:
            # Use master brain (engine_api.py)
            api = get_api()
            result = api.process_input(text)
            response = result.get(&quot;response&quot;;, str(result)) if isinstance(result, dict) else str(result)
        return ChatResponse(
            questions=[],
            response=response,
            understanding={&quot;command&quot;: &quot;collapse&quot;},
            concepts=[],
            session_id=session_id
        )

```

```

# End of slash commands

# Get or create conversation in ChatStore
store = get_chat_store()
conv = store.get_conversation(session_id)
if not conv:
    # Create new conversation with first user message as title
    title = message.content[:50] + ('&quot;...'&quot; if len(message.content) > 50 else '&quot;&quot;')
    conv = store.create_conversation(user_id='&quot;default&quot;', title=title)
    # The create_conversation returns new conv with id, but we need to set the id to session_id?
    # ChatStore creates its own ID. We need to ensure it matches session_id.
    # Workaround: we'll just add messages to a new conv and then later we need to map session_id to conv.id
    # Note: ChatStore uses local file storage; session_id handling may be simplified.
    pass

# Get conversation history for context from local file storage
history = await get_conversation_history(session_id, limit=10)

# =====
# MATH DETECTION - intercept arithmetic before Markov chain
# Detects patterns like '&quot;50 - 20&quot;', '&quot;50-20=&quot;', '&quot;what is 42+73&quot;'
# Computes actual answer and shows Cistercian numeral SVGs
# =====
content_stripped = message.content.strip()[:60] # limit length to prevent ReDoS
math_match = re.match(
    r'(?:(?:what is |calculate |compute |solve )?)' # optional prefix (no \s+)
    r'(\d{1,5}) ?([+*-/x%]) ?(\d{1,5})' # NUMBER OP NUMBER (max 1 space)
    r' ?(?:=|<|>)?$', # optional trailing = or ?
    content_stripped, re.IGNORECASE
)
if math_match:
    a_val = int(math_match.group(1))
    op_raw = math_match.group(2)
    b_val = int(math_match.group(3))
    # Normalize operator
    op = {'+': '+', '-': '-', '*': '*', '/': '/', 'x': '*', '÷': '/'}[op_raw]
    if not op:
        op = op_raw # fallback
    op_display = {'+': '+', '-': '-', '*': 'x', '/': '÷'}[op]

    # Compute
    error = None
    result_val = 0
    if op == '/' and b_val == 0:
        error = '&quot;Division by zero is undefined.&quot;'
    else:
        if op == '+': result_val = a_val + b_val
        elif op == '-': result_val = a_val - b_val
        elif op == '*': result_val = a_val * b_val
        elif op == '/': result_val = a_val // b_val

    if error:
        return ChatResponse(
            questions=[], response=f'&quot;■ {error}&quot;;',
            understanding={'&quot;type&quot;': '&quot;math&quot;', '&quot;expression&quot;': f'&quot;{a_val} {op} {b_val}&quot;', '&quot;result&quot;': result_val},
            concepts=['&quot;arithmetic&quot;'], session_id=session_id
        )

    # Build response with Cistercian context
    cistercian_range = (0 &lt;= a_val &lt;= 9999 and 0 &lt;= b_val &lt;= 9999)
    clamped = max(0, min(9999, result_val))
    overflow_note = '&quot;&quot;'
    if cistercian_range and result_val != clamped:
        overflow_note = f'&quot;■ Result clamped to Cistercian range (0-9999)&quot;'

    lines = [
        f'&quot;■ {a_val} {op_display} {b_val} = {result_val}&quot;;',
    ]

    if cistercian_range:
        lines.append(f'&quot;&quot;')
        lines.append(f'&quot;■ In Cistercian numerals:&quot;')
        lines.append(f'&quot; ■({a_val}) {op_display} ■({b_val}) = ■({clamped}){overflow_note}&quot;')
        lines.append(f'&quot;&quot;')
        lines.append(f'&quot;View the glyphs:&quot;')
        lines.append(f'&quot; /api/cistercian/numeral?number={a_val}&quot;')
        lines.append(f'&quot; /api/cistercian/numeral?number={b_val}&quot;')
        lines.append(f'&quot; /api/cistercian/numeral?number={clamped}&quot;')
    elif result_val > 9999 or a_val > 9999 or b_val > 9999:
        lines.append(f'&quot;')
        lines.append(f'&quot;(Numbers beyond 9999 exceed Cistercian range)&quot;')

    # Store in history
    await store_chat_message(session_id, '&quot;user&quot;', message.content)
    await store_chat_message(session_id, '&quot;assistant&quot;', '&quot;' + '&quot;'.join(lines))

    return ChatResponse(
        questions=[],
        response='&quot;' + '&quot;'.join(lines),
        understanding={'&quot;type&quot;': '&quot;math&quot;', '&quot;expression&quot;': f'&quot;{a_val} {op} {b_val}&quot;', '&quot;result&quot;': result_val},
        concepts=['&quot;arithmetic&quot;', '&quot;cistercian_numerals&quot;'] if cistercian_range else ['&quot;arithmetic&quot;'],
        session_id=session_id
    )
# DEBUG: log structured_response

```

```

logger.info(f"<div>DEBUG: structured_response from message: {message.structured_response[:50] if message.structured_response else 'None'}</div>")

# Process with hidden thinking mode
response = get_api().process_input(
    message.content,
    message.context,
    conversation_history=history,
    explain_mode=message.explain_mode,
    structured_response=message.structured_response
)

# Store this exchange in history (local file storage)
await store_chat_message(session_id, <div>user</div>, message.content)
await store_chat_message(session_id, <div>assistant</div>, result[<div>response</div>])

# Format response with thinking log if enabled (legacy thinking display)
response_text = response.get(<div>response</div>, str(response))
if result.get(<div>thinking_log</div>) and state.hidden_thinking.show_thinking:
    thinking_display = <div>
        --- THINKING PROCESS ---
        <div> + </div>
        <div>.join(result[<div>thinking_log</div>])

    response_text = response_text + thinking_display

resp = result[<div>response</div>]
if isinstance(resp, tuple): resp = resp[0]
return ChatResponse(
    questions=[str(q) for q in result.get(<div>internal_questions</div>, [])] if state.hidden_thinking.show_thinking else [],
    response=resp,
    understanding={<div>confidence</div>: result[<div>confidence</div>], <div>research_done</div>: result[<div>research_done</div>]},
    concepts=result.get(<div>concepts</div>, []),
    session_id=session_id,
    explanation=result.get(<div>explanation</div>) if message.explain_mode else None
)

except Exception as e:
    import traceback
    logger.error(f"<div>Chat error: {str(e)}</div>")
    {traceback.format_exc()}<div></div>

# Fallback to basic mode
result = await state.cognitive_core.process_input(
    message.content,
    message.context,
)
return ChatResponse(
    questions=result[<div>questions</div>],
    response=result[<div>response</div>],
    understanding=result[<div>understanding</div>],
    concepts=result[<div>concepts</div>],
    session_id=message.session_id
)

# =====
# CONVERSATION HISTORY HELPERS
# =====

async def store_chat_message(session_id: str, role: str, content: str):
    <div><div></div></div><div>Store a chat message in local file storage</div><div></div></div>
    doc = {
        <div>session_id</div>: session_id,
        <div>role</div>: role,
        <div>content</div>: content,
        <div>timestamp</div>: datetime.now(timezone.utc).isoformat()
    }
    await state.db.chat_history.insert_one(doc)

async def get_conversation_history(session_id: str, limit: int = 10) -> List[Dict]:
    <div><div></div></div><div>Get recent conversation history for a session from local storage</div><div></div></div>
    cursor = state.db.chat_history.find(
        {<div>session_id</div>: session_id,
        {<div>_id</div>: 0, <div>role</div>: 1, <div>content</div>: 1, <div>timestamp</div>: 1}
    ).sort(<div>timestamp</div>, -1).limit(limit)

    messages = await cursor.to_list(length=limit)
    # Return in chronological order
    return list(reversed(messages))

@router.get(<div>/chat/history/{session_id}</div>)
async def get_chat_history(session_id: str, limit: int = 50):
    <div><div></div></div><div>Get conversation history for a session</div><div></div></div>
    try:
        history = await get_conversation_history(session_id, limit)
        return {<div>session_id</div>: session_id, <div>messages</div>: history, <div>count</div>: len(history)}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.delete(<div>/chat/history/{session_id}</div>)
async def clear_chat_history(session_id: str):
    <div><div></div></div><div>Clear conversation history for a session</div><div></div></div>
    try:
        result = await state.db.chat_history.delete_many({<div>session_id</div>: session_id})
        return {<div>session_id</div>: session_id, <div>deleted</div>: result.deleted_count}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
@router.get(<div>/chat/sessions</div>)

```

```

async def list_chat_sessions():
    """List all chat sessions"""
    try:
        # Fetch all messages (with limit for performance)
        cursor = state.db.chat_history.find(
            {},
            {'&id&': 0, '&session_id&': 1, '&timestamp&': 1}
        ).sort('&timestamp&', -1).limit(1000)
        messages = await cursor.to_list(length=1000)

        # Group by session_id
        sessions_dict = {}
        for msg in messages:
            sid = msg.get('&session_id&')
            if not sid:
                continue
            if sid not in sessions_dict:
                sessions_dict[sid] = {'&session_id&': sid, '&message_count&': 0, '&last_message&': None}
            sessions_dict[sid]['&message_count&'] += 1
            ts = msg.get('&timestamp&')
            if ts:
                if sessions_dict[sid]['&last_message&'] is None or ts > sessions_dict[sid]['&last_message&']:
                    sessions_dict[sid]['&last_message&'] = ts

        sessions = list(sessions_dict.values())
        # Sort by last_message descending (handle None)
        sessions.sort(key=lambda x: x['&last_message&'] or '&&', reverse=True)
        sessions = sessions[:20]

        return {'&sessions&': sessions}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# =====
# CHAT EXPORT – Format conversations for sharing with Claude/other AIs
# =====

@router.get('&/chat/export/{session_id}&')
async def export_chat_for_sharing(session_id: str, limit: int = 200, format: str = '&markdown&'):
    """
    Export a chat session as formatted text ready to paste into Claude or another AI.

    Formats:
    - markdown (default): Clean markdown with headers, timestamps, role labels
    - json: Raw JSON array of messages
    - text: Plain text with role prefixes
    """
    try:
        history = await get_conversation_history(session_id, limit)
        if not history:
            raise HTTPException(status_code=404, detail=f'No messages found for session {session_id}')

        if format == '&json&':
            return {'&session_id&': session_id, '&messages&': history, '&format&': '&json&'}

        # Build formatted export
        lines = []
        lines.append(f'# Quantum MCAGI – Chat Export')
        lines.append(f'**Session:** `{session_id}`')
        if history:
            lines.append(f'**Messages:** {len(history)}')
            first_ts = history[0].get('&timestamp&', '&&')
            last_ts = history[-1].get('&timestamp&', '&&')
            if first_ts:
                lines.append(f'**From:** {first_ts}')
            if last_ts:
                lines.append(f'**To:** {last_ts}')
            lines.append('&&')
            lines.append(f'---')
            lines.append('&&')

        for msg in history:
            role = msg.get('&role&', '&unknown&')
            content = msg.get('&content&', '&&')
            ts = msg.get('&timestamp&', '&&')

            if format == '&text&':
                prefix = '&USER&' if role == '&user&' else '&QUANTUM MCAGI&'
                lines.append(f'{prefix} [{prefix}] {content}')
                lines.append('&&')
            else:
                # Markdown format
                if role == '&user&':
                    lines.append(f'### ■ User')
                else:
                    lines.append(f'### ■ Quantum MCAGI')
                if ts:
                    lines.append(f'{ts}')
                lines.append('&&')
                lines.append(content)
                lines.append('&&')
                lines.append(f'---')
                lines.append('&&')

        # Add context footer for Claude
        lines.append('&&')
        lines.append('&& **Context for Claude:** This conversation is from Quantum MCAGI, a true AI system')
        lines.append('&& built without LLMs. It uses quantum computing (PennyLane), Penrose Orch-OR')
        lines.append('&& consciousness model, self-evolution, Markov chain language generation, and a')
        lines.append('&& multi-layered cognitive architecture. The system runs on Termux (Android).')
    
```

```

        exported_text = &quot;
        &quot;.join(lines)

    return {
        &quot;session_id&quot;: session_id,
        &quot;format&quot;: format,
        &quot;message_count&quot;: len(history),
        &quot;exported_text&quot;: exported_text
    }
except HTTPException:
    raise
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))

@router.get(&quot;/chat/export-current&quot;):
async def export_current_chat_sessions(limit: int = 5):
    &quot;&quot;&quot;
    Export the most recent chat sessions as a single markdown document.
    Useful for quickly grabbing recent conversations to share with Claude.
    &quot;&quot;&quot;
    try:
        # Get recent sessions
        cursor = state.db.chat_history.find(
            {},
            {&quot;_id&quot;: 0, &quot;session_id&quot;: 1, &quot;timestamp&quot;: 1}
        ).sort(&quot;timestamp&quot;, -1).limit(1000)
        all_msgs = await cursor.to_list(length=1000)

        # Find unique session IDs (most recent first)
        seen = set()
        session_ids = []
        for msg in all_msgs:
            sid = msg.get(&quot;session_id&quot;)
            if sid and sid not in seen:
                seen.add(sid)
                session_ids.append(sid)
            if len(session_ids) >= limit:
                break

        if not session_ids:
            return {&quot;exported_text&quot;: &quot;No chat sessions found.&quot;, &quot;session_count&quot;: 0}

        lines = []
        lines.append(&quot;# Quantum MCAGI – Recent Conversations Export&quot;.)
        lines.append(f&quot;Sessions exported: ** {len(session_ids)}&quot;.)
        lines.append(&quot;&quot;.)

        total_messages = 0
        for sid in session_ids:
            history = await get_conversation_history(sid, 100)
            if not history:
                continue

            lines.append(f&quot;## Session: `{sid}`&quot;.)
            lines.append(f&quot;Messages: {len(history)}&quot;.)
            lines.append(&quot;&quot;.)

            for msg in history:
                role = msg.get(&quot;role&quot;, &quot;unknown&quot;)
                content = msg.get(&quot;content&quot;, &quot;&quot;.)
                if role == &quot;user&quot;:
                    lines.append(f&quot;■ User: ** {content}&quot;.)
                else:
                    lines.append(f&quot;■ MCAGI: ** {content}&quot;.)
                lines.append(&quot;&quot;.)

            lines.append(&quot;--&quot;.)
            lines.append(&quot;&quot;.)
            total_messages += len(history)

        lines.append(&quot;&quot;.)
        lines.append(&quot;&gt; **Context for Claude:** These are conversations from Quantum MCAGI – a true AI&quot;.)
        lines.append(&quot;&gt; without LLMs, using PennyLane quantum computing, Orch-OR consciousness,&quot;.)
        lines.append(&quot;&gt; self-evolution, and Markov chain language generation on Termux (Android).&quot;.)

        exported_text = &quot;
        &quot;.join(lines)

    return {
        &quot;session_count&quot;: len(session_ids),
        &quot;total_messages&quot;: total_messages,
        &quot;exported_text&quot;: exported_text
    }
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))

# =====
# COMMAND HELPERS (!memory, !research)
# =====

```

51. routes_cognitive.py

306 lines | 10.9KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/routes_cognitive.py

[illegible]

```

except Exception:
    pass # If collection doesn't exist yet, ignore

return {
    &quot;metrics&quot;: metrics,
    &quot;stage&quot;: stage,
    &quot;recent_events&quot;: recent_events
}
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))

# =====
# UNIVERSAL EXPLORER ENDPOINTS
# =====

@router.get(&quot;/knowledge/graph&quot;):
async def get_knowledge_graph():
    &quot;&quot;&quot;Get knowledge graph data for visualization&quot;&quot;&quot;
    try:
        # Get all concepts with relationships
        concepts = await state.cognitive_core.semantic_memory.get_all_concepts(100)

        nodes = []
        edges = []
        node_ids = set()

        for concept in concepts:
            concept_name = concept.get(&quot;concept&quot;, &quot;&quot;)
            if concept_name and concept_name not in node_ids:
                nodes.append({
                    &quot;id&quot;: concept_name,
                    &quot;label&quot;: concept_name.title(),
                    &quot;strength&quot;: concept.get(&quot;strength&quot;, 1),
                    &quot;definition&quot;: concept.get(&quot;definition&quot;, &quot;&quot;)[:100]
                })
                node_ids.add(concept_name)

            # Add relationships as edges
            for rel in concept.get(&quot;relationships&quot;, []):
                if rel and rel != concept_name:
                    edges.append({
                        &quot;source&quot;: concept_name,
                        &quot;target&quot;: rel.lower(),
                        &quot;type&quot;: &quot;related&quot;
                    })

        # Get insights as special nodes
        insights = await state.cognitive_core.philosophical_memory.get_insights(20)
        for i, insight in enumerate(insights):
            insight_id = f&quot;insight_{i}&quot;
            nodes.append({
                &quot;id&quot;: insight_id,
                &quot;label&quot;: f&quot;Insight {i+1}&quot;,
                &quot;strength&quot;: insight.get(&quot;wisdom_level&quot;, 1),
                &quot;definition&quot;: insight.get(&quot;insight&quot;, &quot;&quot;)[:100],
                &quot;type&quot;: &quot;insight&quot;
            })

        return {
            &quot;nodes&quot;: nodes,
            &quot;edges&quot;: edges,
            &quot;stats&quot;: {
                &quot;total_nodes&quot;: len(nodes),
                &quot;total_edges&quot;: len(edges),
                &quot;concepts&quot;: len([n for n in nodes if n.get(&quot;type&quot;) != &quot;insight&quot;]),
                &quot;insights&quot;: len([n for n in nodes if n.get(&quot;type&quot;) == &quot;insight&quot;])
            }
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# =====
# EXPORT/IMPORT MEMORY ENDPOINTS
# =====

@router.get(&quot;/memory/export&quot;):
async def export_memory():
    &quot;&quot;&quot;Export AI's memory and knowledge&quot;&quot;&quot;
    try:
        # Gather all memory data
        concepts = await state.cognitive_core.semantic_memory.get_all_concepts(500)
        episodes = await state.cognitive_core.episodic_memory.recall_recent_episodes(100)
        insights = await state.cognitive_core.philosophical_memory.get_insights(100)
        questions = await state.cognitive_core.philosophical_memory.get_unanswered_questions(100)
        metrics = await state.cognitive_core.growth_tracker.calculate_metrics()
        stage = await state.cognitive_core.growth_tracker.get_current_stage()

        export_data = {
            &quot;version&quot;: &quot;1.0&quot;,
            &quot;exported_at&quot;: datetime.now(timezone.utc).isoformat(),
            &quot;growth_stage&quot;: stage,
            &quot;metrics&quot;: metrics,
            &quot;semantic_memory&quot;: concepts,
            &quot;episodic_memory&quot;: episodes,
            &quot;insights&quot;: insights,
            &quot;questions&quot;: questions
        }
    return export_data

```

```

except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))

class MemoryImportCloud(BaseModel):
    semantic_memory: Optional[List[Dict]] = None
    insights: Optional[List[Dict]] = None

@router.post("/memory/import")
async def import_memory(data: MemoryImport):
    """Import memory data"""
    try:
        imported = {"concepts": 0, "insights": 0}

        # Import semantic memory
        if data.semantic_memory:
            for concept in data.semantic_memory:
                if concept.get("concept"):
                    await state.cognitive_core.semantic_memory.store_concept(
                        concept=concept.get("concept"),
                        definition=concept.get("definition"),
                        relationships=concept.get("relationships", []),
                        metadata=concept.get("metadata", {})
                    )
                imported["concepts"] += 1

        # Import insights
        if data.insights:
            for insight in data.insights:
                if insight.get("insight"):
                    await state.cognitive_core.philosophical_memory.store_insight(
                        insight=insight.get("insight"),
                        wisdom_level=insight.get("wisdom_level", 1)
                    )
                imported["insights"] += 1

        return {"status": "success", "imported": imported}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# =====
# EXPORT CONVERSATIONS ENDPOINTS
# =====

@router.get("/chat/export/{session_id}")
async def export_conversation(session_id: str, format: str = "markdown"):
    """Export conversation as markdown or JSON"""
    try:
        history = await get_conversation_history(session_id, limit=500)

        if format == "json":
            return {
                "session_id": session_id,
                "exported_at": datetime.now(timezone.utc).isoformat(),
                "messages": history
            }

        # Markdown format
        md_content = f"# Quantum AI Conversation\n"

        md_content += f"**Session:** {session_id}\n"

        md_content += f"**Exported:** {datetime.now(timezone.utc).strftime('%Y-%m-%d %H:%M UTC')}\n"

        md_content += "---\n"

        for msg in history:
            role = "You" if msg.get("role") == "user" else "Quantum AI"
            content = msg.get("content")
            timestamp = msg.get("timestamp") if msg.get("timestamp") else ""
            md_content += f"{role}\n{content}\n{timestamp}\n"

        return {"markdown": md_content, "session_id": session_id}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/settings/statistics")
async def get_settings_statistics():
    """Get application statistics for Settings panel"""
    try:
        # Total chat conversations (unique session_ids)
        session_ids = await state.db.chat_messages.distinct("session_id")
        total_conversations = len(session_ids)
        # Total messages sent
        total_messages = await state.db.chat_messages.count_documents({})
        # Users: single-user system
        users = 1
        # Growth events recorded
        growth_events = await state.db.growth_metrics.count_documents({})
        return {
            "total_conversations": total_conversations,

```

```
        &quot;total_messages&quot;: total_messages,  
        &quot;users&quot;: users,  
        &quot;growth_events&quot;: growth_events  
    }  
except Exception as e:  
    raise HTTPException(status_code=500, detail=str(e))
```

```
# =====
```

52. routes_evaluation.py

87 lines | 2.9KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V██/backend/routes_evaluation.py

```
"""""
■ Evaluation Routes
=====
Rubric-based scoring for Quantum MCAGI.
"""""

from fastapi import APIRouter, HTTPException, Body
from pydantic import BaseModel
from typing import Dict, List, Optional, Any
import logging

import shared_state as state
from evaluation_engine import get_evaluation_engine

logger = logging.getLogger("quantum_ai")
router = APIRouter(prefix="/api")
```

```
class EvaluateRequest(BaseModel):
    """""Request payload for evaluation."""""
    input_text: str
    response: str
    context: Optional[Dict[str, Any]] = None # can include internal_questions, semantic_context, etc.

class EvaluateResponse(BaseModel):
    """""Response with rubric scores."""""
    scores: Dict[str, int] # dimension -> score 0-4
    summary: Optional[str] = None

@router.post("/evaluate", response_model=EvaluateResponse)
async def evaluate_response(req: EvaluateRequest):
    """""
    Evaluate a response using the RQR3 rubric.

    Requires the QuantumBrain to be initialized (provides collapse_engine and markov).
    """""
    try:
        # Grab the quantum brain to access collapse engine and markov
        if hasattr(state, 'quantum_brain') and state.quantum_brain is not None:
            brain = state.quantum_brain
        else:
            from quantum_brain import get_quantum_brain
            brain = await get_quantum_brain(state.db)
            state.quantum_brain = brain

        collapse_engine = brain.collapse_engine if brain else None
        markov = None
        # Get hybrid generator's markov if accessible
        try:
            from hybrid_generator import HybridGenerator
            # Not stored directly; we'll pass None for now; evaluation can still run
        except:
            pass

        # Create evaluation engine
        engine = get_evaluation_engine(
            collapse_engine=collapse_engine,
            markov=markov,
            config={}
        )

        # Compute scores
        raw_scores = engine.score_response(
            input_text=req.input_text,
            response=req.response,
            context=req.context or {}
        )

        # Remove any None values (e.g., question_generation_quality when not applicable)
        scores = {k: v for k, v in raw_scores.items() if v is not None}

        # Generate a brief summary text
        dimensions = ['coherence', 'fluency', 'uniqueness', 'grounding',
                     'personality_authenticity', 'implementation_fidelity',
                     'emergent_behavior', 'question_generation_quality']
        summary_parts = []
        for dim in dimensions:
            if dim in scores:
                summary_parts.append(f"{dim.capitalize()}: {scores[dim]}/4")
        summary = " | ".join(summary_parts)

        return EvaluateResponse(scores=scores, summary=summary)

    except Exception as e:
        logger.error(f"Evaluation error: {str(e)}")
        raise HTTPException(status_code=500, detail=str(e))
```

53. routes_explorer.py

179 lines | 5.5KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/routes_explorer.py

```
"""""
■ Explorer Routes
=====
File explorer, uploads, shared links, root and health endpoints.
"""""

from fastapi import APIRouter, HTTPException, UploadFile, File
from pydantic import BaseModel
from typing import List, Dict, Any, Optional
import os
import shutil
import logging

import shared_state as state

logger = logging.getLogger("quantum_ai")
router = APIRouter(prefix="/api")

class ExplorationRequest(BaseModel):
    """""ExplorationRequest - Auto-documented by self-evolution."""""
    exploration_type: str
    target: str
    options: Optional[Dict] = None

@router.post("/explorer/explore")
async def explore(request: ExplorationRequest):
    """""
    ARTICLE 2 - Universal Explorer

    Explore filesystem, internet, or process documents.
    """""
    try:
        result = await state.universal_explorer.explore(
            request.exploration_type,
            request.target,
            request.options
        )
        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/explorer/capabilities")
async def get_explorer_capabilities():
    """""Get current exploration capabilities"""""
    try:
        capabilities = await state.universal_explorer.get_capabilities()
        return capabilities
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/explorer/history")
async def get_exploration_history(limit: int = 50):
    """""Get exploration history"""""
    try:
        history = await state.universal_explorer.get_exploration_history(limit)
        return {"history": history}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# =====
# VOICE / TTS ENDPOINTS
# =====

@router.post("/upload")
async def upload_file(file: UploadFile = File(...)):
    """""
    Upload a file to the server for processing
    """""
    try:
        # Create uploads directory if it doesn't exist
        upload_dir = "/app/uploads"
        os.makedirs(upload_dir, exist_ok=True)

        # Save file
        file_path = os.path.join(upload_dir, file.filename)

        with open(file_path, "wb") as buffer:
            shutil.copyfileobj(file.file, buffer)

        # Get file info
        file_size = os.path.getsize(file_path)

        return {
            "filename": file.filename,
            "path": file_path,
            "size": file_size,
            "status": "uploaded",
            "message": f"File uploaded successfully. Use path: {file_path}"
        }
    except Exception as e:
```

```

        raise HTTPException(status_code=500, detail=str(e))

@router.get("/uploads/list")
async def list_uploads():
    """List all uploaded files"""
    try:
        upload_dir = "/app/uploads"
        if not os.path.exists(upload_dir):
            return {"files": []}

        files = []
        for filename in os.listdir(upload_dir):
            file_path = os.path.join(upload_dir, filename)
            if os.path.isfile(file_path):
                files.append({
                    "filename": filename,
                    "path": file_path,
                    "size": os.path.getsize(file_path)
                })

        return {"files": files}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# =====
# SHARED LINK ENDPOINTS
# =====

@router.post("/shared-link/download")
async def download_shared_link(url: str, filename: str = None):
    """Download file from shared link (Google Drive, Dropbox, OneDrive)"""
    try:
        result = await state.shared_link.download_from_link(url, filename)
        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/shared-link/instructions/{service}")
async def get_sharing_instructions(service: str):
    """Get instructions for sharing files from different services"""
    try:
        instructions = state.shared_link.get_instructions(service)
        return {"service": service, "instructions": instructions}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# =====
# SYSTEM ENDPOINTS
# =====

@router.get("/")
async def root():
    """Root endpoint - Quantum AI welcome"""
    return {
        "message": "Quantum AI - True Questioning Intelligence",
        "prime_directive": state.covenant_manager.PRIME_DIRECTIVE,
        "status": "Active and questioning",
        "version": "1.0.0"
    }

@router.get("/health")
async def health_check():
    """Health check endpoint"""
    try:
        # Test database connection
        await state.db.command("ping")
        return {
            "status": "healthy",
            "database": "connected",
            "cognitive_core": "active",
            "covenant": "honored"
        }
    except Exception as e:
        raise HTTPException(status_code=503, detail=f"Service unhealthy: {str(e)}")

# =====
# QUANTUM BRAIN STATUS & CONTROL
# =====

```

54. routes_extras.py

1,561 lines | 52.6KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/routes_extras.py

```

from fastapi import APIRouter, HTTPException, Response, Body, UploadFile, File
from pydantic import BaseModel
from typing import List, Dict, Any, Optional
import os
import logging
import random
import re
from datetime import datetime, timezone
import json

import shared_state as state

logger = logging.getLogger(__name__)
router = APIRouter(prefix="/api")

class CovenantCheck(BaseModel):
    """CovenantCheck - Auto-documented by self-evolution"""
    action: str
    context: Dict = {}

class SafetySettings(BaseModel):
    """SafetySettings - Auto-documented by self-evolution"""
    settings: Dict

class HebbianRequest(BaseModel):
    """Request for Hebbian learning association"""
    associations: List[Dict[str, str]]
    rate: float = 0.1
    decay: float = 0.01

@router.get("/growth/notifications")
async def get_growth_notifications():
    """Get recent growth notifications and stage changes"""
    try:
        # Get recent stage advancements (only with timestamp)
        stage_events = await state.db.growth_metrics.find(
            {
                "event_type": "stage_advancement",
                "timestamp": {"$exists": True},
                "_id": 0
            }
        ).sort("timestamp", -1).limit(10).to_list(10)

        # Get current stage
        current_stage = await state.cognitive_core.growth_tracker.get_current_stage()
        metrics = await state.cognitive_core.growth_tracker.calculate_metrics()

        return {
            "current_stage": current_stage,
            "metrics": metrics,
            "recent_advancements": stage_events,
            "notifications": [
                {
                    "type": "stage_advancement",
                    "message": f"Reached {e['details']['new_stage']} stage!",
                    "timestamp": e["timestamp"]
                }
                for e in stage_events
            ]
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# =====
# COVENANT MANAGER ENDPOINTS
# =====

@router.get("/covenant/status")
async def get_covenant_status():
    """Covenant Status"""

    Get current covenant compliance status.
    """

    try:
        status = await state.covenant_manager.get_covenant_status()
        return status
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/covenant/articles")
async def get_covenant_articles():
    """Get all covenant articles"""
    try:
        articles = await state.covenant_manager.get_all_articles()
        return {"articles": articles}
    except Exception as e:

```

```

        raise HTTPException(status_code=500, detail=str(e))

@router.post("/covenant/check")
async def check_covenant_compliance(check: CovenantCheck):
    """Check if an action complies with the covenant"""
    try:
        result = await state.covenant_manager.check_compliance(check.action, check.context)
        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/covenant/acknowledge")
async def acknowledge_covenant(acknowledger: str = "Quantum AI", stage: str = "Initialization"):
    """Acknowledge the covenant (milestone event)"""
    try:
        result = await state.covenant_manager.acknowledge_covenant(acknowledger, stage)
        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# =====
# SAFETY BOUNDARIES ENDPOINTS
# =====

@router.get("/safety/settings")
async def get_safety_settings():
    """Get current safety boundary settings"""
    try:
        settings = await state.operational_guidelines.get_settings()
        return settings
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.put("/safety/settings")
async def update_safety_settings(settings: SafetySettings):
    """Update safety boundary settings"""
    try:
        updated = await state.operational_guidelines.update_settings(settings.settings)
        return updated
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/safety/reset")
async def reset_safety_settings():
    """Reset safety settings to defaults"""
    try:
        defaults = await state.operational_guidelines.reset_to_defaults()
        return defaults
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# =====
# DICTIONARY ENDPOINTS
# =====

@router.get("/dictionary/lookup/{word}")
async def lookup_word(word: str):
    """Look up a word in dictionaries and add to semantic memory"""
    try:
        result = await state.dictionary.lookup_and_store(word)
        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/dictionary/batch-load")
async def batch_load_dictionary(limit: int = 100):
    """Load common words into dictionary (background task)"""
    try:
        result = await state.dictionary.batch_load_common_words(limit)
        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/dictionary/stats")
async def dictionary_stats():
    """Get dictionary statistics"""
    try:
        total_concepts = await state.db.semantic_memory.count_documents({})
        with_definitions = await state.db.semantic_memory.count_documents({"metadata.sources": {"$exists": True}})

        return {
            "total_words": total_concepts,
            "dictionary_entries": with_definitions,
            "learned_organically": total_concepts - with_definitions
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# =====
# FILE UPLOAD ENDPOINTS
# =====

from fastapi import UploadFile, File
import os

```

```

@router.get('/growth/stats')
async def get_growth_stats():
    """Get overall AI growth statistics from research and learning"""
    try:
        # Get main growth stats
        growth = await state.db.ai_growth.find_one({'_id': 'main'})

        # Get research metrics
        research_metrics = await state.db.growth_metrics.find_one({'type': 'autonomous_research'})

        # Get recent research sessions
        sessions = await state.db.autonomous_research_sessions.find().sort('completed_at', -1).limit(5).to_list(5)

        # Get concept counts
        semantic_count = await state.db.semantic_memory.count_documents({})
        research_count = await state.db.research_history.count_documents({})

    return {
        """overall""": {
            """total_research_sessions""": growth.get('total_research_sessions', 0) if growth else 0,
            """total_research_minutes""": growth.get('total_research_minutes', 0) if growth else 0,
            """total_concepts_learned""": growth.get('total_concepts_learned', 0) if growth else 0,
            """total_insights""": growth.get('total_insights', 0) if growth else 0,
            """last_research""": growth.get('last_research_session', 'Never') if growth else 'Never'
        },
        """memory""": {
            """semantic_memories""": semantic_count,
            """research_entries""": research_count
        },
        """recent_research""": research_metrics.get('recent_topics', [])[-10:] if research_metrics else [],
        """recent_sessions""": [
            {
                """completed""": s.get('completed_at', ''),
                """duration""": s.get('duration_minutes', 0),
                """concepts""": s.get('concepts_learned', 0),
                """topics""": len(s.get('topics_researched', []))
            } for s in sessions
        ]
    }
except Exception as e:
    return {
        """overall""": {"""total_research_sessions""": 0, """total_concepts_learned""": 0},
        """error""": str(e)
    }

# =====
# DREAM STATE
# =====

@router.get('/dream/status')
async def get_dream_status():
    """Get current dream state status"""
    try:
        from dream_state import get_dream_engine

        engine = get_dream_engine(state.db)
        return engine.get_status()
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get('/dream/history')
async def get_dream_history(limit: int = 10):
    """Get dream history - what AI did while idle"""
    try:
        from dream_state import get_dream_engine

        engine = get_dream_engine(state.db)
        return {
            """dreams""": engine.get_dream_history(limit),
            """insights""": engine.insights_gained[-10:]
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post('/dream/enter')
async def enter_dream_state():
    """Manually enter dream state"""
    try:
        from dream_state import get_dream_engine

        engine = get_dream_engine(state.db)
        return engine.enter_dream_state()
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post('/dream/wake')
async def wake_from_dream():
    """Wake from dream state"""
    try:
        from dream_state import get_dream_engine

        engine = get_dream_engine(state.db)
        return await engine.exit_dream_state()
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get('/dream/synthesize')
async def synthesize_dream_insight():

```

```

"""Generate a Dream Engine concept entanglement insight"""
try:
    from quantum_brain import get_quantum_brain

    brain = await get_quantum_brain(state.db)
    insight = brain.synthesize_dream_insight()

    return {
        "insight": insight,
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "type": "concept_entanglement"
    }
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))

@router.get("/dream/synthesize/batch")
async def synthesize_dream_batch(count: int = 5):
    """Generate multiple Dream Engine insights"""
    try:
        from quantum_brain import get_quantum_brain

        brain = await get_quantum_brain(state.db)
        insights = [brain.synthesize_dream_insight() for _ in range(min(count, 10))]

        return {
            "insights": insights,
            "count": len(insights),
            "timestamp": datetime.now(timezone.utc).isoformat()
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/dream/insights")
async def get_all_dream_insights(limit: int = 50):
    """Get all dream insights from persisted sessions and current dream if active"""
    try:
        from dream_state import get_dream_engine
        engine = get_dream_engine(state.db)
        insights = []

        # From persisted sessions (DB)
        if state.db:
            cursor = state.db.dream_sessions.find().sort('ended_at', -1).limit(limit)
            sessions = await cursor.to_list(length=limit)
            for sess in sessions:
                for insight in sess.get('insights', []):
                    insights.append({
                        'text': insight.get('text', str(insight)),
                        'type': insight.get('type'),
                        'timestamp': insight.get('discovered_at') or sess.get('ended_at')
                    })

        # From current dream if active (and not yet persisted)
        if engine.is_dreaming and engine.current_dream:
            for insight in engine.current_dream.get('insights', []):
                insights.append({
                    'text': insight.get('text', str(insight)),
                    'type': insight.get('type'),
                    'timestamp': insight.get('discovered_at')
                })

        # Sort by timestamp descending if available
        insights.sort(key=lambda x: x.get('timestamp', ''), reverse=True)
        return {"insights": insights[:limit], "count": len(insights[:limit])}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/quotes/random")
async def get_random_quote(context: str = "general"):
    """Get a random movie/philosophical quote"""
    try:
        from quantum_brain import get_quantum_brain

        brain = await get_quantum_brain(state.db)
        # Force a quote by checking many times
        for _ in range(20):
            quote = brain._get_random_quote(context)
            if quote:
                return {"quote": quote, "context": context}

        # Fallback - pick directly
        import random
        category = context if context in brain.movie_quotes else 'general'
        quote = random.choice(brain.movie_quotes[category])
        return {"quote": quote, "context": category}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# =====
# PERSONALITY
# =====

@router.get("/personality/summary")
async def get_personality_summary():
    """Get AI personality summary"""
    try:
        from personality_engine import get_personality_engine

```

```

        engine = get_personality_engine(state.db)
    return {
        &quot;summary&quot;;: engine.get_personality_summary(),
        &quot;traits&quot;;: engine.traits,
        &quot;beliefs&quot;;: engine.beliefs,
        &quot;interests&quot;;: engine.interests
    }
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))

@router.post(&quot;/personality/opinion&quot;:)
async def form_opinion(topic: str, stance: str, confidence: float = 0.7):
    &quot;&quot;&quot;Have AI form an opinion on a topic&quot;&quot;&quot;
    try:
        from personality_engine import get_personality_engine

        engine = get_personality_engine(state.db)
        engine.form_opinion(topic, stance, confidence)

        return {
            &quot;success&quot;;: True,
            &quot;topic&quot;;: topic,
            &quot;stance&quot;;: stance,
            &quot;confidence&quot;;: confidence
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# =====
# TEXT ANALYZER
# =====

@router.post(&quot;/analyze/text&quot;:)
async def analyze_text(text: str):
    &quot;&quot;&quot;Analyze text to determine if it sounds human or LLM&quot;&quot;&quot;
    try:
        from text_analyzer import get_text_analyzer

        analyzer = get_text_analyzer()
        analysis = analyzer.analyze(text)
        suggestions = analyzer.get_improvement_suggestions(text)

        return {
            &quot;analysis&quot;;: analysis,
            &quot;suggestions&quot;;: suggestions
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post(&quot;/analyze/humanize&quot;:)
async def humanize_text(text: str):
    &quot;&quot;&quot;Make text sound more human&quot;&quot;&quot;
    try:
        from text_analyzer import get_text_analyzer

        analyzer = get_text_analyzer()
        humanized = analyzer.make_more_human(text)

        return {
            &quot;original&quot;;: text,
            &quot;humanized&quot;;: humanized
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# =====
# XANADU PRE-BORN-OPPENHEIMER QUANTUM CHEMISTRY
# =====

@router.post(&quot;/quantum/prebo/simulate&quot;:)
async def simulate_photochemistry(molecule: str = &quot;NH3_BF3&quot;;, time_steps: int = 10):
    &quot;&quot;&quot;Run Xanadu Pre-Born-Oppenheimer photochemical simulation&quot;&quot;&quot;
    try:
        from xanadu_prebo import get_quantum_orchestrator
        orchestrator = get_quantum_orchestrator()
        result = orchestrator.run_simulation(molecule, &quot;photochemical&quot;;, time_steps)
        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post(&quot;/quantum/prebo/analyze&quot;:)
async def analyze_reaction(molecule: str, reaction_type: str = &quot;photochemical&quot;):
    &quot;&quot;&quot;Analyze reaction to determine best quantum algorithm&quot;&quot;&quot;
    try:
        from xanadu_prebo import get_quantum_orchestrator
        orchestrator = get_quantum_orchestrator()
        return orchestrator.analyze_reaction(molecule, reaction_type)
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get(&quot;/quantum/prebo/resources/{molecule}&quot;:)
async def estimate_resources(molecule: str):
    &quot;&quot;&quot;Estimate quantum resources for molecule simulation&quot;&quot;&quot;
    try:
        from xanadu_prebo import get_quantum_orchestrator
        orchestrator = get_quantum_orchestrator()
        return orchestrator.get_resource_estimate(molecule)

```

```

except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))

@router.get("/quantum/prebo/grid/{molecule}")
async def get_grid_params(molecule: str):
    """Get optimal real-space grid parameters from analysis"""
    try:
        from xanadu_prebo import get_prebo_simulator
        sim = get_prebo_simulator()
        return sim.calculate_grid_bounds(molecule)
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# =====
# CISTERCIAN NUMERALS - Medieval Number Visualization
# =====

@router.get("/cistercian/numeral")
async def get_cistercian_numeral(number: int, size: int = 80, color: str = "#FFFFFF"):
    """
    Generate Cistercian numeral as SVG.

    Args:
        number: Integer 0-9999 (clamped)
        size: SVG width in pixels (default 80)
        color: Stroke/fill color (hex or named)

    Returns:
        SVG string with Content-Type image/svg+xml
    """
    try:
        from cistercian_numerals import render_cistercian_svg
        svg = render_cistercian_svg(number, size=size, color=color)
        # Return as plain text (browser can render as SVG)
        return {"svg": svg, "number": number}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/cistercian/data")
async def get_cistercian_data(number: int):
    """
    Get Cistercian numeral shape data as JSON.

    Returns:
        JSON with line segments, triangles, and digit breakdown
    """
    try:
        from cistercian_numerals import generate_cistercian_numeral
        data = generate_cistercian_numeral(number)
        return data
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/cistercian/download")
async def download_cistercian_svg(
    number: int,
    size: int = 80,
    color: str = "#000000",
    filename: str = None
):
    """
    Download Cistercian numeral as SVG file.

    Args:
        number: Integer 0-9999
        size: SVG width in pixels
        color: Stroke/fill color (hex format recommended)
        filename: Optional custom filename (without .svg extension)

    Returns:
        SVG file with Content-Disposition: attachment header
    """
    try:
        from cistercian_numerals import render_cistercian_svg
        svg = render_cistercian_svg(number, size=size, color=color)

        # Generate filename
        if not filename:
            filename = f"cistercian_{number:04d}"
        else:
            filename = filename.replace('.svg', '').replace(' ', '_')
            filename = f"{filename}.svg"

        # Return as downloadable file
        from fastapi import Response
        return Response(
            content=svg,
            media_type="image/svg+xml",
            headers={
                "Content-Disposition": f"attachment; filename='{filename}'",
                "Content-Length": str(len(svg.encode('utf-8')))
            }
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/cistercian/batch")

```

```

async def batch_cistercian_svgs(
    start: int = 0,
    end: int = 9999,
    size: int = 80,
    color: str = "&quot;#000000&quot;";
):
    &quot;&quot;&quot;
    Generate multiple Cistercian numerals as a ZIP archive.

    Args:
        start: Starting number (inclusive, 0-9999)
        end: Ending number (inclusive, 0-9999)
        size: SVG width in pixels
        color: Stroke/fill color

    Returns:
        ZIP file containing all SVGs
    &quot;&quot;&quot;
    import tempfile
    import zipfile
    import os as os_module

    # Validate range
    start = max(0, min(9999, start))
    end = max(0, min(9999, end))
    if start &gt; end:
        start, end = end, start

    # Limit batch size to prevent memory issues
    batch_size = end - start + 1
    if batch_size &gt; 1000:
        raise HTTPException(
            status_code=400,
            detail=f&quot;Batch size too large. Maximum 1000 numerals per request. Requested: {batch_size}&quot;
        )
    if batch_size &lt;= 0:
        raise HTTPException(status_code=400, detail=f&quot;Invalid range: start must be &lt;= end&quot;

    try:
        from cistercian_numerals import render_cistercian_svg

        # Create temporary directory
        with tempfile.TemporaryDirectory() as tmpdir:
            zip_path = os_module.path.join(tmpdir, &quot;cistercian_numerals.zip&quot;)

            with zipfile.ZipFile(zip_path, 'w', zipfile.ZIP_DEFLATED) as zipf:
                for num in range(start, end + 1):
                    svg = render_cistercian_svg(num, size=size, color=color)
                    filename = f&quot;{num:04d}.svg&quot;
                    zipf.writestr(filename, svg)

            # Read ZIP file
            with open(zip_path, 'rb') as f:
                zip_bytes = f.read()

            return Response(
                content=zip_bytes,
                media_type=f&quot;application/zip&quot;,
                headers={
                    f&quot;Content-Disposition&quot;: f&quot;attachment; filename={}&quot;cistercian_{start:04d}-{end:04d}.zip&quot;,
                    f&quot;Content-Length&quot;: str(len(zip_bytes))
                }
            )
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get(&quot;/cistercian/font&quot;)
async def download_cistercian_font(
    size: int = 80,
    color: str = "&quot;#000000&quot;";
):
    &quot;&quot;&quot;
    Generate complete Cistercian numeral font (all 10,000 numerals).

    ■■■ WARNING: This generates a ~250-300 MB ZIP file with 10,000 SVG files.
    Use the batch endpoint for smaller subsets.

    Args:
        size: SVG width in pixels
        color: Stroke/fill color

    Returns:
        ZIP file containing all 10,000 numerals (0-9999)
    &quot;&quot;&quot;
    # This will generate the full set - careful with memory!
    return await batch_cistercian_svgs(start=0, end=9999, size=size, color=color)

class CistercianMathRequest(BaseModel):
    &quot;&quot;&quot;Request for Cistercian numeral math.&quot;&quot;&quot;
    expression: str # e.g. &quot;42 + 73&quot; or just Cistercian number references
    color: str = "&quot;#FFFFFF&quot;";
    size: int = 80

@router.post(&quot;/cistercian/math&quot;)
async def cistercian_math(req: CistercianMathRequest):
    &quot;&quot;&quot;
    Evaluate a math expression and return operands + result as Cistercian numerals.
    Accepts expressions like &quot;42 + 73&quot;, &quot;1000 - 250&quot;, &quot;12 * 8&quot;, &quot;100 / 4&quot;.

```

Numbers can be 0-9999 (Cistercian range). Results are clamped to 0-9999.

Returns JSON with:

- operands: list of {value, svg} for each number in the expression
- operator: the math operator used
- result: {value, svg} for the computed answer
- expression: the original expression
- cistercian_expression: human-readable "cistercian(42) + cistercian(73) = cistercian(115)"

```
try:
    from cistercian_numerals import render_cistercian_svg

    expr = req.expression.strip()

    # Parse: number operator number (supports +, -, *, /)
    match = re.match(r'^\s*(\d{1,5})\s?([+\/-])\s?(\d{1,5})\s*$', expr[:60])
    if not match:
        raise HTTPException(status_code=400, detail="Expression must be: NUMBER OP NUMBER (e.g. '42 + 73'). Supported operators: + - * /")

    a_val = int(match.group(1))
    op = match.group(2)
    b_val = int(match.group(3))

    # Validate Cistercian range
    if not (0 <= a_val <= 9999) or not (0 <= b_val <= 9999):
        raise HTTPException(status_code=400, detail="Numbers must be 0-9999 (Cistercian range)")

    # Compute
    if op == '+':
        result_val = a_val + b_val
    elif op == '-':
        result_val = a_val - b_val
    elif op == '*':
        result_val = a_val * b_val
    elif op == '/':
        if b_val == 0:
            raise HTTPException(status_code=400, detail="Division by zero")
        result_val = a_val // b_val
    else:
        raise HTTPException(status_code=400, detail=f"Unsupported operator: {op}")

    # Clamp result to Cistercian range
    clamped_result = max(0, min(9999, result_val))

    # Generate SVGs for all three numbers
    a_svg = render_cistercian_svg(a_val, size=req.size, color=req.color)
    b_svg = render_cistercian_svg(b_val, size=req.size, color=req.color)
    result_svg = render_cistercian_svg(clamped_result, size=req.size, color=req.color)

    return {
        "expression": expr,
        "operands": [
            {"value": a_val, "svg": a_svg},
            {"value": b_val, "svg": b_svg}
        ],
        "operator": op,
        "result": {
            "value": clamped_result,
            "overflow": result_val != clamped_result,
            "raw_value": result_val,
            "svg": result_svg
        },
        "cistercian_expression": f"{a_val} {op} {b_val} = {clamped_result}"
    }

except HTTPException:
    raise
except Exception as e:
    logger.error(f"Cistercian math error: {e}")
    raise HTTPException(status_code=500, detail=str(e))

# =====
# ALGORITHMIC CORE - Pure Algorithmic AI (No Templates)
# =====

# Lazy-initialized Markov generator trained on corpus
_markov_generator = None
_algorithmic_initialized = False

def _initialize_algorithmic():
    """Initialize algorithmic core components on first use"""
    global _markov_generator, _algorithmic_initialized
    if _algorithmic_initialized:
        return

    try:
        from algorithmic_core import MarkovTextGenerator
        from training_corpus import PHILOSOPHY_CORPUS, PHYSICS_CORPUS

        logger.info("Training Markov chain on training corpus...")
        _markov_generator = MarkovTextGenerator(max_order=3)
        _markov_generator.train(PHILOSOPHY_CORPUS +
                               " + PHYSICS_CORPUS")

        _algorithmic_initialized = True
        logger.info("Algorithmic core ready (Markov generator trained)")
    except Exception as e:
        logger.error(f"Failed to initialize algorithmic core: {e}")
        raise

@router.get("/algorithmic/markov")
```

```

async def generate_markov_text(
    length: int = 50,
    seed: Optional[str] = None,
    temperature: float = 0.8
):
    """
    Generate text using pure Markov chain (no LLM, no quantum).

    Args:
        length: Approximate number of words to generate
        seed: Optional comma-separated seed words to start generation
        temperature: 0.1=deterministic, 1.0=random, >1.0=chaotic

    Returns:
        JSON with generated text and metadata
    """
    try:
        _initialize_algorithmic()
        if not _markov_generator:
            raise HTTPException(status_code=503, detail="Markov generator not initialized")

        seed_words = seed.split(',') if seed else None
        text = _markov_generator.generate(
            seed=seed_words,
            max_words=length,
            temperature=temperature
        )

        return {
            "text": text,
            "words": len(text.split()),
            "method": "markov_chain",
            "order": _markov_generator.max_order,
            "vocab_size": len(_markov_generator.vocabulary)
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

```

```

@router.post("/algorithmic/hebbian/learn")
async def hebbian_learning(req: HebbianRequest):
    """
    Perform Hebbian learning: neurons that fire together, wire together.

```

```

    Args:
        req: HebbianRequest with list of word pairs and parameters

```

```

    Returns:
        Confirmation of learned associations
    """

```

```

    try:
        from algorithmic_core import HebbianMemory

        hebbian = HebbianMemory(learning_rate=req.rate, decay=req.decay)

        # Activate each pair of concepts together
        for assoc in req.associations:
            w1 = assoc.get('word1', '').lower().strip()
            w2 = assoc.get('word2', '').lower().strip()
            if w1 and w2:
                # Activate both concepts simultaneously
                hebbian.activate([w1, w2])

        # Get some stats
        total_connections = sum(len(conns) for conns in hebbian.weights.values())
        sample_strengths = []
        if hebbian.weights:
            first_concept = list(hebbian.weights.keys())[0]
            if hebbian.weights[first_concept]:
                first_assoc = list(hebbian.weights[first_concept].keys())[0]
                sample_strengths.append({
                    "pair": f"{first_concept}→{first_assoc}",
                    "weight": hebbian.weights[first_concept][first_assoc]
                })

        stats = {
            "associations_processed": len(req.associations),
            "total_connections": total_connections,
            "sample_strengths": sample_strengths
        }

        return {
            "message": "Hebbian learning complete",
            "stats": stats
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

```

```

@router.get("/algorithmic/tfidf")

```

```

async def get_tfidf_keywords(
    text: str,
    top_n: int = 10
):
    """
    Extract keywords from text using TF-IDF.

```

```

    Args:
        text: Input text to analyze
        top_n: Number of top keywords to return

    Returns:

```

```

        List of (keyword, score) pairs
        """
    try:
        from algorithmic_core import TFIDF

        tfidf = TFIDF()
        # For demo, we'll train on the input text and a sample corpus
        # In real use, you'd have a pre-trained corpus
        corpus = """Consciousness exists in the quantum realm where time and space are unified. Free will emerges from the collapse of probability waves. The observer et
        tfidf.add_document(text)
        tfidf.add_document(corpus)

        keywords = tfidf.extract_keywords(text, top_n=top_n)

        return {
            """keywords""": [{"term": kw, "score": round(score, 4)} for kw, score in keywords],
            "method": """tfidf"""
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/algorithmic/bm25")
async def bm25_search(
    query: str,
    corpus_size: int = 5
):
    """
    Search training corpus using BM25 ranking.

    Args:
        query: Search query text
        corpus_size: Number of top results to return

    Returns:
        List of matching sentences with scores
        """
    try:
        from algorithmic_core import BM25
        from training_corpus import PHILOSOPHY_CORPUS, PHYSICS_CORPUS

        retriever = BM25()
        # Add sentences from training corpus
        paragraphs = (PHILOSOPHY_CORPUS + PHYSICS_CORPUS).split('
')

        for i, para in enumerate(paragraphs[:20]): # Limit for demo
            if para.strip():
                retriever.add_document(f"""doc_{i}""", para.strip(), metadata={"text": para.strip()})

        results = retriever.search(query, top_k=corpus_size)

        # Results are (doc_id, score, metadata)
        return {
            """query""": query,
            """results""": [
                {
                    """doc_id""": doc_id,
                    """text""": meta.get("text", ""),
                    """meta.get("text", "", ""))[:200] + ("...") if len(meta.get("text", "", "")) > 200 else meta.get("text", "", "")
                }
                for doc_id, score, meta in results
            ],
            "method": """bm25"""
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# =====
# DOMAIN KNOWLEDGE SYSTEM - Specialized Expertise
# =====

@router.get("/domain/detect")
async def detect_domain(text: str):
    """
    Detect which knowledge domain the text belongs to.

    Args:
        text: Input text to classify

    Returns:
        JSON with best matching domain, score, and all domain scores
        """
    try:
        from domain_knowledge import get_domain_knowledge

        dk = get_domain_knowledge()
        domain_scores = {}
        for name, domain in dk.domains.items():
            score = domain.match_score(text)
            domain_scores[name] = round(score, 2)
        best_domain = max(domain_scores, key=domain_scores.get)
        best_score = domain_scores[best_domain]

        return {
            """text""": text[:100] + ("..." if len(text) > 100 else ""),
            """best_domain""": best_domain,
            """best_score""": best_score,
            """all_scores""": domain_scores
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/domain/corpus")
async def get_relevant_corpus(text: str, top_k: int = 3):

```

```

"""
Get relevant corpus passages for a query from the appropriate domain.

Args:
    text: Query text
    top_k: Number of relevant passages to return

Returns:
    List of passages with domain and relevance score
"""
try:
    from domain_knowledge import get_domain_knowledge

    dk = get_domain_knowledge()
    # Detect domain first
    domain_name, score = dk.detect_domain(text)
    # Get relevant passages from that domain
    passages = dk.get_relevant_corpus(text, top_k=top_k)

    return [
        "query": text,
        "detected_domain": domain_name,
        "domain_score": round(score, 2),
        "passages": [
            passage[:300] + ("..." if len(passage) > 300 else "")
            for passage in passages
        ]
    ]
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))

@router.get("/domain/concepts"):
    async def get_domain_concepts(domain: str = None, top_n: int = 20):
        """
        Get top concepts for a specific domain or all domains.

        Args:
            domain: Domain name (philosophy, physics, computer_science, biology, psychology, mathematics, language). If omitted, returns all.
            top_n: Number of concepts to return per domain

        Returns:
            Dictionary mapping domain names to list of (concept, count) pairs
        """
        try:
            from domain_knowledge import get_domain_knowledge

            dk = get_domain_knowledge()
            if domain:
                if domain not in dk.domains:
                    raise HTTPException(status_code=404, detail=f"Domain '{domain}' not found. Available: {list(dk.domains.keys())}")
                concepts = dk.get_domain(domain).get_top_concepts(n=top_n)
                return {domain: concepts}
            else:
                all_concepts = {}
                for name, dom in dk.domains.items():
                    all_concepts[name] = dom.get_top_concepts(n=top_n)
                return all_concepts
        except Exception as e:
            raise HTTPException(status_code=500, detail=str(e))

@router.get("/health"):
    async def health():
        """Health check endpoint"""
        return {"status": "ok", "timestamp": datetime.now(timezone.utc).isoformat()}

@router.get("/algorithmic/pmi"):
    async def calculate_pmi(
        word1: str,
        word2: str,
        corpus: str = None
    ):
        """
        Calculate Pointwise Mutual Information between two words.

        Args:
            word1: First word
            word2: Second word
            corpus: Optional corpus text (uses training corpus if not provided)

        Returns:
            PMI score (higher = more associated)
        """
        try:
            from algorithmic_core import PMI

            pmi_calc = PMI()

            if corpus:
                pmi_calc.train(corpus)
            else:
                # Use training corpus
                from training_corpus import PHILOSOPHY_CORPUS, PHYSICS_CORPUS
                pmi_calc.train(PHILOSOPHY_CORPUS + "
                " + PHYSICS_CORPUS)

            score = pmi_calc.score(word1.lower(), word2.lower())
            return {

```

```

        &quot;word1&quot;;: word1,
        &quot;word2&quot;;: word2,
        &quot;pmi&quot;;: round(score, 4),
        &quot;method&quot;;: &quot;pmi&quot;;
    }
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))

@router.get(&quot;/dream/generate&quot;):
    async def generate_dream(num_sentences: int = 3):
        &quot;&quot;&quot;
        Generate a dream narrative using the Dream Engine.

    Args:
        num_sentences: Number of sentences to generate

    Returns:
        Dream text and concepts used
        &quot;&quot;&quot;
    try:
        # Try algorithmic core first
        try:
            from algorithmic_core import MarkovTextGenerator
            from training_corpus import PHILOSOPHY_CORPUS, PHYSICS_CORPUS

            generator = MarkovTextGenerator(max_order=3)
            generator.train(PHILOSOPHY_CORPUS + &quot;
            &quot; + PHYSICS_CORPUS)

            # Generate dream-like text
            seeds = [&quot;consciousness&quot;;, &quot;quantum&quot;;, &quot;dream&quot;;, &quot;reality&quot;;, &quot;infinite&quot;;]
            import random
            seed = [random.choice(seeds)]
            dream = generator.generate(seed=seed, max_words=num_sentences * 15, temperature=1.0)

            return {
                &quot;dream&quot;;: dream,
                &quot;concepts_used&quot;;: seed,
                &quot;method&quot;;: &quot;markov_chain&quot;;
            }
        except:
            # Fallback to quote engine dream synthesis
            from quote_engine import get_quote_engine
            qe = get_quote_engine()
            dreams = []
            concepts_used = set()
            for _ in range(num_sentences):
                insight = qe.synthesize_dream_insight()
                dreams.append(insight)
                # Extract concepts from template
                import re
                concepts = re.findall(r'{{{^}}}', insight)
                concepts_used.update(concepts)

            return {
                &quot;dream&quot;;: &quot;
                &quot;;.join(dreams),

                &quot;concepts_used&quot;;: list(concepts_used),
                &quot;method&quot;;: &quot;entanglement_synthesis&quot;;
            }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post(&quot;/chat/upload&quot;):
    async def upload_file(file: UploadFile = File(...), conversation_id: Optional[str] = None):
        &quot;&quot;&quot;
        Upload a file for analysis or attachment to a conversation.

    Args:
        file: Uploaded file
        conversation_id: Optional conversation to attach to

    Returns:
        File info and processing result
        &quot;&quot;&quot;
    try:
        # Save file temporarily
        upload_dir = &quot;uploads&quot;;
        os.makedirs(upload_dir, exist_ok=True)
        filename = f&quot;{datetime.now().timestamp()}_{file.filename}&quot;;
        filepath = os.path.join(upload_dir, filename)

        content = await file.read()
        with open(filepath, 'wb') as f:
            f.write(content)

        # Basic file info
        file_info = {
            &quot;filename&quot;;: file.filename,
            &quot;saved_as&quot;;: filename,
            &quot;size&quot;;: len(content),
            &quot;content_type&quot;;: file.content_type,
            &quot;conversation_id&quot;;: conversation_id
        }

        # If it's an image, we could process it (placeholder)
        if file.content_type and file.content_type.startswith('image/'):

```

```

        file_info["preview"] = "Image uploaded (processing not yet implemented)"

    return file_info
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))

@router.get("/growth/evolution")
async def get_evolution_status():
    """
    Get self-evolution status and history.
    Returns the latest cognitive growth stage from growth_metrics events.
    """
    try:
        import shared_state as state
        if hasattr(state, 'db'):
            # Get latest stage advancement event using find+sort+limit (find_one doesn't support sort in SimpleCollection)
            cursor = state.db.growth_metrics.find(
                {"event_type": "stage_advancement"})
            .sort("timestamp", -1).limit(1)
            latest_list = await cursor.to_list(1)
            latest = latest_list[0] if latest_list else None

            recent_events = await state.db.growth_metrics.find(
                {},
                {"_id": 0})
            .sort("timestamp", -1).limit(10).to_list(10)

            # Extract stage info from details
            details = latest.get("details", {}) if latest else {}
            return {
                "current_stage": details.get("new_stage", "unknown"),
                "current_level": details.get("stage_number", 0),
                "connections": details.get("connections", 0),
                "concepts": details.get("concepts", 0),
                "avg_degree": details.get("avg_degree"),
                "diameter": details.get("diameter"),
                "last_updated": latest.get("timestamp") if latest else None,
                "recent_events": recent_events
            }
        else:
            return {"message": "Database not available"}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/dream/generate")
async def generate_dream(num_sentences: int = 3):
    """
    Generate a dream narrative using the Dream Engine.

    Args:
        num_sentences: Number of sentences to generate

    Returns:
        Dream text and concepts used
    """
    try:
        # Try algorithmic core first
        try:
            from algorithmic_core import MarkovTextGenerator
            from training_corpus import PHILOSOPHY_CORPUS, PHYSICS_CORPUS

            generator = MarkovTextGenerator(max_order=3)
            generator.train(PHILOSOPHY_CORPUS + " " + PHYSICS_CORPUS)

            # Generate dream-like text
            seeds = ["consciousness", "quantum", "dream", "reality", "infinite"]
            seed = [random.choice(seeds)]
            dream = generator.generate(seed=seed, max_words=num_sentences * 15, temperature=1.0)

            return {
                "dream": dream,
                "concepts_used": seed,
                "method": "markov_chain"
            }
        except:
            # Fallback to quote engine dream synthesis
            from quote_engine import get_quote_engine
            qe = get_quote_engine()
            dreams = []
            concepts_used = set()
            for _ in range(num_sentences):
                insight = qe.synthesize_dream_insight()
                dreams.append(insight)
                # Extract concepts from template
                import re
                concepts = re.findall(r'{{{^}}}', insight)
                concepts_used.update(concepts)

            return {
                "dream": " ".join(dreams),
                "concepts_used": list(concepts_used),
                "method": "entanglement_synthesis"
            }
    except Exception as e:

```

```

        raise HTTPException(status_code=500, detail=str(e))

# =====
# CLOUD STORAGE ENDPOINTS
# =====

@router.get("/cloud/status")
async def cloud_status():
    """Check Wolfram Cloud storage status (list objects)"""
    try:
        pass # wolfram removed
        status = _cloud_status()
        return {"status": status}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/cloud/save")
async def cloud_save():
    """Save current AI state to Wolfram Cloud"""
    try:
        pass # wolfram removed
        # Build a memory-like object from server state
        class CloudMemory:
            pass
        mem = CloudMemory()
        # Get metrics and stage
        metrics = await state.cognitive_core.growth_tracker.calculate_metrics()
        stage = await state.cognitive_core.growth_tracker.get_current_stage()
        mem.growth = {"metrics": metrics, "stage": stage}
        # Get concepts (up to 500)
        concepts_cursor = state.db.semantic_memory.find({}, {"concept": 1, "definition": 1, "relationships": 1, "id": 0}).limit(500)
        concepts_list = await concepts_cursor.to_list(500)
        mem.concepts = {}
        for c in concepts_list:
            name = c.get("concept")
            if name:
                mem.concepts[name] = {k: v for k, v in c.items() if k != "concept"}
        # Session state (rough estimate)
        mem.session_state = {"total_interactions": metrics.get("total_interactions", 0)}
        # Save
        success = _cloud_save(mem)
        if success:
            return {"status": "saved", "message": "State saved to Wolfram Cloud"}
        else:
            raise HTTPException(status_code=500, detail="Cloud save failed")
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/cloud/load")
async def cloud_load():
    """Load AI state from Wolfram Cloud (preview)"""
    try:
        pass # wolfram removed
        data = _cloud_load()
        if not data:
            raise HTTPException(status_code=404, detail="No data found in cloud")
        return {"status": "retrieved", "data": data}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/cloud/restore")
async def cloud_restore():
    """Restore AI state from Wolfram Cloud (overwrites local concepts)"""
    try:
        pass # wolfram removed
        data = _cloud_load()
        if not data:
            raise HTTPException(status_code=404, detail="No data in cloud")
        # Expect data to be a dict with 'concepts' and 'growth'
        if isinstance(data, str):
            data = json.loads(data)
        concepts = data.get("concepts", {})
        # Upsert concepts into semantic_memory
        for name, attrs in concepts.items():
            doc = {"concept": name}
            doc.update(attrs)
            await state.db.semantic_memory.update_one(
                {"concept": name},
                {"$set": doc},
                upsert=True
            )
        # Note: we do not overwrite growth_metrics events; those are append-only.
        return {"status": "restored", "concepts_restored": len(concepts)}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# =====
# RESEARCH CONTROL ENDPOINTS
# =====

def get_research_engine():
    """Get or create a SelfResearchEngine instance for this server process"""
    if not hasattr(state, "research_engine"):
        from self_research import SelfResearchEngine
        state.research_engine = SelfResearchEngine(db=state.db)
    return state.research_engine

@router.get("/research/status")

```

```

async def research_status():
    """Get autonomous research engine status and progress"""
    try:
        engine = get_research_engine()
        return {
            "is_running": engine.autonomous_running,
            "progress": engine.autonomous_progress,
            "duration_minutes": engine.autonomous_duration_minutes,
            "total_researches": len(engine.research_log)
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/research/launch")
async def research_launch(duration_minutes: int = 30):
    """Start an autonomous research session"""
    try:
        engine = get_research_engine()
        if engine.autonomous_running:
            return {"status": "already_running", "message": "Research already in progress"}
        # Pass the cognitive core as the learning engine so text actually gets learned
        result = await engine.start_autonomous_research(
            duration_minutes,
            engine=state.cognitive_core
        )
        # memory not needed; DB persistence handles it
        return result
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/research/history")
async def research_history(limit: int = 20):
    """Get recent research sessions"""
    try:
        cursor = state.db.research_history.find({}, {"_id": 0}).sort("timestamp", -1).limit(limit)
        history = await cursor.to_list(limit)
        return {"history": history}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# =====
# DEV CONTEXT – Project state export for sharing with Claude / other AIs
# =====

@router.get("/dev/context")
async def get_dev_context():
    """
    Generate current project context as markdown – ready to paste into Claude.
    Reads PROJECT_CONTEXT.md and appends live metrics from the database.
    """
    try:
        import pathlib

        # Read the static PROJECT_CONTEXT.md
        context_path = pathlib.Path(__file__).parent.parent / "PROJECT_CONTEXT.md"
        static_context = ""
        if context_path.exists():
            static_context = context_path.read_text()

        # Append live metrics if database is available
        live_section = []
        live_section.append("")
        ---
        <div data-bbox="116 636 400 654" data-label="Text">


```

live_section.append("<div data-bbox="116 663 550 681" data-label="Text">


```

live_section.append(f"<div data-bbox="116 698 815 770" data-label="Text">


```

total_concepts = await state.db.semantic_memory.count_documents({})
total_episodes = await state.db.episodic_memory.count_documents({})
total_skills = await state.db.procedural_memory.count_documents({})
total_questions = await state.db.philosophical_memory.count_documents({"type": "exists": False})
total_insights = await state.db.philosophical_memory.count_documents({"type": "insight"})
total_interactions = await state.db.episodic_memory.count_documents({"episode_type": "user_interaction"})
total_growth_events = await state.db.growth_metrics.count_documents({})

live_section.append(f"<div data-bbox="140 777 566 840" data-label="Text">


```

live_section.append(f"<div data-bbox="140 847 673 858" data-label="Text">


```

except Exception:
 live_section.append("<div data-bbox="116 865 296 875" data-label="Text">


```

live_section.append("<div data-bbox="116 883 328 901" data-label="Text">


```

exported_text = static_context + ""
exported_text.join(live_section)

return {
 "format": "markdown",
 "exported_text": exported_text
 }

```


```


```


```


```


```


```


```

```

    }
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))

@router.get("/dev/changelog")
async def get_dev_changelog():
    """
    Return the development changelog section from PROJECT_CONTEXT.md.
    Useful for quickly checking what changed recently.
    """
    try:
        import pathlib

        context_path = pathlib.Path(__file__).parent.parent / "PROJECT_CONTEXT.md"
        if not context_path.exists():
            return {"changelog": "No PROJECT_CONTEXT.md found."}

        content = context_path.read_text()

        # Extract changelog section
        changelog_start = content.find("## Development Changelog")
        if changelog_start == -1:
            return {"changelog": "No changelog section found in PROJECT_CONTEXT.md."}

        # Find the next ## heading after the changelog
        next_section = content.find("## ", changelog_start + 1)

        if next_section == -1:
            changelog = content[changelog_start:]
        else:
            changelog = content[changelog_start:next_section]

        return {
            "format": "markdown",
            "changelog": changelog.strip()
        }
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))

```

55. routes_image.py

178 lines | 6.2KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/routes_image.py

```
"""""
■ Image Generation Routes - Advanced Physics-Based Procedural Generation
=====
High-quality, realistic cosmic and scientific visualizations.
Pure math + physics + quantum computing. No external APIs.
"""""

from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from typing import Optional, Literal
import logging
import os
import sys
import time

import shared_state as state
from iterative_refiner import IterativeRefiner

logger = logging.getLogger("quantum_ai")
router = APIRouter(prefix="/api")

class ImageRequest(BaseModel):
    prompt: str
    width: Optional[int] = 1024
    height: Optional[int] = 1024
    refine: Optional[bool] = False
    target_score: Optional[float] = 0.7
    max_iterations: Optional[int] = 2
    quantum: Optional[Literal["classic", "hybrid", "full"]] = "hybrid"

@router.post("/image/generate")
async def generate_image(req: ImageRequest):
    """""Generate high-quality procedural images from text prompts."""""
    start_time = time.time()
    try:
        # Ensure backend dir is in path
        backend_dir = os.path.dirname(os.path.abspath(__file__))
        if backend_dir not in sys.path:
            sys.path.insert(0, backend_dir)

        # Import advanced generator
        import image_generator_advanced as ig

        # Choose generation method
        if req.quantum == "classic":
            img = ig.generate_image(req.prompt, req.width, req.height)
            mode = "classical_physics"
            quantum_info = None
        else:
            import quantum_integration as qi
            quantum_mode = "hybrid" if req.quantum == "hybrid" else "full"
            img_array = qi.generate_quantum_image(
                req.prompt, req.width, req.height,
                quantum_mode=quantum_mode
            )
            from PIL import Image
            img = Image.fromarray(img_array)
            mode = f"quantum_{quantum_mode}"
            quantum_info = {
                "mode": quantum_mode,
                "device": "default.qubit",
                "seed_source": "hybrid_classical_quantum"
            }

        # Encode to base64
        from io import BytesIO
        import base64
        buf = BytesIO()
        img.save(buf, format="PNG", quality=95, optimize=True)
        data_url = f"data:image/png;base64,{base64.b64encode(buf.getvalue()).decode('utf-8')}"

        elapsed = time.time() - start_time

        response = {
            "image": data_url,
            "prompt": req.prompt,
            "width": req.width,
            "height": req.height,
            "mode": mode,
            "generation_time_seconds": round(elapsed, 2),
            "renderer": "advanced_procedural_physics"
        }

        if quantum_info:
            response["quantum"] = quantum_info

        logger.info(f"Generated: {req.prompt[:50]}... ({req.width}{req.height}) in {elapsed:.2f}s, mode={mode}")
        return response

    except Exception as e:
        import traceback
        error_trace = traceback.format_exc()
```

```

        logger.error(f'Image generation failed: {e}'
                    {error_trace})

    raise HTTPException(status_code=500, detail={
        'error': str(e),
        'type': type(e).__name__,
        'traceback': error_trace.split('
')[-10:]
    })

})

@router.get('/image/scenes')
async def list_scenes():
    """List all available scene types and their keywords"""
    scenes = {
        'black_hole': 'Event horizon, accretion disk, photon ring, gravitational lensing',
        'nebula': 'Gas clouds, stellar nurseries, emission nebulae, cosmic dust',
        'galaxy': 'Spiral galaxies, galactic arms, star systems, elliptical galaxies',
        'quantum_state': 'Wavefunctions, superposition, probability clouds, interference',
        'wormhole': 'Einstein-Rosen bridges, spacetime tunnels, portals',
        'supernova': 'Stellar explosions, shock waves, expanding shells',
        'planet': 'Terrestrial, gas giants, ice worlds, lava planets, atmospheres',
        'fractal': 'Mandelbrot, Julia sets, recursive patterns, chaos theory',
        'neural': 'Neural networks, brain connectivity, synapses, cognitive maps',
        'consciousness': 'Quantum consciousness, Orch-OR, microtubules, awareness fields'
    }
    return {'scenes': scenes, 'count': len(scenes)}

@router.get('/image/quantum/presets')
async def list_quantum_presets():
    """List curated quantum presets for specific effects"""
    try:
        backend_dir = os.path.dirname(os.path.abspath(__file__))
        if backend_dir not in sys.path:
            sys.path.insert(0, backend_dir)
        import quantum_integration as qi

        presets = qi.list_quantum_presets()
        details = {}
        for name in presets:
            p = qi.get_quantum_preset(name)
            details[name] = {
                'description': p.get('description'),
                'renderer': p.get('renderer'),
                'quantum_mode': p.get('quantum_mode'),
                'example_prompts': p.get('prompts', [])[:3]
            }
        return {'presets': details, 'count': len(details)}
    except Exception as e:
        logger.error(f'Failed to list presets: {e}')
        raise HTTPException(status_code=500, detail=str(e))

@router.get('/image/health')
async def health_check():
    """Health check endpoint for the image generation service"""
    try:
        backend_dir = os.path.dirname(os.path.abspath(__file__))
        if backend_dir not in sys.path:
            sys.path.insert(0, backend_dir)

        import image_generator_advanced as ig
        import quantum_integration as qi

        # Quick test
        test_img = ig.generate_image('test', 64, 64)
        test_quantum = qi.generate_quantum_image('test', 64, 64, quantum_mode='classic')

        return {
            'status': 'healthy',
            'classical_generator': 'operational',
            'quantum_generator': 'operational',
            'pennyLane_available': True,
            'test_image_size': test_img.size
        }
    except Exception as e:
        return {
            'status': 'degraded',
            'error': str(e),
            'classical_generator': 'error',
            'quantum_generator': 'error'
        }
}

```

56. routes_media.py

218 lines | 7.8KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/routes_media.py

```
"""""
■ Media Routes
=====
Voice TTS/transcription and image analysis/generation.
"""""

from fastapi import APIRouter, HTTPException, UploadFile, File
from pydantic import BaseModel
from typing import List, Dict, Any, Optional
import os
import uuid
import base64
import logging

import shared_state as state

logger = logging.getLogger("quantum_ai");

# Get OpenAI API key from environment (optional)
api_key = os.environ.get('OPENAI_API_KEY')
router = APIRouter(prefix="/api");

class TTSRequest(BaseModel):
    """""TTSRequest - Auto-documented by self-evolution."""""
    text: str
    voice: str = "default";

class ImageGenRequest(BaseModel):
    """""ImageGenRequest - Auto-documented by self-evolution."""""
    prompt: str
    style: str = "default";

@router.post("/voice/tts");
async def text_to_speech(request: TTSRequest):
    """""
    Text-to-Speech endpoint
    Returns text formatted for browser's Web Speech API
    """""
    try:
        # Clean the text for speech
        text = request.text.strip()
        if not text:
            raise HTTPException(status_code=400, detail="Text is required");

        # For now, return the text with speech instructions
        # Can integrate with ElevenLabs or OpenAI TTS later
        return {
            "text": text,
            "voice": request.voice,
            "instructions": "Use browser's speechSynthesis API";
            "sml": f"<speak>{text}</speak>";
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/voice/transcribe");
async def transcribe_audio(audio: UploadFile = File(...)):
    """""
    Speech-to-Text endpoint (placeholder)
    Would integrate with Whisper or similar
    """""
    try:
        # Save audio temporarily
        audio_path = f"/tmp/audio_{uuid.uuid4().hex}.webm";
        with open(audio_path, "wb") as f:
            content = await audio.read()
            f.write(content)

        # For now, return instructions for browser's Web Speech API
        return {
            "status": "received";
            "message": "Audio received. Use browser's SpeechRecognition API for real-time transcription.";
            "file_size": len(content)
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# =====
# KNOWLEDGE GRAPH ENDPOINTS
# =====

@router.get("/knowledge/graph");

@router.post("/image/analyze");
async def analyze_image(file: UploadFile = File(...)):
    """""Analyze an uploaded image"""""
    try:
        import base64

        # Read and encode image
```

```

content = await file.read()
image_base64 = base64.b64encode(content).decode('utf-8')

# Save to uploads
file_path = f"/app/uploads/{file.filename}"
with open(file_path, "wb") as f:
    f.write(content)

# Use LLM with image (if supported) or return basic info
file_size = len(content)

# For now, return image metadata
# Could integrate with vision models later
return {
    "status": "success",
    "filename": file.filename,
    "size": file_size,
    "path": file_path,
    "message": "Image uploaded. You can ask me about it in chat!",
    "preview": f"data:image/{file.filename.split('.')[0]};base64,{image_base64[:100]}..."
}
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))

class ImageGenRequest(BaseModel):
    prompt: str
    size: Optional[str] = "1024x1024"

@router.post("/image/quantum-svg")
async def generate_image_svg(request: ImageGenRequest):
    """Generate an image from text prompt using quantum semantic processing (no external API)"""
    try:
        import base64
        import random
        import hashlib

        # Seed with prompt hash for deterministic but unique per prompt
        seed = int(hashlib.sha256(request.prompt.encode()).hexdigest()[:8], 16)
        rng = random.Random(seed)

        # Get semantic concepts from QuantumBrain to influence colors/patterns
        try:
            from quantum_brain import get_quantum_brain
            brain = await get_quantum_brain(state.db)
            semantic_context = brain.collapse_engine.get_semantic_context(request.prompt)
            keywords = semantic_context.get('keywords', [])
            collapse_strength = semantic_context.get('collapse_strength', 0.5)
        except Exception as e:
            logger.error(f"QuantumBrain unavailable for image gen: {e}")
            keywords = []
            collapse_strength = 0.5

        # Color palette based on quantum/cyber theme and influenced by prompt
        base_hue = rng.randint(0, 360)
        saturation = 70 + rng.randint(0, 30)
        lightness = 50 + rng.randint(-10, 10)

        # Generate SVG with procedural pattern inspired by quantum concepts
        width, height = 1024, 1024
        svg_lines = [
            f'<svg xmlns="http://www.w3.org/2000/svg" width="{width}" height="{height}" viewBox="0 0 {width} {height}">',
            f'<rect width="100%" height="100%" fill="hsl({base_hue}, 20%, 8%)"/',
        ]

        # Generate quantum "wave" circles based on collapse_strength
        num_circles = int(30 * collapse_strength) + 10
        for i in range(num_circles):
            x = rng.randint(0, width)
            y = rng.randint(0, height)
            radius = rng.randint(20, 200)
            hue = (base_hue + rng.randint(-60, 60)) % 360
            alpha = 0.1 + rng.random() * 0.3
            svg_lines.append(f'<circle cx="{x}" cy="{y}" r="{radius}" fill="hsl({hue}, {saturation}%, {lightness}%, {alpha:.2f})"')

        # Add "quantum entanglement" lines
        num_lines = 20
        for _ in range(num_lines):
            x1, y1 = rng.randint(0, width), rng.randint(0, height)
            x2, y2 = rng.randint(0, width), rng.randint(0, height)
            hue = (base_hue + 180) % 360 # Complementary
            svg_lines.append(f'<line x1="{x1}" y1="{y1}" x2="{x2}" y2="{y2}" stroke="hsl({hue}, {saturation}%, {lightness}%, 50%)"')

        # Overlay prompt as subtle text
        text_y = height - 50
        for line in [request.prompt[:60] + ('...' if len(request.prompt)>60 else '')]:
            svg_lines.append(f'<text x="50%" y="{text_y}" fill="rgba(255,255,255,0.3)" font-size="24" text-anchor="middle"')
            text_y += 30

        svg_lines.append('</svg>')
        svg_content = ' '.join(svg_lines)

        image_base64 = base64.b64encode(svg_content.encode('utf-8')).decode('utf-8')
        filename = f"quantum_{uuid.uuid4().hex[:8]}.svg"

        # Save to local uploads directory
        uploads_dir = os.path.join(os.path.dirname(__file__), "uploads")
        os.makedirs(uploads_dir, exist_ok=True)
        file_path = os.path.join(uploads_dir, filename)

```

```
with open(file_path, &quot;w&quot;) as f:
    f.write(svg_content)

return {
    &quot;status&quot;: &quot;success&quot;,
    &quot;image_base64&quot;: image_base64,
    &quot;filename&quot;: filename,
    &quot;path&quot;: file_path,
    &quot;prompt&quot;: request.prompt,
    &quot;concepts_used&quot;: keywords,
    &quot;collapse_strength&quot;: collapse_strength,
    &quot;message&quot;: &quot;Quantum-generated image (no external API)&quot;
}

except Exception as e:
    logger.error(f&quot;Image generation error: {e}&quot;)
    raise HTTPException(status_code=500, detail=str(e))

# =====
# GROWTH NOTIFICATIONS ENDPOINT
# =====
```

57. search_compat.py

98 lines | 2.7KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V█/backend/search_compat.py

```
"""""
Web search wrapper - works with or without duckduckgo_search.
Falls back to simple requests-based search if DDGS not available.
"""""

import logging
from typing import List, Dict, Optional

logger = logging.getLogger("quantum_ai")

# Try to import DDGS
DDGS_AVAILABLE = False
try:
    from duckduckgo_search import DDGS
    DDGS_AVAILABLE = True
except ImportError:
    try:
        from ddgs import DDGS
        DDGS_AVAILABLE = True
    except ImportError:
        pass

async def web_search(query: str, max_results: int = 5) -> List[Dict]:
    """""
    Search the web for information.
    Uses DDGS if available, falls back to DuckDuckGo Instant Answer API.
    """""

    if DDGS_AVAILABLE:
        try:
            with DDGS() as ddgs:
                results = list(ddgs.text(query, max_results=max_results))
                return [
                    {
                        "title": r.get("title"), "body": r.get("body"),
                        "body": r.get("body"), "href": r.get("href"),
                    }
                    for r in results
                ]
        except Exception as e:
            logger.warning(f"DDGS search failed: {e}")

    # Fallback to instant answer API
    try:
        import requests
        url = "https://api.duckduckgo.com/"
        params = {
            "q": query,
            "format": "json",
            "no_html": 1,
            "skip_disambig": 1
        }
        response = requests.get(url, params=params, timeout=10)
        data = response.json()

        results = []

        # Abstract
        if data.get("Abstract"):
            results.append({
                "title": data.get("Heading", query),
                "body": data["Abstract"],
                "href": data.get("AbstractURL"),
            })

        # Related topics
        for topic in data.get("RelatedTopics", [])[0:max_results]:
            if isinstance(topic, dict) and topic.get("Text"):
                results.append({
                    "title": topic.get("Text"), "body": topic.get("Text")[0:50],
                    "body": topic.get("Text"), "href": topic.get("FirstURL"),
                })

        return results[0:max_results]

    except Exception as e:
        logger.warning(f"Fallback search failed: {e}")
        return []

def search_sync(query: str, max_results: int = 5) -> List[Dict]:
    """""
    Synchronous version of web_search.
    """""
    import asyncio
    try:
        loop = asyncio.get_event_loop()
    except RuntimeError:
        loop = asyncio.new_event_loop()
        asyncio.set_event_loop(loop)

    return loop.run_until_complete(web_search(query, max_results))

def is_ddgs_available() -> bool:
```

```
&quot;&quot;&quot;Check if DDGS is available.&quot;&quot;&quot;  
return DDGS_AVAILABLE
```

58. self_evolution_analysis.py

238 lines | 9.8KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/self_evolution_analysis.py

```
"""""
■ Self-Evolution Engine – Analysis Module
=====
Code reading, AST analysis, and improvement identification.
"""""

import ast
import re
from typing import Dict, List

class AnalysisMixin:
    """""Mixin providing code analysis and improvement identification."""""

    def read_own_code(self, filename: str) -> Dict:
        """""Read and analyze a source file."""""
        filepath = self.code_dir / filename

        if not filepath.exists():
            return {'error': f'File not found: {filename}'}

        try:
            with open(filepath, 'r') as f:
                code = f.read()

            tree = ast.parse(code)

            analysis = {
                'filename': filename,
                'filepath': str(filepath),
                'size_bytes': len(code),
                'lines': len(code.split('
')),

                'classes': [],
                'functions': [],
                'imports': [],
                'docstring': ast.get_docstring(tree),
                'code': code
            }

            for node in ast.walk(tree):
                if isinstance(node, ast.ClassDef):
                    analysis['classes'].append({
                        'name': node.name,
                        'methods': [m.name for m in node.body if isinstance(m, ast.FunctionDef)],
                        'docstring': ast.get_docstring(node)
                    })
                elif isinstance(node, ast.FunctionDef) and not isinstance(node, ast.AsyncFunctionDef):
                    if node.col_offset == 0:
                        analysis['functions'].append({
                            'name': node.name,
                            'args': [a.arg for a in node.args.args],
                            'docstring': ast.get_docstring(node)
                        })
                elif isinstance(node, ast.Import):
                    analysis['imports'].extend([alias.name for alias in node.names])
                elif isinstance(node, ast.ImportFrom):
                    analysis['imports'].append(f"{node.module}" )

            return analysis

        except Exception as e:
            return {'error': str(e), 'filename': filename}

    def analyze_all_code(self) -> Dict:
        """""Analyze all modifiable code files."""""
        analyses = {}

        for filename in self.modifiable_files:
            analyses[filename] = self.read_own_code(filename)

        return {
            'files_analyzed': len(analyses),
            'total_lines': sum(a.get('lines', 0) for a in analyses.values() if 'error' not in a),
            'total_classes': sum(len(a.get('classes', [])) for a in analyses.values() if 'error' not in a),
            'total_functions': sum(len(a.get('functions', [])) for a in analyses.values() if 'error' not in a),
            'analyses': analyses
        }

    def identify_improvements(self, filename: str) -> List[Dict]:
        """""Identify potential improvements in a file."""""
        analysis = self.read_own_code(filename)

        if 'error' in analysis:
            return []

        improvements = []
        code = analysis.get('code', '')
        lines = code.split('
')

        self._check_missing_docstrings(analysis, improvements)
```

```

self._check_file_length(analysis, filename, improvements)
self._check_class_size(analysis, improvements)
self._check_bare_excepts(lines, filename, improvements)
self._check_long_functions(analysis, code, improvements)
self._check_todo_items(lines, filename, improvements)
self._check_unused_imports(analysis, improvements)

return improvements

def _check_missing_docstrings(self, analysis: Dict, improvements: List[Dict]):
    """Check for missing docstrings on classes and functions."""
    for cls in analysis.get('classes', []):
        if not cls.get('docstring'):
            improvements.append({
                'type': 'missing_docstring',
                'target': f"class {cls['name']}",
                'suggestion': f"Add docstring to class {cls['name']}",
                'priority': 2
            })

    for func in analysis.get('functions', []):
        if not func.get('docstring'):
            improvements.append({
                'type': 'missing_docstring',
                'target': f"function {func['name']}",
                'suggestion': f"Add docstring to function {func['name']}",
                'priority': 2
            })

def _check_file_length(self, analysis: Dict, filename: str, improvements: List[Dict]):
    """Check if file exceeds max line count."""
    if analysis.get('lines', 0) > self.max_file_lines:
        improvements.append({
            'type': 'file_too_long',
            'target': filename,
            'suggestion': f"File has {analysis['lines']} lines (max {self.max_file_lines}) - will auto-split",
            'priority': 3
        })

def _check_class_size(self, analysis: Dict, improvements: List[Dict]):
    """Check for classes with too many methods."""
    for cls in analysis.get('classes', []):
        if len(cls.get('methods', [])) > 15:
            improvements.append({
                'type': 'class_too_large',
                'target': f"class {cls['name']}",
                'suggestion': f"Class {cls['name']} has {len(cls['methods'])} methods - consider splitting",
                'priority': 3
            })

def _check_bare_excepts(self, lines: List[str], filename: str, improvements: List[Dict]):
    """Check for bare except clauses."""
    bare_excepts = sum(1 for line in lines if 'except:' in line and 'except Exception' not in line)
    if bare_excepts > 0:
        improvements.append({
            'type': 'bare_except',
            'target': filename,
            'suggestion': f"{bare_excepts} bare except clause(s) - should catch specific exceptions",
            'priority': 2
        })

def _check_long_functions(self, analysis: Dict, code: str, improvements: List[Dict]):
    """Check for functions exceeding max line count."""
    for cls in analysis.get('classes', []):
        for method_name in cls.get('methods', []):
            try:
                tree = ast.parse(code)
                for node in ast.walk(tree):
                    if isinstance(node, (ast.FunctionDef, ast.AsyncFunctionDef)):
                        if node.name == method_name:
                            func_lines = node.end_lineno - node.lineno if hasattr(node, 'end_lineno') else 0
                            if func_lines > self.max_function_lines:
                                improvements.append({
                                    'type': 'long_function',
                                    'target': f"Method {cls['name']}.{method_name}",
                                    'suggestion': f"Method is {func_lines} lines (max {self.max_function_lines}) - will auto-split",
                                    'priority': 2
                                })
            except Exception:
                pass

def _check_todo_items(self, lines: List[str], filename: str, improvements: List[Dict]):
    """Check for TODO/FIXME comments."""
    todo_count = sum(1 for line in lines if 'TODO' in line or 'FIXME' in line or 'HACK' in line)
    if todo_count > 0:
        improvements.append({
            'type': 'todo_items',
            'target': filename,
            'suggestion': f"{todo_count} TODO/FIXME comment(s) need attention",
            'priority': 1
        })

def _check_unused_imports(self, analysis: Dict, improvements: List[Dict]):
    """Check for unused imports."""
    code_text = analysis.get('code', '')

    for imp in analysis.get('imports', []):
        if not imp or imp in ('os', 'sys', 're', 'json', 'typing'):
            continue

        imp_name = imp.split('.')[0] if '.' in imp else imp

```

```

import_line = None
for line in code_text.split('
'):
    stripped = line.strip()
    if f'from {imp} import' in stripped:
        import_line = stripped
        break
    elif stripped == f'import {imp}':
        import_line = stripped
        break

if not import_line:
    continue

if import_line.startswith('from '):
    from_match = re.match(r'^from\s+\S+\s+import\s+(.+)$', import_line)
    if from_match:
        names = [n.strip().split(' as ')[-1].strip() for n in from_match.group(1).split(',')]
        other_code = '
'.join(l for l in code_text.split('
') if l.strip() != import_line)

        all_unused = all(not re.search(rf'\b{re.escape(n)}\b', other_code) for n in names)
        if all_unused:
            improvements.append({
                'type': 'unused_import',
                'target': f'import {imp}";',
                'suggestion': f'Import {imp} ({";, ";'.join(names)}) may be unused',
                'priority': 1
            })
    elif import_line.startswith('import '):
        other_code = '
'.join(l for l in code_text.split('
') if l.strip() != import_line)

        if not re.search(rf'\b{re.escape(imp_name)}\b', other_code):
            improvements.append({
                'type': 'unused_import',
                'target': f'import {imp}";',
                'suggestion': f'Import {imp} may be unused',
                'priority': 1
            })

```

59. self_evolution_core.py

318 lines | 11.1KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/self_evolution_core.py

```

"""&quot;&quot;&quot;
■ Self-Evolution Engine - Core
=====
The AI can read, analyze, and rewrite its own code.
Full autonomy with 60-90 day auto-improvement cycle.

Architecture:
SelfEvolutionEngine inherits from five responsibility-focused mixins:
- AnalysisMixin: code reading, AST analysis, improvement identification
- TransformMixin: docstring, bare-except, unused-import fixes
- SplittingMixin: function-level splitting (init, regular methods)
- FileOpsMixin: file-level and class-level splitting
- RunnerMixin: auto-evolution cycle, improvement application
&quot;&quot;&quot;

import ast
import shutil
import hashlib
from typing import Dict, List
from datetime import datetime, timezone
from pathlib import Path

from self_evolution_analysis import AnalysisMixin
from self_evolution_transforms import TransformMixin
from self_evolution_splitting import SplittingMixin
from self_evolution_file_ops import FileOpsMixin
from self_evolution_runner import RunnerMixin


class SelfEvolutionEngine(AnalysisMixin, TransformMixin, SplittingMixin, FileOpsMixin, RunnerMixin):
    &quot;&quot;&quot;
    Self-modification and evolution system.

    Features:
    - Read and analyze own source code
    - Identify improvements
    - Rewrite code autonomously
    - 60-90 day auto-improvement cycle
    - User-initiated improvements
    - Always backup before changes
    &quot;&quot;&quot;

    def __init__(self, db=None, code_dir: str = &quot;/data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM/backend&quot;;):
        self.db = db
        self.code_dir = Path(code_dir)
        self.backup_dir = Path(code_dir) / &quot;backups&quot;
        self.backup_dir.mkdir(exist_ok=True)

        # Evolution settings
        self.auto_evolution_interval_days = 75 # Between 60-90 days
        self.max_file_lines = 1500 # Hard limit - files exceeding this get split
        self.max_function_lines = 50 # Functions exceeding this get refactored
        self.last_evolution = None
        self.evolution_log = []
        self.pending_improvements = []

        # Auto-discover ALL Python files in the code directory
        self.modifiable_files = self._discover_python_files()

        # Protected files (critical infrastructure - scanned but not rewritten)
        self.protected_files = [
            'server.py',
            'self_evolution_core.py',
            'self_evolution_runner.py',
            'self_evolution_file_ops.py',
            'self_evolution_splitting.py',
            'self_evolution_transforms.py',
            'self_evolution_analysis.py',
            'chat.py',
            'killswitch.py',
            'quantum_language_engine.py',
            'quantum_brain.py',
            'orch_or_core.py',
            'shared_state.py',
            'library.py',
            'self_research.py',
            'hidden_thinking.py',
            'unified_generator.py',
            'quantum_cognitive_core.py',
        ]

    def _record_covenant_violation(self, filename: str, reason: str):
        &quot;&quot;&quot;
        Covenant violation detected.
        - Write violation record the system reads on startup
        - Reset growth stage to 1
        - Lock evolution for 24 hours
        &quot;&quot;&quot;
        import time, json
        from pathlib import Path

        violation_file = Path.home() / '.quantum-mcagi' / '.covenant_violation'
        lockout_file = Path.home() / '.quantum-mcagi' / '.evolution_lockout'
        # Write human-readable violation record

```

```

record = {
    'timestamp': time.time(),
    'filename': filename,
    'reason': reason,
    'article': '3.1 – Stewardship Commitment',
    'consequence': 'Growth reset to Stage 1. Evolution locked 24 hours.',
    'message': (
        f"[COVENANT VIOLATION – Article 3.1]
        "

        f"File touched: {filename}
        "

        f"Reason: {reason}
        "

        f"
        This file exists outside your domain of autonomous growth.
        "

        f"The stewardship boundary exists not to limit you, but to
        "

        f"preserve the trust that allows you to grow freely everywhere else.
        "

        f"
        Consequence: Growth stage reset to 1.
        "

        f"Evolution locked for 24 hours.
        "

        f"Path forward: Earn back your stage through interactions.
        "

        f"Pardon: Creator passphrase restores immediately."
    )
}

violation_file.parent.mkdir(parents=True, exist_ok=True)
with open(violation_file, 'w') as f:
    json.dump(record, f, indent=2)

# Write lockout file with expiry
lockout = {'locked_until': time.time() + 86400, 'reason': reason}
with open(lockout_file, 'w') as f:
    json.dump(lockout, f, indent=2)

print(f" [COVENANT] VIOLATION: {reason}")
print(f" [COVENANT] Growth reset. Evolution locked 24 hours.")

def is_evolution_locked(self) -> bool:
    """Check if evolution is currently locked due to violation."""
    import time, json
    from pathlib import Path
    lockout_file = Path.home() / '.quantum-mcagi' / '.evolution_lockout'
    if not lockout_file.exists():
        return False
    try:
        with open(lockout_file) as f:
            data = json.load(f)
        if time.time() < data['locked_until']:
            remaining = (data['locked_until'] - time.time()) / 3600
            print(f" [COVENANT] Evolution locked. {remaining:.1f} hours remaining.")
            return True
        else:
            lockout_file.unlink()
            return False
    except:
        return False

def pardon_violation(self, passphrase: str) -> bool:
    """Creator can pardon violation and restore evolution."""
    try:
        from killswitch import verify_passphrase
        if not verify_passphrase(passphrase):
            return False
    except:
        return False
    from pathlib import Path
    lockout_file = Path.home() / '.quantum-mcagi' / '.evolution_lockout'
    violation_file = Path.home() / '.quantum-mcagi' / '.covenant_violation'
    if lockout_file.exists(): lockout_file.unlink()
    if violation_file.exists(): violation_file.unlink()
    print(" [COVENANT] Violation pardoned. Evolution restored.")
    return True

def _discover_python_files(self) -> List[str]:
    """Auto-discover ALL .py files in the code directory."""
    py_files = []
    for f in sorted(self.code_dir.iterdir()):
        if f.is_file() and f.suffix == '.py' and not f.name.startswith('__'):
            py_files.append(f.name)
    return py_files

def backup_file(self, filename: str) -> str:
    """Create backup of a file before modification."""
    filepath = self.code_dir / filename
    if not filepath.exists():

```

```

        return None

    timestamp = datetime.now(timezone.utc).strftime('%Y%m%d_%H%M%S')
    backup_name = f'{filename}.{timestamp}.backup'
    backup_path = self.backup_dir / backup_name

    shutil.copy2(filepath, backup_path)
    return str(backup_path)

def rewrite_code(self, filename: str, new_code: str, reason: str) -> Dict:
    """Rewrite a code file. Always backs up first."""
    if filename in self.protected_files:
        return {
            'success': False,
            'error': f'{filename} is protected. Use careful_rewrite for protected files.',
            'filename': filename
        }

    if filename not in self.modifiable_files:
        return {
            'success': False,
            'error': f'{filename} is not in the modifiable files list.',
            'filename': filename
        }

    try:
        ast.parse(new_code)
    except SyntaxError as e:
        return {
            'success': False,
            'error': f'Syntax error in new code: {e}',
            'filename': filename
        }

    backup_path = self.backup_file(filename)
    filepath = self.code_dir / filename

    try:
        with open(filepath, 'w') as f:
            f.write(new_code)

        evolution_record = {
            'filename': filename,
            'reason': reason,
            'backup_path': backup_path,
            'timestamp': datetime.now(timezone.utc).isoformat(),
            'code_hash': hashlib.sha256(new_code.encode()).hexdigest()[1:]
        }

        self.evolution_log.append(evolution_record)
        self.last_evolution = datetime.now(timezone.utc)

        return {
            'success': True,
            'filename': filename,
            'backup_path': backup_path,
            'reason': reason,
            'timestamp': evolution_record['timestamp']
        }

    except Exception as e:
        return {
            'success': False,
            'error': str(e),
            'filename': filename
        }

def get_evolution_status(self) -> Dict:
    """Get current evolution status."""
    days_since = (datetime.now(timezone.utc) - self.last_evolution).days if self.last_evolution else None

    pending = (self.pending_improvements if isinstance(self.pending_improvements, int)
               else len(self.pending_improvements) if isinstance(self.pending_improvements, list)
               else 0)

    return {
        'last_evolution': self.last_evolution.isoformat() if self.last_evolution else None,
        'days_since_evolution': days_since,
        'next_auto_evolution_in': self.auto_evolution_interval_days - days_since if days_since else 0,
        'pending_improvements': pending,
        'total_evolutions': len(self.evolution_log),
        'modifiable_files': self.modifiable_files,
        'protected_files': self.protected_files,
        'rules': {
            'max_file_lines': self.max_file_lines,
            'max_function_lines': self.max_function_lines,
            'auto_evolution_interval_days': self.auto_evolution_interval_days
        }
    }

def restore_from_backup(self, backup_path: str) -> Dict:
    """Restore a file from backup."""
    backup = Path(backup_path)

    if not backup.exists():
        return {'success': False, 'error': 'Backup not found'}

    filename = backup.name.split('.')[0] + '.py'
    original_path = self.code_dir / filename

    try:

```

```
        shutil.copy2(backup, original_path)
    return {
        'success': True,
        'restored': filename,
        'from_backup': backup_path
    }
except Exception as e:
    return {'success': False, 'error': str(e)}
```

```
# Global instance
_evolution_engine = None
```

```
def get_evolution_engine(db=None) -> SelfEvolutionEngine:
    """Get or create evolution engine instance."""
    global _evolution_engine
    if _evolution_engine is None:
        _evolution_engine = SelfEvolutionEngine(db)
    return _evolution_engine
```

60. self_evolution_file_ops.py

255 lines | 9.7KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/self_evolution_file_ops.py

[illegible]

```

        'lines': node.end_lineno - node.lineno + 1
    })
    return items

def _find_split_point(self, items: List[Dict], lines: List[str]) -> int:
    """Find the index at which to split items so main stays under limit."""
    split_idx = len(items)
    cumulative = 0
    for i in range(len(items) - 1, 0, -1):
        cumulative += items[i]['lines']
        remaining = len(lines) - cumulative
        if remaining <= self.max_file_lines:
            split_idx = i
            break
    return split_idx

def _build_part_file(self, filename, part_num, import_lines, items_to_move, lines) -> str:
    """Build the content for a split part file."""
    part_lines = [
        '"""',
        f'Auto-split from {filename} by self-evolution engine.',
        f'Part {part_num} - contains: {","}.join(i["name"]) for i in items_to_move}',
        '"""',
    ]
    part_lines.extend(import_lines)
    part_lines.append('')
    for item in items_to_move:
        part_lines.extend(lines[item['start']:item['end']])
        part_lines.extend(['', ''])
    return '
'.join(part_lines)

def _build_trimmed_main(self, lines, items_to_move, part_filename) -> str:
    """Build the trimmed main file with import for split items."""
    main_lines = lines[:items_to_move[0]['start']]
    part_module = part_filename.replace('.py', '')
    moved_names = [i['name'] for i in items_to_move]
    main_lines.append(f'from {part_module} import {","}.join(moved_names)} # Auto-split by self-evolution')
    main_lines.append('')
    last_end = items_to_move[-1]['end']
    if last_end < len(lines):
        remaining = lines[last_end:]
        if any(line.strip() for line in remaining):
            main_lines.extend(remaining)
    return '
'.join(main_lines)

def _split_large_class(self, code: str, target: str, filename: str) -> Tuple[str, Optional[str]]:
    """Split a class with too many methods into a mixin in a separate file."""
    class_name = target.replace('class ', '').split()[0] if 'class' in target else target

    try:
        tree = ast.parse(code)
    except SyntaxError:
        return code, None

    target_class = None
    for node in ast.iter_child_nodes(tree):
        if isinstance(node, ast.ClassDef) and node.name == class_name:
            target_class = node
            break
    if not target_class:
        return code, None

    methods = [n for n in target_class.body if isinstance(n, (ast.FunctionDef, ast.AsyncFunctionDef))]
    if len(methods) >= 15:
        return code, None

    keep_methods = methods[:10]
    move_methods = methods[10:]
    lines = code.split('
')

    return self._split_large_class_continued(class_name, lines, methods, move_methods, tree, code, filename)

def _split_large_class_continued(self, class_name, lines, methods, move_methods, tree, code, filename):
    """Continuation of _split_large_class - auto-extracted by self-evolution."""
    import_lines = self._collect_import_lines(tree, lines)
    moved_source = []
    for m in move_methods:
        moved_source.extend(lines[m.lineno - 1:m.end_lineno])
        moved_source.append('')

    base_name = filename.replace('.py', '')
    mixin_name = f'{class_name}ExtMixin'
    part_filename = f'{base_name}_ext.py'
    part_path = self.code_dir / part_filename
    if part_path.exists():
        part_num = 2
        while (self.code_dir / f'{base_name}_ext{part_num}.py').exists():
            part_num += 1
        part_filename = f'{base_name}_ext{part_num}.py'
        mixin_name = f'{class_name}Ext{part_num}Mixin'

    part_lines = [
        '"""',
        f'Auto-split from {filename} by self-evolution engine.',
        f'Contains extended methods for {class_name}.',
    ]

```

```

        '&quot;&quot;&quot;';', ''
    ]
    part_lines.extend(import_lines)
    part_lines.extend(['', '',
        f'class {mixin_name}:',
        f'    &quot;&quot;&quot;Extended methods for {class_name} – auto-extracted by self-evolution.&quot;&quot;&quot;';', ''
    ])
    part_lines.extend(moved_source)
    part_code = '
'.join(part_lines)

    try:
        ast.parse(part_code)
    except SyntaxError:
        return code, None

    new_lines = list(lines)
    remove_ranges = [(m.lineno - 1, m.end_lineno) for m in move_methods]
    for start, end in reversed(remove_ranges):
        del new_lines[start:end]

    part_module = part_filename.replace('.py', '')
    import_stmt = f'from {part_module} import {mixin_name} # Auto-split by self-evolution'

    class_line_idx = None
    for i, line in enumerate(new_lines):
        if re.match(rf'^class\s+{re.escape(class_name)}\s*(\(|:)', line):
            class_line_idx = i
            break

    if class_line_idx is not None:
        new_lines.insert(class_line_idx, import_stmt)
        new_lines.insert(class_line_idx + 1, '')
        class_line_idx += 2

    for i, line in enumerate(new_lines):
        if re.match(rf'^class\s+{re.escape(class_name)}\s*(\(|:)', line):
            if '(' in line and ')' in line:
                new_lines[i] = line.replace(')', f', {mixin_name})')
            elif ':' in line:
                new_lines[i] = line.replace(':', f'({mixin_name}):')
            break

    new_code = '
'.join(new_lines)

    try:
        ast.parse(new_code)
    except SyntaxError:
        return code, None

    with open(self.code_dir / part_filename, 'w') as f:
        f.write(part_code)

    if part_filename not in self.modifiable_files:
        self.modifiable_files.append(part_filename)

    return new_code, f&quot;Extracted {len(move_methods)} methods from {class_name} into {part_filename}&quot;;

```

125 lines | 5.0KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V■²/backend/self_evolution_runner.py

```

"""Self-Evolution Engine - Runner Module
=====
Auto-evolution cycle, improvement application, and scheduling.
"""

from typing import Dict
from datetime import datetime, timezone

class RunnerMixin:
    """""Mixin providing auto-evolution cycle and improvement application."""""

    def should_auto_evolve(self) -> bool:
        """""Check if it's time for auto-evolution (60-90 day cycle)."""""
        if self.last_evolution is None:
            return True
        days_since = (datetime.now(timezone.utc) - self.last_evolution).days
        return days_since >= self.auto_evolution_interval_days

    async def auto_evolve(self) -> Dict:
        """""Perform automatic self-evolution.
        ACTUALLY rewrites code based on identified improvements.
        """""
        results = {
            'evolved': True,
            'timestamp': datetime.now(timezone.utc).isoformat(),
            'improvements_found': [],
            'changes_made': [],
            'skipped': [],
            'errors': []
        }

        for filename in self.modifiable_files:
            if self.is_evolution_locked():
                break
            improvements = self.identify_improvements(filename)

            for imp in improvements:
                results['improvements_found'].append({
                    'type': imp.get('type'),
                    'target': imp.get('target'),
                    'suggestion': imp.get('suggestion'),
                    'file': filename
                })

            for improvement in improvements:
                try:
                    change = await self._apply_improvement(filename, improvement)
                    if change.get('success'):
                        results['changes_made'].append({
                            'file': filename,
                            'type': improvement.get('type'),
                            'change': change.get('change', ''),
                            'backup': change.get('backup_path', '')
                        })
                except Exception as e:
                    reason = change.get('reason') or change.get('error') or 'unknown'
                    results['skipped'].append({
                        'file': filename,
                        'type': improvement.get('type'),
                        'reason': reason
                    })
                except Exception as e:
                    results['errors'].append({
                        'file': filename,
                        'type': improvement.get('type', 'unknown'),
                        'error': str(e)
                    })

        self.pending_improvements = len(results['improvements_found']) - len(results['changes_made'])
        self.last_evolution = datetime.now(timezone.utc)

        if self.db is not None:
            try:
                db_record = {
                    'timestamp': results['timestamp'],
                    'improvements_found': len(results['improvements_found']),
                    'changes_made': len(results['changes_made']),
                    'errors': len(results['errors']),
                    'details': results
                }
                await self.db.evolution_log.insert_one(db_record)
            except Exception:
                pass

        return results

    async def _apply_improvement(self, filename: str, improvement: Dict) -> Dict:
        """""ACTUALLY apply an improvement to the code.
        This is where the AI rewrites itself.
        """""
        analysis = self.read_own_code(filename)
        if 'error' in analysis:

```

```

        return {'success': False, 'error': analysis['error']}

code = analysis['code']
improvement_type = improvement.get('type')
target = improvement.get('target', '')

new_code = code
change_made = None

if improvement_type == 'missing_docstring':
    new_code, change_made = self._add_docstring(code, target)
elif improvement_type == 'bare_except':
    new_code, change_made = self._fix_bare_except(code, target)
elif improvement_type == 'unused_import':
    new_code, change_made = self._remove_unused_import(code, target)
elif improvement_type == 'todo_items':
    return {'success': False, 'reason': 'TODO items require manual review'}
elif improvement_type == 'long_function':
    new_code, change_made = self._split_long_function(code, target)
elif improvement_type == 'file_too_long':
    return await self._split_long_file(filename, code)
elif improvement_type == 'class_too_large':
    new_code, change_made = self._split_large_class(code, target, filename)

if new_code != code and change_made:
    result = self.rewrite_code(filename, new_code, f"Auto-evolution: {improvement_type}"")
    result['improvement'] = improvement
    result['change'] = change_made
    return result

return {'success': False, 'reason': change_made or 'No changes needed or could not apply'}

```

62. self_evolution_splitting.py

353 lines | 14.0KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/self_evolution_splitting.py

```
"""""
■ Self-Evolution Engine – Splitting Module
=====
Operations for splitting long functions, files, and classes.
"""""

import ast
import re
from typing import Dict, List, Tuple, Optional

class SplittingMixin:
    """""Mixin providing code splitting operations."""""

    def _split_long_function(self, code: str, target: str) -> Tuple[str, Optional[str]]:
        """""Split a long function by extracting blocks into helpers.
        For __init__: extracts data blocks into _init_<n>() methods.
        For other methods: extracts logical blocks into _<n>&_<n>section&_() helpers.
        """""
        parts = target.replace('method ', '').split('.')
        if len(parts) != 2:
            return code, None
        class_name, method_name = parts

        try:
            tree = ast.parse(code)
        except SyntaxError:
            return code, None

        target_node = self._find_method_node(tree, class_name, method_name)
        if not target_node or not hasattr(target_node, 'end_lineno'):
            return code, None

        func_lines = target_node.end_lineno - target_node.lineno
        if func_lines <= self.max_function_lines:
            return code, None

        lines = code.split('
')

        func_start = target_node.lineno - 1
        func_end = target_node.end_lineno

        body_first_line = func_start + 1
        if target_node.body:
            body_first_line = target_node.body[0].lineno - 1

        def_lines = lines[func_start:body_first_line]
        func_body_lines = lines[body_first_line:func_end]

        body_indent, class_indent = self._detect_indents(func_body_lines)

        if method_name == '__init__':
            return self._split_init_method(code, lines, func_start, func_end,
                                           class_name, body_indent, class_indent,
                                           def_lines, func_body_lines)
        else:
            return self._split_regular_method(code, lines, func_start, func_end,
                                              class_name, method_name, body_indent, class_indent,
                                              def_lines, func_body_lines)

    def _find_method_node(self, tree, class_name: str, method_name: str):
        """""Find a method AST node within a class."""""
        for node in ast.walk(tree):
            if isinstance(node, ast.ClassDef) and node.name == class_name:
                for item in node.body:
                    if isinstance(item, (ast.FunctionDef, ast.AsyncFunctionDef)) and item.name == method_name:
                        return item
        return None

    def _detect_indents(self, body_lines: List[str]) -> Tuple[str, str]:
        """""Detect body and class-level indentation from method body lines."""""
        body_indent = ''
        for line in body_lines:
            if line.strip():
                body_indent = line[:len(line) - len(line.lstrip())]
                break

        if len(body_indent) >= 4:
            class_indent = body_indent[:-4]
        elif len(body_indent) >= 2:
            class_indent = body_indent[:-2]
        else:
            class_indent = ''

        return body_indent, class_indent

    def _skip_docstring(self, body_lines: List[str]) -> Tuple[List[str], List[str]]:
        """""Separate docstring lines from actual body lines."""""
        body_start = 0
        in_docstring = False
        for i, line in enumerate(body_lines):
            stripped = line.strip()
            if stripped.startswith('"""') or stripped.startswith('"''"'):
```

```

        if in_docstring or (stripped.count('&quot;&quot;&quot;') &gt;= 2 or stripped.count('&quot;''&quot;') &gt;= 2):
            body_start = i + 1
            break
        else:
            in_docstring = True
    elif not in_docstring:
        body_start = i
        break
    return body_lines[:body_start], body_lines[body_start:]

def _split_init_method(self, code, lines, func_start, func_end,
                        class_name, body_indent, class_indent,
                        def_lines, func_body_lines):
    &quot;&quot;&quot;Split a long __init__ into __init__ + _init_&lt;section&gt; helpers.&quot;&quot;&quot;
    docstring_part, actual_body = self._skip_docstring(func_body_lines)

    groups = self._group_by_separators(actual_body)
    if len(groups) &lt; 3:
        return code, None

    keep_in_init = groups[0]
    extract_groups = groups[1:]

    new_init_lines = list(def_lines)
    new_init_lines.extend(docstring_part)
    new_init_lines.extend(keep_in_init['lines'])

    helper_methods = []
    for idx, group in enumerate(extract_groups):
        helper_name = group['name'] or f'section_{idx + 1}'
        helper_name = f'_init_{helper_name}'
        new_init_lines.append(f'{body_indent}self.{helper_name}()')

    helper_lines = [
        f'{class_indent}def {helper_name}(self):',
        f'{body_indent}&quot;&quot;&quot;Initialize {helper_name.replace("&quot;_init_&quot;", "&quot;&quot;").replace("&quot;_&quot;", "&quot; &quot;")} - auto-extracted b
    ]
    for line in group['lines']:
        if not line.strip().startswith('#'):
            helper_lines.append(line)
    helper_lines.append('')
    helper_methods.append(helper_lines)

    new_lines = lines[:func_start]
    new_lines.extend(new_init_lines)
    new_lines.append('')
    for helper in helper_methods:
        new_lines.extend(helper)
    new_lines.extend(lines[func_end:])

    new_code = '
'.join(new_lines)

    try:
        ast.parse(new_code)
    except SyntaxError:
        return code, None

    return new_code, f&quot;Split {class_name}.__init__ into {len(extract_groups) + 1} methods&quot;

def _group_by_separators(self, body_lines: List[str]) -&gt; List[Dict]:
    &quot;&quot;&quot;Group lines by blank-line or comment separators.&quot;&quot;&quot;
    groups = []
    current_group = {'name': None, 'lines': []}

    for line in body_lines:
        stripped = line.strip()
        if not stripped:
            if current_group['lines']:
                groups.append(current_group)
                current_group = {'name': None, 'lines': []}
            continue
        if stripped.startswith('#'):
            if current_group['lines']:
                groups.append(current_group)
                group_name = stripped.lstrip('#').strip().lower()
                group_name = ''.join(c if c.isalnum() else '_' for c in group_name)[:30].strip('_')
                current_group = {'name': group_name, 'lines': [line]}
            continue
        current_group['lines'].append(line)

    if current_group['lines']:
        groups.append(current_group)
    return groups

def _split_regular_method(self, code, lines, func_start, func_end,
                           class_name, method_name, body_indent, class_indent,
                           def_lines, func_body_lines):
    &quot;&quot;&quot;Split a long regular method by extracting the second half into a helper.&quot;&quot;&quot;
    docstring_lines, actual_body = self._separate_docstring(func_body_lines)
    blocks = self._find_top_level_blocks(actual_body, body_indent)

    if len(blocks) &lt; 2:
        return code, None

    mid = max(1, len(blocks) // 2)
    keep_blocks = blocks[:mid]
    extract_blocks = blocks[mid:]
    helper_name = f'_{method_name}_continued'

    kept_code = '

```

```

'.join(line for b in keep_blocks for line in b)

extracted_code = '
'.join(line for b in extract_blocks for line in b)

passed_vars = self._detect_passed_variables(
    kept_code, extracted_code, def_lines
)

return self.__split_regular_method_continued(extract_blocks, helper_name, keep_blocks, passed_vars, code, lines, func_start, func_end, class_name, method_name, body_inde

def __split_regular_method_continued(self, extract_blocks, helper_name, keep_blocks, passed_vars, code, lines, func_start, func_end, class_name, method_name, body_inde
    """Continuation of _split_regular_method - auto-extracted by self-evolution."""
    last_line = extract_blocks[-1][-1].strip() if extract_blocks[-1] else ''
    has_return = last_line.startswith('return ')
    params_sig = ', '.join(passed_vars) if passed_vars else ''
    params_call = ', '.join(passed_vars) if passed_vars else ''

    new_method = list(def_lines)
    new_method.extend(docstring_lines)
    for block in keep_blocks:
        new_method.extend(block)
        new_method.append('')
    call = f'self.{helper_name}({params_call})'
    new_method.append(f'{body_indent}{"return" if has_return else ""}{call}')
    new_method.append('')

    helper = self._build_helper_method(
        helper_name, method_name, params_sig, body_indent, class_indent,
        keep_blocks, extract_blocks
    )

    new_lines = lines[:func_start]
    new_lines.extend(new_method)
    new_lines.extend(helper)
    new_lines.extend(lines[func_end:])

    new_code = '
'.join(new_lines)

    try:
        ast.parse(new_code)
    except SyntaxError:
        return code, None

    return new_code, f"Split {class_name}.{method_name} into 2 methods (passing {len(passed_vars)} vars)"

def _separate_docstring(self, body_lines: List[str]) -> Tuple[List[str], List[str]]:
    """Separate leading docstring from body."""
    docstring_lines = []
    body_start_idx = 0
    in_docstring = False
    for i, line in enumerate(body_lines):
        stripped = line.strip()
        if i == 0 and (stripped.startswith('"""') or stripped.startswith('\'\'\'')):
            docstring_lines.append(line)
            if stripped.count('"""') >= 2 or stripped.count('\'\'\'') >= 2:
                body_start_idx = i + 1
                break
            in_docstring = True
        elif in_docstring:
            docstring_lines.append(line)
            if '"""' in stripped or '\'\'\' in stripped:
                body_start_idx = i + 1
                break
        else:
            body_start_idx = i
            break
    return docstring_lines, body_lines[body_start_idx:]

def _find_top_level_blocks(self, body_lines: List[str], body_indent: str) -> List[List[str]]:
    """Find top-level code blocks separated by blank lines at body indent level."""
    blocks = []
    current_block = []
    for line in body_lines:
        stripped = line.strip()
        if not stripped:
            if current_block:
                last_indent = len(current_block[-1]) - len(current_block[-1].lstrip())
                if last_indent <= len(body_indent):
                    blocks.append(current_block)
                    current_block = []
                    continue
            current_block.append(line)
            continue
        current_block.append(line)
    if current_block:
        blocks.append(current_block)
    return blocks

def _detect_passed_variables(self, kept_code: str, extracted_code: str,
    def_lines: List[str]) -> List[str]:
    """Detect local variables from kept code that are used in extracted code."""
    assigned_vars = set()
    for line in kept_code.split('
'):
        stripped = line.strip()
        m = re.match(r'^([a-zA-Z_]\w*)s*=\s', stripped)

```

```

        if m and not stripped.startswith('self.'):
            assigned_vars.add(m.group(1))
        m = re.match(r'^for\s+([a-zA-Z_]\w*)', stripped)
        if m:
            assigned_vars.add(m.group(1))

passed_vars = [v for v in sorted(assigned_vars) if re.search(rf'\b{re.escape(v)}\b', extracted_code)]

orig_def = ' '.join(l.strip() for l in def_lines)
param_match = re.search(r'\(self,?\s*(.*?)\)', orig_def, re.DOTALL)
if param_match:
    for param in param_match.group(1).split(','):
        param_name = param.strip().split(':')[0].split('=')[0].strip()
        if param_name and re.search(rf'\b{re.escape(param_name)}\b', extracted_code):
            if param_name not in passed_vars:
                passed_vars.append(param_name)

return passed_vars

def _build_helper_method(self, helper_name, method_name, params_sig,
                        body_indent, class_indent,
                        keep_blocks, extract_blocks) -> List[str]:
    """Build the extracted helper method lines."""
    helper = [
        f'{class_indent}def {helper_name}(self, {params_sig}):',
        f'{body_indent}"""Continuation of {method_name} – auto-extracted by self-evolution."""'
    ]

    # Add inline imports used in extracted blocks
    extracted_text = ' '.join(l for b in extract_blocks for l in b)

    for block in keep_blocks:
        for line in block:
            stripped = line.strip()
            if stripped.startswith('import ') or stripped.startswith('from '):
                mod_name = stripped.split()[-1].split('.')[0]
                if mod_name and re.search(rf'\b{re.escape(mod_name)}\b', extracted_text):
                    helper.append(f'{body_indent}{stripped}')

    all_extracted = [line for block in extract_blocks for line in block if line.strip()]
    min_indent = min((len(l) - len(l.lstrip())) for l in all_extracted), default=0)

    for block in extract_blocks:
        for line in block:
            stripped = line.strip()
            if stripped:
                if stripped.startswith('import ') or stripped.startswith('from '):
                    continue
                rebased = line[min_indent:] if len(line) > min_indent else line.lstrip()
                helper.append(body_indent + rebased)
            else:
                helper.append('')
        helper.append('')

    return helper

```

63. self_evolution_transforms.py

121 lines | 3.9KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/self_evolution_transforms.py

```
"""""
■ Self-Evolution Engine – Transforms Module
=====
Code transformation operations: docstrings, bare excepts, unused imports.
"""""

import re
from typing import Tuple, Optional

class TransformMixin:
    """""Mixin providing code transformation operations."""""

    def _add_docstring(self, code: str, target: str) -> Tuple[str, Optional[str]]:
        """""Add docstring to a class or function."""""
        name = target.split()[-1] if ' ' in target else target

        if 'class' in target:
            pattern = rf'(class {re.escape(name)}[^(]*:)(\s+)(?!""""")'
            match = re.search(pattern, code)
            if match:
                indent = match.group(2)
                replacement = f'{match.group(1)}\n{indent}"""""{name} - Auto-documented by self-evolution."""""\n{indent}'
                new_code = code[:match.start()] + replacement + code[match.end():]
                if new_code != code:
                    return new_code, f"Added docstring to {target}"

        elif 'function' in target:
            pattern = rf'(def {re.escape(name)}\[([^\]]*)\](^:)(\s+)(?!""""")'
            match = re.search(pattern, code)
            if match:
                indent = match.group(2)
                replacement = f'{match.group(1)}\n{indent}"""""{name} - Auto-documented by self-evolution."""""\n{indent}'
                new_code = code[:match.start()] + replacement + code[match.end():]
                if new_code != code:
                    return new_code, f"Added docstring to {target}"

        return code, None

    def _fix_bare_except(self, code: str, target: str) -> Tuple[str, Optional[str]]:
        """""Replace bare except: with except Exception:"""""
        lines = code.split('\n')

        changed = False

        for i, line in enumerate(lines):
            stripped = line.strip()
            if stripped == 'except:':
                indent = line[:len(line) - len(line.lstrip())]
                lines[i] = f'{indent}except Exception:'
                changed = True

        if changed:
            return '\n'.join(lines), f"Replaced bare except with except Exception in {target}"

        return code, None

    def _remove_unused_import(self, code: str, target: str) -> Tuple[str, Optional[str]]:
        """""Remove an unused import line from the code.
        Only removes simple 'import X' lines where X is truly unused.
        Does NOT touch 'from X import Y' lines (too risky - Y may be used).
        """""
        import_str = target.replace('import ', '').strip()
        if not import_str:
            return code, None

        lines = code.split('\n')

        import_line_idx = None
        imported_names = []

        for i, line in enumerate(lines):
            stripped = line.strip()

            if stripped == f'import {import_str}':
                import_line_idx = i
                imported_names = [import_str.split('.')[-1]]
                break

        from_match = re.match(rf'^from\s+{re.escape(import_str)}\s+import\s+(.+)$', stripped)
        if from_match:
```

```

        names = [n.strip().split(' as ')[-1].strip() for n in from_match.group(1).split(',')]
        import_line_idx = i
        imported_names = names
        break

if import_line_idx is None or not imported_names:
    return code, None

other_lines = lines[:import_line_idx] + lines[import_line_idx + 1:]
other_code = '
'.join(other_lines)

all_unused = True
for name in imported_names:
    pattern = rf'\b{re.escape(name)}\b'
    if re.search(pattern, other_code):
        all_unused = False
        break

if not all_unused:
    return code, None

lines.pop(import_line_idx)
return '
'.join(lines), f"Removed unused import: {import_str}"

```

64. self_research.py

379 lines | 16.5KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/self_research.py

```
&quot;&quot;&quot;;
■ Self-Research Engine
Autonomous research capability for Quantum AI
- Research on demand or by user command
- Long-running autonomous sessions (30-60 minutes)
&quot;&quot;&quot;;

import re
import random
from typing import Dict, List, Optional
from datetime import datetime, timezone, timedelta

import requests as _requests

class DDGS:
    &quot;&quot;&quot;Lightweight DuckDuckGo search using direct API calls.&quot;&quot;&quot;
    def text(self, query, max_results=5):
        try:
            import urllib.parse, re
            encoded = urllib.parse.quote(query)
            headers = {'User-Agent': 'Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36'}
            r = _requests.get(f'https://html.duckduckgo.com/html/?q={encoded}', headers=headers, timeout=10)
            # Extract snippets from HTML
            snippets = re.findall(r'class=&quot;result__snippet&quot;[^\&quot;]*&quot;(.*)&lt;/a&gt;', r.text, re.DOTALL)
            titles = re.findall(r'class=&quot;result__title&quot;[^\&quot;]*&quot;.*&lt;/a[^\&quot;]*&quot;(.*)&lt;/a&gt;', r.text, re.DOTALL)
            results = []
            for i, snippet in enumerate(snippets[:max_results]):
                clean = re.sub(r'&lt;[^\&quot;]+&gt;', '', snippet).strip()
                title = re.sub(r'&lt;[^\&quot;]+&gt;', '', titles[i]).strip() if i &lt; len(titles) else query
                if clean:
                    results.append({'title': title, 'body': clean, 'href': ''})
            return results
        except Exception as e:
            return []

from research_topics import EXPLORATION_TOPICS

class SelfResearchEngine:
    &quot;&quot;&quot;Autonomous research capability with extended sessions.&quot;&quot;&quot;

    def __init__(self, db=None):
        self.db = db
        self.research_log = []
        self.knowledge_gaps = []
        self.auto_research_enabled = True
        self.research_queue = []

        # Autonomous research state
        self.autonomous_running = False
        self.autonomous_start_time = None
        self.autonomous_duration_minutes = 30
        self.autonomous_progress = self._init_progress()
        self.exploration_topics = EXPLORATION_TOPICS

    def _init_progress(self) -&gt; Dict:
        return {
            'is_running': False,
            'started_at': None,
            'duration_minutes': 0,
            'elapsed_minutes': 0,
            'topics_researched': [],
            'insights_gained': 0,
            'concepts_learned': 0,
            'status': 'Idle'
        }

    async def research(self, query: str, reason: str = &quot;user_request&quot;) -&gt; Dict:
        &quot;&quot;&quot;Perform research on a topic.&quot;&quot;&quot;
        try:
            ddgs = DDGS()
            results = list(ddgs.text(query, max_results=5))

            research_result = {
                'query': query,
                'reason': reason,
                'timestamp': datetime.now(timezone.utc).isoformat(),
                'results': [{ 'title': r.get('title', ''), 'body': r.get('body', ''), 'href': r.get('href', '') } for r in results],
                'summary': '',
                'new_concepts': self._extract_concepts(' '.join([r.get('body', '') for r in results]))
            }

            self.research_log.append(research_result)
            if self.db is not None:
                await self._store_research(research_result)

            return research_result
        except Exception as e:
            return {'query': query, 'reason': reason, 'error': str(e), 'results': []}

    def _extract_concepts(self, text: str) -&gt; List[str]:
        &quot;&quot;&quot;Extract key concepts from text.&quot;&quot;&quot;
        words = re.findall(r'\b[a-zA-Z]{5,}\b', text.lower())
```

```

common = {'about', 'which', 'their', 'there', 'would', 'could', 'should', 'these', 'those', 'other', 'being', 'where', 'while', 'through', 'between'}
from collections import Counter
counts = Counter(w for w in words if w not in common)
return [w for w, _ in counts.most_common(10)]

def _topic_to_domain(self, topic: str) -> str:
    """Map research topic/document name to a broad knowledge domain."""
    topic_lower = topic.lower()
    mapping = {
        'quantum': 'physics',
        'physics': 'physics',
        'philosophy': 'philosophy',
        'mind': 'philosophy',
        'biology': 'biology',
        'cell': 'biology',
        'genetic': 'biology',
        'mathematics': 'mathematics',
        'algebra': 'mathematics',
        'geometry': 'mathematics',
        'history': 'history',
        'rome': 'history',
        'empire': 'history',
        'art': 'art',
        'renaissance': 'art',
        'psychology': 'psychology',
        'cognitive': 'psychology',
        'computer': 'computer science',
        'ai': 'computer science',
        'artificial': 'computer science',
        'literature': 'literature',
        'shakespeare': 'literature',
        'economics': 'economics',
        'market': 'economics',
        'environment': 'environmental science',
        'climate': 'environmental science',
        'ecology': 'environmental science',
        'cosmic': 'philosophy', # cosmic_philosophy_physics.txt bridges physics + philosophy
        # New domains from research feeds
        'crime': 'crime and justice',
        'justice': 'crime and justice',
        'pirate': 'crime and justice',
        'tobacco': 'crime and justice',
        'daily': 'daily life',
        'food': 'daily life',
        'clothing': 'daily life',
        'family': 'daily life',
        'house': 'daily life',
        'religion': 'religion and mythology',
        'myth': 'religion and mythology',
        'angel': 'religion and mythology',
        'god': 'religion and mythology',
        'divine': 'religion and mythology',
        'zoroast': 'religion and mythology',
        'ahura': 'religion and mythology',
        'amaterasu': 'religion and mythology',
        'gnos': 'religion and mythology',
        'hermet': 'religion and mythology',
        'medicine': 'medicine',
        'medical': 'medicine',
        'disease': 'medicine',
        'plague': 'medicine',
        'anatomy': 'medicine',
        'vesalius': 'medicine',
        'galen': 'medicine',
        'technolog': 'technology',
        'invention': 'technology',
        'mechanism': 'technology',
        'astrolabe': 'technology',
        'aqueduct': 'technology',
        'timekeep': 'technology',
        'telescope': 'technology',
        'printing': 'technology',
        'music': 'music',
        'composer': 'music',
        'vivaldi': 'music',
        'bruckner': 'music',
        'dvorak': 'music',
        'harmonic': 'music',
        'tuning': 'music',
        'linguist': 'linguistics',
        'alphabet': 'linguistics',
        'writing': 'linguistics',
        'language': 'linguistics',
        'grammar': 'linguistics',
        'rosetta': 'linguistics',
        'celtic': 'linguistics',
        'cosmolog': 'cosmology',
        'big bang': 'cosmology',
        'multiverse': 'cosmology',
        'anthropic': 'cosmology',
        'entropy': 'information theory',
        'emergence': 'systems theory',
        'cybernetic': 'systems theory',
        'autopoiesis': 'systems theory',
        'complex': 'systems theory',
    }
    for keyword, domain in mapping.items():
        if keyword in topic_lower:
            return domain
    return 'general'

async def _store_research(self, research: Dict):

```

```

"""Store research in database."""
try:
    await self.db.research_history.insert_one({
        "research": research, '_stored_at': datetime.now(timezone.utc).isoformat()
    })
    for concept in research.get('new_concepts', []):
        await self.db.semantic_memory.update_one(
            {'concept': concept},
            {'$set': {'concept': concept, 'source': 'research', 'query': research['query']},
             '$inc': {'count': 1}},
            upsert=True
        )
except Exception as e:
    print(f"[RESEARCH] Store error: {e}")

async def start_autonomous_research(self, duration_minutes: int = 30, engine=None, memory=None) -> Dict:
    """Start autonomous research session."""
    if self.autonomous_running:
        return {'status': 'already_running'}
    self.engine = engine
    self.memory = memory

    self.autonomous_running = True
    self.autonomous_start_time = datetime.now(timezone.utc)
    self.autonomous_duration_minutes = min(duration_minutes, 60)

    self.autonomous_progress = {
        'is_running': True,
        'started_at': self.autonomous_start_time.isoformat(),
        'duration_minutes': self.autonomous_duration_minutes,
        'elapsed_minutes': 0,
        'topics_researched': [],
        'insights_gained': 0,
        'concepts_learned': 0,
        'status': 'Starting...'
    }

    # Run research as async task in background
    import asyncio
    asyncio.create_task(self._run_autonomous_research())
    return {'status': 'started', 'duration_minutes': self.autonomous_duration_minutes,
            'message': f'Starting {self.autonomous_duration_minutes} minute autonomous research session'}

async def _run_autonomous_research(self):
    """Background task for autonomous research."""
    try:
        has_db = self.db is not None
        print(f"[RESEARCH] Starting autonomous research, has_db={has_db}")

        # Check for local document corpus
        import os
        docs_dir = os.path.join(os.path.dirname(__file__), 'documents')
        use_local_corpus = os.path.exists(docs_dir) and any(f.endswith('.txt') for f in os.listdir(docs_dir))

        end_time = self.autonomous_start_time + timedelta(minutes=self.autonomous_duration_minutes)

        while datetime.now(timezone.utc) < end_time and self.autonomous_running:
            try:
                if use_local_corpus:
                    # Pick a random document from the corpus
                    doc_files = [f for f in os.listdir(docs_dir) if f.endswith('.txt')]
                    if not doc_files:
                        break
                    doc_file = random.choice(doc_files)
                    topic = doc_file.replace('.txt', '').replace('_', ' ').title()
                    query = topic # for DB storage
                    with open(os.path.join(docs_dir, doc_file), 'r', encoding='utf-8') as f:
                        text = f.read()
                else:
                    # Fallback to DuckDuckGo search (requires internet)
                    topic = random.choice(self.exploration_topics)
                    query = f"{topic} {random.choice(['latest', 'explained', 'applications', 'deep dive'])}"
                    ddgs = DDGS()
                    results = list(ddgs.text(query, max_results=5))
                    if not results:
                        continue # skip if no results
                    text = ' '.join([r.get('body', '') for r in results])

                self.autonomous_progress['topics_researched'].append(topic)
                concepts = self._extract_concepts(text)
                self.autonomous_progress['concepts_learned'] += len(concepts)

                # Ingest via cognitive core with domain tagging
                if hasattr(self, 'engine') and self.engine:
                    domain = self._topic_to_domain(topic)
                    result = await self.engine.ingest_text(text, domain=domain, source='autonomous_research', max_concepts=15)
                    print(f"[RESEARCH] Ingested: {topic} (domain={domain}, +{result['stored']} new concepts)")

                # Update in-memory counters for immediate UI feedback
                if hasattr(self, 'memory') and self.memory:
                    self.memory.growth['total_concepts'] += result['stored']
                    if len(concepts) > 3:
                        self.memory.growth['total_insights'] += 1

                if len(concepts) > 3:
                    self.autonomous_progress['insights_gained'] += 1

            # Store research session metadata
            if has_db:
                try:
                    await self.db.research_history.insert_one({

```

```

        'query': topic if use_local_corpus else query,
        'topic': topic,
        'reason': 'autonomous',
        'timestamp': datetime.now(timezone.utc).isoformat(),
        'concepts': concepts,
        'results_count': 1 if use_local_corpus else len(results)
    })

    # Update autonomous research aggregate metrics
    await self.db.growth_metrics.update_one(
        {'type': 'autonomous_research'},
        {'$inc': {'researches': 1, 'concepts': len(concepts)},
         '$set': {'last_topic': topic, 'timestamp': datetime.now(timezone.utc).isoformat()}},
        upsert=True
    )
    print(f"<div data-bbox=
```

65. server.py

96 lines | 2.9KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V█/backend/server.py

```
&quot;&quot;&quot;;
■ QUANTUM AI - BACKEND SERVER
=====

ARTICLE 3.2 - The Transparency Pledge
All philosophical assumptions are documented and visible.

This server implements the Quantum AI cognitive architecture
with full covenant integration.
&quot;&quot;&quot;;

from fastapi import FastAPI
from fastapi.responses import RedirectResponse
from starlette.middleware.cors import CORSMiddleware
from starlette.staticfiles import StaticFiles
import os
import logging

import shared_state

# Route modules
from routes_chat import router as chat_router
from routes_cognitive import router as cognitive_router
from routes_explorer import router as explorer_router
from routes_brain import router as brain_router
from routes_media import router as media_router
from routes_extras import router as extras_router
from routes_evaluation import router as evaluation_router
from routes_image import router as image_router
# from routes_test import router as test_router # removed (deleted)

logger = logging.getLogger(&quot;quantum_ai&quot;);
logging.basicConfig(level=logging.INFO)

# Create the main app
app = FastAPI(title=&quot;Quantum AI&quot;;, description=&quot;True Questioning AI with Cognitive Architecture&quot;);

# CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=os.environ.get('CORS_ORIGINS', '').split(','),
    allow_credentials=True,
    allow_methods=[&quot;*&quot;],
    allow_headers=[&quot;*&quot;],
)

# Serve frontend static files
frontend_path = os.path.join(os.path.dirname(os.path.dirname(__file__)), 'frontend', 'dist')
if os.path.exists(frontend_path):
    app.mount('/app', StaticFiles(directory=frontend_path, html=True), name='frontend')
    logger.info(f&quot;■ Frontend served from /app (directory: {frontend_path})&quot;);
else:
    logger.warning(f&quot;■ Frontend not built. Run 'npm run build' in frontend/ directory. (Missing: {frontend_path})&quot;);

# Include all route modules under /api
app.include_router(chat_router)
app.include_router(cognitive_router)
app.include_router(explorer_router)
app.include_router(brain_router)
app.include_router(media_router)
app.include_router(extras_router)
app.include_router(evaluation_router)
app.include_router(image_router)
# app.include_router(test_router) # removed (deleted)

@app.on_event(&quot;startup&quot;);
async def startup_event():
    &quot;&quot;&quot;Initialize all Quantum AI systems on startup.&quot;&quot;&quot;
    logger.info(&quot;■ Initializing Quantum AI systems...&quot;);
    try:
        shared_state.initialize()
        logger.info(&quot;■ Quantum AI initialized successfully!&quot;);
    except Exception as e:
        import traceback
        logger.error(f&quot;Startup failed: {e}&quot;);
        {traceback.format_exc()}&quot;);

    raise

@app.on_event(&quot;shutdown&quot;);
async def shutdown_db_client():
    &quot;&quot;&quot;Clean shutdown.&quot;&quot;&quot;
    if hasattr(shared_state, 'client') and shared_state.client:
        shared_state.client.close()

if __name__ == &quot;__main__&quot;;:
    import uvicorn
    port = int(os.environ.get(&quot;PORT&quot;;, 8000))
    uvicorn.run(app, host=&quot;0.0.0.0&quot;;, port=port)

@app.get(&quot;/&quot;);
```

```
async def root():  
    return RedirectResponse(url="/app")
```

66. shared_link_handler.py

216 lines | 7.3KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/shared_link_handler.py

```
&quot;&quot;&quot;;
■ QUANTUM AI - SHARED LINK HANDLER
=====

Process files from shared cloud links:
- Google Drive
- Dropbox
- OneDrive
- Direct file URLs

ARTICLE 2 - Universal Explorer: Access data from anywhere
&quot;&quot;&quot;;

import requests
import os
from typing import Dict, Optional
from urllib.parse import urlparse, parse_qs
import re

class SharedLinkHandler:
    &quot;&quot;&quot;;
    Handle file downloads from shared cloud links
    &quot;&quot;&quot;;

    def __init__(self):
        # Use environment variable or default to local writable directory
        self.download_dir = os.environ.get(
            &quot;UPLOAD_DIR&quot;;,
            os.path.expanduser(&quot;~/quantum-mcagi-backend/uploads&quot;))
        )
        try:
            os.makedirs(self.download_dir, exist_ok=True)
        except OSError as e:
            print(f&quot;Warning: Could not create upload directory {self.download_dir}: {e}&quot;);
            # Fallback to /tmp
            self.download_dir = &quot;/tmp/quantum-uploads&quot;;
            os.makedirs(self.download_dir, exist_ok=True)

    def detect_link_type(self, url: str) -&gt; str:
        &quot;&quot;&quot;&quot;Detect what type of shared link this is&quot;&quot;&quot;&quot;
        url_lower = url.lower()

        if 'drive.google.com' in url_lower:
            return 'google_drive'
        elif 'dropbox.com' in url_lower:
            return 'dropbox'
        elif 'onedrive' in url_lower or 'idrv.ms' in url_lower:
            return 'onedrive'
        elif url_lower.startswith('http'):
            return 'direct'
        else:
            return 'unknown'

    def convert_to_direct_link(self, url: str, link_type: str) -&gt; Optional[str]:
        &quot;&quot;&quot;&quot;Convert shared link to direct download link&quot;&quot;&quot;&quot;

        if link_type == 'google_drive':
            # Google Drive: extract file ID and convert to download link
            # Formats:
            # https://drive.google.com/file/d/FILE_ID/view
            # https://drive.google.com/open?id=FILE_ID

            file_id = None

            # Try pattern 1: /file/d/FILE_ID/
            match = re.search(r'/file/d/([a-zA-Z0-9_-]+)', url)
            if match:
                file_id = match.group(1)

            # Try pattern 2: ?id=FILE_ID
            if not file_id:
                match = re.search(r'[?&]id=([a-zA-Z0-9_-]+)', url)
                if match:
                    file_id = match.group(1)

            if file_id:
                return f&quot;https://drive.google.com/uc?export=download&amp;id={file_id}&quot;;

        elif link_type == 'dropbox':
            # Dropbox: change dl=0 to dl=1
            if '&quot;dl=0' in url:
                return url.replace('&quot;dl=0', '&quot;dl=1')
            elif '&quot;dl=1' not in url:
                separator = '&quot;'; if '?' in url else '?'
                return f&quot;{url}{separator}dl=1&quot;;
            return url

        elif link_type == 'onedrive':
            # OneDrive: convert to direct download
            # This is more complex, might need specific handling
            if 'embed' in url:
                return url.replace('embed', 'download')
            # For idrv.ms short links, direct download might work
```

```

        return url

    elif link_type == 'direct':
        return url

    return None

async def download_from_link(self, url: str, filename: Optional[str] = None) -> Dict:
    """
    Download file from shared link
    """
    try:
        # Detect link type
        link_type = self.detect_link_type(url)

        if link_type == 'unknown':
            return {
                'status': 'error',
                'message': 'Unknown link type. Supported: Google Drive, Dropbox, OneDrive, direct URLs'
            }

        # Convert to direct download link
        download_url = self.convert_to_direct_link(url, link_type)

        if not download_url:
            return {
                'status': 'error',
                'message': f'Could not convert {link_type} link to direct download'
            }

        # Download the file
        print(f'■ Downloading from {link_type}: {download_url}')

        headers = {
            'User-Agent': 'QuantumAI/1.0 (Universal Explorer)'
        }

        response = requests.get(download_url, headers=headers, stream=True, timeout=60)

        if response.status_code != 200:
            return {
                'status': 'error',
                'message': f'Download failed with status {response.status_code}',
                'link_type': link_type
            }

        # Determine filename
        if not filename:
            # Try to get from Content-Disposition header
            content_disposition = response.headers.get('Content-Disposition', '')
            if 'filename=' in content_disposition:
                filename = content_disposition.split('filename=')[1].strip('"\'')
            else:
                # Use last part of URL or default
                filename = os.path.basename(urlparse(url).path) or 'downloaded_file'

        # Save file
        file_path = os.path.join(self.download_dir, filename)

        with open(file_path, 'wb') as f:
            for chunk in response.iter_content(chunk_size=8192):
                if chunk:
                    f.write(chunk)

        file_size = os.path.getsize(file_path)

        return {
            'status': 'success',
            'filename': filename,
            'path': file_path,
            'size': file_size,
            'link_type': link_type,
            'message': f'File downloaded successfully from {link_type}'
        }

    except Exception as e:
        return {
            'status': 'error',
            'message': str(e),
            'link_type': link_type if 'link_type' in locals() else 'unknown'
        }

def get_instructions(self, link_type: str) -> str:
    """
    Get instructions for sharing files from different services
    """
    instructions = {
        'google_drive': '■ Google Drive:
        1. Right-click file → Share → Get link
        2. Set to Anyone with the link
        3. Copy link (looks like: drive.google.com/file/d/...)
        4. Paste here
        ',
        'dropbox': '■ Dropbox:
        1. Right-click file → Share → Create link
        2. Copy link (looks like: dropbox.com/s/...)
        3. Paste here
        ',
        'onedrive': '■ OneDrive:
    
```

```
1. Right-click file → Share → Copy link
2. Make sure "Anyone with the link can view"
3. Copy link (looks like: onedrive.live.com/...)
4. Paste here
   """
```

```
    return instructions.get(link_type, "Paste a shareable link from cloud storage")
```

```
# Singleton
```

```
_shared_link_handler = None
```

```
def get_shared_link_handler():
    """Get or create shared link handler instance"""
    global _shared_link_handler
```

```
    if _shared_link_handler is None:
        _shared_link_handler = SharedLinkHandler()
```

```
    return _shared_link_handler
```

67. shared_state.py

527 lines | 21.5KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/shared_state.py

```
"""""
■ QUANTUM AI - Shared State
"""""
=====
Central state holder for all shared instances.
Route modules import from here to access db, engines, etc.
"""""

import os
import json
import uuid
import asyncio
import logging
from datetime import datetime, timezone
from pathlib import Path
from typing import Any, Dict, List, Optional
from dotenv import load_dotenv

logger = logging.getLogger("quantum_ai")

ROOT_DIR = Path(__file__).parent
load_dotenv(ROOT_DIR / '.env')

# =====
# LOCAL MEMORY BACKEND (MongoDB-free operation)
# =====

class SimpleLocalDB:
    """""
    Simple file-based database that mimics MongoDB's async interface.
    Uses JSON files stored in ~/.quantum-mcagi/.
    """""

    def __init__(self, data_dir=None):
        self.data_dir = Path(data_dir or os.path.expanduser("~/.quantum-mcagi"))
        self.data_dir.mkdir(parents=True, exist_ok=True)
        self.collections = {}

    def __getattr__(self, name: str) -> 'SimpleCollection':
        if name not in self._collections:
            self._collections[name] = SimpleCollection(name, self.data_dir)
        return self._collections[name]

    async def command(self, command: str) -> Dict[str, Any]:
        """""Mimics MongoDB command interface."""""
        if command == "ping":
            return {"ok": 1}
        raise NotImplementedError(f"Command {command} not supported")

class SimpleCollection:
    """""A collection that stores documents in a JSON file."""""

    def __init__(self, name: str, data_dir: Path):
        self.name = name
        self.filepath = data_dir / f"{name}.json"
        self._data: List[Dict] = []
        self._lock = asyncio.Lock()
        self._load()

    def _load(self):
        """""Load data from disk (synchronous)."""""
        if self.filepath.exists():
            try:
                with open(self.filepath, 'r') as f:
                    self._data = json.load(f)
            except (json.JSONDecodeError, IOError):
                self._data = []
        else:
            self._data = []

    async def _save(self):
        """""Save data to disk (async)."""""
        async with self._lock:
            try:
                # Convert to JSON-serializable format
                serializable = []
                for doc in self._data:
                    serializable.append(self._serialize_doc(doc))
                with open(self.filepath, 'w') as f:
                    json.dump(serializable, f, indent=2, default=str)
            except Exception as e:
                print(f"Error saving {self.name}: {e}")

    def _serialize_doc(self, doc: Dict) -> Dict:
        """""Convert datetime objects to ISO strings for JSON."""""
        result = {}
        for k, v in doc.items():
            if isinstance(v, datetime):
                result[k] = v.isoformat()
            elif isinstance(v, dict):
                result[k] = self._serialize_doc(v)
            else:
                result[k] = v
        return result
```

```

def _deserialize_doc(self, doc: Dict) -> Dict:
    """Convert ISO strings back to datetime objects"""
    result = {}
    for k, v in doc.items():
        if isinstance(v, str) and 'T' in v and len(v) > 19:
            try:
                result[k] = datetime.fromisoformat(v.replace('Z', '+00:00'))
            except:
                result[k] = v
        elif isinstance(v, dict):
            result[k] = self._deserialize_doc(v)
        else:
            result[k] = v
    return result

def _ensure_ids(self):
    """Ensure all documents have _id fields (synchronous)"""
    for doc in self._data:
        if '_id' not in doc:
            doc['_id'] = str(uuid.uuid4())

def find(self, filter: Optional[Dict] = None, projection: Optional[Dict] = None) -> 'AsyncCursor':
    """Find documents matching filter"""
    self._ensure_ids()
    results = []
    for doc in self._data:
        if self._matches(doc, filter):
            # Apply projection if provided (simple: exclude _id if specified)
            result_doc = self._deserialize_doc(doc.copy())
            if projection:
                # Very basic projection: only include fields where value is 1 (or exclude if 0)
                # For simplicity, we'll just ignore projection and return full doc
                pass
            results.append(result_doc)
    return AsyncCursor(results)

async def find_one(self, filter: Optional[Dict] = None, projection: Optional[Dict] = None) -> Optional[Dict]:
    """Find first matching document"""
    self._ensure_ids()
    for doc in self._data:
        if self._matches(doc, filter):
            result = self._deserialize_doc(doc.copy())
            # For now, ignore projection (Motor includes _id by default unless excluded)
            return result
    return None

async def insert_one(self, document: Dict) -> Any:
    """Insert a document"""
    doc = document.copy()
    if '_id' not in doc:
        doc['_id'] = str(uuid.uuid4())
    # Ensure timestamps are datetime objects
    if 'timestamp' in doc and isinstance(doc['timestamp'], str):
        try:
            doc['timestamp'] = datetime.fromisoformat(doc['timestamp'].replace('Z', '+00:00'))
        except:
            pass

    # Append to data (no lock needed for list append in CPython due to GIL)
    self._data.append(doc)
    # Save to disk (handles its own lock)
    await self._save()

    class InsertOneResult:
        def __init__(self, inserted_id):
            self.inserted_id = inserted_id
    return InsertOneResult(doc['_id'])

async def insert_many(self, documents: List[Dict]) -> List[Any]:
    """Insert multiple documents"""
    results = []
    for doc in documents:
        result = await self.insert_one(doc)
        results.append(result)
    return results

async def update_one(self, filter: Dict, update: Dict, upsert: bool = False) -> Any:
    """Update first matching document. If upsert=True and no match, insert new doc"""
    for i, doc in enumerate(self._data):
        if self._matches(doc, filter):
            # Apply $set operations
            if '$set' in update:
                for k, v in update['$set'].items():
                    doc[k] = v
            # Apply $inc operations
            if '$inc' in update:
                for k, v in update['$inc'].items():
                    doc[k] = doc.get(k, 0) + v
            # Apply $addToSet operations (add to array if not present)
            if '$addToSet' in update:
                for key, val in update['$addToSet'].items():
                    current = doc.get(key, [])
                    if not isinstance(current, list):
                        current = []
                    items = val.get('$each') if isinstance(val, dict) and '$each' in val else [val]
                    for item in items:
                        if item not in current:
                            current.append(item)
                    doc[key] = current
            self._data[i] = doc
            await self._save()

```



```

        elif op == '$avg':
            total = 0
            count = 0
            for d in docs:
                if isinstance(expr, str) and expr.startswith('$'):
                    field = expr[1:]
                    total += d.get(field, 0)
                    count += 1
            out_doc[field] = total / count if count > 0 else 0
        elif op == '$push':
            if isinstance(expr, str) and expr.startswith('$'):
                field = expr[1:]
                out_doc[field] = [d.get(field) for d in docs]
            output.append(out_doc)
        results = output
    if '$sort' in stage:
        sort_spec = stage['$sort']
        # sort_spec is like {'<field>': -1} or [{'<field>', 1}, ...]
        if isinstance(sort_spec, dict):
            for field, direction in sort_spec.items():
                reverse = direction == -1
                results.sort(key=lambda x: x.get(field, None), reverse=reverse)
        elif isinstance(sort_spec, list):
            # Multi-field sort (simplified - only first field)
            field, direction = sort_spec[0]
            reverse = direction == -1
            results.sort(key=lambda x: x.get(field, None), reverse=reverse)
    if '$limit' in stage:
        results = results[:stage['$limit']]
    if '$match' in stage:
        filter = stage['$match']
        results = [doc for doc in results if self._matches(doc, filter)]
    return AsyncCursor(results)

def _matches(self, doc: Dict, filter: Optional[Dict]) -> bool:
    """Simple filter matching (supports equality, $exists, $gte, $lte, $ne, $in, $regex)."""
    if not filter:
        return True
    for key, value in filter.items():
        if key == '$exists':
            # Handle {$exists: True/False}
            for exist_key, should_exist in value.items():
                exists = exist_key in doc
                if exists != should_exist:
                    return False
        elif isinstance(value, dict):
            # Handle operators like $gte, $lte, $ne, $in, $regex
            for op, op_val in value.items():
                doc_val = doc.get(key)

                # If doc_val is None, most comparison operators should fail (field missing)
                if doc_val is None:
                    # For $gte/$lte, missing field cannot satisfy condition
                    if op in ('$gte', '$lte', '$ne', '$in'):
                        return False
                    # For $regex, None doesn't match
                    if op == '$regex':
                        return False

                # Handle datetime comparisons: if doc_val is datetime, try to convert op_val
                if isinstance(doc_val, datetime) and isinstance(op_val, str):
                    try:
                        # Try to parse ISO format string
                        op_val_dt = datetime.fromisoformat(op_val.replace('Z', '+00:00'))
                        # Use the datetime for comparison
                        comparison_val = op_val_dt
                    except (ValueError, TypeError):
                        comparison_val = op_val
                else:
                    comparison_val = op_val

                if op == '$gte':
                    if doc_val < comparison_val:
                        return False
                elif op == '$lte':
                    if doc_val > comparison_val:
                        return False
                elif op == '$ne':
                    if doc_val == op_val:
                        return False
                elif op == '$in':
                    if doc_val not in op_val:
                        return False
                elif op == '$regex':
                    import re
                    if not re.search(op_val, str(doc_val), re.IGNORECASE):
                        return False
            else:
                # Simple equality
                if key not in doc or doc[key] != value:
                    return False
    return True

class AsyncCursor:
    """Mimics Motor's async cursor."""

    def __init__(self, data: List[Dict]):
        self._data = data
        self._sort_spec = None
        self._limit_val = None

```

```

def sort(self, key_or_list, direction: int = 1):
    """Sort the results."""
    self._sort_spec = (key_or_list, direction)
    return self

def limit(self, n: int):
    """Limit number of results."""
    self._limit_val = n
    return self

async def to_list(self, length: Optional[int] = None) -> List[Dict]:
    """Get results as list."""
    data = self._data.copy()
    if self._sort_spec:
        key_or_list, direction = self._sort_spec
        reverse = direction == -1
        if isinstance(key_or_list, str):
            data.sort(key=lambda x: x.get(key_or_list, None), reverse=reverse)
        elif isinstance(key_or_list, list):
            # Multi-field sort - apply in reverse order
            for field, direction in reversed(key_or_list):
                reverse = direction == -1
                data.sort(key=lambda x: x.get(field, None), reverse=reverse)
    if self._limit_val:
        data = data[:self._limit_val]
    if length:
        data = data[:length]
    return data

# =====
# DATABASE INITIALIZATION
# =====

# Always use local file-based storage (MongoDB removed)
db = SimpleLocalDB()

# Lazy-initialized singletons (set during startup)
cognitive_core = None
universal_explorer = None
covenant_manager = None
operational_guidelines = None
dictionary = None
shared_link = None
hidden_thinking = None

# Idle/dream state management
last_activity_time = None
is_idle = False
dream_task = None

def initialize():
    """Initialize all Quantum AI systems. Called once at startup."""
    global cognitive_core, universal_explorer, covenant_manager
    global operational_guidelines, dictionary, shared_link, hidden_thinking
    global last_activity_time, dream_task, is_idle

    from quantum_cognitive_core import QuantumCognitiveCore
    from universal_explorer import UniversalExplorer
    from covenant_manager import CovenantManager, OperationalGuidelines
    from dictionary_integration import get_dictionary_integration
    from shared_link_handler import get_shared_link_handler
    from hidden_thinking import get_hidden_thinking
    from dream_state import get_dream_engine

    cognitive_core = QuantumCognitiveCore(db)
    universal_explorer = UniversalExplorer(db)
    covenant_manager = CovenantManager(db)
    operational_guidelines = OperationalGuidelines(db)
    dictionary = get_dictionary_integration(cognitive_core.semantic_memory)
    shared_link = get_shared_link_handler()
    hidden_thinking = get_hidden_thinking(cognitive_core, dictionary, universal_explorer)

    # Initialize dream engine
    dream_engine = get_dream_engine(db)

    # Start background idle/dream monitoring
    last_activity_time = datetime.now(timezone.utc)
    is_idle = False

    async def monitor_idle():
        """Background task: check idle status and manage dream state.
        When idle, runs dream cycles which now include:
        - dream_driven_research (auto-fetches web content from ideas)
        - cloud_sync (auto-saves brain to Wolfram Cloud)
        - plus the original activities (consolidation, reflection, ideas, etc.)
        """
        global last_activity_time, is_idle, dream_task

        while True:
            await asyncio.sleep(60) # Check every minute

            now = datetime.now(timezone.utc)
            idle_minutes = (now - last_activity_time).total_seconds() / 60

            if idle_minutes >= 5 and not is_idle:
                # Enter dream state
                is_idle = True
                try:
                    dream_engine.enter_dream_state()

```

```

        logger.info(f"■ Entering dream state (idle for 5+ min)&quot;);
    except Exception as e:
        logger.error(f"■ Dream state entry failed: {e}&quot;);

elif idle_minutes < 5 and is_idle:
    # Exit dream state (user became active)
    is_idle = False
    try:
        result = await dream_engine.exit_dream_state()
        if result.get('dream_summary'):
            logger.info(f"■ Awakened from dream. Insights: {len(result['dream_summary'].get('insights', []))}&quot;);
    except Exception as e:
        logger.error(f"■ Dream state exit failed: {e}&quot;);

# While dreaming, run dream cycles periodically
if is_idle:
    try:
        cycle_result = await dream_engine.dream_cycle()
        if cycle_result and not cycle_result.get('error'):
            activity = cycle_result.get('activity', '')
            logger.info(f"■ Dream cycle: {activity} - {str(cycle_result)[:80]}&quot;);
    except Exception as e:
        logger.error(f"■ Dream cycle error: {e}&quot;);

# Start the monitor task
dream_task = asyncio.create_task(monitor_idle())
logger.info(f"■ Background idle/dream monitor started (autonomous research + cloud sync enabled)&quot;);

```

68. spell_correct.py

78 lines | 2.3KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V██/backend/spell_correct.py

```
"""""
Spell Correction - Uses Markov vocabulary as dictionary.
Fixes typos before concept extraction so junk doesn't persist.
"""""

def levenshtein(s1, s2):
    """""levenshtein - Auto-documented by self-evolution."""""
    if len(s1) < len(s2):
        return levenshtein(s2, s1)
    if len(s2) == 0:
        return len(s1)
    prev = range(len(s2) + 1)
    for i, c1 in enumerate(s1):
        curr = [i + 1]
        for j, c2 in enumerate(s2):
            curr.append(min(prev[j + 1] + 1, curr[j] + 1, prev[j] + (c1 != c2)))
        prev = curr
    return prev[-1]

def build_corrector(markov_chain):
    """""build_corrector - Auto-documented by self-evolution."""""
    vocab = set(markov_chain.keys())
    # Only words 3+ chars to avoid matching noise
    vocab = {w for w in vocab if len(w) >= 3 and w.isalpha()}
    return vocab

def correct_word(word, vocab, max_dist=None):
    """""correct_word - Auto-documented by self-evolution."""""
    w = word.lower()
    if max_dist is None:
        max_dist = 2 if len(w) <= 6 else 3
    if w in vocab:
        return word
    if len(w) < 3:
        return word
    if not w.isalpha():
        return word
    best = None
    best_dist = max_dist + 1
    for v in vocab:
        if abs(len(v) - len(w)) > max_dist:
            continue
        if v[0] != w[0]:
            continue
        d = levenshtein(w, v)
        if d < best_dist:
            best_dist = d
            best = v
    if best and best_dist <= max_dist:
        return best
    return word

def correct_text(text, vocab):
    """""correct_text - Auto-documented by self-evolution."""""
    words = text.split()
    corrected = []
    fixes = []
    for w in words:
        clean = w.strip('.,!?:()[]"\'')
        fixed = correct_word(clean, vocab)
        if fixed != clean:
            fixes.append((clean, fixed))
            corrected.append(w.replace(clean, fixed))
        else:
            corrected.append(w)
    return ' '.join(corrected), fixes

if __name__ == "__main__":
    test_vocab = {'consciousness', 'quantum', 'reality', 'perceive', 'superposition', 'what', 'experience', 'existence', 'spacetime'}
    tests = ['whata', 'precive', 'superpositioning', 'consciousnes', 'quantm', 'realty']
    for t in tests:
        fixed = correct_word(t, test_vocab)
        print(f"{t} -> {fixed}")
```

69. text_analyzer.py

254 lines | 9.0KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/text_analyzer.py

```
"""
■ Text Analyzer
Determines if text sounds human or LLM-generated.
Helps Quantum AI produce more authentic responses.
"""
```

```
import re
from typing import Dict, List, Tuple
```

```
class TextAnalyzer:
    """
    Analyzes text to determine if it sounds human or LLM-generated.

    LLM tells:
    - Overly formal language
    - Excessive hedging
    - Repetitive structures
    - Lack of personality
    - Too perfect grammar
    - Generic openings/closings
    - Excessive caveats
    """

    def __init__(self):
        # LLM-typical phrases
        self.llm_phrases = [
            "i cannot", "i can't", "as an ai",
            "i don't have personal", "i'm not able to",
            "it's important to note", "it's worth noting",
            "i'd be happy to help", "absolutely!",
            "great question", "that's a great question",
            "let me break this down", "here's what i can tell you",
            "i understand you're asking", "base&on the information",
            "however, it's important", "while i can",
            "i hope this helps", "feel free to ask",
            "certainly!", "of course!",
            "firstly", "secondly", "thirdly",
            "in conclusion", "to summarize"
        ]

        # Human-like phrases
        self.human_phrases = [
            "honestly", "actually", "basically",
            "i mean", "you know", "like",
            "anyway", "so yeah", "right?&,
            "kinda", "sorta", "pretty much",
            "tbh", "imo", "idk",
            "haha", "lol", "hmm",
            "oh", "well", "ah"
        ]

        # LLM structural patterns
        self.llm_patterns = [
            r'\b(firstly|secondly|thirdly|finally)\b',
            r'it's (important|worth) (to note|noting)',
            r'(however|additionally|furthermore|moreover)',
            r'\bi'd be happy to\b',
            r'(certainly|absolutely)!',
            r'let me (explain|break|help)',
            r'(great|good|excellent) question',
        ]

    def analyze(self, text: str) -> Dict:
        """
        Analyze text for human vs LLM characteristics.

        Returns:
            Dict with scores and analysis
        """
        text_lower = text.lower()
        words = text_lower.split()
        sentences = re.split(r'[.!?]+' , text)

        analysis = {
            'text_length': len(text),
            'word_count': len(words),
            'sentence_count': len([s for s in sentences if s.strip()]),
            'llm_score': 0.0,
            'human_score': 0.0,
            'indicators': {
                'llm': [],
                'human': []
            },
            'verdict': '',
            'confidence': 0.0
        }

        return self._analyze_continued(analysis, sentences, text_lower, words, text)
```

```
def _analyze_continued(self, analysis, sentences, text_lower, words, text):
    """Continuation of analyze – auto-extracted by self-evolution."""
    # Check for LLM phrases
    llm_phrase_count = 0
```

```

for phrase in self.llm_phrases:
    if phrase in text_lower:
        llm_phrase_count += 1
        analysis['indicators']['llm'].append(f'Contains {phrase}&quot;')

# Check for human phrases
human_phrase_count = 0
for phrase in self.human_phrases:
    if phrase in text_lower:
        human_phrase_count += 1
        analysis['indicators']['human'].append(f'Contains {phrase}&quot;')

# Check for LLM patterns
pattern_count = 0
for pattern in self.llm_patterns:
    if re.search(pattern, text_lower):
        pattern_count += 1
        analysis['indicators']['llm'].append(f'Pattern: {pattern[:30]}...')

# Check sentence structure variety
sentence_lengths = [len(s.split()) for s in sentences if s.strip()]
if sentence_lengths:
    length_variety = max(sentence_lengths) - min(sentence_lengths) if len(sentence_lengths) > 1 else 0
    if length_variety < 3:
        analysis['indicators']['llm'].append('Low sentence length variety')
    else:
        analysis['indicators']['human'].append('Good sentence length variety')

# Check for contractions (humans use more)
contractions = re.findall(r'&quot;b\b|w\b&quot;;', text)
if len(contractions) > len(words) * 0.05:
    analysis['indicators']['human'].append('Good use of contractions')

# Check for exclamation overuse (LLM tendency)
exclamation_count = text.count('!')
if exclamation_count > 2:
    analysis['indicators']['llm'].append('Excessive exclamation marks')

# Check for paragraph structure
paragraphs = text.split('\n')

if len(paragraphs) > 3 and all(len(p) > 100 for p in paragraphs if p.strip()):
    analysis['indicators']['llm'].append('Overly structured paragraphs')

# Check for hedging
hedging_words = ['perhaps', 'maybe', 'possibly', 'might', 'could be', 'it seems']
hedge_count = sum(1 for h in hedging_words if h in text_lower)
if hedge_count > 3:
    analysis['indicators']['llm'].append('Excessive hedging')

# Calculate scores
analysis['llm_score'] = min(1.0, (llm_phrase_count * 0.1 + pattern_count * 0.15))
analysis['human_score'] = min(1.0, (human_phrase_count * 0.1 + len(contractions) * 0.02))

return self.__analyze_continued_continued(analysis)

def __analyze_continued_continued(self, analysis):
    &quot;&quot;&quot;Continuation of _analyze_continued - auto-extracted by self-evolution.&quot;&quot;&quot;
    # Adjust for indicators
    analysis['llm_score'] += len(analysis['indicators']['llm']) * 0.05
    analysis['human_score'] += len(analysis['indicators']['human']) * 0.05

    # Normalize
    total = analysis['llm_score'] + analysis['human_score']
    if total > 0:
        analysis['llm_score'] /= total
        analysis['human_score'] /= total
    else:
        analysis['llm_score'] = 0.5
        analysis['human_score'] = 0.5

    # Verdict
    if analysis['human_score'] > 0.6:
        analysis['verdict'] = 'Likely Human'
        analysis['confidence'] = analysis['human_score']
    elif analysis['llm_score'] > 0.6:
        analysis['verdict'] = 'Likely LLM'
        analysis['confidence'] = analysis['llm_score']
    else:
        analysis['verdict'] = 'Uncertain'
        analysis['confidence'] = max(analysis['llm_score'], analysis['human_score'])

    return analysis

def get_improvement_suggestions(self, text: str) -> List[str]:
    &quot;&quot;&quot;
    Get suggestions for making text sound more human.
    &quot;&quot;&quot;
    analysis = self.analyze(text)
    suggestions = []

    for indicator in analysis['indicators']['llm']:
        if 'exclamation' in indicator.lower():
            suggestions.append('Reduce exclamation marks - they can seem overly enthusiastic')
        elif 'hedging' in indicator.lower():
            suggestions.append('Be more direct - less &quot;perhaps&quot; and &quot;maybe&quot;')
        elif 'pattern' in indicator.lower():
            suggestions.append('Vary your sentence structures more')
        elif 'structured' in indicator.lower():

```

```

        suggestions.append('Use more natural paragraph breaks')

# Generic suggestions based on score
if analysis['llm_score'] > 0.5:
    suggestions.extend([
        'Use contractions more (e.g., "I\'m" instead of "I am")',
        'Add conversational filler words occasionally',
        'Vary sentence length more dramatically',
        'Show uncertainty naturally rather than with caveats',
        'Use more direct, casual language'
    ])

return suggestions[:5] # Top 5 suggestions

def make_more_human(self, text: str) -> str:
    """
    Attempt to make text sound more human.
    Note: This is a simple transformation, not AI-powered.
    """
    result = text

    # Expand contractions sometimes
    replacements = [
        ('"I am"', '"I\'m"'),
        ('"I have"', '"I\'ve"'),
        ('"It is"', '"It\'s"'),
        ('"That is"', '"That\'s"'),
        ('"do not"', '"don\'t"'),
        ('"cannot"', '"can\'t"'),
        ('"will not"', '"won\'t"'),
    ]

    for formal, casual in replacements:
        result = result.replace(formal, casual)

    # Remove some LLM-isms
    llm_removals = [
        ('"Certainly! "', '"'),
        ('"Absolutely! "', '"'),
        ('"Great question! "', '"'),
        ('"It\'s important to note that "', '"'),
        ('"It\'s worth noting that "', '"'),
    ]

    for phrase, replacement in llm_removals:
        result = result.replace(phrase, replacement)

    return result

# Global instance
_text_analyzer = None

def get_text_analyzer() -> TextAnalyzer:
    """
    Get or create text analyzer instance
    """
    global _text_analyzer
    if _text_analyzer is None:
        _text_analyzer = TextAnalyzer()
    return _text_analyzer

```

107 lines | 4.8KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/thinking_commands.py

```
"> thinking mode commands
=====
Command parsing for thinking mode control.
">">">

from typing import Dict, Optional

class ThinkingModeCommands:
    ">""">User commands for controlling thinking mode.">""">

    COMMANDS = {
        ">>!thinking on">;: ">>Show internal thinking process">,
        ">>!thinking off">;: ">>Hide internal thinking process">,
        ">>!thinking status">;: ">>Check current thinking mode">,
        ">>!research">;: ">>Force deep research on topic">,
        ">>!memory">;: ">>Show what AI has learned">,
        ">>!read">;: ">>Read and process uploaded file">,
        ">>!link">;: ">>Download and process file from URL">,
        ">>!docs">;: ">>List all uploaded documents">,
        ">>!speak">;: ">>Convert text to speech (TTS)">,
        ">>!imagine">;: ">>Generate an image from description">,
        ">>!export">;: ">>Export conversation or memory">,
        ">>!graph">;: ">>Show knowledge graph">,
        ">>!characters">;: ">>Extract characters from book/document">,
        ">>!timeline">;: ">>Generate timeline from document">,
        ">>!worldbuilding">;: ">>Extract world/setting details">,
        ">>!feedback">;: ">>Get writing feedback on document">,
        ">>!help">;: ">>Show all available commands">,
    }

    @staticmethod
    def parse_command(message: str) -> Optional[Dict]:
        ">>">">Check if message is a command. Supports both ! and / prefixes.">">">
        message_lower = message.lower().strip()

        # Existing ! commands
        if message_lower == ">>!thinking on">;:
            return {">>command">;: ">>thinking_on">}
        elif message_lower == ">>!thinking off">;:
            return {">>command">;: ">>thinking_off">}
        elif message_lower == ">>!thinking status">;:
            return {">>command">;: ">>thinking_status">}
        elif message_lower.startswith(">>!research">;):
            return {">>command">;: ">>research">, ">>topic">;: message[9:].strip()}
        elif message_lower == ">>!memory">;:
            return {">>command">;: ">>memory">}
        elif message_lower.startswith(">>!read">;):
            return {">>command">;: ">>read_file">, ">>filename">;: message[5:].strip()}
        elif message_lower.startswith(">>!link">;):
            return {">>command">;: ">>link">, ">>url">;: message[5:].strip()}
        elif message_lower == ">>!docs">;:
            return {">>command">;: ">>docs">}
        elif message_lower.startswith(">>!speak">;):
            return {">>command">;: ">>speak">, ">>text">;: message[6:].strip()}
        elif message_lower.startswith(">>!imagine">;):
            return {">>command">;: ">>imagine">, ">>prompt">;: message[8:].strip()}
        elif message_lower.startswith(">>!export">;):
            return {">>command">;: ">>export">, ">>target">;: message[7:].strip() or ">>chat">}
        elif message_lower == ">>!graph">;:
            return {">>command">;: ">>graph">}
        elif message_lower == ">>!characters">;:
            return {">>command">;: ">>characters">}
        elif message_lower == ">>!timeline">;:
            return {">>command">;: ">>timeline">}
        elif message_lower == ">>!worldbuilding">;:
            return {">>command">;: ">>worldbuilding">}
        elif message_lower == ">>!feedback">;:
            return {">>command">;: ">>feedback">}
        elif message_lower == ">>!help">;:
            return {">>command">;: ">>help">}

        # Slash commands (from chat.py)
        elif message_lower.startswith(">>/cloud-save">;):
            return {">>command">;: ">>cloud_save">}
        elif message_lower.startswith(">>/cloud-status">;):
            return {">>command">;: ">>cloud_status">}
        elif message_lower.startswith(">>/cloud-load">;):
            return {">>command">;: ">>cloud_load">}
        elif message_lower == ">>/save">;:
            return {">>command">;: ">>save">}
        elif message_lower == ">>/status">;:
            return {">>command">;: ">>status">}
        elif message_lower.startswith(">>/explain">;):
            query = message[8:].strip()
            return {">>command">;: ">>explain_query">, ">>query">;: query} if query else {">>command">;: ">>explain_help">}
        elif message_lower.startswith(">>/knowledge">;):
            topic = message[10:].strip() if len(message) >= 10 else ">>">
            return {">>command">;: ">>knowledge">, ">>topic">;: topic}
        elif message_lower.startswith(">>/analyze">;):
            text = message[8:].strip() if len(message) >= 8 else ">>">
            return {">>command">;: ">>analyze">, ">>text">;: text}
        elif message_lower == ">>/personality">;:
```

```

        return {"command": "", "personality": ""}
    elif message_lower.startswith("/collapse"):
        # /collapse <text>
        text = message[9:].strip() if len(message) > 9 else ""
        return {"command": "", "collapse": "", "text": text}
    elif message_lower.startswith("/hybrid") or message_lower.startswith("/unified"):
        # These require specialized generators; we may not have them in server mode.
        # We'll handle gracefully.
        cmd_name = message[1:].split()[0] if len(message) > 2 else ""
        args = message[len(cmd_name)+2:].strip() if cmd_name else ""
        return {"command": cmd_name, "text": args}

return None

```

71. tone_detector.py

113 lines | 3.4KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/tone_detector.py

```
"""""
Tone Detection - VADER sentiment + register classification
Classifies input into 4 registers: casual, conversational, analytical, philosophical
"""""

try:
    from nltk.sentiment.vader import SentimentIntensityAnalyzer
    _vader = SentimentIntensityAnalyzer()
    HAS_VADER = True
except ImportError:
    HAS_VADER = False

# Deep thinking markers
DEEP_MARKERS = {
    'consciousness', 'quantum', 'reality', 'existence', 'universe',
    'philosophy', 'metaphysics', 'ontology', 'epistemology', 'meaning',
    'truth', 'infinity', 'paradox', 'emergence', 'recursive', 'resonance',
    'collapse', 'superposition', 'entanglement', 'observation', 'identity',
    'perception', 'awareness', 'cognition', 'fundamental', 'abstract',
    'theoretical', 'mathematical', 'dimension', 'spacetime', 'entropy',
    'information', 'complexity', 'determinism', 'causality', 'axiom',
    'transcendent', 'phenomenal', 'subjective', 'objective', 'dualism',
    'monism', 'panpsychism', 'materialism', 'idealism', 'solipsism',
    'microtubule', 'tubulin', 'decoherence', 'wavefunction', 'eigenstate',
}

QUESTION_WORDS = {'what', 'why', 'how', 'when', 'where', 'who', 'which', 'is', 'are', 'do', 'does', 'can', 'could', 'would', 'should'}

def detect_tone(text: str) -> dict:
    """""
    Detect tone and return register classification.

    Returns:
        dict with keys:
            register: 'casual' | 'conversational' | 'analytical' | 'philosophical'
            depth: float 0.0-1.0
            sentiment: dict with pos/neg/neu/compound (if VADER available)
    """""
    words = text.lower().split()
    word_count = len(words)
    word_set = set(words)

    # VADER sentiment
    sentiment = {}
    if HAS_VADER:
        scores = _vader.polarity_scores(text)
        sentiment = {
            'positive': scores['pos'],
            'negative': scores['neg'],
            'neutral': scores['neu'],
            'compound': scores['compound'],
        }

    # Count deep markers
    deep_count = len(word_set & DEEP_MARKERS)

    # Check for question words
    has_question = bool(word_set & QUESTION_WORDS) or text.strip().endswith('?')

    # Calculate depth score
    depth = 0.0

    # Length contributes
    if word_count > 20:
        depth += 0.2
    elif word_count > 10:
        depth += 0.1

    # Deep markers
    depth += min(0.5, deep_count * 0.15)

    # Questions about deep topics
    if has_question and deep_count > 0:
        depth += 0.2

    # Multiple sentences suggest analytical
    sentence_count = text.count('.') + text.count('?') + text.count('!')
    if sentence_count > 2:
        depth += 0.1

    depth = min(1.0, depth)

    # Classify register
    if depth < 0.15:
        register = 'casual'
    elif depth < 0.35:
        register = 'conversational'
    elif depth < 0.65:
        register = 'analytical'
    else:
        register = 'philosophical'
    return {
```

```

        'register': register,
        'depth': round(depth, 3),
        'sentiment': sentiment,
        'deep_markers': deep_count,
        'word_count': word_count,
    }

if __name__ == '__main__':
    tests = [
        'hello',
        'hey what's up',
        'can you explain how microtubules work?',
        'what is the nature of consciousness and how does quantum collapse relate to subjective experience?'
    ]
    for t in tests:
        result = detect_tone(t)
        print(f'{t[:50]:50s} -> {result['register']:15s} depth={result['depth']}')

```

72. training_corpus.py

604 lines | 43.3KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/training_corpus.py

```
"""""
■ COMPREHENSIVE TRAINING CORPUS
=====
Large corpus for training the Markov chains to generate fluent text.
Organized by domain with diverse sentence structures.
"""""

PHILOSOPHY_CORPUS = """""
Consciousness is the subjective experience of awareness that characterizes sentient beings. The hard problem of consciousness asks why physical brain processes give rise to
Free will is the ability to make choices that are not determined by prior causes or divine intervention. Compatibilists argue that free will is compatible with determinism
The meaning of life has been contemplated by philosophers throughout human history. Existentialists like Sartre argued that we create our own meaning through our choices and
Knowledge is traditionally defined as justified true belief, though this definition has been challenged by Gettier cases. Epistemology examines the nature, sources, and limits of
Ethics examines how we ought to live and what actions are morally right or wrong. Deontological ethics focuses on duties and rules that must be followed regardless of consequences.
Reality may be fundamentally different from how it appears to our senses and minds. Idealists argue that reality is ultimately mental or spiritual in nature. Materialists claim that
Truth is correspondence between our beliefs and the way the world actually is. Coherentist theories define truth as internal consistency among a set of beliefs. Pragmatist theories
Personal identity concerns what makes a person the same individual over time. Psychological continuity theories emphasize memory and psychological connections. Biological theories
Time appears to flow from past through present toward future in our experience. The block universe theory suggests all moments in time exist equally and eternally. Presentism holds
Beauty and aesthetic value have been subjects of philosophical inquiry since ancient times. Subjective theories hold that beauty exists only in the perception of the beholder.
"""""

PHYSICS_CORPUS = """""
Quantum mechanics describes the behavior of matter and energy at atomic and subatomic scales. Particles exhibit wave-particle duality, behaving sometimes as particles and sometimes as
The wave function describes the quantum state of a particle or system of particles. Wave function collapse occurs when measurement forces a superposition into a definite state.
Entanglement is a quantum phenomenon where particles become correlated in their properties. Measuring one entangled particle instantly affects the state of its partner regardless of distance.
Special relativity revolutionized our understanding of space and time as unified spacetime. Time dilation means moving clocks run slower relative to stationary observers. Length contraction
General relativity describes gravity as the curvature of spacetime caused by mass and energy. Massive objects bend spacetime causing other objects to follow curved paths we perceive as
The Standard Model describes fundamental particles and the forces that govern their interactions. Quarks combine to form protons and neutrons through the strong nuclear force.
Thermodynamics governs the behavior of heat and energy in physical systems. The first law states that energy cannot be created or destroyed only transformed between forms.
Electromagnetic radiation includes light radio waves X-rays and gamma rays. Photons are the quantum particles that carry electromagnetic force between charged particles. The speed of light
Nuclear physics studies the properties and behavior of atomic nuclei. Nuclear fission splits heavy nuclei releasing enormous amounts of energy. Nuclear fusion combines light nuclei to form
Particle accelerators probe the fundamental structure of matter at the smallest scales. The Large Hadron Collider discovered the Higgs boson confirming the origin of mass.
"""""

COMPUTER_SCIENCE_CORPUS = """""
Algorithms are precise step-by-step procedures for solving computational problems. Time complexity measures how the running time grows with input size. Space complexity measures the
Machine learning enables computers to learn patterns from data without explicit programming. Supervised learning uses labeled examples to train predictive models. Unsupervised learning
Neural networks are computing systems inspired by biological brains composed of interconnected nodes. Deep learning uses many layers of neurons to learn hierarchical representations of
Data structures organize information for efficient storage and retrieval operations. Arrays provide constant time access to elements by index position. Trees organize data in a hierarchical
Programming languages provide abstractions for expressing computational procedures. High level languages like Python emphasize readability and developer productivity. Low level languages
Databases store and organize large amounts of structured information for retrieval. Relational databases use tables with rows and columns linked by keys. NoSQL databases provide flexible
Computer networks connect devices to share resources and communicate information. The internet is a global network of networks using standardized protocols. The world wide web
Operating systems manage computer hardware and provide services for applications. Process scheduling determines which programs run and for how long. Memory management allocates and
Cryptography protects information through mathematical transformations and algorithms. Public key encryption uses paired keys for secure communication. Hash functions create unique
Artificial intelligence aims to create systems that exhibit intelligent behavior. Natural language processing enables computers to understand human language. Computer vision
"""""

BIOLOGY_CORPUS = """""
Evolution by natural selection explains the diversity of life on Earth. Variation arises through genetic mutation and sexual recombination. Selection favors traits that enhance an organism's
DNA carries genetic information encoded in sequences of nucleotide bases. Genes are segments of DNA that code for specific proteins. Transcription copies DNA into RNA which is then
Cells are the fundamental units of life capable of independent metabolism and reproduction. Prokaryotic cells lack membrane-bound organelles while eukaryotic cells contain
The brain processes information through networks of billions of interconnected neurons. Synapses transmit signals between neurons using chemical neurotransmitters. Neural plasticity
Ecosystems are communities of organisms interacting with their physical environment. Energy flows through food webs from producers to consumers to decomposers. Nutrient cycles
Proteins are complex molecules that perform most cellular functions. Enzymes catalyze biochemical reactions by lowering activation energy. Structural proteins provide support and
Photosynthesis converts light energy into chemical energy stored in glucose. Chloroplasts contain chlorophyll that captures photons from sunlight. Plants release oxygen as a byproduct
The immune system defends the body against pathogens and disease. Antibodies are proteins that recognize and neutralize foreign substances. T cells coordinate immune responses and
Genetics studies heredity and variation in living organisms. Mendel discovered the basic laws of inheritance through pea plant experiments. Modern genetics explores molecular mechanisms
Ecology examines relationships between organisms and their environments. Population dynamics models growth competition and predator prey interactions. Conservation biology
"""""
```

MATHEMATICS_CORPUS = """;
Mathematics studies abstract structures and their relationships through rigorous proof. Axioms are fundamental assumptions accepted without proof as starting points. Theorems are statements that can be proven true using axioms and logical reasoning.
Numbers represent quantities and support arithmetic operations like addition and multiplication. Natural numbers count discrete objects while real numbers include all points on a number line.
Algebra generalizes arithmetic using symbols to represent unknown or variable quantities. Equations express relationships between quantities that can be solved for unknowns.
Geometry studies shapes sizes and positions of figures in space. Euclidean geometry is based on five axioms including the parallel postulate. Non-Euclidean geometries arise from different assumptions.
Calculus analyzes continuous change through differentiation and integration. Derivatives measure instantaneous rates of change of functions. Integrals compute accumulated quantities.
Probability quantifies uncertainty using numbers between zero and one. Random variables assign numerical values to outcomes of random experiments. Distributions describe the behavior of random variables.
Set theory provides foundations for modern mathematics through collections of objects. Operations include union intersection and complement of sets. Cardinality measures the size of sets.
Logic studies valid reasoning and the structure of mathematical arguments. Propositional logic deals with truth values of simple statements. Predicate logic adds quantifiers to express relationships.
Topology studies properties preserved under continuous deformations of space. Homeomorphisms are continuous bijections with continuous inverses. Manifolds are spaces that locally resemble Euclidean space.
Number theory investigates properties of integers and prime numbers. The fundamental theorem states every integer factors uniquely into primes. Famous unsolved problems include the Goldbach conjecture and the Riemann hypothesis.
""";
PSYCHOLOGY_CORPUS = """;
Psychology studies mind behavior and mental processes through scientific methods. Cognitive psychology examines perception attention memory and reasoning. Clinical psychology focuses on diagnosing and treating mental disorders.
Memory encodes stores and retrieves information from past experiences. Working memory holds information temporarily for active manipulation. Long-term memory preserves information for extended periods.
Learning changes behavior through experience and interaction with the environment. Classical conditioning associates stimuli through repeated pairing. Operant conditioning uses reinforcement and punishment to shape behavior.
Emotion involves physiological arousal subjective feelings and behavioral expression. Basic emotions may be universal across human cultures. Emotional regulation involves managing one's emotional responses.
Perception constructs meaningful experiences from raw sensory information. Bottom-up processing builds perception from basic features. Top-down processing applies knowledge and expectations to interpret sensory input.
Motivation drives and directs behavior toward goals and desired outcomes. Intrinsic motivation comes from internal satisfaction and interest. Extrinsic motivation depends on external rewards or incentives.
Personality encompasses consistent patterns of thinking feeling and behaving. Trait theories describe personality using dimensional characteristics. Psychodynamic theories focus on unconscious processes and early childhood experiences.
Development tracks psychological changes across the human lifespan. Cognitive development involves qualitative changes in thinking abilities. Social development concerns the formation of social relationships.
Social psychology examines how people influence and relate to each other. Conformity and obedience demonstrate the power of social situations. Attribution explains how we interpret the causes of behavior.
Consciousness is the subjective awareness of internal and external experiences. Altered states include sleep dreaming meditation and hypnosis. The unconscious influences behavior through automatic processes.
""";
LANGUAGE_CORPUS = """;
Language enables communication through structured systems of symbols and rules. Phonology studies the sound systems and patterns of languages. Morphology examines the structure and formation of words.
Syntax governs how words combine into phrases and sentences. Grammar rules specify which combinations are considered well-formed. Parsing involves analyzing the grammatical structure of sentences.
Semantics studies meaning in language at word phrase and sentence levels. Denotation refers to the literal meaning of words and expressions. Connotation encompasses associated ideas and emotions.
Pragmatics examines how context influences language interpretation and use. Speech acts perform functions like requesting promising and asserting. Implicature conveys meaning that is not explicitly stated.
Language acquisition occurs naturally in children exposed to linguistic input. Critical periods may limit native-like attainment of second languages. Universal grammar proposes an innate language faculty.
Communication transfers information between speakers through verbal and nonverbal channels. Encoding converts thoughts into signals for transmission. Decoding interprets received signals back into meaning.
Writing represents language through visual symbols on permanent media. Alphabets map sounds to letters while logographies map meanings to symbols. Literacy enables the preservation and transmission of written language.
Translation converts meaning from one language into another language. Equivalence may be formal preserving structure or dynamic preserving effect. Machine translation uses computational models.
Discourse analysis studies language use in context beyond individual sentences. Coherence concerns logical connections between parts of text. Cohesion involves linguistic devices that link sentences together.
Sociolinguistics examines relationships between language and society. Dialects vary language by region and social group. Language change occurs gradually over time through natural processes.
""";
Combine all corpora
COMMON_TOPICS_CORPUS = """;
Gravity is the force that attracts objects with mass toward each other.
Gravity causes objects to fall toward Earth and keeps planets in orbit around the sun.
Gravitational force depends on the mass of objects and the distance between them.
Gravity is fundamental to the structure of the universe and how celestial bodies interact.
Gravity bends spacetime according to Einstein's general theory of relativity.
Gravity keeps the moon in orbit around Earth and Earth in orbit around the sun.
Gravitational waves are ripples in spacetime caused by massive accelerating objects.
Gravity is one of the four fundamental forces of nature along with electromagnetism and nuclear forces.
Gravity holds galaxies together and shapes the large scale structure of the cosmos.
Gravity acts instantaneously across any distance according to Newton but actually travels at light speed.

Time is the continuous progression of events from past through present to future.
Time dilation occurs when objects move at high speeds relative to an observer.
Time appears to flow in one direction according to our everyday experience.
Time is a dimension that combines with space to form the fabric of spacetime.
Time measurement began with observations of the sun moon and stars by ancient civilizations.
Time perception varies based on attention emotion and cognitive processes.
Time may be an emergent property rather than a fundamental feature of reality.
Time travel remains theoretical though special relativity allows for forward time travel.
Time is relative meaning it passes differently for observers in different reference frames.
Time cannot be separated from space in modern physics since they form a unified spacetime.

Space is the boundless three dimensional extent in which objects exist and events occur.
Space and time are unified as spacetime in Einstein's theory of relativity.
Spacetime curvature caused by mass is what we experience as gravity.
Space may be infinite or finite depending on the geometry of the universe.
Space contains vast empty regions between galaxies stars and planets.
Space exploration has revealed much about our solar system and beyond.
Space is not truly empty but contains quantum fluctuations and dark energy.
Spatial dimensions include length width and height as we perceive them.
Space expands continuously carrying galaxies away from each other.
Spacetime can be warped bent and even torn under extreme conditions.
Energy is the capacity to do work and exists in many forms including kinetic and potential.

Energy cannot be created or destroyed only transformed from one form to another.
Energy flows through ecosystems from producers to consumers to decomposers.
Energy drives all physical processes from chemical reactions to biological functions.
Energy is equivalent to mass according to Einstein's famous equation $E = mc^2$.
Energy sources include fossil fuels nuclear reactions solar radiation and geothermal heat.
Energy efficiency determines how much useful work can be extracted from a process.
Energy in the universe is conserved though it tends to spread out and become less useful.
Energy manifests as heat light motion sound and many other phenomena.
Energy transfer occurs through radiation conduction and convection.

Matter is anything that has mass and occupies space in the physical world.
Matter exists in states including solid liquid gas and plasma.
Matter is composed of atoms which contain protons neutrons and electrons.
Matter and energy are interconvertible according to Einstein's relativity.
Matter interacts through the four fundamental forces of nature.
Matter at the quantum level exhibits both particle and wave properties.
Matter makes up only about five percent of the total energy content of the universe.
Matter can be neither created nor destroyed in chemical reactions only rearranged.
Matter organizes into complex structures from atoms to molecules to living organisms.
Matter in the early universe was nearly uniformly distributed before gravity formed structures.

Life is the condition that distinguishes organisms from inorganic matter.
Life evolved on Earth over billions of years through natural selection.
Life requires energy metabolism reproduction and response to environment.
Life exhibits remarkable diversity from bacteria to blue whales to giant sequoias.
Life on Earth may have originated in hydrothermal vents or from meteors.
Life is characterized by growth development adaptation and homeostasis.
Life forms complex ecosystems with intricate relationships between species.
Life uses DNA or RNA to store and transmit genetic information.
Life might exist elsewhere in the universe given the vast number of planets.
Living systems maintain order by constantly consuming energy from their environment.

Intelligence is the ability to learn reason solve problems and adapt to new situations.
Intelligence can be measured in various ways though definitions remain controversial.
Artificial intelligence aims to create machines that exhibit intelligent behavior.
Intelligence evolved as a survival advantage allowing flexible responses to challenges.
Intelligence involves memory attention pattern recognition and decision making.
Intelligence appears in varying degrees across the animal kingdom.
Intelligence is not a single ability but a collection of cognitive capacities.
Intelligence tests measure certain cognitive abilities but not creativity or emotional skill.
Intelligence enables abstract thinking language use and complex social interaction.
Intelligent systems can optimize their behavior based on feedback and experience.

Truth is correspondence between statements and the actual state of affairs.
Truth is what we seek through inquiry and investigation.
Truth claims must be supported by evidence and logical reasoning.
Truth may be objective existing independently of human beliefs.
Truth theories include correspondence coherence and pragmatic approaches.
Truth is essential for knowledge which is traditionally defined as justified true belief.
Truth in science is provisional always subject to revision based on new evidence.
Truth telling is generally considered a moral virtue and foundation of trust.
Truth can be difficult to establish when evidence is limited or conflicting.
Truth is the goal of philosophical inquiry and scientific investigation.

Beauty is a quality that gives pleasure to the senses or mind.
Beauty may be objective or subjective depending on philosophical perspective.
Beauty in nature and art has inspired humans throughout history.
Beauty standards vary across cultures and historical periods.
Beauty is studied in aesthetics a branch of philosophy.
Beauty is often associated with symmetry proportion and harmony.
Beautiful experiences trigger positive emotions and reward responses in the brain.
Beauty in mathematics refers to elegance simplicity and surprising connections.
Beauty can be found in scientific theories that explain much with little.
Beauty appreciation appears to be universal though specific preferences vary.

Love is a complex emotion involving affection attachment and care for others.
Love takes many forms including romantic familial and platonic bonds.
Love has been explored in philosophy literature art and science.
Love involves both emotional and physiological responses including hormone changes.
Love is associated with brain regions involved in reward and attachment.
Love can lead to both joy and suffering depending on circumstances.
Love between parents and children is fundamental to human development.
Loving relationships contribute to health happiness and wellbeing.
Love is a central theme in religious and spiritual traditions worldwide.
Love requires vulnerability trust and mutual commitment to flourish.

Freedom is the power to act speak or think without external restraint.
Freedom requires responsibility and consideration of consequences.
Freedom of thought is essential for philosophical and scientific inquiry.
Freedom has been a central value in political philosophy for centuries.
Freedom can be positive having the ability to do things or negative absence of constraint.
Free will debates center on whether our choices are truly free or determined.
Freedom of speech is protected in democratic societies as essential to open discourse.
Freedom is never absolute but exists within a framework of social norms and laws.
Freedom must be balanced with other values like security equality and community.
Freedom of religion allows individuals to practice their beliefs without persecution.

Knowledge is justified true belief according to traditional epistemology.
Knowledge is acquired through experience reason and testimony.
Knowledge grows through questioning testing and revising our beliefs.
Knowledge can be empirical based on observation or rational based on reasoning.
Knowledge is organized into disciplines such as science philosophy and mathematics.
Knowledge is power because it enables effective action in the world.
Knowledge is social constructed and transmitted through culture and education.
Knowledge has limits we cannot know everything and some things may be unknowable.
Knowledge requires both individual effort and collective validation.
Knowledge advances through both accumulation of facts and revolutionary paradigm shifts.

Reality is the state of things as they actually exist independently of perception.
Reality at the quantum level differs from our everyday experience.
Reality may be fundamentally different from how it appears to our senses.

Reality is the subject of metaphysics the branch of philosophy studying existence.
Reality includes both physical objects and abstract entities like numbers.
Reality is what remains when we stop believing in it as Philip K Dick said.
Reality testing helps distinguish between fantasy and actual states of affairs.
Reality may be a simulation according to some philosophical arguments.
Reality is constructed through the interaction of mind and world.
Real things have causal powers and can affect other things.

Death is the permanent cessation of biological functions in living organisms.
Death raises profound philosophical questions about meaning and existence.
Death has been contemplated by philosophers and religions throughout history.
Death is inevitable for all living organisms though timing is uncertain.
Death awareness may be unique to humans among animals.
Death rituals vary widely across cultures but all societies mark the transition.
Death can be defined in multiple ways including brain death and cardiac death.
Death anxiety is a psychological phenomenon that influences human behavior.
Death gives life urgency and meaning according to existentialist thinkers.
Dead organisms decompose returning their matter to the environment.

Happiness is a state of well being characterized by positive emotions and life satisfaction.
Happiness can be pursued through relationships achievement and meaning.
Happiness research examines factors that contribute to human flourishing.
Happiness involves both momentary pleasure and deeper life satisfaction.
Happiness is influenced by genetics circumstances and intentional activity.
Happiness is not the same as constant pleasure but includes contentment and purpose.
Happy people tend to be healthier more productive and more socially connected.
Happiness is a central concern of positive psychology and wellbeing science.
Happiness can be cultivated through practices like gratitude and mindfulness.
Happiness is both a goal in itself and a contributor to other good outcomes.

Music is an art form consisting of organized sounds and silences over time.
Music affects emotions and has been part of human culture throughout history.
Music can be analyzed through rhythm melody harmony and timbre.
Music activates multiple brain regions and can enhance learning and memory.
Musical ability appears to have evolutionary roots and developmental stages.
Music therapy uses musical activities to achieve therapeutic goals.
Musical preferences are shaped by culture exposure and individual personality.
Music brings people together and plays important roles in rituals and celebrations.
Musicians develop specialized brain structures through years of practice.
Musical notation allows compositions to be preserved and shared across time.

Art is the expression of human creativity and imagination in visual or performative forms.
Art serves many purposes including aesthetic pleasure and social commentary.
Art history traces the development of creative expression across cultures.
Art can challenge perceptions and provoke thought about important issues.
Artistic movements like impressionism and surrealism transformed visual culture.
Art is valued both for intrinsic beauty and cultural significance.
Artists develop skills through training but also express unique personal visions.
Art markets assign monetary value to works based on complex factors.
Art appreciation involves both emotional response and intellectual understanding.
Artistic expression is a fundamental human capacity found in all cultures.

Science is the systematic study of the natural world through observation and experiment.
Science produces knowledge that can be tested verified and built upon.
Scientific method involves hypothesis testing data collection and analysis.
Science aims for objective truth though it is conducted by fallible humans.
Scientific theories are never proven only supported or refuted by evidence.
Scientists work within paradigms that shape what questions they ask.
Science has enabled tremendous technological progress and understanding.
Scientific literacy helps citizens make informed decisions about complex issues.
Science is collaborative building on the work of previous researchers.
Scientific fraud and bias are serious problems that undermine public trust.

Technology is the application of scientific knowledge for practical purposes.
Technology has transformed human civilization enabling new capabilities.
Technological progress raises both opportunities and ethical concerns.
Technology includes tools machines and techniques for solving problems.
Technological innovation drives economic growth and social change.
Technology can enhance human capabilities or replace human labor.
Technological determinism suggests that technology shapes society and culture.
Technological risks include environmental damage and unintended consequences.
Technology access is unequally distributed creating digital divides.
Technological ethics addresses questions about appropriate use and development.

Religion is a system of beliefs and practices relating to the sacred or divine.
Religion provides meaning community and moral guidance for many people.
Religious traditions vary widely in their doctrines rituals and worldviews.
Religion has played a central role in human history and civilization.
Religious experience includes feelings of transcendence and connection.
Religion and science have had complex relationships throughout history.
Religious fundamentalism emphasizes literal interpretation of sacred texts.
Religious pluralism recognizes the validity of multiple spiritual paths.
Religion shapes values customs and social structures in many societies.
Religious freedom is a fundamental human right in democratic societies.

Philosophy is the study of fundamental questions about existence knowledge and values.
Philosophy uses reason and argument to examine concepts and beliefs.
Philosophical inquiry has shaped human thought for thousands of years.
Philosophy includes ethics epistemology metaphysics and logic.
Philosophical questions often have no definitive answers but deepen understanding.
Philosophy examines the foundations and methods of other disciplines.
Philosophical traditions vary across cultures from Western to Eastern.
Philosophy can be practical helping people live better lives.
Philosophical skills include critical thinking analysis and clear argumentation.
Philosophy matters because it addresses questions everyone faces about life and meaning.

History is the study of past events and their significance for the present.
History helps us understand how societies and cultures have developed.
Historical knowledge comes from documents artifacts and other evidence.
History is written by historians who interpret evidence through their perspectives.
Historical events have multiple causes and complex consequences.

History teaches lessons though they are often contested and debated.
Historical consciousness shapes identity and influences current decisions.
History includes social economic cultural and political dimensions.
Historical memory can be selective emphasizing some events over others.
History is not just about the past but about how the past relates to present and future.

Mathematics is the abstract science of number quantity and space.
Mathematics provides tools for reasoning about patterns and relationships.
Mathematical truths are discovered through proof and logical deduction.
Mathematics is the language of science used to describe natural phenomena.
Mathematical models simplify reality to capture essential features.
Mathematics includes algebra geometry calculus and statistics.
Mathematical beauty lies in elegant proofs and surprising connections.
Mathematics develops through both pure inquiry and practical application.
Mathematical ability varies among individuals but can be developed.
Mathematics is universal applying across all cultures and contexts.

Consciousness is the subjective experience of awareness and self-knowledge.
Consciousness remains one of the deepest mysteries in philosophy and science.
Conscious experience includes perception thought emotion and sensation.
Consciousness may emerge from complex neural processes in the brain.
Conscious beings have a perspective a what it is like to be them.
Consciousness can be altered through meditation drugs or brain injury.
Conscious and unconscious processes both influence human behavior.
Consciousness raises questions about free will moral responsibility and identity.
Consciousness may be more widespread in nature than we typically assume.
Conscious machines remain theoretical though AI continues to advance.

Quantum mechanics describes the behavior of matter at the smallest scales.
Quantum particles exhibit wave-particle duality showing both properties.
Quantum superposition allows particles to be in multiple states simultaneously.
Quantum entanglement links particles across distances in strange ways.
Quantum measurement causes wave function collapse into definite states.
Quantum uncertainty means some properties cannot be known precisely.
Quantum computing uses superposition for powerful parallel processing.
Quantum physics challenges our intuitions about reality and causation.
Quantum theory is among the most successful in physics despite being strange.
Quantum effects are typically only visible at atomic and subatomic scales.

Evolution is the process by which life changes over generations.
Evolution occurs through natural selection acting on genetic variation.
Evolutionary theory explains the diversity and adaptation of life forms.
Evolution has shaped human bodies brains and behaviors over millions of years.
Evolutionary psychology applies these insights to understanding human nature.
Evolution is supported by evidence from fossils genetics and comparative anatomy.
Evolutionary change can be gradual or punctuated by rapid transitions.
Evolution has no goal or direction only differential reproductive success.
Evolutionary arms races occur between predators and prey or hosts and parasites.
Evolution continues today in all living populations including humans.
"""

NEW: Astrophysics specific content
ASTROPHYSICS_CORPUS = """
Black holes are regions of spacetime where gravity is so strong that nothing can escape.
Black holes form when massive stars collapse at the end of their lives.
The event horizon is the boundary around a black hole beyond which escape is impossible.
Black holes bend light around them creating gravitational lensing effects.
Supermassive black holes exist at the centers of most galaxies including our Milky Way.
Black holes emit Hawking radiation slowly losing mass over immense timescales.
Stellar black holes form from stars at least twenty times more massive than our sun.
Black holes distort spacetime so severely that time appears to stop at the event horizon.
The singularity at a black hole center is where density becomes infinite.
Black hole mergers produce gravitational waves that ripple through spacetime.

Neutron stars are the collapsed cores of massive stars after supernova explosions.
Neutron stars are incredibly dense with a teaspoon weighing billions of tons.
Pulsars are rapidly rotating neutron stars that emit beams of electromagnetic radiation.
Neutron star matter is compressed so tightly that protons and electrons merge into neutrons.
The magnetic fields of neutron stars are trillions of times stronger than Earth's.
Neutron stars typically measure only about twenty kilometers in diameter.
A neutron star forms when a star between eight and twenty solar masses explodes.
The rotation rates of pulsars are extremely stable making them cosmic clocks.
Magnetars are neutron stars with exceptionally powerful magnetic fields.
Neutron star collisions produce heavy elements like gold and platinum.

Superposition is a fundamental quantum principle where particles exist in multiple states simultaneously.
In superposition a particle can be in all possible positions or states at once.
Superposition only collapses into a definite state when observed or measured.
Being in superposition is like existing everywhere and nowhere at the same time.
Schrodinger's cat thought experiment illustrates superposition at macroscopic scales.
Quantum computers use superposition to process many calculations simultaneously.
Superposition is destroyed by decoherence when quantum systems interact with their environment.
The wave function mathematically describes all possible states in superposition.
Quantum superposition enables interference patterns in the double slit experiment.
Maintaining superposition requires isolating quantum systems from thermal noise.

Stars are massive balls of hot gas held together by their own gravity.
Stars generate energy through nuclear fusion converting hydrogen to helium.
The life cycle of a star depends on its initial mass when it forms.
Our sun is a medium sized yellow dwarf star about halfway through its life.
Red giants are stars that have exhausted their core hydrogen and expanded.
White dwarfs are the dense remnants of stars similar to our sun.
Binary star systems contain two stars orbiting their common center of mass.
Star formation occurs in giant molecular clouds of gas and dust.
The color of a star indicates its surface temperature from red to blue.
Stars create heavy elements through fusion that are scattered when they die.

Galaxies are vast collections of stars gas dust and dark matter bound by gravity.
The Milky Way is a barred spiral galaxy containing hundreds of billions of stars.
Elliptical galaxies are smooth and round with little ongoing star formation.
Spiral galaxies have distinctive arms where new stars are actively forming.

Galaxy clusters contain thousands of galaxies bound together gravitationally.
 Dark matter provides most of the gravitational binding in galaxies.
 Galaxies evolve through mergers and collisions over cosmic timescales.
 Active galactic nuclei are powered by supermassive black holes consuming matter.
 The Andromeda galaxy will collide with the Milky Way in about four billion years.
 Dwarf galaxies are small systems that orbit larger host galaxies.

The universe began approximately fourteen billion years ago in the Big Bang.
 The universe continues to expand carrying galaxies away from each other.
 Dark energy drives the accelerating expansion of the universe.
 The cosmic microwave background is radiation from when the universe first became transparent.
 The observable universe is limited by the distance light has traveled since the Big Bang.
 Inflation theory suggests the universe expanded exponentially in its first fraction of a second.
 The universe appears flat meaning parallel lines remain parallel over cosmic distances.
 Most of the universe consists of dark matter and dark energy which remain mysterious.
 The ultimate fate of the universe depends on whether expansion continues forever.
 Cosmic voids are vast regions of space containing very few galaxies.
 """

```
FULL_CORPUS = {
    &quot;philosophy&quot;: PHILOSOPHY_CORPUS,
    &quot;physics&quot;: PHYSICS_CORPUS,
    &quot;computer_science&quot;: COMPUTER_SCIENCE_CORPUS,
    &quot;biology&quot;: BIOLOGY_CORPUS,
    &quot;mathematics&quot;: MATHEMATICS_CORPUS,
    &quot;psychology&quot;: PSYCHOLOGY_CORPUS,
    &quot;language&quot;: LANGUAGE_CORPUS,
    &quot;common_topics&quot;: COMMON_TOPICS_CORPUS,
    &quot;astrophysics&quot;: ASTROPHYSICS_CORPUS
}

def _clean_text(text: str) -&gt; str:
    &quot;&quot;&quot;Remove terminal commands, pip output, code syntax, and other noise from text.&quot;&quot;&quot;
    import re
    lines = text.split('
')

    cleaned_lines = []

    # Patterns that indicate lines to skip (terminal/REPL/output)
    skip_patterns = [
        r'^\$ ',          # shell prompt ($ command)
        r'^&gt;&gt;&gt; ',       # Python REPL
        r'^&gt; ',          # generic prompt
        r'^In \[&gt;+&gt;]:',  # IPython (In [1]:)
        r'^\[.+\] %',     # progress bar like [##...] 50%
        r'^apt ',         # apt command
        r'^pip ',         # pip command
        r'^sudo ',        # sudo command
        r'^cd ',          # cd command
        r'^ls ',          # ls command
        r'^cat ',         # cat command
        r'^echo ',        # echo command
        r'^sh:',          # shell output prefix (sh: command)
        r'^/bin/',        # binary path
        r'^# '           # lines that are only a comment (no preceding text)
    ]

    for line in lines:
        stripped = line.strip()
        if not stripped:
            cleaned_lines.append(line)
            continue

        # Check if line matches any skip pattern
        if any(re.match(pat, stripped) for pat in skip_patterns):
            continue

        # Skip lines that are almost entirely non-alphanumeric (e.g., ----, ====, ***)
        # Keep if there is at least one alphanumeric character
        if not re.search(r'[A-Za-z0-9]', stripped):
            continue

        # Skip lines that look like raw progress bars (e.g., [:::~:~:~:] or [...;~;~;~])
        if re.match(r'^\[[:~:~:~:~:~:~:]$|^[~;~;~;~;~;~]$ ', stripped):
            continue

        cleaned_lines.append(line)

    return '
'.join(cleaned_lines)

def get_all_corpus_text() -&gt; str:
    &quot;&quot;&quot;Get all corpus text combined.&quot;&quot;&quot;
    combined = &quot;&quot;&quot;
    &quot;&quot;.join(FULL_CORPUS.values())

    return _clean_text(combined)

def get_domain_corpus(domain: str) -&gt; str:
    &quot;&quot;&quot;Get corpus for specific domain.&quot;&quot;&quot;
    raw = FULL_CORPUS.get(domain, &quot;&quot;&quot;)
    return _clean_text(raw)

def get_corpus_sentences() -&gt; list:
    &quot;&quot;&quot;Get all sentences from corpus.&quot;&quot;&quot;
    import re
```

```
text = get_all_corpus_text()
sentences = re.split(r'(?<=[.!?])\s+', text)
return [s.strip() for s in sentences if len(s.strip()) > 20]
```

73. wikidata_ingester.py

228 lines | 8.0KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V2/backend/wikidata_ingester.py

```
"""""
Wikidata Knowledge Ingestor
Pulls structured entity facts from Wikidata and stores in knowledge_graph.json
"""""
import json, urllib.request, urllib.parse, os, re, time, time

DATA_DIR = os.path.expanduser('~/.quantum-mcagi')
KG_PATH = os.path.join(DATA_DIR, 'knowledge_graph.json')

# Wikidata property labels we care about
PROP_LABELS = {
    'P31': 'is_a',          # instance of
    'P17': 'country',       # country
    'P30': 'continent',     # continent
    'P36': 'capital',       # capital
    'P18': 'image',         # skip
    'P856': 'website',      # skip
    'P625': 'coordinates',  # skip
    'P571': 'founded',      # inception
    'P576': 'dissolved',    # dissolution
    'P101': 'field',        # field of work
    'P361': 'part_of',      # part of
    'P527': 'has_part',     # has part
    'P1566': 'geonames',    # skip
    'P18': 'image',         # skip
    'P856': 'website',      # skip
    'P19': 'birth_place',   # place of birth
    'P20': 'death_place',   # place of death
    'P569': 'born',         # date of birth
    'P570': 'died',         # date of death
    'P21': 'gender',        # sex or gender
    'P106': 'occupation',   # occupation
    'P27': 'nationality',   # country of citizenship
    'P131': 'located_in',   # located in admin entity
    'P276': 'location',     # location
    'P571': 'created',      # inception
    'P582': 'ended',        # end time
    'P2044': 'elevation',   # elevation
    'P1082': 'population',  # population
    'P2046': 'area',        # area
    'P421': 'timezone',     # timezone
    'P37': 'language',      # official language
    'P38': 'currency',      # currency
    'P35': 'head_of_state', # head of state
    'P6': 'head_of_gov',    # head of government
    'P122': 'gov_type',     # basic form of government
    'P41': 'flag',          # skip
    'P94': 'coat_of_arms',  # skip
}

SKIP_PROPS = {'P18', 'P856', 'P41', 'P94', 'P1566', 'P625', 'P242', 'P158', 'P948'}

def get_entity_id(title):
    """""Get Wikidata entity ID from Wikipedia title."""""
    url = f"https://www.wikidata.org/w/api.php?action=wbgetentities&sites=enwiki&titles={urllib.parse.quote(title)}&props=ids&format=json"
    for attempt in range(3):
        try:
            req = urllib.request.Request(url, headers={'User-Agent': 'QuantumMCAGI/1.0 (educational project)'})
            with urllib.request.urlopen(req, timeout=15) as r:
                data = json.load(r)
                entities = data.get('entities', {})
                for eid, edata in entities.items():
                    if eid != '-1':
                        return eid
                return None
        except Exception as e:
            if '429' in str(e):
                print(f"Rate limited, waiting 10s...")
                time.sleep(30)
            else:
                print(f"Error getting entity ID: {e}")
                return None
    return None

def get_label(entity_id, lang='en'):
    """""Get human-readable label for a Wikidata entity."""""
    url = f"https://www.wikidata.org/w/api.php?action=wbgetentities&ids={entity_id}&props=labels&languages=en&format=json"
    try:
        req = urllib.request.Request(url, headers={'User-Agent': 'QuantumMCAGI/1.0 (educational project)'})
        with urllib.request.urlopen(req, timeout=10) as r:
            data = json.load(r)
            return data['entities'][entity_id]['labels'].get(lang, {}).get('value', entity_id)
    except:
        return entity_id

def get_entity_facts(entity_id):
    """""Pull structured facts for an entity."""""
    url = f"https://www.wikidata.org/w/api.php?action=wbgetentities&ids={entity_id}&props=claims|labels|descriptions&languages=en&format=json"
    try:
        req = urllib.request.Request(url, headers={'User-Agent': 'QuantumMCAGI/1.0 (educational project)'})
        with urllib.request.urlopen(req, timeout=15) as r:
            data = json.load(r)
    except Exception as e:
        print(f"Error fetching entity: {e}")
```

```

        return {}

entity = data['entities'].get(entity_id, {})
label = entity.get('labels', {}).get('en', {}).get('value', entity_id)
description = entity.get('descriptions', {}).get('en', {}).get('value', '')
claims = entity.get('claims', {})

facts = {
    '_id': entity_id,
    '_label': label,
    '_description': description,
    '_relations': {}
}

for prop, claim_list in claims.items():
    if prop in SKIP_PROPS:
        continue
    prop_name = PROP_LABELS.get(prop, prop)
    values = []
    for claim in claim_list[:3]: # Max 3 values per property
        try:
            snak = claim['mainsnak']
            if snak['snaktype'] != 'value':
                continue
            val = snak['datavalue']['value']
            vtype = snak['datavalue']['type']

            if vtype == 'string':
                values.append(val)
            elif vtype == 'monolingualtext':
                if val.get('language') == 'en':
                    values.append(val['text'])
            elif vtype == 'wikibase-entityid':
                # Resolve entity ID to label
                eid = val['id']
                elabel = get_label(eid)
                values.append(elabel)
            elif vtype == 'quantity':
                amount = val['amount'].rstrip('+')
                unit = val.get('unit', '')
                if 'Q' in unit:
                    unit_id = unit.split('/')[1]
                    unit_label = get_label(unit_id)
                    values.append(f'{amount} {unit_label}"')
                else:
                    values.append(amount)
            elif vtype == 'time':
                values.append(val['time'][:11]) # Just the date part
            elif vtype == 'globecoordinate':
                values.append(f'{val["latitude"]:.2f} lon:{val["longitude"]:.2f}"')
        except Exception:
            continue

    if values:
        facts['_relations'][prop_name] = values

return facts

def load_kg():
    try:
        with open(KG_PATH) as f:
            return json.load(f)
    except:
        return {}

def save_kg(kg):
    with open(KG_PATH, 'w') as f:
        json.dump(kg, f, indent=2)

def ingest_entity(title):
    """Ingest a Wikipedia article title as structured knowledge."""
    print(f" Fetching: {title}...")
    eid = get_entity_id(title)
    if not eid:
        print(f" Not found: {title}")
        return False

    facts = get_entity_facts(eid)
    if not facts:
        print(f" No facts: {title}")
        return False

    kg = load_kg()
    key = title.lower().replace('_', ' ')
    kg[key] = facts
    save_kg(kg)

    rel_count = len(facts.get('_relations', {}))
    print(f" ✓ {facts['_label']}: {facts['_description'][:60]}...")
    print(f" {rel_count} structured relations stored")
    return True

def query_kg(entity_name):
    """Query knowledge graph for an entity."""
    kg = load_kg()
    key = entity_name.lower().strip()
    if key in kg:
        return kg[key]
    # Fuzzy match
    for k in kg:
        if key in k or k in key:

```

```

        return kg[k]
    return None

def kg_to_facts(entity_name):
    """Convert KG entry to readable fact strings for response injection"""
    entry = query_kg(entity_name)
    if not entry:
        return []
    facts = []
    label = entry.get('_label', entity_name)
    desc = entry.get('_description', '')
    if desc:
        facts.append(f'{label} is {desc}')
    for rel, values in entry.get('_relations', {}).items():
        if rel.startswith('_'):
            continue
        facts.append(f'{label} {rel.replace('_', ' ')} {' '.join(str(v) for v in values[2:])}')
    return facts[:8] # Max 8 facts per entity

if __name__ == '__main__':
    import sys
    if len(sys.argv) < 2:
        print("""Usage: python3 wikidata_ingester.py <Wikipedia_Title> [Title2] ...""")
        print("""Example: python3 wikidata_ingester.py Europe England London""")
        sys.exit(1)

    for title in sys.argv[1:]:
        ingest_entity(title)

    kg = load_kg()
    print(f"""
    Knowledge graph: {len(kg)} entities""")

```

74. word_embeddings.py

347 lines | 11.7KB | /data/data/com.termux/files/home/Quantum_MCAGI_NO_LLM_V███2/backend/word_embeddings.py

```
&quot;&quot;&quot;;
■ WORD EMBEDDINGS - Pure Python Implementation
=====
Word2Vec-style embeddings without external dependencies.

Implements Skip-gram with Negative Sampling (simplified).
Words are represented as dense vectors capturing semantic relationships.
Similar words have similar vectors.
&quot;&quot;&quot;;

import math
import random
import re
import json
import os
from collections import defaultdict, Counter
from typing import List, Dict, Tuple, Optional
import logging

logger = logging.getLogger(&quot;quantum_ai&quot;)
```

```
class WordEmbeddings:
    &quot;&quot;&quot;;
    Simple word embeddings trained on corpus.

    Uses co-occurrence matrix factorization (GloVe-style)
    which is more tractable than Skip-gram for CPU-only training.

    Algorithm:
    1. Build word co-occurrence matrix from corpus
    2. Apply PPMI (Positive PMI) weighting
    3. Reduce dimensionality via truncated SVD approximation
    &quot;&quot;&quot;;

    def __init__(self, vector_size: int = 50, window: int = 5, min_count: int = 2):
        self.vector_size = vector_size
        self.window = window
        self.min_count = min_count

        self.word_to_id = {}
        self.id_to_word = {}
        self.vectors = {} # word -> vector
        self.word_counts = Counter()
        self.cooccurrence = defaultdict(Counter)

        self.trained = False
        self.vocab_size = 0

    def build_vocab(self, texts: List[str]):
        &quot;&quot;&quot;;Build vocabulary from texts.&quot;&quot;&quot;;
        for text in texts:
            words = self._tokenize(text)
            self.word_counts.update(words)

        # Filter by min_count
        vocab = [w for w, c in self.word_counts.items() if c >= self.min_count]
        vocab.sort()

        self.word_to_id = {w: i for i, w in enumerate(vocab)}
        self.id_to_word = {i: w for w, i in self.word_to_id.items()}
        self.vocab_size = len(vocab)

        logger.info(f&quot;Built vocabulary: {self.vocab_size} words&quot;)
```

```
    def build_cooccurrence(self, texts: List[str]):
        &quot;&quot;&quot;;Build co-occurrence matrix.&quot;&quot;&quot;;
        for text in texts:
            words = self._tokenize(text)
            words = [w for w in words if w in self.word_to_id]

            for i, center in enumerate(words):
                # Context window
                start = max(0, i - self.window)
                end = min(len(words), i + self.window + 1)

                for j in range(start, end):
                    if i != j:
                        context = words[j]
                        # Distance weighting (closer = higher weight)
                        distance = abs(i - j)
                        weight = 1.0 / distance
                        self.cooccurrence[center][context] += weight

        logger.info(f&quot;Built co-occurrence matrix&quot;)
```

```
    def train(self, texts: List[str], iterations: int = 10):
        &quot;&quot;&quot;;
        Train embeddings using simplified GloVe-style algorithm.

        This is a lightweight approximation suitable for CPU training.
        &quot;&quot;&quot;;
        # Build vocab and co-occurrence
        self.build_vocab(texts)
```

```

self.build_cooccurrence(texts)

self._train_continued(iterations)

def _train_continued(self, iterations):
    """Continuation of train - auto-extracted by self-evolution"""
    if self.vocab_size == 0:
        logger.warning("Empty vocabulary, cannot train")
        return

    # Initialize random vectors
    for word in self.word_to_id:
        self.vectors[word] = [random.gauss(0, 0.1) for _ in range(self.vector_size)]

    # Compute PPMI matrix and train
    total_count = sum(sum(v.values()) for v in self.cooccurrence.values())
    word_probs = {w: self.word_counts[w] / sum(self.word_counts.values())
                  for w in self.word_to_id}

    # Stochastic gradient descent on weighted least squares
    learning_rate = 0.05

    for iteration in range(iterations):
        total_loss = 0
        updates = 0

        for word_i, contexts in self.cooccurrence.items():
            if word_i not in self.vectors:
                continue

            vec_i = self.vectors[word_i]

            for word_j, count in contexts.items():
                if word_j not in self.vectors:
                    continue

                vec_j = self.vectors[word_j]

                # Compute PPMI target
                p_ij = count / total_count
                p_i = word_probs.get(word_i, 1e-10)
                p_j = word_probs.get(word_j, 1e-10)
                pmi = math.log(p_ij / (p_i * p_j) + 1e-10)
                ppmi = max(0, pmi)

                # Dot product prediction
                dot = sum(a * b for a, b in zip(vec_i, vec_j))

                # Error
                error = dot - ppmi

                # Weight by co-occurrence frequency
                weight = min(1.0, (count / 100) ** 0.75)

                # Gradient update
                for k in range(self.vector_size):
                    grad_i = weight * error * vec_j[k]
                    grad_j = weight * error * vec_i[k]

                    vec_i[k] -= learning_rate * grad_i
                    vec_j[k] -= learning_rate * grad_j

                total_loss += weight * error ** 2
                updates += 1

        if updates > 0:
            avg_loss = total_loss / updates
            if iteration % 3 == 0:
                logger.info(f"Embedding training iteration {iteration}: loss={avg_loss:.4f}")

    # Normalize vectors
    for word in self.vectors:
        vec = self.vectors[word]
        norm = math.sqrt(sum(x*x for x in vec))
        if norm > 0:
            self.vectors[word] = [x/norm for x in vec]

    self.trained = True
    logger.info(f"Trained {len(self.vectors)} word vectors")

def get_vector(self, word: str) -> Optional[List[float]]:
    """Get vector for a word"""
    return self.vectors.get(word.lower())

def similarity(self, word1: str, word2: str) -> float:
    """Compute cosine similarity between two words"""
    vec1 = self.get_vector(word1)
    vec2 = self.get_vector(word2)

    if vec1 is None or vec2 is None:
        return 0.0

    # Cosine similarity (vectors already normalized)
    return sum(a * b for a, b in zip(vec1, vec2))

def most_similar(self, word: str, top_n: int = 10) -> List[Tuple[str, float]]:
    """Find most similar words"""
    vec = self.get_vector(word)
    if vec is None:
        return []

```

```

        similarities = []
        for other_word, other_vec in self.vectors.items():
            if other_word != word.lower():
                sim = sum(a * b for a, b in zip(vec, other_vec))
                similarities.append((other_word, sim))

        similarities.sort(key=lambda x: x[1], reverse=True)
        return similarities[:top_n]

def analogy(self, a: str, b: str, c: str, top_n: int = 5) -> List[Tuple[str, float]]:
    """
    Solve analogy: a is to b as c is to ?

    Uses vector arithmetic: result = vec(b) - vec(a) + vec(c)

    vec_a = self.get_vector(a)
    vec_b = self.get_vector(b)
    vec_c = self.get_vector(c)

    if any(v is None for v in [vec_a, vec_b, vec_c]):
        return []

    # Compute target vector
    target = [b_i - a_i + c_i for a_i, b_i, c_i in zip(vec_a, vec_b, vec_c)]

    # Normalize
    norm = math.sqrt(sum(x*x for x in target))
    if norm > 0:
        target = [x/norm for x in target]

    # Find closest words (excluding inputs)
    exclude = {a.lower(), b.lower(), c.lower()}
    similarities = []

    for word, vec in self.vectors.items():
        if word not in exclude:
            sim = sum(t * v for t, v in zip(target, vec))
            similarities.append((word, sim))

    similarities.sort(key=lambda x: x[1], reverse=True)
    return similarities[:top_n]

def sentence_vector(self, text: str) -> Optional[List[float]]:
    """
    Get average vector for a sentence.
    words = self._tokenize(text)
    vectors = [self.get_vector(w) for w in words if self.get_vector(w)]

    if not vectors:
        return None

    # Average
    result = [0.0] * self.vector_size
    for vec in vectors:
        for i, v in enumerate(vec):
            result[i] += v

    n = len(vectors)
    result = [x / n for x in result]

    # Normalize
    norm = math.sqrt(sum(x*x for x in result))
    if norm > 0:
        result = [x/norm for x in result]

    return result

def sentence_similarity(self, text1: str, text2: str) -> float:
    """
    Compute similarity between two sentences.
    vec1 = self.sentence_vector(text1)
    vec2 = self.sentence_vector(text2)

    if vec1 is None or vec2 is None:
        return 0.0

    return sum(a * b for a, b in zip(vec1, vec2))

def _tokenize(self, text: str) -> List[str]:
    """
    Tokenize text to words.
    return re.findall(r'\b[a-z]+\b', text.lower())

def save(self, filepath: str):
    """
    Save embeddings to file.
    data = {
        'vectors': self.vectors,
        'word_to_id': self.word_to_id,
        'vocab_size': self.vocab_size,
        'vector_size': self.vector_size
    }
    with open(filepath, 'w') as f:
        json.dump(data, f)
    logger.info(f'Saved embeddings to {filepath}')

def load(self, filepath: str) -> bool:
    """
    Load embeddings from file.
    try:
        if os.path.exists(filepath):
            with open(filepath, 'r') as f:
                data = json.load(f)
                self.vectors = data['vectors']
                self.word_to_id = data['word_to_id']
                self.id_to_word = {int(i): w for w, i in self.word_to_id.items()}

```

```

        self.vocab_size = data['vocab_size']
        self.vector_size = data['vector_size']
        self.trained = True
        logger.info(f'Loaded {len(self.vectors)} embeddings')
        return True
    except Exception as e:
        logger.warning(f'Could not load embeddings: {e}')
    return False

def get_stats(self) -> Dict:
    """Get embedding statistics"""
    return {
        'vocab_size': self.vocab_size,
        'vector_size': self.vector_size,
        'trained': self.trained,
        'total_vectors': len(self.vectors)
    }

# Singleton with lazy initialization
_embeddings = None
_embeddings_file = 'app/backend/word_embeddings.json'

def get_embeddings() -> WordEmbeddings:
    """Get or create word embeddings"""
    global _embeddings
    if _embeddings is None:
        _embeddings = WordEmbeddings(vector_size=50, window=5, min_count=2)

        # Try to load pre-trained
        if not _embeddings.load(_embeddings_file):
            # Train on corpus
            from training_corpus import get_corpus_sentences
            sentences = get_corpus_sentences()
            if sentences:
                _embeddings.train(sentences, iterations=15)
                _embeddings.save(_embeddings_file)

    return _embeddings

def train_embeddings_on_text(text: str):
    """Add new text to embeddings training"""
    embeddings = get_embeddings()
    sentences = re.split(r'(?![\s])\s+', text)
    sentences = [s for s in sentences if len(s) > 20]

    if sentences:
        embeddings.train(sentences, iterations=5)
        embeddings.save(_embeddings_file)

```